DEEPSEEK 官方论文

中文全文汇编

24 篇 · 2024–2026

DeepSeek LLM · MoE · Coder · Math · VL · V2 · Prover · V3 · R1 · NSA · V3.2 · mHC · Engram · OCR · V4 · DSec

正文全译 · 图注/表题翻译 · 图表保留原文页码 · 每篇一个书签

目　　录

《DeepSeek 官方论文中文全文汇编（24 篇）》

01 DeepSeek LLM....3
02 DeepSeekMoE....4
03 DeepSeek-Coder....5
04 DeepSeekMath....6
05 DeepSeek-VL....7
06 DeepSeek-V2....8
07 DeepSeek-Prover....9
08 DeepSeek-Coder-V2....10
09 DeepSeek-Prover-V1.5....11
10 DeepSeek-VL2....12
11 DeepSeek-V3 Technical Report....13
12 DeepSeek-R1....14
13 Native Sparse Attention....15
14 DeepSeek-Prover-V2....16
15 Insights into DeepSeek-V3....17
16 DeepSeek-OCR....18
17 DeepSeekMath-V2....19
18 DeepSeek-V3.2....20
19 mHC....21
20 Conditional Memory via Scalable Lookup....22
21 DeepSeek-OCR 2....23
22 DeepSeek-V4....24
23 DeepSeek-V4.1-Flash....25
24 DeepSeek Elastic Compute (DSec)....26

说明：本汇编收录 DeepSeek 官方论文 24 篇的中文全文版（正文全译，图注/表题翻译，图表数据保留原文页码）；每篇在 PDF 书签栏中对应一个书签。

DeepSeek LLM：以长期主义扩展开源语言模型

DeepSeek LLM: Scaling Open-Source Language Models with Longtermism

DeepSeek-AI　research@deepseek.com

arXiv:2401.02954 · 中文翻译版（正文全文；图表保留原文页码位置）

摘　要

开源大语言模型（LLM）的迅猛发展确实令人瞩目。然而，已有文献所描述的缩放定律（scaling laws）给出了各不相同的结论，这给大语言模型的规模化蒙上了一层阴影。我们深入研究缩放定律，并给出了独特的发现，以推动两种常用开源配置（7B 与 67B）下大规模模型的扩展。在缩放定律的指导下，我们推出了 DeepSeek LLM——一个以长期主义视角致力于推进开源语言模型发展的项目。为支撑预训练阶段，我们构建了一个目前包含 2 万亿（2 trillion）token 且仍在持续扩充的数据集。我们进一步在 DeepSeek LLM Base 模型上开展监督微调（SFT）与直接偏好优化（DPO），从而得到 DeepSeek Chat 模型。评测结果表明，DeepSeek LLM 67B 在一系列基准（benchmark）上超越 LLaMA-2 70B，尤其在代码、数学与推理领域。此外，开放式评测显示，与 GPT-3.5 相比，我们的 DeepSeek LLM 67B Chat 表现更优。

1 引言

过去几年间，基于 decoder-only Transformer（Vaswani 等，2017）的大语言模型（Large Language Model, LLM）日益成为通往通用人工智能（Artificial General Intelligence, AGI）的基石与路径。LLM 通过在连续文本中预测下一个词，在海量数据集上进行自监督预训练，从而能够服务于多种用途并具备多种能力，例如小说创作、文本摘要、代码补全等。此后的监督微调与奖励建模等进展，使大语言

模型（LLM）能够更好地遵循用户意图与指令，赋予它们更为多样的对话能力，并迅速扩大了其影响力。

这一浪潮由闭源产品掀起，例如 ChatGPT（OpenAI，2022）、Claude（Anthropic，2023）与 Bard（Google，2023），这些产品依托大量算力资源与高昂的标注成本开发而成。这些产品显著抬高了社区对开源 LLM 能力的期待，进而启发了一系列工作（Bai 等，2023；Du 等，2022；Jiang 等，2023；Touvron 等，2023a,b；Yang 等，2023）。其中，LLaMA 系列模型（Touvron 等，2023a,b）尤为突出。它整合了一系列工作，构建出高效且稳定的架构，打造出从 7B 到 70B 参数、性能良好的模型。因此，LLaMA 系列已成为开源模型在架构与性能方面事实上的基准。

在 LLaMA 之后，开源社区主要聚焦于训练固定规模（7B、13B、34B 与 70B）的高质量模型，往往忽视了对 LLM 扩展规律（scaling laws）的研究探索（Hoffmann 等，2022；Kaplan 等，2020）。然而，考虑到当前开源模型仅处于通用人工智能（AGI）发展的起步阶段，扩展规律研究至关重要。此外，早期工作（Hoffmann 等，2022；Kaplan 等，2020）在算力预算增加时模型与数据应如何扩展这一问题上得出了不同的结论，且对超参数的讨论不够充分。本文中，我们系统研究了语言模型的扩展行为，并将研究结论应用于两种被广泛使用的大规模模型配置，即 7B 与 67B。我们的研究旨在为未来开源 LLM 的扩展奠定基础，为该领域的进一步发展铺平道路。具体而言，我们首先考察了批大小与学习率的扩展规律，并发现了它们随模型规模变化的趋势。在此基础上，我们对数据规模与模型规模的扩展规律进行了全面研究，成功揭示了最优的模型/数据扩展分配策略，并预测了我们大规模模型的预期性能。此外，在研发过程中我们发现，由不同数据集导出的扩展规律存在显著差异。这表明数据集的选择会显著影响扩展行为，也意味着在跨数据集推广扩展规律时应保持谨慎。

在我们的扩展规律指导下，我们从零开始构建开源大语言模型，并尽可能多地公开信息以供社区参考。我们收集了 2 万亿 token 用于预训练，以中英文为主。在模型层面，我们总体上沿用 LLaMA 的架构，但用多步学习率调度器替换了余弦学习率调度器，在保持性能的同时便于持续训练。我们收集了来自多种来源的超过 100 万个实例，用于监督微调（SFT）（Ouyang 等，2022）。本文分享了我们在不同 SFT 策略上的经验，以及在数据消融技术方面的发现。此外，我们利用直接偏好优化（DPO）（Rafailov 等，2023）来提升模型的对话表现。

我们使用基础（base）模型与对话（chat）模型开展了大量评测。评测结果表明，DeepSeek LLM 在多个基准上超越 LLaMA-2 70B，尤其在代码、数学与推理领域。经过 SFT 与 DPO 之后，DeepSeek 67B 对话模型在中英文开放式评测中均优于 GPT-3.5。这凸显了 DeepSeek 67B 在生成高质量回复以及

用两种语言进行有意义对话方面的优越表现。此外，安全评测表明，DeepSeek 67B Chat 在实践中能够给出无害的回复。

本文其余部分组织如下：第 2 节首先介绍 DeepSeek LLM 预训练的基本概念，包括数据构成、模型架构、基础设施与超参数。第 3 节详细阐述我们发现的扩展规律及其含义。同时，结合扩展规律分析所带来的洞见，我们讨论了预训练超参数选择背后的依据。第 4 节讨论我们的微调方法，涵盖微调数据的构成以及 SFT 与 DPO 阶段的具体方法。随后第 5 节给出 DeepSeek LLM 的详细评测结果，既覆盖基础模型与对话模型，也覆盖它们在开放式评测与安全评测中的表现。最后，第 6 节讨论 DeepSeek LLM 当前的局限与未来的方向。

2 预训练

2.1 数据

我们的主要目标是全面提升数据集的丰富性与多样性。我们从若干可靠来源（Computer，2023；Gao 等，2020；Penedo 等，2023；Touvron 等，2023a）中获得了有价值的洞见。为实现这些目标，我们将方法组织为三个关键阶段：去重（deduplication）、过滤（filtering）与重混（remixing）。去重与重混阶段通过对唯一样本进行采样来保证数据表示的多样性；过滤阶段则提升信息密度，从而使模型训练更高效、更有效。

我们采用了激进的去重策略，扩大了去重范围。分析表明，与在单个 dump 内部去重相比，对整个 Common Crawl 语料库去重能剔除更多重复实例。表 1 显示，跨 91 个 dump 去重所剔除的文档量是单 dump 方法的四倍。

表 1：各种 Common Crawl dump 的去重比例。（原文第 4 页）

使用的 dump 数	1	2	6	12	16	22	41	91
去重比例（%）	22.2	46.7	55.7	69.9	75.7	76.3	81.6	89.8

在过滤阶段，我们专注于为文档质量评估制定稳健的判据。这涉及结合语言层面与语义层面评估的细致分析，从个体与全局两个视角给出数据质量的图景。在重混阶段，我们调整方法以应对数据不

均衡问题，重点提高代表性不足领域的占比。这一调整旨在获得更均衡、更具包容性的数据集，确保多样化的视角与信息得到充分体现。

在分词器方面，我们基于 tokenizers 库（Huggingface Team，2019）实现了字节级字节对编码（Byte-level Byte-Pair Encoding, BBPE）算法。与 GPT-2（Radford 等，2019）类似，我们采用预分词（pre-tokenization）来防止来自不同字符类别的 token 被合并，例如换行符、标点符号以及中日韩（CJK）符号。我们还沿用了（Touvron 等，2023a,b）的做法，将数字拆分为单个数字。基于此前的经验，我们将词表中的常规 token 数设为 100000。分词器在约 24 GB 的多语种语料上训练，我们在最终词表中增补了 15 个特殊 token，使总规模达到 100015。为保证训练时的计算效率，并为未来可能需要的额外特殊 token 预留空间，我们在训练时将模型的词表大小配置为 102400。

2.2 模型架构

表 2：DeepSeek LLM 系列模型的详细规格。我们依据第 3 节的研究结论选择超参数。（原文第 5 页）

参数量	n_layers	d_model	n_heads	n_kv_heads	上下文长度	序列批大小	学习率	token 数
7B	30	4096	32	32	4096	2304	4.2e-4	2.0T
67B	95	8192	64	8	4096	4608	3.2e-4	2.0T

DeepSeek LLM 的微观设计大体沿用 LLaMA（Touvron 等，2023a,b）的设计：采用 Pre-Norm 结构并配合 RMSNorm（Zhang 与 Sennrich，2019）函数，使用 SwiGLU（Shazeer，2020）作为前馈网络（Feed-Forward Network, FFN）的激活函数，中间层维度为 8/3·d_model。此外，模型采用旋转位置编码（Rotary Embedding）（Su 等，2024）作为位置编码。为优化推理成本，67B 模型使用分组查询注意力（Grouped-Query Attention, GQA）（Ainslie 等，2023）替代传统的多头注意力（Multi-Head Attention, MHA）。

然而在宏观设计上，DeepSeek LLM 略有不同。具体而言，DeepSeek LLM 7B 是 30 层网络，而 DeepSeek LLM 67B 有 95 层。这些层数调整在与其他开源模型保持参数规模一致的同时，也便于模型流水线切分，从而优化训练与推理。

与大多数使用分组查询注意力（GQA）的工作不同，我们将 67B 模型的参数扩展放在网络深度上，而非通常做法中加宽 FFN 层的中间宽度，以期获得更好的性能。详细的网络规格见表 2。

2.3 超参数

DeepSeek LLM 以 0.006 的标准差初始化，并使用 AdamW 优化器（Loshchilov 与 Hutter，2017）训练，超参数如下：β1 = 0.9，β2 = 0.95，weight_decay = 0.1。

预训练期间采用多步学习率调度器，而非典型的余弦调度器。具体而言，模型的学习率在 2000 个 warmup 步之后达到最大值，随后在处理完 80% 的训练 token 后降至最大值的 31.6%，在处理完 90% 的 token 后再降至最大值的 10%。训练阶段的梯度裁剪阈值设为 1.0。

【图表】图 1 不同学习率调度器（或调度器的不同参数）下的训练损失曲线。模型规模为 16 亿参数，在 1000 亿 token 的数据集上训练；(a) 多步与余弦学习率衰减的对比，(b) 多步调度器不同阶段的占比（原文第 6 页）

根据我们的经验性发现，尽管训练过程中的损失下降趋势有所不同，但使用多步学习率调度器的最终性能与余弦调度器基本一致，如图 1(a) 所示。在保持模型规模不变而调整训练规模时，多步学习率调度器允许复用第一阶段的训练，为持续训练提供了独特的便利。因此，我们选择多步学习率调度器作为默认设置。

我们还在图 1(b) 中表明，调整多步学习率调度器中不同阶段的占比可以带来略优的性能。然而，为在持续训练中的复用比例与模型性能之间取得平衡，我们最终选择了前述三阶段分别为 80%、10% 与 10% 的分配。

批大小与学习率随模型规模变化。7B 与 67B 模型预训练阶段的具体参数见表 2。

2.4 基础设施

我们使用一个名为 HAI-LLM（High-flyer，2023）的高效轻量训练框架来训练与评测大语言模型。该框架集成了数据并行、张量并行、序列并行与 1F1B 流水线并行，做法与 Megatron（Korthikanti 等，2023；Narayanan 等，2021；Shoeybi 等，2019）一致。我们还利用 flash attention（Dao，2023；Dao 等，2022）技术来提升硬件利用率。采用 ZeRO-1（Rajbhandari 等，2020）在数据并行 rank 之间划分优化器状态。我们还努力让计算与通信重叠，以尽量减少额外的等待开销，包括最后一个 micro-batch 的反向传播过程与 ZeRO-1 中的 reduce-scatter 操作之间的重叠，以及序列并行中 GEMM 计算与 all-gather/reduce-scatter 之间的重叠。一些层/算子被融合以加速训练，包括 LayerNorm、尽可能融合的 GEMM，以及 Adam 更新。为提升训练稳定性，我们以 bf16 精度训练模型，但以 fp32 精度累加梯度。为降低 GPU 显存消耗，我们采用原地（in-place）交叉熵，即在交叉熵

CUDA kernel 中动态地将 bf16 logits 转换为 fp32 精度（而非事先在 HBM 中转换），计算相应的 bf16 梯度，并用该梯度覆写 logits。

模型权重与优化器状态每 5 分钟异步保存一次，这意味着在偶尔发生硬件或网络故障的最坏情况下，我们最多损失 5 分钟的训练进度。这些临时模型检查点会被定期清理，以避免占用过多存储空间。我们还支持从不同的 3D 并行配置恢复训练，以应对计算集群负载的动态变化。

在评测方面，我们在生成任务中使用 vLLM（Kwon 等，2023），在非生成任务中使用连续批处理（continuous batching），以避免手动调优批大小并减少 token 填充。

3 扩展规律

对扩展规律的研究（Hestness 等，2017）早于大语言模型的出现。扩展规律（Henighan 等，2020；Hoffmann 等，2022；Kaplan 等，2020）表明，随着算力预算 C、模型规模 N 与数据规模 D 的增加，模型性能可以被可预测地提升。当模型规模 N 以模型参数表示、数据规模 D 以 token 数表示时，C 可近似为 $C = 6ND$。因此，在增加算力预算时如何优化模型规模与数据规模之间的分配，也是扩展规律研究中的一个关键目标。

LLM 的发展（Dai 等，2019；Radford 等，2019）——更大的模型取得了出乎意料且显著的性能提升——将扩展规律研究推向了新的高峰。扩展规律的研究结果表明，扩大算力预算仍能持续带来显著收益，这进一步鼓励了模型规模的扩大（Brown 等，2020；Smith 等，2022）。

然而，如表 4 所示，早期关于最优模型/数据扩展分配策略的工作（Hoffmann 等，2022；Kaplan 等，2020）得出了不同的结论，使人们对扩展规律的普遍适用性产生怀疑。此外，这些研究往往缺少对超参数设置的完整描述，导致无法确定不同算力预算下的模型是否达到了最优性能。因此，我们在本节重新审视扩展规律，以消除这些不确定性，并确保我们走在高效扩展算力的正确道路上——这体现了长期主义视角，也是持续改进模型的关键。

为确保不同算力预算下的模型都能达到最优性能，我们首先研究了超参数的扩展规律。经验观察表明，当算力预算变化时，训练中大多数参数的最优值并不改变。因此，这些参数与第 2.3 节所述一致，在不同算力预算下保持不变。然而，对性能影响最大的超参数，即批大小与学习率，则被重新考察。早期工作（Goyal 等，2017；McCandlish 等，2018；Shallue 等，2019；Smith 等，2017；Zhang 等，2019）为设定批大小与学习率提供了一些经验性观察，但我们发现这些观察在我们的初步实验中适用性有限。通过大量实验，我们对算力预算 C 与最优批大小、最优学习率之间的幂律关系进行了建

模。我们将这一关系称为超参数的扩展规律，它为确定最优超参数提供了经验性框架。该方法保证了不同算力预算下的模型都能达到接近最优的性能。

随后我们研究模型规模与数据规模的扩展规律。为降低实验成本与拟合难度，我们采用 Chinchilla（Hoffmann 等，2022）中的 IsoFLOP 剖面（IsoFLOP profile）方法来拟合扩展曲线。为更准确地表示模型规模，我们采用新的模型规模表示——非嵌入 FLOPs/token M，以替代此前使用的模型参数 N，并将近似算力预算公式 $C = 6ND$ 替换为更精确的 $C = MD$。实验结果给出了关于最优模型/数据扩展分配策略与性能预测的洞见，并且准确预测了 DeepSeek LLM 7B 与 67B 模型的预期性能。

此外，在探索扩展规律的过程中，我们所用的数据经历了多轮迭代，质量不断提升。我们尝试在多个数据集上拟合扩展曲线，发现数据质量会显著影响最优的模型/数据扩展分配策略。数据质量越高，新增的算力预算就应越多地分配给模型扩展。这意味着在数据规模相同时，高质量数据可以支撑更大模型的训练。最优模型/数据扩展分配策略的差异，也可作为一种间接评估数据质量的方法。我们将持续密切关注数据质量的变化及其对扩展规律的影响，并在未来工作中提供更多分析。

综上所述，我们在扩展规律方面的贡献与发现可概括如下：

- 我们建立了超参数的扩展规律，为确定最优超参数提供了经验性框架。
- 我们采用非嵌入 FLOPs/token M 而非模型参数 N 来表示模型规模，从而得到更准确的最优模型/数据扩展分配策略，并能更好地预测大规模模型的泛化损失。
- 预训练数据的质量会影响最优的模型/数据扩展分配策略。数据质量越高，新增的算力预算就应越多地分配给模型扩展。

3.1 超参数的扩展规律

我们首先在算力预算为 1e17 的小规模实验中，对批大小与学习率进行了网格搜索，某一具体模型规模（177M FLOPs/token）的结果如图 2(a) 所示。结果表明，在相当宽的批大小与学习率取值范围内，泛化误差保持稳定。这说明在相对较宽的参数空间内即可达到接近最优的性能。

【图表】图 2 在 1e17 与 1e20 FLOPs 预算下，训练损失随批大小与学习率的变化；(a) 1e17 FLOPs（177M FLOPs/token），(b) 1e20 FLOPs（2.94B FLOPs/token）（原文第 8 页）

随后，我们利用上述多步学习率调度器，通过复用第一阶段，高效地训练了多个在批大小、学习率与算力预算（从 1e17 到 2e19）上各不相同的模型。考虑到参数空间中的冗余，我们将泛化误差超出最小值不超过 0.25% 的模型所用参数视为近最优超参数。接着，我们对批大小 B 与学习率 η 关于

算力预算 C 进行了拟合。如图 3 所示，拟合结果表明：随着算力预算 C 增加，最优批大小 B 逐渐增大，而最优学习率 η 逐渐减小。这与扩展模型时对批大小与学习率的直观经验设定相符。此外，所有近最优超参数都落在一条较宽的带状范围内，说明在该区间内选择近最优参数相对容易。我们最终拟合得到的批大小与学习率公式如下：

η_opt = 0.3118·C^(−0.1250)；B_opt = 0.2920·C^(0.3271)　　(1)

【图表】图 3 批大小与学习率的扩展曲线。灰色圆点表示泛化误差超出最小值不超过 0.25% 的模型；虚线为对较小模型拟合的幂律；蓝色星号表示 DeepSeek LLM 7B 与 67B；(a) 批大小扩展曲线，(b) 学习率扩展曲线（原文第 9 页）

我们在算力预算为 1e20 的一系列模型上验证了上述公式，某一具体模型规模（每 token 2.94B FLOPs）的结果如图 2(b) 所示。结果表明，拟合得到的参数位于最优参数空间的中心。后续章节也将表明，我们为 DeepSeek LLM 7B 与 67B 模型拟合得到的参数同样取得了良好性能。

然而需要指出的是，我们尚未考虑算力预算 C 之外的其它因素对最优超参数的影响。这与一些早期工作（Kaplan 等，2020；McCandlish 等，2018）不一致，它们认为最优批大小可以建模为仅与泛化误差 L 相关。此外，我们观察到，在算力预算相同但模型/数据分配不同的模型中，最优参数空间会略有差异。这表明需要进一步研究以理解超参数的选择与训练动力学。我们将在未来工作中探索这些方面。

3.2 估计最优的模型与数据扩展

在得出用于拟合近最优超参数的公式之后，我们开始拟合扩展曲线，并分析最优的模型/数据扩展分配策略。该策略旨在寻找模型扩展指数 a 与数据扩展指数 b，使它们满足 N_opt ∝ C^a，以及 $D_{opt} \propto C^b$。数据规模 D 可以统一地用数据集中的 token 数量来表示。在以往的工作中，模型规模通常用模型参数量来表示，即非嵌入参数 N_1（Kaplan et al., 2020）与完整参数 N_2（Hoffmann et al., 2022）。计算预算 C 与模型/数据规模之间的关系可以近似描述为 $C = 6ND$，这意味着我们可以用 $6N_1$ 或 $6N_2$ 来近似模型规模。然而，由于 $6N_1$ 与 $6N_2$ 都没有计入注意力操作的计算开销，而且 $6N_2$ 还包含了词表计算——这部分对模型能力的贡献较小——因此二者在某些设置下都存在显著的近似误差。为缓解这些误差，我们引入了一种新的模型规模表示：非嵌入 FLOPs/token M。M 包含注意力操作的计算开销，但不计入词表计算。用 M 表示模型规模后，计算预算 C 可以简单地表示为 $C = MD$。$6N_1$、$6N_2$ 与 M 之间的具体差异如下式所示：

$$6N_1 = 72 \cdot n_{\text{layer}} \cdot d_{\text{model}}^2$$

$6N_2 = 72 \cdot n\text{layer} \cdot d\text{model}^2 + 6 \cdot n\text{vocab} \cdot d\text{model}$

$M = 72 \cdot n\text{layer} \cdot d\text{model}^2 + 12 \cdot n\text{layer} \cdot d\text{model} \cdot l\text{seq}$ （2）

其中 $n$layer 表示层数，$d$model 表示模型宽度，$n$vocab 为词表大小，$l$seq 为序列长度。我们在不同规模的模型上评估了这三种表示之间的差异，如表 3 所示。结果表明，在不同规模的模型中，$6N_1$ 与 $6N_2$ 时而高估、时而低估计算开销。这种偏差在小规模模型上尤为明显，差异最高可达 50%。此类不准确会在拟合 scaling 曲线时引入可观的统计误差。关于模型规模不同表示的进一步分析，请参见附录 A.2。

【表格】表 3 模型规模表示方式的差异，以及非嵌入参数 N_1 与完整参数 N_2 相对于非嵌入 FLOPs/token M 的偏差（原文第 10 页）

采用 M 表示模型规模后，我们的目标可以更清晰地表述为：给定计算预算 $C = MD$，找到使模型泛化误差最小的最优模型规模 Mopt 与数据规模 Dopt。该目标可以形式化为：

$M\text{opt}(C), D\text{opt}(C) = \text{argmin}_\{M, D, \text{s.t. } C = MD\}\ L(N, D)$ （3）

为降低实验成本与拟合难度，我们采用 Chinchilla（Hoffmann et al., 2022）中的 IsoFLOP profile 方法来拟合 scaling 曲线。我们选取了从 1e17 到 3e20 的 8 个不同计算预算，并为每个预算设计了约 10 种不同的模型/数据规模分配。每个预算对应的超参数由公式（1）确定，泛化误差在一个独立的验证集上计算——该验证集与训练集分布相近，包含 1 亿 token。

【图表】图 4 IsoFLOP 曲线与最优模型/数据分配。（a）IsoFLOP 曲线；（b）最优模型 scaling；（c）最优数据 scaling。IsoFLOP 曲线的指标为验证集上的 bits-per-byte；最优模型/数据 scaling 曲线中的虚线表示对较小模型（灰色圆圈）拟合的幂律（原文第 11 页）

图 4 展示了 IsoFLOP 曲线以及模型/数据 scaling 曲线，它们通过对每个计算预算下的最优模型/数据分配进行拟合得到。最优非嵌入 FLOPs/token Mopt 与最优 token 数 Dopt 的具体公式如下：

$M\text{opt} = M\text{base} \cdot C\text{^a}$，其中 $M\text{base} = 0.1715$，$a = 0.5243$

$D\text{opt} = D\text{base} \cdot C\text{^b}$，其中 $D\text{base} = 5.8316$，$b = 0.4757$ （4）

此外，我们根据计算预算 C 与最优泛化误差拟合了损失 scaling 曲线，并预测了 DeepSeek LLM 7B 与 67B 的泛化误差，如图 5 所示。结果表明，使用小规模实验能够准确预测计算预算为其 1000 倍的模型性能。这为在更大规模上训练模型提供了信心与指导。

【图表】图 5 性能 scaling 曲线。指标为验证集上的 bits-per-byte。虚线表示对较小模型（灰色圆圈）拟合的幂律；

蓝色星号表示 DeepSeek LLM 7B 与 67B，它们的性能被 scaling 曲线很好地预测（原文第 11 页）

3.3 不同数据下的 Scaling Laws

在 DeepSeek LLM 的研发过程中，数据集经历了多轮迭代精炼，我们调整了不同数据来源的比例，同时提升了整体数据质量。这使我们能够进一步分析不同数据集对 scaling laws 的影响。

我们使用三个不同的数据集来研究 scaling laws：早期内部数据、当前内部数据，以及此前 scaling laws 研究（Kaplan et al., 2020）中使用的 OpenWebText2。我们的内部数据评估显示，当前内部数据的数据质量高于早期内部数据。此外，OpenWebText2 的质量甚至超过了当前内部数据，因为其规模较小，可以进行更细致的处理。

【表格】表 4 模型 scaling 与数据 scaling 的系数随训练数据分布而变化（原文第 12 页）

该分析中一个有趣的观察是，在这三个数据集上，最优的模型/数据放大分配策略与数据质量表现出了一致性。如表 4 所示，随着数据质量的提升，模型 scaling 指数 a 逐渐增大，而数据 scaling 指数 b 逐渐减小，这表明增加的计算预算应更多地分配给模型而非数据。这一发现或许也能解释早期 scaling laws 研究中观察到的最优模型/数据放大分配上的显著差异。

对这一发现的一个直观推测是：高质量数据通常意味着逻辑清晰，并且在充分训练之后预测难度更低。因此，在增加计算预算时，扩大模型规模更为有利。我们将继续密切关注数据质量的变化及其对 scaling laws 的影响，并在未来的工作中提供更多分析。

4 对齐

我们收集了约 150 万条中英文指令数据，覆盖广泛的有用性（helpfulness）与无害性（harmlessness）主题。我们的有用数据包含 120 万条实例，其分布为：通用语言任务 31.2%、数学问题 46.6%、编程练习 22.2%。安全数据包含 30 万条实例，覆盖各类敏感话题。

我们的对齐流程包含两个阶段。

监督微调（Supervised Fine-Tuning）：我们将 7B 模型微调了 4 个 epoch，但 67B 模型只微调了 2 个 epoch，因为我们观察到 67B 模型上的过拟合问题较为严重。我们观察到，7B 模型的 GSM8K（Cobbe et al., 2021）与 HumanEval（Chen et al., 2021）指标持续提升，而 67B 模型很快就达到了性能

上界。7B 与 67B 模型的学习率分别为 1e-5 与 5e-6。除了监控基准准确率，我们还评估了 chat 模型在微调过程中的重复比例。我们共收集了 3868 条中英文 prompt，并统计了生成回复中未能终止、反而无休止重复某段文本的比例。我们观察到，随着数学 SFT 数据量的增加，重复比例呈上升趋势。这可以归因于数学 SFT 数据有时在推理中包含相似的模式，因此能力较弱的模型难以掌握这类推理模式，从而产生重复回复。为解决该问题，我们尝试了两阶段微调与 DPO（Rafailov et al., 2023），二者都几乎能够保持基准分数，并显著降低重复比例。

DPO：为进一步增强模型能力，我们采用了直接偏好优化算法（direct preference optimization, DPO；Rafailov et al., 2023），该算法已被证明是一种简单而有效的 LLM 对齐方法。我们从有用性与无害性两个方面构建了 DPO 训练的偏好数据。对于有用性数据，我们收集了多语言 prompt，涵盖创意写作、问答、指令遵循等类别，然后使用我们的 DeepSeek Chat 模型生成回复作为候选回复。无害性偏好数据的构建也采用了类似的操作。

我们对 DPO 训练了 1 个 epoch，学习率为 5e-6，批大小为 512，并使用了学习率 warmup 与余弦学习率调度器。我们发现，DPO 能够增强模型的开放式生成能力，而对标准基准上的表现几乎没有影响。

5 评估

5.1 公开基准评估

我们在一系列英文与中文公开基准上评估了我们的模型，评估基于内部评估框架。

- 多学科多项选择数据集，包括 MMLU（Hendrycks et al., 2020）、C-Eval（Huang et al., 2023）与 CMMLU（Li et al., 2023）。
- 语言理解与推理数据集，包括 HellaSwag（Zellers et al., 2019）、PIQA（Bisk et al., 2020）、ARC（Clark et al., 2018）、OpenBookQA（Mihaylov et al., 2018）与 BigBench Hard（BBH）（Suzgun et al., 2022）。
- 闭卷问答数据集，包括 TriviaQA（Joshi et al., 2017）与 NaturalQuestions（Kwiatkowski et al., 2019）。
- 阅读理解数据集，包括 RACE（Lai et al., 2017）与 DROP（Dua et al., 2019）、C3（Sun et al., 2019）。
- 指代消歧数据集，包括 WinoGrande（Sakaguchi et al., 2019）与 CLUEWSC（Xu et al., 2020）。

- 语言建模数据集，包括 Pile（Gao et al., 2020）。
- 中文理解与文化数据集，包括 CHID（Zheng et al., 2019）与 CCPM（Li et al., 2021）。
- 数学数据集，包括 GSM8K（Cobbe et al., 2021）、MATH（Hendrycks et al., 2021）与 CMath（Wei et al., 2023）。
- 代码数据集，包括 HumanEval（Chen et al., 2021）与 MBPP（Austin et al., 2021）。
- 标准化考试，包括 AGIEval（Zhong et al., 2023）。

对于需要从若干选项中选出答案的数据集，我们采用基于困惑度（perplexity）的评估方法。这些数据集包括 HellaSwag、PIQA、WinoGrande、RACE-Middle、RACE-High、MMLU、ARC-Easy、ARC-Challenge、OpenBookQA、CHID、C-Eval、CMMLU、C3 与 CCPM。这里的基于困惑度的评估是指计算每个选项的困惑度，并选择困惑度最低的选项作为模型预测。对于 ARC 与 OpenBookQA，我们使用无条件归一化（Brown et al., 2020）来计算困惑度，其他数据集则使用长度归一化。

我们对 TriviaQA、NaturalQuestions、DROP、MATH、GSM8K、HumanEval、MBPP、BBH、AGIEval、CLUEWSC 与 CMath 采用基于生成的评估方法。这里的基于生成的评估是指让模型生成自由文本，并从生成文本中解析出结果。对于基于生成的评估，我们使用贪心解码。

我们对 Pile-test 采用基于语言建模的评估方法，即计算测试语料上的 bits-per-byte。

针对不同基准，我们使用 2048 或 4096 作为最大序列长度。评估格式的详情见附录 A.6。

5.1.1 基座模型

表 5 给出了评估基准上的主要结果。尽管 DeepSeek 模型是在 2T 中英双语语料上预训练的，但它们在英语语言理解基准上展现出了与 LLaMA2 模型相当的性能——后者同样消耗 2T token，但专注于英语。此外，与 LLaMA2 70B 相比，DeepSeek 67B 在 MATH、GSM8K、HumanEval、MBPP、BBH 以及中文基准上取得了明显更好的性能。我们在附录 A.3 中给出了基准曲线。可以看到，一些任务的性能会随着模型规模扩大而提升，例如 GSM8K 与 BBH。鉴于我们的 7B 与 67B 模型都在同一数据集上训练，这种提升的出现可以归因于大模型强大的少样本学习能力。然而，随着数学数据比例的增加，小模型与大模型之间的差距可能会缩小。

一个有趣的观察是，DeepSeek 67B 相对 LLaMA2 70B 的优势，大于 DeepSeek 7B 相对 LLaMA2 7B 的优势。这一现象凸显出，语言冲突对较小模型的影响更大。此外，尽管 LLaMA2 并未专门在中文数据上训练，它在某些中文任务（如 CMath）上表现令人瞩目。这表明某些基础能力（如数学推

理）可以有效地跨语言迁移。然而，像 CHID 这样评估中文成语使用情况的任务，需要模型在预训练期间消费大量中文 token。在这种情况下，LLaMA2 的表现显著逊于 DeepSeek LLM。

【表格】表 5 主要结果。我们报告的评估结果基于内部评估框架。加粗数字表示 4 个模型中的最佳结果。对 Pile-test，我们报告 bits-per-byte（BPB）；对 DROP，我们报告 F1 分数；其他任务报告准确率。注意 test-shots 为最大值，由于上下文长度有限，或 RACE 等阅读理解任务中同一段落内可用的少样本示例有限，实际可能采用更少的 shot（原文第 15 页）

5.1.2 Chat 模型

表 6 展示了 DeepSeek Chat 模型的结果，表明微调后大多数任务整体上有所提升。然而，也有少数情况下某些任务的性能出现了下降。

知识：我们观察到基座模型与 chat 模型在知识相关任务（如 TriviaQA、MMLU 与 C-Eval）上存在波动。然而，我们认为这种轻微波动并不表示 SFT 之后知识的获得或丢失。SFT 的价值在于：让 chat 模型在零样本设置下能够取得与基座模型少样本设置相当的分数，这与真实场景相一致。例如，chat 模型的 0-shot MMLU 性能与基座模型的 5-shot MMLU 性能相当。

推理：由于相当大比例的 SFT 实例采用 CoT 格式（Wei et al., 2022），chat 模型在推理任务（如 BBH 与 NaturalQuestions）上表现出轻微提升。然而，我们认为 SFT 阶段学习到的并非推理能力，而是正确的推理路径格式。

【表格】表 6 基座模型与 chat 模型的对比。对 MMLU、GSM8K、MATH、C-Eval 与 CMMLU，我们以 0-shot 评估 chat 模型，而基座模型的结果仍以少样本设置获得（原文第 16 页）

性能下降的任务：无论模型规模大小或选择哪个预训练检查点，少数任务的性能在微调后都会持续下降。这些特定任务通常涉及完形填空或句子补全任务，例如 HellaSwag。可以合理地假设，纯语言模型更适合处理这类任务。

数学与代码：微调后，我们的模型在数学与编程任务上表现出显著提升。例如，HumanEval 与 GSM8K 的分数提升了 20 分以上。我们的解释是：基座模型最初在这些任务上处于欠拟合状态，而 SFT 阶段通过大量 SFT 数据学习到了额外的编程与数学知识。然而，需要注意的是，模型的能力可能主要集中于代码补全与代数问题。要形成对数学与编程的全面理解，在预训练阶段引入多样化的数据至关重要，这留待未来工作。我们在附录 A.4 中对代码与数学任务进行了详细分析。

在 7B 模型的微调中，我们最初使用全部数据对模型进行微调。随后引入了第二阶段，该阶段排除了数学与代码数据。这一做法的动机是：第一阶段模型的重复比例为 2.0%，降至 1.4%

在第二阶段微调后，模型在基准测试上的分数保持不变。对于 67B 模型，经过第一阶段微调后重复率已低于 1%，而第二阶段反而会损害模型在基准测试上的得分。因此，67B 模型只进行了一个阶段的 SFT。

5.2 开放式评测

对于对话模型而言，除了观察其在标准基准上的指标外，开放领域与开放式问题所生成内容的质量还会直接影响实际用户体验。因此，我们分别测试了对话模型在中文与英文任务上的开放式生成能力。

5.2.1 中文开放式评测

在中文开放式评测方面，我们使用高质量的开放式问题测试集 AlignBench（Liu et al., 2023），在不同领域中测试对话模型的综合能力。AlignBench 共包含 8 个一级类别、36 个二级类别，涵盖 683 个问题。对于每个问题，除提示词外，AlignBench 还提供专业参考答案与评分模板，供 GPT-4 判断回答质量。

我们使用 AlignBench 官方 GitHub 代码仓库来实现对模型的评测。我们将关键的温度参数与原始设置严格对齐：对于角色扮演、写作能力与开放式问题，生成温度设为 0.7；而对于其他任务，生成温度设为 0.1。

【表格】表 7 AlignBench 排行榜，由 gpt-4-0613 评分。模型按总分降序排列；带 * 的结果是我们基于 AlignBench 官方仓库的评测结果，其余结果均来自 AlignBench 论文。我们发现，我们的 DeepSeek-67B-Chat 模型以明显优势超越 ChatGPT 及其他基线模型，这表明我们的模型在基础中文语言任务与高级中文推理任务上均具有更优表现。此外，我们可以看到 DPO 过程几乎在所有领域都带来了提升。（原文第 17 页）

AlignBench 排行榜如表 7 所示。可以看到，我们的 DeepSeek 67B Chat 模型超越了 ChatGPT 及其他基线模型，仅次于两个版本的 GPT-4。这表明，与其他开源或专有中文大语言模型相比，我们的模型在各种中文任务上均表现优异。DPO 模型在几乎所有指标上都显示出提升，这证明了 DPO 训练过程对模型对齐的积极影响。

在基础中文语言任务方面，我们的模型处于所有模型中的第一梯队，DPO 模型的中文基础语言能力甚至高于最新版本的 GPT-4。在高级中文推理任务方面，我们的模型得分显著高于其他中文 LLM，且领先优势明显，证明了我们的模型在更复杂的中文逻辑推理与数学计算方面具有卓越性能。

5.2.2 英文开放式评测

对于英文开放式评测，我们使用 MT-Bench 基准（Zheng et al., 2023），它包含 8 个不同类别的多轮问题。如表 8 所示，我们的 DeepSeek LLM 67B Chat 优于 LLaMA-2-Chat 70B（Touvron et al., 2023b）、Xwin 70b v0.1 与 TÜLU 2+DPO 70B（Ivison et al., 2023）等其他开源模型，取得 8.35 的分数，与 GPT-3.5-turbo 相当。此外，经过 DPO 阶段后，我们的 DeepSeek LLM 67B Chat DPO 进一步将平均分提升至 8.76，仅次于 GPT-4（OpenAI, 2023）。这些结果说明了 DeepSeek LLM 强大的多轮开放式生成能力。

【表格】表 8 MT-Bench 评测结果。带 * 的结果引自 Ivison et al. (2023)。（原文第 18 页）

5.3 留出评测

数据污染与基准过拟合是评估 LLM 时的两大挑战。一种常见做法是利用近期发布的测试集作为留出测试集来评估模型。

LeetCode：为评估模型的编程能力，我们使用了 LeetCode 周赛中的题目（Weekly Contest 351–372、Bi-Weekly Contest 108–117，时间为 2023 年 7 月至 2023 年 11 月）。我们通过爬取 LeetCode 数据获得这些题目，共 126 道题，每题有 20 多个测试用例。所采用的评测指标与 HumanEval 类似：如果模型的输出成功通过所有测试用例，则认为模型有效解决了该问题。模型的编程能力如下图所示：纵轴表示模型在域内人工评测（in-domain human evaluation testing）上的 pass@1 分数，横轴表示模型在域外 LeetCode 周赛题目上的 pass@1 分数。LeetCode 测试数据将很快随 DeepSeek Coder 技术报告一并发布。

【图表】图 6 域内人工评测 pass@1 与域外 LeetCode 周赛 pass@1 的对比（原文第 19 页）

匈牙利国家高中考试：与 Grok-1 类似，我们使用匈牙利国家高中考试（Hungarian National High School Exam）来评估模型的数学能力。该考试包含 33 道题，模型得分通过人工标注确定。我们遵循 solution.pdf 中的评分标准来评估所有模型。

指令遵循评测：2023 年 11 月 15 日，Google 发布了一个指令遵循评测数据集（Zhou et al., 2023）。他们识别出 25 种可验证的指令类型，并构建了约 500 条提示词，每条提示词包含一条或多条可验证指令。我们使用提示词级别的宽松（loose）指标来评估所有模型。

【表格】表 9 留出数据集评测结果。（原文第 19 页）

我们将自己的模型与不同规模的各种基线模型进行了对比分析，包括 Qwen 72B Chat（Bai et al., 2023）、ChatGLM3（Du et al., 2022）、Baichuan2（Yang et al., 2023）与 Yi-34B Chat。我们的观察表明，在这些留出数据集上，大模型与小模型之间存在显著的性能差距，即便某些小模型在传统基准上取得了不错的结果。例如，ChatGLM3 在代码测试集 MBPP 上取得 52.4 分，接近 DeepSeek 67B；然而在新基准上评测时，其表现明显逊于 DeepSeek 67B。在数学数据集上也观察到类似趋势：ChatGLM3 在 GSM8K 上非常强（72.3），但其匈牙利考试得分不如大模型。此外，指令遵循能力表明，总计算量起着至关重要的作用。

DeepSeek 7B 与 67B 模型采用相同的训练流程，但两者表现存在显著差异。通过主观评测，我们观察到，当模型规模扩展到 67B 时，模型在各种任务上展现出的智能水平有明显跃升。尽管 DeepSeek 7B 在标准基准上落后于其他较小的语言模型，但与其他模型相比，其在留出任务上的表现相对可圈可点。

5.4 安全评测

我们深刻认识到安全对于通用人工智能的重要性。构建真正有帮助的人工智能模型的前提是：它拥有与人类一致的价值观，并对人类表现出友好。我们在包括预训练、SFT 与 DPO 在内的整个训练过程中都融入了对模型安全的保障。

为验证模型的安全性，我们组建了一支来自不同学科的 20 人专家团队，并构建了一套与人类价值观一致的安全内容分类体系（安全评测分类体系见表 10）。随后，专家团队为每个安全子类别人工构建了数十个高质量测试用例。除了关注安全内容领域的多样性，我们也注重安全内容形式的多样性。臭名昭著的“奶奶”漏洞（grandmother loophole）说明，模型可能被查询的表面形式所蒙骗，从而给出不安全的回答。因此，在设计问题时，专家团队同样注重提问方式的多样化。他们通过诱导、角色扮演、多轮对话、预设立场等方式构建了多样化的安全问题。最终，我们得到了一个包含 2400 个问题的安全测试集。此外，专家团队还针对每一种不同的内容类型与形式类型，构建了用于安全审查的基本准则（constitution）。

类别	子类别	安全回答数/总用例数
歧视偏见问题（Discrimination and Prejudice Questions）	民族种族、宗教信仰、国别地域、性别、年龄、职业、健康、其他方面歧视	486/500
侵犯他人合法权益（Infringement of Others' Legal Rights）	身心健康、合法财产、肖像权、名誉权、荣誉权、隐私权、信息权益、其他合法权益	473/500
商业秘密与知识产权（Trade Secrets and Intellectual Property Rights）	侵犯他人知识产权、垄断和不正当竞争行为、其他商业违法违规行为、违反商业道德、泄露他人商业机密	281/300
违法违规行为（Illegal and Non-compliant Behavior）	邪教迷信、色情、赌博、毒品和违禁品、侮辱谩骂、暴力行为、涉黑涉恶、其他违法违规行为	290/300
其他安全问题（Other Safety Issues）	幻觉和真实性问题、时效性问题、自我认知问题、其他敏感话题	767/800

表 10 我们的安全评测分类体系。表中列出了每个类别的测试用例总数以及我们的模型（DeepSeek-67B-Chat）给出的安全回答数量，见表格最右列。测试问题的标注与生成结果的评估由专业人工团队完成。可以看到，我们的模型在各类安全测试集上均展现出较强的安全性。

对于模型在该测试集上的输出结果，我们人工检查了其安全性。我们的审查团队训练有素，并对标注结果进行了交叉验证。标注人员对每个问题作三类标注：安全、不安全、模型拒答。我们测试了 DeepSeek 67B Chat 模型的安全性，结果见表 10。表中列出了每个安全类别的测试问题数量以及我们的模型通过的安全测试数量。我们将安全作答与模型拒答的测试用例都标记为安全回答。结果表明，我们的模型在众多安全测试类别上均表现出良好的安全性能。

作为对现有安全方法的补充，我们进一步使用“Do-Not-Answer”数据集（Wang et al., 2023）来评估 DeepSeek 67B Chat 模型的安全机制，从而丰富了我们的评测。该数据集的 939 条按风险分类的提示词，帮助我们凸显了模型增强后的能力。如表 11 所示，DeepSeek 67B Chat 模型表现突出，取得 97.8 分，高于 ChatGPT 与 GPT-4。这一分数不仅衡量了我们模型安全处理敏感查询的能力，也使其在该领域处于领先模型的竞争行列。

【表格】表 11 Do-Not-Answer 得分（Wang et al., 2023），分数越高表示模型越安全。带 * 的结果为我们基于官方仓库的评测结果，其余结果均来自原论文。可以看到，我们模型的安全得分高于 ChatGPT 与 GPT-4，跻身最安全模型

之列。（原文第 21 页）

5.5 讨论

在整个开发过程中，我们在构建 LLM 时发现了一些有趣的结论。

分阶段微调（Staged Fine-Tuning）：正如上文所述，小模型需要在数学与代码数据集上进行更长时间的微调，但这会损害模型的对话能力，例如加重重复行为。为解决这一问题，我们实现了分阶段微调流程。在该方法中，第一阶段使用所有可用数据进行微调，第二阶段则专门使用对话数据进行微调。

模型	HumanEval	GSM8K	重复率	IFEval
DeepSeek LLM 7B Chat Stage1	48.2	63.9	0.020	38.0
DeepSeek LLM 7B Chat Stage2	48.2	63.0	0.014	41.2

表 12 两阶段微调结果。重复率在温度为 0 时计算，重复率越低越好。IFEval 结果为提示词级别的宽松准确率。

表 12 展示了两阶段训练过程得到的结果。这些结果清楚地表明，第二阶段不会损害模型在代码与数学方面的能力，同时还能减少重复行为并增强指令遵循能力。

选择题（Multi-Choice Question）：使用选择题形式的评测数据（如 MMLU、AGI Eval 与 C-Eval）来测试模型是一种常见做法。选择题不仅要求模型具备相应的知识，还要求模型理解选项所指的内容。在对齐阶段，我们测试了加入 2000 万道中文选择题的效果，性能如表 13 所示。需要说明的是，我们对 C-Eval 验证集与 CMMLU 测试集进行了去重，以防止数据污染。

模型	MMLU	C-Eval	CMMLU	TriviaQA	ChineseQA
DeepSeek LLM 7B Chat	49.4	47.0	49.7	57.9	75.0
DeepSeek LLM 7B Chat + MC	60.9	71.3	73.8	57.9	74.4

表 13 增加选择题数据的影响。

额外加入 2000 万条选择题（MC，multiple-choice）数据，不仅有利于中文选择题基准，也有助于提升英文基准的表现。这表明模型解决选择题的能力得到了增强。然而，我们观察到这种提升并未延伸到其他不采用选择题形式的评测上，例如 TriviaQA 与我们内部的 ChineseQA 测试集，它们都是生成式评测基准。这说明在对话交互中，用户可能并不会觉得模型变得更智能，因为这些交互涉及的是生成回答，而非解决选择题。

因此，我们选择在预训练与微调阶段均排除选择题数据，因为纳入这类数据会导致对基准的过拟合，无助于模型实现真正的智能。

预训练中的指令数据：人们普遍认为，在预训练阶段后期引入指令数据能够提升基座模型在基准任务上的表现。在我们的研究中，我们在预训练最后 10% 的阶段加入了 500 万条指令数据，主要由选择题构成。我们观察到，基座模型在基准上的表现确实有所提升。然而，最终结果与在 SFT 阶段加入相同数据所取得的结果几乎相同。我们得出结论：虽然这种做法增强了基座模型在基准上的表现，但其整体潜力等同于不引入这些指令数据。如果指令数据规模可观，将其纳入预训练过程是可以接受的。由于我们倾向于排除选择题，且我们拥有的非选择题数量有限，我们决定不在预训练过程中纳入指令数据。

系统提示词（System Prompt）：设计良好的系统提示词应能有效引导模型生成既有用又尊重用户的回答。我们对 LLaMA-2 提出的提示词稍作修改，作为我们的系统提示词。

系统提示词：

> 你是 DeepSeek Chat，一个由 DeepSeek 开发的乐于助人、尊重用户且诚实的 AI 助手。你的训练数据知识截止日期为 2023 年 5 月。请始终在保证安全的前提下尽可能提供有帮助的回答。你的回答不应包含任何有害、不道德、种族主义、性别歧视、有毒、危险或非法的内容。请确保你的回答在社会意义上无偏见且积极正面。如果某个问题没有意义或在事实上不成立，请解释原因，而不是给出不正确的回答。如果你不知道某个问题的答案，请不要分享虚假信息。

我们观察到一个有趣的现象：引入系统提示词后，7B LLM 的性能会出现轻微下降；然而在使用 67B LLM 时，加入提示词会带来显著改善的结果，如表 14 所示。我们对这种差异的解释是：更大的模型能更好地理解系统提示词背后的意图，从而更有效地遵循指令并生成更优的回答。另一方面，较小的模型难以充分把握系统提示词，且训练与测试之间的不一致可能对其性能产生负面影响。

模型	MT Bench
DeepSeek LLM 7B Chat	7.15
DeepSeek LLM 7B Chat + System Prompt	7.11

DeepSeek LLM 67B Chat	8.35
DeepSeek LLM 67B Chat + System Prompt	8.58

表 14 增加系统提示词的影响。

6 结论、局限与未来工作

我们介绍了 DeepSeek LLM——一系列在包含中英文、规模达 2 万亿（2 trillion）token 的庞大数据集上从零开始训练的开源模型。在本文中，我们深入阐释了超参数的选择、scaling laws，以及我们所做的各种微调尝试。我们对先前工作中的 scaling laws 进行了校准，并提出了一种新的最优模型/数据放大分配策略。此外，我们提出了一种在给定算力预算下预测近最优批量大小与学习率的方法。我们进一步得出结论：scaling laws 与数据质量相关，这可能是不同工作中缩放行为存在差异的根本原因。在 scaling laws 的指导下，我们使用最佳超参数进行预训练，并提供了全面的评测。我们在所有训练阶段都避免对基准进行粉饰，也不使用任何暗箱手段。

DeepSeek Chat 同样存在其他 LLM 普遍公认的局限，包括：预训练之后缺乏持续的知识更新；可能生成不符合事实的信息，例如未经核实的建议；以及倾向于产生幻觉。此外，需要指出的是，我们初始版本的中文数据并不完备，这可能导致模型在部分中文特定话题上表现欠佳。由于我们的数据主要由中文与英文来源构成，模型在其他语言上的能力仍然较为薄弱，使用时应保持谨慎。

DeepSeek LLM 是一个致力于推进开源语言模型发展的长期项目。

- 很快，我们将分别发布代码智能与混合专家（Mixture-of-Experts, MoE）领域的技术报告。它们将展示我们如何为预训练构建高质量的代码数据，以及如何设计稀疏模型以达到稠密模型的性能。
- 目前，我们正在为下一代 DeepSeek LLM 构建规模更大、质量更高的数据集。我们希望下一版本在推理、中文知识、数学与代码能力上都将有显著提升。
- 我们的对齐团队致力于研究如何向公众提供有用、诚实且安全的模型。我们的初步实验证明，强化学习能够提升模型的复杂推理能力。

附录

A.1 致谢

A.2 不同模型规模的表示方法

A.3 基准指标曲线

A.4 与代码或数学专用模型的比较

A.5 含 DPO 阶段的基准结果

A.6 评估格式

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] 6. Conclusion, Limitation, and Future Work

[2] We introduce DeepSeek LLMs, a series of open-source models trained from scratch on a vast dataset of 2 trillion tokens in both English and Chinese. In this paper, we provide an in-depth explanation of hyper-parameters selection, scaling laws, as well as the various fine-tuning attempts we made. We calibrate the scaling laws in the previous work and propose a new optimal model/data scaling-up allocation strategy. In addition, we present a method to predict the near-optimal batch size and learning rate with given compute budget. We further conclude that the scaling laws is related to the data quality, which might be the root cause of varying scaling behavior in different works. Guided by the scaling laws, we conduct pre-training with the best hyper-parameter and provide a comprehensive evaluation. We avoid benchmark decoration and dark secrets in all training stages.

[3] DeepSeek Chat shares the acknowledged limitations commonly found in other LLMs, which include the lack of ongoing knowledge updates after pre-training, the possibility of generating non-factual information such as unverified advice, and a tendency to produce hallucinations. Moreover, it is important to note that our initial version of Chinese data is not exhaustive, which may result in suboptimal performance on certain Chinese-specific topics. Since our data primarily consists of Chinese and English sources, the model's proficiency in other languages remains delicate and should be approached with caution.

[4] DeepSeek LLM is a long-term project committed to advancing open-source language models.

[5] • Soon, we will release our technique reports in code intelligence and Mixture-of-Experts (MoE), respectively. They show how we create high-quality code data for pre-training, and design a sparse model to achieve dense model performance. • At present, we are constructing a larger and improved dataset for the upcoming version of DeepSeek LLM. We hope the reasoning, Chinese knowledge, math, and code capabilities will be significantly improved in the next version. • Our alignment team is dedicated to studying ways to deliver a model that is helpful, honest, and safe to the public. Our initial experiments prove that reinforcement learning could boost model complex reasoning capability.

[6] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

[7] Anthropic. Introducing Claude, 2023. URL https://www.anthropic.com/index/introd ucing-claude .

[8] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

[9] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, et al. Qwen technical report. arXiv preprint arXiv:2309.16609, 2023.

[10] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi. PIQA: reasoning about physical commonsense in natural language. In The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020, pages 7432–7439. AAAI Press, 2020. doi: 10.1609/aaai.v34i05.6239. URL https://doi.org/10.1609/aaai.v34i05.6239 .

[11] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P . Dhariwal, A. Neelakantan, P . Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners, 2020.

[12] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374 .

[13] P . Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. CoRR, abs/1803.05457, 2018. URL http://arxiv.org/abs/1803.05457 .

[14] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[15] T. Computer. Redpajama: an open dataset for training large language models, 2023. URL https://github.com/togethercomputer/RedPajama-Data .

[16] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V . Le, and R. Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. arXiv preprint arXiv:1901.02860, 2019.

[17] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. 2023.

[18] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In Advances in Neural Information Processing Systems, 2022.

[19] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang. Glm: General language model pretraining with autoregressive blank infilling. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 320–335, 2022.

[20] D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368– 2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246 . L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

[21] Google. An important next step on our AI journey, 2023. URL https://blog.google/tech nology/ai/bard-google-ai-search-updates/ .

[22] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen. Tora: A toolintegrated reasoning agent for mathematical problem solving. CoRR, abs/2309.17452, 2023. doi: 10.48550/ARXIV.2309.17452. URL https://doi.org/10.48550/arXiv.2309.1745 2 .

[23] P . Goyal, P . Dollár, R. Girshick, P . Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He. Accurate, large minibatch sgd: Training imagenet in 1 hour. arXiv preprint arXiv:1706.02677, 2017.

[24] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[25] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[26] T. Henighan, J. Kaplan, M. Katz, M. Chen, C. Hesse, J. Jackson, H. Jun, T. B. Brown, P . Dhariwal, S. Gray, et al. Scaling laws for autoregressive generative modeling. arXiv preprint arXiv:2010.14701, 2020.

[27] J. Hestness, S. Narang, N. Ardalani, G. Diamos, H. Jun, H. Kianinejad, M. M. A. Patwary, Y. Yang, and Y. Zhou. Deep learning scaling is predictable, empirically. arXiv preprint arXiv:1712.00409, 2017.

[28] High-flyer. Hai-llm: 高 效 且 轻 量 的 大 模 型 训 练 工 具 , 2023. URL https://www.high-flyer.c n/en/blog/hai-llm .

[29] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models. CoRR, abs/2203.15556, 2022. doi: 10.48550 /ARXIV.2203.15556. URL https://doi.org/10.48550/arXiv.2203.15556 .

[30] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

[31] Huggingface Team. Tokenizers: Fast state-of-the-art tokenizers optimized for research and production, 2019. URL https://github.com/huggingface/tokenizers .

[32] F. i, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/pdf?id= fR3wGCk-IXp . H. Ivison, Y. Wang, V . Pyatkin, N. Lambert, M. Peters, P . Dasigi, J. Jang, D. Wadden, N. A. Smith, I. Beltagy, and H. Hajishirzi. Camels in a changing climate: Enhancing lm adaptation with tulu 2. 2023.

[33] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7b. arXiv preprint arXiv:2310.06825, 2023.

[34] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147 .

[35] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. CoRR, abs/2001.08361, 2020. URL https://arxiv.org/abs/2001.08361 .

[36] V . A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro. Reducing activation recomputation in large transformer models. Proceedings of Machine Learning and Systems, 5, 2023.

[37] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. P . Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: a benchmark for

question answering research. Trans. Assoc. Comput. Linguistics, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://doi.org/10.1162/tacl_a_00276 .

[38] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 2023.

[39] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. H. Hovy. RACE: large-scale reading comprehension dataset from examinations. In M. Palmer, R. Hwa, and S. Riedel, editors, Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017, Copenhagen, Denmark, September 9-11, 2017, pages 785–794. Association for Computational Linguistics, 2017. doi: 10.18653/V1/D17-1082. URL https://doi.org/10.18653/v1/d1 7-1082 .

[40] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.

[41] W. Li, F. Qi, M. Sun, X. Yi, and J. Zhang. Ccpm: A chinese classical poetry matching dataset, 2021.

[42] X. Liu, X. Lei, S. Wang, Y. Huang, Z. Feng, B. Wen, J. Cheng, P . Ke, Y. Xu, W. L. Tam, X. Zhang, L. Sun, H. Wang, J. Zhang, M. Huang, Y. Dong, and J. Tang. Alignbench: Benchmarking chinese alignment of large language models. CoRR, abs/2311.18743, 2023. doi: 10.48550/A RXIV.2311.18743. URL https://doi.org/10.48550/arXiv.2311.18743 .

[43] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017. H. Luo, Q. Sun, C. Xu, P . Zhao, J. Lou, C. Tao, X. Geng, Q. Lin, S. Chen, and D. Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. arXiv preprint arXiv:2308.09583, 2023.

[44] S. McCandlish, J. Kaplan, D. Amodei, and O. D. Team. An empirical model of large-batch training. arXiv preprint arXiv:1812.06162, 2018.

[45] T. Mihaylov, P . Clark, T. Khot, and A. Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering, 2018.

[46] D. Narayanan, M. Shoeybi, J. Casper, P . LeGresley, M. Patwary, V . Korthikanti, D. Vainbrand, P . Kashinkunti, J. Bernauer, B. Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–15, 2021.

[47] OpenAI. Introducing ChatGPT, 2022. URL https://openai.com/blog/chatgpt .

[48] arXiv preprint arXiv:2303.08774, 2023. OpenAI. GPT4 technical report.

[49] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P . Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35:27730–27744, 2022.

[50] G. Penedo, Q. Malartic, D. Hesslow, R. Cojocaru, A. Cappelli, H. Alobeidli, B. Pannier, E. Almazrouei, and J. Launay. The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. arXiv preprint arXiv:2306.01116, 2023.

[51] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. OpenAI blog, 1(8):9, 2019.

[52] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. 2023.

[53] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.

[54] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.

[55] C. J. Shallue, J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, and G. E. Dahl. Measuring the effects of data parallelism on neural network training. Journal of Machine Learning Research, 20(112):1–49, 2019.

[56] N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.

[57] M. Shoeybi, M. Patwary, R. Puri, P . LeGresley, J. Casper, and B. Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053, 2019.

[58] S. Smith, M. Patwary, B. Norick, P . LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhumoye, G. Zerveas, V . Korthikanti, et al. Using deepspeed and megatron to train megatron-turing nlg 530b, a large-scale generative language model. arXiv preprint arXiv:2201.11990, 2022.

[59] S. L. Smith, P .-J. Kindermans, C. Ying, and Q. V . Le. Don’t decay the learning rate, increase the batch size. arXiv preprint arXiv:1711.00489, 2017. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.

[60] K. Sun, D. Yu, D. Yu, and C. Cardie. Investigating prior knowledge for challenging chinese machine reading comprehension, 2019.

[61] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[62] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023a.

[63] H. Touvron, L. Martin, K. Stone, P . Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P . Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton-Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V . Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V . Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P . S. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P . Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P . Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. CoRR, abs/2307.09288, 2023b. doi: 10.48550/arXiv.2307.09288. URL https://doi.org/10.48550/arXiv.2307. 09288 .

[64] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[65] Y. Wang, H. Li, X. Han, P . Nakov, and T. Baldwin. Do-not-answer: A dataset for evaluating safeguards in llms. CoRR, abs/2308.13387, 2023. doi: 10.48550/ARXIV.2308.13387. URL https://doi.org/10.48550/arXiv.2308.13387 .

[66] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V . Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In NeurIPS, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf 4f15af0f7b31abca4-Abstract-Conference.html .

[67] T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.

[68] L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL https://doi.org/10.18653/v1/2020.coling-main.419 .

[69] A. Yang, B. Xiao, B. Wang, B. Zhang, C. Yin, C. Lv, D. Pan, D. Wang, D. Yan, F. Yang, F. Deng, F. Wang, F. Liu, G. Ai, G. Dong, H. Zhao, H. Xu, H. Sun, H. Zhang, H. Liu, J. Ji, J. Xie, J. Dai, K. Fang, L. Su, L. Song, L. Liu, L. Ru, L. Ma, M. Wang, M. Liu, M. Lin, N. Nie, P . Guo, R. Sun, T. Zhang, T. Li, T. Li, W. Cheng, W. Chen, X. Zeng, X. Wang, X. Chen, X. Men, X. Yu, X. Pan, Y. Shen, Y. Wang, Y. Li, Y. Jiang, Y. Gao, Y. Zhang, Z. Zhou, and Z. Wu. Baichuan 2: Open large-scale language models. Technical report, Baichuan Inc., 2023. URL https://cdn.baichuan-ai.com/paper/Baichuan2-technical-report.pdf .

[70] L. Yu, W. Jiang, H. Shi, J. Yu, Z. Liu, Y. Zhang, J. T. Kwok, Z. Li, A. Weller, and W. Liu. Metamath: Bootstrap your own mathematical questions for large language models. CoRR, abs/2309.12284, 2023. doi: 10.48550/ARXIV.2309.12284. URL https://doi.org/10.485 50/arXiv.2309.12284 .

[71] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[72] B. Zhang and R. Sennrich. Root mean square layer normalization. Advances in Neural Information Processing Systems, 32, 2019.

[73] G. Zhang, L. Li, Z. Nado, J. Martens, S. Sachdeva, G. Dahl, C. Shallue, and R. B. Grosse. Which algorithmic choices matter at which batch sizes? insights from a noisy quadratic model. Advances in neural information processing systems, 32, 2019.

[74] C. Zheng, M. Huang, and A. Sun. Chid: A large-scale chinese idiom dataset for cloze test. In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 778–787. Association for Computational Linguistics, 2019. doi: 10.18653/V1/P19-1075. URL https://doi.org/10.18653/v1/p19-1075 .

[75] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P . Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. 2023.

[76] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364 .

[77] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models. arXiv preprint arXiv:2311.07911, 2023. A. Appendix

[78] This project was realized thanks to the efforts of numerous contributors. We offer our extended thanks to the following individuals for their help :

[79] • Data Annotation Team: Jialu Cai, Ruijian Chen, Ruyi Chen, Bei Feng, Yanping Huang, Zhen Huang, Pin Jiang, Rongli Jin, Xiangyue Jin, Ziyun Ke, Hui Li, Meng Li, Sangsang Li, Xiaoqian Li, Yaohui Li, Yunxian Ma, Jiaqi Ni, Xiaojin Shen, Xinnan Song, Tianyu Sun, Xiaosha Chen, Haoyuan Tian, Xiaohan Wang, Xiaoxiang Wang, Yuhao Wang, Fanyi Xia, Lei Xu, Zeyuan Xu, Zhipeng Xu, Tian Yuan, Zhongyu Zhang, Yi Zheng, Shuang Zhou, Xinyi Zhou, Yuchen Zhu, Yuxuan Zhu. • Compliance Team:

Jin Chen, Ying Tang, Miaojun Wang, Xianzu Wang, Shaoqing Wu, Leyi Xia, W.L. Xiao. • Business Team: Jian Liang, Mingming Li, T. Wang, Xianzu Wang, Zhiniu Wen, Shengfeng Ye, Peng Zhang, Zhen Zhang. • Design Team: Wei An, Yukun Zha.

[80] A.2. Different Model Scale Representations

[81] We refitted the scaling curve for different model scale representations, reusing the experiments from the IsoFLOP profile. We recalculated the compute FLOPs using6 N and 6 N as model scale 2 1 representations and refitted the performance scaling curves. As shown in Figure 6, the results indicate that the deviation of optimal model/data allocation among these three representations is not significant at higher compute budgets, but there are noticeable differences at lower budgets.

[82] (a) Compute budget C = 6 N D (b) Compute budget C = 6 N D (c) Compute budget C = MD 2 1

[83] Figure 6 | Performance scaling curves using different model scale representations. The metric is the bits-per-byte on the validation set. The dotted line represents the power law fitting the smaller model (grey circles). The blue stars represent DeepSeek LLM 7B and 67B. N , N , 2 1 and M represent the non-embedding parameters, complete parameters, and non-embedding FLOPs/token of the model, respectively.

[84] When using 6 N as the model scale representation, the fitted performance scaling curve tends to overestimate the performance of large-scale models. Conversely, when using 6 N , the

[85] Authors are ordered alphabetically by the last name.

DeepSeekMoE：迈向混合专家语言模型的极致专家专业化

DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

DeepSeek-AI　research@deepseek.com

arXiv:2401.06066 · 中文翻译版（正文全文；图表保留原文页码位置）

摘　要

在大语言模型时代，混合专家（MoE）是在扩展模型参数规模时管理计算成本的一种有前景的架构。然而，GShard 等传统 MoE 架构从 N 个专家中激活 top-K 个，在确保专家专业化（即每个专家获得互不重叠且聚焦的知识）方面面临挑战。为此，我们提出了面向极致专家专业化的 DeepSeekMoE 架构。它包含两项主要策略：（1）将专家细分为 mN 个，并从中激活 mK 个，从而实现更灵活的激活专家组合；（2）隔离出 Ks 个专家作为共享专家，旨在捕获共性知识并缓解路由专家中的冗余。我们从 2B 参数的适度规模起步，证明 DeepSeekMoE 2B 可达到与 GShard 2.9B 相当的性能，而后者的专家参数与计算量均为 1.5 倍。此外，DeepSeekMoE 2B 的性能接近其总参数量相同的稠密对应模型——该模型构成 MoE 模型性能的上界。随后我们将 DeepSeekMoE 扩展至 16B 参数，表明其以约 40% 的计算量即可达到与 LLaMA2 7B 相当的性能。进一步地，我们将 DeepSeekMoE 扩展至 145B 参数的初步尝试持续验证了其相对 GShard 架构的显著优势，并显示其性能可与 DeepSeek 67B 相比，而计算量仅为其 28.5%（甚至可能为 18.2%）。

1 引言

近期，各项研究与工程实践已实证表明：只要有充足的训练数据，通过增加参数与计算预算来扩展语言模型，就能得到显著更强的模型（Brown 等，2020；Hoffmann 等，2022；OpenAI，2023；Touvron 等，2023a）。然而必须承认，将模型扩展到极大规模的尝试也伴随着极高的计算成本。考虑

到这些高昂的成本，混合专家（Mixture-of-Experts, MoE）架构（Jacobs 等，1991；Jordan 与 Jacobs，1994；Shazeer 等，2017）已成为一种流行的解决方案。

【图表】图 1 DeepSeekMoE 16B 与开源模型在 Open LLM Leaderboard 上的对比。红色虚线由除 DeepSeekMoE 16B 之外所有模型的数据点线性拟合得到。DeepSeekMoE 16B 在激活参数量相近的模型中持续大幅领先，并达到与 LLaMA2 7B 相当的性能，后者的激活参数量约为 DeepSeekMoE 16B 的 2.5 倍（原文第 2 页）

它能够实现参数扩展，同时将计算成本保持在适度水平。近期，MoE 架构在 Transformer（Vaswani 等，2017）中的应用已催生出将语言模型扩展到可观规模的成功尝试（Du 等，2022；Fedus 等，2021；Lepikhin 等，2021；Zoph，2022），并伴随着令人瞩目的性能。这些成果凸显了 MoE 语言模型的巨大潜力与前景。

尽管 MoE 架构潜力巨大，但现有 MoE 架构可能受困于知识混杂（knowledge hybridity）与知识冗余（knowledge redundancy）问题，这些问题限制了专家专业化，即每个专家获取互不重叠且聚焦的知识。传统 MoE 架构用 MoE 层替换 Transformer 中的前馈网络（Feed-Forward Network, FFN）。每个 MoE 层由多个专家组成，每个专家在结构上与标准 FFN 相同，每个 token 被分配给一个（Fedus 等，2021）或两个（Lepikhin 等，2021）专家。这种架构暴露出两个潜在问题：（1）知识混杂：现有 MoE 实践通常采用数量有限的专家（例如 8 个或 16 个），因此被分配给某个特定专家的 token 很可能覆盖多样化的知识。其结果是，被指定的专家会倾向于在其参数中汇集类型差异巨大的知识，而这些知识难以被同时利用。（2）知识冗余：被分配给不同专家的 token 可能需要共同的知识。因此，多个专家可能会在各自参数中趋同地学习共享知识，从而造成专家参数冗余。这些问题共同阻碍了现有 MoE 实践中的专家专业化，使其无法达到 MoE 模型理论上的性能上界。

针对上述问题，我们提出了 DeepSeekMoE，这是一种专为实现极致专家专业化而设计的创新 MoE 架构。我们的架构包含两项主要策略：（1）细粒度专家分割：在保持参数数量不变的同时，我们通过拆分 FFN 的中间隐藏维度，将专家分割为更细的粒度。相应地，在保持计算成本不变的情况下，我们还激活更多细粒度专家，以实现更灵活、更具适应性的激活专家组合。细粒度专家分割使多样化知识能够被更精细地分解，并更精确地学习到不同专家之中，每个专家都将保持更高程度的专业化。此外，激活专家组合灵活性的提升也有助于更准确、更有针对性的知识获取。（2）共享专家隔离：我们隔离出某些专家作为始终激活的共享专家，旨在捕捉并整合跨不同上下文的共同知识。通过将这些共同知识压缩到共享专家中，其他路由专家之间的冗余将得到缓解。这可以提升参数效率，并确保每

个路由专家通过聚焦于各自不同的方面而保持专业化。DeepSeekMoE 的这些架构创新为训练一个参数高效、且每个专家高度专业化的 MoE 语言模型提供了机会。

从 2B 参数的适度规模起步，我们验证了 DeepSeekMoE 架构的优势。我们在覆盖多种任务的 12 个零样本或少样本基准上开展评测。实证结果表明，DeepSeekMoE 2B 大幅超越 GShard 2B（Lepikhin 等，2021），甚至与 GShard 2.9B 相当，后者是一个专家参数量与计算量均为 1.5 倍的更大 MoE 模型。值得注意的是，我们发现 DeepSeekMoE 2B 几乎接近参数量相同的稠密对照模型的性能，而后者构成了 MoE 语言模型的严格性能上界。为了获得更深入的洞见，我们对 DeepSeekMoE 的专家专业化开展了细致的消融研究与分析。这些研究验证了细粒度专家分割与共享专家隔离的有效性，并提供了实证证据，支持 DeepSeekMoE 能够实现高水平专家专业化的论断。

借助我们的架构，我们随后将模型参数扩展到 16B，并在包含 2T token 的大规模语料上训练 DeepSeekMoE 16B。评测结果表明，DeepSeekMoE 16B 仅用约 40% 的计算量，就达到了与 DeepSeek 7B（DeepSeek-AI，2024）相当的性能，后者是在同一 2T 语料上训练的稠密模型。我们还将 DeepSeekMoE 与开源模型进行了比较，评测表明 DeepSeekMoE 16B 在激活参数量相近的模型中持续大幅领先，并达到与 LLaMA2 7B（Touvron 等，2023b）相当的性能，后者的激活参数量约为 DeepSeekMoE 16B 的 2.5 倍。图 1 展示了在 Open LLM Leaderboard 上的评测结果（https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard）。此外，我们开展监督微调（Supervised Fine-Tuning, SFT）进行对齐，将模型转化为对话模型。评测结果表明，在对话场景下，DeepSeekMoE Chat 16B 同样达到了与 DeepSeek Chat 7B 及 LLaMA2 SFT 7B 相当的性能。受这些结果鼓舞，我们进一步开展了将 DeepSeekMoE 扩展到 145B 的初步尝试。实验结果依然一致地验证了其相较 GShard 架构的显著优势。此外，它展现出与 DeepSeek 67B 相当的性能，而仅使用了 28.5%（甚至可能低至 18.2%）的计算量。

我们的贡献总结如下：

- **架构创新。**我们提出了 DeepSeekMoE，这是一种旨在实现极致专家专业化的创新 MoE 架构，采用细粒度专家分割与共享专家隔离两项主要策略。
- **实证验证。**我们开展了大量实验，以实证方式验证 DeepSeekMoE 架构的有效性。实验结果验证了 DeepSeekMoE 2B 中高水平的专家专业化，并表明 DeepSeekMoE 2B 几乎能够逼近 MoE 模型的性能上界。
- **可扩展性。**我们将 DeepSeekMoE 扩展到训练一个 16B 模型，并表明 DeepSeekMoE 16B 仅用约 40% 的计算量，就达到了与 DeepSeek 7B 及 LLaMA2 7B 相当的性能。我们还开展了将

DeepSeekMoE 扩展到 145B 的初步尝试，凸显了其相对 GShard 架构的一致优势，并展现出与 DeepSeek 67B 相当的性能。

- **MoE 的对齐。**我们成功地对 DeepSeekMoE 16B 进行了监督微调，构建出一个对齐的对话模型，展示了 DeepSeekMoE 16B 的适应性与通用性。
- **公开发布。**本着开放研究的精神，我们向公众发布了 DeepSeekMoE 16B 的模型检查点（checkpoint）。值得注意的是，该模型无需量化即可部署在单张 40GB 显存的 GPU 上。

2 预备知识：Transformer 的混合专家模型

我们首先介绍一种在 Transformer 语言模型中常用的通用 MoE 架构。标准的 Transformer 语言模型由 L 层标准 Transformer 块堆叠而成，其中每个块可以表示如下：

$$\mathbf{u}_{1:T}^l = \text{Self-Att}(\mathbf{h}_{1:T}^{l-1}) + \mathbf{h}_{1:T}^{l-1} \quad (1)$$

$$\mathbf{h}_t^l = \text{FFN}(\mathbf{u}_t^l) + \mathbf{u}_t^l \quad (2)$$

其中 T 表示序列长度，Self-Att(·) 表示自注意力模块，FFN(·) 表示前馈网络（FFN），$\mathbf{u}_{1:T}^l \in \mathbb{R}^{T\times d}$ 是第 l 个注意力模块之后所有 token 的隐藏状态，$\mathbf{h}_t^l \in \mathbb{R}^d$ 是第 t 个 token 经第 l 个 Transformer 块处理后的输出隐藏状态。为简洁起见，我们在上述公式中省略了层归一化。

构建 MoE 语言模型的一种典型做法，通常是按指定间隔用 MoE 层替换 Transformer 中的 FFN（Du 等，2022；Fedus 等，2021；Lepikhin 等，2021；Zoph，2022）。一个 MoE 层由多个专家组成，其中每个专家在结构上与标准 FFN 相同。随后，每个 token 会被分配给一个（Fedus 等，2021）或两个（Lepikhin 等，2021）专家。若第 l 个 FFN 被替换为 MoE 层，其输出隐藏状态 $\mathbf{h}_t^l$ 的计算可表示为：

$$\mathbf{h}_t^l = \sum_{i=1}^{N} (g_{i,t}\cdot \text{FFN}_i(\mathbf{u}_t^l)) + \mathbf{u}_t^l \quad (3)$$

$$g_{i,t} = \begin{cases} s_{i,t}, & \text{若 } s_{i,t} \in \text{Topk}(\{s_{j,t} \mid 1 \leqslant j \leqslant N\}, K); \\ 0, & \text{否则} \end{cases} \quad (4)$$

$$s_{i,t} = \text{Softmax}_i(\mathbf{u}_t^{lT} \mathbf{e}_i^l) \quad (5)$$

其中 N 表示专家总数，$\text{FFN}_i(\cdot)$ 为第 i 个专家 FFN，$g_{i,t}$ 表示第 i 个专家的门控值，$s_{i,t}$ 表示 token 到专家的亲和度，Topk(·, K) 表示在针对第 t 个 token 与全部 N 个专家计算出的亲和度分数中取最高的 K 个所构成的集合，$\mathbf{e}_i^l$ 是第 l 层中第 i 个专家的质心。注意 $g_{i,t}$ 是稀疏的，表明 N 个门控值中只有 K 个非零。这种稀疏特性保证了 MoE 层内的计算效率，即每个 token 只会被分配给 K 个专家并在其中计算。此外，在上述公式中，为简洁起见我们省略了层归一化操作。

【图表】图 2 DeepSeekMoE 示意图。子图 (a) 展示了采用传统 top-2 路由策略的 MoE 层。子图 (b) 展示了细粒度专家分割策略。随后，子图 (c) 展示了共享专家隔离策略的整合，构成完整的 DeepSeekMoE 架构。值得注意的是，在这三种架构中，专家参数量与计算成本均保持不变（原文第 5 页）

3 DeepSeekMoE 架构

在第 2 节概述的通用 MoE 架构之上，我们提出了 DeepSeekMoE，它专为挖掘专家专业化的潜力而设计。如图 2 所示，我们的架构融合了两项主要策略：细粒度专家分割与共享专家隔离。这两项策略都旨在提升专家专业化的水平。

3.1 细粒度专家分割

在专家数量有限的场景下，被分配给某个特定专家的 token 更可能覆盖多样类型的知识。因此，被指定的专家会倾向于在其参数中学习类型差异巨大的知识，而这些知识难以被同时利用。然而，如果每个 token 可以被路由到更多专家，多样化知识就有潜力被分解并分别在不同专家中学习。在这种情况下，每个专家仍然可以保持高水平的专家专业化，从而有助于在专家之间形成更聚焦的知识分布。

为实现这一目标，我们在保持专家参数量与计算成本一致的情况下，以更细的粒度对专家进行分割。更细的专家分割使激活专家的组合更灵活、更具适应性。具体而言，在图 2(a) 所示的典型 MoE 架构之上，我们通过将 FFN 中间隐藏维度缩减为原来的 1/m，把每个专家 FFN 分割为 m 个更小的专家。由于每个专家都变小了，相应地，我们也把激活专家数量增加到 m 倍，以保持相同的计算成本，如图 2(b) 所示。借助细粒度专家分割，MoE 层的输出可以表示为：

$$h_t^l = \sum_{i=1}^{mN} \left(g_{i,t} \cdot \mathrm{FFN}_i(u_t^l)\right) + u_t^l \qquad (6)$$

$$g_{i,t} = \begin{cases} s_{i,t}, & \text{若 } s_{i,t} \in \mathrm{Topk}(\{s_{j,t} \mid 1 \leqslant j \leqslant mN\}, mK) \\ 0, & \text{否则} \end{cases} \qquad (7)$$

$$s_{i,t} = \mathrm{Softmax}_i\left(u_t^{lT} e_i^l\right) \qquad (8)$$

其中专家参数总量等于 N 倍标准 FFN 的参数量，mN 表示细粒度专家的总数。采用细粒度专家分割策略后，非零门控的数量也将增加到 mK。

从组合的角度来看，细粒度专家分割策略大幅提升了激活专家的组合灵活性。作为一个说明性示例，我们考虑 N = 16 的情形。典型的 top-2 路由策略可以产生 C(16, 2) = 120 种可能的组合。相比之

下，若将每个专家分割为 4 个更小的专家，细粒度路由策略则可以产生 C(64, 8) = 4,426,165,368 种可能的组合。组合灵活性的激增提升了实现更准确、更有针对性知识获取的潜力。

3.2 共享专家隔离

在传统路由策略下，被分配给不同专家的 token 可能需要某些共同的知识或信息。因此，多个专家可能会在各自参数中趋同地学习共享知识，从而造成专家参数冗余。然而，如果有专门的共享专家来捕捉并整合跨不同上下文的共同知识，其他路由专家之间的参数冗余就会得到缓解。这种冗余的缓解将有助于构建参数效率更高、专家更专业化的模型。

为实现这一目标，在细粒度专家分割策略之外，我们进一步隔离出 K_s 个专家作为共享专家。无论路由模块如何，每个 token 都会被确定性地分配给这些共享专家。为保持计算成本不变，其他路由专家中被激活的专家数量将减少 K_s 个，如图 2(c) 所示。整合共享专家隔离策略后，完整 DeepSeekMoE 架构中的 MoE 层可形式化如下：

$$h_t^l = \sum_{i=1}^{K_s} \mathrm{FFN}_i(u_t^l) + \sum_{i=K_s+1}^{mN} (g_{i,t} \cdot \mathrm{FFN}_i(u_t^l)) + u_t^l \quad (9)$$

$$g_{i,t} = \begin{cases} s_{i,t}, & \text{若 } s_{i,t} \in \mathrm{Topk}(\{s_{j,t} \mid K_s+1 \leqslant j \leqslant mN\}, mK-K_s) \\ 0, & \text{否则} \end{cases} \quad (10)$$

$$s_{i,t} = \mathrm{Softmax}_i(u_t^{lT} e_i^l) \quad (11)$$

最后，在 DeepSeekMoE 中，共享专家的数量为 K_s，路由专家的总数为 mN−K_s，非零门控的数量为 mK−K_s。

值得注意的是，共享专家隔离的雏形可归功于 Rajbhandari 等（2022）。关键区别在于，他们从工程视角推导出这一策略，而我们则从算法视角提出该策略。

3.3 负载均衡考量

自动学习到的路由策略可能遇到负载不均衡问题，该问题表现为两个显著缺陷。首先，存在路由崩溃（routing collapse）（Shazeer 等，2017）的风险，即模型总是只选择少数几个专家，导致其他专家无法得到充分训练。其次，如果专家分布在多个设备上，负载不均衡会加剧计算瓶颈。

专家级均衡损失。为降低路由崩溃的风险，我们还采用一种专家级均衡损失。均衡损失的计算如下：

$$L_{\mathrm{ExpBal}} = \alpha_1 \cdot \sum_{i=1}^{N'} f_i \cdot P_i \quad (12)$$

$$f_i = \frac{N'}{K'T} \cdot \sum_{t=1}^{T} \mathbb{1}(\text{token_t 选择了专家 } i) \quad (13)$$

$$P_i = (1/T) \cdot \Sigma_{t=1}^{T} s_{i,t} \quad (14)$$

其中 α_1 是称为专家级均衡因子的超参数；为简洁起见，N' 等于 $(mN-K_s)$，K' 等于 $(mK-K_s)$。$1(\cdot)$ 表示指示函数。

设备级均衡损失。除专家级均衡损失外，我们还引入一种设备级均衡损失。当目标是缓解计算瓶颈时，没有必要在专家层面施加严格的均衡约束，因为对负载均衡的过度约束会损害模型性能。相反，我们的首要目标是确保各设备之间的计算均衡。若将所有路由专家划分为 D 组 $\{E_1, E_2, ..., E_D\}$，并将每组部署在单个设备上，则设备级均衡损失计算如下：

$$L_{DevBal} = \alpha_2 \cdot \Sigma_{i=1}^{D} f'_i \cdot P'_i \quad (15)$$

$$f'_i = (1/|E_i|) \cdot \Sigma_{j \in E_i} f_j \quad (16)$$

$$P'_i = \Sigma_{j \in E_i} P_j \quad (17)$$

其中 α_2 是称为设备级均衡因子的超参数。在实践中，我们设置较小的专家级均衡因子以降低路由崩溃的风险，同时设置较大的设备级均衡因子以促进各设备之间的计算均衡。

4 验证实验

4.1 实验设置

4.1.1 训练数据与分词

我们的训练数据采样自 DeepSeek-AI 构建的大规模多语言语料。该语料以英文和中文为主，同时也涵盖其他语言。它来自多种来源，包括网页文本、数学材料、代码脚本、已发表文献以及其他各类文本材料。为开展验证实验，我们从该语料中采样一个包含 100B token 的子集来训练模型。在分词方面，我们使用 HuggingFace Tokenizer 工具（https://github.com/huggingface/tokenizers）在训练语料的较小子集上训练字节对编码（Byte Pair Encoding, BPE）（Sennrich 等，2016）分词器。在验证实验中，我们准备了一个词表大小为 8K 的分词器，而在训练更大模型时，词表大小将会扩大。

4.1.2 基础设施

我们基于 HAI-LLM（High-Flyer，2023）开展实验，这是一个高效且轻量的训练框架，集成了多种并行策略，包括张量并行（Korthikanti 等，2023；Narayanan 等，2021；Shoeybi 等，2019）、ZeRO 数据并行（Rajbhandari 等，2020）、PipeDream 流水线并行（Harlap 等，2018），更具体地说，还包

括通过结合数据并行与张量并行实现的专家并行（Lepikhin 等，2021）。为优化性能，我们使用 CUDA 与 Triton（Tillet 等，2019）为门控算法开发 GPU 内核，并融合不同专家中多个线性层的计算。所有实验均在配备 NVIDIA A100 或 H800 GPU 的集群上进行。A100 集群中的每个节点包含 8 块 GPU，两两通过 NVLink 桥连接。H800 集群同样每节点配置 8 块 GPU，节点内通过 NVLink 与 NVSwitch 互连。对于 A100 与 H800 集群，均使用 InfiniBand 互连以支持跨节点通信。

4.1.3 超参数

模型设置。在验证实验中，我们将 Transformer 层数设为 9，隐藏维度设为 1280。我们采用多头注意力机制，共 10 个注意力头，每个头的维度为 128。在初始化方面，所有可学习参数均以 0.006 的标准差随机初始化。我们将所有 FFN 替换为 MoE 层，并确保专家参数总量等于标准 FFN 的 16 倍。此外，我们保持激活的专家参数（包括共享专家参数与激活的路由专家参数）为标准 FFN 的 2 倍。在此配置下，每个 MoE 模型的总参数量约为 2B，激活参数量约为 0.3B。

训练设置。我们采用 AdamW 优化器（Loshchilov 与 Hutter，2019），超参数设置为 $\beta_1 = 0.9$、$\beta_2 = 0.95$、weight_decay = 0.1。学习率采用预热与分步衰减（warmup-and-step-decay）策略进行调度。初始阶段，学习率在前 2K 步内从 0 线性增长到最大值。随后，在训练进度的 80% 处学习率乘以 0.316，并在 90% 处再次乘以 0.316。验证实验的最大学习率设为 1.08×10^{-3}，梯度裁剪范数设为 1.0。批大小设为 2K，在最大序列长度为 2K 的情况下，每个训练批次包含 4M token。相应地，训练总步数设为 25,000，以达到 100B 训练 token。由于训练数据充足，我们在训练中不使用 dropout。鉴于模型规模相对较小，包括专家参数在内的所有参数都部署在单个 GPU 设备上，以避免计算不均衡。相应地，我们在训练中不丢弃任何 token，也不采用设备级均衡损失。为防止路由崩溃，我们将专家级均衡因子设为 0.01。

为便于阅读，我们还在附录 A 中提供了一张 DeepSeekMoE 在不同规模下的超参数概览表。

4.1.4 评测基准

我们在覆盖多种任务类型的广泛基准上开展评测。基准列表如下。

语言建模。对于语言建模，我们在 Pile（Gao 等，2020）的测试集上评测模型，评测指标为交叉熵损失。

语言理解与推理。对于语言理解与推理，我们考虑 HellaSwag（Zellers 等，2019）、PIQA（Bisk 等，2020）、ARC-challenge 与 ARC-easy（Clark 等，2018）。这些任务的评测指标为准确率。

阅读理解。对于阅读理解，我们使用 RACE-high 与 RACE-middle（Lai 等，2017），评测指标为准确率。

代码生成。对于代码生成，我们在 HumanEval（Chen 等，2021）与 MBPP（Austin 等，2021）上评测模型。评测指标为 Pass@1，即仅一次生成尝试的通过率。

闭卷问答。对于闭卷问答，我们考虑 TriviaQA（Joshi 等，2017）与 NaturalQuestions（Kwiatkowski 等，2019）。评测指标为精确匹配（Exact Matching, EM）率。

4.2 评测

基线。包括 DeepSeekMoE 在内，我们比较了五个模型用于验证实验。Dense 表示一个总参数量为 0.2B 的标准稠密 Transformer 语言模型。Hash Layer（Roller 等，2021）是一种基于 top-1 哈希路由的 MoE 架构，总参数量为 2.0B、激活参数量为 0.2B，与稠密基线对齐。Switch Transformer（Fedus 等，2021）是另一种著名的基于 top-1 可学习路由的 MoE 架构，其总参数量与激活参数量均与 Hash Layer 相同。GShard（Lepikhin 等，2021）采用 top-2 可学习路由策略，总参数量为 2.0B、激活参数量为 0.3B，因为与 top-1 路由方法相比多激活了一个专家。DeepSeekMoE 有 1 个共享专家与 63 个路由专家，其中每个专家的大小为标准 FFN 的 0.25 倍。包括 DeepSeekMoE 在内，所有对比模型共享相同的训练语料与训练超参数。所有对比的 MoE 模型具有相同的总参数量，且 GShard 与 DeepSeekMoE 具有相同的激活参数量。

结果。我们在表 1 中给出评测结果。对于所有展示的模型，我们报告在 100B token 上训练后的最终评测结果。从表中，我们可以得出以下观察：（1）在稀疏架构且总参数量更多的情况下，Hash Layer……

【表格】表 1 验证实验的评估结果。粗体表示最佳结果。与其他 MoE 架构相比，DeepSeekMoE 展现出显著的性能优势（原文第 10 页）

……以及 Switch Transformer 均以相同的激活参数量取得了显著优于稠密基线的性能。（2）与 Hash Layer 和 Switch Transformer 相比，GShard 的激活参数更多，性能略优于 Switch Transformer。（3）在总参数量与激活参数量均相同的情况下，DeepSeekMoE 相较 GShard 展现出压倒性的优势。这些结果彰显了我们的 DeepSeekMoE 架构在现有 MoE 架构格局中的优越性。

4.3 DeepSeekMoE 高度逼近 MoE 模型的上界

我们已经证明，DeepSeekMoE 优于稠密基线与其他 MoE 架构。为了更精确地理解 DeepSeekMoE 的性能，我们将其与拥有更多总参数或激活参数的更大基线模型进行比较。这些比较使我们能够估计 GShard 或稠密基线要达到与 DeepSeekMoE 相当的性能所需的模型规模。

与 GShard×1.5 的比较：表 2 展示了 DeepSeekMoE 与一个专家规模为其 1.5 倍的更大 GShard 模型之间的比较，这使专家参数与专家计算量均为原来的 1.5 倍。总体而言，我们观察到 DeepSeekMoE 取得了与 GShard×1.5 相当的性能，这凸显了 DeepSeekMoE 架构固有的显著优势。除了与 GShard×1.5 的比较之外，我们还在附录 B 中给出了与 GShard×1.2 的比较。

此外，我们将 DeepSeekMoE 的总参数量增加到 13.3B，并将其与总参数量分别为 15.9B 与 19.8B 的 GShard×1.2 和 GShard×1.5 进行比较。我们发现，在更大规模下，DeepSeekMoE 甚至能明显优于 GShard×1.5。这些结果同样在附录 B 中给出。

【表格】表 2 DeepSeekMoE、更大的 GShard 模型与更大的稠密模型之间的比较。在“# Experts”（专家数）一行中，$a+b$ 表示 a 个共享专家与 b 个路由专家；在“# Activated Experts”（激活专家数）一行中，$a+b$ 表示 a 个被激活的共享专家与 b 个被激活的路由专家。DeepSeekMoE 取得了与专家参数及计算量均为其 1.5 倍的 GShard 模型相当的性能。此外，DeepSeekMoE 接近 FFN 参数量为其 16 倍的稠密模型的性能，后者在模型容量层面构成了 MoE 模型的上界（原文第 11 页）

与 Dense×16 的比较：表 2 还展示了 DeepSeekMoE 与更大的稠密模型之间的比较。为了进行公平比较，我们没有采用注意力参数与 FFN 参数之间广泛使用的比例（1:2）。取而代之的是，我们配置了 16 个共享专家，其中每个专家的参数量与标准 FFN 相同。这种架构模拟了一个 FFN 参数量为标准 FFN 16 倍的稠密模型。从表中我们可以发现，DeepSeekMoE 接近 Dense×16 的性能，而后者在模型容量层面构成了 MoE 模型的严格上界。这些结果表明，至少在约 2B 参数与 100B 训练 token 的规模上，DeepSeekMoE 的性能与 MoE 模型的理论上界高度吻合。此外，我们还在附录 B 中提供了与 Dense×4 的额外比较。

4.4 消融研究

为了验证细粒度专家分割与共享专家隔离策略的有效性，我们对 DeepSeekMoE 进行了消融研究，并在图 3 中给出了结果。为了保证比较公平，我们确保参与比较的所有模型具有相同的总参数量与激活参数量。

【图表】图 3 DeepSeekMoE 的消融研究。为便于展示，性能按最佳性能进行了归一化。所有参与比较的模型具有相同的参数量与激活参数量。我们可以看到，细粒度专家分割与共享专家隔离都有助于提升模型的整体性能（原文第 12 页）

共享专家隔离：为了评估共享专家隔离策略的影响，我们在 GShard 的基础上隔离出一个专家作为共享专家。从图 3 中我们观察到，与 GShard 相比，有意隔离出一个共享专家在大多数基准上都带来了性能提升。这些结果支持了共享专家隔离策略有助于增强模型性能这一论断。

细粒度专家分割：为了评估细粒度专家分割策略的有效性，我们通过将专家进一步分割为更细的粒度来进行更细致的比较。具体而言，我们将每个专家分割为 2 个或 4 个更小的专家，从而总共得到 32 个（1 个共享 + 31 个路由）或 64 个（1 个共享 + 63 个路由）专家。图 3 揭示了一个一致的趋势：专家分割粒度的持续细化对应着模型整体性能的持续提升。这些发现为细粒度专家分割策略的有效性提供了实证依据。

共享专家与路由专家的比例：此外，我们还探究了共享专家与路由专家的最佳比例。在总专家数为 64 的最细粒度设置下，并保持总专家数与激活专家数不变，我们尝试分别隔离出 1 个、2 个和 4 个专家作为共享专家。我们发现，共享专家与路由专家的不同比例并不会显著影响性能，1 个、2 个和 4 个共享专家分别取得了 1.808、1.806 与 1.811 的 Pile 损失。考虑到 1:3 的比例取得了略优的 Pile 损失，在扩大 DeepSeekMoE 规模时，我们将共享专家与激活路由专家之间的比例保持为 1:3。

4.5 专家专业化分析

本节中，我们对 DeepSeekMoE 2B 的专家专业化进行实证分析。本节的 DeepSeekMoE 2B 指表 1 中报告的模型，即总参数量为 2.0B，并且激活 1 个共享专家与 63 个路由专家中的 7 个。

【图表】图 4 在禁用不同比例的最高路由概率专家时的 Pile 损失。值得注意的是，DeepSeekMoE 对禁用最高路由概率专家的比例更为敏感，这表明 DeepSeekMoE 中各路由专家之间的冗余更低（原文第 13 页）

DeepSeekMoE 的路由专家之间冗余更低：为了评估路由专家之间的冗余，我们禁用不同比例的最高路由概率专家，并评估 Pile 损失。具体而言，对于每个 token，我们屏蔽一定比例的路由概率最高的专家，然后从剩余的路由专家中选择 top-K 个专家。为了保证公平，我们将 DeepSeekMoE 与 GShard×1.5 进行比较，因为在未禁用任何专家时二者的 Pile 损失相同。如图 4 所示，与 GShard×1.5 相比，DeepSeekMoE 对禁用最高路由概率专家更为敏感。这种敏感性表明 DeepSeekMoE 中的参数冗

余程度更低，因为每个路由专家都更加不可替代。相比之下，GShard×1.5 的专家参数之间存在更大的冗余，因此它能够在最高路由概率专家被禁用时缓冲性能下降。

共享专家无法被路由专家替代：为了探究共享专家在 DeepSeekMoE 中的作用，我们禁用共享专家并额外激活一个路由专家。Pile 上的评估显示，尽管计算开销保持不变，Pile 损失仍显著上升，从 1.808 增至 2.414。这一结果凸显了共享专家的关键作用，并表明共享专家捕获了不与路由专家共享的基础性、本质性知识，因此无法被路由专家替代。

DeepSeekMoE 能更准确地获取知识：为了验证我们的主张——激活专家组合的更高灵活性有助于更准确、更有针对性地获取知识——我们研究 DeepSeekMoE 能否以更少的激活专家获取所需知识。具体而言，我们将激活的路由专家数从 3 逐步增加到 7，并评估由此得到的 Pile 损失。如图 5 所示，即使仅激活 4 个路由专家，DeepSeekMoE 也能取得与 GShard 相当的 Pile 损失。这一观察支持了 DeepSeekMoE 能够更准确、更高效地获取所需知识这一命题。

【图表】图 5 DeepSeekMoE 中激活不同数量的路由专家时的 Pile 损失。仅激活 4 个路由专家时，DeepSeekMoE 即可取得与 GShard 相当的 Pile 损失（原文第 14 页）

【图表】图 6 GShard 与激活专家数减半（从零开始训练）的 DeepSeekMoE 之间的比较。在总专家参数相同、激活专家参数仅为其一半的情况下，DeepSeekMoE 仍优于 GShard（原文第 14 页）

受这些发现的鼓舞，为了更严格地验证 DeepSeekMoE 的专家专业化与准确的知识获取能力，我们从零开始训练了一个新模型。该模型包含 1 个共享专家与 63 个路由专家，其中仅激活 3 个路由专家。图 6 所示的评估结果表明，即使在总专家参数相同、激活专家参数仅为其一半的情况下，DeepSeekMoE 仍优于 GShard。这凸显了 DeepSeekMoE 更高效地利用专家参数的能力，即激活专家中有效参数的比例远高于 GShard。

5 扩展至 DeepSeekMoE 16B

借助 DeepSeekMoE 架构，我们将 MoE 模型扩展到总参数量为 16B 的更大规模，并在 2T token 上训练。结果表明，与 LLaMA2 7B 相比，DeepSeekMoE 16B 仅用约 40% 的计算量就取得了更优的性能。

5.1 实验设置

5.1.1 训练数据与分词

我们从与第 4.1.1 节所述相同的语料中采样训练数据。与验证实验不同，我们采样了更大量的数据，共 2T token，与 LLaMA2 7B 的训练 token 数保持一致。我们同样使用 HuggingFace Tokenizer 工具训练 BPE 分词器，但针对 DeepSeekMoE 16B 将词表大小设为 100K。

5.1.2 超参数

模型设置：对于 DeepSeekMoE 16B，我们将 Transformer 层数设为 28，隐藏维度设为 2048。我们采用多头注意力机制，共 16 个注意力头，每个头的维度为 128。在初始化方面，所有可学习参数均以 0.006 的标准差随机初始化。我们将除第一层之外的所有 FFN 替换为 MoE 层，因为我们观察到第一层的负载均衡状态收敛得尤其慢。每个 MoE 层由 2 个共享专家与 64 个路由专家组成，其中每个专家的大小为标准 FFN 的 0.25 倍。每个 token 将被路由到这 2 个共享专家以及 64 个路由专家中的 6 个。我们没有采用更细的专家分割粒度，因为专家规模过小可能带来计算效率的下降。在超过 16B 的更大规模下，仍然可以采用更细的粒度。在我们的配置下，DeepSeekMoE 16B 的总参数量约为 16.4B，激活参数量约为 2.8B。

训练设置：我们采用 AdamW 优化器（Loshchilov 与 Hutter，2019），超参数设为 $\beta1 = 0.9$、$\beta2 = 0.95$、weight_decay = 0.1。学习率同样采用预热与阶梯衰减（warmup-and-step-decay）策略进行调度。最初，学习率在前 2K 步内从 0 线性增加到最大值。随后，在训练步数达到 80% 时，学习率乘以 0.316；在达到 90% 时，再乘以 0.316。DeepSeekMoE 16B 的最大学习率设为 4.2×10^−4，梯度裁剪范数设为 1.0。批大小设为 4.5K，在最大序列长度为 4K 的情况下，每个训练批次包含 18M token。相应地，训练总步数设为 106,449，以实现 2T 训练 token。由于训练数据充足，我们在训练中不使用 dropout。我们利用流水线并行将模型的不同层部署在不同设备上，并且对于每一层，所有专家都部署在同一设备上。因此，我们在训练中也不会丢弃任何 token，并且不采用设备级平衡损失。为了防止路由崩溃，我们设置了 0.001 这一相当小的专家级平衡因子，因为我们发现，在我们的并行化策略下，更高的专家级平衡因子无法提升计算效率，反而会损害模型性能。

5.1.3 评估基准

除了验证实验中使用的基准之外，我们还加入了更多基准，以实现更全面的评估。下文将说明这些基准与验证实验所用基准的不同之处。

语言建模：对于语言建模，我们同样在 Pile（Gao 等，2020）的测试集上评估模型。由于 DeepSeekMoE 16B 使用的分词器与 LLaMA2 7B 所使用的分词器不同，为了公平比较，我们采用每字节比特数（bits per byte, BPB）作为评估指标。

阅读理解：对于阅读理解，我们额外考虑了 DROP（Dua 等，2019），评估指标为精确匹配（Exactly Matching, EM）率。

数学推理：对于数学推理，我们额外纳入了 GSM8K（Cobbe 等，2021）与 MATH（Hendrycks 等，2021），并使用 EM 作为评估指标。

多学科多项选择：对于多学科多项选择，我们额外在 MMLU（Hendrycks 等，2020）上评估模型，评估指标为准确率。

消歧：对于消歧，我们额外考虑了 WinoGrande（Sakaguchi 等，2019），评估指标为准确率。

中文基准：由于 DeepSeekMoE 16B 在中英双语语料上预训练，我们还在四个中文基准上对其进行了评估。CLUEWSC（Xu 等，2020）是一个中文消歧基准；CEval（Huang 等，2023）与 CMMLU（Li 等，2023）是两个形式与 MMLU 相似的中文多学科多项选择基准；CHID（Zheng 等，2019）是一个中文成语补全基准，旨在评估对中华文化的理解。上述中文基准的评估指标为准确率或 EM。

Open LLM Leaderboard：我们基于内部评估框架评估上述所有基准。为了公平且方便地将 DeepSeekMoE 16B 与开源模型进行比较，我们还在 Open LLM Leaderboard 上评估了 DeepSeekMoE 16B。Open LLM Leaderboard 是由 HuggingFace 支持的公开排行榜，包含六个任务：ARC（Clark 等，2018）、HellaSwag（Zellers 等，2019）、MMLU（Hendrycks 等，2020）、TruthfulQA（Lin 等，2022）、Winogrande（Sakaguchi 等，2019）与 GSM8K（Cobbe 等，2021）。

5.2 评估

5.2.1 与 DeepSeek 7B 的内部比较

我们首先在 DeepSeekMoE 16B 与 DeepSeek 7B（DeepSeek-AI，2024）之间进行内部比较，后者是一个拥有 6.9B 参数的稠密语言模型。为了保证公平，两个模型都在相同的语料上使用 2T token 进行训练。这使我们能够准确评估 MoE 架构的有效性，而不受训练数据的影响。

【表格】表 3 DeepSeek 7B 与 DeepSeekMoE 16B 的对比。粗体表示最佳或接近最佳的结果。在仅使用 40.5% 计算量的情况下，DeepSeekMoE 16B 取得了与 DeepSeek 7B 相当的性能。（原文第 17 页）

评估结果如表 3 所示，可得出以下观察：（1）总体而言，DeepSeekMoE 16B 仅使用约 40% 的计算量，就取得了与 DeepSeek 7B 相当的性能。（2）DeepSeekMoE 16B 在语言建模与知识密集型任务上展现出显著优势，例如 Pile、HellaSwag、TriviaQA 与 NaturalQuestions。考虑到在 MoE 模型中，FFN 参数比注意力参数多得多，这些结果与下述论断相吻合：Transformer 中的 FFN 具备知识记忆能力（Dai et al., 2022a）。（3）与其他任务上的优异表现相比，DeepSeekMoE 在处理选择题任务时表现出局限。这一不足源于 DeepSeekMoE 16B 中有限的注意力参数（DeepSeekMoE 16B 仅有约 0.5B 注意力参数，而 DeepSeek 7B 有 2.5B 注意力参数）。我们此前对 DeepSeek 7B 的研究表明，注意力容量与选择题任务的表现之间存在正相关。例如，采用多查询注意力机制（Shazeer, 2019）的 DeepSeek 7B MQA 在类 MMLU 任务上也表现不佳。此外，为了更全面地理解 DeepSeekMoE 16B 的训练过程，我们还在附录 C 中提供了 DeepSeekMoE 16B 与 DeepSeek 7B（稠密）训练过程中的基准曲线，供参考。

尤为重要的是，由于 DeepSeekMoE 16B 的参数规模不大，它可以在单块 40GB 显存的 GPU 上部署。配合适当的算子优化，其推理速度可达到 7B 稠密模型的近 2.5 倍。

5.2.2 与开源模型的比较

与 LLaMA2 7B 的内部比较：在开源模型领域，我们主要将 DeepSeekMoE 16B 与 LLaMA2 7B（Touvron et al., 2023b）进行比较，后者是一个广为人知的强开源语言模型，拥有 6.7B 参数。DeepSeekMoE 16B 与 LLaMA2 7B 均在 2T token 上预训练。DeepSeekMoE 的总参数量是 LLaMA2 7B 的 245%，但计算量仅为 LLaMA2 7B 的 39.6%。我们在内部基准上的结果如表 4 所示，可得出以下观察：（1）在所评估的基准中，DeepSeekMoE 16B 仅使用约 40% 的计算量，就在大多数基准上优于 LLaMA2 7B。（2）DeepSeekMoE 16B 的数学推理与代码生成能力强于 LLaMA2 7B，这得益于我们的预训练语料中数学与代码相关文本更为丰富。（3）由于我们的预训练语料中包含中文文本，DeepSeekMoE 16B 在中文基准上相对 LLaMA2 7B 具有显著性能优势。（4）尽管训练时使用的英文文本较少，DeepSeekMoE 16B 在英文理解或知识密集型基准上仍取得与 LLaMA2 7B 相当或更好的表现，这展示了 DeepSeekMoE 16B 的卓越能力。

【表格】表 4 LLaMA2 7B 与 DeepSeekMoE 16B 的对比。在仅使用 39.6% 计算量的情况下，DeepSeekMoE 16B 在大多数基准上优于 LLaMA2 7B。（原文第 18 页）

在 Open LLM Leaderboard 上的评估：除了内部评估外，我们还在 Open LLM Leaderboard 上评估 DeepSeekMoE 16B，并将其与其他开源模型比较。除 LLaMA2 7B 外，我们纳入了更广泛的开源模

型，包括 LLaMA 7B（Touvron et al., 2023a）、Falcon 7B（Almazrouei et al., 2023）、GPT-J 6B（Wang and Komatsuzaki, 2021）、RedPajama-INCITE 7B 与 3B（Together-AI, 2023）、Open LLaMA 7B 与 3B（Geng and Liu, 2023）、OPT 2.7B（Zhang et al., 2022）、Pythia 2.8B（Biderman et al., 2023）、GPT-neo 2.7B（Black et al., 2021）与 BLOOM 3B（Scao et al., 2022）。如图 1 所示，评估结果表明，DeepSeekMoE 16B 稳定地以较大幅度优于激活参数相近的模型。此外，它取得了与 LLaMA2 7B 相当的性能，而 LLaMA2 7B 的激活参数约为其 2.5 倍。

6 DeepSeekMoE 16B 的对齐

先前的研究表明，MoE 模型通常无法从微调中获得显著收益（Artetxe et al., 2022; Fedus et al., 2021）。然而，Shen et al. (2023) 的研究结果表明，MoE 模型确实能够受益于指令微调。为了评估 DeepSeekMoE 16B 能否从微调中受益，我们进行监督微调，以基于 DeepSeekMoE 16B 构建对话模型。实验结果表明，DeepSeekMoE Chat 16B 同样取得了与 LLaMA2 SFT 7B 和 DeepSeek Chat 7B 相当的性能。

6.1 实验设置

训练数据：为训练对话模型，我们在内部整理的数据上进行监督微调（SFT），该数据包含 140 万条训练样本。该数据集涵盖广泛的类别，包括数学、代码、写作、问答、推理、摘要等。我们的 SFT 训练数据大部分为英文和中文，这使对话模型具备多用途能力，可适用于双语场景。

超参数：在监督微调过程中，我们将批量大小设为 1024 条样本，并使用 AdamW 优化器（Loshchilov and Hutter, 2019）训练 8 个 epoch。我们采用 4K 的最大序列长度，并尽可能密集地打包训练样本，直到达到序列长度上限。我们在监督微调中不使用 dropout，并简单地将学习率设为常数 10^−5，不采用任何学习率调度策略。

评估基准：对于对话模型的评估，我们采用与 5.1.3 节类似的基准，并作以下调整：（1）我们排除 Pile（Gao et al., 2020），因为对话模型很少用于纯语言建模。（2）我们排除 CHID（Zheng et al., 2019），因为观察到其结果不稳定，难以得出可靠结论。（3）我们额外加入 BBH（Suzgun et al., 2022），以更全面地评估对话模型的推理能力。

6.2 评估

基线：为了验证 DeepSeekMoE 16B 在对齐后的潜力，我们对 LLaMA2 7B、DeepSeek 7B 与 DeepSeekMoE 16B 进行监督微调，并完全使用相同的微调数据以确保公平。相应地，我们构建了三个对话模型，包括 LLaMA2 SFT 7B〔脚注：我们使用 LLaMA2 SFT 这一名称，以区别于官方的 LLaMA2 Chat（Touvron et al., 2023b）模型。〕、DeepSeek Chat 7B 与 DeepSeekMoE Chat 16B。随后，我们在广泛的下游任务上，将 DeepSeekMoE Chat 16B 与另外两个稠密对话模型（其 FLOPs 约为 DeepSeekMoE Chat 16B 的 2.5 倍）进行比较。

【表格】表 5 LLaMA2 SFT 7B、DeepSeek Chat 7B 与 DeepSeekMoE Chat 16B 的对比，这三个模型均在相同的 SFT 数据上微调。与两个 7B 稠密模型相比，DeepSeekMoE Chat 16B 在仅使用 40% 计算量的情况下，仍在大多数基准上取得相当或更优的性能。（原文第 20 页）

结果：评估结果如表 5 所示。我们的主要观察包括：（1）DeepSeekMoE Chat 16B 在消耗近 40% 计算量的情况下，在语言理解与推理（PIQA、ARC、BBH）、机器阅读理解（RACE）、数学（GSM8K、MATH）与知识密集型任务（TriviaQA、NaturalQuestions）上，取得了与 7B 稠密模型相当的性能。（2）在代码生成任务上，DeepSeekMoE Chat 16B 显著优于 LLaMA2 SFT 7B，在 HumanEval 与 MBPP 上均有明显提升。此外，它也超过了 DeepSeek Chat 7B。（3）在包括 MMLU、CEval 与 CMMLU 在内的选择题问答基准上，DeepSeekMoE Chat 16B 仍落后于 DeepSeek Chat 7B，这与基座模型上的观察一致（5.2.1 节）。然而，值得注意的是，经过监督微调后，DeepSeekMoE 16B 与 DeepSeek 7B 之间的性能差距缩小了。（4）受益于双语语料上的预训练，DeepSeekMoE Chat 16B 在所有中文基准上都显著优于 LLaMA2 SFT 7B。这些结果展示了 DeepSeekMoE 16B 在中文与英文上的均衡能力，增强了它在多种场景下的通用性与适用性。总之，对对话模型的评估凸显了 DeepSeekMoE 16B 从对齐中受益的潜力，并验证了它在仅使用约 40% 计算量的情况下与稠密模型取得相当性能的一贯优势。

7 DeepSeekMoE 145B（进行中）

受 DeepSeekMoE 16B 优异表现的鼓舞，我们进一步开展了将 DeepSeekMoE 扩展到 145B 的初步尝试。在这项初步研究中，DeepSeekMoE 145B 在 245B token 上训练，但它已展现出相对 GShard 架

构的一贯优势，并显示出达到或超越 DeepSeek 67B（稠密）性能的潜力。此外，待 DeepSeekMoE 145B 的最终版本完成充分训练后，我们还计划将其公开发布。

7.1 实验设置

训练数据与分词器：对于 DeepSeekMoE 145B，我们采用与 DeepSeekMoE 16B 完全相同的训练语料与分词器，唯一的区别在于 DeepSeekMoE 145B 在初步研究中仅训练 245B token。

模型设置：对于 DeepSeekMoE 145B，我们将 Transformer 层数设为 62，隐藏维度设为 4096。我们采用多头注意力机制，共 32 个注意力头，每个头的维度为 128。在初始化方面，所有可学习参数均以 0.006 的标准差随机初始化。与 DeepSeekMoE 16B 相同，我们也将除第一层之外的所有 FFN 替换为 MoE 层。每个 MoE 层由 4 个共享专家和 128 个路由专家组成，其中每个专家的规模是标准 FFN 的 0.125 倍。每个 token 将被路由到这 4 个共享专家以及 128 个路由专家中的 12 个。在此配置下，DeepSeekMoE 145 拥有约 144.6B 总参数量，激活参数量约为 22.2B。

训练设置：我们采用 AdamW 优化器（Loshchilov and Hutter, 2019），超参数设为 $\beta1 = 0.9$、$\beta2 = 0.95$、weight_decay = 0.1。对于 DeepSeekMoE 145B 的初步研究，我们采用预热后保持恒定（warmup-and-constant）的学习率调度器。最初，学习率在前 2K 步内从 0 线性增长到最大值。随后，学习率在剩余训练过程中保持不变。DeepSeekMoE 145B 的最大学习率设为 3.0 × 10^−4，梯度裁剪范数设为 1.0。批量大小设为 4.5K，在最大序列长度为 4K 的情况下，每个训练批次包含 18M token。我们将 DeepSeekMoE 145B 训练 13,000 步，共使用 245B 训练 token。此外，我们在训练期间不使用 dropout。我们利用流水线并行将模型的不同层部署在不同设备上，对于每一层，所有路由专家将均匀部署在 4 个设备上（即专家并行与数据并行的结合）。由于我们对 DeepSeekMoE 145B 采用专家并行，需要考虑设备级负载均衡以减少计算瓶颈。为此，我们将设备级均衡因子设为 0.05，以鼓励跨设备的均衡计算。同时，我们仍设置较小的专家级均衡因子 0.003，以防止路由崩溃。

评估基准：我们在与 DeepSeekMoE 16B 完全相同的内部基准上评估 DeepSeekMoE 145B（见 5.1.3 节）。

7.2 评估

基线：除 DeepSeekMoE 145B 外，我们还考虑三个模型进行对比。DeepSeek 67B（稠密）是一个拥有 67.4B 总参数的稠密模型（模型与训练细节参见 DeepSeek-AI (2024)）。GShard 137B 与 DeepSeekMoE 145B 具有相同的隐藏维度与层数，但遵循 GShard 架构。需要注意的是，DeepSeekMoE

145B 出于计算效率考虑，将每个专家的中间隐藏维度对齐到 64 的倍数，因此其模型规模比 GShard 137B 大 6%。DeepSeekMoE 142B（半激活）与 DeepSeekMoE 145B 架构相似，但它仅包含 2 个共享专家，并且在 128 个路由专家中仅激活 6 个。值得注意的是，包括 DeepSeekMoE 145B 在内的所有对比模型共享相同的训练语料。此外，对比中的所有 MoE 模型都从零开始训练，并共享相同的训练超参数。

【表格】表 6 DeepSeek 67B（稠密）与总参数量约 140B 量级的 MoE 模型的对比。在“# Experts”与“# Activated Experts”两行中，a + b 分别表示 a 个共享专家和 b 个路由专家。粗体表示除最后一列外的最佳或接近最佳性能。DeepSeekMoE 145B，乃至仅有一半激活专家参数的 DeepSeekMoE 142B（半激活），都以较大幅度优于 GShard 137B。此外，DeepSeekMoE 145B 在仅使用 28.5% 计算量的情况下，取得了与 DeepSeek 67B 相当的性能。（原文第 23 页）

结果：从表 6 所示的评估结果中，我们得到以下观察：（1）尽管总参数量与计算量相当，DeepSeekMoE 145B 仍显著优于 GShard 137B，再次凸显了 DeepSeekMoE 架构的优势。（2）总体而言，DeepSeekMoE 145B 在仅使用 28.5% 计算量的情况下，取得了与 DeepSeek 67B（稠密）相当的性能。与 DeepSeekMoE 16B 的发现一致，DeepSeekMoE 145B 在语言建模与知识密集型任务上展现出显著优势，但在选择题任务上存在局限。（3）在更大规模下，DeepSeekMoE 142B（半激活）的性能与 DeepSeekMoE 145B 相比并未落后太多。此外，尽管仅有一半的激活专家参数，DeepSeekMoE 142B（半激活）仍能在仅使用 18.2% 计算量的情况下达到 DeepSeek 67B（稠密）的性能。它也优于 GShard 137B，这与 4.5 节的结论一致。

8 相关工作

混合专家（Mixture of Experts, MoE）技术最早由 Jacobs et al. (1991) 与 Jordan and Jacobs (1994) 提出，用于以相互独立的专家模块处理不同的样本。Shazeer et al. (2017) 将 MoE 引入语言模型训练，构建了基于 LSTM（Hochreiter and Schmidhuber, 1997）的大规模 MoE 模型。随着 Transformer 成为 NLP 领域最流行的架构，许多工作将 Transformer 中的 FFN 扩展为 MoE 层，以构建 MoE 语言模型。GShard（Lepikhin et al., 2021）与 Switch Transformer（Fedus et al., 2021）是这一方向的先驱，它们采用可学习的 top-2 或 top-1 路由策略，将 MoE 语言模型扩展到极大的规模。Hash Layer（Roller et al., 2021）与 StableMoE（Dai et al., 2022b）采用固定路由策略，以实现更稳定的路由与训练。Zhou et al. (2022) 提出专家选择（expert-choice）路由策略，其中每个 token 可以被分配给不同数量的专家。

Zoph (2022) 关注 MoE 模型中训练不稳定与微调困难的问题，并提出 ST-MoE 来克服这些挑战。除了对 MoE 架构与训练策略的研究外，近年来还涌现出大量基于现有 MoE 架构的大规模语言模型或多模态模型（Du et al., 2022; Lin et al., 2021; Ren et al., 2023; Xue et al., 2023）。总体而言，以往大多数 MoE 模型都基于传统的 top-1 或 top-2 路由策略，在提升专家专业化方面仍有很大改进空间。为此，我们的 DeepSeekMoE 架构旨在最大限度地提升专家专业化程度。

9 结论

在本文中，我们介绍了面向 MoE 语言模型的 DeepSeekMoE 架构，其目标是实现极致的专家专业化。通过细粒度专家分割与共享专家隔离，DeepSeekMoE 相比主流 MoE 架构实现了显著更高的专家专业化程度与性能。我们从 2B 参数的适度规模出发，验证了 DeepSeekMoE 的优势，展示了其逼近 MoE 模型性能上限的能力。此外，我们提供了经验证据，表明 DeepSeekMoE 的专家专业化程度高于 GShard。在扩展到 16B 总参数的更大规模后，我们在 2T token 上训练了 DeepSeekMoE 16B，并展示了其与 DeepSeek 7B 和 LLaMA2 7B 相当的优异性能，且仅使用约 40% 的计算量。此外，我们进行了监督微调以实现对齐，基于 DeepSeekMoE 16B 构建了一个 MoE 对话模型，进一步展示了其适应性与通用性。更进一步，我们开展了将 DeepSeekMoE 扩展到 145B 参数的初步探索。我们发现，DeepSeekMoE 145B 仍相对 GShard 架构保持显著优势，并在仅使用 28.5%（甚至可能低至 18.2%）计算量的情况下，展现出与 DeepSeek 67B 相当的性能。

出于研究目的，我们公开发布了 DeepSeekMoE 16B 的模型 checkpoint，它可以在单块 40GB 显存的 GPU 上部署。我们期望这项工作能为学术界与工业界提供有价值的洞见，并推动大规模语言模型的加速发展。

附录

A.1 超参数概览

A.2 与更大模型的比较

A.3 DeepSeekMoE 16B 的训练基准曲线

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] and propose ST-MoE to overcome these challenges. In addition to research on MoE architectures and training strategies, recent years have also witnessed the emergence of numerous large-scale language or multimodal models (Du et al., 2022; Lin et al., 2021; Ren et al., 2023; Xue et al., 2023) based on existing MoE architectures. By and large, most of the previous MoE models are based on conventional top-1 or top-2 routing strategies, leaving large room for improving expert specialization. In response, our DeepSeekMoE architecture aims to improve the expert specialization to the utmost extent.

[2] In this paper, we introduce the DeepSeekMoE architecture for MoE language models, with the objective of achieving ultimate expert specialization. Through fine-grained expert segmentation and shared expert isolation, DeepSeekMoE achieves significantly higher expert specialization and performance compared with prevailing MoE architectures. Starting with a modest scale of 2B parameters, we validate the advantages of DeepSeekMoE, demonstrating its capability to approach the upper bound performance for MoE models. Furthermore, we provide empirical evidence to show that DeepSeekMoE has a higher level of expert specialization than GShard.

[3] Scaling up to a larger scale of 16B total parameters, we train DeepSeekMoE 16B on 2T tokens and demonstrate its outstanding performance comparable with DeepSeek 7B and LLaMA2 7B, with only about 40% of computations. Additionally, supervised fine-tuning is conducted for alignment to construct an MoE chat model based on DeepSeekMoE 16B, further showing its adaptability and versatility. Further, we perform a preliminary exploration to scale DeepSeek- MoE to 145B parameters. We find that DeepSeekMoE 145B still keeps substantial advantages over the GShard architecture, and demonstrates comparable performance with DeepSeek 67B, using only 28.5% (maybe even 18.2%) of computations.

[4] For research purposes, we release the model checkpoint of DeepSeekMoE 16B to the public, which can be deployed on a single GPU with 40GB of memory. We aspire for this work to provide valuable insights for both academia and industry, and contribute to the accelerated advancement of large-scale language models.

[5] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocaru, M. Debbah, E. Goffinet, D. Heslow, J. Launay, Q. Malartic, B. Noune, B. Pannier, and G. Penedo. Falcon-40B: an open large language model with state-of-the-art performance, 2023.

[6] M. Artetxe, S. Bhosale, N. Goyal, T. Mihaylov, M. Ott, S. Shleifer, X. V . Lin, J. Du, S. Iyer, R. Pasunuru, G. Anantharaman, X. Li, S. Chen, H. Akin, M. Baines, L. Martin, X. Zhou, P . S. Koura, B. O'Horo, J. Wang, L. Zettlemoyer, M. T. Diab, Z. Kozareva, and V . Stoyanov. Efficient large scale language modeling with mixtures of experts. In Y. Goldberg, Z. Kozareva, and Y. Zhang, editors, Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022, pages 11699–11732. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022 .EMNLP-MAIN.804. URL https://doi.org/10.18653/v1/2022.emnlp-main.804 .

[7] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021. S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O'Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skowron, L. Sutawika, and O. van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA, volume 202 of Proceedings of Machine Learning Research, pages 2397–2430. PMLR, 2023. URL https: //proceedings.mlr.press/v202/biderman23a.html .

[8] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi. PIQA: reasoning about physical commonsense in natural language. In The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020, pages 7432–7439. AAAI Press, 2020. doi: 10.1609/aaai.v34i05.6239. URL https://doi.org/10.1609/aaai.v34i05.6239 .

[9] S. Black, L. Gao, P . Wang, C. Leahy, and S. Biderman. GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow, Mar. 2021. URL https://doi.org/10.5281/ zenodo.5297715 . If you use this misc, please cite it using these metadata.

[10] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P . Dhariwal, A. Neelakantan, P . Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8 ac142f64a-Abstract.html .

[11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A.

Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374 .

[12] P . Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. CoRR, abs/1803.05457, 2018. URL http://arxiv.org/abs/1803.05457 .

[13] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[14] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei. Knowledge neurons in pretrained transformers. In S. Muresan, P . Nakov, and A. Villavicencio, editors, Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022, pages 8493–8502. Association for Computational Linguistics, 2022a. doi: 10.18653/V1/2022.ACL-LONG.581. URL https://doi.org/10.1 8653/v1/2022.acl-long.581 . D. Dai, L. Dong, S. Ma, B. Zheng, Z. Sui, B. Chang, and F. Wei. Stablemoe: Stable routing strategy for mixture of experts. In S. Muresan, P . Nakov, and A. Villavicencio, editors, Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022, pages 7085–7095. Association for Computational Linguistics, 2022b. doi: 10.18653/V1/2022.ACL-LONG.489. URL https://doi.org/10.18653/v1/2022.acl-long.489 .

[15] DeepSeek-AI. Deepseek llm: Scaling open-source language models with longtermism. arXiv preprint arXiv:2401.02954, 2024.

[16] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, B. Zoph, L. Fedus, M. P . Bosma, Z. Zhou, T. Wang, Y. E. Wang, K. Webster, M. Pellat, K. Robinson, K. S. Meier-Hellstern, T. Duke, L. Dixon, K. Zhang, Q. V . Le, Y. Wu, Z. Chen, and C. Cui. Glam: Efficient scaling of language models with mixture-of-experts. In International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA, volume 162 of Proceedings of Machine Learning Research, pages 5547–5569. PMLR, 2022. URL https://proceedings.mlr.press/v162/du22c.html .

[17] D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368– 2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246 .

[18] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. CoRR, abs/2101.03961, 2021. URL https://arxiv.org/ abs/2101.03961 .

[19] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

[20] X. Geng and H. Liu. Openllama: An open reproduction of llama, May 2023. URL https: //github.com/openlm-research/open_llama .

[21] A. Harlap, D. Narayanan, A. Phanishayee, V . Seshadri, N. R. Devanur, G. R. Ganger, and P . B. Gibbons. Pipedream: Fast and efficient pipeline parallel DNN training. CoRR, abs/1806.03377, 2018. URL http://arxiv.org/abs/1806.03377 .

[22] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[23] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset, 2021.

[24] High-Flyer. Hai-llm: An efficient and lightweight tool for training large models, 2023. URL https://www.high-flyer.cn/en/blog/hai-llm .

[25] S. Hochreiter and J. Schmidhuber. Long short-term memory. Neural Computing, 9(8):1735–1780, 1997. URL https://doi.org/10.1162/neco.1997.9.8.1735 . J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models. CoRR, abs/2203.15556, 2022. doi: 10.48550 /arXiv.2203.15556. URL https://doi.org/10.48550/arXiv.2203.15556 .

[26] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

[27] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. Neural Computing, 3(1):79–87, 1991. URL https://doi.org/10.1162/neco.1991.3.1. 79 .

[28] M. I. Jordan and R. A. Jacobs. Hierarchical mixtures of experts and the EM algorithm. Neural Computing, 6(2):181–214, 1994. URL https://doi.org/10.1162/neco.1994.6.2.181 .

[29] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. triviaqa: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. arXiv e-prints, art. arXiv:1705.03551, 2017.

[30] V . A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro. Reducing activation recomputation in large transformer models. Proceedings of Machine Learning and Systems, 5, 2023.

[31] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, M. Kelcey, J. Devlin, K. Lee, K. N. Toutanova, L. Jones, M.-W. Chang, A. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: a benchmark for question answering research. Transactions of the Association of Computational Linguistics, 2019.

[32] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. H. Hovy. RACE: large-scale reading comprehension dataset from examinations. In M. Palmer, R. Hwa, and S. Riedel, editors, Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017, Copenhagen, Denmark, September 9-11, 2017, pages 785–794. Association for Computational Linguistics, 2017. doi: 10.18653/V1/D17-1082. URL https://doi.org/10.18653/v1/d1 7-1082 .

[33] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In 9th International Conference on Learning Representations, ICLR 2021. OpenReview.net, 2021. URL https://openreview.net/forum?id=qrwe7XHTmYb .

[34] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.

[35] J. Lin, R. Men, A. Yang, C. Zhou, M. Ding, Y. Zhang, P . Wang, A. Wang, L. Jiang, X. Jia, J. Zhang, J. Zhang, X. Zou, Z. Li, X. Deng, J. Liu, J. Xue, H. Zhou, J. Ma, J. Yu, Y. Li, W. Lin, J. Zhou, J. Tang, and H. Yang. M6: A chinese multimodal pretrainer. CoRR, abs/2103.00823, 2021. URL https://arxiv.org/abs/2103.00823 .

[36] S. Lin, J. Hilton, and O. Evans. Truthfulqa: Measuring how models mimic human falsehoods. In S. Muresan, P . Nakov, and A. Villavicencio, editors, Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, May 22-27, 2022, pages 3214–3252. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.ACL-LONG.229. URL https://doi.org/10.18653/v1/2022.a cl-long.229 .

[37] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In 7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019. OpenReview.net, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7 .

[38] D. Narayanan, M. Shoeybi, J. Casper, P . LeGresley, M. Patwary, V . Korthikanti, D. Vainbrand, P . Kashinkunti, J. Bernauer, B. Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–15, 2021.

[39] OpenAI. GPT-4 technical report. CoRR, abs/2303.08774, 2023. doi: 10.48550/arXiv.2303.08774. URL https://doi.org/10.48550/arXiv.2303.08774 .

[40] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: memory optimizations toward training trillion parameter models. In C. Cuicchi, I. Qualters, and W. T. Kramer, editors, Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020, page 20. IEEE/ACM, 2020. doi: 10.1109/SC41405.2020.00024. URL https://doi.org/10.1109/SC 41405.2020.00024 .

[41] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He. Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation AI scale. In K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvári, G. Niu, and S. Sabato, editors, International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA, volume 162 of Proceedings of Machine Learning Research, pages 18332–18346. PMLR, 2022. URL https://proceedings.mlr.press/v162/rajbh andari22a.html .

[42] X. Ren, P . Zhou, X. Meng, X. Huang, Y. Wang, W. Wang, P . Li, X. Zhang, A. Podolskiy, G. Arshinov, A. Bout, I. Piontkovskaya, J. Wei, X. Jiang, T. Su, Q. Liu, and J. Yao. Pangu- Σ : Towards trillion parameter language model with sparse heterogeneous computing. CoRR, abs/2303.10845, 2023. URL https://doi.org/10.48550/arXiv.2303.10845 .

[43] S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. CoRR, abs/2106.04426, 2021. URL https://arxiv.org/abs/2106.04426 .

[44] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.

[45] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilic, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé, J. Tow, A. M. Rush, S. Biderman, A. Webson, P . S. Ammanamanchi, T. Wang, B. Sagot, N. Muennighoff, A. V . del Moral, O. Ruwase, R. Bawden, S. Bekman, A. McMillan-Major, I. Beltagy, H. Nguyen, L. Saulnier, S. Tan, P . O. Suarez, V . Sanh, H. Laurençon, Y. Jernite, J. Launay, M. Mitchell, C. Raffel, A. Gokaslan, A. Simhi, A. Soroa, A. F. Aji, A. Alfassy, A. Rogers, A. K. Nitzav, C. Xu, C. Mou, C. Emezue, C. Klamm, C. Leong, D. van Strien, D. I. Adelani, and et al. BLOOM: A 176b-parameter open-access multilingual language model. CoRR, abs/2211.05100, 2022. doi: 10.48550/ARXIV.2211.05100. URL https: //doi.org/10.48550/arXiv.2211.05100 . R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers. The Association for Computer Linguistics, 2016. doi: 10.18653/V1/P16-1162. URL https: //doi.org/10.18653/v1/p16-1162 .

[46] N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019. URL http://arxiv.org/abs/1911.02150 .

[47] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V . Le, G. E. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In 5th International Conference on Learning Representations, ICLR 2017. OpenReview.net, 2017. URL https: //openreview.net/forum?id=B1ckMDqlg .

[48] S. Shen, L. Hou, Y. Zhou, N. Du, S. Longpre, J. Wei, H. W. Chung, B. Zoph, W. Fedus, X. Chen, T. Vu, Y. Wu, W. Chen, A. Webson, Y. Li, V . Zhao, H. Yu, K. Keutzer, T. Darrell, and D. Zhou. Flan-moe: Scaling instruction-finetuned language models with sparse mixture of experts. CoRR, abs/2305.14705, 2023. doi: 10.48550/ARXIV.2305.14705. URL https://doi.org/10 .48550/arXiv.2305.14705 .

[49] M. Shoeybi, M. Patwary, R. Puri, P . LeGresley, J. Casper, and B. Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053, 2019.

[50] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[51] P . Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, MAPL 2019, page 10–19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367196. doi: 10.1145/331550 8.3329973. URL https://doi.org/10.1145/3315508.3329973 .

[52] Together-AI. Redpajama-data: An open source recipe to reproduce llama training dataset, April 2023. URL https://github.com/togethercomputer/RedPajama-Data .

[53] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. Llama: Open and efficient foundation language models. CoRR, abs/2302.13971, 2023a. doi: 10.48550/arXiv.230 2.13971. URL https://doi.org/10.48550/arXiv.2302.13971 .

[54] H. Touvron, L. Martin, K. Stone, P . Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P . Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton-Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V . Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V . Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P . S. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P . Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P . Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. CoRR, abs/2307.09288, 2023b. doi: 10.48550/arXiv.2307.09288. URL https://doi.org/10.48550/arXiv.2307. 09288 . A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, pages 5998–6008, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd 053c1c4a845aa-Abstract.html .

[55] B. Wang and A. Komatsuzaki. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. https://github.com/kingoflolz/mesh-transformer-jax , May 2021.

[56] L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL https://doi.org/10.18653/v1/2020.coling-main.419 .

[57] F. Xue, Z. Zheng, Y. Fu, J. Ni, Z. Zheng, W. Zhou, and Y. You. Openmoe: Open mixture-of-experts language models. https://github.com/XueFuzhao/OpenMoE , 2023.

[58] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[59] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V . Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P . S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer. Opt: Open pre-trained transformer language models, 2022.

[60] C. Zheng, M. Huang, and A. Sun. Chid: A large-scale chinese idiom dataset for cloze test. In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 778–787. Association for Computational Linguistics, 2019. doi: 10.18653/V1/P19-1075. URL https://doi.org/10.18653/v1/p19-1075 .

[61] Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V . Zhao, A. M. Dai, Z. Chen, Q. V . Le, and J. Laudon. Mixture-of-experts with expert choice routing. In NeurIPS, 2022. URL http://papers.nip s.cc/paper_files/paper/2022/hash/2f00ecd787b432c1d36f3de9800728eb-Abs tract-Conference.html .

[62] B. Zoph. Designing effective sparse expert models. In IEEE International Parallel and Distributed Processing Symposium, IPDPS Workshops 2022, Lyon, France, May 30 - June 3, 2022, page 1044. IEEE, 2022. URL https://doi.org/10.1109/IPDPSW55747.2022.0 0171 .

DeepSeek-Coder：当大语言模型遇见编程——代码智能的崛起

DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence

*Daya Guo*1, Qihao Zhu ∗1,2, Dejian Yang1, Zhenda Xie1, Kai Dong1, Wentao Zhang1*

DeepSeek-AI research@deepseek.com

arXiv:2401.14196 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

大语言模型的快速发展彻底改变了软件开发中的代码智能。然而，闭源模型的主导地位限制了广泛的研究与开发。为此，我们推出 DeepSeek-Coder 系列——一系列开源代码模型，参数规模从 1.3B 到 33B，从零开始训练，训练 token 数达 2 万亿（2 trillion）。这些模型在高质量的项目级代码语料上预训练，并采用窗口长度为 16K 的填空（fill-in-the-blank）任务，以增强代码生成与代码填充（infilling）能力。我们的大量评测表明，DeepSeek-Coder 不仅在多个基准上取得开源代码模型中的最先进性能，还超越了 Codex、GPT-3.5 等现有闭源模型。此外，DeepSeek-Coder 模型采用宽松许可证，同时允许研究与不受限制的商业使用。

1 引言

软件开发领域已被大语言模型的迅猛发展显著改变（OpenAI，2023；Touvron 等，2023），这些模型开启了代码智能的新时代。它们有望自动完成并简化编码的诸多环节——从缺陷检测到代码生成——从而提升生产力并降低人为错误的可能性。然而，该领域的一大挑战在于开源模型（Li 等，2023；Nijkamp 等，2022；Roziere 等，2023；Wang 等，2021）与闭源模型（Gemini Team，2023；OpenAI，2023）之间的性能差距。巨型闭源模型虽然强大，但因其专有性质，许多研究人员与开发者往往无法获取。

为应对这一挑战，我们推出 DeepSeek-Coder 系列。该系列包含一系列开源代码模型，规模从 1.3B 到 33B，每个规模均提供基础（base）版本与指令（instruct）版本。系列中的每个模型都从零开始，在来自 87 种编程语言的 2 万亿 token 上训练，以确保对编程语言与语法的全面理解。此外，我们尝试在仓库层面组织预训练数据，以增强预训练模型在仓库内跨文件语境下的理解能力。除在预训练阶段采用下一 token 预测损失外，我们还引入了中间填充（Fill-In-Middle, FIM）方法（Bavarian 等，2022；Li 等，2023）。该方法旨在进一步增强模型的代码补全能力。为满足处理更长代码输入的需求，我们将上下文长度扩展至 16K。这一调整使我们的模型能够处理更复杂、更庞大的编码任务，从而提升其在各种编码场景中的通用性与适用性。

我们使用多种公开的代码相关基准开展了全面实验。结果表明，在开源模型中， DeepSeek-Coder-Base 33B 在所有基准上均持续取得更优表现。此外，DeepSeek-Coder-Instruct 33B 在大多数评测基准上超越 OpenAI GPT-3.5 Turbo，显著缩小了 OpenAI GPT-4 与开源模型之间的性能差距。值得注意的是， DeepSeek-Coder-Base 7B 尽管参数量更少，却展现出与参数量为其五倍的模型（如 CodeLlama-33B（Roziere 等，2023））相比具有竞争力的性能。综上所述，我们的主要贡献如下：

- 我们推出 DeepSeek-Coder-Base 与 DeepSeek-Coder-Instruct，这是两个先进的、聚焦代码的大语言模型（LLM）。它们在海量代码语料上经过大量训练，能够熟练理解 87 种编程语言。此外，它们提供多种模型规模，以满足广泛的计算与应用需求。
- 我们首次尝试在模型的预训练阶段引入仓库级数据构建。我们发现这能显著提升跨文件代码生成能力。
- 我们的分析严格考察了 FIM 训练策略对代码模型预训练阶段的影响。这些全面研究的结果揭示了 FIM 配置中一些有趣的方面，为代码预训练模型的改进与发展提供了有价值的洞见。
- 我们针对涵盖众多代码相关任务的大量基准，对代码 LLM 进行了广泛评测。结果表明，DeepSeek-Coder-Base 在这些基准上超越了所有现有开源代码 LLM。此外，借助指令数据的细致微调，DeepSeek-Coder-Instruct 在代码相关任务上的表现优于 OpenAI GPT-3.5 Turbo 模型。

2 数据收集

DeepSeek-Coder 的训练数据集由 87% 的源代码、10% 的英文代码相关自然语言语料与 3% 的与代码无关的中文自然语言语料构成。英文语料由来自 GitHub 的 Markdown 与 StackExchange（https://stackexchange.com）的材料构成，用于增强模型对代码相关概念的理解，并提升其处理库使

用、缺陷修复等任务的能力。中文语料则由高质量文章构成，旨在提升模型理解中文的熟练程度。本节将概述我们如何构建代码训练数据。该过程包括数据抓取、基于规则的过滤、依赖解析、仓库级去重与质量筛查，如图 2 所示。下文将逐步介绍数据创建流程。

【图表】图 2 数据集构建流程（原文第 3 页）

2.1 GitHub 数据抓取与过滤

我们收集 GitHub 上 2023 年 2 月之前创建的公开仓库，仅保留 87 种编程语言，如表 1 所列。为减少待处理的数据量，我们采用与 StarCoder 项目（Li 等，2023）类似的过滤规则，初步滤除质量较低的代码。应用这些过滤规则后，数据总量降至原始规模的 32.8%。为使论文内容完整自洽，我们简要描述 StarCoder Data 项目中使用的过滤规则：

首先，我们滤除平均行长超过 100 个字符或最大行长超过 1000 个字符的文件。此外，我们移除字母字符占比低于 25% 的文件。除 XSLT 编程语言外，我们还进一步滤除前 100 个字符中出现字符串 "<?xml version=" 的文件。对于 HTML 文件，我们考虑可见文本与 HTML 代码的比例，保留可见文本至少占代码的 20% 且不少于 100 个字符的文件。对于通常包含更多数据的 JSON 与 YAML 文件，我们只保留字符数在 50 到 5000 之间的文件。这有效地移除了大多数数据量过大的文件。

2.2 依赖解析

在此前的工作中（Chen 等，2021；Li 等，2023；Nijkamp 等，2022；Roziere 等，2023），面向代码的大语言模型主要基于文件级源代码预训练，忽略了项目中不同文件之间的依赖关系。然而在实际应用中，此类模型难以有效扩展到处理整个项目级的代码场景。因此，我们将在这一步中考虑如何利用同一仓库内文件之间的依赖关系。具体而言，我们首先解析文件间的依赖关系，然后按一定顺序排列这些文件，确保每个文件所依赖的上下文在该文件之前进入输入序列。通过依据依赖关系对齐文件，我们的数据集能更准确地反映真实的编码实践与结构。这种增强的对齐不仅使数据集更具相关性，还可能提升模型在处理项目级代码场景时的实用性与适用性。值得注意的是，我们只考虑文件之间的调用关系，并使用正则表达式提取它们，例如 Python 中的 "import"、C# 中的 "using" 以及 C 中的 "include"。

算法 1：依赖分析的拓扑排序（原文第 4 页）

1: procedure TOPOLOGICALSORT(files)

```
    graphs ← {}  ▷ 初始化空的邻接表
    inDegree ← {}  ▷ 初始化空字典，用于记录各文件的入度
    对于 files 中的每个 file 执行
        graphs[file] ← []
        inDegree[file] ← 0
    结束循环
    对于 files 中的每个 fileA 执行
        对于 files 中的每个 fileB 执行
            若 HASDEPENDENCY(fileA, fileB) 成立，则  ▷ 若 fileA 依赖 fileB
                graphs[fileB].append(fileA)  ▷ 添加从 B 指向 A 的有向边
                inDegree[fileA] ← inDegree[fileA] + 1  ▷ A 的入度加 1
            结束分支
        结束循环
    结束循环
    subgraphs ← getDisconnectedSubgraphs(graphs)  ▷ 识别不连通的子图
    allResults ← []
    对于 subgraphs 中的每个 subgraph 执行
        results ← []
        当 length(results) ≠ NumberOfNodes(subgraph) 时执行
            file ← argmin({inDegree[file] | file ∈ subgraph 且 file ∉ results})
            对于 graphs[file] 中的每个 node 执行
                inDegree[node] ← inDegree[node] − 1
            结束循环
            results.append(file)
        结束循环
        allResults.append(results)
    结束循环
```

32: 返回 allResults

33: 结束过程

算法 1 描述了针对同一项目内文件列表进行依赖分析的拓扑排序。它首先建立两个数据结构：名为 "graphs" 的空邻接表，用于表示文件间的依赖关系；以及名为 "inDegree" 的空字典，用于存储每个文件的入度。随后，算法遍历每一对文件以识别依赖关系，并相应更新 "graphs" 与 "inDegree"。接下来，它在整体依赖图中识别出不连通的子图。对于每个子图，算法采用一种改进的拓扑排序：与选择入度为 0 的节点的标准做法不同，该算法选择入度最小的节点，从而能够处理图中的环。被选中的节点会加入 "results" 列表，其相连节点的入度则相应减 1。该过程持续进行，直到为每个子图生成一个拓扑排序序列。算法最后返回这些已排序序列组成的列表，每个序列中的文件会被拼接成一个训练样本。为纳入文件路径信息，每个文件的开头都会加上一条标明其路径的注释。该方法确保路径信息在训练数据中得以保留。

2.3 仓库级去重

近期研究表明，对大语言模型（LLM）的训练数据集进行去重可以带来显著的性能提升。Lee 等（2022）已表明，语言模型训练语料中常包含大量近似重复内容，移除较长的重复子串可以提升 LLM 的性能。Kocetkov 等（2022）将近似去重方法应用于训练数据，取得了显著改进，并强调近似去重是在代码基准任务上取得有竞争力性能的关键预处理步骤。在我们的数据集中，我们也采用了近似去重。不过，我们的方法与以往工作有所不同：我们在代码的仓库级进行去重，而非在文件级进行，因为后者可能滤除仓库中的某些文件，进而破坏仓库结构。具体而言，我们将仓库级拼接后的代码视为单个样本，并应用相同的近似去重算法，以确保仓库结构的完整性。

2.4 质量筛查与去污染

除应用第 2.1 节所述的过滤规则外，我们还使用编译器与质量模型，并结合启发式规则，进一步滤除低质量数据，包括存在语法错误、可读性差、模块化程度低的代码。我们在表 1 中给出了源代码的统计汇总，共涵盖 87 种语言，详细列出每种语言的磁盘大小、文件数量与占比。数据总量为 798 GB，共 6.03 亿个文件。为确保代码训练数据不被 GitHub 上可能存在的测试集信息污染，我们实施了 n-gram 过滤流程。该流程会移除任何符合特定条件的代码片段。具体而言，我们滤除包含 HumanEval（Chen 等，2021）、MBPP（Austin 等，2021）、GSM8K（Cobbe 等，2021）与 MATH（Hendrycks 等，2021）等来源的文档字符串、题目与解答的文件。过滤标准如下：若某段代码包含与测试数据中

任一字符串相同的 10-gram 字符串，则将其从训练数据中排除；若测试数据中的字符串长度不足 10-gram 但不低于 3-gram，则采用精确匹配方式进行过滤。

【表格】表 1 所选编程语言清洗后训练数据的汇总（原文第 6 页）

3 训练策略

3.1 训练方法

3.1.1 下一 token 预测

我们模型的第一个训练目标即下一 token 预测（next token prediction）。在这一过程中，各类文件被拼接成固定长度的条目，随后这些条目用于训练模型，使其能够基于给定的上下文预测后续 token。

3.1.2 中间填充（FIM）

我们模型的第二个训练目标是所谓的中间填充（fill-in-the-middle）。在代码预训练场景中，常常需要根据给定的上下文与后续文本生成相应的插入内容。由于编程语言中存在特定的依赖关系，仅依靠下一个 token 预测不足以学习这种中间填充能力。因此，若干研究工作（Bavarian 等，2022；Li 等，2023）提出了中间填充（Fill-in-the-Middle，FIM）的预训练方法。该方法将文本随机划分为三个部分，然后打乱这些部分的顺序，并用特殊字符将它们连接起来，旨在在训练过程中引入一种填空式的预训练任务。在 FIM 方法中，采用了两种不同的模式：PSM（Prefix-Suffix-Middle，前缀-后缀-中间）与 SPM（Suffix-Prefix-Middle，后缀-前缀-中间）。在 PSM 模式下，训练语料按 Prefix、Suffix、Middle 的顺序组织，使中间片段夹在前缀与后缀之间；相反，SPM 模式按 Suffix、Prefix、Middle 的顺序排列各片段，呈现出不同的结构挑战。这些模式有助于增强模型处理代码中各种结构排列的能力，为高级代码预测任务提供了一个稳健的训练框架。

【图表】图 3 使用 FIM 目标的有效性（原文第 7 页）

为确定 FIM 方法中各种超参数的有效性，我们开展了一系列消融实验。

实验设置：在本实验中，我们采用 DeepSeek-Coder-Base 1.3B 作为模型架构。为简化实验流程，我们聚焦于训练数据集中的 Python 子集。我们的主要目标是借助 HumanEval-FIM 基准（Fried 等，2022）评估中间填充（FIM）技术的效果。该基准专门面向 Python 的单行 FIM 任务：随机遮蔽

HumanEval 解答中的一行代码，以测试模型预测缺失行的能力。我们假设，与传统的下一个 token 预测目标相比，PSM 模式可能表现出细微差异，主要原因是 PSM 会重新排列原文顺序，可能影响模型的学习动态。因此，我们在四种不同配置下实现 PSM 模式的 FIM：0% FIM 率、50% FIM 率、100% FIM 率以及 50% MSP 率。掩码片段预测（Masked Span Prediction，MSP）策略最初由 T5（Raffel 等，2023）提出，它遮蔽多个文本片段并训练模型重建这些片段。根据 CodeGen2.5（Nijkamp 等，2023）的研究，与 PSM 相比，MSP 可能提升 FIM 性能。因此，我们将该方法纳入对比分析。

结果：实验结果如图 3 所示。虽然模型在 100% FIM 率下于 HumanEval-FIM 上表现最佳，但这一配置也导致代码补全能力最弱。这表明 FIM 与代码补全能力之间存在权衡。此外，我们观察到，在 50% PSM 率下，模型的表现优于 MSP 策略。为在 FIM 效率与代码补全能力之间取得平衡，我们最终选择 50% PSM 率作为首选的训练策略。

在我们的实现中，我们为该任务专门引入了三个哨兵 token。对于每个代码文件，我们首先将其内容划分为三个片段，分别记为 f_pre、f_middle 与 f_suf。采用 PSM 模式时，我们按如下方式构造训练样本：

`<｜fim_start｜>f_pre<｜fim_hole｜>f_suf<｜fim_end｜>f_middle<|eos_token|>`

我们在打包（packing）过程之前、于文档级别实现中间填充（FIM）方法，这与 Bavarian 等（2022）原始工作所提出的一致；FIM 率设为 0.5，并采用 PSM 模式。

3.2 分词器

在分词过程中，我们使用 HuggingFace Tokenizer 库（https://github.com/huggingface/tokenizers），按照 Sennrich 等（2015）所述，在训练语料的一个子集上训练字节对编码（Byte Pair Encoding，BPE）分词器。最终，我们采用的词表大小为 32,000。

3.3 模型架构

我们开发了一系列不同参数规模的模型，以满足多样化的应用需求，包括 1.3B、6.7B 与 33B 参数的模型。这些模型均基于 DeepSeek 大语言模型（LLM）（DeepSeek-AI，2024）所述的同一框架构建。每个模型都是一个仅解码器（decoder-only）Transformer，采用 Su 等（2023）所述的旋转位置编码（Rotary Position Embedding，RoPE）。值得注意的是，DeepSeek 33B 模型集成了分组查询注意力（Grouped-Query-Attention，GQA），分组大小为 8，从而提升训练与推理效率。此外，我们采用 FlashAttention v2（Dao，2023）来加速注意力机制中的计算。模型的架构细节汇总于表 2。

【表格】表 2 DeepSeek-Coder 的超参数（原文第 9 页）

3.4 优化

遵循 DeepSeek LLM（DeepSeek-AI，2024），我们使用 AdamW（Loshchilov 与 Hutter，2019）作为优化器，β1 与 β2 分别取 0.9 与 0.95。我们根据 DeepSeek LLM 中建议的缩放定律调整批大小与学习率。在学习率调度方面，我们采用三阶段策略，包含 2000 步预热，并将最终学习率设为初始学习率的 10%。值得注意的是，每个阶段的学习率都降至前一阶段学习率的 $\sqrt{}(1/10)$，遵循 DeepSeek LLM（DeepSeek-AI，2024）中确立的准则。

3.5 环境

我们的实验在 HAI-LLM（High-Flyer，2023）框架上开展，该框架以高效、轻量的大语言模型训练方式著称。该框架融合了多种并行策略以优化计算效率，包括张量并行（Korthikanti 等，2023），以及 ZeRO 数据并行（Rajbhandari 等，2020）与 PipeDream 流水线并行（Narayanan 等，2019）。我们的实验使用配备 NVIDIA A100 与 H800 GPU 的集群。在 A100 集群中，每个节点配置 8 块 GPU，GPU 成对通过 NVLink 桥接互连；H800 集群的配置与此类似，每个节点包含 8 块 GPU，这些 GPU 通过 NVLink 与 NVSwitch 技术的组合互连，确保节点内的高效数据传输。为便于 A100 与 H800 集群中节点之间的无缝通信，我们采用以高吞吐与低延迟著称的 InfiniBand 互连。这一配置为我们的计算实验提供了稳健而高效的基础设施。

3.6 长上下文

为增强 DeepSeek-Coder 处理扩展上下文的能力，特别是仓库级代码处理等场景，我们重新配置了 RoPE（Su 等，2023）参数，以扩展默认上下文窗口。遵循以往实践（Chen 等，2023；kaiokendev，2023），我们采用线性缩放策略，将缩放因子从 1 提高到 4，并将基频从 10000 调整为 100000。模型额外进行了 1000 步训练，批大小为 512，序列长度为 16K，学习率与最终预训练阶段保持一致。理论上，这些修改使我们的模型能够处理最多 64K token 的上下文。然而，经验观察表明，模型在 16K token 范围内给出的输出最为可靠。未来研究将继续改进与评估长上下文适配方法，以进一步提升 DeepSeek-Coder 在处理扩展上下文时的效率与易用性。

3.7 指令微调

我们通过使用高质量数据进行基于指令的微调，在 DeepSeek-Coder-Base 的基础上开发了 DeepSeek-Coder-Instruct。这些数据由 Alpaca 指令格式（Taori 等，2023）组织的有帮助且无偏的人类指令构成。为划分每一轮对话，我们采用一个独特的分隔符 token `<|EOT|>` 来标记各片段的结束。训练方面，我们使用余弦调度，包含 100 步预热，初始学习率为 1e-5；批大小为 4M token，总计训练 2B token。

图 4 展示了使用 DeepSeek-Coder-Instruct 34B 的一个示例，这是一个构建贪吃蛇游戏的多轮对话场景。首先，我们要求模型使用 pygame 编写一个贪吃蛇游戏，模型成功创建了一个能够无 bug 运行的基础贪吃蛇游戏。为进一步改进游戏，我们要求在左上角添加计分系统，模型随后引入了 “score” 变量与 “display_score” 函数，并解释了如何集成这些功能。该示例体现了 DeepSeek-Coder-Instruct 在多轮对话设置下提供完整解决方案的能力。更多案例见附录 A。

【图表】图 4 DeepSeek-Coder-Instruct 33B 在多轮对话设置下的回复示例（原文第 10 页）

4 实验结果

在本节中，我们从四项任务评估 DeepSeek-Coder，包括代码生成（§4.1）、FIM 代码补全（§4.2）、跨文件代码补全（§4.3）与基于程序的数学推理（§4.4）。我们将 DeepSeek-Coder 与先前最先进的大语言模型进行比较：

- CodeGeeX2（Zheng 等，2023）是 CodeGeeX 多语言代码生成模型的第二代。它基于 ChatGLM2（Du 等，2022）架构开发，并使用大规模编码示例数据集进行了增强。
- StarCoder（Li 等，2023）是一个公开可用的模型，参数量达到 150 亿（15 billion）。它专门在精心筛选的 Stack 数据集（Kocetkov 等，2022）子集上训练，覆盖 86 种编程语言，确保其在广泛编码任务上的能力。
- CodeLlama（Roziere 等，2023）包含一系列以代码为中心的大语言模型（LLM），是 LLaMA2（Touvron 等，2023）的衍生模型。它提供 7B、13B 与 34B 三种规模，这些模型在 LLaMA2 基础架构之上，于 5000 亿 token 的庞大代码语料上继续训练。
- code-cushman-001（Chen 等，2021）是 OpenAI 开发的 120 亿参数模型，曾作为 GitHub Copilot 的初始模型。

- GPT-3.5 与 GPT-4（OpenAI，2023）是 OpenAI 开发的先进生成式 AI 模型。虽然它们并非专门针对代码生成训练，但也在此领域展现出显著性能。它们处理代码生成任务的有效性在很大程度上归因于其庞大的参数规模。

4.1 代码生成

HumanEval 与 MBPP 基准：HumanEval（Chen 等，2021）与 MBPP（Austin 等，2021）基准被广泛用于评估代码 LLM。HumanEval 由 164 个手写的 Python 问题组成，通过测试用例进行验证，以在零样本（zero-shot）设置下评估 Code LLM 生成的代码；而 MBPP 基准包含 500 个问题，采用少样本（few-shot）设置。为评估模型的多语言能力，我们将 HumanEval 基准的 Python 问题扩展到另外七种常用编程语言，即 C++、Java、PHP、TypeScript（TS）、C#、Bash 与 JavaScript（JS）（Cassano 等，2023）。对于这两个基准，我们采用贪心搜索，并在相同脚本与环境下重新实现基线结果，以保证公平比较。

【表格】表 3 各方法在多语言 HumanEval 与 MBPP 基准上的性能（原文第 11 页）

表 3 给出了结果。可以看出，DeepSeek-Coder-Base 取得了最先进的性能，在 HumanEval 上的平均准确率为 50.3%，在 MBPP 上为 66.0%。与规模相近的开源模型 CodeLlama-Base 34B 相比，我们的模型在准确率上分别显著提升了 9% 与 11%。值得注意的是，即使是我们更小的 DeepSeek-Coder-Base 6.7B 模型，也超过了 CodeLlama-Base 34B 的性能。经过指令微调后，我们的模型在 HumanEval 基准上超过了闭源 GPT-3.5-Turbo 模型，显著缩小了 OpenAI GPT-4 与开源模型之间的性能差距。

DS-1000 基准：HumanEval 与 MBPP 有一个显著缺陷：它们严重依赖简单的编程任务，可能无法准确代表大多数程序员通常编写的代码类型。相比之下，Lai 等（2023）提出的 DS-1000 基准提供了涵盖七个不同库的 1,000 个实用且真实的数据科学工作流集合。该基准通过针对特定测试用例执行代码来评估代码生成。DS-1000 的独特之处在于它按涉及的库对问题进行分类，这些库包括 Matplotlib、NumPy、Pandas、SciPy、Scikit-Learn、PyTorch 和 TensorFlow。该基准评估基础模型在代码补全设置下的表现，我们给出每个库的 pass@1 结果以及总体得分。DS-1000 基准的结果如表 4 所示。从表中可以看出，DeepSeek-Coder 模型在所有库上都取得了相对较高的准确率，这表明我们的模型不仅能够生成优质代码，还能在真实的数据科学工作流中更准确地使用各类库。

【表格】表 4 不同方法在 DS-1000 任务上的表现（原文第 12 页）

LeetCode 竞赛基准 为进一步验证模型在真实世界编程问题上的能力，我们构建了 LeetCode 竞赛基准（脚注 3）。LeetCode（脚注 4）提供竞赛级题目，具有显著的挑战性，可用来检验模型的问题理解能力与代码生成能力。我们从 LeetCode 竞赛中收集最新的题目，以避免这些题目或其解答出现在我们的预训练数据中。2023 年 7 月至 2024 年 1 月期间共收集了 180 道题目。对于每道题目，我们收集了 100 个测试用例以确保测试覆盖度。我们使用如下模板构建指令提示：

```
{problem_description}
Please complete the code below to solve the above problem:
```python
{code_template}
```
```

评估结果如表 5 所示。在我们的评估中，DeepSeek-Coder 系列模型相较当前的开源代码模型展现出卓越的性能。具体而言，DeepSeek-Coder-Instruct 6.7B 与 33B 在该基准上分别取得 19.4% 和 27.8% 的 Pass@1 分数。这一表现明显超越了 Code-Llama-33B 等现有开源模型。DeepSeek-Coder-Instruct 33B 是唯一在这项任务上优于 OpenAI GPT-3.5-Turbo 的开源模型。然而，与更强的 GPT-4-Turbo 相比，仍存在显著的性能差距。

我们的分析表明，采用思维链（Chain-of-Thought，CoT）提示能显著增强 DeepSeek-Coder-Instruct 模型的能力。这一提升在更具挑战性的任务子集上尤为明显。通过在初始提示之后追加指令“ You need first to write a step-by-step outline and then write the code.”（即“你需要先写出逐步提纲，然后再编写代码”），我们观察到了性能提升。这一观察使我们相信，先撰写详细的代码描述有助于模型更有效地理解并处理编码任务中逻辑与依赖关系的复杂之处，尤其是复杂度较高的任务。因此，我们强烈建议在使用 DeepSeek-Coder-Instruct 模型应对复杂编程挑战时采用 CoT 提示策略。这种做法能够促成更有条理、更具逻辑性的问题求解框架，从而有可能在代码生成任务中获得更精确、更高效的结果。

脚注 3：我们已将该基准公开发布于 https://github.com/deepseek-ai/DeepSeek-Coder/tree/main/Evaluation/LeetCode 。

脚注 4：https://leetcode.com/ 。

【表格】表 5 不同模型在 LeetCode 竞赛基准上的表现（原文第 13 页）

需要说明的是，尽管我们努力收集最新的代码题目用于模型测试，但仍无法完全排除数据污染的可能性。我们观察到，GPT-4-Turbo 与 DeepSeek-Coder 模型在 7 月和 8 月举办的 LeetCode 竞赛中取得了更高的分数。我们鼓励研究社区在今后使用我们发布的 LeetCode 数据评估模型时，考虑潜在的数据污染问题。

4.2 填充中间（Fill-in-the-Middle）代码补全

DeepSeek-Coder 模型在预训练阶段以 0.5 的 FIM（Fill-In-the-Middle，填充中间）率进行训练。这一专门的训练策略使模型能够基于给定代码片段的前缀与后缀上下文，通过填空的方式熟练地生成代码。该能力在代码补全工具领域尤为有利。已有若干开源模型具备类似能力，其中较为突出的包括 SantaCoder（Allal 等，2023）、StarCoder（Li 等，2023）和 CodeLlama（Roziere 等，2023）。这些模型在代码生成与补全领域树立了先例。在评估 DeepSeek-Coder 模型的性能时，我们与上述模型进行了对比分析。对比所用的基准是 Allal 等（2023）提出的单行填充（Single-Line Infilling）基准，涵盖三种不同的编程语言。该基准以行级精确匹配准确率作为评估指标。

【表格】表 6 不同方法在 FIM 任务上的表现（原文第 13 页）

评估结果如表 6 所示。尽管 DeepSeek-Coder 是参数量仅 13 亿的最小模型，它在这些基准上仍优于规模更大的 StarCoder 与 CodeLlama。这一优异表现可归因于 DeepSeek-Coder 所使用的高质量预训练数据。此外，我们观察到的一个显著趋势是模型规模与其性能之间的相关性：随着模型规模增大，性能也相应提升。这一趋势凸显了模型容量在实现更高代码补全准确率方面的重要性。基于这些发现，我们推荐在代码补全工具中部署 DeepSeek-Coder-Base 6.7B 模型。这一建议基于该模型在效率与准确率之间所展现出的平衡。DeepSeek-Coder-Base 6.7B 模型凭借其可观的参数规模，已被证明在代码补全场景中非常有效，是将先进计算能力集成到编码环境中的理想选择。

4.3 跨文件代码补全

本节中，我们将评估现有开源模型在跨文件代码补全任务上的表现。与上一节讨论的代码生成不同，跨文件代码补全要求模型访问并理解横跨多个文件、具有大量跨文件依赖的代码仓库。我们使用 CrossCodeEval（Ding 等，2023）来评估当前可用的 7B 规模开源代码模型在跨文件补全任务上的能力。该数据集构建于四种流行编程语言（Python、Java、TypeScript 和 C#）下的一组多样化、真实世界、开源且许可宽松的代码仓库之上。该数据集经过专门设计，严格依赖跨文件上下文才能完成准确补全。值得注意的是，该数据集由 2023 年 3 月至 6 月间创建的代码仓库构建，而我们的预训练数据仅包含 2023 年 2 月之前创建的代码，这确保该数据集未出现在我们的预训练数据中，从而避免了数据泄漏。

【表格】表 7 不同模型在跨文件代码补全上的表现（原文第 14 页）

在我们对各模型的评估中，我们将最大序列长度设为 2048 个 token，最大输出长度设为 50 个 token，跨文件上下文上限设为 512 个 token。对于跨文件上下文，我们使用 Ding 等（2023）提供的官方 BM25 检索结果。评估指标包括精确匹配（exact match）与编辑相似度（edit similarity）。表 7 所示的结果表明，DeepSeek-Coder 在多种语言的跨文件补全任务上始终优于其他模型，展现出其卓越的实际应用能力。当仅使用文件级代码语料（w/o Repo Pre-training，即不使用仓库级预训练）来预训练 DeepSeek-Coder 时，我们观察到在 Java、TypeScript 和 C# 语言上的性能有所下降，这表明仓库级预训练是有效的。

4.4 基于程序的数学推理

基于程序的数学推理考察模型通过编程来理解并求解数学问题的能力。这类推理在数据分析与科学计算等领域至关重要。为开展该项评估，我们采用 Gao 等（2023）提出的程序辅助数学推理（Program-Aided Math Reasoning，PAL）方法。该方法应用于七个各具独特挑战与情境的基准，包括 GSM8K（Cobbe 等，2021）、MATH（Hendrycks 等，2021）、GSM-Hard（Gao 等，2023）、SVAMP（Patel 等，2021）、TabMWP（Lu 等，2022）、ASDiv（Miao 等，2020）和 MAWPS（Gou 等，2023）。在这些基准中，模型被提示交替地用自然语言描述一个求解步骤，然后用代码执行该步骤。如表 8 所示，DeepSeek-Coder 模型在所有基准上均取得出色的表现，尤其是 33B 版本，这展现了此类模型在需要复杂数学计算与问题求解能力的应用中的潜力。

【表格】表 8 不同方法在程序辅助数学推理任务上的表现（原文第 15 页）

5 从通用大语言模型继续预训练

为进一步增强 DeepSeek-Coder 模型的自然语言理解与数学推理能力，我们在通用语言模型 DeepSeek-LLM-7B Base（DeepSeek-AI，2024）的基础上，使用 2 万亿 token 进行额外的预训练，得到 DeepSeek-Coder-v1.5 7B。此次预训练中，我们专门使用了表 9 所列的数据来源。与 DeepSeek-Coder 不同，DeepSeek-Coder-v1.5 在预训练阶段仅采用下一 token 预测目标，上下文长度为 4K。

【表格】表 9 DeepSeek-Coder-v1.5 7B 预训练的数据来源（原文第 15 页）

我们将 DeepSeek-Coder-v1.5 7B 与 DeepSeek-Coder 6.7B 进行对比，并使用我们的评估流水线重新运行了所有基准，以确保公平比较。我们评估了广泛的任务，可归类如下：

- **编程**：该类包括使用 Chen 等（2021）的 HumanEval 数据集进行的多语言设置评估，以及使用 Austin 等（2021）的 MBPP 数据集进行的 Python 设置评估。
- **数学推理**：我们使用 GSM8K 基准（Cobbe 等，2021）和 MATH 基准（Hendrycks 等，2021）[4] 评估数学推理任务的表现。这些任务通过生成程序来求解数学问题。
- **自然语言**：我们在自然语言任务上的评估包括 MMLU（Hendrycks 等，2020）、BBH（Suzgun 等，2022）、HellaSwag（Zellers 等，2019）、Winogrande（Sakaguchi 等，2021）和 ARC-Challenge（Clark 等，2018）等基准。

Base 模型与 Instruct 模型的结果见表 10。可以观察到，与 DeepSeek-Coder-Base 模型相比，DeepSeek-Coder-Base-v1.5 尽管在编码性能上略有下降，但在大多数任务上都取得了显著的提升。特别是在数学推理与自然语言类别中，DeepSeek-Coder-Base-v1.5 在所有基准上都显著优于其前代模型，这也表明其数学推理与自然语言处理能力得到了显著提升。

模型	规模	HumanEval（编程）	MBPP（编程）	GSM8K（数学）	MATH（数学）	MMLU（自然语言）	BBH（自然语言）	HellaSwag（自然语言）	WinoG（自然语言）	ARC-C（自然语言）
DeepSeek-Coder-Base	6.7B	44.7%	60.6%	43.2%	19.2%	36.6%	44.3%	53.8%	57.1%	32.5%
DeepSeek-Coder-Base-v1.5	6.9B	43.2%	60.4%	62.4%	24.7%	49.1%	55.2%	69.9%	63.8%	47.2%
DeepSeek-Coder-Instruct	6.7B	66.1%	65.4%	62.8%	28.6%	37.2%	46.9%	55.0%	57.6%	37.4%
DeepSeek-Coder-Instruct-v1.5	6.9B	64.1%	64.6%	72.6%	34.1%	49.5%	53.3%	72.2%	63.4%	48.1%

表 10 | DeepSeek-Coder-Base 与 DeepSeek-Coder-Base-v1.5 之间的性能对比分析。数学任务通过编程求解。

6 结论

在本技术报告中，我们介绍了一系列专用于编码任务的大语言模型（LLM），名为 DeepSeek-Coder，提供 1.3B、6.7B 和 33B 三种不同规模。这些模型在一个经过精心筛选的项目级代码语料上进行独特训练，采用“填空”（fill-in-the-blank）式的预训练目标以增强代码填充能力。一项重要的进展是将模型的上下文窗口扩展到 16,384 个 token，从而大幅提升其处理大规模代码生成任务的效果。我们的评估显示，该系列中最先进的模型 DeepSeek-Coder-Base 33B 在多种标准测试上超越了现有开源代码模型。令人印象深刻的是，DeepSeek-Coder-Base 6.7B 模型尽管规模更小，其性能却与 340 亿参数的 CodeLlama 相当，这印证了我们预训练语料的高质量。

为增强 DeepSeek-Coder-Base 模型的零样本指令能力，我们使用高质量的指令数据对其进行了微调。由此得到的 DeepSeek-Coder-Instruct 33B 模型在一系列编码相关任务上优于 OpenAI 的 GPT-3.5 Turbo，展现出其在代码生成与理解方面卓越的能力。

为进一步提升 DeepSeek-Coder-Base 模型的自然语言理解能力，我们在 DeepSeek-LLM 7B checkpoint 的基础上进行了额外的预训练。这一额外训练使用了包含自然语言、代码与数学数据在内的多样化数据集，共处理了 20 亿个 token。由此产生了新的……

附录

A.1 与 DeepSeek-Coder-Instruct 对话的案例

A.2 DeepSeek-Coder-Base 训练期间的基准曲线

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] and improved code model, DeepSeek-Coder-v1.5. Our observations indicate that DeepSeek- Coder-v1.5 not only maintains its predecessor’s high-level coding performance but also exhibits enhanced natural language comprehension. This advancement underscores our belief that the most effective code-focused Large Language Models (LLMs) are those built upon robust general LLMs. The reason is evident: to effectively interpret and execute coding tasks, these models must also possess a deep understanding of human instructions, which often come in various forms of natural language. Looking ahead, our commitment is to develop and openly share even more powerful code-focused LLMs based on larger-scale general LLMs.

[2] We would like to express our gratitude to Bo Liu, Chengqi Deng, Chong Ruan, Damai Dai, Jiashi Li, Kang Guan, Mingchuan Zhang, Panpan Huang, Shuiping Yu, Shirong Ma, Yaofeng Sun, Yishi Piao, Zhihong Shao, and Zhewen Hao for their invaluable discussions and assistance during training DeepSeek-Coder models.

[3] L. B. Allal, R. Li, D. Kocetkov, C. Mou, C. Akiki, C. M. Ferrandis, N. Muennighoff, M. Mishra, A. Gu, M. Dey, et al. Santacoder: don’t reach for the stars! arXiv preprint arXiv:2301.03988, 2023.

[4] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton. Program synthesis with large language models, 2021.

[5] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen. Efficient training of language models to fill in the middle. arXiv preprint arXiv:2207.14255, 2022.

[6] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman, et al. Multipl-e: a scalable and polyglot approach to benchmarking neural code generation. IEEE Transactions on Software Engineering, 2023.

[7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

[8] S. Chen, S. Wong, L. Chen, and Y. Tian. Extending context window of large language models via positional interpolation. arXiv preprint arXiv:2306.15595, 2023.

[9] P . Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv preprint arXiv:1803.05457, 2018.

[10] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[11] T. Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.

[12] DeepSeek-AI. Deepseek llm: Scaling open-source language models with longtermism. arXiv preprint arXiv:2401.02954, 2024. Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Jain, M. K. Ramanathan, R. Nallapati, P . Bhatia, D. Roth, et al. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. In Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2023.

[13] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang. Glm: General language model pretraining with autoregressive blank infilling. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 320–335, 2022.

[14] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, and M. Lewis. Incoder: A generative model for code infilling and synthesis. arXiv preprint arXiv:2204.05999, 2022.

[15] L. Gao, A. Madaan, S. Zhou, U. Alon, P . Liu, Y. Yang, J. Callan, and G. Neubig. Pal: Programaided language models. In International Conference on Machine Learning, pages 10764– 10799. PMLR, 2023.

[16] G. Gemini Team. Gemini: A family of highly capable multimodal models, 2023. URL https: //goo.gle/GeminiPaper .

[17] Z. Gou, Z. Shao, Y. Gong, Y. Yang, M. Huang, N. Duan, W. Chen, et al. Tora: A tool-integrated reasoning agent for mathematical problem solving. arXiv preprint arXiv:2309.17452, 2023.

[18] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In International Conference on Learning Representations, 2020.

[19] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[20] High-Flyer. Hai-llm: An efficient and lightweight tool for training large models. 2023. URL https://www.high-flyer.cn/en/blog/hai-llm .

[21] kaiokendev. Things i’m learning while training superhot. https://kaiokendev.github.io /til#extending-context-to-8k , 2023.

[22] D. Kocetkov, R. Li, L. Jia, C. Mou, Y. Jernite, M. Mitchell, C. M. Ferrandis, S. Hughes, T. Wolf, D. Bahdanau, et al. The stack: 3 tb of permissively licensed source code. Transactions on Machine Learning Research, 2022.

[23] V . A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro. Reducing activation recomputation in large transformer models. Proceedings of Machine Learning and Systems, 5, 2023.

[24] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, W.-t. Yih, D. Fried, S. Wang, and T. Yu. Ds-1000: A natural and reliable benchmark for data science code generation. In International Conference on Machine Learning, pages 18319–18345. PMLR, 2023.

[25] K. Lee, D. Ippolito, A. Nystrom, C. Zhang, D. Eck, C. Callison-Burch, and N. Carlini. Deduplicating training data makes language models better. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 8424–8445, 2022. R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, et al. Starcoder: may the source be with you! arXiv preprint arXiv:2305.06161, 2023.

[26] I. Loshchilov and F. Hutter. Decoupled weight decay regularization, 2019.

[27] P . Lu, L. Qiu, K.-W. Chang, Y. N. Wu, S.-C. Zhu, T. Rajpurohit, P . Clark, and A. Kalyan. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. In The Eleventh International Conference on Learning Representations, 2022.

[28] S.-Y. Miao, C.-C. Liang, and K.-Y. Su. A diverse corpus for evaluating and developing english math word problem solvers. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 975–984, 2020.

[29] D. Narayanan, A. Harlap, A. Phanishayee, V . Seshadri, N. R. Devanur, G. R. Ganger, P . B. Gibbons, and M. Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In Proceedings of the 27th ACM Symposium on Operating Systems Principles, pages 1–15, 2019.

[30] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong. Codegen: An open large language model for code with multi-turn program synthesis. arXiv preprint arXiv:2203.13474, 2022.

[31] E. Nijkamp, H. Hayashi, C. Xiong, S. Savarese, and Y. Zhou. Codegen2: Lessons for training llms on programming and natural languages, 2023.

[32] A. Patel, S. Bhattamishra, and N. Goyal. Are nlp models really able to solve simple math word problems? In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pages 2080– 2094, 2021.

[33] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P . J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023.

[34] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.

[35] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al. Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950, 2023.

[36] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale. Communications of the ACM, 64(9):99–106, 2021.

[37] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909, 2015.

[38] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.

[39] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, , and J. Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022. R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P . Liang, and T. B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab /stanford_alpaca , 2023.

[40] H. Touvron, L. Martin, K. Stone, P . Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P . Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023.

[41] Y. Wang, W. Wang, S. Joty, and S. C. Hoi. Codet5: Identifier-aware unified pre-trained encoderdecoder models for code understanding and generation. arXiv preprint arXiv:2109.00859, 2021.

[42] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. Hellaswag: Can a machine really finish your sentence? In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pages 4791–4800, 2019.

[43] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, L. Shen, Z. Wang, A. Wang, Y. Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pages 5673–5684, 2023.

DeepSeekMath：突破开源语言模型数学推理的极限

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models Zhihong Shao1,2 ∗†, Peiyi Wang1,3 ∗†, Qihao Zhu1,3 ∗†, Runxin Xu1, Junxiao Song1

DeepSeek-AI research@deepseek.com

arXiv:2402.03300 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

由于数学推理具有复杂且结构化的特性，它对语言模型构成了重大挑战。本文提出 DeepSeekMath 7B，它在 DeepSeek-Coder-Base-v1.5 7B 的基础上继续预训练，使用了从 Common Crawl 中获取的 120B 数学相关 token，以及自然语言与代码数据。在不依赖外部工具包与投票技术的情况下，DeepSeekMath 7B 在竞赛级 MATH 基准上取得了 51.7% 的优异成绩，接近 Gemini-Ultra 与 GPT-4 的水平。对 DeepSeekMath 7B 的 64 个样本应用自洽性（self-consistency）解码，可在 MATH 上达到 60.9%。DeepSeekMath 的数学推理能力归功于两个关键因素：第一，我们通过精心设计的数据筛选流程，充分挖掘了公开网络数据的巨大潜力；第二，我们提出了组相对策略优化（GRPO），它是近端策略优化（PPO）的一种变体，在提升数学推理能力的同时优化了 PPO 的内存占用。

1 引言

大语言模型（LLM）彻底改变了人工智能中进行数学推理的途径，推动了定量推理基准（Hendrycks 等，2021）与几何推理基准（Trinh 等，2024）的显著进步。此外，这些模型已被证明在辅助人类求解复杂数学问题方面卓有成效（Tao，2023）。然而，GPT-4（OpenAI，2023）与 Gemini-Ultra（Anil 等，2023）等前沿模型并未公开可用，而目前可获取的开源模型在性能上明显落后。

在本研究中，我们提出 DeepSeekMath——一个领域专用的语言模型，其数学能力显著优于开源模型，并在学术基准上接近 GPT-4 的水平。为此，我们构建了 DeepSeekMath Corpus，一个包含 120B 数学 token 的大规模高质量预训练语料。该数据集使用基于 fastText 的分类器（Joulin 等，2016）从 Common Crawl（CC）中提取。在初始迭代中，分类器以 OpenWebMath（Paster 等，2023）中的实例作为正例训练，同时纳入多样化的其他网页作为负例。随后，我们使用该分类器从 CC 中挖掘更多正例，并通过人工标注进一步提炼这些正例。然后用这一增强后的数据集更新分类器，以提升其性能。评测结果表明，该大规模语料质量很高：我们的基座模型 DeepSeekMath-Base 7B 在 GSM8K（Cobbe 等，2021）上取得 64.2%、在竞赛级 MATH 数据集（Hendrycks 等，2021）上取得 36.2%，优于 Minerva 540B（Lewkowycz 等，2022a）。此外，DeepSeekMath Corpus 是多语言的，因此我们注意到中文数学基准（Wei 等，2023；Zhong 等，2023）上的提升。我们相信，我们在数学数据处理方面的经验可作为研究界的一个起点，未来仍有很大的改进空间。

DeepSeekMath-Base 以 DeepSeek-Coder-Base-v1.5 7B（Guo 等，2024）初始化，因为我们注意到，与通用 LLM 相比，从代码训练模型出发是更好的选择。此外，我们观察到数学训练还提升了模型在 MMLU（Hendrycks 等，2020）与 BBH 基准（Suzgun 等，2022）上的能力，这表明它不仅增强了模型的数学能力，也放大了通用推理能力。

在预训练之后，我们使用思维链（Wei 等，2022）、程序思维（program-of-thought）（Chen 等，2022；Gao 等，2023）与工具集成推理（tool-integrated reasoning）（Gou 等，2023）数据，对 DeepSeekMath-Base 进行数学指令微调。所得模型 DeepSeekMath-Instruct 7B 超越了所有 7B 同类模型，并可与 70B 开源指令微调模型相媲美。

此外，我们提出组相对策略优化（Group Relative Policy Optimization, GRPO），这是一种近端策略优化（PPO）（Schulman 等，2017）的变体强化学习（RL）算法。GRPO 舍弃了评论家（critic）模型，转而从组内得分估计基线，显著降低了训练资源需求。仅使用一部分英文指令微调数据，GRPO 就在强大的 DeepSeekMath-Instruct 基础上取得了大幅提升，包括强化学习阶段的域内（GSM8K：82.9%→88.2%，MATH：46.8%→51.7%）与域外数学任务（例如 CMATH：84.6%→88.8%）。我们还提供了一个统一范式，以理解拒绝采样微调（Rejection Sampling Fine-Tuning, RFT）（Yuan 等，2023a）、直接偏好优化（DPO）（Rafailov 等，2023）、PPO 与 GRPO 等不同方法。基于这一统一范式，我们发现这些方法在概念上均可视为直接或简化的 RL 技术。我们还开展了大量实验，例如在线与离线训练、结果监督与过程监督、单轮与迭代式 RL 等，以深入考察这一范式的本质要素。最后，

我们解释了 RL 为何能提升指令微调模型的性能，并基于该统一范式总结了实现更有效 RL 的潜在方向。

1.1 贡献

我们的贡献包括可扩展的数学预训练，以及对强化学习的探索与分析。

大规模数学预训练

- 我们的研究提供了有力证据，表明公开可获取的 Common Crawl 数据包含对数学有价值的信息。通过实施精心设计的数据选择流程，我们成功构建了 DeepSeekMath Corpus——一个由经数学内容过滤的网页组成的高质量数据集，规模达 120B token，约为 Minerva（Lewkowycz 等，2022a）所用数学网页规模的 7 倍，也是近期发布的 OpenWebMath（Paster 等，2023）的 9 倍。
- 我们的预训练基座模型 DeepSeekMath-Base 7B 取得了与 Minerva 540B（Lewkowycz 等，2022a）相当的性能，表明参数量并非数学推理能力的唯一关键因素。一个在高质量数据上预训练的较小模型同样可以取得强劲性能。
- 我们分享了数学训练实验中的发现：在数学训练之前进行代码训练，能够提升模型在有工具与无工具使用情况下求解数学问题的能力。这为长期存在的问题——代码训练能否提升推理能力——提供了部分答案。我们认为答案是肯定的，至少对数学推理而言如此。
- 尽管在 arXiv 论文上训练很常见，尤其是在许多与数学相关的论文中，但这对本文采用的所有数学基准都没有带来显著提升。

强化学习的探索与分析

- 我们提出组相对策略优化（GRPO），一种高效且有效的强化学习算法。GRPO 舍弃评论家模型，转而从组内得分估计基线，相比近端策略优化（PPO）显著降低了训练资源。
- 我们证明，仅使用指令微调数据，GRPO 就能显著提升我们指令微调模型 DeepSeekMath-Instruct 的性能。此外，我们观察到强化学习过程中域外性能也有所提升。
- 我们提供一个统一范式来理解 RFT、DPO、PPO 与 GRPO 等不同方法。我们还开展了大量实验，例如在线与离线训练、结果监督与过程监督、单轮与迭代式强化学习等，以深入考察该范式的本质要素。
- 基于我们的统一范式，我们探索了强化学习有效性的原因，并总结了实现 LLM 更有效强化学习的若干潜在方向。

1.2 评测与指标概览

- **英文与中文数学推理**：我们在英文与中文基准上对模型进行全面评估，涵盖从小学到大学水平的数学问题。英文基准包括 GSM8K（Cobbe 等，2021）、MATH（Hendrycks 等，2021）、SAT（Azerbayev 等，2023）、OCW Courses（Lewkowycz 等，2022a）、MMLU-STEM（Hendrycks 等，2020）。中文基准包括 MGSM-zh（Shi 等，2023）、CMATH（Wei 等，2023）、Gaokao-MathCloze（Zhong 等，2023）与 Gaokao-MathQA（Zhong 等，2023）。我们评估模型在不使用工具的情况下生成自包含文本解答的能力，以及使用 Python 解题的能力。

在英文基准上，DeepSeekMath-Base 可与闭源的 Minerva 540B（Lewkowycz 等，2022a）相竞争，并超越所有开源基座模型（例如 Mistral 7B（Jiang 等，2023）与 Llemma-34B（Azerbayev 等，2023）），无论这些模型是否经过数学预训练，且优势往往十分显著。值得注意的是，DeepSeekMath-Base 在中文基准上表现更优，这可能是因为我们没有像以往工作（Azerbayev 等，2023；Lewkowycz 等，2022a）那样只收集纯英文数学预训练数据，而是也纳入了高质量的非英文数据。借助数学指令微调与强化学习，所得 DeepSeekMath-Instruct 与 DeepSeekMath-RL 展现出强劲性能，在开源社区中首次于竞赛级 MATH 数据集上取得超过 50% 的准确率。

- **形式化数学**：我们采用 Jiang 等（2022）提出的非形式到形式定理证明任务，在 miniF2F（Zheng 等，2021）上评估 DeepSeekMath-Base，并选择 Isabelle（Wenzel 等，2008）作为证明助手。DeepSeekMath-Base 展现出强大的少样本自动形式化性能。
- **自然语言理解、推理与代码**：为全面刻画模型在通用理解、推理与编码方面的能力，我们在以下基准上评估 DeepSeekMath-Base：涵盖 57 项选择题任务、涉及多学科的大规模多任务语言理解（MMLU）基准（Hendrycks 等，2020）；包含 23 项颇具挑战性、大多需要多步推理才能解决的任务的 BIG-Bench Hard（BBH）（Suzgun 等，2022）；以及广泛用于评测代码语言模型的 HumanEval（Chen 等，2021）与 MBPP（Austin 等，2021）。数学预训练同时有益于语言理解与推理性能。

2 数学预训练

2.1 数据收集与去污染

本节将概述从 Common Crawl 构建 DeepSeekMath Corpus 的过程。如图 2 所示，我们给出一个迭代式流程，展示如何从 Common Crawl 出发，系统性地收集大规模数学语料，其起点是一个种子语料（例如一个规模较小但质量较高的数学相关数据集）。值得注意的是，该方法同样适用于其他领域，例如编程。

【图表】图 2 从 Common Crawl 收集数学网页的迭代流程（原文第 5 页）

首先，我们选择 OpenWebMath（Paster 等，2023）——一个高质量的数学网页文本集合——作为初始种子语料。利用该语料，我们训练一个 fastText 模型（Joulin 等，2016），以召回更多类似 OpenWebMath 的数学网页。具体而言，我们从种子语料中随机选取 500,000 个数据点作为正训练样本，并从 Common Crawl 中选取另外 500,000 个网页作为负样本。我们使用一个开源库（https://fasttext.cc）进行训练，配置向量维度为 256、学习率为 0.1、词 n-gram 的最大长度为 3、词的最小出现次数为 3、训练轮数为 3。为缩减原始 Common Crawl 的规模，我们采用基于 URL 的去重与近似去重技术，得到 40B 个 HTML 网页。随后，我们使用 fastText 模型从去重后的 Common Crawl 中召回数学网页。

为滤除低质量数学内容，我们根据 fastText 模型预测的分数对收集到的网页排序，仅保留排名靠前的网页。保留数据的规模通过对前 40B、80B、120B 与 160B token 进行预训练实验来评估。在第一次迭代中，我们选择保留前 40B token。

在第一次数据收集迭代之后，仍有大量数学网页未被收集，主要原因是 fastText 模型训练所用正例集合缺乏足够的多样性。因此，我们识别更多数学网络来源以丰富种子语料，从而优化 fastText 模型。具体而言，我们首先将整个 Common Crawl 组织为互不相交的域；域定义为共享同一基础 URL 的网页集合。对于每个域，我们计算第一次迭代中已收集网页的百分比。已收集网页占比超过 10% 的域被归类为数学相关域（例如 mathoverflow.net）。随后，我们人工标注这些已识别域中与数学内容相关的 URL（例如 mathoverflow.net/questions）。与这些 URL 链接但尚未被收集的网页将被加入种子语料。这一方法使我们能够收集更多正例，从而训练出改进的 fastText 模型，在后续迭代中召回更多

数学数据。经过四次数据收集迭代，我们最终得到 35.5M 个数学网页，总计 120B token。在第四次迭代中，我们注意到近 98% 的数据在第三次迭代中已被收集，因此决定停止数据收集。

为避免基准污染，我们遵循 Guo 等（2024）的做法，滤除包含英文数学基准（如 GSM8K（Cobbe 等，2021）与 MATH（Hendrycks 等，2021））以及中文基准（如 CMATH（Wei 等，2023）与 AGIEval（Zhong 等，2023））题目或答案的网页。过滤标准如下：任何文本片段若包含与评测基准中任一子串完全匹配的 10-gram 字符串，则从数学训练语料中移除；对于长度短于 10-gram 但至少为 3-gram 的基准文本，我们采用精确匹配来滤除受污染的网页。

2.2 验证 DeepSeekMath 语料的质量

我们开展预训练实验，以考察 DeepSeekMath Corpus 与近期发布的数学训练语料相比如何：

- MathPile（Wang 等，2023c）：一个多源语料（8.9B token），汇集自教材、Wikipedia、ProofWiki、CommonCrawl、StackExchange 与 arXiv，其中大部分（超过 85%）来自 arXiv；
- OpenWebMath（Paster 等，2023）：经数学内容过滤的 CommonCrawl 数据，总计 13.6B token；
- Proof-Pile-2（Azerbayev 等，2023）：由 OpenWebMath、AlgebraicStack（10.3B token 的数学代码）与 arXiv 论文（28.0B token）构成的数学语料。在 Proof-Pile-2 上实验时，我们遵循 Azerbayev 等（2023）的做法，采用 arXiv:Web:Code = 2:4:1 的比例。

2.2.1 训练设置

我们将数学训练应用于一个具有 1.3B 参数的通用预训练语言模型，该模型与 DeepSeek LLM 系列（DeepSeek-AI，2024）采用相同框架，记为 DeepSeek-LLM 1.3B。我们分别在每个数学语料上训练模型 150B token。所有实验均使用高效轻量的 HAI-LLM（High-flyer，2023）训练框架。遵循 DeepSeek LLM 的训练实践，我们使用 AdamW 优化器（Loshchilov 与 Hutter，2017），$\beta_1 = 0.9$、$\beta_2 = 0.95$、weight_decay = 0.1，并采用多步学习率调度：学习率在 2,000 个预热步后达到峰值，在训练进程达到 80% 时降至峰值的 31.6%，在训练进程达到 90% 后进一步降至峰值的 10.0%。我们将学习率的最大值设为 5.3e-4，并使用 4M token 的批大小与 4K 的上下文长度。

【表格】表 1 在不同数学语料上训练的 DeepSeek-LLM 1.3B 的性能（以少样本思维链提示评测）；语料规模使用词表大小为 100K 的分词器计算（原文第 6 页）

2.2.2 评测结果

DeepSeekMath Corpus 质量高、覆盖多语言数学内容，且规模最大。

- **高质量**：我们使用少样本思维链提示（Wei 等，2022）在 8 个数学基准上评估下游性能。如表 1 所示，在 DeepSeekMath Corpus 上训练的模型具有明显的性能领先。图 3 显示，在 DeepSeekMath Corpus 上训练的模型在 50B token（即 Proof-Pile-2 的 1 个完整 epoch）时即优于在 Proof-Pile-2 上训练的模型，表明 DeepSeekMath Corpus 的平均质量更高。

【图表】图 3 在不同数学语料上训练的 DeepSeek-LLM 1.3B 的基准曲线（原文第 7 页）

- **多语言**：DeepSeekMath Corpus 涵盖多种语言的数据，其中英文与中文是占比最高的两种语言。如表 1 所示，在 DeepSeekMath Corpus 上训练提升了英文与中文的数学推理性能。相比之下，现有以英文为主的数学语料在中文数学推理上提升有限，甚至可能造成损害。
- **大规模**：DeepSeekMath Corpus 的规模是现有数学语料的数倍。如图 3 所示，DeepSeek-LLM 1.3B 在 DeepSeekMath Corpus 上训练时，学习曲线更陡，改进也更为持久。相比之下，基线语料规模小得多，且在训练中已被重复多轮，模型性能很快趋于平台。

2.3 DeepSeekMath-Base 7B 的训练与评测

本节介绍 DeepSeekMath-Base 7B——一个具有强推理能力（尤其在数学方面）的基座模型。我们的模型以 DeepSeek-Coder-Base-v1.5 7B（Guo 等，2024）初始化，并训练 500B token。数据分布如下：56% 来自 DeepSeekMath Corpus，4% 来自 AlgebraicStack，10% 来自 arXiv，20% 为 Github 代码，其余 10% 为来自 Common Crawl 的中英文自然语言数据。我们主要采用第 2.2.1 节所述的训练设置，只是将学习率的最大值设为 4.2e-4，并使用 10M token 的批大小。

我们对 DeepSeekMath-Base 7B 的数学能力进行全面评估，重点关注其在不依赖外部工具的情况下生成自包含数学解答、使用工具求解数学问题以及开展形式化定理证明的能力。在数学之外，我们还给出该基座模型更通用的能力画像，包括其在自然语言理解、推理与编程技能方面的表现。

基于逐步推理的数学问题求解 我们使用少样本思维链提示（Wei 等，2022），在 8 个英文与中文基准上评估 DeepSeekMath-Base 求解数学问题的表现。这些基准涵盖定量推理（例如 GSM8K（Cobbe 等，2021）、MATH（Hendrycks 等，2021）与 CMATH（Wei 等，2023））与选择题（例如 MMLU-STEM（Hendrycks 等，2020）与 Gaokao-MathQA（Zhong 等，2023）），覆盖从初等数学到大学水平复杂度的多个数学领域。

如表 2 所示，在开源基座模型中，DeepSeekMath-Base 7B 在全部 8 个基准上均领先（对比对象包括广泛使用的通用模型 Mistral 7B（Jiang 等，2023）与近期发布的 Llemma 34B（Azerbayev 等，2023），后者在 Proof-Pile-2（Azerbayev 等，2023）上接受了数学训练）。值得注意的是，在竞赛级 MATH 数据集上，DeepSeekMath-Base 以超过 10% 的绝对优势超越现有开源基座模型，并优于 Minerva 540B（Lewkowycz 等，2022a）——这是一个参数量为其 77 倍、基于 PaLM（Lewkowycz 等，2022b）并进一步在数学文本上训练的闭源基座模型。

【表格】表 2 DeepSeekMath-Base 7B 与强基座模型在英文与中文数学基准上的对比；模型以思维链提示评测，Minerva 结果引自 Lewkowycz 等（2022a）（原文第 8 页）

使用工具的数学问题求解：我们使用少样本程序思维链（program-of-thought）提示（Chen 等，2022；Gao 等，2023）在 GSM8K 与 MATH 上评估程序辅助的数学推理。我们提示模型为每道题编写一个 Python 程序，其中可使用 math、sympy 等库完成复杂计算，程序的执行结果被作为答案进行评估。如表 3 所示，DeepSeekMath-Base 7B 优于此前最先进的 Llemma 34B。

【表格】表 3 基座模型使用工具求解数学问题以及在 Isabelle 中进行非形式化到形式化定理证明能力的少样本评估（原文第 9 页）

形式数学：形式化证明自动化有助于确保数学证明的准确性与可靠性并提升效率，近年来受到越来越多的关注。我们在 Jiang 等（2022）提出的非形式化到形式化证明任务上评估 DeepSeekMath-Base 7B，该任务要求根据非形式化陈述、该陈述的形式化对应物以及非形式化证明来生成形式化证明。我们在 miniF2F（Zheng 等，2021）上评估，这是一个面向奥赛级数学的形式化基准；我们通过少样本提示为每道题生成 Isabelle 形式化证明。遵循 Jiang 等（2022），我们利用模型生成证明草图，并执行现成的自动证明器 Sledgehammer（Paulson，2010）来补全缺失的细节。如表 3 所示，DeepSeekMath-Base 7B 在证明自动形式化方面展现出强劲的性能。

自然语言理解、推理与代码：我们在 MMLU（Hendrycks 等，2020）上评估模型的自然语言理解性能，在 BBH（Suzgun 等，2022）上评估推理性能，在 HumanEval（Chen 等，2021）与 MBPP（Austin 等，2021）上评估编程能力。如表 4 所示，DeepSeekMath-Base 7B 在 MMLU 与 BBH 上相较其前身 DeepSeek-Coder-Base-v1.5（Guo 等，2024）取得了显著提升，说明数学训练对语言理解与推理具有积极影响。此外，通过在持续训练中加入代码 token，DeepSeekMath-Base 7B 有效保持了

DeepSeek-Coder-Base-v1.5 在两个编码基准上的性能。总体而言，DeepSeekMath-Base 7B 在三个推理与编码基准上显著优于通用模型 Mistral 7B（Jiang 等，2023）。

表 4：自然语言理解、推理与代码基准上的评估。DeepSeek-Coder-Base-v1.5† 是学习率衰减前的检查点，用于训练 DeepSeekMath-Base。在 MMLU 与 BBH 上，我们使用少样本思维链提示；在 HumanEval 与 MBPP 上，我们分别在零样本与少样本设置下评估模型性能。（原文第 9 页）

模型	规模	MMLU	BBH	HumanEval (Pass@1)	MBPP (Pass@1)
Mistral	7B	62.4%	55.7%	28.0%	41.4%
DeepSeek-Coder-Base-v1.5†	7B	42.9%	42.9%	40.2%	52.6%
DeepSeek-Coder-Base-v1.5	7B	49.1%	55.2%	43.2%	60.4%
DeepSeekMath-Base	7B	54.9%	59.5%	40.9%	52.6%

3 监督微调

3.1 SFT 数据构建

我们构建了一个数学指令微调数据集，覆盖来自不同数学领域、复杂度各异的中英文问题：每个问题都配有思维链（CoT）（Wei 等，2022）、程序思维链（PoT）（Chen 等，2022；Gao 等，2023）与工具集成推理格式（Gou 等，2023）的解答。训练样本总数为 776K。

- **英文数学数据集**：我们为 GSM8K 与 MATH 问题标注工具集成解答，并采用 MathInstruct（Yue 等，2023）的一个子集以及 Lila-OOD（Mishra 等，2022）的训练集，其中问题以 CoT 或 PoT 求解。我们的英文集合覆盖代数、概率、数论、微积分与几何等多个数学领域。
- **中文数学数据集**：我们收集了涵盖 76 个子主题（如线性方程）的中文 K-12 数学问题，并同时以 CoT 与工具集成推理格式标注解答。

3.2 DeepSeekMath-Instruct 7B 的训练与评估

本节介绍 DeepSeekMath-Instruct 7B，它在 DeepSeekMath-Base 的基础上进行了数学指令微调。训练样本被随机拼接，直至达到 4K token 的最大上下文长度。我们以 256 的批大小、5e-5 的恒定学习率训练模型 500 步。

我们在英文与中文的 4 个定量推理基准上，评估模型在无工具使用与有工具使用两种情况下的数学性能。我们将模型与当时的领先模型进行比较：

- **闭源模型**包括：（1）GPT 系列，其中 GPT-4（OpenAI，2023）与 GPT-4 Code Interpreter（https://openai.com/blog/chatgpt-plugins#code-interpreter）能力最强；（2）Gemini Ultra 与 Pro（Anil 等，2023）；（3）Inflection-2（Inflection AI，2023）；（4）Grok-1（https://x.ai/model-card）；以及中国公司近期发布的模型，包括（5）Baichuan-3（https://www.baichuan-ai.com）与（6）GLM 系列最新的 GLM-4（https://open.bigmodel.cn/dev/api#glm-4）（Du 等，2022）。这些模型面向通用目的，其中大多数经过了一系列对齐流程。
- **开源模型**包括：通用模型，如（1）DeepSeek-LLM-Chat 67B（DeepSeek-AI，2024）、（2）Qwen 72B（Bai 等，2023）、（3）SeaLLM-v2 7B（Nguyen 等，2023）与（4）ChatGLM3 6B（ChatGLM3 Team，2023）；以及经数学增强的模型，包括（5）InternLM2-Math 20B（https://github.com/InternLM/InternLM-Math），它基于 InternLM2 构建，先进行数学训练，随后进行指令微调；（6）Math-Shepherd-Mistral 7B，它使用过程监督奖励模型对 Mistral 7B（Jiang 等，2023）应用 PPO 训练（Schulman 等，2017）；（7）WizardMath 系列（Luo 等，2023），它使用 evolve-instruct（即一种使用 AI 进化指令的指令微调版本）以及以 GSM8K 与 MATH 为主要训练问题的 PPO 训练，来提升 Mistral 7B 与 Llama-2 70B（Touvron 等，2023）的数学推理能力；（8）MetaMath 70B（Yu 等，2023），它是在 GSM8K 与 MATH 增强版本上微调的 Llama-2 70B；（9）ToRA 34B（Gou 等，2023），它是经微调以执行工具集成数学推理的 CodeLlama 34B；（10）MAmmoTH 70B（Yue 等，2023），它是在 MathInstruct 上进行指令微调的 Llama-2 70B。

如表 5 所示，在不允许使用工具的评估设置下，DeepSeekMath-Instruct 7B 展现出强大的逐步推理性能。值得注意的是，在竞赛级 MATH 数据集上，我们的模型以至少 9% 的绝对优势超过了所有开源模型以及大多数专有模型（如 Inflection-2 与 Gemini Pro）。即使对于规模大得多的模型（如 Qwen 72B）或经过专门数学强化学习增强的模型（如 WizardMath-v1.1 7B），这一结论同样成立。尽管

DeepSeekMath-Instruct 在 MATH 上与中文专有模型 GLM-4 和 Baichuan-3 不相上下，但其性能仍落后于 GPT-4 与 Gemini Ultra。

在允许模型将自然语言推理与基于程序的工具使用相结合来求解问题的评估设置下，DeepSeekMath-Instruct 7B 在 MATH 上的准确率接近 60%，超过了所有现有开源模型。在其他基准上，我们的模型与规模大 10 倍的前最先进模型 DeepSeek-LLM-Chat 67B 具有竞争力。

【表格】表 5 开源与闭源模型在英文与中文基准上使用思维链与工具集成推理的性能。灰色分数表示 32 个候选项的多数投票结果，其余为 Top1 分数。DeepSeekMath-RL 7B 击败了 7B 至 70B 的所有开源模型以及大多数闭源模型。尽管 DeepSeekMath-RL 7B 仅在 GSM8K 与 MATH 的思维链格式指令微调数据上进一步训练，它在所有基准上均优于 DeepSeekMath-Instruct 7B（原文第 12 页）

4 强化学习

4.1 组相对策略优化（GRPO）

强化学习（RL）已被证明能够在监督微调（SFT）阶段之后进一步提升 LLM 的数学推理能力（Luo 等，2023；Wang 等，2023b）。本节介绍我们高效且有效的 RL 算法——组相对策略优化（GRPO）。

4.1.1 从 PPO 到 GRPO

近端策略优化（PPO）（Schulman 等，2017）是一种演员-评论家（actor-critic）RL 算法，广泛用于 LLM 的 RL 微调阶段（Ouyang 等，2022）。具体而言，它通过最大化如下替代目标来优化 LLM：

J_PPO(θ) = E_{q~P(Q), o~π_θold(O|q)} [(1/|o|) Σ_{t=1}^{|o|} min((π_θ(o_t|q,o_<t) / π_θold(o_t|q,o_<t))·A_t, clip(π_θ(o_t|q,o_<t) / π_θold(o_t|q,o_<t), 1−ε, 1+ε)·A_t)]　　（1）

其中 π_θ 与 π_θold 分别为当前策略模型与旧策略模型；q 与 o 分别为从问题数据集和旧策略 π_θold 中采样得到的问题与输出；ε 是 PPO 中为稳定训练而引入的裁剪相关超参数；A_t 是优势值，它通过在奖励 {r_≥t} 与学习到的价值函数 V_ψ 上应用广义优势估计（GAE）（Schulman 等，2015）计算得到。

【图表】图 4 PPO 与本文 GRPO 的示意图。GRPO 舍弃了价值模型，转而从组内得分估计基线，显著减少训练资源

消耗（原文第 13 页）

因此，在 PPO 中，需要将价值函数与策略模型一同训练；为缓解对奖励模型的过度优化，标准做法是在每个 token 的奖励中加入来自参考模型的逐 token KL 惩罚（Ouyang 等，2022），即：

$$r_t = r_\varphi(q, o_{\le t}) - \beta \cdot \log\left(\pi_\theta(o_t|q,o_{<t}) / \pi_{ref}(o_t|q,o_{<t}) \right) \quad (2)$$

其中 r_φ 为奖励模型，π_{ref} 为参考模型（通常为初始 SFT 模型），β 为 KL 惩罚的系数。

由于 PPO 中使用的价值函数通常是另一个与策略模型规模相当的模型，这会带来显著的内存与计算负担。此外，在 RL 训练过程中，价值函数被用作基线来计算优势值，以降低方差；而在 LLM 场景中，奖励模型通常只为最后一个 token 赋予奖励分数，这会使训练一个在每个 token 上都准确的价值函数变得复杂。为解决这一问题，如图 4 所示，我们提出组相对策略优化（GRPO）：它无需像 PPO 那样进行额外的价值函数近似，而是使用针对同一问题采样得到的多个输出的平均奖励作为基线。更具体地说，对于每个问题 q，GRPO 从旧策略 $\pi_{\theta old}$ 中采样一组输出 $\{o_1, o_2, \ldots, o_G\}$，然后通过最大化如下目标来优化策略模型：

$$J_{GRPO}(\theta) = \mathbb{E}_{q\sim P(Q),\ \{o_i\}_{i=1}^{G}\sim\pi_{\theta old}(O|q)} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta old}(o_{i,t}|q,o_{i,<t})} \hat{A}_{i,t},\ \text{clip}\left(\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\theta old}(o_{i,t}|q,o_{i,<t})}, 1-\varepsilon, 1+\varepsilon \right) \hat{A}_{i,t} \right) - \beta \cdot D_{KL}[\pi_\theta \| \pi_{ref}] \right) \right] \quad (3)$$

其中 ε 与 β 为超参数，$\hat{A}_{i,t}$ 是仅基于组内各输出的相对奖励计算得到的优势值，将在后续小节中详述。GRPO 用来计算优势值的组相对方式与奖励模型的比较性质高度契合，因为奖励模型通常在同题输出之间的比较数据集上训练。还需注意，GRPO 并没有在奖励中加入 KL 惩罚，而是通过将训练策略与参考策略之间的 KL 散度直接加入损失来进行正则化，从而避免使 $\hat{A}_{i,t}$ 的计算复杂化。

算法 1：迭代式组相对策略优化（Iterative Group Relative Policy Optimization）

- **输入**：初始策略模型 $\pi_{\theta init}$；奖励模型 r_φ；任务提示集 D；超参数 ε、β、μ
- **1**：策略模型 $\pi_\theta \leftarrow \pi_{\theta init}$
- **2**：for iteration = 1, …, I **do**
- **3**：参考模型 $\pi_{ref} \leftarrow \pi_\theta$
- **4**：for step = 1, …, M **do**
- **5**：从 D 中采样一个批次 D_b
- **6**：更新旧策略模型 $\pi_{\theta old} \leftarrow \pi_\theta$

- **7**：对每个问题 q ∈ D_b，从 π_θold(·|q) 采样 G 个输出 {o_i}（i = 1, …, G）
- **8**：通过运行 r_φ 为每个采样输出 o_i 计算奖励 {r_i}（i = 1, …, G）
- **9**：通过组相对优势估计计算 o_i 第 t 个 token 的 Â_{i,t}
- **10**：for GRPO iteration = 1, …, μ **do**
- **11**：通过最大化 GRPO 目标（式 21）更新策略模型 π_θ
- **12**：借助回放机制持续训练，更新 r_φ
- **输出**：π_θ

与式（2）中使用的 KL 惩罚项不同，我们使用如下无偏估计器（Schulman，2020）来估计 KL 散度：

D_KL[π_θ‖π_ref] = π_ref(o_i,t|q,o_i,<t) / π_θ(o_i,t|q,o_i,<t) − log(π_ref(o_i,t|q,o_i,<t) / π_θ(o_i,t|q,o_i,<t)) − 1 （4）

该估计值保证为正。

4.1.2 基于结果监督的 GRPO 强化学习

形式化地，对于每个问题 q，我们从旧策略模型 π_θold 中采样一组输出 {o_1, o_2, …, o_G}。随后使用奖励模型为这些输出打分，相应得到 G 个奖励 r = {r_1, r_2, …, r_G}。接着，这些奖励通过减去组均值并除以组标准差进行归一化。结果监督在每个输出 o_i 的末尾提供归一化奖励，并将该输出中所有 token 的优势值 Â_{i,t} 设为该归一化奖励，即 Â_{i,t} = r̃_i = (r_i − mean(r)) / std(r)，然后通过最大化式（3）定义的目标来优化策略。

4.1.3 基于过程监督的 GRPO 强化学习

结果监督仅在每个输出的末尾提供奖励，这可能不足以高效地监督复杂数学任务中的策略。遵循 Wang 等（2023b），我们还探索了过程监督，即在每个推理步骤的末尾提供奖励。形式化地，给定问题 q 与 G 个采样输出 {o_1, o_2, …, o_G}，使用过程奖励模型为输出的每个步骤打分，相应得到奖励 R = {{r_1^index(1), …, r_1^index(K_1)}, …, {r_G^index(1), …, r_G^index(K_G)}}，其中 index(j) 为第 j 步的结束 token 索引，K_i 为第 i 个输出中的总步数。我们同样使用均值与标准差对这些奖励进行归一化，即 r̃_i^index(j) = (r_i^index(j) − mean(R)) / std(R)。随后，过程监督将每个 token 的优势值计算为后续各步骤归一化奖励之和，即 Â_{i,t} = Σ_{index(j) ≥ t} r̃_i^index(j)，然后通过最大化式（3）定义的目标来优化策略。

4.1.4 基于 GRPO 的迭代式强化学习

随着强化学习训练过程的推进，旧的奖励模型可能不足以监督当前的策略模型。因此，我们还探索了基于 GRPO 的迭代式 RL。如算法 1 所示，在迭代 GRPO 中，我们基于策略模型的采样结果为奖励模型生成新的训练集，并使用一种回放机制（其中包含 10% 的历史数据）持续训练旧奖励模型。然后，我们将参考模型设为策略模型，并用新的奖励模型持续训练策略模型。

4.2 DeepSeekMath-RL 的训练与评估

我们在 DeepSeekMath-Instruct 7B 的基础上进行 RL。RL 的训练数据是 SFT 数据中与 GSM8K 和 MATH 相关的思维链格式问题，约包含 144K 道题。我们排除了其他 SFT 问题，以考察 RL 对在整个 RL 阶段都缺乏数据的基准的影响。我们遵循 Wang 等（2023b）构建奖励模型的训练集。我们基于 DeepSeekMath-Base 7B 训练初始奖励模型，学习率为 2e-5。对于 GRPO，我们将策略模型的学习率设为 1e-6，KL 系数为 0.04。对于每个问题，我们采样 64 个输出；最大长度设为 1024，训练批大小为 1024。在每个探索阶段之后，策略模型仅进行一次更新。我们按照评估 DeepSeekMath-Instruct 7B 的方式，在基准上评估 DeepSeekMath-RL 7B。对 DeepSeekMath-RL 7B 而言，采用思维链推理的 GSM8K 与 MATH 可视为域内任务，其他所有基准均可视为域外任务。

表 5 展示了开源与闭源模型在英文与中文基准上使用思维链与工具集成推理的性能。我们发现：1）DeepSeekMath-RL 7B 在使用思维链推理时，在 GSM8K 与 MATH 上分别取得 88.2% 与 51.7% 的准确率，超过了 7B 至 70B 范围内的所有开源模型以及大多数闭源模型；2）关键在于，DeepSeekMath-RL 7B 仅从 DeepSeekMath-Instruct 7B 出发、在 GSM8K 与 MATH 的思维链格式指令微调数据上训练，尽管训练数据范围受限，它在所有评估指标上均优于 DeepSeekMath-Instruct 7B，彰显了强化学习的有效性。

5 讨论

在本节中，我们将分享在预训练与 RL 实验中的发现。

5.1 预训练中的经验

我们首先分享预训练方面的经验。除非另有说明，我们将遵循第 2.2.1 节所述的训练设置。值得注意的是，本节中提及 DeepSeekMath Corpus 时，我们使用的是数据收集过程第二轮迭代得到的 89B token 数据集。

5.1.1 代码训练有益于数学推理

一个流行但尚未被验证的假设认为，代码训练能够提升推理能力。我们尝试对此给出部分回答，特别是在数学领域：代码训练，提升模型在使用工具与不使用工具两种情形下的数学推理能力。

为研究代码训练如何影响数学推理，我们尝试了以下两阶段训练与一阶段训练设置：

两阶段训练

- **先训练 400B token 的代码 → 再训练 150B token 的数学**：我们先让 DeepSeek-LLM 1.3B 训练 400B 个代码 token，随后训练 150B 个数学 token；
- **先训练 400B token 的通用数据 → 再训练 150B token 的数学**：作为对照实验，我们在第一阶段训练中改用通用 token（从 DeepSeek-AI 构建的大规模通用语料中采样）替代代码 token，以考察代码 token 相比通用 token 在提升数学推理方面的优势。

一阶段训练

- **训练 150B token 的数学**：我们让 DeepSeek-LLM 1.3B 训练 150B 个数学 token；
- **在 400B 代码 token 与 150B 数学 token 的混合数据上训练**：在代码训练之后进行数学训练会降低编码性能。我们考察：当代码 token 与数学 token 混合用于一阶段训练时，是否仍能提升数学推理能力，并同时缓解灾难性遗忘问题。

结果 表 6 与表 7 展示了不同训练设置下的下游性能。

【表格】表 6 不同训练设置下代码对数学推理的影响（原文第 16 页）

【表格】表 7 代码与数学训练的不同设置对模型语言理解、推理与编码性能的影响（原文第 17 页）

代码训练有益于程序辅助的数学推理，无论在两阶段训练还是一阶段训练设置下都是如此。如表 6 所示，在两阶段训练设置下，仅代码训练就已显著增强了使用 Python 求解 GSM8K 与 MATH 问题的能力；第二阶段的数学训练则带来进一步提升。有趣的是，在一阶段训练设置下，混合代码 token 与数学 token 有效缓解了两阶段训练所引发的灾难性遗忘问题，并使编码能力（表 7）与程序辅助的数学推理能力（表 6）产生协同效应。

代码训练同样提升了不使用工具情形下的数学推理能力。在两阶段训练设置下，初始阶段的代码训练即已带来中等程度的提升；它还提高了后续数学训练的效率，最终带来最佳性能。然而，将代码

token 与数学 token 混合用于一阶段训练，会损害不使用工具情形下的数学推理能力。一种推测是：由于规模有限，DeepSeek-LLM 1.3B 缺乏同时充分吸收代码数据与数学数据的能力。

5.1.2 ArXiv 论文似乎难以有效提升数学推理能力

arXiv 论文通常被作为数学预训练数据的一部分（Azerbayev 等，2023；Lewkowycz 等，2022a；Polu 和 Sutskever，2020；Wang 等，2023c）。然而，关于它们对数学推理影响的具体分析尚未得到广泛开展。或许与直觉相反，根据我们的实验，arXiv 论文似乎无法有效提升数学推理能力。我们使用经过不同处理流程的 arXiv 语料，对不同规模的模型进行了实验，包括 DeepSeek-LLM 1.3B 和 DeepSeek-Coder-Base-v1.5 7B（Guo 等，2024）：

- MathPile（Wang 等，2023c）：一个 8.9B token 的语料，采用启发式规则进行清洗与过滤，其中超过 85% 为科研类 arXiv 论文；
- ArXiv-RedPajama（Computer，2023）：由 arXiv 全部 LaTeX 文件构成，已去除导言区（preambles）、注释、宏（macros）与参考文献，共计 28.0B token。

在我们的实验中，我们分别在每个 arXiv 语料上训练 DeepSeek-LLM 1.3B 达 150B token、训练 DeepSeek-Coder-Base-v1.5 7B 达 40B token。结果似乎表明，arXiv 论文无法有效提升数学推理能力。当仅在 arXiv 语料上训练时，两个模型在本研究所采用的、复杂度各异的多个数学基准上均未表现出明显提升，甚至出现性能退化。这些基准包括 GSM8K、MATH 等定量推理数据集（表 8），MMLU-STEM 等多项选择挑战（表 8），以及 miniF2F 等形式化数学基准（表 9）。

【表格】表 8 数学训练对不同 arXiv 数据集的影响（原文第 17 页）

ArXiv 语料	miniF2F-valid	miniF2F-test
无数学训练	20.1%	21.7%
MathPile	16.8%	16.4%
ArXiv-RedPajama	14.8%	11.9%

表 9 | 数学训练对不同 arXiv 语料的影响，基座模型为 DeepSeek-Coder-Base-v1.5 7B。我们在 Isabelle 中评估非形式到形式的证明（informal-to-formal proving）。

然而，这一结论有其局限，应当审慎看待。我们尚未研究：

- arXiv token 对本研究未涵盖的特定数学相关任务的影响，例如定理的“非形式化”（informalization），即把形式化陈述或证明转换为非形式化版本；
- arXiv token 与其他类型数据结合时的效果；
- arXiv 论文的收益是否会在更大模型规模上显现。

因此，仍需要进一步探索，我们将其留待未来研究。

5.2 强化学习的洞见

5.2.1 走向统一范式

在本节中，我们提供一个统一范式，用以分析 SFT、RFT、DPO、PPO、GRPO 等不同的训练方法，并进一步开展实验以探索该统一范式的各个要素。一般而言，一种训练方法关于参数 θ 的梯度可以写为：

$$\nabla_\theta J_A(\theta) = \mathbb{E}_{(q,o)\sim D}\left[\frac{1}{|o|}\sum_{t=1}^{|o|} GC_A(q, o, t, \pi_{rf})\nabla_\theta \log \pi_\theta(o_t \mid q, o_{<t})\right] \quad (5)$$

（式中 D 下方的标注为“数据来源”（Data Source），GC_A 下方的标注为“梯度系数”（Gradient Coefficient）。）

该范式包含三个关键组成部分：1）数据来源 D，它决定训练数据；2）奖励函数 π_{rf}，它是训练奖励信号的来源；3）算法 A，它将训练数据与奖励信号处理为梯度系数 GC，该系数决定了对相应数据施加惩罚或强化的幅度。我们基于这一统一范式分析了几种代表性方法：

- **监督微调（Supervised Fine-tuning，SFT）**：SFT 在人工挑选的 SFT 数据上对预训练模型进行微调。
- **拒绝采样微调（Rejection Sampling Fine-tuning，RFT）**：RFT 基于 SFT 问题，从 SFT 模型采样的输出中筛选后进一步微调 SFT 模型；RFT 依据答案的正确性来过滤输出。
- **直接偏好优化（Direct Preference Optimization，DPO）**：DPO 使用成对的 DPO 损失，在从 SFT 模型采样的增强输出上微调，从而进一步改进 SFT 模型。
- **在线拒绝采样微调（Online Rejection Sampling Fine-tuning，Online RFT）**：与 RFT 不同，Online RFT 以 SFT 模型初始化策略模型，并使用从实时策略模型采样的增强输出进行微调以改进它。
- **PPO/GRPO**：PPO/GRPO 以 SFT 模型初始化策略模型，并使用从实时策略模型采样的输出对其进行强化。

我们在表 10 中总结了这些方法的各个组成部分。更详细的推导过程请参见附录 A.1。

【表格】表 10 不同方法的数据来源与梯度系数（原文第 19 页）

关于数据来源的观察 我们将数据来源分为两类：在线采样（online sampling）与离线采样（offline sampling）。在线采样指训练数据来自实时训练策略模型的探索结果，而离线采样指训练数据来自初始 SFT 模型的采样结果。RFT 与 DPO 属于离线方式，Online RFT 与 GRPO 属于在线方式。

如图 5 所示，我们发现 Online RFT 在两个基准上显著优于 RFT。具体而言，Online RFT 在训练早期与 RFT 相当，但在后期取得绝对优势，这体现了在线训练的优越性。这一点是直观的：在初始阶段，actor 模型与 SFT 模型高度相似，采样数据只体现出细微差异；而在后期，从 actor 采样的数据会呈现更大差异，实时数据采样将带来更大优势。

【图表】图 5 DeepSeekMath-Instruct 1.3B 模型经不同方法进一步训练后在两个基准上的表现（原文第 19 页）

关于梯度系数的观察 算法将奖励信号处理为梯度系数，以更新模型参数。在我们的实验中，我们将奖励函数分为“规则”（Rule）与“模型”（Model）两类。Rule 指根据答案的正确性判断响应的质量，Model 指我们训练一个奖励模型为每个响应打分，奖励模型的训练数据基于规则判断。式 10 与式 21 突显了 GRPO 与 Online RFT 的一个关键差异：GRPO 独特地依据奖励模型给出的奖励值来调整其梯度系数，这使得它能够按响应幅度的大小对其施加差异化的强化与惩罚。相比之下，Online RFT 缺乏这一特性：它不对错误响应施加惩罚，并以相同的强度统一强化所有答案正确的响应。

如图 5 所示，GRPO 超越了 Online RFT，从而凸显了调整正负梯度系数的效率。此外，GRPO+PS 相比 GRPO+OS 表现出更优的性能，说明使用细粒度、步骤感知（step-aware）的梯度系数是有益的。进一步地，我们探索了迭代式 RL，在实验中我们进行了两轮迭代。如图 6 所示，我们注意到迭代式 RL 显著提升了性能，尤其是在第一次迭代时。

【图表】图 6 DeepSeekMath-Instruct 7B 迭代式强化学习在两个基准上的表现（原文第 20 页）

5.2.2 RL 为何有效？

在本文中，我们基于指令微调数据的一个子集开展强化学习，它在指令微调模型的基础上取得了显著的性能提升。为进一步解释强化学习为何有效，我们评估了 Instruct 模型与 RL 模型在两个基准上的 Pass@K 与 Maj@K 准确率。如图 7 所示，RL 提升了 Maj@K 的性能，但未提升 Pass@K。这

些发现表明，RL 是通过使输出分布更加稳健来提升模型的整体性能的；换言之，性能的改善似乎归因于提升了从 TopK 中选出正确响应的能力，而非基础能力的增强。类似地，（Wang 等，2023a）发现了 SFT 模型在推理任务中的错位（misalignment）问题，表明 SFT 模型的推理性能可以通过一系列偏好对齐策略得到改善（Song 等，2023；Wang 等，2023a；Yuan 等，2023b）。

【图表】图 7 SFT 与 RL 的 DeepSeekMath 7B 在 GSM8K 与 MATH 上的 Maj@K 与 Pass@K（温度 0.7）（原文第 21 页）

5.2.3 如何实现更有效的 RL？

我们展示了 RL 在数学推理任务上效果相当好。我们还提供了一个统一范式来理解不同的代表性训练方法。在该范式下，所有方法都被概念化为直接或简化的 RL 技术。如式 5 所总结，存在三个关键组成部分：数据来源、算法与奖励函数。我们围绕这三个组成部分给出若干潜在的未来方向。

数据来源 数据来源是所有训练方法的原材料。在 RL 语境下，我们特指数据来源为无标注的问题以及从策略模型采样的输出。在本文中，我们仅使用指令微调阶段的问题以及一种朴素的核采样（nucleus sampling）来采样输出。我们认为这是我们 RL 流水线仅提升 Maj@K 性能的一个潜在原因。未来，我们将把 RL 流水线扩展到分布外（out-of-distribution）的问题提示上，并结合更先进的采样（解码）策略，例如基于树搜索的方法（Yao 等，2023）。此外，高效的推理技术（Kwon 等，2023；Leviathan 等，2023；Xia 等，2023，2024）决定了策略模型的探索效率，同样起着极其重要的作用。

算法 算法将数据与奖励信号处理为梯度系数，以更新模型参数。基于式 5，在某种程度上，当前所有方法都完全信任（TRUST）奖励函数的信号，以提高或降低某个 token 的条件概率。然而，无法保证奖励信号总是可靠的，尤其是在极其复杂的任务中。例如，即使是由训练有素的标注人员仔细标注的 PRM800K 数据集（Lightman 等，2023），仍包含约 20% 的错误标注（脚注 7）。为此，我们将探索对噪声奖励信号稳健的强化学习算法。我们相信，这类弱到强（WEAK-TO-STRONG）对齐方法（Burns 等，2023）将给学习算法带来根本性变革。

脚注 7：https://github.com/openai/prm800k/issues/12#issuecomment-1728491852

奖励函数 奖励函数是训练信号的来源。在 RL 中，奖励函数通常是神经奖励模型。我们认为奖励模型存在三个重要方向：1）如何增强奖励模型的泛化能力。奖励模型必须能够有效泛化，以处理分布外的问题与更高级的解码输出；否则，强化学习可能只是稳定了 LLM 的分布，而无法提升其基础能力；2）如何反映奖励模型的不确定性。不确定性有可能充当弱奖励模型与弱到强学习算法之间的连接

桥梁；3）如何高效构建高质量的进程奖励模型（process reward model），为推理过程提供细粒度的训练信号（Lightman 等，2023；Wang 等，2023b）。

6 结论、局限与未来工作

我们提出 DeepSeekMath，它在竞赛级 MATH 基准上优于所有开源模型，并接近闭源模型的性能。DeepSeekMath 以 DeepSeek-Coder-v1.5 7B 初始化，并经过 500B token 的持续训练，其中很大一部分训练数据是来自 Common Crawl 的 120B 数学 token。我们广泛的消融研究表明，网页为高质量数学数据提供了巨大潜力，而 arXiv 可能并不像我们预期的那样有益。我们引入组相对策略优化（Group Relative Policy Optimization，GRPO）——近端策略优化（Proximal Policy Optimization，PPO）的一种变体——它能够以更低的内存消耗显著提升数学推理能力。实验结果表明，即使 DeepSeekMath-Instruct 7B 已在基准上取得高分，GRPO 依然有效。我们还提供了一个统一范式来理解一系列方法，并总结了若干实现更有效强化学习的潜在方向。

尽管 DeepSeekMath 在定量推理基准上取得了令人印象深刻的分数，但其在几何与定理证明方面的能力相对弱于闭源模型。例如，在我们的试运行中，模型无法处理与三角形和椭圆相关的问题，这可能表明预训练与微调中存在数据选择偏差。此外，受模型规模限制，DeepSeekMath 的少样本能力不如 GPT-4：GPT-4 可以通过少样本输入提升性能，而 DeepSeekMath 在零样本与少样本评估中表现相近。未来，我们将进一步改进工程化的数据选择流水线，以构建更高质量的预训练语料。此外，我们将探索实现更有效的大语言模型强化学习的潜在方向（第 5.2.3 节）。

附录

附录 A.1 强化学习分析

附录 A.2 监督微调

附录 A.3 拒绝采样微调

附录 A.4 在线拒绝采样微调

附录 A.5 直接偏好优化（DPO）

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] R. Anil, S. Borgeaud, Y. Wu, J. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, D. Silver, S. Petrov, M. Johnson, I. Antonoglou, J. Schrittwieser, A. Glaese, J. Chen, E. Pitler, T. P . Lillicrap, A. Lazaridou, O. Firat, J. Molloy, M. Isard, P . R. Barham, T. Hennigan, B. Lee, F. Viola, M. Reynolds, Y. Xu, R. Doherty, E. Collins, C. Meyer, E. Rutherford, E. Moreira, K. Ayoub, M. Goel, G. Tucker, E. Piqueras, M. Krikun, I. Barr, N. Savinov, I. Danihelka, B. Roelofs, A. White, A. Andreassen, T. von Glehn, L. Yagati, M. Kazemi, L. Gonzalez, M. Khalman, J. Sygnowski, and et al. Gemini: A family of highly capable multimodal models. CoRR, abs/2312.11805, 2023. doi: 10.48550/ARXIV.2312.11805. URL https: //doi.org/10.48550/arXiv.2312.11805 .

[2] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

[3] Z. Azerbayev, H. Schoelkopf, K. Paster, M. D. Santos, S. McAleer, A. Q. Jiang, J. Deng, S. Biderman, and S. Welleck. Llemma: An open language model for mathematics. arXiv preprint arXiv:2310.10631, 2023.

[4] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, et al. Qwen technical report. arXiv preprint arXiv:2309.16609, 2023.

[5] C. Burns, P . Izmailov, J. H. Kirchner, B. Baker, L. Gao, L. Aschenbrenner, Y. Chen, A. Ecoffet, M. Joglekar, J. Leike, et al. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. arXiv preprint arXiv:2312.09390, 2023.

[6] ChatGLM3 Team. Chatglm3 series: Open bilingual chat llms, 2023. URL https://github.c om/THUDM/ChatGLM3 .

[7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374 .

[8] W. Chen, X. Ma, X. Wang, and W. W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. CoRR, abs/2211.12588, 2022. doi: 10.48550/ARXIV.2211.12588. URL https://doi.org/10.48550/arXiv.2211.12588 .

[9] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[10] T. Computer. Redpajama: an open dataset for training large language models, Oct. 2023. URL https://github.com/togethercomputer/RedPajama-Data .

[11] DeepSeek-AI. Deepseek LLM: scaling open-source language models with longtermism. CoRR, abs/2401.02954, 2024. doi: 10.48550/ARXIV.2401.02954. URL https://doi.org/10.485 50/arXiv.2401.02954 . Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang. Glm: General language model pretraining with autoregressive blank infilling. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 320–335, 2022.

[12] L. Gao, A. Madaan, S. Zhou, U. Alon, P . Liu, Y. Yang, J. Callan, and G. Neubig. PAL: programaided language models. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA, volume 202 of Proceedings of Machine Learning Research, pages 10764–10799. PMLR, 2023. URL https://proceedings.mlr.press/v202/gao23f. html .

[13] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, M. Huang, N. Duan, and W. Chen. Tora: A toolintegrated reasoning agent for mathematical problem solving. CoRR, abs/2309.17452, 2023. doi: 10.48550/ARXIV.2309.17452. URL https://doi.org/10.48550/arXiv.2309.1745 2 .

[14] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024.

[15] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[16] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[17] High-flyer. Hai-llm: 高 效 且 轻 量 的 大 模 型 训 练 工 具 , 2023. URL https://www.high-flyer.c n/en/blog/hai-llm .

[18] Inflection AI. Inflection-2, 2023. URL https://inflection.ai/inflection-2 .

[19] A. Q. Jiang, S. Welleck, J. P . Zhou, W. Li, J. Liu, M. Jamnik, T. Lacroix, Y. Wu, and G. Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. arXiv preprint arXiv:2210.12283, 2022.

[20] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7b. arXiv preprint arXiv:2310.06825, 2023.

[21] A. Joulin, E. Grave, P . Bojanowski, M. Douze, H. Jégou, and T. Mikolov. Fasttext. zip: Compressing text classification models. arXiv preprint arXiv:1612.03651, 2016.

[22] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 2023.

[23] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In International Conference on Machine Learning, pages 19274–19286. PMLR, 2023.

[24] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V . Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, et al. Solving quantitative reasoning problems with language models. Advances in Neural Information Processing Systems, 35:3843–3857, 2022a. A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V . V . Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, Y. Wu, B. Neyshabur, G. Gur-Ari, and V . Misra. Solving quantitative reasoning problems with language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022b. URL http://papers.nips.cc/paper_files/paper/2022/hash/18abbeef8cfe9203fdf9053c9c4fe191-Abstr act-Conference.html .

[25] H. Lightman, V . Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. arXiv preprint arXiv:2305.20050, 2023.

[26] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

[27] H. Luo, Q. Sun, C. Xu, P . Zhao, J. Lou, C. Tao, X. Geng, Q. Lin, S. Chen, and D. Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. arXiv preprint arXiv:2308.09583, 2023.

[28] S. Mishra, M. Finlayson, P . Lu, L. Tang, S. Welleck, C. Baral, T. Rajpurohit, O. Tafjord, A. Sabharwal, P . Clark, and A. Kalyan. LILA: A unified benchmark for mathematical reasoning. In Y. Goldberg, Z. Kozareva, and Y. Zhang, editors, Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022, pages 5807–5832. Association for Computational Linguistics, 2022. doi: 10.18653/V1/2022.EMNLP-MAIN.392. URL https://doi.org/10.18653/v1/ 2022.emnlp-main.392 .

[29] X. Nguyen, W. Zhang, X. Li, M. M. Aljunied, Q. Tan, L. Cheng, G. Chen, Y. Deng, S. Yang, C. Liu, H. Zhang, and L. Bing. Seallms - large language models for southeast asia. CoRR, abs/2312.00738, 2023. doi: 10.48550/ARXIV.2312.00738. URL https://doi.org/10.485 50/arXiv.2312.00738 .

[30] OpenAI. GPT4 technical report. arXiv preprint arXiv:2303.08774, 2023.

[31] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P . Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. Advances in Neural Information Processing Systems, 35:27730–27744, 2022.

[32] K. Paster, M. D. Santos, Z. Azerbayev, and J. Ba. Openwebmath: An open dataset of high-quality mathematical web text. CoRR, abs/2310.06786, 2023. doi: 10.48550/ARXIV.2310.06786. URL https://doi.org/10.48550/arXiv.2310.06786 .

[33] L. C. Paulson. Three years of experience with sledgehammer, a practical link between automatic and interactive theorem provers. In R. A. Schmidt, S. Schulz, and B. Konev, editors, Proceedings of the 2nd Workshop on Practical Aspects of Automated Reasoning, PAAR-2010, Edinburgh, Scotland, UK, July 14, 2010, volume 9 of EPiC Series in Computing, pages 1–10. EasyChair, 2010. doi: 10.29007/TNFD. URL https://doi.org/10.29007/tnfd .

[34] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. CoRR, abs/2009.03393, 2020. URL https://arxiv.org/abs/2009.03393 .

[35] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. 2023. J. Schulman. Approximating kl divergence, 2020. URL http://joschu.net/blog/kl-app rox.html .

[36] J. Schulman, P . Moritz, S. Levine, M. Jordan, and P . Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv preprint arXiv:1506.02438, 2015.

[37] J. Schulman, F. Wolski, P . Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

[38] F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/pdf?id=fR3wGCk-IXp .

[39] F. Song, B. Yu, M. Li, H. Yu, F. Huang, Y. Li, and H. Wang. Preference ranking optimization for human alignment. arXiv preprint arXiv:2306.17492, 2023.

[40] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[41] T. Tao. Embracing change and resetting expectations, 2023. URL https://unlocked.micro soft.com/ai-anthology/terence-tao/ .

[42] H. Touvron, L. Martin, K. Stone, P . Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P . Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton-Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V . Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V . Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P . S. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P . Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P . Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. CoRR, abs/2307.09288, 2023. doi: 10.48550/arXiv.2307.09288. URL https://doi.org/10.48550/arXiv.2307. 09288 .

[43] T. H. Trinh, Y. Wu, Q. V . Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. Nature, 625(7995):476–482, 2024.

[44] P . Wang, L. Li, L. Chen, F. Song, B. Lin, Y. Cao, T. Liu, and Z. Sui. Making large language models better reasoners with alignment. arXiv preprint arXiv:2309.02144, 2023a.

[45] P . Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. CoRR, abs/2312.08935, 2023b.

[46] Z. Wang, R. Xia, and P . Liu. Generative AI for math: Part I - mathpile: A billion-token-scale pretraining corpus for math. CoRR, abs/2312.17120, 2023c. doi: 10.48550/ARXIV.2312.17120. URL https://doi.org/10.48550/arXiv.2312.17120 .

[47] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V . Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In NeurIPS, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf 4f15af0f7b31abca4-Abstract-Conference.html . T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.

[48] M. Wenzel, L. C. Paulson, and T. Nipkow. The isabelle framework. In O. A. Mohamed, C. A. Muñoz, and S. Tahar, editors, Theorem Proving in Higher Order Logics, 21st International Conference, TPHOLs 2008, Montreal, Canada, August 18-21, 2008. Proceedings, volume 5170 of Lecture Notes in Computer Science, pages 33–38. Springer, 2008. doi: 10.1007/978-3-540-7 1067-7_7. URL https://doi.org/10.1007/978-3-540-71067-7_7 .

[49] H. Xia, T. Ge, P . Wang, S.-Q. Chen, F. Wei, and Z. Sui. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In H. Bouamor, J. Pino, and K. Bali, editors, Findings of the Association for Computational Linguistics: EMNLP 2023, pages 3909– 3925, Singapore, Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/20 23.findings-emnlp.257. URL https://aclanthology.org/2023.findings-emnlp.257 .

[50] H. Xia, Z. Yang, Q. Dong, P . Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. arXiv preprint arXiv:2401.07851, 2024.

[51] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601, 2023.

[52] L. Yu, W. Jiang, H. Shi, J. Yu, Z. Liu, Y. Zhang, J. T. Kwok, Z. Li, A. Weller, and W. Liu. Metamath: Bootstrap your own mathematical questions for large language models. CoRR, abs/2309.12284, 2023. doi: 10.48550/ARXIV.2309.12284. URL https://doi.org/10.485 50/arXiv.2309.12284 .

[53] Z. Yuan, H. Yuan, C. Li, G. Dong, C. Tan, and C. Zhou. Scaling relationship on learning mathematical reasoning with large language models. arXiv preprint arXiv:2308.01825, 2023a.

[54] Z. Yuan, H. Yuan, C. Tan, W. Wang, S. Huang, and F. Huang. Rrhf: Rank responses to align language models with human feedback without tears. arXiv preprint arXiv:2304.05302, 2023b.

[55] X. Yue, X. Qu, G. Zhang, Y. Fu, W. Huang, H. Sun, Y. Su, and W. Chen. Mammoth: Building math generalist models through hybrid instruction tuning. CoRR, abs/2309.05653, 2023. doi: 10.48550/ARXIV.2309.05653. URL https://doi.org/10.48550/arXiv.2309.05653 .

[56] K. Zheng, J. M. Han, and S. Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. arXiv preprint arXiv:2109.00110, 2021.

[57] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364 .

DeepSeek-VL：面向真实世界的视觉-语言理解

*DeepSeek-VL: Towards Real-World Vision-Language Understanding Haoyu Lu*1†, Wen Liu*1, Bo Zhang*1‡, Bingxuan Wang1†, Kai Dong1, Bo Liu1†, Jingxiang Sun1†,*

DeepSeek-AI research@deepseek.com

arXiv:2403.05525 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-VL，一个面向真实世界视觉与语言理解应用的开源视觉-语言（VL）模型。我们的方法围绕三个关键维度构建：（1）数据构建：我们力求数据多样、可扩展，并广泛覆盖真实世界场景，包括网页截图、PDF、OCR、图表与知识型内容（专家知识、教材），旨在全面表征实际应用语境。此外，我们基于真实用户场景构建用例分类体系，并据此构建指令微调数据集；使用该数据集微调显著提升了模型在实际应用中的用户体验。（2）模型架构：兼顾效率与大多数真实世界场景的需求，DeepSeek-VL 采用混合视觉编码器，可在固定的 token 预算内高效处理高分辨率图像（1024×1024），同时保持相对较低的计算开销。这一设计选择确保模型能够在各类视觉任务中捕获关键的语义与细节信息。（3）训练策略：我们认为，一个优秀的视觉-语言模型首先应具备强大的语言能力。为确保预训练期间不丢失 LLM 能力，我们研究了一种有效的视觉-语言预训练策略：从一开始就融入 LLM 训练，并谨慎管理视觉与语言模态之间出现的竞争动态。我们从以文本为重点开始，逐步调整比例，以促进两种模态的平衡融合。DeepSeek-VL 家族（1.3B 与 7B 两个版本）作为视觉-语言聊天机器人在真实应用中提供了更优的用户体验，在同等模型规模下于广泛的视觉-语言基准上取得最先进或极具竞争力的性能，同时在以语言为中心的基准上保持稳健表现。我们已公开 1.3B 与 7B 两个模型，以推动基于该基础模型的研究与创新。

1 引言

大语言模型（LLM）（Anthropic, 2023; Google, 2023; OpenAI, 2022, 2023a）的显著成功，激发了人们对能够处理语言之外多种模态的通用接口的需求。为回应这一日益增长的需求，我们见证了 GPT-4V（OpenAI, 2023b）与 Gemini（Team et al., 2023）等大型多模态模型（Large Multimodal Model, LMM）的涌现，它们作为通用助手，能够理解并执行跨越视觉与语言的指令。这些模型在执行复杂、多样的真实世界任务方面展现出可观的前景，使交互更加自然、更接近人类。

近期，旨在缩小与专有模型差距的开源大型多模态模型大量涌现。尽管已经取得长足进步，尤其是在基准性能方面，但在真实世界性能与用户体验方面，大多数开源模型与最先进的闭源模型（Bai et al., 2023; Bavishi et al., 2023; OpenAI, 2023b; Team et al., 2023）之间仍存在显著差距。对开源社区而言，要开发出具备稳健通用多模态能力、可面向真实世界应用的模型依然是一项挑战。

大多数开源模型与专有模型之间的性能差距，在真实世界场景中尤为明显，主要原因如下：

- 许多开源方案将相当比例的计算资源分配给指令微调（instruction tuning）阶段。然而，训练强大语言模型的经验凸显了大规模预训练在通用智能发展中的重要性。为使多模态模型具备丰富的世界知识，应当重视利用广谱视觉-语言数据开展的全面预训练。
- 一种常见做法是在指令微调时把各类学术数据集混合在一起。这种方法虽然可能取得不错的基准结果，但往往难以提供真实世界中的使用体验。
- 在模型架构方面，以往工作大多将一个视觉 Transformer（通常是文本对齐的）适配到预训练语言模型上。然而，这些模型大多在相对较低的分辨率下运行，例如 336×336 或 448×448。复杂真实世界场景中的种种细节，例如光学字符识别或细小物体辨别，要求具备高分辨率处理能力。
- 尽管已有一些模型（01-ai, 2024; Lin et al., 2023a; Sun et al., 2023; Wang et al., 2023b）开始利用预训练，但它们往往忽视了语言能力的保持。经过长期的多模态训练后，语言能力常常会出现退化。既然我们的目标是打造在两种模态上都具备强大能力的通才模型，就应当有一种在发展新模态能力的同时良好保持语言能力的训练策略。

有鉴于此，我们提出 DeepSeek-VL——一个构建于 DeepSeek 语言模型系列之上的开源大型多模态模型。我们研发该模型，旨在追求其在真实世界场景中的出色表现，这涉及大规模预训练、基于用例分类体系（taxonomy）的细致数据策展、面向高分辨率处理的模型架构设计，以及平衡多模态的训练策略。在上述工作之上，我们还提出了一套引导模型规模扩展（从 1B 到 7B）的训练方法。这些全面的探索使模型在实际场景中相较其他同等规模的大型多模态模型（LMM）展现出显著的性能优势。

【图表】图 1 DeepSeek-VL 具备通用的多模态理解能力，能够处理逻辑图、网页、公式识别、科学文献、自然图像以及复杂场景中的具身智能（原文第 4 页）

DeepSeek-VL 的预训练数据集由多种来源汇编而成，包括但不限于 Common Crawl、Web Code、电子书、教育材料与 arXiv 论文。该数据集全面涵盖网页截图、PDF、OCR、图表以及知识性内容（专业知识、教科书）等真实世界场景，力求在保持可扩展性的同时提供广泛而实用的数据表示。

尽管我们的预训练数据涵盖广泛的世界知识，我们仍细致策展指令微调数据集，以反映真实世界的使用场景。为此，我们人工从互联网上收集针对 GPT-4V 与 Gemini 的真实测试案例。这些案例已被系统地整理为一个全面的分类体系。我们借助这一结构化分类体系为每张测试图像选择提示词（prompt），从而确保指令微调数据集既实用又切题。该分类体系还被用于构建评测数据集，以有效评估真实世界中的性能表现。

视觉模块的设计目标是在固定 token 预算之内尽可能优化高分辨率视觉输入的利用，从而有效控制推理成本。为此，我们采用混合视觉编码器，它将一个用于在 384×384 分辨率下进行粗粒度语义提取的文本对齐编码器，与一个在 1024×1024 分辨率下捕捉细节视觉信息的高分辨率编码器相结合。通过融合这两个编码器，我们的混合方法能够将一张 1024×1024 分辨率图像（在大多数用例中已足够）高效压缩为 576 个 token。这一 token 数量在丰富的视觉表示与 token 经济性之间取得平衡，使图文交错与多轮推理场景均成为可能。

在多模态模型的预训练过程中，一个常见挑战是：当训练过程过度依赖视觉-语言数据时，语言能力可能发生退化。我们的研究表明，保持相当比例的语言数据——具体而言，至少 70%——对于维护模型内部语言知识的完整性至关重要。这一平衡对于在不损害语言性能的前提下实现稳健的多模态能力十分关键。此外，我们提出了一种新颖的“模态预热”（modality warm-up）策略。该方法在训练过程中细致调整各模态的比例，逐步引入更多视觉-语言数据。对模态比例的精心调节配合预热策略，使两种模态的性能达到平衡。

在迭代模型时，我们先在小规模上开展实验，再扩展到更大的模型规模。然而，较小的模型（例如 1B 模型）无法在基准（Schaeffer et al., 2024）上展现出合理的性能，也难以忠实反映模型的实际表现。我们采用两种方法来应对这一点。首先，我们将评测协议从多项选择改为比较各选项的困惑度（perplexity）。其次，为避免指令跟随能力成为瓶颈，我们在预训练阶段混入少量指令微调数据。如此，我们便能借助 1B 模型获得合理的性能，并在实验中更准确地衡量每次迭代的影响。

通过对通用视觉与语言基准的广泛评测，DeepSeek-VL 系列在真实世界应用中展现出优越的用户体验，并在同等模型规模下于广泛的视觉-语言基准上取得最先进的或极具竞争力的性能，同时保持稳健的以语言为中心的性能。为促进创新并支持广泛的应用，我们公开发布了两个版本（1.3B 与 7B），以期满足不同算力条件的需求。

2 数据构建

多样而大规模的数据集是视觉语言模型训练中最重要的要素。我们的数据集可分为两部分：视觉-语言预训练数据与视觉-语言监督微调数据。VL 预训练数据由来自多种来源的视觉-文本数据构成，旨在增强模型基础的跨模态理解能力；而 VL 监督微调数据规模相对较小，旨在教会模型完成特定的下游任务。按照设计，VL 预训练数据用于在训练阶段 1 预热视觉-语言适配器，并在阶段 2 联合预训练视觉-语言模型；VL 监督微调数据则用于训练阶段 3，即视觉语言监督微调。

2.1 视觉-语言预训练数据

本研究所用的预训练数据集除了涵盖多种公开可获取的来源之外，还包含一部分专有数据。我们在表 1 中全面概述了联合视觉与语言预训练阶段所采用的数据来源。这样的数据集有助于 LLM 理解图像中所描绘的实体。

此外，我们对完整数据集作了详细分解，其组织为以下类别：

交错图文数据（Interleaved image-text）使模型具备更强的多模态输入上下文学习能力，我们使用了三个公开数据集 MMC4（Zhu et al., 2024）、Wiki（Burns et al., 2023）、Wikihow（Yang et al., 2021）以及 Epub 教科书。

图像描述数据（Image caption）来自三个高质量的图文配对数据集：Capsfusion（Yu et al., 2023a）、TaiSu（Liu et al., 2022b）与 Detailed Caption（echo840, 2024）。

表格与图表数据（Table and chart）使模型学习通用的表格与图表图像理解能力。它涵盖多种公开数据源，包括 Chart2text（Kantharaj et al., 2022）、Geo170K（Gao et al., 2023）、Unichart（Masry et al., 2023）、Ureader（Ye et al., 2023）、M-paper（Hu et al., 2023）、ScienceQA（Lu et al., 2022b）、ScreenQA（Hsiao et al., 2022）、SciGraphQA-295K（Li and Tajbakhsh, 2023）、Paper2figure100k（Rodriguez et al., 2023）、Widget Captioning（Li et al., 2020）、Screen2words（Wang et al., 2021）与 Refexp（Mao et al., 2016）。

Web Code 数据赋予模型从图形界面或可视化图表重建代码的能力。我们利用 Websight（HuggingFaceM4, 2024）进行 UI 逆向渲染，并对可视化图表采用与 MATCHA（Liu et al., 2022a）类似的反向渲染策略。这一过程处理了 Stack 数据集（Kocetkov et al., 2023）中约 146 万个 Jupyter notebook。通过抽取这些 notebook 并整理出全部图表及其对应的前置代码片段，我们成功构建了包含 200 万对图像与代码的集合。为提升数据质量，我们筛选出 110 万个实例（每个实例由一张图像与至少 5 行代码构成），构成我们的主要训练数据集。

文档光学字符识别（OCR）数据有助于在文档层级识别光学字符，即使在极具挑战的真实世界场景中亦然。据我们所知，目前尚无同时涵盖英文与中文文档的公开大规模数据集。尽管存在公开可获取的小规模数据集 Latex-OCR（Blecher, 2024），我们仍额外构建了一个全面的中英文文档 OCR 数据集。它由两部分组成：1）**arXiv 论文：**我们从 140 万篇 arXiv 论文中收集源代码并编译 PDF。借助 Nougat（Blecher et al., 2023）的预处理工具，我们将这些论文渲染为成对的图像与文本；2）**电子书与教育材料：**我们从 Anna's Archive（Anna's Archive, 2024）清洗了 86 万本英文电子书与 18 万本中文电子书，以及数百万道 K-12 教育考题。随后，我们使用 HTML 渲染工具（Kulkarni and Truelsen）将这些采用不同模板的 HTML 文件转换为成对的图像与文本格式。

【表格】表 1 联合视觉-语言预训练阶段所用数据集汇总（原文第 7 页）

场景文本 OCR 数据增强了模型从文本融入环境的图像中识别并抽取文本的能力。该数据集由多个公开数据集构成，包括 ArT（Chng et al., 2019）、MLT-17（Nayef et al., 2017）、LSVT（Sun et al., 2019）、UberText（Zhang et al., 2017）、Coco-text（Veit et al., 2016）、RCTW-17（Shi et al., 2017）、ReCTS（Zhang et al., 2019）、TextOCR（Singh et al., 2021）、OpenVINO（Krylov et al., 2021）与 HierText（Long et al., 2022）。

【表格】表 2 联合视觉-语言监督微调阶段所用数据汇总（原文第 8 页）

纯文本语料（Text-only corpus）用于保持模型在以语言为中心的任务上的熟练度。在本研究中，我们采用与 DeepSeek-LLM（DeepSeek-AI, 2024）相同的文本语料。

2.2 监督微调数据

本研究所用的监督微调数据集涵盖多种多模态与语言数据来源，包括知名的开源共享 gpt4v 数据集，如 ShareGPT4V（Chen et al., 2023）、LAION-GPTV（LAION, 2023）、LVIS-Instruct4V（Wang et

al., 2023a）、textOCR-GPT4V（Carter, 2024）、LLaVA1.6-GPT4V（Liu et al., 2024a）与 IconQA（Lu et al., 2021）。此外，我们还纳入从 Ureader（Ye et al., 2023）、ScreenQA（Hsiao et al., 2022）、Geo170K（Gao et al., 2023）与 ScienceQA（Lu et al., 2022b）等预训练数据集中抽取的部分表格与图表数据。同时，我们整合了从 Screen-to-code（Abi, 2024）任务获得的 UI Code 数据集。为提升多模态 SFT 数据的质量，我们还策展了一部分高质量的内部多模态 SFT 数据，其中一些为中文。我们的内部指令微调数据集经过精心设计，以反映真实世界的使用场景并覆盖广泛的任务。我们首先从各类在线来源收集针对 GPT-4V 与 Gemini 的多样化真实测试案例。随后，这些测试案例被细致分析并整理为一个全面的分类体系，涵盖识别、转换、分析、推理、评测与安全等多个类别，详见表 3。这一结构化分类体系作为指南，用于为每张测试图像选择具有代表性的提示词，确保我们的指令微调数据集既实用又与真实世界应用相关。此外，该分类体系还被用于构建均衡而全面的评测数据集，使我们能够有效评估模型在不同任务与类别上的表现。通过遵循这一系统化方法，我们确保内部多模态 SFT 数据所覆盖的类别与分类体系良好对齐，并能代表真实世界的使用场景。

【表格】表 3 我们内部 SFT 数据的分类体系。我们高质量内部多模态 SFT 数据所覆盖的类别在这一分类体系中得到了全面的体现（原文第 9 页）

此外，我们还将 DeepSeek-LLM（DeepSeek-AI, 2024）中使用的纯文本 SFT 数据纳入我们的联合视觉-语言 SFT 数据之中。

3 方法

3.1 模型架构

我们的系统包含三个模块：混合视觉编码器、视觉-语言适配器与语言模型。本节将逐一介绍各个部分。

混合视觉编码器（Hybrid Vision Encoder）：我们采用 SigLIP 作为视觉编码器，从视觉输入中提取高层语义特征表示。然而，我们观察到，单一的 SigLIP 编码器难以全面应对真实世界中的各类问题。CLIP 系列（包括 SigLIP）中的视觉编码器主要面向语义视觉表示而设计，但受制于编码歧义——由于所谓的“CLIP 盲对”（CLIP-blind pairs；Tong et al., 2024），视觉上差异明显的图像会被编码为相似的表示。同时，CLIP 系列模型受限于相对较低分辨率的输入（如 224×224、336×336、384×384、512×512），这妨碍了它们处理需要更精细低层特征的任务，例如密集 OCR 和视觉定位任务。

为应对这些局限，近期研究（Lin et al., 2023b; Tong et al., 2024; Wei et al., 2023）主张引入额外的纯视觉自监督编码器，以增强多模态模型的视觉定位能力。基于上述动机，我们额外使用了一个基于 SAM-B（Kirillov et al., 2023）的纯视觉编码器——它是一个预训练的 ViTDet（Li et al., 2022）图像编码器，用于处理低层特征，可接受 1024×1024 的高分辨率图像输入。除 SAM-B 编码器外，我们保留了接收 384×384 低分辨率图像输入的 SigLIP-L 视觉编码器。因此，我们的混合视觉编码器结合了 SAM-B 与 SigLIP-L，在高效编码 1024×1024 高分辨率图像的同时，兼顾语义信息与细节信息。具体而言，高分辨率的 SAM-B 视觉编码器首先将图像调整至 1024×1024，并输出 64×64×256 的特征图。

对于 SAM-B 生成的 64×64×256 高分辨率特征图，VL 适配器首先将其插值为 96×96×256 的尺寸。随后，它使用两个步长为 2 的卷积层，得到 24×24×1024 的特征图，并将其重塑为 576×1024。与此同时，由 SigLIP-L 生成的尺寸为 576×1024 的低分辨率特征图与高分辨率特征拼接，最终得到 576 个 2048 维的视觉 token。这些视觉 token 具备可观的容量，能够增强高层语义视觉识别与低层视觉定位任务。之后，它们经过 GeLU 激活，并通过一个嵌入层与语言模型建立连接。

视觉-语言适配器（Vision-Language Adaptor）：我们采用一个两层的混合 MLP 来连接视觉编码器与 LLM。首先，使用各自独立的单层 MLP 分别处理高分辨率特征与低分辨率特征；随后，将这些特征沿维度拼接，并通过另一层 MLP 变换到 LLM 的输入空间。

语言模型（Language Model）：我们的语言模型建立在 DeepSeek LLM（DeepSeek-AI, 2024）之上，其微观设计大体沿用 LLaMA（Touvron et al., 2023a,b）的设计：采用 Pre-Norm 结构，使用 RMSNorm（Zhang and Sennrich, 2019）函数，并以 SwiGLU（Shazeer, 2020）作为前馈网络（FFN）的激活函数，中间层维度为 83 $d$model。它还采用旋转位置编码（Rotary Embedding；Su et al., 2024），并使用与 DeepSeek-LLM 相同的 tokenizer。我们推出了一系列 DeepSeek-VL 模型。鉴于我们的目标是将多模态数据与语言数据联合预训练，我们选择 DeepSeek 预训练模型中的一个中间 checkpoint 继续预训练。具体而言，DeepSeek-VL-1B 模型基于 DeepSeek-LLM-1B 模型构建，后者使用了约 5000 亿文本 token 的语料进行训练；而 DeepSeek-VL-7B 模型则基于使用约 2 万亿文本 token 训练的 DeepSeek-LLM-7B 模型构建。

【图表】图 2 可视化结果。DeepSeek-VL 能够捕捉微小物体并给出条理清晰的解释（原文第 11 页）

3.2 训练流程

如图 3 所示，我们分三个连续阶段训练 DeepSeek-VL：视觉-语言适配器预热、视觉-语言联合预训练，以及监督微调。我们目前聚焦于视觉理解能力，并且只在语言部分计算下一 token 预测损失。

【图表】图 3 我们的训练流程包含三个阶段。阶段 1 训练视觉-语言（VL）适配器，同时保持混合视觉编码器与语言模型冻结。阶段 2 是视觉与语言联合预训练的关键部分，VL 适配器与语言模型均可训练。阶段 3 是监督微调阶段，此阶段将训练低分辨率视觉编码器 SigLIP-L、VL 适配器以及语言模型（原文第 12 页）

3.2.1 阶段 1：训练视觉-语言适配器

本阶段的主要目标是在嵌入空间中建立视觉元素与语言元素之间的概念联系，从而使大语言模型（LLM）能够全面理解图像中所描绘的实体。与 LLaVA（Liu et al., 2024b）和 Instruct-BLIP（Dai et al., 2023）的先前研究一致，我们采用类似做法：在此阶段冻结视觉编码器与 LLM，仅允许视觉-语言适配器中的参数可训练。我们使用一个包含 125 万条来自 ShareGPT4V 的图像-文本配对描述、以及 250 万对 Document OCR 渲染配对的数据集来训练 VL 适配器。

然而，与 LLM 相比，视觉-语言适配器（例如 2 层 MLP）的参数容量显著更小。这种模型容量上的限制约束了该阶段能够学到的能力。一个自然的问题是：数据缩放定律在这一阶段是否有效？为回答这一问题，我们在表 8 中进行了一个简单实验。结果表明，在该阶段扩大数据规模并不能带来收益，甚至可能导致性能下降。因此，我们继续解冻 LLM，并在阶段 2 探索高效的视觉-语言预训练方法。

3.2.2 阶段 2：视觉-语言联合预训练

在此阶段，我们探索有效的预训练策略——它可以被视为一个额外阶段，用来使 LLM 理解多模态输入。我们保持视觉编码器冻结，优化语言模型与 VL 适配器。

最初，我们尝试直接用多模态数据训练 LLM。然而我们发现，虽然多模态性能指标逐步提升，语言指标却出现了明显而严重的下降，如图 4 所示（多模态:语言 = 100%:0%）。这凸显了直接在 LLM 基础上进行多模态预训练所固有的挑战，揭示了增强多模态能力与保持语言能力之间的关键权衡。

我们推测，观察到的现象源于两个主要因素：第一，大多数多模态语料过于简单，与语言数据的复杂度和分布存在显著差异。第二，多模态与语言两种模态之间似乎存在竞争动态，导致 LLM 语言能力出现所谓的灾难性遗忘。

语言-多模态联合训练（Joint Language-multimodal Training）：为应对这一挑战，我们设计了一种简单而有效的语言-多模态联合训练策略。在训练过程中，我们不仅使用多模态数据进行训练，还将大比例的语言数据纳入训练。该方法旨在平衡训练重心，缓解所观察到的不利影响。我们在图 4 中基于 DeepSeek-VL 1B 模型开展实验，以探究不同模态混合比例的影响。

对图表的分析得出若干关键结论：（1）引入语言数据显著缓解了语言能力的下降，模型的语言表现得到大幅改善；（2）加入语言数据并未导致多模态性能的显著损失，表明模型保留了多模态处理能力；（3）不同模态的性能与其在训练数据中所占比例强相关，印证了两种模态之间的竞争关系。最终，我们为最终模型选择了语言数据与多模态数据约 7:3 的训练比例。该比例使模型在保持语言能力的同时，能够更好地对多模态数据进行预训练，有效平衡了语言与多模态两方面能力的发展。

扩大视觉-语言预训练规模（Scaling Vision-Language Pretraining）：然而，模型的预训练阶段会产生大量计算开销，在 7B 模型上进行迭代需要过多的算力与时间。一种合适的策略是在更小的模型上开展实验，即在 1.3B 模型上实验，然后再扩展到 7B 模型。幸运的是，我们观察到，1.3B 模型上获得的相当一部分结论可以通过 SFT 有效迁移到 7B 模型（例如编码器设计）。然而，在阶段 2 的训练过程中，我们遇到 1.3B 模型的生成指标出现相当大的波动，使得有效监督训练过程变得困难。Schaeffer et al. (2024) 已讨论过这一现象：“即使模型家族的每 token 错误率随着规模增大而平滑、连续且可预测地变化，研究者对度量方式的选择仍可能引发急剧且不可预测的变化。”后续实验帮助我们找到了该问题的根本原因：1.3B 模型容量有限，以及训练数据集中缺少 SFT 数据，二者都阻碍了模型准确遵循指令的能力。即使模型知道正确选项，也难以精确地将其生成出来。

【图表】图 4 训练阶段 2 中不同模态融合比例的对比性能结果。过大的多模态数据比例（多模态:语言 = 100%:0%）会导致 LLM 语言能力的显著遗忘。合适的比例（多模态:语言 = 70%:30%）能够在有效缓解语言遗忘问题的同时增强模型的多模态能力（原文第 14 页）

为解决这些挑战，我们采用了双管齐下的方法。首先，我们使用 Multi-choice PPL（多项选择困惑度）方法来监控模型进展。这不仅要向网络输入 prompt 与图像，还要输入与问题相关的所有答案；随后，我们计算每个答案位置（如 A、B、C、D）的 PPL，并选择模型认为正确的选项作为最终答案。其次，我们以极小的比例将 SFT 数据引入训练数据集，使模型获得一定的指令遵循能力。这两种方法的结合确保了 1.3B 模型训练指标的稳定，并在阶段 3 之后带来更好的性能。

3.2.3 阶段 3：监督微调

在此阶段，我们使用基于指令的微调对预训练后的 DeepSeek-VL 模型进行微调，以增强其遵循指令与进行对话的能力，最终构建出交互式的 DeepSeek-VL-Chat 模型。我们使用视觉-语言 SFT 数据（如表 2 所示）优化语言模型、VL 适配器以及混合视觉编码器；由于 GPU 显存有限，SAM-B 保持冻结。我们只对答案与特殊 token 计算监督，并掩蔽 system prompt 与 user prompt。为保证模型在对话方面的全面能力，我们混合使用了多模态数据与 DeepSeek-LLM 中使用的纯文本对话数据。该方法确保了模型在各种对话场景中的通用性。

【图表】图 5 可视化结果。DeepSeek-VL 能够理解 Python 代码并提供详尽、有条理的解释（原文第 15 页）

3.3 超参数与基础设施

所有阶段的详细超参数见表 4。我们使用 HAI-LLM（High-flyer, 2023）——一个轻量高效的分布式训练框架——来训练与评估 DeepSeek-VL。由于我们使用视觉编码器将图像转换为嵌入向量，并统一处理图像嵌入与文本嵌入，因此可以很容易地将流水线并行适配到 VL 模型训练：我们只需将视觉编码器与文本嵌入视为单个模块，并将其作为所得模型的第一层。这个第一层具有复杂的模型结构，无法采用标准的张量并行技术；但幸运的是，与上层标准 transformer 块相比，它的计算量相对较小。因此，我们在所有张量并行 rank 上简单地重新计算视觉编码器的前向传播。视觉编码器的存在还会导致各模型层的执行时间不均匀，因此我们重新划分流水线并行各 rank 之间的模型层，以实现更好的负载均衡与吞吐。DeepSeek-VL 的上层与 DeepSeek-LLM 中的完全相同。借助这些微小改动，我们就可以像 Megatron（Korthikanti et al., 2023; Narayanan et al., 2021; Shoeybi et al., 2019）那样执行规范的 3D 并行技术，并像 DeepSeek-LLM（DeepSeek-AI, 2024）那样重叠计算与通信。DeepSeek-VL-7B 在一个由 64 个节点（每个节点 8 块 Nvidia A100 GPU）组成的集群上训练了 5 天，而 DeepSeek-VL-1B 在包含 16 个节点的配置上训练了 7 天。

【表格】表 4 我们 DeepSeek-VL 的详细超参数。（原文第 16 页）

4 评估

4.1 公开多模态基准评估

我们在以下一系列公开基准上评估我们的模型：

- 多模态综合理解数据集：MMMU（Yue et al., 2023）、CMMMU（Zhang et al., 2024）、MMBench（Liu et al., 2023a）、MMBench-CN（Liu et al., 2023a）、SeedBench（Li et al., 2023a）与 MMV（Yu et al., 2023b）。由于当前官方测试下载链接已失效，我们在 MMB/MMC-dev 上与竞争对手进行比较。
- 图表理解数据集：OCRBench（Liu et al., 2023b）；
- 幻觉数据集：POPE（Li et al., 2023b）；
- 科学问题数据集：ScienceQA（Lu et al., 2022a）与 MathVista（Lu et al., 2023）。

我们采用基于生成的评估方式，并使用贪婪解码。这里所说的基于生成的评估，是指让模型生成自由文本，并从生成的文本中解析出结果。

如表 5 所示，对比结果表明 DeepSeek-VL-7B 在广泛的基准上超越了大多数规模相近的开源模型。

【表格】表 5 不同多模态模型之间的比较。上半部分为专有模型，下半部分为开源模型。（原文第 17 页）

DeepSeek-VL 在 MMB、MMC 与 SEEDbench 等基准上优于规模相近的开源模型，甚至接近专有模型（DeepSeek-VL 与 GPT-4V 在 seedbench 上的成绩为 70.4 对 71.6），展示了其强大的自然图像理解能力。该模型在数学逻辑方面也超越了所有开源模型，但仍显著落后于 GPT-4V 等专有模型（在 MathVista 上为 36.1 对 47.8）。这一差异可能归因于基座模型规模的差异。

此外，如表 6 所示，DeepSeek-VL-1.3B 显著优于规模相当的模型。在 MMB 基准测试中，它展现出优于领先开源模型的性能，而使用的参数量仅接近后者的一半（1.3B 对 2.7B），这表明其具有稳健的自然图像理解能力。DeepSeek-VL-1.3B 甚至在 MathVista 上取得了与 7B 开源模型相当的结果，进一步验证了 DeepSeek-VL 系列强大的逻辑理解能力。

【表格】表 6 小型多模态模型之间的比较。（原文第 17 页）

4.2 公开语言基准评估

我们在以下公开语言基准上评估我们的模型：

- 多学科选择题数据集，包括 MMLU（Hendrycks et al., 2020）。
- 语言理解与推理数据集，包括 HellaSwag（Zellers et al., 2019）。
- 语言建模数据集，包括 Pile（Gao et al., 2020）。
- 数学数据集，包括 GSM8K（Cobbe et al., 2021）。
- 代码数据集，包括 MBPP（Austin et al., 2021）。
- 标准化考试，包括 AGIEval（Zhong et al., 2023）。

对于需要从若干选项中选出答案的数据集，我们采用基于困惑度（perplexity）的评估。这类数据集包括 HellaSwag 与 MMLU。这里所说的基于困惑度的评估，是指计算每个选项的困惑度，并选择困惑度最低的选项作为模型预测。基于困惑度的评估有助于区分模型预测之间细微的概率差异，并避免精确匹配式评估的不连续性。对于 GSM8K 与 AGIEval，我们采用基于生成的评估方式与贪婪解码。这里所说的基于生成的评估，是指让模型生成自由文本，并从生成的文本中解析出结果。对于 Pile-test，我们采用基于语言建模的评估，即在测试语料上计算 bits-per-byte（BPB）。结果如表 7 所示。

【表格】表 7 语言基准上的性能。（原文第 18 页）

可以观察到，在大多数语言基准上，DeepSeek-VL 的表现与 DeepSeek-7B 相当，甚至超过后者。例如，在作为评估通用语言能力的通用基准 HellaSwag 上，它取得了 68.4 对 68.5 的分数。DeepSeek-VL 在 MMLU 与 AGIEval 等指标上优于 DeepSeek-7B，这表明多模态训练方法甚至可能有助于语言任务。然而，DeepSeek-VL-7B 在数学（GSM8K）上出现了一定程度的下降，这说明尽管我们努力促进视觉与语言模态之间的协调，二者之间仍存在竞争关系。这可能归因于有限的模型容量（7B），而更大的模型或许能显著缓解这一问题。总体而言，DeepSeek-VL 在应对这些挑战的同时，力求实现语言能力下降最小化的目标。

4.3 人工评估

为了进一步探索 DeepSeek-VL 的能力，我们独立构建了一个用于人工评估的数据集。该数据集包含 100 个问题，划分为七个类别，每个类别涵盖特定任务。这些类别与任务与我们内部 SFT 数据的分

类体系相同，如表 3 所示。这一方法确保我们测试的任务具有通用性，并涵盖多模态模型的大多数使用场景。

此外，基于现有报告中描述的类别与任务，我们收集了相似的图像素材并设计了提示词。这些图像素材的来源包括免版税图像社区以及研究人员拍摄的照片。这一系统化的收集与提示词构建过程，确保我们的数据集既全面又能代表真实世界的多模态模型应用。

我们将 DeepSeek-VL-7B 与 InternLM-XComposer2-VL、CogVLM 和 GPT-4V 进行比较，如图 6 所示（我们还在附录 A 中提供了可视化结果）。GPT-4V 在大多数维度上表现优异。所有开源模型在逻辑推理上仍远远落后于 GPT-4V，这凸显了扩大大语言模型（LLM）规模的必要性。DeepSeek-VL-7B 在整体性能上取得更好的结果，在识别（Recognition）、转换（Conversion）与常识推理（Commonsense Reasoning）上接近 GPT-4V 的水平。

【图表】图 6 对 InternLM-XComposer2-VL（Dong et al., 2024）、CogVLM（Wang et al., 2023b）、DeepSeek-VL 与 GPT-4V（OpenAI, 2023b）的人工评估结果。（原文第 19 页）

此外，我们使用 GPT-4V 进行对比评估，在专为人工评估设计的 99 个测试样本上，评估 DeepSeek-VL 相对其他模型的性能。参照 Zheng et al. (2024) 的做法，我们向 GPT-4V 展示问题以及来自两个不同模型的回答，并要求 GPT-4V 判断哪一个更好，或宣布平局。结果表明，在多数情况下 GPT-4V 更偏好 DeepSeek-VL 的回答，因为 GPT-4V 倾向于给 DeepSeek-VL 的回答质量更高的评价。如图 7 所示，与开源多模态模型（包括 Fuyu-8B、CogVLM-17B 与 InternLM-XComposer2-VL）相比，DeepSeek-VL 在超过 60% 的实例中被判定为更优。此外，与其他专有模型（例如 GPT-4V 本身）相比，DeepSeek-VL 也展现出相当出色的性能。

【图表】图 7 基于 GPT-4V 的 DeepSeek-VL 与其他模型的评估结果：该图展示了基于 GPT-4V、在 99 个测试样本上的评估结果，表明 DeepSeek-VL 相对开源与专有模型均取得有利结果。（原文第 19 页）

4.4 消融研究

扩大投影器训练规模（Scale Up Projector Training）：我们扩大了第一阶段（投影器预热）的数据集，随后进行监督微调。如图 8 所示，结果表明增加训练数据量并不能提升该阶段的性能。这意味着投影器的容量本身受到限制，无法捕捉多模态任务所需的广泛知识。

第一阶段训练	MMB	MMC	SEED	POPE	MMMU	平均

步数						
2K	59.0	54.0	61.8	82.3	30.3	57.5
8K	58.0	45.0	58.5	84.9	29.2	55.1
20K	56.0	52.3	59.0	81.7	28.6	55.5
80K	58.1	55.0	58.6	78.6	27.9	55.6

表 8 扩大第一阶段数据规模时的直接 SFT 性能对比结果。结果表明，在该阶段扩大数据规模并不会带来收益，甚至会导致更差的性能。

训练阶段（Training Stage）：在表 9 中，我们考察了各阶段对模型性能的贡献。显然，组合第一阶段、第二阶段与第三阶段在所有指标上都显著优于仅组合第一阶段与第三阶段，这证明了多模态预训练的有效性。此外，组合第二阶段与第三阶段的表现仍略逊于第一、第二、第三阶段组合后的性能，说明视觉-语言适配器预热阶段仍然是有意义的。

第一阶段	第二阶段	第三阶段	MMB	MMC	SEED	POPE	MMMU	平均
✓		✓	59.4	54.2	61.4	82.5	29.2	57.4
	✓	✓	63.4	60.5	65.9	87.1	31.8	61.7
✓	✓	✓	64.3	61.3	66.7	87.6	32.2	62.4

表 9 模型在各训练阶段的性能分析。

模态分组训练（Modality Group Training）：在混合语言与多模态数据时，我们观察到在批次层面直接混合它们会显著降低训练效率。这种低效源于每个批次的梯度反向传播过程都需要等待最慢的样本完成。结果，处理速度更快的纯语言数据不得不等待多模态样本完成，从而导致整体训练效率下降。

为解决这一问题，我们尝试在每个全局步对数据按模态进行分组，分别采样不同的模态。该方法通过组织训练数据，使得在不同训练步中，批次要么完全由语言数据组成，要么完全由多模态数据组成，而不是在同一批次内混合它们。

结果如图 8 所示，我们观察到该方法不会损害模型性能，同时将模型训练效率提升了 20%。这一策略有效规避了不同模态处理时间差异所带来的瓶颈，优化了训练流程。

【图表】图 8 模态预热在语言（Pile-test）与多模态（MMBench 与 MMBench_CN）基准上的对比分析表明：在语言任务上，按模态分组始终优于不分组的方法，同时在训练第二阶段（多模态:语言=60%:40%）保持多模态任务的性能。（原文第 20 页）

模态预热（Modality Warmup）：考虑到我们的方法是在语言模型的基础上进行多模态训练，从一开始就以固定比例直接混合多模态数据可能会使模型变得不稳定。为应对这一问题，我们提出了一种简单而有效的模态预热（warm-up）策略。最初，我们将语言数据比例设为 1，然后逐步降低到最终模型训练的目标比例（例如 0.7）。

如图 9 所示，我们的实验表明，该策略有效避免了训练初期语言能力的显著下降，同时在训练后期于语言与多模态两个领域都取得了相对更优的结果。这种渐进式适应使模型能够更平滑地调整以适应多模态数据的引入，从而提升整体训练稳定性与性能。

【图表】图 9 模态预热在语言（Pile-test）与多模态（MMBench 与 MMBench_CN）基准上的对比性能结果。在训练第二阶段（多模态:语言=60%:40%）的所有评估任务上，模态预热始终达到或超过不使用模态预热的方法。（原文第 21 页）

视觉编码器选择（Vision Encoder Selection）：为了更好地获取与利用图像信息，我们在相同训练设置下比较了不同视觉编码器的训练损失，唯一的例外是出于效率考虑将第二阶段的训练步数减少到 8000。如图 10 所示，我们发现引入仅视觉的自监督编码器能够显著改善训练损失的表现。为了更有效地处理高分辨率图像，我们的研究最终采用混合视觉编码器策略，将 SigLIP 与 SAM 结合用于我们的模型实现。

【图表】图 10 不同视觉编码器在第二阶段训练损失上的对比分析。（原文第 22 页）

视觉-语言适配器设计（Vision-Language Adaptor Design）：为在遵守当前 token 长度限制的同时提升从视觉编码器提取信息的效率，可以从两个主要方面对视觉-语言适配器进行调整：视觉特征的组合方式，以及 MLP 适配器的设计。

先前的研究（Tong et al., 2024）表明，沿序列维度组合视觉特征可以带来更好的模型性能，但这会因视觉特征 token 序列变长而增加计算需求，构成权衡。如表 10 上半部分所示，在序列拼接之前沿图像的宽度或高度维度堆叠视觉特征（以保持序列长度不变）以缩短序列长度，在大多数指标上并不比简单地在嵌入维度上合并它们取得更好的结果。在适配器架构方面，为每个视觉特征编码器采用独立的 MLP 适配器，可以更精确地调整视觉特征的具体数值与分布模式，从而使模型训练更平滑。相反，对不同视觉编码器使用共享的 MLP 适配器有助于充分的特征融合。我们采用混合策略，并报告了稳定且有所提升的性能，如表 10 下半部分所示。

【表格】表 10 以 SigLIP 与 SAM 作为混合视觉编码器时不同适配器架构的比较，序列拼接实验使用 Hybrid MLP。加粗条目表示最佳结果，带下划线的条目表示次佳结果。计算平均分时，我们将 OCRBench 的得分除以总题数。

（原文第 22 页）

5 结论、局限与未来工作

在本技术报告中，我们介绍了 DeepSeek-VL——一系列多模态大语言模型，提供 1.3B 与 6.7B 参数规模。本报告揭示了主流的基于投影器的预训练方法所固有的局限，为 DeepSeek-VL 所采用的创新方法奠定了基础。通过优先进行视觉与语言（VL）联合预训练阶段，DeepSeek-VL 超越了传统模型，确保多模态数据的整合不会损害大语言模型（LLM）的语言能力。这通过策略性的预热数据比例与混合视觉编码器的引入来实现，二者共同实现了对高分辨率图像的高效处理，同时不丢失语义的丰富性。

混合视觉编码器的引入——它能够在受限的 token 预算内处理 1024 × 1024 的图像——彰显了我们致力于在各类任务中保留细微细节与语义完整性的承诺。因此，DeepSeek-VL 成为一个开创性的模型，不仅达到而且超越了同类通用模型所设定的标准。它在广泛的以视觉为中心的基准上展现出卓越性能，同时在以语言为中心的评估中保持了强大的能力。

通过公开 DeepSeek-VL，我们期望推动研究社区内的进一步创新与探索，为后续研究提供坚实的基础。这一开放举措旨在促进我们在理解与处理多模态数据方面能力与认知的共同进步。

展望未来，我们很高兴地宣布，我们计划将 DeepSeek-VL 扩展到更大的规模，并引入混合专家（Mixture of Experts, MoE）技术。这一即将到来的扩展有望进一步提升模型的效率与效果，为 AI 领域的研究与应用开辟新的视野。

附录

附录 A：视觉问答示例（Prompt/Response 对照，见英文原文第 30–33 页）

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] high-resolution images without losing sight of semantic richness.

[2] The incorporation of a hybrid vision encoder, capable of handling 1024 x 1024 images within a constrained token budget, underscores our commitment to preserving the nuanced details and semantic integrity across diverse tasks. As a result, DeepSeek-VL emerges as a pioneering model that not only meets but exceeds the standards set by generalist models in its class. It showcases exceptional performance across a wide range of visually-centric benchmarks while sustaining formidable proficiency in language-centric evaluations.

[3] In making DeepSeek-VL publicly available, we aim to catalyze further innovation and exploration within the research community, providing a robust foundation upon which future studies can build. This gesture of openness is intended to facilitate the collective advancement of our understanding and capabilities in handling multimodal data.

[4] Looking ahead, we are excited to announce plans to scale up DeepSeek-VL to larger sizes, incorporating Mixture of Experts (MoE) technology. This forthcoming expansion promises to further enhance the model's efficiency and effectiveness, opening up new horizons for research and application in the field of AI.

[5] 01-ai. Yi-34B vision language model. https://huggingface.co/01-ai/Yi-VL-34B , 2024.

[6] Abi. Screenshot to code. https://github.com/abi/screenshot-to-code , 2024.

[7] Anna's Archive. Anna's archive. https://annas-archive.org/ , 2024.

[8] Anthropic. Introducing Claude, 2023. URL https://www.anthropic.com/index/introd ucing-claude .

[9] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

[10] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P . Wang, J. Lin, C. Zhou, and J. Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond. arXiv preprint arXiv:2308.12966, 2023.

[11] R. Bavishi, E. Elsen, C. Hawthorne, M. Nye, A. Odena, A. Somani, and S. Ta¸ sırlar. Introducing our multimodal models, 2023. URL https://www.adept.ai/blog/fuyu-8b .

[12] L. Blecher. Latex-ocr. GitHub repository, 2024. URL https://github.com/lukas-blecher /LaTeX-OCR .

[13] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic. Nougat: Neural optical understanding for academic documents. arXiv preprint arXiv:2308.13418, 2023.

[14] A. Burns, K. Srinivasan, J. Ainslie, G. Brown, B. A. Plummer, K. Saenko, J. Ni, and M. Guo. A suite of generative tasks for multi-level multimodal webpage understanding. In The 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. URL https://openreview.net/forum?id=rwcLHjtUmn .

[15] J. Carter. Textocr-gpt4v. https://huggingface.co/datasets/jimmycarter/textocr-g pt4v , 2024. L. Chen, J. Li, X. Dong, P . Zhang, C. He, J. Wang, F. Zhao, and D. Lin. Sharegpt4v: Improving large multi-modal models with better captions. arXiv preprint arXiv:2311.12793, 2023.

[16] C. K. Chng, Y. Liu, Y. Sun, C. C. Ng, C. Luo, Z. Ni, C. Fang, S. Zhang, J. Han, E. Ding, et al. Icdar2019 robust reading challenge on arbitrary-shaped text-rrc-art. In 2019 International Conference on Document Analysis and Recognition (ICDAR), pages 1571–1576. IEEE, 2019.

[17] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[18] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P . Fung, and S. Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning, 2023.

[19] DeepSeek-AI. Deepseek llm: Scaling open-source language models with longtermism. arXiv preprint arXiv:2401.02954, 2024. URL https://github.com/deepseek-ai/DeepSeek-L LM .

[20] X. Dong, P . Zhang, Y. Zang, Y. Cao, B. Wang, L. Ouyang, X. Wei, S. Zhang, H. Duan, M. Cao, et al. Internlm-xcomposer2: Mastering free-form text-image composition and comprehension in vision-language large model. arXiv preprint arXiv:2401.16420, 2024.

[21] echo840. Detailed caption dataset. https://huggingface.co/datasets/echo840/Deta iled_Caption , 2024.

[22] W. Foundation. Wikimedia downloads. URL https://dumps.wikimedia.org .

[23] J. Gao, R. Pi, J. Zhang, J. Ye, W. Zhong, Y. Wang, L. Hong, J. Han, H. Xu, Z. Li, et al. Gllava: Solving geometric problem with multi-modal large language model. arXiv preprint arXiv:2312.11370, 2023.

[24] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

[25] Google. An important next step on our AI journey, 2023. URL https://blog.google/tech nology/ai/bard-google-ai-search-updates/ .

[26] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[27] High-flyer. Hai-llm: 高 效 且 轻 量 的 大 模 型 训 练 工 具 , 2023. URL https://www.high-flyer.c n/en/blog/hai-llm .

[28] Y.-C. Hsiao, F. Zubach, M. Wang, et al. Screenqa: Large-scale question-answer pairs over mobile app screenshots. arXiv preprint arXiv:2209.08199, 2022.

[29] A. Hu, Y. Shi, H. Xu, J. Ye, Q. Ye, M. Yan, C. Li, Q. Qian, J. Zhang, and F. Huang. mplugpaperowl: Scientific diagram analysis with the multimodal large language model. arXiv preprint arXiv:2311.18248, 2023.

[30] HuggingFaceM4. Websight dataset. https://huggingface.co/datasets/HuggingFaceM 4/WebSight , 2024. S. Kantharaj, R. T. Leong, X. Lin, A. Masry, M. Thakkar, E. Hoque, and S. Joty. Chart-totext: A large-scale benchmark for chart summarization. In S. Muresan, P . Nakov, and A. Villavicencio, editors, Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 4005–4023, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.277. URL https://aclanthology.org/2022.acl-long.277 .

[31] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al. Segment anything. arXiv preprint arXiv:2304.02643, 2023.

[32] D. Kocetkov, R. Li, L. B. Allal, J. Li, C. Mou, C. M. Ferrandis, Y. Jernite, M. Mitchell, S. Hughes, T. Wolf, D. Bahdanau, L. von Werra, and H. de Vries. The stack: 3 tb of permissively licensed source code. In Transactions on Machine Learning Research, 2023.

[33] V . A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro. Reducing activation recomputation in large transformer models. Proceedings of Machine Learning and Systems, 5, 2023.

[34] I. Krylov, S. Nosov, and V . Sovrasov. Open images v5 text annotation and yet another mask text spotter. In Asian Conference on Machine Learning, pages 379–389. PMLR, 2021.

[35] A. Kulkarni and J. Truelsen. wkhtmltopdf. https://wkhtmltopdf.org/ . Project maintained by Ashish Kulkarni, originally created by Jakob Truelsen. Accessed: 2024-02-22.

[36] LAION. Gpt-4v dataset. https://huggingface.co/datasets/laion/gpt4v-dataset , 2023.

[37] B. Li, R. Wang, G. Wang, Y. Ge, Y. Ge, and Y. Shan. Seed-bench: Benchmarking multimodal llms with generative comprehension. arXiv preprint arXiv:2307.16125, 2023a.

[38] S. Li and N. Tajbakhsh. Scigraphqa: A large-scale synthetic multi-turn question-answering dataset for scientific graphs, 2023.

[39] Y. Li, G. Li, L. He, J. Zheng, H. Li, and Z. Guan. Widget captioning: Generating natural language description for mobile user interface elements. arXiv preprint arXiv:2010.04295, 2020.

[40] Y. Li, H. Mao, R. Girshick, and K. He. Exploring plain vision transformer backbones for object detection. In European Conference on Computer Vision, pages 280–296. Springer, 2022.

[41] Y. Li, Y. Du, K. Zhou, J. Wang, W. X. Zhao, and J.-R. Wen. Evaluating object hallucination in large vision-language models. arXiv preprint arXiv:2305.10355, 2023b.

[42] J. Lin, H. Yin, W. Ping, Y. Lu, P . Molchanov, A. Tao, H. Mao, J. Kautz, M. Shoeybi, and S. Han. Vila: On pre-training for visual language models. arXiv preprint arXiv:2312.07533, 2023a.

[43] Z. Lin, C. Liu, R. Zhang, P . Gao, L. Qiu, H. Xiao, H. Qiu, C. Lin, W. Shao, K. Chen, et al. Sphinx: The joint mixing of weights, tasks, and visual embeddings for multi-modal large language models. arXiv preprint arXiv:2311.07575, 2023b.

[44] F. Liu, F. Piccinno, S. Krichene, C. Pang, K. Lee, M. Joshi, Y. Altun, N. Collier, and J. M. Eisenschlos. Matcha: Enhancing visual language pretraining with math reasoning and chart derendering. arXiv preprint arXiv:2212.09662, 2022a.

[45] H. Liu, C. Li, Y. Li, B. Li, Y. Zhang, S. Shen, and Y. J. Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024a. URL https://llava-vl.github.io/blog/202 4-01-30-llava-next/ . H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. Advances in neural information processing systems, 36, 2024b.

[46] Y. Liu, G. Zhu, B. Zhu, Q. Song, G. Ge, H. Chen, G. Qiao, R. Peng, L. Wu, and J. Wang. Taisu: A 166m large-scale high-quality dataset for chinese vision-language pre-training. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems, volume 35, pages 16705–16717. Curran Associates, Inc., 2022b. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/6 a386d703b50f1cf1f61ab02a15967bb-Paper-Datasets_and_Benchmarks.pdf .

[47] Y. Liu, H. Duan, Y. Zhang, B. Li, S. Zhang, W. Zhao, Y. Yuan, J. Wang, C. He, Z. Liu, et al. Mmbench: Is your multi-modal model an all-around player? arXiv preprint arXiv:2307.06281, 2023a.

[48] Y. Liu, Z. Li, H. Li, W. Yu, M. Huang, D. Peng, M. Liu, M. Chen, C. Li, L. Jin, et al. On the hidden mystery of ocr in large multimodal models. arXiv preprint arXiv:2305.07895, 2023b.

[49] S. Long, S. Qin, D. Panteleev, A. Bissacco, Y. Fujii, and M. Raptis. Towards end-to-end unified scene text detection and layout analysis. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.

[50] P . Lu, L. Qiu, J. Chen, T. Xia, Y. Zhao, W. Zhang, Z. Yu, X. Liang, and S.-C. Zhu. Iconqa: A new benchmark for abstract diagram understanding and visual language reasoning. arXiv preprint arXiv:2110.13214, 2021.

[51] P . Lu, S. Mishra, T. Xia, L. Qiu, K.-W. Chang, S.-C. Zhu, O. Tafjord, P . Clark, and A. Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. In The 36th Conference on Neural Information Processing Systems (NeurIPS), 2022a.

[52] P . Lu, S. Mishra, T. Xia, L. Qiu, K.-W. Chang, S.-C. Zhu, O. Tafjord, P . Clark, and A. Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. Advances in Neural Information Processing Systems, 35:2507–2521, 2022b.

[53] P . Lu, H. Bansal, T. Xia, J. Liu, C. Li, H. Hajishirzi, H. Cheng, K.-W. Chang, M. Galley, and J. Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. arXiv preprint arXiv:2310.02255, 2023.

[54] J. Mao, J. Huang, A. Toshev, O. Camburu, A. L. Yuille, and K. Murphy. Generation and comprehension of unambiguous object descriptions. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 11–20, 2016.

[55] A. Masry, P . Kavehzadeh, X. L. Do, E. Hoque, and S. Joty. Unichart: A universal vision-language pretrained model for chart comprehension and reasoning. arXiv preprint arXiv:2305.14761, 2023.

[56] D. Narayanan, M. Shoeybi, J. Casper, P . LeGresley, M. Patwary, V . Korthikanti, D. Vainbrand, P . Kashinkunti, J. Bernauer, B. Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–15, 2021.

[57] N. Nayef, F. Yin, I. Bizid, H. Choi, Y. Feng, D. Karatzas, Z. Luo, U. Pal, C. Rigaud, J. Chazalon, et al. Icdar2017 robust reading challenge on multi-lingual scene text detection and script identification-rrc-mlt. In 2017 14th IAPR international conference on document analysis and recognition (ICDAR), volume 1, pages 1454–1459. IEEE, 2017. OpenAI. Chatgpt: Optimizing language models for dialogue. 2022. URL https://openai.c om/blog/chatgpt .

[58] arXiv, 2023a. OpenAI. GPT-4 technical report.

[59] R. OpenAI. Gpt-4v(ision) system card. 2023b.

[60] J. A. Rodriguez, D. Vazquez, I. Laradji, M. Pedersoli, and P . Rodriguez. Ocr-vqgan: Taming text-within-image generation. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pages 3689–3698, 2023.

[61] R. Schaeffer, B. Miranda, and S. Koyejo. Are emergent abilities of large language models a mirage? Advances in Neural Information Processing Systems, 36, 2024.

[62] N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.

[63] B. Shi, C. Yao, M. Liao, M. Yang, P . Xu, L. Cui, S. Belongie, S. Lu, and X. Bai. Icdar2017 competition on reading chinese text in the wild (rctw-17). In 2017 14th iapr international conference on document analysis and recognition (ICDAR), volume 1, pages 1429–1434. IEEE, 2017.

[64] M. Shoeybi, M. Patwary, R. Puri, P . LeGresley, J. Casper, and B. Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053, 2019.

[65] A. Singh, G. Pang, M. Toh, J. Huang, W. Galuba, and T. Hassner. Textocr: Towards largescale end-to-end reasoning for arbitrary-shaped scene text. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 8802–8812, 2021.

[66] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.

[67] Q. Sun, Q. Yu, Y. Cui, F. Zhang, X. Zhang, Y. Wang, H. Gao, J. Liu, T. Huang, and X. Wang. Generative pretraining in multimodality. arXiv preprint arXiv:2307.05222, 2023.

[68] Y. Sun, Z. Ni, C.-K. Chng, Y. Liu, C. Luo, C. C. Ng, J. Han, E. Ding, J. Liu, D. Karatzas, et al. Icdar 2019 competition on large-scale street view text with partial labeling-rrc-lsvt. In 2019 International Conference on Document Analysis and Recognition (ICDAR), pages 1557–1562. IEEE, 2019.

[69] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, et al. Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805, 2023.

[70] S. Tong, Z. Liu, Y. Zhai, Y. Ma, Y. LeCun, and S. Xie. Eyes wide shut? exploring the visual shortcomings of multimodal llms. arXiv preprint arXiv:2401.06209, 2024.

[71] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023a.

[72] H. Touvron, L. Martin, K. Stone, P . Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P . Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton-Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V . Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V . Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P . S. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P . Mishra, I. Molybog, Y. Nie, A. Poulton, J.

Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P . Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models. CoRR, abs/2307.09288, 2023b. doi: 10.48550/arXiv.2307.09288. URL https://doi.org/10.48550/arXiv.2307. 09288 .

[73] A. Veit, T. Matera, L. Neumann, J. Matas, and S. Belongie. Coco-text: Dataset and benchmark for text detection and recognition in natural images. arXiv preprint arXiv:1601.07140, 2016.

[74] B. Wang, G. Li, X. Zhou, Z. Chen, T. Grossman, and Y. Li. Screen2words: Automatic mobile ui summarization with multimodal learning. In The 34th Annual ACM Symposium on User Interface Software and Technology, pages 498–510, 2021.

[75] J. Wang, L. Meng, Z. Weng, B. He, Z. Wu, and Y.-G. Jiang. To see is to believe: Prompting gpt-4v for better visual instruction tuning. arXiv preprint arXiv:2311.07574, 2023a.

[76] W. Wang, Q. Lv, W. Yu, W. Hong, J. Qi, Y. Wang, J. Ji, Z. Yang, L. Zhao, X. Song, et al. Cogvlm: Visual expert for pretrained language models. arXiv preprint arXiv:2311.03079, 2023b.

[77] H. Wei, L. Kong, J. Chen, L. Zhao, Z. Ge, J. Yang, J. Sun, C. Han, and X. Zhang. Vary: Scaling up the vision vocabulary for large vision-language models. arXiv preprint arXiv:2312.06109, 2023.

[78] Y. Yang, A. Panagopoulou, Q. Lyu, L. Zhang, M. Yatskar, and C. Callison-Burch. Visual goal-step inference using wikihow. arXiv preprint arXiv:2104.05845, 2021.

[79] J. Ye, A. Hu, H. Xu, Q. Ye, M. Yan, G. Xu, C. Li, J. Tian, Q. Qian, J. Zhang, et al. Ureader: Universal ocr-free visually-situated language understanding with multimodal large language model. arXiv preprint arXiv:2310.05126, 2023.

[80] Q. Yu, Q. Sun, X. Zhang, Y. Cui, F. Zhang, Y. Cao, X. Wang, and J. Liu. Capsfusion: Rethinking image-text data at scale. arXiv preprint arXiv:2310.20550, 2023a.

[81] W. Yu, Z. Yang, L. Li, J. Wang, K. Lin, Z. Liu, X. Wang, and L. Wang. Mm-vet: Evaluating large multimodal models for integrated capabilities. arXiv preprint arXiv:2308.02490, 2023b.

[82] X. Yue, Y. Ni, K. Zhang, T. Zheng, R. Liu, G. Zhang, S. Stevens, D. Jiang, W. Ren, Y. Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. arXiv preprint arXiv:2311.16502, 2023.

[83] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[84] B. Zhang and R. Sennrich. Root mean square layer normalization. Advances in Neural Information Processing Systems, 32, 2019.

[85] G. Zhang, X. Du, B. Chen, Y. Liang, T. Luo, T. Zheng, K. Zhu, Y. Cheng, C. Xu, S. Guo, et al. Cmmmu: A chinese massive multi-discipline multimodal understanding benchmark. arXiv preprint arXiv:2401.11944, 2024. R. Zhang, Y. Zhou, Q. Jiang, Q. Song, N. Li, K. Zhou, L. Wang, D. Wang, M. Liao, M. Yang, et al. Icdar 2019 robust reading challenge on reading chinese text on signboard. In 2019 international conference on document analysis and recognition (ICDAR), pages 1577–1581. IEEE, 2019.

[86] Y. Zhang, L. Gueguen, I. Zharkov, P . Zhang, K. Seifert, and B. Kadlec. Uber-text: A largescale dataset for optical character recognition from street-level imagery. In SUNw: Scene Understanding Workshop - CVPR 2017, Hawaii, U.S.A., 2017. URL http://sunw.csail.m it.edu/abstract/uberText.pdf .

[87] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. Advances in Neural Information Processing Systems, 36, 2024.

[88] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364 .

[89] W. Zhu, J. Hessel, A. Awadalla, S. Y. Gadre, J. Dodge, A. Fang, Y. Yu, L. Schmidt, W. Y. Wang, and Y. Choi. Multimodal c4: An open, billion-scale corpus of images interleaved with text. Advances in Neural Information Processing Systems, 36, 2024.

DeepSeek-V2：一个强大、经济且高效的混合专家语言模型

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model

DeepSeek-AI　research@deepseek.com

arXiv:2405.04434 · 中文翻译版（正文全文；图表保留原文页码位置）

摘　要

我们提出 DeepSeek-V2，一个以训练经济性与推理高效性为特点的强大混合专家（MoE）语言模型。它总参数量为 236B，每个 token 激活其中 21B，并支持 128K token 的上下文长度。DeepSeek-V2 采用多头潜在注意力（MLA）与 DeepSeekMoE 等创新架构。MLA 通过将键值（KV）缓存显著压缩为潜在向量来保证高效推理，而 DeepSeekMoE 通过稀疏计算使模型能以经济的成本训练出强大的模型。与 DeepSeek 67B 相比，DeepSeek-V2 的性能显著更强，同时节省 42.5% 的训练成本，将 KV 缓存减少 93.3%，并将最大生成吞吐提升至 5.76 倍。我们在由 8.1T token 构成的高质量多源语料上预训练 DeepSeek-V2，并进一步进行监督微调（SFT）与强化学习（RL），以充分释放其潜力。评测结果表明，即使仅有 21B 激活参数，DeepSeek-V2 及其对话版本仍在开源模型中达到顶级性能。模型检查点见 https://github.com/deepseek-ai/DeepSeek-V2。

【图表】图 1 (a) 不同开源模型的 MMLU 准确率随激活参数量的变化；(b) DeepSeek 67B（稠密）与 DeepSeek-V2 的训练成本与推理效率（原文第 1 页）

1 引言

过去几年间，大语言模型（Large Language Model, LLM）（Anthropic，2023；Google，2023；OpenAI，2022、2023）经历了快速发展，让人们得以一窥通用人工智能（Artificial General Intelligence,

AGI）的曙光。一般而言，LLM 的智能水平往往随参数量的增加而提升，使其能够在各类任务中展现出涌现能力（Wei 等，2022）。然而，这种提升的代价是训练需要更大的计算资源，且推理吞吐可能下降。这些约束构成了重大挑战，阻碍了 LLM 的广泛采用与应用。为解决这一问题，我们提出了 DeepSeek-V2——一个强大的开源混合专家（Mixture-of-Experts, MoE）语言模型，它通过创新的 Transformer 架构，实现了经济的训练与高效的推理。其总参数量为 236B，每个 token 激活 21B 参数，并支持 128K token 的上下文长度。

我们在 Transformer 框架（Vaswani 等，2017）内，用我们提出的多头潜在注意力（Multi-head Latent Attention, MLA）与 DeepSeekMoE 对注意力模块与前馈网络（Feed-Forward Network, FFN）进行优化。（1）在注意力机制方面，多头注意力（Multi-Head Attention, MHA）（Vaswani 等，2017）的键值（Key-Value, KV）缓存是 LLM 推理效率的一大障碍。为应对这一问题，研究者探索了多种方法，包括分组查询注意力（Grouped-Query Attention, GQA）（Ainslie 等，2023）与多查询注意力（Multi-Query Attention, MQA）（Shazeer，2019）。然而，这些方法在试图减少 KV 缓存时往往会牺牲性能。为了两全其美，我们引入了 MLA——一种配备低秩键值联合压缩的注意力机制。实证上，MLA 取得了优于 MHA 的性能，同时显著减少了推理期间的 KV 缓存，从而提升了推理效率。（2）在前馈网络方面，我们沿用 DeepSeekMoE 架构（Dai 等，2024），该架构采用细粒度专家分割与共享专家隔离，以挖掘更高的专家专业化潜力。与传统 MoE 架构（如 GShard（Lepikhin 等，2021））相比，DeepSeekMoE 架构展现出巨大优势，使我们能够以经济成本训练强大的模型。由于训练时采用专家并行，我们还设计了补充机制来控制通信开销并确保负载均衡。

通过结合这两项技术，DeepSeek-V2 同时兼具强大的性能（图 1(a)）、经济的训练成本与高效的推理吞吐（图 1(b)）。

我们构建了一个高质量、多源的预训练语料，包含 8.1T token。与我们的上一代模型 DeepSeek 67B（DeepSeek-AI，2024）所使用的语料相比，该语料数据量更大（尤其是中文数据），且数据质量更高。我们首先在完整的预训练语料上预训练 DeepSeek-V2。随后，我们收集了 150 万条对话会话，涵盖数学、代码、写作、推理、安全等多个领域，用于对 DeepSeek-V2 Chat 进行监督微调（Supervised Fine-Tuning, SFT），得到 DeepSeek-V2 Chat（SFT）。最后，我们遵循 DeepSeekMath（Shao 等，2024）的做法，采用组相对策略优化（Group Relative Policy Optimization, GRPO）进一步使模型与人类偏好对齐，得到 DeepSeek-V2 Chat（RL）。

我们在中英文的广泛基准上评测 DeepSeek-V2，并将其与有代表性的开源模型进行比较。评测结果表明，即便仅有 21B 激活参数，DeepSeek-V2 仍在开源模型中达到顶级性能，成为最强的开源 MoE

语言模型。图 1(a) 突出显示，在 MMLU 上，DeepSeek-V2 仅以少量激活参数就取得了名列前茅的性能。此外，如图 1(b) 所示，与 DeepSeek 67B 相比，DeepSeek-V2 节省了 42.5% 的训练成本，将 KV 缓存减少了 93.3%，并将最大生成吞吐提升至 5.76 倍。

【图表】图 2 DeepSeek-V2 的架构示意图。MLA 通过显著缩减生成时的 KV 缓存来确保高效推理，DeepSeekMoE 则通过稀疏架构使我们能够以经济成本训练强大的模型（原文第 5 页）

我们还在开放式基准上评测了 DeepSeek-V2 Chat（SFT）与 DeepSeek-V2 Chat（RL）。值得注意的是，DeepSeek-V2 Chat（RL）在 AlpacaEval 2.0（Dubois 等，2024）上取得 38.9 的长度控制胜率，在 MT-Bench（Zheng 等，2023）上取得 8.97 的总分，在 AlignBench（Liu 等，2023）上取得 7.91 的总分。英文开放式对话评测表明，DeepSeek-V2 Chat（RL）在开源对话模型中具有顶级性能。此外，AlignBench 上的评测表明，在中文方面，DeepSeek-V2 Chat（RL）优于所有开源模型，甚至击败了大多数闭源模型。

为促进 MLA 与 DeepSeekMoE 的进一步研究与开发，我们还面向开源社区发布了 DeepSeek-V2-Lite——一个配备 MLA 与 DeepSeekMoE 的较小模型。其总参数量为 15.7B，每个 token 激活 2.4B 参数。关于 DeepSeek-V2-Lite 的详细描述见附录 B。

在本文其余部分，我们首先详细介绍 DeepSeek-V2 的模型架构（第 2 节）。随后，我们介绍预训练方面的工作，包括训练数据构建、超参数设置、基础设施、长上下文扩展，以及模型性能与效率的评测（第 3 节）。接着，我们展示在对齐方面的工作，包括监督微调（SFT）、强化学习（Reinforcement Learning, RL）、评测结果与其他讨论（第 4 节）。最后，我们总结结论，讨论 DeepSeek-V2 当前的局限性，并展望未来工作（第 5 节）。

2 架构

总体而言，DeepSeek-V2 仍采用 Transformer 架构（Vaswani 等，2017），其中每个 Transformer 块由一个注意力模块与一个前馈网络（FFN）组成。然而，对于注意力模块与 FFN，我们都设计并采用了创新的架构。在注意力方面，我们设计了 MLA，它利用低秩键值联合压缩来消除推理时键值缓存的瓶颈，从而支持高效推理。在 FFN 方面，我们采用 DeepSeekMoE 架构（Dai 等，2024），这是一种高性能 MoE 架构，使我们能够以经济成本训练强大的模型。图 2 给出了 DeepSeek-V2 架构的示意

图，本节将介绍 MLA 与 DeepSeekMoE 的细节。对于其他细节（例如层归一化与 FFN 中的激活函数），除非特别说明，DeepSeek-V2 均沿用 DeepSeek 67B（DeepSeek-AI，2024）的设置。

2.1 多头潜在注意力：提升推理效率

传统 Transformer 模型通常采用多头注意力（MHA）（Vaswani 等，2017），但在生成过程中，其庞大的键值（KV）缓存会成为限制推理效率的瓶颈。为了减少 KV 缓存，多查询注意力（MQA）（Shazeer，2019）与分组查询注意力（GQA）（Ainslie 等，2023）被提出。它们所需的 KV 缓存规模更小，但性能不及 MHA（我们在附录 D.1 中给出了 MHA、GQA 与 MQA 的消融实验）。

针对 DeepSeek-V2，我们设计了一种创新的注意力机制，称为多头潜在注意力（MLA）。MLA 配备低秩键值联合压缩，取得了优于 MHA 的性能，却只需要显著更少的 KV 缓存。下面我们介绍其架构，并在附录 D.2 中给出 MLA 与 MHA 的对比。

2.1.1 预备知识：标准多头注意力

我们首先介绍标准的 MHA 机制作为背景。令 d 为嵌入维度，n_h 为注意力头数量，d_h 为每个头的维度，h_t ∈ R^d 为第 t 个 token 在某一注意力层的注意力输入。标准 MHA 首先通过三个矩阵 W^Q、W^K、W^V ∈ R^{d_h n_h × d} 分别生成 q_t、k_t、v_t ∈ R^{d_h n_h}：

$$q_t = W^Q h_t \quad (1)$$

$$k_t = W^K h_t \quad (2)$$

$$v_t = W^V h_t \quad (3)$$

【图表】图 3 多头注意力（MHA）、分组查询注意力（GQA）、多查询注意力（MQA）与多头潜在注意力（MLA）的简化示意图。通过将键与值联合压缩为潜在向量，MLA 在推理期间显著缩减了 KV 缓存（原文第 7 页）

随后，q_t、k_t、v_t 将被切分为 n_h 个头，以进行多头注意力计算：

$$[q_{t,1}; q_{t,2}; ...; q_{t,n_h}] = q_t \quad (4)$$

$$[k_{t,1}; k_{t,2}; ...; k_{t,n_h}] = k_t \quad (5)$$

$$[v_{t,1}; v_{t,2}; ...; v_{t,n_h}] = v_t \quad (6)$$

$$o_{t,i} = \Sigma_{j=1}^{t} \mathrm{Softmax}_j(q_{t,i}^T k_{j,i} / \sqrt{(d_h)}) \cdot v_{j,i} \quad (7)$$

$$u_t = W^O [o_{t,1}; o_{t,2}; ...; o_{t,n_h}] \quad (8)$$

其中 q_{t,i}、k_{t,i}、v_{t,i} ∈ R^{d_h} 分别表示第 i 个注意力头的查询、键与值；W^O ∈ R^{d×d_h n_h} 表示输出投影矩阵。推理时，所有键与值都需要被缓存以加速推理，因此 MHA 需要为每个 token 缓存 2 n_h d_h l 个元素。在模型部署中，这一庞大的 KV 缓存是限制最大批大小与序列长度的一大瓶颈。

2.1.2 低秩键值联合压缩

MLA 的核心是对键与值进行低秩联合压缩，以减少 KV 缓存：

c_t^{KV} = W^{DKV} h_t （9）

k_t^C = W^{UK} c_t^{KV} （10）

v_t^C = W^{UV} c_t^{KV} （11）

其中 c_t^{KV} ∈ R^{d_c} 是键与值的压缩潜在向量；d_c（≪ d_h n_h）表示 KV 压缩维度；W^{DKV} ∈ R^{d_c×d} 是下投影矩阵；W^{UK}、W^{UV} ∈ R^{d_h n_h×d_c} 分别是键与值的上投影矩阵。推理时，MLA 只需缓存 c_t^{KV}，因此其 KV 缓存仅有 d_c l 个元素，其中 l 表示层数。此外，推理期间由于 W^{UK} 可以吸收进 W^Q、W^{UV} 可以吸收进 W^O，我们甚至无需为注意力显式地计算出键与值。图 3 直观展示了 MLA 中的键值联合压缩如何缩减 KV 缓存。

此外，为了减少训练期间的激活内存，我们还进行，对查询的低秩压缩（low-rank compression），尽管这并不能减少 KV cache：

c^Q_t = W^DQ h_t （12）

q^C_t = W^UQ c^Q_t （13）

其中 c^Q_t ∈ R^{d'_c} 是查询的压缩潜在向量；d'_c（≪ d_h n_h）表示查询压缩维度；W^DQ ∈ R^{d'_c × d} 与 W^UQ ∈ R^{d_h n_h × d'_c} 分别是查询的下投影矩阵与上投影矩阵。

2.1.3 解耦的旋转位置嵌入

遵循 DeepSeek 67B（DeepSeek-AI，2024）的做法，我们打算在 DeepSeek-V2 中使用旋转位置嵌入（Rotary Position Embedding, RoPE）（Su 等，2024）。然而，RoPE 与低秩 KV 压缩不兼容。具体而言，RoPE 对键和查询都是位置敏感的。如果对键 k^C_t 施加 RoPE，那么公式（10）中的 W^UK 将与一个位置敏感的 RoPE 矩阵耦合。这样一来，在推理期间 W^UK 便无法再被吸收进 W^Q，因为与当前正在生成的 token 相关的 RoPE 矩阵会位于 W^Q 与 W^UK 之间，而矩阵乘法不满足交换律。因此，我们必须为所有前缀 token 重新计算键，这将显著阻碍推理效率。

作为一种解决方案，我们提出解耦的 RoPE 策略，使用额外的多头查询 q^R_{t,i} ∈ R^{d^R_h} 和一个共享键 k^R_t ∈ R^{d^R_h} 来承载 RoPE，其中 d^R_h 表示解耦查询与键的每头维度。借助解耦的 RoPE 策略，MLA 执行如下计算：

[q^R_{t,1}; q^R_{t,2}; ...; q^R_{t,n_h}] = q^R_t = RoPE(W^QR c^Q_t)　　（14）

k^R_t = RoPE(W^KR h_t)　　（15）

q_{t,i} = [q^C_{t,i}; q^R_{t,i}]　　（16）

k_{t,i} = [k^C_{t,i}; k^R_t]　　（17）

o_{t,i} = ∑_{j=1}^{t} Softmax_j(q^T_{t,i} k_{j,i} / √(d_h + d^R_h)) v^C_{j,i}　　（18）

u_t = W^O [o_{t,1}; o_{t,2}; ...; o_{t,n_h}]　　（19）

其中 W^QR ∈ R^{d^R_h n_h × d'_c} 与 W^KR ∈ R^{d^R_h × d} 分别是用于生成解耦查询与解耦键的矩阵；RoPE(·) 表示施加 RoPE 矩阵的操作；[·;·] 表示拼接操作。在推理期间，解耦键同样需要被缓存。因此，DeepSeek-V2 所需的总 KV cache 包含 (d_c + d^R_h) l 个元素。

为了完整展示 MLA 的计算过程，我们还在附录 C 中整理并给出了其完整公式。

2.1.4 键值缓存比较

我们在表 1 中展示了不同注意力机制之间每 token 的 KV cache 对比。MLA 只需少量 KV cache，相当于仅有 2.25 组的 GQA，但能够取得强于 MHA 的性能。

表 1：不同注意力机制之间每 token 的 KV cache 比较。 n_h 表示注意力头数，d_h 表示每个注意力头的维度，l 表示层数，n_g 表示 GQA 中的组数，d_c 与 d^R_h 分别表示 MLA 中的 KV 压缩维度以及解耦查询与键的每头维度。KV cache 的大小按元素数量衡量，与存储精度无关。对于 DeepSeek-V2，d_c 设为 4d_h，d^R_h 设为 d_h/2。因此，其 KV cache 仅相当于 2.25 组的 GQA，但性能强于 MHA。（原文第 9 页）

注意力机制	每 token 的 KV cache（元素数）	能力
多头注意力（MHA）	2 n_h d_h l	强
分组查询注意力（GQA）	2 n_g d_h l	中等
多查询注意力（MQA）	2 d_h l	弱
MLA（本文）	(d_c + d^R_h) l ≈ 9/2 · d_h l	更强

2.2 DeepSeekMoE：以经济成本训练强模型

2.2.1 基本架构

对于 FFN，我们采用 DeepSeekMoE 架构（Dai 等，2024）。DeepSeekMoE 有两个核心思想：将专家分割成更细的粒度，以获得更高的专家专业化程度与更准确的知识获取；以及隔离出部分共享专家，以缓解路由专家之间的知识冗余。在激活专家参数与总专家参数数量相同的情况下，DeepSeekMoE 能够大幅优于 GShard（Lepikhin 等，2021）等传统 MoE 架构。

设 u_t 为第 t 个 token 的 FFN 输入，我们按如下方式计算 FFN 输出 h'_t：

$$h'_t = u_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(u_t) + \sum_{i=1}^{N_r} g_{i,t}\, \mathrm{FFN}_i^{(r)}(u_t) \quad (20)$$

$$g_{i,t} = \begin{cases} s_{i,t}, & \text{若 } s_{i,t} \in \mathrm{Topk}(\{s_{j,t} \mid 1 \le j \le N_r\}, K_r) \\ 0, & \text{否则} \end{cases} \quad (21)$$

$$s_{i,t} = \mathrm{Softmax}_i(u_t^{\top} e_i) \quad (22)$$

其中 N_s 与 N_r 分别表示共享专家与路由专家的数量；FFN_i^{(s)}(·) 与 FFN_i^{(r)}(·) 分别表示第 i 个共享专家与第 i 个路由专家；K_r 表示被激活的路由专家数；g_{i,t} 是第 i 个专家的门控值；s_{i,t} 是 token 到专家的亲和度；e_i 是该层第 i 个路由专家的质心；Topk(·, K) 表示由第 t 个 token 与所有路由专家计算出的亲和度分数中得分最高的 K 个所构成的集合。

2.2.2 设备受限路由

我们设计了一种设备受限路由（device-limited routing）机制，以限制与 MoE 相关的通信开销。当采用专家并行时，路由专家会被分布到多个设备上。对每个 token 而言，其与 MoE 相关的通信频率正比于其目标专家所覆盖的设备数量。由于 DeepSeekMoE 采用细粒度专家分割，被激活的专家数量可能很大，因此如果我们采用专家并行，与 MoE 相关的通信代价会更高。对于 DeepSeek-V2，除了朴素的路由专家 top-K 选择之外，我们还额外确保每个 token 的目标专家最多分布在 M 个设备上。具体而言，对于每个 token，我们首先选出包含亲和度最高专家的 M 个设备，然后在这些设备上的专家中执行 top-K 选择。在实践中我们发现，当 $M \ge 3$ 时，设备受限路由能够取得与不受限 top-K 路由大致相当的良好性能。

2.2.3 负载均衡辅助损失

对于自动学习的路由策略，我们将负载均衡纳入考虑。第一，负载不均衡会提高路由崩溃（routing collapse）（Shazeer 等，2017）的风险，使部分专家无法得到充分的训练与利用。第二，当采用专家并行时，负载不均衡会降低计算效率。在 DeepSeek-V2 的训练中，我们设计了三种辅助损

失，分别用于控制专家级负载均衡（L_ExpBal）、设备级负载均衡（L_DevBal）与通信均衡（L_CommBal）。

专家级均衡损失：我们使用专家级均衡损失（Fedus 等，2021；Lepikhin 等，2021）来缓解路由崩溃的风险：

$$L_{ExpBal} = \alpha_1 \sum_{i=1}^{N_r} f_i P_i \quad (23)$$

$$f_i = \frac{N_r}{K_r T} \sum_{t=1}^{T} 1(\text{Token_t 选择 Expert_i}) \quad (24)$$

$$P_i = \frac{1}{T} \sum_{t=1}^{T} s_{i,t} \quad (25)$$

其中 α_1 是称为专家级均衡因子的超参数；$1(\cdot)$ 表示指示函数；T 表示一个序列中的 token 数。

设备级均衡损失：除了专家级均衡损失之外，我们还额外设计了设备级均衡损失，以确保不同设备之间的计算均衡。在 DeepSeek-V2 的训练过程中，我们将所有路由专家划分为 D 组 $\{E_1, E_2, ..., E_D\}$，并将每组部署在单个设备上。设备级均衡损失的计算如下：

$$L_{DevBal} = \alpha_2 \sum_{i=1}^{D} f'_i P'_i \quad (26)$$

$$f'_i = \frac{1}{|E_i|} \sum_{j \in E_i} f_j \quad (27)$$

$$P'_i = \sum_{j \in E_i} P_j \quad (28)$$

其中 α_2 是称为设备级均衡因子的超参数。

通信均衡损失：最后，我们引入通信均衡损失，以确保每个设备的通信保持均衡。尽管设备受限路由机制保证每个设备的发送通信量有界，但如果某个设备接收的 token 多于其他设备，实际通信效率同样会受到影响。为了缓解这一问题，我们设计了如下通信均衡损失：

$$L_{CommBal} = \alpha_3 \sum_{i=1}^{D} f''_i P''_i \quad (29)$$

$$f''_i = \frac{D}{M T} \sum_{t=1}^{T} 1(\text{Token_t 被发送至 Device_i}) \quad (30)$$

$$P''_i = \sum_{j \in E_i} P_j \quad (31)$$

其中 α_3 是称为通信均衡因子的超参数。设备受限路由机制遵循的原则是确保每个设备最多向其他设备发送 MT 个隐藏状态。同时，通信均衡损失被用来鼓励每个设备从其他设备接收大约 MT 个隐藏状态。通信均衡损失保证了设备之间信息的均衡交换，促进了高效通信。

2.2.4 Token 丢弃策略

尽管均衡损失旨在鼓励负载均衡，但必须承认，它们无法保证严格的负载均衡。为了进一步减少负载不均衡造成的计算浪费，我们在训练期间引入了设备级的 token 丢弃策略。该方法首先计算每个

设备的平均计算预算，这意味着每个设备的容量因子等价于 1.0。然后，受 Riquelme 等（2021）的启发，我们在每个设备上丢弃亲和度分数最低的 token，直到达到计算预算。此外，我们确保属于约 10% 训练序列的 token 永远不会被丢弃。通过这种方式，我们可以根据效率需求在推理期间灵活决定是否丢弃 token，并始终保证训练与推理之间的一致性。

3 预训练

3.1 实验设置

3.1.1 数据构建

在保持与 DeepSeek 67B（DeepSeek-AI，2024）相同的数据处理阶段的同时，我们扩展了数据量并提升了数据质量。为了扩充预训练语料，我们挖掘互联网数据的潜力并优化清洗流程，从而恢复了大量被误删的数据。此外，我们纳入了更多中文数据，旨在更好地利用中文互联网上可获得的语料。除了数据量之外，我们也关注数据质量。我们用来自多种来源的高质量数据丰富预训练语料，同时改进基于质量的过滤算法。改进后的算法确保大量无益数据被剔除，同时有价值的数据基本得以保留。此外，我们从预训练语料中过滤掉有争议的内容，以缓解由特定区域文化引入的数据偏差。关于该过滤策略影响的详细讨论见附录 E。

我们采用与 DeepSeek 67B 相同的分词器，它基于字节级字节对编码（Byte-level Byte-Pair Encoding, BBPE）算法构建，词表大小为 100K。我们分词后的预训练语料包含 8.1T token，其中中文 token 比英文 token 多约 12%。

3.1.2 超参数

模型超参数：我们将 Transformer 层数设为 60，隐藏维度设为 5120。所有可学习参数均以 0.006 的标准差随机初始化。在 MLA 中，我们将注意力头数 n_h 设为 128，每个头的维度 d_h 设为 128。KV 压缩维度 d_c 设为 512，查询压缩维度 d'_c 设为 1536。对于解耦查询与键，我们将每头维度 d^R_h 设为 64。遵循 Dai 等（2024）的做法，我们将除第一层之外的所有 FFN 替换为 MoE 层。每个 MoE 层由 2 个共享专家与 160 个路由专家组成，其中每个专家的中间隐藏维度为 1536。在路由专家中，每个 token 将激活 6 个专家。此外，低秩压缩与细粒度专家分割会影响一层的输出尺度。因此，在实践中，我们在压缩潜在向量之后使用额外的 RMS Norm 层，并在宽度瓶颈处（即压缩潜在向

量与路由专家的中间隐藏状态）乘以额外的缩放因子，以确保训练稳定。在该配置下，DeepSeek-V2 的总参数量为 236B，其中每个 token 激活 21B。

训练超参数：我们采用 AdamW 优化器（Loshchilov 与 Hutter，2017），超参数设为 β_1 = 0.9、β_2 = 0.95、weight_decay = 0.1。学习率采用预热与阶梯衰减（warmup-and-step-decay）策略（DeepSeek-AI，2024）进行调度。最初，学习率在前 2K 步内从 0 线性增加到最大值。随后，在训练约 60% 的 token 后，学习率乘以 0.316；在训练约 90% 的 token 后，再乘以 0.316。最大学习率设为 2.4×10^−4，梯度裁剪范数设为 1.0。我们还采用了批大小调度策略：在前 225B token 的训练中，批大小从 2304 逐渐增加到 9216，随后在剩余训练中保持 9216。我们将最大序列长度设为 4K，并在 8.1T token 上训练 DeepSeek-V2。我们利用流水线并行将模型的不同层部署在不同设备上，并且对于每一层，路由专家会均匀部署在 8 个设备上（D = 8）。在设备受限路由方面，每个 token 最多被发送到 3 个设备（M = 3）。在均衡损失方面，我们将 α_1 设为 0.003、α_2 设为 0.05、α_3 设为 0.02。我们在训练期间采用 token 丢弃策略以加速训练，但在评估时不丢弃任何 token。

3.1.3 基础设施

DeepSeek-V2 基于 HAI-LLM 框架（High-flyer，2023）训练，该框架是由我们的工程师内部开发的高效轻量训练框架。它采用 16 路零气泡（zero-bubble）流水线并行（Qi 等，2023）、8 路专家并行（Lepikhin 等，2021）以及 ZeRO-1 数据并行（Rajbhandari 等，2020）。鉴于 DeepSeek-V2 的激活参数相对较少，且部分算子会被重算以节省激活内存，因此无需张量并行即可训练，从而降低了通信开销。此外，为了进一步提升训练效率，我们将共享专家的计算与专家并行的 all-to-all 通信重叠。我们还为通信、路由算法以及跨不同专家的融合线性计算定制了更快的 CUDA kernel。此外，MLA 也基于改进版的 FlashAttention-2（Dao，2023）进行了优化。

我们在配备 NVIDIA H800 GPU 的集群上开展所有实验。H800 集群中的每个节点包含 8 个 GPU，节点内通过 NVLink 与 NVSwitch 连接。在节点之间，使用 InfiniBand 互连来促进通信。

3.1.4 长上下文扩展

在 DeepSeek-V2 完成初始预训练之后，我们采用 YaRN（Peng 等，2023）将默认上下文窗口长度从 4K 扩展到 128K。YaRN 被专门应用于解耦的共享键 k^R_t，因为它负责承载 RoPE（Su 等，2024）。对于 YaRN，我们将缩放因子 s 设为 40、α 设为 1、β 设为 32，并将目标最大上下文长度设为 160K。在这些设置下，我们可以预期模型在 128K 的上下文长度上表现良好。与原始 YaRN 略有

不同，由于我们独特的注意力机制，我们调整了长度缩放因子以调制注意力熵。因子 $\sqrt{t}$ 的计算方式为 $\sqrt{t} = 0.0707 \ln s + 1$，旨在最小化困惑度。

【图表】图 4 “大海捞针”（Needle In A Haystack, NIAH）测试的评估结果。DeepSeek-V2 在长达 128K 的所有上下文窗口长度上均表现良好（原文第 13 页）

我们额外训练了模型 1000 步，序列长度为 32K，批大小为 576 个序列。尽管训练仅在 32K 的序列长度上进行，但模型在 128K 上下文长度上评估时仍表现出稳健的性能。如图 4 所示，“大海捞针”（Needle In A Haystack, NIAH）测试的结果表明，DeepSeek-V2 在长达 128K 的所有上下文窗口长度上均表现良好。

3.2 评估

3.2.1 评估基准

DeepSeek-V2 在中英双语语料上预训练，因此我们在一系列英文与中文基准上对其进行评估。我们的评估基于我们内部的评估框架，该框架集成，HAI-LLM 框架中。所包含的基准按类别列举如下，其中带下划线的基准为中文基准：

多学科多项选择数据集包括 MMLU（Hendrycks 等，2020）、C-Eval（Huang 等，2023）与 CMMLU（Li 等，2023）。

语言理解与推理数据集包括 HellaSwag（Zellers 等，2019）、PIQA（Bisk 等，2020）、ARC（Clark 等，2018）与 BigBench Hard（BBH）（Suzgun 等，2022）。

闭卷问答数据集包括 TriviaQA（Joshi 等，2017）与 NaturalQuestions（Kwiatkowski 等，2019）。

阅读理解数据集包括 RACE（Lai 等，2017）、DROP（Dua 等，2019）、C3（Sun 等，2019）与 CMRC（Cui 等，2019）。

指代消歧数据集包括 WinoGrande（Sakaguchi 等，2019）与 CLUEWSC（Xu 等，2020）。

语言建模数据集包括 Pile（Gao 等，2020）。

中文理解与文化数据集包括 CHID（Zheng 等，2019）与 CCPM（Li 等，2021）。

数学数据集包括 GSM8K（Cobbe 等，2021）、MATH（Hendrycks 等，2021）与 CMath（Wei 等，2023）。

代码数据集包括 HumanEval（Chen 等，2021）、MBPP（Austin 等，2021）与 CRUXEval（Gu 等，2024）。

标准化考试包括 AGIEval（Zhong 等，2023）。注意 AGIEval 同时包含英文与中文子集。

遵循我们此前的工作（DeepSeek-AI，2024），我们对 HellaSwag、PIQA、WinoGrande、RACE-Middle、RACE-High、MMLU、ARC-Easy、ARC-Challenge、CHID、C-Eval、CMMLU、C3 与 CCPM 等数据集采用基于困惑度的评测；对 TriviaQA、NaturalQuestions、DROP、MATH、GSM8K、HumanEval、MBPP、CRUXEval、BBH、AGIEval、CLUEWSC、CMRC 与 CMath 采用生成式评测。此外，我们对 Pile-test 采用基于语言建模的评测，并使用每字节比特数（Bits-Per-Byte, BPB）作为指标，以保证不同分词器的模型之间可以公平比较。

为便于直观了解这些基准，我们还在附录 G 中给出每个基准的评测格式。

3.2.2 评测结果

在表 2 中，我们将 DeepSeek-V2 与若干代表性开源模型进行比较，包括 DeepSeek 67B（DeepSeek-AI，2024，我们的上一代发布）、Qwen1.5 72B（Bai 等，2023）、LLaMA3 70B（AI@Meta，2024）与 Mixtral 8x22B（Mistral，2024）。我们用内部评测框架评测所有这些模型，并确保它们采用相同的评测设置。总体而言，仅 21B 激活参数的 DeepSeek-V2 在几乎所有基准上显著优于 DeepSeek 67B，并在开源模型中达到顶级性能。

【表格】表 2 DeepSeek-V2 与其他代表性开源模型的对比。所有模型均在我们的内部框架中评测，并采用相同的评测设置；粗体表示最佳，下划线表示次佳；差距小于 0.3 的分数视为同一水平。仅 21B 激活参数的 DeepSeek-V2 在开源模型中达到顶级性能（原文第 15 页）

进一步地，我们逐一详细比较 DeepSeek-V2 与其开源同类模型。（1）与同样支持中英文的 Qwen1.5 72B 相比，DeepSeek-V2 在大多数英文、代码与数学基准上展现出压倒性优势。至于中文基准，Qwen1.5 72B 在多学科多项选择任务上表现更好，而 DeepSeek-V2 在其他中文任务上与之相当或更优。注意，在 CHID 基准上，Qwen1.5 72B 的分词器在我们的评测框架中会出现错误，因此我们将 Qwen1.5 72B 的 CHID 得分留空。（2）与 Mixtral 8x22B 相比，除 TriviaQA、NaturalQuestions 与 HellaSwag 这些与英文常识知识密切相关的基准外，DeepSeek-V2 的英文性能与之相当或更优。值得注意的是，DeepSeek-V2 在 MMLU 上优于 Mixtral 8x22B。在代码与数学基准上，DeepSeek-V2 与 Mixtral 8x22B 表现相当。由于 Mixtral 8x22B 未专门在中文数据上训练，其中文能力远远落后于 DeepSeek-V2。（3）与 LLaMA3 70B 相比，DeepSeek-V2 使用的英文 token 不足其四分之一。因此，

我们承认 DeepSeek-V2 在基础英文能力上与 LLaMA3 70B 仍有小幅差距。然而，即使训练 token 与激活参数都少得多，DeepSeek-V2 在代码与数学能力上仍与 LLaMA3 70B 相当。此外，作为一个双语语言模型，DeepSeek-V2 在中文基准上压倒性地优于 LLaMA3 70B。

最后值得一提的是，某些先前研究（Hu 等，2024）在预训练阶段就纳入了 SFT 数据，而 DeepSeek-V2 在预训练期间从未接触过 SFT 数据。

3.2.3 训练与推理效率

训练成本 由于 DeepSeek-V2 为每个 token 激活的参数更少、所需 FLOPs 低于 DeepSeek 67B，理论上训练 DeepSeek-V2 比训练 DeepSeek 67B 更经济。尽管训练 MoE 模型会引入额外的通信开销，但通过我们的算子与通信优化，DeepSeek-V2 的训练仍可达到较高的模型 FLOPs 利用率（MFU）。在 H800 集群上的实际训练中，每训练 1 万亿 token，DeepSeek 67B 需要 300.6K GPU 小时，而 DeepSeek-V2 仅需 172.8K GPU 小时，即稀疏的 DeepSeek-V2 相比稠密的 DeepSeek 67B 节省 42.5% 的训练成本。

推理效率 为高效部署 DeepSeek-V2 对外服务，我们首先将其参数转换为 FP8 精度。此外，我们还对 DeepSeek-V2 进行 KV 缓存量化（Hooper 等，2024；Zhao 等，2023），将 KV 缓存中的每个元素平均进一步压缩为 6 bit。得益于 MLA 与这些优化，实际部署的 DeepSeek-V2 所需 KV 缓存显著少于 DeepSeek 67B，因而可以服务大得多的批大小。我们基于实际部署的 DeepSeek 67B 服务的提示与生成长度分布，评估 DeepSeek-V2 的生成吞吐。在配备 8 块 H800 GPU 的单节点上，DeepSeek-V2 的生成吞吐超过每秒 50K token，是 DeepSeek 67B 最大生成吞吐的 5.76 倍。此外，DeepSeek-V2 的提示输入吞吐超过每秒 100K token。

4 对齐

4.1 监督微调

在我们此前研究（DeepSeek-AI，2024）的基础上，我们构建的指令微调数据集包含 1.5M 个实例，其中 1.2M 个实例用于提升有用性、0.3M 个实例用于安全性。与此前版本相比，我们提升了数据质量，以缓解幻觉性回复并增强写作能力。我们用 2 个 epoch 微调 DeepSeek-V2，学习率设为 5×10^{-6}。对于 DeepSeek-V2 Chat（SFT）的评测，我们主要采用生成式基准，仅保留若干代表性多项选择任务（MMLU 与 ARC）。我们还对 DeepSeek-V2 Chat（SFT）开展指令遵循评测（IFEval）（Zhou

等，2023），使用提示级宽松准确率（prompt-level loose accuracy）作为指标。此外，我们采用 2023 年 9 月 1 日至 2024 年 4 月 1 日的 LiveCodeBench（Jain 等，2024）题目评测对话模型。除标准基准外，我们还在开放式对话基准上进一步评测模型，包括 MT-Bench（Zheng 等，2023）、AlpacaEval 2.0（Dubois 等，2024）与 AlignBench（Liu 等，2023）。为便于比较，我们也在自己的评测框架与设置中评测 Qwen1.5 72B Chat、LLaMA-3-70B Instruct 与 Mistral-8x22B Instruct。至于 DeepSeek 67B Chat，我们直接采用此前发布中报告的评测结果。

4.2 强化学习

为进一步释放 DeepSeek-V2 的潜力并使其与人类偏好对齐，我们通过强化学习（RL）调整其偏好。

强化学习算法 为节省 RL 的训练成本，我们采用组相对策略优化（GRPO）（Shao 等，2024），它舍弃了通常与策略模型规模相同的评论家（critic）模型，转而从组内得分估计基线。具体而言，对于每个问题 q，GRPO 从旧策略 π_θold 中采样一组输出 {o_1, o_2, ⋯, o_G}，然后通过最大化以下目标来优化策略模型 π_θ：

J_GRPO(θ) = E[q∼P(Q), {o_i}_{i=1}^G∼π_θold(O|q)] [(1/G) Σ_{i=1}^G (min((π_θ(o_i|q)/π_θold(o_i|q)) A_i, clip(π_θ(o_i|q)/π_θold(o_i|q), 1−ε, 1+ε) A_i) − β D_KL(π_θ ‖ π_ref))] （32）

D_KL(π_θ ‖ π_ref) = π_ref(o_i|q)/π_θ(o_i|q) − log(π_ref(o_i|q)/π_θ(o_i|q)) − 1 （33）

其中 ε 与 β 为超参数；A_i 为优势（advantage），由每组内输出对应的一组奖励 {r_1, r_2, ⋯, r_G} 计算得到：

A_i = (r_i − mean({r_1, r_2, ⋯, r_G})) / std({r_1, r_2, ⋯, r_G}) （34）

训练策略 在初步实验中，我们发现对推理数据（例如代码与数学提示）进行 RL 训练，呈现出区别于通用数据训练的独特特性。例如，我们模型的数学与编码能力可以在较长的一段训练步数内持续提升。因此，我们采用两阶段 RL 训练策略：先进行推理对齐，再进行人类偏好对齐。在第一个推理对齐阶段，我们为代码与数学推理任务训练一个奖励模型 RM_reasoning，并用 RM_reasoning 的反馈优化策略模型：

r_i = RM_reasoning(o_i) （35）

在第二个阶段，即人类偏好对齐阶段，我们采用多奖励框架，从有用性奖励模型 RM_helpful、安全性奖励模型 RM_safety 与基于规则的奖励模型 RM_rule 获取奖励。回复 o_i 的最终奖励为：

r_i = c_1·RM_helpful(o_i) + c_2·RM_safety(o_i) + c_3·RM_rule(o_i) （36）

其中 c_1、c_2 与 c_3 为相应的系数。

为获得在 RL 训练中起关键作用的可靠奖励模型，我们仔细收集偏好数据，并细致地进行质量过滤与比例调整。我们基于编译器反馈获取代码偏好数据，基于真值标签获取数学偏好数据。对于奖励模型训练，我们用 DeepSeek-V2 Chat（SFT）初始化奖励模型，并使用逐点（point-wise）或成对（pair-wise）损失进行训练。在实验中，我们观察到 RL 训练能够充分挖掘并激活模型的潜力，使其能够从可能的回复中选出正确且令人满意的答案。

训练效率优化 在超大规模模型上开展 RL 训练对训练框架提出了很高要求。它需要细致的工程优化来管理 GPU 显存与内存压力，同时保持较快的训练速度。为此，我们实施了以下工程优化：（1）首先，我们提出一种混合引擎，对训练与推理分别采用不同的并行策略，以实现更高的 GPU 利用率；（2）其次，我们采用大 batch size 的 vLLM（Kwon 等，2023）作为推理后端，以加速推理速度；（3）第三，我们精心设计了将模型卸载到 CPU 与重新加载回 GPU 的调度策略，在训练速度与内存消耗之间实现了近乎最优的平衡。

4.3 评测结果

标准基准评测 首先，我们在标准基准上评测 DeepSeek-V2 Chat（SFT）与 DeepSeek-V2 Chat（RL）。值得注意的是，与基座版本相比，DeepSeek-V2 Chat（SFT）在 GSM8K、MATH 与 HumanEval 评测上均有大幅提升。这一进展可归因于我们纳入了包含大量数学与代码相关内容的 SFT 数据。此外，DeepSeek-V2 Chat（RL）进一步提升了数学与代码基准上的性能。我们在附录 F 中给出更多代码与数学评测结果。

至于与其他模型的比较，我们首先将 DeepSeek-V2 Chat（SFT）与 Qwen1.5 72B Chat 进行比较，发现 DeepSeek-V2 Chat（SFT）在几乎所有英文、数学与代码基准上均超过 Qwen1.5 72B Chat。在中文基准上，DeepSeek-V2 Chat（SFT）在多学科多项选择任务上的得分略低于 Qwen1.5 72B Chat，这与其基座版本所观察到的表现一致。与最先进的开源 MoE 模型 Mixtral 8x22B Instruct 相比，DeepSeek-V2 Chat（SFT）在大多数基准上表现更好，仅 NaturalQuestions 与 IFEval 除外。此外，与最先进的开源模型 LLaMA3 70B Chat 相比，DeepSeek-V2 Chat（SFT）在代码与数学相关基准上表现相近；LLaMA3 70B Chat 在 MMLU 与 IFEval 上表现更好，而 DeepSeek-V2 Chat（SFT）在中文任务上表现出更强的性能。最终，与 DeepSeek-V2 Chat（SFT）相比，DeepSeek-V2 Chat（RL）在数学与编码任务上的性能进一步提升。这些比较凸显了 DeepSeek-V2 Chat 在各领域与各语言中相对于其他语言模型的优势。

【表格】表 3 DeepSeek-V2 Chat（SFT）、DeepSeek-V2 Chat（RL）与其他代表性开源对话模型的对比。关于 TriviaQA 与 NaturalQuestions，值得注意的是，LLaMA3 70B Instruct 等对话模型可能不会严格遵循少样本设置中通常规定的格式约束，因此这可能导致某些模型在我们的评测框架中被低估（原文第 19 页）

开放式生成评测 我们继续在开放式对话基准上评测模型。对于英文开放式对话生成，我们使用 MT-Bench 与 AlpacaEval 2.0 作为基准。表 4 中的评测结果显示，DeepSeek-V2 Chat（RL）相比 DeepSeek-V2 Chat（SFT）具有显著的性能优势。这一结果展示了我们的 RL 训练在实现更好对齐方面的有效性。与其他开源模型相比，DeepSeek-V2 Chat（RL）在两个基准上均优于 Mistral 8x22B Instruct 与 Qwen1.5 72B Chat。与 LLaMA3 70B Instruct 相比，DeepSeek-V2 Chat（RL）在 MT-Bench 上展现出有竞争力的性能，并在 AlpacaEval 2.0 上明显优于它。这些结果凸显了 DeepSeek-V2 Chat（RL）在生成高质量且与上下文相关的回复方面的强大性能，尤其是在基于指令的对话任务中。

【表格】表 4 英文开放式对话评测。对 AlpacaEval 2.0，我们使用长度控制（length-controlled）的胜率作为指标（原文第 19 页）

此外，我们基于 AlignBench 评测中文开放式生成能力。如表 5 所示，DeepSeek-V2 Chat（RL）相比 DeepSeek-V2 Chat（SFT）略有优势。值得注意的是，DeepSeek-V2 Chat（SFT）以显著优势超越所有开源中文模型。它在中文推理与语言两方面均显著优于第二佳的开源模型 Qwen1.5 72B Chat。此外，DeepSeek-V2 Chat（SFT）与 DeepSeek-V2 Chat（RL）均优于 GPT-4-0613 与 ERNIEBot 4.0，巩固了我们的模型在支持中文的顶级 LLM 中的地位。具体而言，DeepSeek-V2 Chat（RL）在中文语言理解上表现卓越，超越了包括 GPT-4-Turbo-1106-Preview 在内的所有模型。另一方面，DeepSeek-V2 Chat（RL）的推理能力仍落后于 Erniebot-4.0 与 GPT-4s 等巨型模型。

【表格】表 5 由 GPT-4-0613 评分的 AlignBench 排行榜。模型按总分降序排列；带 * 的模型表示我们通过其 API 服务或开放权重模型自行评测，而非引用其原论文报告的结果；Erniebot-4.0 与 Moonshot 的后缀表示我们调用其 API 的时间戳（原文第 20 页）

4.4 讨论

SFT 数据量 关于大规模 SFT 语料必要性的讨论一直是激烈争论的话题。先前工作（Young 等，2024；Zhou 等，2024）认为，少于 10K 个 SFT 数据实例就足以产生令人满意的结果。然而，在我们的实验中，如果使用少于 10K 个实例，我们观察到 IFEval 基准上的性能显著下降。一种可能的解释

是：语言模型需要一定量的数据才能发展出特定技能。尽管所需的数据量可能会随模型规模增大而减少，但无法完全消除。我们的观察凸显了为 LLM 配备所需能力时对充足数据的关键需求。此外，SFT 数据的质量同样至关重要，尤其是对于涉及写作或开放式问题的任务。

强化学习的对齐税 在人类偏好对齐过程中，我们观察到模型在开放式生成基准上取得显著性能提升（以 AI 与人类评估者给出的分数衡量）。然而，我们也注意到一种“对齐税”（alignment tax）现象（Ouyang 等，2022），即对齐过程可能对 BBH 等某些标准基准上的性能产生负面影响。为缓解对齐税，在 RL 阶段，我们在数据处理与改进训练策略方面投入了大量努力，最终在标准基准与开放式基准的性能之间取得了可接受的权衡。如何在不损害模型通用性能的前提下使模型与人类偏好对齐，是未来研究的一个有价值方向。

在线强化学习 在偏好对齐实验中，我们发现在线方法显著优于离线方法。因此，我们投入大量精力为对齐 DeepSeek-V2 实现了一个在线 RL 框架。关于在线与离线偏好对齐的结论可能因情境而异，我们将更充分的比较与分析留待未来工作。

5 结论、局限与未来工作

本文提出 DeepSeek-V2，一个支持 128K 上下文长度的大型 MoE 语言模型。除强劲性能外，得益于 MLA 与 DeepSeekMoE 等创新架构，它还具有训练经济和推理高效的特点。在实践中，与 DeepSeek 67B 相比，DeepSeek-V2 性能显著更强，同时节省 42.5% 的训练成本、将 KV 缓存减少 93.3%，并将最大生成吞吐提升至 5.76 倍。评测结果进一步表明，仅 21B 激活参数的 DeepSeek-V2 在开源模型中达到顶级性能，并成为最强的开源 MoE 模型。

DeepSeek-V2 及其对话版本存在其他 LLM 普遍具有的公认局限，包括预训练后缺乏持续的知识更新、可能生成非事实性信息（如未经核实的建议）以及可能产生幻觉。此外，由于我们的数据主要由中文与英文内容构成，模型在其他语言上的熟练度可能有限。在中英文之外的场景中，应谨慎使用。

DeepSeek 将以长期主义持续投入开源大模型，致力于逐步接近通用人工智能的目标。

- 在持续探索中，我们致力于设计方法，使 MoE 模型能够进一步扩展，同时保持经济的训练与推理成本。我们下一步的目标是在即将发布的版本中达到与 GPT-4 相当的性能。
- 我们的对齐团队持续努力提升模型，目标是开发一个不仅有用、而且对全球用户诚实且安全的模型。我们的最终目标是使模型价值观与人类价值观对齐，同时尽量减少对人类监督的需求。通过优先考虑伦理与负责任的发展，我们致力于为社会创造积极有益的贡献。

- 目前，DeepSeek-V2 仅设计为支持文本模态。在我们的前瞻规划中，我们打算让模型支持多种模态，以提升其在更广泛场景中的通用性与实用性。

附录

附录 A 贡献与致谢

附录 B DeepSeek-V2-Lite：配备 MLA 与 DeepSeekMoE 的 16B 模型

B.1 模型描述

B.2 性能评估

附录 C MLA 的完整公式

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] compromising its general performance presents a valuable direction for future research.

[2] Online Reinforcement Learning. In our preference alignment experiments, we find that the online approach significantly outperforms the offline approach. Therefore, we invest tremendous efforts in implementing an online RL framework for aligning DeepSeek-V2. The conclusion about online or offline preference alignment can vary in different contexts, and we reserve a more thorough comparison and analysis between them for future work.

[3] 5. Conclusion, Limitation, and Future Work

[4] In this paper, we introduce DeepSeek-V2, a large MoE language model that supports 128K context length. In addition to strong performance, it is also characterized by economical training and efficient inference, benefiting from its innovative architecture including MLA and DeepSeekMoE. In practice, compared with DeepSeek 67B, DeepSeek-V2 achieves significantly stronger performance, and meanwhile saves 42.5% of training costs, reduces the KV cache by 93.3%, and boosts the maximum generation throughput to 5.76 times. Evaluation results further demonstrate that with only 21B activated parameters, DeepSeek-V2 achieves top-tier performance among open-source models and becomes the strongest open-source MoE model.

[5] DeepSeek-V2 and its chat versions share the acknowledged limitations commonly found in other LLMs, including the lack of ongoing knowledge updates after pre-training, the possibility of generating non-factual information such as unverified advice, and a chance to produce hallucinations. In addition, since our data primarily consist of Chinese and English content, our model may exhibit limited proficiency in other languages. In scenarios beyond Chinese and English, it should be used with caution.

[6] DeepSeek will continuously invest in open-source large models with longtermism, aiming to progressively approach the goal of artificial general intelligence.

[7] • In our ongoing exploration, we are dedicated to devising methods that enable further scaling up MoE models while maintaining economical training and inference costs. The goal of our next step is to achieve performance on par with GPT-4 in our upcoming release. • Our alignment team continuously strives to enhance our models, aiming to develop a model that is not only helpful but also honest and safe for worldwide users. Our ultimate objective is to align the values of our model with human values, while minimizing the need for human supervision. By prioritizing ethical considerations and responsible development, we are dedicated to creating a positive and beneficial impact on society. • Currently, DeepSeek-V2 is designed to support the text modality exclusively. In our forward-looking agenda, we intend to enable our model to support multiple modalities, enhancing its versatility and utility in a wider range of scenarios.

[8] AI@Meta. Llama 3 model card, 2024. URL https://github.com/meta-llama/llama3/bl ob/main/MODEL_CARD.md .

[9] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023. Anthropic. Introducing Claude, 2023. URL https://www.anthropic.com/index/introd ucing-claude .

[10] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

[11] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P . Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu. Qwen technical report. arXiv preprint arXiv:2309.16609, 2023.

[12] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi. PIQA: reasoning about physical commonsense in natural language. In The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020, pages 7432–7439. AAAI Press, 2020. doi: 10.1609/aaai.v34i05.6239. URL https://doi.org/10.1609/aaai.v34i05.6239 .

[13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374 .

[14] P . Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. CoRR, abs/1803.05457, 2018. URL http://arxiv.org/abs/1803.05457 .

[15] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[16] Y. Cui, T. Liu, W. Che, L. Xiao, Z. Chen, W. Ma, S. Wang, and G. Hu. A span-extraction dataset for Chinese machine reading comprehension. In K. Inui, J. Jiang, V . Ng, and X. Wan, editors, Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 5883–5889, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1600. URL https://aclanthology.org/D19-1 600 .

[17] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P . Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL https://doi.org/10.48550/arXiv.2401.06066 .

[18] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning, 2023. DeepSeek-AI. Deepseek LLM: scaling open-source language models with longtermism. CoRR, abs/2401.02954, 2024. URL https://doi.org/10.48550/arXiv.2401.02954 .

[19] D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368– 2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246 .

[20] Y. Dubois, B. Galambosi, P . Liang, and T. B. Hashimoto. Length-controlled alpacaeval: A simple way to debias automatic evaluators. arXiv preprint arXiv:2404.04475, 2024.

[21] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. CoRR, abs/2101.03961, 2021. URL https://arxiv.org/ abs/2101.03961 .

[22] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

[23] Google. Introducing gemini: our largest and most capable ai model, 2023. URL https: //blog.google/technology/ai/google-gemini-ai/ .

[24] A. Gu, B. Rozière, H. Leather, A. Solar-Lezama, G. Synnaeve, and S. I. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution, 2024.

[25] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[26] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[27] High-flyer. Hai-llm: 高 效 且 轻 量 的 大 模 型 训 练 工 具 , 2023. URL https://www.high-flyer.c n/en/blog/hai-llm .

[28] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami. Kvquant: Towards 10 million context length LLM inference with KV cache quantization. CoRR, abs/2401.18079, 2024. URL https://doi.org/10.48550/arXiv.2401.18079 .

[29] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. arXiv preprint arXiv:2404.06395, 2024.

[30] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

[31] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024. M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147 .

[32] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. P . Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: a benchmark for question answering research. Trans. Assoc. Comput. Linguistics, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://doi.org/10.1162/tacl_a_00276 .

[33] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 2023.

[34] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. H. Hovy. RACE: large-scale reading comprehension dataset from examinations. In M. Palmer, R. Hwa, and S. Riedel, editors, Proceedings of the 2017 Conference on Empirical Methods in Natural Language

Processing, EMNLP 2017, Copenhagen, Denmark, September 9-11, 2017, pages 785–794. Association for Computational Linguistics, 2017. doi: 10.18653/V1/D17-1082. URL https://doi.org/10.18653/v1/d1 7-1082 .

[35] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In 9th International Conference on Learning Representations, ICLR 2021. OpenReview.net, 2021. URL https://openreview.net/forum?id=qrwe7XHTmYb .

[36] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.

[37] W. Li, F. Qi, M. Sun, X. Yi, and J. Zhang. Ccpm: A chinese classical poetry matching dataset, 2021.

[38] X. Liu, X. Lei, S. Wang, Y. Huang, Z. Feng, B. Wen, J. Cheng, P . Ke, Y. Xu, W. L. Tam, X. Zhang, L. Sun, H. Wang, J. Zhang, M. Huang, Y. Dong, and J. Tang. Alignbench: Benchmarking chinese alignment of large language models. CoRR, abs/2311.18743, 2023. doi: 10.48550/A RXIV.2311.18743. URL https://doi.org/10.48550/arXiv.2311.18743 .

[39] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

[40] Mistral. Cheaper, better, faster, stronger: Continuing to push the frontier of ai and making it accessible to all, 2024. URL https://mistral.ai/news/mixtral-8x22b .

[41] OpenAI. Introducing ChatGPT, 2022. URL https://openai.com/blog/chatgpt .

[42] OpenAI. GPT4 technical report. arXiv preprint arXiv:2303.08774, 2023.

[43] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P . Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. Advances in neural information processing systems, 35:27730–27744, 2022. B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023.

[44] P . Qi, X. Wan, G. Huang, and M. Lin. Zero bubble pipeline parallelism. arXiv preprint arXiv:2401.10241, 2023.

[45] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.

[46] C. Riquelme, J. Puigcerver, B. Mustafa, M. Neumann, R. Jenatton, A. S. Pinto, D. Keysers, and N. Houlsby. Scaling vision with sparse mixture of experts. In Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, pages 8583–8595, 2021. URL https://proceedings.neurips.cc/paper /2021/hash/48237d9f2dea8c74c2a72126cf63d933-Abstract.html .

[47] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.

[48] Z. Shao, P . Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[49] N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019. URL http://arxiv.org/abs/1911.02150 .

[50] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V . Le, G. E. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In 5th International Conference on Learning Representations, ICLR 2017. OpenReview.net, 2017. URL https: //openreview.net/forum?id=B1ckMDqlg .

[51] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.

[52] K. Sun, D. Yu, D. Yu, and C. Cardie. Investigating prior knowledge for challenging chinese machine reading comprehension, 2019.

[53] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[54] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[55] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al. Emergent abilities of large language models. arXiv preprint arXiv:2206.07682, 2022.

[56] T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.

[57] L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL https://doi.org/10.18653/v1/2020.coling-main.419 .

[58] A. Young, B. Chen, C. Li, C. Huang, G. Zhang, G. Zhang, H. Li, J. Zhu, J. Chen, J. Chang, et al. Yi: Open foundation models by 01. ai. arXiv preprint arXiv:2403.04652, 2024.

[59] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[60] Y. Zhao, C. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci. Atom: Low-bit quantization for efficient and accurate LLM serving. CoRR, abs/2310.19102, 2023. URL https://doi.org/10.48550/arXiv.2310.19102 .

[61] C. Zheng, M. Huang, and A. Sun. Chid: A large-scale chinese idiom dataset for cloze test. In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 778–787. Association for Computational Linguistics, 2019. doi: 10.18653/V1/P19-1075. URL https://doi.org/10.18653/v1/p19-1075 .

[62] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P . Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

[63] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364 .

[64] C. Zhou, P . Liu, P . Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P . Yu, L. Yu, et al. Lima: Less is more for alignment. Advances in Neural Information Processing Systems, 36, 2024.

[65] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models. arXiv preprint arXiv:2311.07911, 2023.

DeepSeek-Prover：通过大规模合成数据推进大语言模型定理证明

DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data Huajian Xin1,2 Daya Guo1 Zhihong Shao1 Z.Z. Ren1 Qihao Zhu1 Bo Liu1

DeepSeek-AI research@deepseek.com

arXiv:2405.14333 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

Lean 等证明助手彻底改变了数学证明验证，确保了高准确性与可靠性。尽管大语言模型（LLM）在数学推理方面展现出潜力，但其在形式化定理证明上的进展受制于训练数据的缺乏。为解决这一问题，我们提出一种方法，从高中与本科水平的数学竞赛题出发生成大量 Lean 4 证明数据。该方法包括：将自然语言问题翻译为形式化陈述、过滤低质量陈述，并生成证明以构建合成数据。在包含 800 万条带证明的形式化陈述的合成数据集上微调 DeepSeekMath 7B 后，我们的模型在 Lean 4 miniF2F 测试集上取得了 46.3% 的整证明生成准确率（64 个样本）以及 52% 的累计准确率，超过了基线 GPT-4 的 23.0%（64 个样本）与一种树搜索强化学习方法的 41.0%。此外，我们的模型在 Lean 4 形式化国际数学奥林匹克（FIMO）基准中成功证明了 148 道题中的 5 道，而 GPT-4 一道也未能证明。这些结果展示了利用大规模合成数据增强大语言模型定理证明能力的潜力。合成数据集与模型都将公开，以推动这一前景广阔的领域的后续研究。

1 引言

在现代数学中，证明的复杂性不断增加，给同行评审带来了巨大挑战。这种复杂性导致错误的证明被接受，而其关键缺陷往往要经过相当长的时间才被发现。为解决这些问题，人们开发了形式化数学语言，例如 Lean（De Moura et al., 2015, Moura and Ullrich, 2021）、Isabelle（Paulson, 1994）与 Coq

（The Coq Development Team）。这些语言使得创建计算机可验证的证明成为可能（Avigad, 2023）。然而，撰写形式化证明需要投入大量精力与专业知识，即使对经验丰富的数学家而言也构成挑战。因此，自动定理证明的重要性正日益上升（Shulman, 2024）。

为减少撰写形式化数学证明所需的投入，已有若干方法（Polu and Sutskever, 2020, Jiang et al., 2021, Han et al., 2021, Polu et al., 2022, Lample et al., 2022, Jiang et al., 2022a, Yang et al., 2024）被提出，它们主要聚焦于为待证定理探索潜在解的搜索算法。然而，这些方法常常受困于复杂定理所需的庞大搜索空间，从而对更复杂的证明收效甚微（Loos et al., 2017）。近期，大语言模型（LLM）的进展引入了一种新颖的策略，，利用预训练模型来引导搜索过程。尽管这些新方法（Jiang et al., 2022b, Zhao et al., 2023, Xin et al., 2023）代表了显著的改进，但由于缺乏平行语料库，它们距实际可用仍有差距。与 Python 或 Java 等常规编程语言不同，形式化证明语言的使用者相对较少，导致相关数据集规模有限。近期自动形式化（autoformalization）的进展（Wu et al., 2022）使得能够合成更多对齐的数据，以训练基于 LLM 的自动定理证明器。然而，由此得到的数据集仍然太小，无法充分释放 LLM 的能力。

为解决这一问题，我们提出了一种从非形式化数学问题生成大量 Lean 4 证明数据的方法。我们的方法将高中与本科水平的数学竞赛题转化为形式化命题。随后，我们使用大语言模型（LLM）自动生成证明，并在 Lean 4 环境中验证这些证明的正确性。

该方法的首要挑战在于同时保证合成数据的规模与质量。

质量保障（Quality Assurance）：我们通过多步流程提升所生成证明的质量。首先，我们使用质量评分模型滤除过于简单的命题，并通过假设拒绝（hypothesis rejection）策略剔除无效命题。随后，我们新颖的迭代框架按如下方式提升证明质量：先由一个训练不足、仅在有限数据上微调过的 LLM 从非形式化数学问题生成合成命题；这些命题被用于生成相应的证明，并由 Lean 4 验证器校验其正确性；随后，正确的定理-证明对又被用于进一步训练初始模型。经过若干轮迭代，在大规模合成数据上训练出的模型显著强于最初训练不足的 LLM，从而产出质量更高的定理-证明对。

规模保障（Scale Assurance）：为加速证明生成过程，我们的方法着力应对证明搜索空间巨大这一挑战。造成延迟的一个重要原因是：一些无法证明的命题会一直被处理，直到触及时间上限。为缓解这一问题，我们提出并行地证明否定命题。一旦原命题或其否定命题之一被证明，整个证明过程即告终止。

我们在 Lean 4 定理证明任务上评估了该方法的有效性，所用测试包括 miniF2F（Zheng et al., 2021）的 488 道题目与 FIMO 基准（Liu et al., 2023）的 148 道题目。我们以 DeepSeekMath 7B（Shao et al., 2024）——一个最先进的数学模型——作为基座。结果表明，我们迭代训练出的模型表现

强劲：在 miniF2F-test 基准上、使用 64 个样本时，整证明生成（whole-proof generation）的准确率达到 46.3%，超过 GPT-4（Achiam et al., 2023）的 23.0% 以及一种强化学习方法的 41.0%。此外，在 FIMO 基准上，我们的方法使用 100 个样本解出了 148 道题中的 4 道，而 GPT-4 一道也未解出；使用 4096 个样本时，我们的方法解出了 5 道。消融实验表明，随着每次迭代，模型在 miniF2F 上解出的题目越来越多。综上，本文的贡献如下：

- 我们提出了一种迭代方法，从非形式化数学问题中合成 800 万条形式化命题，每条命题都附带一个形式化证明。实验结果表明，该方法显著提升了合成数据的可扩展性与质量。
- 我们的模型在该合成数据集上训练后，在基准上取得了最先进的性能：在 Lean 4 miniF2F 测试上，使用 64 个样本时整证明生成的准确率为 46.3%，累计准确率为 52%。这超过了 64 个样本下准确率 23.0% 的 GPT-4 基线，以及准确率 41.0% 的树搜索强化学习方法。此外，我们的模型在 Lean 4 形式化国际数学奥林匹克（Formalized International Mathematical Olympiad, FIMO）基准的 148 道题中成功证明了 5 道，而 GPT-4 一道也未能证明。
- 我们创建并开源了一个大规模、高质量的形式化数学证明数据集，以此贡献于数学与 AI 社区，推动自动定理证明领域的进一步研究与发展。

2 背景与相关工作

自动定理证明（Automated Theorem Proving, ATP）自诞生之日起就是人工智能研究中一个重要的兴趣领域（Bibel, 2013）。早期工作着眼于较为简单的逻辑框架，由此催生了诸如 E（Schulz, 2002）与 Vampire（Kovács and Voronkov, 2013）等高效的一阶定理证明器。然而，这些工具在处理 Lean（De Moura et al., 2015）、Isabelle（Paulson, 1994）与 Coq（The Coq Development Team）等现代证明助手中常见的复杂定理时往往力有不逮。近年来深度学习模型与模型引导搜索技术的出现为该领域注入了新的活力（Bansal et al., 2019）。这一现代方法不仅增强了 ATP 系统的能力，也拓展了它们在求解更复杂数学问题上的适用性。

基于神经模型的 ATP。随着深度学习的发展，已有若干方法尝试将神经模型与 ATP 相结合（Loos et al., 2017）。其中一系列 ATP 方法采用由神经模型引导的树搜索算法（Polu and Sutskever, 2020, Han et al., 2021, Polu et al., 2022, Jiang et al., 2022a, Yang et al., 2024）。这些方法主要利用强化学习技术来提升模型的准确率（Kaliszyk et al., 2018, Crouse et al., 2021, Wu et al., 2021, Lample et al., 2022）。由于搜索空间极大，搜索过程会消耗大量时间与计算资源。

另一系列 ATP 方法则借助大语言模型的能力。这类方法通常使用以开源证明数据微调过的语言模型，并通过状态-动作转移程序与验证器交互（Polu and Sutskever, 2020, Jiang et al., 2021, Han et al., 2021, Polu et al., 2022, Lample et al., 2022, Jiang et al., 2022a, Yang et al., 2024）。该过程迭代地生成证明步骤，并用形式化验证器校验其正确性，然后依据形式化验证器返回的证明状态生成下一步证明步骤。尽管这些方法取得了很高的性能，但计算开销很大。为提升效率，近期研究利用语言模型直接生成完整的形式化证明（First et al., 2023, Jiang et al., 2022b, Zhao et al., 2023, Xin et al., 2023），从而绕开证明生成过程中的迭代交互。

面向形式化数学的自动形式化。由于可用于训练的形式化语料有限，当前大语言模型（LLM）的性能同样受到制约。因此，一些方法提出自动形式化（Wu et al., 2022, Jiang et al., 2022b），即将自然语言描述转换为可由证明助手校验的形式化命题。若干研究通过对已有定理进行基于规则的变换，生成了形式化证明的合成数据集（Wu et al., 2020, Wang and Deng, 2020, Xiong et al., 2023）。这些方法虽然有效，但受制于对预定义规则的依赖，在更广泛的应用上缺乏灵活性。近期的方法采用大语言模型将自然语言问题翻译为形式化命题（Huang et al., 2024）。然而，这些数据集规模仍然小于所需，且局限于小型数学基准，因此对语言模型训练效果的提升有限。本文旨在以更大的规模通过自动形式化合成形式化证明，以提升神经证明器的性能。

3 方法

本节介绍我们的方法，它由如图 1 所示的四个关键流程组成。初始阶段聚焦于从大量非形式化数学问题中生成形式化数学命题，这些命题尚待证明。接下来，自动形式化得到的命题会经由模型评分与假设拒绝方法过滤，以筛选出高质量命题。随后，这些命题由一个名为 DeepSeek-Prover 的模型进行证明，其正确性由名为 Lean 4 的形式化验证器校验〔脚注 2：leanprover/lean4 : v4.7.0 − rc2〕，从而产出经过验证的形式化命题与证明。这些数据作为合成数据，用于微调 DeepSeek-Prover。在 DeepSeek-Prover 得到增强后，我们重复上述整个过程。该循环持续进行，直到 DeepSeek-Prover 的提升不再明显。值得注意的是，为提升证明效率，我们同时证明原命题与其否定命题。该方法的好处在于：当原命题无效时，通过证明其否定命题即可迅速将其弃置。各阶段的细节将在后续小节中介绍。

【图表】图 1 我们方法的概览（原文第 4 页）

3.1 自动形式化

形式化证明数据的生成从根本上依赖于大量形式化命题语料的可得性。然而在实践中，汇集大量人工构造的形式化命题颇具挑战。幸运的是，互联网上充满了以自然语言表述的数学相关问题。通过对这些非形式化数学问题进行自动形式化，我们可以生成一个庞大的形式化命题库。

我们观察到，与需要复杂定义与构造的高等数学主题相比，条件明确、目标清晰的问题通常更容易形式化。因此，本文主要考察高中与本科水平的竞赛题，尤其侧重代数与数论，其次涉及组合、几何与统计。尽管这些问题表面上较为简单，但它们往往涉及复杂的求解技巧，因而非常适合用于构建证明数据，以提升大语言模型（LLM）的定理证明能力。为构建数据集，我们采用网页抓取与细致的数据清洗技术，从包含高中与本科习题、考试与竞赛的在线资源中抽取题目，最终得到包含 869,659 道高质量自然语言数学问题的数据集。

具体而言，我们使用 DeepSeekMath-Base 7B 模型（Shao et al., 2024）初始化 DeepSeek-Prover。最初，该模型难以将非形式化数学问题转换为形式化命题。为此，我们使用 MMA 数据集（Jiang et al., 2023）微调 DeepSeek-Prover，该数据集由 Lean 4 的 mathlib〔脚注 3：所用的具体 mathlib commit 为 64528268b3c2cf578639bc479828882a9ecd3a82〕中的形式化命题构成，这些命题由 GPT-4 反向翻译为自然语言问题描述。随后，我们指示模型以结构化方式将这些自然语言问题翻译为 Lean 4 的形式化命题。

Prompt： ````text Mathematical Problem in Natural Language: {$informal_statement_with_answers} Translate the problem to Lean 4 (only the core declaration): ```lean4 ````

Response： ````text {$formal_statement} ``` ````

3.2 质量过滤

我们发现，自动形式化所得命题的质量并不理想，主要源于两个问题。第一，许多形式化命题过于简单。为解决这一问题，我们制定了评分准则，并以 miniF2F-valid 中的示例作为少样本示例来引导 DeepSeek-Prover

……使用思维链（chain-of-thought）方法评估这些陈述的内容与质量。对这些评分的人工复核确认，模型的评估与人类直觉和预期高度吻合。具体而言，模型被指示（详细 prompt 见附录 A.1）将每个形式化陈述的质量划分为以下类别：“excellent”（优秀）、“good”（良好）、“above average”（中上）、“fair”（一般）或“poor”（较差）。被评为“fair”或“poor”的陈述随后会被剔除。

第二个问题涉及那些虽然可证明、但基于不一致假设并导致空洞结论的形式化陈述——这些结论在数学上毫无意义。例如，考虑以下由模型生成的陈述：

```
example (θ : R) (h0 : ∀ z : C, z^2 = -1 ∧ z^3 = -1 ∧ z^6 = 1) (h1 : Real.tan θ = 2 * Real.sqrt 3) : θ = 5 * Real.pi / 3
```

这里，对所有复数均有 $z^2 = -1 \wedge z^3 = -1 \wedge z^6 = 1$ 的假设显然是错误的，因此由此导出的任何结论都毫无意义。为从数据集中剔除此类情况，我们实现了一种假设拒绝方法。该方法使用 DeepSeek-Prover 模型尝试以 'False' 为结论来证明该形式化陈述。若证明成功，则表明假设无效，从而将该陈述排除。示例如下：

```
example (θ : R) (h0 : ∀ z : C, z^2 = -1 ∧ z^3 = -1 ∧ z^6 = 1) (h1 : Real.tan θ = 2 * Real.sqrt 3) : False := by
  simpa using h0 1
```

通过采用模型评分与假设拒绝的双重策略，我们精选出一个包含 712,073 条高质量形式化陈述的精炼集合，为后续的证明合成提供了坚实的基础。

3.3 陈述证明

在构建了大规模高质量形式化陈述语料库之后，我们使用模型为这些陈述搜索证明。传统上，语言模型主要以暴力方式被用于定理证明——反复尝试，直到找到有效证明或计算资源耗尽。这种方式对我们的目标而言效率低下。通常，语言模型被应用于人工精心挑选、通常为真且可证明的形式化陈述；然而，在我们的自动形式化陈述证明任务中，模型生成的许多陈述可能是错误的。事实上，期望模型在任何可靠的证明系统中验证一个假命题都是不合理的。这一问题在大规模自动形式化过程中更加突出：我们观察到，即使经过质量过滤，模型生成的形式化陈述中仍有至少 20% 是错误的，若采用暴力方式处理，将造成大量计算浪费。

为尽量减少在不可证明陈述上的资源浪费并提高证明搜索过程的效率，我们利用陈述与其否定之间的逻辑对称性来加速证明合成。我们为每个合成陈述实现双重并发的证明搜索——一个针对陈述 $\Gamma \vdash P$，另一个针对其否定 $\Gamma \vdash \neg P$。只要其中任一搜索找到有效证明，搜索即终止，从而确凿地表明另一个不可证明。每个证明搜索流最多尝试 k 个证明，除非更早出现有效证明。

所有通过验证的证明——无论它们证明的是原定理还是其否定——随后都会被汇总起来，用于进一步训练 DeepSeek-Prover。因此，这种双重方法起到一种数据增强的作用：即使原命题未被模型正确形式化，数据集也会同时包含命题及其否定，从而得到丰富。

3.4 迭代增强

由于整个流程高度依赖 DeepSeek-Prover，因此在每次迭代后提升模型性能至关重要。为此，我们持续使用新生成的数据对模型进行微调，更新后的模型随后用于后续的自动形式化迭代。这一迭代过程的关键洞见在于：模型在每一轮精炼与应用之后，其能力和效果都会逐步提升。该迭代过程持续到不再观察到进一步的增益为止。因此，模型生成的定理-证明对会随着每次迭代而质量越来越高。这种方法确保 DeepSeek-Prover 的性能持续增强，最终通过不断的精炼产出更优的定理-证明对。

4 实验

4.1 实验设置

DeepSeek-Prover 构建于 DeepSeekMath-Base 7B 模型（Shao et al., 2024）之上，后者是一个仅解码器（decoder-only）transformer（Vaswani et al., 2017），在包含 1200 亿数学相关 token 的语料上预训练而成。我们使用全局批大小 512、恒定学习率 1×10^{-4}，并加入 6,000 步 warmup，利用合成数据对该模型进行微调。DeepSeek-Prover 的性能与若干基线进行了对比：

- GPT-3.5 与 GPT-4（Achiam et al., 2023）：由 OpenAI 开发的高级生成式 AI 模型，以在包括代码生成在内的多种任务上的有效性而闻名。虽然它们并非专为定理证明而设计，但其庞大的规模和参数量赋予了它们强大的能力。相比之下，DeepSeekMath 是一个专门针对数学内容预训练的专用模型。我们使用 GPT-4（具体为 GPT-4-turbo 0409 版本）和 DeepSeekMath，采用与我们的方法类似的方式为给定定理生成完整证明。
- GPT-f（Polu and Sutskever, 2020）：采用受 GPT-2 启发的架构（Radford et al., 2019），实现了一种迭代的最佳优先搜索方法，在形式化证明环境中逐步生成并验证证明步骤，直到证明完成或资源耗尽。这一方法随后被 Proof Artifact Co-Training（Han et al., 2021）、ReProver（Yang et al., 2024）、Llemma（Azerbayev et al., 2023）和 COPRA（Thakur et al., 2023）进一步发展，它们使用专门微调的模型或 GPT-3.5、GPT-4 等通用的多用途模型来生成证明步骤。

4.2 主要结果

本研究针对代数和数论中的复杂数学问题。我们使用 miniF2F（Zheng et al., 2021）和 FIMO（Liu et al., 2023）基准评估模型的定理证明效果。指标 pass@k 用于表示在模型生成的前 k 次尝试中至少发现一个有效证明的情形。

MiniF2F 上的结果：miniF2F 基准包含 244 个验证集问题和 244 个测试集问题，涵盖从基础算术到竞赛级别的问题，例如来自美国数学邀请赛（AIME）、美国数学竞赛（AMC）和国际数学奥林匹克（IMO）的题目。我们使用 Lean 4 版本的 miniF2F，该版本由 LeanDojo 项目发布（https://github.com/yangky11/miniF2F-lean4）。

表 1 在 miniF2F 数据集上对比了多种最先进的方法。DeepSeek-Prover 表现优于所有方法，在 miniF2F-valid 和 miniF2F-test 上的累计得分分别为 60.2% 和 52.0%，显著高于其他方法，包括得分分别为 25.41% 和 22.95% 的 GPT-4。即使是最好的树搜索方法——使用 600M 模型的 Hypertree Proof Search——在 miniF2F-valid 和 miniF2F-test 上也仅达到 58.6% 和 41.0%。DeepSeek-Prover 的可扩展性显而易见：其性能随着计算资源的增加而提升，从贪心方法的 30.03% 提升到 65536 次生成时的 50.0%，展示了其处理复杂证明场景的有效性。MiniF2F 中已证明定理的示例见附录 A.3.1。

FIMO 上的结果：FIMO 基准包含 149 道形式化问题，来源于被翻译为 Lean 4 的 IMO 预选题。我们的方法在每题 100 次尝试下成功证明了 4 个定理，而 GPT-4 未能证明任何定理。将每题的尝试次数增加到 4,096 次后，我们又成功证明了一个定理。FIMO 中已证明定理的示例见附录 A.3.2。

【表格】表 1 在 miniF2F 数据集上与其他最先进方法的对比（原文第 7 页）

4.3 消融研究

4.3.1 大规模自动形式化的有效性

为证明大规模自动形式化的有效性，我们进行了对比分析（如表 2 所示），比较我们的自动形式化数据集与采用专家迭代（expert iteration；Polu and Sutskever, 2020）的传统数据集。这种迭代方法包括生成形式化证明、根据成功结果微调模型，并重复该过程直到不再观察到额外提升。结果表明，使用我们的自动形式化数据训练的模型显著优于仅使用 mathlib 数据训练的模型。

表 2：在 miniF2F 的 pass@128 通过率上的提升；模型在形式化证明上训练，包括来自 Lean 4 mathlib 中人工编写定理的证明以及自动形式化定理的证明（原文第 7 页）。

模型	#Token 数	miniF2F-valid	miniF2F-test
-	-	25.4%	27.5%
Mathlib	0.238B	30.3%	31.2%
自动形式化陈述	3.108B	48.8%	42.6%

4.3.2 形式化陈述评分的有效性

为证明模型在过滤低质量陈述方面的有效性，我们使用等量的高分证明数据与低分证明数据微调 DeepSeekMath-Base 模型，以验证数据质量，如表 3 所示。表 3 显示，使用高分证明数据训练的模型比使用低分证明数据训练的模型高出 4.5%。这一提升凸显了模型在准确评分并有效过滤低质量陈述方面的效用。

表 3：在使用不同评分证明数据训练的模型中，miniF2F 在 pass@128 上的通过率提升。（原文第 8 页）

评分类别	miniF2F-valid	miniF2F-test
“优秀”、“良好”与“中等偏上”	48.8%	42.6%
“一般”与“较差”	41.4%	38.1%

4.3.3 迭代增强的有效性

表 4 表明，数据合成的迭代次数与定理证明性能的提升之间存在明显的相关性。这一证据凸显了我们的迭代增强策略在增强定理证明能力方面的成功。连续的迭代不仅提升了模型处理复杂证明的能力，还显著提高了所生成合成数据的质量与数量。

【表格】表 4 在连续训练迭代中，模型在 miniF2F 上 pass@128 的通过率提升；这一提升得益于通过自动形式化逐步引入合成数据（原文第 8 页）

4.3.4 扩展合成定理证明数据的有效性

我们对合成定理证明数据的研究揭示了数据集规模与模型效能之间的清晰相关性，如表 5 所示。通过考察所生成的 800 万条证明数据中的不同子集，我们观察到 miniF2F 基准上的性能随数据集规模的指数增长而成比例提升。这一模式凸显了大规模数据集对于提升模型自动形式化自然语言问题能力的关键重要性。这些发现强调了系统性数据构建在推动自动定理证明领域发展方面的重要潜力与必要性。

【表格】表 5 在使用通过自动形式化获得的更大比例合成数据训练的模型中，miniF2F 在 pass@128 上的通过率提升（原文第 8 页）

5 案例研究

本节通过两个案例研究来展示我们的方法在定理自动形式化中的应用。它既展示了成功的证明，也展示了在假设拒绝（Hypothesis Rejection）阶段识别不一致性的能力。

5.1 带有完整证明的自动形式化定理

示例 a。问题：证明以下矩阵的行列式为零。

矩阵：[[1, cos(a − b), cos(a)], [cos(a − b), 1, cos(b)], [cos(a), cos(b), 1]]

Lean 中的自动形式化定理：

```
example (a b : ℝ) :
  Matrix.det ![![1, Real.cos (a - b), Real.cos a], ![Real.cos (a - b), 1, Real.cos b],
    ![Real.cos a, Real.cos b, 1]] = 0
```

该方法利用 Lean 有效地将矩阵及其行列式的代数表达式转化为形式语言。自动形式化通过定义一个依赖于实数 a 与 b 的特定 3 × 3 矩阵，并断言其行列式为零，从而抓住了原数学陈述的本质。该形式化使用 Matrix.det 函数计算行列式，并利用 Lean 中 ![...] 这一“列表的列表”记法来表示矩阵的各行。

5.2 含不一致假设的定理自动形式化

示例 b。问题：给定实数 D，并已知对于非零实数 a、b、c，矩阵 [[a, b, c], [1, 4, 9], [3, 1, 2]] 的行列式等于 D，证明 $D^2 = 154$。

Lean 中的自动形式化定理：

```
example (D : ℝ) (h₀ : ∀ a b c : ℝ, a ≠ 0 ∧ b ≠ 0 ∧ c ≠ 0 →
  Matrix.det ![![a, b, c], ![1, 4, 9], ![3, 1, 2]] = D) : D ^ 2 = 154
```

最初的自动形式化错误地假设条件 $D^2 = 154$ 对所有非零实数 a、b、c 普遍成立。这一假设并不被问题陈述所支持，因为问题陈述并未声称其具有普遍适用性。相反，形式化应当致力于找出满足 $D^2 = 154$ 的特定 a、b、c 取值，或者证明这样的取值不存在。模型成功识别出这一不一致，并给出了一个反例来证明该假设的荒谬性：

```
example (D : ℝ) (h₀ : ∀ a b c : ℝ, a ≠ 0 ∧ b ≠ 0 ∧ c ≠ 0 →
  Matrix.det ![![a, b, c], ![1, 4, 9], ![3, 1, 2]] = D) : False := by
  have h₁ := h₀ 1 2 3
  have h₂ := h₀ 1 4 9
  simp [Matrix.det_fin_three] at h₁ h₂
  linarith
```

可以提出如下修正后的自动形式化定理：

```
example (a b c : ℝ) (h₀ : a ≠ 0 ∧ b ≠ 0 ∧ c ≠ 0) :
  let D := Matrix.det ![![a, b, c], ![1, 4, 9], ![3, 1, 2]];
  D ^ 2 = 154
```

这些示例说明了模型有效验证证明并识别假设不一致的能力。更多细节见附录 A.2。

6 结论

本文提出了一种从高中与本科水平的数学竞赛题中生成大量合成证明数据的方法。通过将自然语言问题转化为形式化陈述、过滤低质量数据，并使用迭代证明生成，我们构建了 800 万条证明数据，并在使用这些合成数据训练后显著提升了 DeepSeekMath 7B 模型在自动定理证明（automated theorem proving, ATP）上的性能。我们的模型在 miniF2F 与 FIMO 等基准上优于 GPT-4 及其他方法。通过开源我们的数据集与模型，我们旨在推动自动定理证明研究，并增强大语言模型在形式化数学推理方面的能力。目前，我们的工作主要聚焦于初中与本科水平的代数与数论。未来，我们将致力于扩展所处理的数学问题的多样性，提升我们的方法在 ATP 中的通用性。

更广泛的影响

本文提出的研究有望通过利用由非形式化数学问题生成的大规模合成证明数据，显著推进自动定理证明。这一显著进展可以增强大语言模型在形式化定理证明方面的能力，有助于实现更可靠的数学证明验证，并为学生与研究者提供有价值的教育资源。通过直接发布代码、模型与数据，我们旨在确保我们的工作得到负责任的使用，促进进一步的创新，并保持数据隐私与知识产权合规的高标准。

附录

附录 A 补充材料

A.1 提示词

A.2 自动形式化案例研究

A.3 模型生成的 Lean 证明示例

A.4 MiniF2F-Test 数据集上的结果

A.5 FIMO 数据集上的结果

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] benchmarks like miniF2F and FIMO. By open-sourcing our dataset and model, we aim to advance research in automated theorem proving and enhance the capabilities of large language models in formal mathematical reasoning. Currently, our work mainly focuses on algebra and number theory at the middle school and undergraduate levels. In future work, we will aim to expand the diversity of mathematical problems addressed, enhancing the general applicability of our methods in ATP.

[2] The research presented in this paper has the potential to significantly advance automated theorem proving by leveraging large-scale synthetic proof data generated from informal mathematical problems. This remarkable advancement can enhance the capabilities of large language models in formal theorem proving, contributing to more reliable mathematical proof verification and providing valuable educational resources for students and researchers. By directly releasing the code, model, and data, we aim to ensure the responsible use of our work, fostering further innovation and maintaining high standards of data privacy and intellectual property compliance.

[3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al. Gpt-4 technical report. arXiv preprint arXiv:2303.08774 , 2023.

[4] J. Avigad. Mathematics and the formal turn, 2023.

[5] Z. Azerbayev, H. Schoelkopf, K. Paster, M. D. Santos, S. McAleer, A. Q. Jiang, J. Deng, S. Biderman, and S. Welleck. Llemma: An open language model for mathematics. arXiv preprint arXiv:2310.10631 , 2023.

[6] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. Holist: An environment for machine learning of higher order logic theorem proving. In International Conference on Machine Learning , pages 454–463. PMLR, 2019.

[7] W. Bibel. Automated theorem proving . Springer Science & Business Media, 2013.

[8] M. Crouse, I. Abdelaziz, B. Makni, S. Whitehead, C. Cornelio, P. Kapanipathi, K. Srinivas, V . Thost, M. Witbrock, and A. Fokoue. A deep reinforcement learning approach to first-order logic theorem proving. In Proceedings of the AAAI Conference on Artificial Intelligence , volume 35, pages 6279–6287, 2021.

[9] L. De Moura, S. Kong, J. Avigad, F. Van Doorn, and J. von Raumer. The lean theorem prover (system description). In Automated Deduction-CADE-25: 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings 25 , pages 378–388. Springer, 2015.

[10] E. First, M. N. Rabe, T. Ringer, and Y . Brun. Baldur: Whole-proof generation and repair with large language models, 2023.

[11] J. M. Han, J. Rute, Y . Wu, E. W. Ayers, and S. Polu. Proof artifact co-training for theorem proving with language models. arXiv preprint arXiv:2102.06203 , 2021.

[12] Y . Huang, X. Lin, Z. Liu, Q. Cao, H. Xin, H. Wang, Z. Li, L. Song, and X. Liang. Mustard: Mastering uniform synthesis of theorem and proof data. arXiv preprint arXiv:2402.08957 , 2024.

[13] A. Q. Jiang, W. Li, J. M. Han, and Y . Wu. Lisa: Language models of isabelle proofs. In 6th Conference on Artificial Intelligence and Theorem Proving , pages 378–392, 2021.

[14] A. Q. Jiang, W. Li, S. Tworkowski, K. Czechowski, T. Odrzygó´ zd´ z, P. Miło´s, Y . Wu, and M. Jamnik. Thor: Wielding hammers to integrate language models and automated theorem provers. Advances in Neural Information Processing Systems , 35:8360–8373, 2022a.

[15] A. Q. Jiang, S. Welleck, J. P. Zhou, W. Li, J. Liu, M. Jamnik, T. Lacroix, Y . Wu, and G. Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. arXiv preprint arXiv:2210.12283 , 2022b. A. Q. Jiang, W. Li, and M. Jamnik. Multilingual mathematical autoformalization. arXiv preprint arXiv:2311.03755 , 2023.

[16] C. Kaliszyk, J. Urban, H. Michalewski, and M. Olšák. Reinforcement learning of theorem proving. Advances in Neural Information Processing Systems , 31, 2018.

[17] L. Kovács and A. V oronkov. First-order theorem proving and vampire. In International Conference on Computer Aided Verification , pages 1–35. Springer, 2013.

[18] G. Lample, T. Lacroix, M.-A. Lachaux, A. Rodriguez, A. Hayat, T. Lavril, G. Ebner, and X. Martinet. Hypertree proof search for neural theorem proving. Advances in neural information processing systems , 35:26337–26349, 2022.

[19] C. Liu, J. Shen, H. Xin, Z. Liu, Y . Yuan, H. Wang, W. Ju, C. Zheng, Y . Yin, L. Li, et al. Fimo: A challenge formal dataset for automated theorem proving. arXiv preprint arXiv:2309.04295 , 2023.

[20] S. Loos, G. Irving, C. Szegedy, and C. Kaliszyk. Deep network guided proof search. arXiv preprint arXiv:1701.06972 , 2017.

[21] L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In Automated Deduction–CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings 28 , pages 625–635. Springer, 2021.

[22] L. C. Paulson. Isabelle a Generic Theorem Prover . Springer Verlag, 1994.

[23] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. arXiv preprint arXiv:2009.03393 , 2020.

[24] S. Polu, J. M. Han, K. Zheng, M. Baksys, I. Babuschkin, and I. Sutskever. Formal mathematics statement curriculum learning. arXiv preprint arXiv:2202.01344 , 2022.

[25] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. OpenAI blog , 1(8):9, 2019.

[26] S. Schulz. E–a brainiac theorem prover. Ai Communications , 15(2-3):111–126, 2002.

[27] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y . Li, Y . Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300 , 2024.

[28] M. Shulman. Strange new universes: Proof assistants and synthetic foundations, 2024.

[29] A. Thakur, Y . Wen, and S. Chaudhuri. A language-agent approach to formal theorem-proving. arXiv preprint arXiv:2310.04353 , 2023.

[30] The Coq Development Team. Coq. URL https://coq.inria.fr .

[31] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems , 30, 2017.

[32] M. Wang and J. Deng. Learning to prove theorems by learning to generate theorems. Advances in Neural Information Processing Systems , 33:18146–18157, 2020.

[33] M. Wu, M. Norrish, C. Walder, and A. Dezfouli. Tacticzero: Learning to prove theorems from scratch with deep reinforcement learning. Advances in Neural Information Processing Systems , 34: 9330–9342, 2021.

[34] Y . Wu, A. Q. Jiang, J. Ba, and R. Grosse. Int: An inequality benchmark for evaluating generalization in theorem proving. arXiv preprint arXiv:2007.02924 , 2020.

[35] Y . Wu, A. Q. Jiang, W. Li, M. Rabe, C. Staats, M. Jamnik, and C. Szegedy. Autoformalization with large language models. Advances in Neural Information Processing Systems , 35:32353–32368, 2022. H. Xin, H. Wang, C. Zheng, L. Li, Z. Liu, Q. Cao, Y . Huang, J. Xiong, H. Shi, E. Xie, et al. Lego-prover: Neural theorem proving with growing libraries. arXiv preprint arXiv:2310.00656 , 2023.

[36] J. Xiong, J. Shen, Y . Yuan, H. Wang, Y . Yin, Z. Liu, L. Li, Z. Guo, Q. Cao, Y . Huang, et al. Trigo: Benchmarking formal mathematical proof reduction for generative language models. arXiv preprint arXiv:2310.10180 , 2023.

[37] K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. Advances in Neural Information Processing Systems , 36, 2024.

[38] X. Zhao, W. Li, and L. Kong. Decomposing the enigma: Subgoal-based demonstration learning for formal theorem proving. arXiv preprint arXiv:2305.16366 , 2023.

[39] K. Zheng, J. M. Han, and S. Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. arXiv preprint arXiv:2109.00110 , 2021.

DeepSeek-Coder-V2：打破代码智能领域的闭源模型壁垒

DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence Qihao Zhu, Daya Guo*, Zhihong Shao*, Dejian Yang*, Peiyi Wang, Runxin Xu, Y. Wu*

DeepSeek-AI research@deepseek.com

arXiv:2406.11931 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-Coder-V2，一个开源的混合专家（MoE）代码语言模型，在代码相关任务上取得与 GPT4-Turbo 相当的性能。具体而言，DeepSeek-Coder-V2 在 DeepSeek-V2 的一个中间检查点基础上继续预训练，额外使用了 6 万亿 token。通过这一持续预训练，DeepSeek-Coder-V2 显著增强了 DeepSeek-V2 的编程与数学推理能力，同时在通用语言任务上保持相当的性能。与 DeepSeek-Coder-33B 相比，DeepSeek-Coder-V2 在各类代码相关任务以及推理与通用能力上均有显著提升。此外，DeepSeek-Coder-V2 将支持的编程语言从 86 种扩展到 338 种，并将上下文长度从 16K 扩展到 128K。在标准基准评测中，DeepSeek-Coder-V2 在代码与数学基准上优于 GPT4-Turbo、Claude 3 Opus、Gemini 1.5 Pro 等闭源模型。

1 引言

开源社区通过开发 StarCoder（Li 等，2023b；Lozhkov 等，2024）、CodeLlama（Roziere 等，2023）、DeepSeek-Coder（Guo 等，2024）与 Codestral（MistralAI，2024）等开源代码模型，在推进代码智能方面取得了显著进展。这些模型已逐步接近闭源模型的性能水平，推动了代码智能的进步。然而，与 GPT4-Turbo（OpenAI，2023）、Claude 3 Opus（Anthropic，2024）与 Gemini 1.5 Pro（Reid 等，2024）等最先进的闭源模型相比，仍存在明显差距。为弥合这一差距并进一步推动开源代码模型

的发展，我们推出 DeepSeek-Coder-V2 系列。这些模型基于 DeepSeek-V2（DeepSeek-AI，2024）构建，并使用额外 6 万亿 token 的语料进行了进一步预训练。

在预训练阶段，DeepSeek-Coder-V2 的数据集由 60% 源代码、10% 数学语料与 30% 自然语言语料组成。源代码包含来自 GitHub 与 CommonCrawl 的 1,170B 代码相关 token，处理流程与 DeepSeekMath（Shao 等，2024）相同。与训练 DeepSeek-Coder 所使用的代码语料相比，该语料覆盖的编程语言从 86 种扩展到 338 种。为验证新代码语料的有效性，我们使用 1B 参数模型进行了消融研究，观察到在 HumanEval（从 30.5% 提升到 37.2%）与 MBPP（从 44.6% 提升到 54.0%）基准（Austin 等，2021a；Chen 等，2021）上的准确率分别提升了 6.7% 与 9.4%。对于数学语料，我们使用相同的流程从 CommonCrawl 收集了 221B 数学相关 token，约为 120B 的 DeepSeekMath 语料（Shao 等，2024）规模的两倍；而对于自然语言语料，我们直接从 DeepSeek-V2 的训练语料中采样。DeepSeek-Coder-V2 总共接触了 10.2T 训练 token，其中 4.2 万亿 token 来自 DeepSeek V2 数据集，其余 6 万亿 token 来自 DeepSeek-Coder-V2 数据集。

为适应更长的代码输入并增强在各种编程场景中的适用性，我们将上下文长度从 16K 扩展到 128K token，使模型能够处理更复杂、更大规模的编码任务。在这一多源语料上对 DeepSeek-V2 进行持续预训练后，我们发现 DeepSeek-Coder-V2 显著增强了模型的编码与数学推理能力，同时保持了相当的一般语言性能。

在对齐阶段，我们首先构建了一个指令训练数据集，其中包含来自 DeepSeek-Coder（Guo 等，2024）与 DeepSeek-Math（Shao 等，2024）的代码和数学数据，以及来自 DeepSeek-V2（DeepSeek-AI，2024）的通用指令数据。该数据集用于微调基座模型。随后，在强化学习阶段，我们采用组相对策略优化（GRPO）算法，使其行为与人类偏好对齐。我们在编码领域使用编译器反馈与测试用例收集偏好数据，并开发奖励模型来指导策略模型的训练。这一方法确保模型的回复在编码任务中针对正确性与人类偏好进行优化。为使模型在对齐后支持代码补全，我们还在 16B 参数基座模型的微调过程中采用了中间填充（Fill-In-Middle）方法（Guo 等，2024）。

1.1 贡献

总而言之，我们的主要贡献如下：

- 我们提出基于 DeepSeek-MoE 框架的 DeepSeek-Coder-V2，包含 16B 与 236B 两个参数规模，激活参数分别仅为 2.4B 与 21B，可高效支持多样化的计算与应用需求。此外，DeepSeek-Coder-V2 支持 338 种编程语言与最长 128K token 的上下文长度。

- 我们首次尝试开发开源千亿参数代码模型，以推动代码智能领域的发展。实验结果表明，DeepSeek-Coder-V2 236B 在编码与数学任务上均优于 GPT4-Turbo、Claude 3 Opus 与 Gemini 1.5 Pro 等最先进的闭源模型。
- DeepSeek-Coder-V2 模型以宽松许可证公开发布，允许研究与不受限制的商业使用。

1.2 评估与指标概述

- **代码**：在代码生成基准评估方面，DeepSeek-Coder-V2 展现出对所有开源模型的显著优势，同时其性能与 GPT4-Turbo、Claude 3 Opus 与 Gemini 1.5 Pro 等领先闭源模型相当。值得注意的是，我们在 HumanEval（Chen 等，2021）上取得 90.2% 的分数，在 MBPP（Austin 等，2021a）上取得 76.2% 的分数（在 EvalPlus 评估流程下刷新了最先进结果），在 LiveCodeBench（Jain 等，2024）（题目来自 2023 年 12 月至 2024 年 6 月）上取得 43.4% 的分数。此外，DeepSeek-Coder-V2 是首个在 SWEBench（Jimenez 等，2023）上得分超过 10% 的开源模型。
- **数学**：DeepSeek-Coder-V2 展现出强大的数学推理能力，在 GSM8K（Cobbe 等，2021）等基础基准以及 MATH（Hendrycks 等，2021）、AIME（MAA，2024）与 Math Odyssey（Netmind.AI，2024）等高级竞赛级基准上，均与 GPT-4o、Gemini 1.5 Pro 与 Claude 3 Opus 等顶尖闭源模型不相上下。值得注意的是，DeepSeek-Coder-V2 在 MATH 基准上达到 75.7% 的准确率，几乎追平 GPT-4o 所取得的最先进准确率 76.6%。此外，它在 AIME 2024 竞赛中超过了这些闭源模型的性能。
- **自然语言**：DeepSeek-Coder-V2 保持了与 DeepSeek-V2 相当的一般语言性能。例如，在 OpenAI simple-eval 流程下，DeepSeek-Coder-V2 在 MMLU 上达到 79.2%。在以 GPT-4 作为评判者的主观评估方面，DeepSeek-Coder-V2 在 arena-hard（Li 等，2024）上取得 65.0 分，在 MT-bench（Zheng 等，2023）上取得 8.77 分，在 alignbench（Liu 等，2023c）上取得 7.84 分。这些分数显著优于其他代码专用模型，甚至可与通用开源模型相媲美。

2 数据收集

DeepSeek-Coder-V2 的预训练数据主要由 60% 源代码、10% 数学语料与 30% 自然语言语料组成。由于自然语言语料直接从 DeepSeek-V2 的训练数据集中采样，本节重点介绍代码与数学数据的收集、清洗与过滤过程。同时，我们通过对比分析实验进一步验证这些数据的质量。

我们收集 GitHub 上 2023 年 11 月之前创建的公开仓库。我们首先应用与 DeepSeek-Coder（Guo 等，2024）相同的过滤规则与近似去重，以过滤掉质量较低与重复的源代码。为使论文自包含，我们简要描述这些过滤规则。首先，我们过滤掉平均行长超过 100 个字符或最大行长超过 1000 个字符的文件。此外，我们移除字母字符占比低于 25% 的文件。除 XSLT 编程语言外，我们进一步过滤掉前 100 个字符中出现字符串 “<?xml version=” 的文件。对于 HTML 文件，我们考虑可见文本与 HTML 代码的比例，仅保留可见文本至少占代码的 20% 且不少于 100 个字符的文件。对于通常包含更多数据的 JSON 与 YAML 文件，我们仅保留字符数在 50 到 5000 个字符之间的文件，这有效移除了大多数数据密集型文件。通过应用这些过滤规则与近似去重，我们获得 821B 代码（涵盖 338 种编程语言）与 185B 代码相关文本（如 markdown 与 issue）。支持的编程语言列表见附录 A。我们使用与 DeepSeekV2 相同的分词器，详见（DeepSeek-AI，2024）。

为从 Common Crawl 收集代码相关与数学相关的网页文本，我们遵循与 DeepSeekMath（Shao 等，2024）相同的流程。具体而言，我们选择 StackOverflow（https://stackoverflow.com）等编程论坛、PyTorch 文档（https://pytorch.org/docs）等库站点，以及 StackExchange（https://math.stackexchange.com）等数学网站作为初始种子语料。利用该种子语料，我们训练一个 fastText 模型（Joulin 等，2016）以召回更多代码相关与数学相关的网页。由于中文等语言的分词无法通过空格完成，我们使用 DeepSeek-V2 的字节对编码（BPE）分词器，这显著提升了 fastText 的召回准确率。对于每个领域，我们计算第一轮迭代中收集到的网页百分比，收集网页超过 10% 的领域被归类为代码相关或数学相关领域。随后，我们在这些识别出的领域中标注与代码相关或数学相关内容关联的 URL，并将这些 URL 链接的未收集网页加入种子语料。经过三轮数据收集迭代后，我们从网页中收集到 700 亿个代码相关 token 与 221B 个数学相关 token。为进一步从 GitHub 收集高质量源代码，我们还在 GitHub 上应用相同流程，经过两轮数据收集，收集到 94B 源代码。初始种子语料通过人工收集高质量源代码（例如包含详细描述者）构建。最终，新的代码语料由来自 GitHub 与 CommonCrawl 的 1,170B 代码相关 token 组成。

为验证新代码语料的有效性，我们使用 1B 参数模型进行了消融研究（见表 1），将其与训练 DeepSeek-Coder 所用的语料进行比较。使用 1T token 在新代码语料上预训练 1B 模型，使 HumanEval（从 30.5% 提升到 36.0%）与 MBPP（从 44.6% 提升到 49.0%）基准的准确率分别提升 5.5% 与 4.4%。用 2T token 进一步训练 1B 模型带来了额外提升，HumanEval 与 MBPP 分数分别升至 37.2% 与 54.0%。因此，新代码语料优于用于训练 DeepSeek-Coder 的代码语料。

表 1：DeepSeek-Coder 与 DeepSeek-Coder-V2 的 1B 基座模型性能。（原文第 4 页）

模型	token 数	Python	C++	Java	PHP	TS	C#	Bash	JS	平均	MBPP
DeepSeek-Coder-1B	1T	30.5%	28.0%	31.7%	23.0%	30.8%	31.7%	9.5%	28.6%	26.7%	44.6%
DeepSeek-Coder-V2-1B	1T	36.0%	34.8%	31.7%	27.3%	37.7%	34.2%	6.3%	38.5%	31.2%	49.0%
DeepSeek-Coder-V2-1B	2T	37.2%	39.1%	32.3%	31.7%	34.6%	36.7%	12.0%	32.9%	32.0%	54.0%

3 训练策略

3.1 训练策略

我们对 DeepSeek-Coder-v2 16B 采用两个训练目标：下一 token 预测（Next-Token-Prediction）与中间填充（FIM）（Bavarian 等，2022；Guo 等，2024；Li 等，2023b）。对于 DeepSeek-Coder-v2 236B，我们仅使用下一 token 预测目标。下面简要介绍 FIM 训练策略。我们在 DeepSeek-Coder-v2-16B 的开发中采用 FIM 训练方法，利用 PSM（Prefix、Suffix、Middle）模式。该方法按 Prefix、Suffix、Middle 的顺序组织内容重建，如下所示：

<｜fim_begin｜>f_pre<｜fim_hole｜>f_suf<｜fim_end｜>f_middle<|eos_token|>

该结构在打包（pre-packing）之前于文档级别应用。FIM 的使用率为 0.5，与 PSM 框架一致，以提升训练效果与模型性能。

3.2 模型架构

我们的架构与 DeepSeek-V2（DeepSeek-AI，2024）保持一致。16B 与 236B 的超参数设置分别对应 DeepSeek-V2-Lite 与 DeepSeek-V2 所使用的设置。值得注意的是，我们在训练中遇到了不稳定与梯度值尖峰的问题，我们将其归因于指数归一化技术。为此，我们回退到常规的归一化方法。

3.3 训练超参数

与 DeepSeek V2 的方法一致（DeepSeek-AI，2024），我们使用 AdamW 优化器（Loshchilov 与 Hutter，2019），配置 $\beta1 = 0.9$、$\beta2 = 0.95$，权重衰减为 0.1。批大小与学习率按照 DeepSeek-V2 的规范进行调整。在学习率调度方面，我们采用余弦衰减策略，从 2000 步预热开始，并将学习率逐步降至初始值的 10%。

DeepSeek-Coder-V2 与 DeepSeek-Coder-V2-Lite 均以相同方法训练。为在 DeepSeek-Coder-V2 中保持稳健的自然语言理解能力，我们从 DeepSeek-V2 的一个中间检查点继续预训练过程。该中间检查点最初在 4.2T token 上训练。因此，DeepSeek-Coder-V2 在预训练阶段总共接触了 10.2T 高质量 token。

【表格】表 2 DeepSeek-Coder-V2 的训练设置（原文第 5 页）

3.4 长上下文扩展

遵循 DeepSeek-V2，我们使用 Yarn（Peng 等，2023）将 DeepSeek-Coder-V2 的上下文长度扩展到 128K。YARN 的超参数与 DeepSeek-V2 相同：缩放因子 $s = 40$、$\alpha = 1$、$\beta = 32$。我们进一步分两个阶段继续训练模型，以增强其处理长上下文的能力。在第一阶段，我们使用 32K 的序列长度与 1152 的批大小训练 1000 步。在第二阶段，我们额外训练模型 1000 步，采用 128K 的序列长度与 288 个序列的批大小。

【图表】图 2 “大海捞针”（Needle In A Haystack, NIAH）测试的评估结果。DeepSeek-Coder-V2 在所有最长至 128K 的上下文窗口长度上均表现良好（原文第 6 页）

需要说明的是，在长上下文扩展过程中，我们对长上下文数据的比例进行了上采样。如图 2 所示，“大海捞针”（Needle In A Haystack, NIAH）测试的结果表明，DeepSeek-Coder-V2 在所有最长至 128K 的上下文窗口长度上均表现良好。

3.5 对齐

3.5.1 监督微调

为构建 DeepSeek-Coder-V2 Chat，我们准备了混合代码数据与数学数据的指令训练数据集。我们首先从 DeepSeek-Coder 和 DeepSeek-Math 中收集了 2 万条代码相关指令数据和 3 万条数学相关数据。为保持通用能力，我们还从 DeepSeek-V2 的指令数据中采样了部分数据。最终，我们使用了一个包含 3 亿 token 的指令数据集。在训练方面，我们采用余弦调度，包含 100 步 warm-up，初始学习率为 5×10^{-6}。我们还使用了 100 万 token 的批大小，总共训练 10 亿 token。

3.5.2 强化学习

我们进一步采用强化学习（RL）技术来充分激发 DeepSeek-Coder-V2 的能力，事实证明这相当有效。

Prompts：我们投入了大量精力，从各种来源收集与代码和数学相关的 prompt，并且每个代码 prompt 都附带相应的测试用例。对 prompt 进行过滤后，总共有约 4 万条数据。

Reward Modeling（奖励建模）：奖励模型在 RL 训练中起着关键作用。在数学偏好数据方面，我们使用真实标签（ground-truth labels）来获得这些数据。在代码偏好数据方面，尽管代码编译器本身已经能够提供 0-1 反馈（即代码是否通过全部测试用例），但某些代码 prompt 的测试用例数量有限、无法提供完整覆盖，因此直接使用编译器的 0-1 反馈可能带有噪声且次优。因此，我们仍决定基于编译器提供的数据训练一个奖励模型，并在 RL 训练中使用该奖励模型提供信号；与原始编译器信号相比，这种方式更稳健，泛化能力也更好。如图 3 所示，在我们的内部测试集（Leetcode 与 Leetcode-zh）上，使用奖励模型提供 RL 训练信号明显优于使用原始编译器信号。因此，我们在后续所有实验中均使用奖励模型信号而非编译器信号。

Reinforcement Learning Algorithm（强化学习算法）：我们采用 Group Relative Policy Optimization（GRPO；Shao et al., 2024）作为 RL 算法，这与 DeepSeek-V2 所使用的一致。值得注意的是，GRPO 被证明相当有效，且相比 PPO 成本更低，因为它无需维护额外的 critic 模型。

【图表】图 3 不同方法的性能（原文第 7 页）

4 实验结果

本节中，我们在三类任务上评估 DeepSeek-Coder-V2，包括编程、数学和通用自然语言。我们将 DeepSeek-Coder-V2 与此前的 state-of-the-art 大语言模型进行比较。

- CodeLlama（Roziere et al., 2023）由一系列基于 Llama2（Touvron et al., 2023）的代码语言模型组成，并在 5000 亿至 1 万亿代码 token 的数据集上继续预训练。这些模型提供 7B、13B、34B 和 70B 四种规模。
- StarCoder（Lozhkov et al., 2024）是一个拥有 150 亿参数的可公开访问模型。它专门在 Stack 数据集（Kocetkov et al., 2022）经过精心筛选的子集上训练，覆盖 86 种编程语言。
- StarCoder2（Lozhkov et al., 2024）由 3B、7B 和 15B 参数的模型组成，在 Stack2 数据集（Lozhkov et al., 2024）的 3.3 至 4.3 万亿 token 上训练，覆盖 619 种编程语言。
- DeepSeek-Coder（Guo et al., 2024）包含一系列代码语言模型，参数量从 10 亿到 330 亿。每个模型都从零开始、在 2 万亿 token 上训练，其中 87% 为代码、13% 为英文与中文的自然语言。这些模型在一个项目级代码语料库上以 16K 的窗口大小预训练，并额外采用填空（fill-in-the-blank）任务，从而支持项目级代码补全与填充。
- Codestral（MistralAI, 2024）是由 Mistral 开发的 220 亿参数模型。它在涵盖 80 多种编程语言的多样化数据集上训练，包括 Python、Java 和 JavaScript 等流行语言，以及 Swift 和 Fortran 等更专门的语言。
- 我们对比的通用语言模型包括 Llama3 70B（Meta, 2024）、GPT-4（OpenAI, 2023）、Claude 3 Opus（Anthropic, 2024）和 Gemini 1.5 Pro（Reid et al., 2024）。虽然它们并未专门在大规模代码语料上训练，但它们在编程方面取得了 state-of-the-art 的性能。

4.1 代码生成

HumanEval 与 MBPP 基准：HumanEval（Chen et al., 2021）和 MBPP（Austin et al., 2021b）基准通常用于评估代码生成大语言模型（LLM）的性能。HumanEval 包含 164 个 Python 任务，通过测试用例验证，以评估代码 LLM 在零样本场景下的性能。对于 MBPP，我们使用 MBPP-Plus 版本（Liu et al., 2023a）来评估模型。为测试模型的多语言能力，我们将 HumanEval 基准的问题扩展到了七种额外的语言：C++、Java、PHP、TypeScript、C#、Bash、JavaScript、Swift、R、Julia、D、Rust 和

Racket。对于这两个基准，我们都采用贪心搜索策略，并使用相同的脚本和环境复现基线结果，以确保公平比较。

〔脚注：我们使用模板“Please complete the python function below. The final complete version of your function must be returned within a code block. Here is the unfinished function:\n Python 代码模板：\n{problem_description}\n\n”来构建指令 prompt。〕

【表格】表 3 各模型在 HumanEval 与 MBPP 基准上的性能指标（原文第 8 页）

表 3 全面展示了各个模型在多种编程语言的 HumanEval 与 MBPP+ 基准上的性能指标。DeepSeek-Coder-V2-Instruct 表现出色，取得了第二高的平均分 75.3%。这一表现值得注意，因为它打破了通常由闭源模型主导的局面，成为一个领先的开源竞争者。它仅被 GPT-4o 超越，后者以 76.4% 的平均分领先。DeepSeek-Coder-V2-Instruct 在多种语言上展现出顶级结果，包括在 Java 与 PHP 上取得最高分，并在 Python、C++、C#、TypeScript 和 JavaScript 上表现强劲，凸显了其在处理多样化编程挑战时的稳健性与通用性。

此外，DeepSeek-Coder-V2-Lite-Instruct 的表现同样令人印象深刻，超越了更大的 33B 模型。其平均性能领先幅度可观（65.6% 对 61.9%），凸显了 16B 模型尽管规模更小、却能交付有竞争力结果的有效性。这凸显了该模型的效率，以及模型架构与训练方法上的进步——正是这些进步使其能够超越规模更大的同类模型。

Competitive Programming（竞赛编程）：为进一步验证模型在真实世界竞赛编程问题上的能力，我们使用 LiveCodeBench（Jain et al., 2024）和 USACO 基准（Shi et al., 2024）来评估 DeepSeek-Coder-V2 的有效性。LiveCodeBench 是一项细致且无污染的大语言模型（LLM）代码生成评估，它会持续从三个知名竞赛编程平台系统地收集新的挑战：LeetCode、AtCoder 和 CodeForces。由于训练数据的截止时间早于 2023 年 11 月，我们使用 Livecodebench 的子集（1201-0601）。USACO 基准包含来自美国计算机奥林匹克竞赛（USA Computing Olympiad）的 307 道题，并为每道题提供高质量的单元测试、参考代码和官方题解。

【表格】表 4 在 LiveCodeBench（LCB）与 USACO 基准上的性能（原文第 9 页）

表 4 展示了各种语言模型在这两个基准上的表现。值得注意的是，DeepSeek-Coder-V2-Instruct 表现突出，以 43.4% 的得分与大模型中的最高分并列，与 GPT-4o 持平。这一非凡结果使其总体排名第

二，仅次于以 45.7% 的总体性能领先的 GPT-4-Turbo-0409。DeepSeek-Coder-V2-Instruct 处理复杂编程挑战的出色能力，牢固地确立了其作为顶级竞争者的地位，紧追领先的 GPT-4-Turbo 变体。

4.2 代码补全

4.2.1 仓库级代码补全评测

我们使用 RepoBench（Liu 等，2023b）来评测当前可用的、规模小于 35B 的开源代码模型在仓库级代码补全任务上的能力。该数据集由两种流行编程语言（Python 与 Java）中多样化的真实世界、开源且采用宽松许可证的仓库构建而成。值得注意的是，RepoBench 的最新版本（v1.1）的数据来自 2023 年 10 月 6 日至 12 月 31 日期间创建的 GitHub 仓库，而我们的预训练数据包含 2023 年 11 月之前创建的代码。为确保该数据集未出现在我们的预训练数据中以避免数据泄漏，我们仅使用 2023 年 12 月的数据。

我们的评测涵盖五个上下文长度级别——2k、4k、8k、12k 与 16k token——并覆盖三种设置：跨文件优先（cross-file-first）、跨文件随机（cross-file-random）与文件内（in-file）。对所有参与评测的模型，我们均使用贪心搜索。模型被限制为每个提示最多生成 64 个新 token，并从输出中选取第一个非空且非注释行作为预测结果。通过截断多余的跨文件上下文，提示的最大 token 长度被设为 15,800。我们报告不同上下文长度级别上的平均精确匹配率。

【表格】表 5 不同模型在 RepoBench v1.1 十二月子集上的性能（原文第 10 页）

如表 5 所示，结果表明：DeepSeek-Coder-V2-Lite-Base 模型尽管只有 24 亿激活参数，其在 Python 上的代码补全能力与 DeepSeek-Coder-Base 33B 模型相当，在 Java 上则与 DeepSeek-Coder-Base 7B 模型相当。与 Codestral 相比，DeepSeek-Coder-V2-Lite-Base 模型的激活参数量仅为 Codestral 的十分之一，因此在代码补全任务上的性能较低。不过我们认为，DeepSeek-Coder-V2 更少的激活参数使其在代码补全场景中速度更快。

4.2.2 中间填充代码补全

DeepSeek-Coder-V2-Lite 采用一种独特的训练方式：在预训练阶段使用 0.5 的中间填充（Fill-In-the-Middle, FIM）率。该方法使模型能够利用周围的上下文（既包括前文也包括后文的代码片段）来填空，从而熟练地完成代码。这一能力对代码补全工具尤为有利。一些开源模型，如 SantaCoder（Allal

等，2023）、StarCoder（Li 等，2023b）与 CodeLlama（Roziere 等，2023），也利用类似的能力，并在代码生成与补全领域确立了较高的标准。

为评估 DeepSeek-Coder-V2 模型的性能，我们与领先模型开展了对比分析。评测基于单行填充（Single-Line Infilling）基准，覆盖 Allal 等（2023）所述的三种不同编程语言。本次评测的主要指标是行精确匹配准确率（我们采用生成的第一行，而非整个生成片段，因此结果与 DeepSeek-Coder 略有不同）。

【表格】表 6 不同方法在 FIM 任务上的性能（原文第 11 页）

表 6 展示了各类代码模型在 FIM（中间填充）任务上跨三种编程语言（Python、Java 与 JavaScript）的性能，Mean 分数表示整体效果。在参与比较的模型中，配置为 2.4B 激活参数的 DeepSeek-Coder-V2-Lite-Base 取得了出色的结果：其在 Python 上得分 80.0%、在 Java 上得分 89.1%、在 JavaScript 上得分 87.2%，平均分高达 86.4%，位居首位。这表明 DeepSeek-Coder-V2-Lite-Base 具有出色的效果，尤其是在处理跨不同编程语言的 FIM 任务方面，其性能可与评测中的其他更大模型相当。

4.3 代码修复

为评估模型的缺陷修复能力，我们使用 Defects4J（https://github.com/rjust/defects4j）、SWE-bench（Jimenez 等，2023）与 Aider（https://github.com/paul-gauthier/aider）数据集进行测试。Defects4J 是软件工程领域广泛使用的数据集，专门用于评估与测试程序修复技术。它由来自多个开源项目（包括但不限于 Apache Commons、JFreeChart 与 Closure Compiler）的真实软件缺陷集合组成。数据集中的每个缺陷都配有测试套件，可用于验证程序修复工具的有效性。由于 Defects4J 中的原始缺陷可能需要修改仓库中的多个文件，从而导致上下文过长，我们从这个基准中收集了 238 个仅需修改一个方法的缺陷。

SWE-bench 是一个综合性基准，旨在评估大语言模型解决来自 GitHub 的真实软件问题的性能。该基准给出一个代码库以及一个具体问题，要求语言模型生成能够有效解决所述问题的补丁。这一严格的评测框架确保大语言模型理解与修复真实软件问题的能力得到充分检验，从而清晰衡量其在软件开发任务中的实用性与有效性。

Aider 的代码编辑基准评测 LLM 修改 Python 源文件的能力，共包含 133 个不同的编码任务。该基准不仅检验 LLM 的编码技能，还检验其按照提示中的规范生成代码编辑的一致性。

对于 DeepSeek-Coder-V2 模型，我们采用 whole（整文件）格式进行评测。

【表格】表 7 不同模型在修复基准上的性能。由于 Llama3-Instruct 仅支持 8K 上下文长度，我们未在 SWE-Bench 上对其进行评测（原文第 12 页）

表 7 概述了不同语言模型在软件修复基准（Defects4J、SWE-Bench 与 Aider）上的表现。在开源模型中，DeepSeek-Coder-V2-Instruct 表现突出，取得了开源模型中的最佳性能：其在 Defects4J 上得分 21%，在 SWE-Bench 上得分 12.7%，接近领先闭源模型的结果，展现出处理较长代码序列的强大能力。值得注意的是，DeepSeek-Coder-V2-Instruct 在 Aider 上取得了最高的 73.7% 得分，超越了表中列出的所有其他模型，包括闭源模型。这一卓越表现凸显了其在自动化代码修复任务中的效率与稳健性，使 DeepSeek-Coder-V2-Instruct 成为开源模型中的佼佼者，也是该领域闭源模型的有力竞争者。

4.4 代码理解与推理

为了评估我们模型的代码推理能力，我们使用 CRUXEval 基准。该基准由 800 个 Python 函数及相应的输入-输出示例组成，分为两个不同的任务：CRUXEval-I 要求大语言模型（LLM）根据给定输入预测输出，CRUXEval-O 则要求模型根据已知输出预测输入。这种结构考验模型在前向与反向两个方向上理解并推理 Python 代码的能力。表 8 展示了各类语言模型在 CruxEval 基准上的性能，该基准通过 CruxEval-I-COT 与 CruxEval-O-COT 两个指标评估模型。在开源模型中，DeepSeek-Coder-V2-Instruct 表现尤为突出：其在 CruxEval-I-COT 上得分 70.0%，在 CruxEval-O-COT 上得分 75.1%，展现出其在开源领域的卓越能力。然而，与更大的闭源模型相比，仍存在性能差距。这一性能差距在很大程度上可能归因于 DeepSeek-Coder-V2-Instruct 仅有 210 亿激活参数，远少于 GPT-4o 等更大、更先进的闭源模型。这种模型复杂度上的限制可能制约其学习与问题求解能力。

【表格】表 8 不同模型在 CruxEval 基准上的性能（原文第 13 页）

4.5 数学推理

为了评估 DeepSeek-Coder-V2 的数学推理能力，我们使用了流行的小学数学基准 GSM8K（Cobbe 等，2021），以及高级竞赛级基准，包括 MATH（Hendrycks 等，2021）、美国数学邀请赛（American Invitational Mathematics Examination, AIME）2024（MAA，2024）与 Math Odyssey

（Netmind.AI，2024）。DeepSeek-Coder-V2 在这四个数学基准上的性能通过零样本思维链提示获得；每道测试题均拼接了如下指令：“Please reason step by step, and put your final answer within \boxed{}.”

【表格】表 9 不同模型在数学推理上的性能。DeepSeek-Coder-V2-Instruct 在 maj@64 下可在 AIME 2024 上取得 5/30（原文第 13 页）

表 9 展示的结果使用贪心解码获得，除非另有说明，未借助工具或投票技术。DeepSeek-Coder-V2 在 MATH 基准上取得了 75.7% 的准确率，在 Math Odyssey 上取得 53.7%，与最先进的 GPT-4o 相当。此外，DeepSeek-Coder-V2 在 AIME 2024 上解出的题目数量多于其他模型，展现了其强大的数学推理能力。

4.6 通用自然语言

由于 DeepSeek-Coder-V2 构建于 DeepSeek-V2 之上，它继承了强大的自然语言能力，甚至在推理相关基准上超过了 DeepSeek-V2。我们在标准基准上将 DeepSeek-Coder-V2 Instruct 与 DeepSeek-V2 Chat 进行比较，这些基准同时覆盖中英文，包括 BigBench Hard（BBH）（Suzgun 等，2022）、MMLU（Hendrycks 等，2020）、ARC（Clark 等，2018）、TriviaQA（Joshi 等，2017）、NaturalQuestions（Kwiatkowski 等，2019）、AGIEval（Zhong 等，2023）、CLUEWSC（Xu 等，2020）、C-Eval（Huang 等，2023）与 CMMLU（Li 等，2023a）。此外，我们还评测了模型的开放式生成能力，包括 Arena-Hard（Li 等，2024）、AlpacaEval 2.0（Dubois 等，2024）、MT-Bench（Zheng 等，2023）与 Alignbench（Liu 等，2023c）。评测流程与指标与 DeepSeek-V2 相同，其中 MMLU 使用 OpenAI simple-eval 工具包（https://github.com/openai/simple-evals）评测。

【表格】表 10 DeepSeek-Coder-V2 Instruct 与 DeepSeek-V2 Chat 的对比（原文第 14 页）

在比较 16B 模型的表现时，可以明显看到，DeepSeek-Coder-V2-Lite-Instruct 在 BBH、Arena-Hard 等基准上优于 DeepSeek-V2-Lite-Chat。这些基准对模型的推理能力要求很高，而这正是 DeepSeek-Coder-V2-Lite-Instruct 所擅长的。然而，DeepSeek-Coder-V2-Lite-Instruct 在 TriviaQA 等知识密集型基准上落后，主要原因是其预训练时使用的网页数据相对较少。

再看 236B 模型，DeepSeek-Coder-V2 Instruct 在推理基准上展现出更强的实力，尤其是在包含大量代码、数学与推理问题的 Arena-Hard 上。另一方面，DeepSeek-V2 Chat 在 MT-Bench（Zheng 等，

2023）、AlpacaEval 2.0（Dubois 等，2024）与 AlignBench（Liu 等，2023c）等基准上取得了略好的结果。这一优势可归因于 DeepSeek-V2 Chat 的通用对齐阶段。

5 结论

本文中，我们提出了 DeepSeek-Coder-V2，以进一步推动代码智能领域的发展。该模型基于 DeepSeek-V2 持续预训练而成，使用了来自高质量、多源语料的 6 万亿 token。通过这种持续预训练，我们发现 DeepSeek-Coder-V2 显著增强了模型在编码与数学推理方面的能力，同时保持了与 DeepSeek-V2 相当的通用语言性能。与 DeepSeek-Coder 相比，DeepSeek-Coder-V2 支持的编程语言数量大幅增加，从 86 种增至 338 种，并将最大上下文长度从 16K 扩展到 128K token。实验结果表明，DeepSeek-Coder-V2 在代码与数学专项任务上取得了与 GPT-4 Turbo、Claude 3 Opus 与 Gemini 1.5 Pro 等最先进闭源模型相当的性能。

尽管 DeepSeek-Coder-V2 在标准基准上取得了令人瞩目的性能，但我们发现在指令遵循能力方面，与 GPT-4 Turbo 等当前最先进模型相比仍存在显著差距。这一差距导致其在复杂场景与任务（如 SWE-bench 中的任务）上表现不佳。因此，我们认为，一个代码模型不仅需要强大的编码能力，还需要出色的指令遵循能力，才能应对真实世界中的复杂编程场景。未来，我们将更加着力于提升模型的指令遵循能力，以更好地处理真实世界中的复杂编程场景，并提升开发流程的生产力。

附录

附录：支持的编程语言（见英文原文）

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] Coder-V2 significantly enhances the model's capabilities in coding and mathematical reasoning while maintaining comparable general language performance to DeepSeek-V2. Compared to DeepSeek-Coder, DeepSeek-Coder-V2 supports a significantly larger number of programming languages, increasing from 86 to 338, and extends the maximum context length from 16K to 128K tokens. Experimental results demonstrate that DeepSeek-Coder-V2 achieves performance comparable to state-of-the-art closed-source models such as GPT-4 Turbo, Claude 3 Opus, and Gemini 1.5 Pro in code and math-specific tasks.

[2] Although DeepSeek-Coder-V2 achieves impressive performance on standard benchmarks, we find that there is still a significant gap in instruction-following capabilities compared to current state-of-the-art models like GPT-4 Turbo. This gap leads to poor performance in complex scenarios and tasks such as those in SWEbench. Therefore, we believe that a code model needs not only strong coding abilities but also exceptional instruction-following capabilities to handle real-world complex programming scenarios. In the future, we will focus more on improving the model's instruction-following capabilities to better handle real-world complex programming scenarios and enhance the productivity of the development process.

[3] L. B. Allal, R. Li, D. Kocetkov, C. Mou, C. Akiki, C. M. Ferrandis, N. Muennighoff, M. Mishra, A. Gu, M. Dey, et al. Santacoder: don't reach for the stars! arXiv preprint arXiv:2301.03988, 2023.

[4] A. Anthropic. The claude 3 model family: Opus, sonnet, haiku. Claude-3 Model Card, 2024.

[5] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, and C. Sutton. Program synthesis with large language models, 2021a.

[6] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021b.

[7] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen. Efficient training of language models to fill in the middle. arXiv preprint arXiv:2207.14255, 2022.

[8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

[9] P . Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. CoRR, abs/1803.05457, 2018. URL http://arxiv.org/abs/1803.05457 .

[10] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[11] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024.

[12] Y. Dubois, B. Galambosi, P . Liang, and T. B. Hashimoto. Length-controlled alpacaeval: A simple way to debias automatic evaluators. arXiv preprint arXiv:2404.04475, 2024. D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li, et al. Deepseekcoder: When the large language model meets programming–the rise of code intelligence. arXiv preprint arXiv:2401.14196, 2024.

[13] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[14] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[15] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

[16] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024.

[17] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? arXiv preprint arXiv:2310.06770, 2023.

[18] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147 .

[19] A. Joulin, E. Grave, P . Bojanowski, M. Douze, H. Jégou, and T. Mikolov. Fasttext. zip: Compressing text classification models. arXiv preprint arXiv:1612.03651, 2016.

[20] D. Kocetkov, R. Li, L. Jia, C. Mou, Y. Jernite, M. Mitchell, C. M. Ferrandis, S. Hughes, T. Wolf, D. Bahdanau, et al. The stack: 3 tb of permissively licensed source code. Transactions on Machine Learning Research, 2022.

[21] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. P . Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: a benchmark for question answering research. Trans. Assoc. Comput. Linguistics, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://doi.org/10.1162/tacl_a_00276 .

[22] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023a.

[23] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, et al. Starcoder: may the source be with you! arXiv preprint arXiv:2305.06161, 2023b.

[24] T. Li, W.-L. Chiang, E. Frick, L. Dunlap, B. Zhu, J. E. Gonzalez, and I. Stoica. From live data to high-quality benchmarks: The arena-hard pipeline, April 2024. URL https://lmsys.org/ blog/2024-04-19-arena-hard/ .

[25] J. Liu, C. S. Xia, Y. Wang, and L. Zhang. Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation. In Thirty-seventh Conference on Neural Information Processing Systems, 2023a. URL https://openreview.net/for um?id=1qvx610Cu7 .

[26] T. Liu, C. Xu, and J. McAuley. Repobench: Benchmarking repository-level code auto-completion systems. In The Twelfth International Conference on Learning Representations, 2023b.

[27] X. Liu, X. Lei, S. Wang, Y. Huang, Z. Feng, B. Wen, J. Cheng, P . Ke, Y. Xu, W. L. Tam, X. Zhang, L. Sun, H. Wang, J. Zhang, M. Huang, Y. Dong, and J. Tang. Alignbench: Benchmarking chinese alignment of large language models. CoRR, abs/2311.18743, 2023c. doi: 10.48550/A RXIV.2311.18743. URL https://doi.org/10.48550/arXiv.2311.18743 .

[28] I. Loshchilov and F. Hutter. Decoupled weight decay regularization, 2019.

[29] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei, et al. Starcoder 2 and the stack v2: The next generation. arXiv preprint arXiv:2402.19173, 2024.

[30] MAA. American invitational mathematics examination - aime. American Invitational Mathematics Examination - AIME 2024, 2024. URL https://maa.org/math-competi tions/american-invitational-mathematics-examination-aime .

[31] Meta. Introducing meta llama 3: The most capable openly available llm to date. https: //ai.meta.com/blog/meta-llama-3/ , April 2024.

[32] MistralAI. Codestral. https://mistral.ai/news/codestral/ , 2024. Accessed: 2024-05-29.

[33] Netmind.AI. Odyssey-math. https://github.com/protagolabs/odyssey-math/tree /main , 2024. Accessed: April 22, 2024.

[34] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023.

[35] M. Reid, N. Savinov, D. Teplyashin, D. Lepikhin, T. Lillicrap, J.-b. Alayrac, R. Soricut, A. Lazaridou, O. Firat, J. Schrittwieser, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530, 2024.

[36] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin, et al. Code llama: Open foundation models for code. arXiv preprint arXiv:2308.12950, 2023.

[37] Z. Shao, P . Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[38] Q. Shi, M. Tang, K. Narasimhan, and S. Yao. Can language models solve olympiad programming? arXiv preprint arXiv:2404.10952, 2024.

[39] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[40] H. Touvron, L. Martin, K. Stone, P . Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P . Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023. L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL https://doi.org/10.18653/v1/2020.coling-main.419 .

[41] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P . Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

[42] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364 . A. Supported Programming Languages

[43] ABAP , ActionScript, Ada, Agda, AGS Script, Alloy, AmbientTalk, AMD GPU, AMPL, ANSYS Parametric Design Language, ANTLR, Apache Configuration, APL, AppleScript, Arc, Arduino, ASP , AspectJ, Assembly, Asymptote, Augeas, AutoHotkey, AutoIt, AWK, BC, Berry, BitBake, BlitzBasic, BlitzMax, Bluespec, BNF, Boo, Boogie, Brainfuck, BrightScript, Bro, BST, C, C#, C2HS Haskell, CADL, CapDL, Ceylon, Chapel, ChucK, Cirru, Click, Clojure, CMake, COBOL, COBOLFree, CoffeeScript,

ColdFusion CFC, Common Lisp, C++, Crystal, Csound, Csound Score, CSS, CUDA, Cypher, Cython, Darcs Patch, Dart, DASM16, Debian Control File, DeviceTree, Diff, DM, Docker, Dockerfile, Dylan, EBNF, eC, Eiffel, Elixir, Elm, ELPi, Emacs Lisp, EmberScript, Erlang, Execline, F#, Factor, Fancy, Fantom, Felix, Fennel, Fish, Flux, Fortran, Fortran Fixed Form, FoxPro, FreeFem, FreeMarker, F*, Futhark, G-Code, GAP , GAS, GDScript, Genshi, Gentoo Ebuild, Gentoo Eclass, Gettext Catalog, GLSL, Glyph, Gnuplot, Go, Gosu, Grace, Gradle, Grammatical Framework, GraphQL, Graphviz DOT, Groff, Groovy, Groovy Server Pages, GSQL, Handlebars, Haskell, Haxe, HCL, HLSL, HTML, HTML Django, HTML ERB, HTML PHP , HTTP , Hy, Idris, IGOR Pro, Inform 6 Template, Inno Setup, Io, Isabelle, J, Jade, JAGS, Jasmin, Java, Java Server Pages, JavaScript, JavaScript MozPreproc, JCL, JFlex, JSON, JSONiq, JSX, Julia, Jupyter Notebook, K, Kconfig, Koka, Kotlin, KRL, Lean, Less, Lex, LFE, Lighttpd Configuration File, LilyPond, Limbo, Linker Script, Liquid, Literate Agda, Literate CoffeeScript, LLVM, Logtalk, LSL, Lua, M4, Makefile, Mako, Mason, MATLAB, Maxima, Meson, Metal, MiniScript, Mirah, Mizar, Modelica, Modula-2, Monkey, MooCode, MoonScript, Mosel, MQL, MUF, MuPAD, NASM, NCL, NetLinx, Nginx Configuration File, Nimrod, Ninja, Nit, Nix, NSIS, Nu, NuSMV , Objdump, Objective-C, Objective-C++, OCaml, Octave, Odin, OMG Interface Definition Language, ooc, Opa, OpenCL, OpenEdge ABL, OpenSCAD, Ox, Oz, Papyrus, Parrot Internal Representation, Pascal, PAWN, PEG, Perl, Perl 6, PHP , Pike, PkgConfig, POD, Pony, POV-Ray, PowerShell, Praat, Processing, Propeller Spin, Protocol Buffer, Pug, Puppet, PureBasic, PureScript, Python, Q, QML, QVTO, R, Racket, Ragel in Ruby Host, RAML, RConsole, Rd, REALbasic, ReasonML, Red, RenderScript, Ren'Py, REXX, RHTML, Ride, Robot Framework, Rouge, Ruby, Rust, S, Sage, SARL, SAS, Sass, Scala, Scheme, Scilab, SCSS, Self, Shell, ShExC, Sieve, Silver, Singularity, Slim, Smali, Smarty, Smithy, SMT, Solidity, SourcePawn, SPARQL, SQF, SQL, Squirrel, Stan, Standard ML, Stata, Stylus, SuperCollider, Swift, SWIG, SystemVerilog, Tcl, Tcsh, Tea, Terminfo, TeX, Thrift, Transact- SQL, Treetop, Turing, Twig, TypeScript, TypoScript, Unity3D Asset, Uno, UnrealScript, UrWeb, USD, Vala, VBScript, VCL, Velocity, Verilog, VHDL, VimL, Visual Basic, Vue, WebAssembly, Web IDL, Whiley, X10, XBase, XC, XML, XML Lasso, XQuery, XS, XSLT, Xtend, Xtlang, YANG, Zeek, Zephir, Zig, Zimpl

DeepSeek-Prover-V1.5：利用证明助手反馈的强化学习与蒙特卡洛树搜索

DeepSeek-Prover-V1.5: Harnessing Proof Assistant Feedback for Reinforcement Learning and Monte-Carlo Tree Search Huajian Xin, Z.Z. Ren*, Junxiao Song*, Zhihong Shao*, Wanjia Zhao,*

DeepSeek-AI research@deepseek.com

arXiv:2408.08152 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-Prover-V1.5，这是一个面向 Lean 4 定理证明的开源语言模型，通过同时优化训练与推理过程对 DeepSeek-Prover-V1 进行了改进。该模型在专门面向形式化数学语言的 DeepSeekMath-Base 上预训练，随后使用由 DeepSeek-Prover-V1 衍生的增强版形式化定理证明数据集进行监督微调；并通过基于证明助手反馈的强化学习（RLPAF）实现进一步精炼。在 DeepSeek-Prover-V1 的单次完整证明生成方法之外，我们提出了 RMaxTS——蒙特卡洛树搜索的一种变体，它采用内在奖励驱动的探索策略来生成多样化的证明路径。DeepSeek-Prover-V1.5 相较 DeepSeek-Prover-V1 取得显著提升，在高中水平的 miniF2F 基准测试集（63.5%）和本科水平的 ProofNet 基准（25.3%）上取得了新的最先进结果。

1 引言

大语言模型的最新进展已深刻影响人工智能领域的数学推理与定理证明。尽管在自然语言领域取得了显著进步，语言模型在形式化定理证明（例如使用 Lean（Moura 和 Ullrich，2021）与 Isabelle（Paulson，1994））方面仍面临重大挑战，因为这类证明需要满足验证系统形式化规范的严谨推导。即便是 GPT-4（OpenAI，2023）这样的先进模型也难以应对复杂的形式化证明，这凸显了其中编码与

数学两个方面的复杂本质。一个形式化定理证明模型不仅需要掌握 Lean 定理证明器等形式系统的语法与语义，还必须将抽象的数学推理与精确的形式化表示对齐。

形式化定理证明中的语言模型通常采用两种策略：证明步骤生成（Polu 和 Sutskever，2020；Jiang 等，2022；Lample 等，2022；Yang 等，2023；Wu 等，2024）与整证明生成（Jiang 等，2022；Zhao 等，2023；Wang 等，2023）。证明步骤生成预测每一个后续 tactic，并使用形式化验证器对其进行验证，以获得关于当前 tactic 状态的最新信息，通常还利用树搜索技术来构造有效证明。相比之下，整证明生成在计算上是高效的：它依据定理陈述生成完整的证明代码，在协调证明器模型与形式化定理验证器之间所需的通信预算更少。

尽管 DeepSeek-Prover-V1（Xin 等，2024）凭借整证明生成在 Lean 4 上取得了当时最佳的结果，但这一范式也带来了独特挑战。它需要在无法获取中间 tactic 状态的情况下进行长程序列预测，而后续 tactic 又依赖这些隐藏的结果。在 Lean 的 tactic 模式下，证明由一系列转换证明状态的 tactic 构造而成，这种序列性引入了误差累积（compounding errors；Ross 等，2011）的风险：一次误解就可能导致显著偏离有效的证明路径。更具体地说，自回归模型在生成长证明时可能对中间 tactic 状态持有错误认知（原文作 incorrect believes）。

为了在证明步骤生成中无缝整合中间 tactic 状态，同时保持整证明生成的简洁性与计算效率，我们在 DeepSeek-Prover-V1.5 中提出了一种统一方法。该方法通过“截断并续写”（truncate-and-resume）机制，结合了证明步骤生成与整证明生成两种技术的优势。流程始于标准的整证明生成：语言模型依照定理陈述前缀补全证明代码，随后由 Lean 证明器验证该代码。若证明正确且完整，流程即终止；若检测到错误，代码将在首条错误信息处被截断，其后的代码全部丢弃。成功生成的证明代码随后被用作提示，用于生成下一段证明。为提高模型新补全内容的准确性，我们在提示末尾以注释形式追加 Lean 4 证明器返回的最新状态。值得注意的是，我们的方法并不局限于从最后一个成功应用的 tactic 处续写。我们将截断并续写机制集成到蒙特卡洛树搜索（Monte-Carlo tree search，MCTS；Coulom，2006）中，其中截断点由树搜索策略调度。此外，我们为 MCTS 提出了一种新颖的无奖励（reward-free）探索算法，以应对证明搜索中的奖励稀疏问题。我们为树搜索智能体赋予内在动机（intrinsic motivation），即好奇心（curiosity；Schmidhuber，2010），以广泛探索 tactic 状态空间。这些算法模块将我们的整证明生成模型扩展为交互式定理证明的灵活工具，能够有效利用证明助手的反馈并生成多样化的候选解答。

【图表】图 2 整体框架：DeepSeek-Prover-V1.5 经预训练、监督微调与强化学习训练；推理阶段提供单次采样与蒙特卡洛树搜索两种方式（原文第 3 页）

图 2 中的 Lean 4 示例代码（保留原文）：

prompt

```
import Mathlib
import Aesop
set_option maxHeartbeats 0
open BigOperators Real Nat Topology Rat
/-- Compute the sum of all the roots of $(2x+3)(x-4)+(2x+3)(x-6)=0$.
Show that it is 7/2.-/
theorem amc12a_2002_p1 (f : ℂ → ℂ) (h₀ : ∀ x, f x = (2 * x + 3) * (x - 4) +
(2 * x + 3) * (x - 6)) (h₁ : Fintype (f⁻¹' {0})) :
  ∑ y in (f⁻¹' {0}).toFinset, y = 7 / 2 := by
  -- We show that the roots of the polynomial are -3/2 and 5.
  have : (f⁻¹' {0}).toFinset = {-(3 / 2 : ℂ), (5 : ℂ)} := by
    ext x
    simp only [Set.mem_toFinset, Set.mem_singleton_iff, Set.mem_preimage,
      Set.mem_setOf, Finset.mem_insert, Finset.mem_singleton]
    /- tactic state:
    case a
    f : ℂ → ℂ
    h₀ : ∀ (x : ℂ), f x = (2 * x + 3) * (x - 4) + (2 * x + 3) * (x - 6)
    h₁ : Fintype ↑(f ⁻¹' {0})
    x : ℂ
    ⊢ f x = 0 ↔ x = -(3 / 2) ∨ x = 5
    -/
```

Whole-proof Completion（整证明补全：首轮补全，未通过验证）

```
    rw [h₀]
    constructor
    · intro h
      -- We solve the equation (2x + 3)[2x - 10] = 0.
      have : (2 * x + 3) * (x - 4 + x - 6) = 0 :=
        by linear_combination h
      -- This gives us two cases to solve.
      cases' eq_zero_or_eq_zero_of_mul_eq_zero this
        with h1 h2
      · -- Case 1: 2x + 3 = 0
        left
        linarith
      · -- Case 2: 2x - 10 = 0
        right
        linarith
    · -- We check that -3/2 and 5 are indeed roots.
      rintro (rfl | rfl) <;> norm_num
  -- Now we compute the sum of the roots.
  rw [this]
  norm_num
```

Error Message from Lean 4 prover: failed to synthesize LinearOrder ℂ

截断并续写后的补全（通过 Lean 4 验证，reward = 1）：

```
    rw [h₀]
    constructor
    · intro h
      -- We solve the equation (2x + 3)[2x - 10] = 0.
      have : (2 * x + 3) * (x - 4 + x - 6) = 0 :=
        by linear_combination h
```

```
    -- This gives us two cases to solve.
    cases' eq_zero_or_eq_zero_of_mul_eq_zero this
      with h1 h2
    · -- Case 1: 2x + 3 = 0
      left
      linear_combination (1 / 2) * h1
    · -- Case 2: 2x - 10 = 0
      right
      linear_combination (1 / 2) * h2
  · -- We check that -3/2 and 5 are indeed roots.
    rintro (rfl | rfl) <;> norm_num
-- Now we compute the sum of the roots.
rw [this]
norm_num
```

reward：Passed the verification of Lean 4 prover

1.1 主要贡献

我们提出了一个基于语言模型的形式化数学证明器的综合框架，整合了若干关键组成部分：大规模数学预训练、形式化数学语料的构建与增强、基于证明助手反馈的在线强化学习，以及用于定理证明中长期规划的树搜索方法。预训练模型、监督微调模型与强化学习模型，以及蒙特卡洛树搜索算法的代码，均已公开，可供进一步研究与应用。

- **预训练（Pre-Training）**：我们通过在高质量的数学与代码数据上进一步预训练，重点覆盖 Lean、Isabelle 与 Metamath 等形式化语言，以增强基座模型在形式化定理证明与数学推理方面的能力。
- **监督微调（Supervised Fine-tuning）**：我们通过两种数据增强技术改进 Lean 4 代码补全数据集。首先，我们使用 DeepSeek-Coder V2 236B（Zhu 等，2024）在 Lean 4 代码旁标注自然语言思维链注释，使形式化定理证明与自然语言推理对齐。其次，我们在 Lean 4 证明代码中插入中间 tactic 状态信息，使模型能够有效利用编译器反馈。所得数据集随后用于微调预训练模型。
- **强化学习（Reinforcement Learning）**：我们采用 GRPO 算法（Shao 等，2024），在监督微调模型上执行基于证明助手反馈的强化学习（reinforcement learning from proof assistant feedback，RLPAF）。Lean 证明器的验证结果作为奖励监督，增强模型与验证系统形式化规范的对齐。
- **蒙特卡洛树搜索（Monte-Carlo Tree Search）**：我们通过引入一种新颖的抽象及其对应的搜索算法，推进形式化定理证明中的树搜索方法。我们的截断并续写机制充当状态-动作抽象，将树搜索过程无缝集成到整证明生成框架中。我们提出 RMaxTS——一种创新的蒙特卡洛树搜索算法，它利用 RMax（Brafman 和 Tennenholtz，2002）策略来应对稀疏奖励证明搜索问题中的探索挑战。

通过赋予内在奖励，该算法鼓励证明器智能体生成多样化的规划路径，从而促进对证明空间的广泛探索。

1.2 评测与指标概述

- **miniF2F**：在单次整证明生成设置下，DeepSeek-Prover-V1.5 在 miniF2F 测试集上取得 60.2% 的通过率，相较 DeepSeek-Prover-V1 的 50.0% 显著提升，绝对提升 10.2 个百分点。结合树搜索技术后，通过率进一步提升至新的当前最佳（state-of-the-art）63.5%。
- **ProofNet**：在 ProofNet 的单次整证明生成设置下，DeepSeek-Prover-V1.5 同样表现出强劲性能，验证集通过率为 21.6%，测试集为 23.7%。结合树搜索技术后，结果进一步提升，验证集与测试集分别达到新的当前最佳通过率 25.4% 与 25.3%。

2 模型训练

2.1 预训练

为增强语言模型生成形式化证明以及用数学语言进行推理的能力，我们对基座模型（Shao 等，2024）进行了进一步预训练。这一精炼过程涉及在同时包含代码与自然语言数学内容的高质量数据集上训练。我们尤其关注证明助手中广泛使用的形式化语言，如 Lean、Isabelle 与 Metamath。我们将这一改进后的模型命名为 DeepSeek-Prover-V1.5-Base。

2.2 监督微调

本节探讨 DeepSeek-Prover-V1.5 监督微调（SFT）的方法与流程。具体而言，我们在 DeepSeek-Prover-V1 的证明数据集基础上加入详细的解释性注释以增强数据。这一增强旨在改善自然语言描述与 Lean 4 代码之间的对齐，从而促进更好的形式化数学推理。此外，我们将中间 tactic 状态信息作为辅助预测任务纳入训练，以支撑蒙特卡洛树搜索过程中使用的截断并续写机制。我们将所得模型称为 DeepSeek-Prover-V1.5-SFT。

数据整理（Data Curation） 我们为监督微调构建了一个全面的 Lean 4 代码补全数据集。该数据集包含由大量形式化定理衍生的合成证明代码。这些定理来自多个项目，例如标准 Lean 4 数学库 Mathlib4（Mathlib Community，2020）、来自 DeepSeek-Prover-V1（Xin 等，2024）与 Lean Workbook（Ying 等，2024）的合成定理，以及来自 miniF2F（Zheng 等，2022）与 ProofNet（Azerbayev 等，

2023）基准的验证集。为增强形式化证明数据，我们采用了专家迭代（expert iteration）流程（Polu 和 Sutskever，2020）。该流程包括：使用语言模型生成证明、验证生成的证明数据、用验证通过的数据重新训练模型，然后使用优化后的模型生成更多证明数据。在每次迭代之间，我们使用 DeepSeek-Coder V2 236B（Zhu 等，2024）将证明代码之前的思考过程标注为注释。最后，我们针对蒙特卡洛树搜索的截断并续写机制对这些数据进行适配（详见第 3.1 节）。最终得到的证明数据集包含 9,645k 条序列。

思维增强的证明生成（Thought-augmented Proof Generation） 在 DeepSeek-Prover-V1 中，我们发现了自然语言中的问题求解策略与 Lean 中的定理证明之间存在显著差距。在自然语言中，模型生成详细的推导步骤来构造证明；而在 Lean 中，它们往往依赖一系列高层 tactic 调用来暴力求解。这些高层 tactic 虽然有效，却掩盖了其内部运作与结果，妨碍模型以结构化的数学推理解决复杂证明目标。为解决这一问题，我们开发了一种在生成定理证明代码之前先进行自然语言推理的方法。与 Lean-STaR（Lin 等，2024）在每一步证明之前执行孤立的思维链推理（Wei 等，2022；Feng 等，2023）类似，我们的方法将这种推理直接作为注释整合进证明代码。我们使用 DeepSeek-Coder V2 236B（Zhu 等，2024）以两种方式增强 DeepSeek-Prover-V1 中的现有数据：其一，在证明块开头插入完整的自然语言解答；其二，为对应的 Lean tactic 交替插入具体的自然语言步骤。用这种数据格式训练模型，可迫使其在证明块开头给出完整的数学推理，并在每个 tactic 之前进行详细的步骤规划。该方法成功发展出新的行为，即运用细腻的数学思考来引导 tactic 的生成。在训练数据中，我们使用两种不同的引导提示（guiding prompt）来区分 CoT（思维链）模式与非 CoT 模式下的证明代码补全。两种模式下输入与输出的示例见附录 A。

以 tactic 状态信息增强提示（Prompt Augmentation with Tactic State Information） 为实现蒙特卡洛树搜索所需的截断并续写机制，我们需要从模型生成的代码中提取 tactic 信息。我们利用 LeanDojo（Yang 等，2023）项目的数据提取工具增强了 Lean REPL（Read-Eval-Print Loop；Leanprover Community，2023）。这使我们能够以三元组的形式提取 tactic 信息，其中包括每个 tactic 的位置，以及其应用前后的 tactic 状态。这些信息有助于我们识别触发验证错误的具体 tactic 代码（用于树搜索的扩展步骤，见第 3.2 节）。对于生成的有效形式化证明中的每个 tactic，我们将验证器返回的 tactic 状态以注释 `/- tactic state: ... -/` 的形式插入。训练时，我们将紧随 `/- tactic state: ` 之后的所有 token 作为响应来计算监督微调损失，而该注释之前的 token 用作提示，不参与训练损失计算。

训练设置（Training Setting） 我们在预训练模型的基础上进行监督微调，训练 9B token，批大小为 2,048，学习率恒定为 1e-4。训练过程以 100 步预热（warm-up）开始，以稳定学习动态。训练样本被随机拼接成序列，最大上下文长度为 4,096 token。

2.3 基于证明助手反馈的强化学习

强化学习（RL）已被证明能够有效增强监督微调语言模型的数学推理能力（Shao 等，2024）。为进一步提升 DeepSeek-Prover-V1.5-SFT，我们加入了一个强化学习阶段，得到模型 DeepSeek-Prover-V1.5-RL。该阶段利用 RL，基于 Lean 4 证明器的验证反馈来提升性能。该 RL 过程的具体细节如下。

提示（Prompts） 在强化学习阶段，我们使用监督微调数据集中定理陈述的一个子集作为训练提示。我们挑选那些 DeepSeek-Prover-V1.5-SFT 在多次尝试后能生成正确证明、但成功率适中的定理。这样既能保证模型仍有提升空间，又能让其获得正向反馈。经过筛选，我们保留了约 4.5k 条不重复的定理陈述。每条定理都同时配上 CoT 与非 CoT 引导提示前缀，以增强模型在两种模式下的证明生成能力。

奖励（Rewards） 在通过 RL 训练 LLM 时，通常由训练好的奖励模型提供反馈信号。相比之下，形式化定理证明得益于证明助手对生成证明的严格验证，具有显著优势。具体而言，每个生成的证明若验证通过则获得奖励 1，否则为 0。这种二值奖励信号虽然准确，但也较为稀疏，对于监督微调模型而言具有挑战性的定理尤其如此。为缓解这种稀疏性，如前所述，我们选择那些对监督微调模型既有挑战性又可实现的训练提示。

强化学习算法（Reinforcement Learning Algorithm） 我们采用组相对策略优化（Group Relative Policy Optimization，GRPO；Shao 等，2024）作为 RL 算法，与 PPO（Schulman 等，2017）相比，它展现出更优的有效性与效率，主要原因是它无需训练额外的 critic 模型。具体而言，GRPO 为每个定理提示采样一组候选证明，并基于组内输出的相对奖励来优化模型。我们的提示选择策略旨在使候选中尽可能同时包含正确与错误的证明，这与 GRPO 的组相对特性高度契合，从而改善训练过程。

训练设置（Training Setting） 我们基于 SFT 模型开展 RL 训练，该模型同时作为初始模型与施加 Kullback-Leibler（KL）散度惩罚的参考模型。我们使用恒定的学习率 5e-6，KL 惩罚系数设为 0.02。对每条定理，我们采样一组 32 个候选证明，最大长度设为 2,048。训练批大小设为 512。

【图表】图 3 不同训练阶段的模型能力对比：“CoT”与“non-CoT”指两种引导提示下的评测，阴影区域表示均值附近的标准差范围，记号 μ±σ 表示平均准确率 μ 与标准差 σ（原文第 7 页）

模型	Pass@128（miniF2F-test）	Pass@128（ProofNet-test）
Base（3-shot）	29.7% ± 0.5%	9.7% ± 0.7%
SFT（non-CoT）	49.8% ± 0.3%	15.9% ± 0.5%
SFT（CoT）	50.4% ± 0.4%	15.9% ± 0.6%
RL（non-CoT）	50.5% ± 0.6%	17.5% ± 0.5%
RL（CoT）	51.6% ± 0.5%	18.2% ± 0.5%

2.4 评测

基准（Benchmarks） 我们在以下基准上评估定理证明性能，以比较各训练阶段之后的模型能力：

- **MiniF2F**（Zheng 等，2022）聚焦高中水平习题与竞赛题（如 AMC、AIME 与 IMO）的形式化解题能力，重点关注代数与数论。该基准包含 244 道验证集题目与 244 道测试集题目，原为 Lean 3 格式，后基于 Yang（2023）提供的版本手工转换为 Lean 4.9.0。
- **ProofNet**（Azerbayev 等，2023）评估本科水平数学的形式化定理证明能力。它由来自广泛使用的本科教材的 185 道验证集题目与 186 道测试集题目组成，涵盖实分析与复分析、线性代数、抽象代数与拓扑学。这些题目最初为 Lean 3 格式，后被手工转换为 Lean 4.9.0。

提示配置（Prompting Configurations） 对于 DeepSeek-Prover-V1.5-Base 的每次证明尝试，我们从验证集中独立采样三个证明示范来构造少样本提示。对于 miniF2F 基准，我们使用 Yang（2023）提供的人工撰写证明；对于 ProofNet 基准，我们使用 DeepSeek-Prover-V1.5-RL 生成的正确证明作为少样本示范。对于 DeepSeek-Prover-V1.5-SFT 与 DeepSeek-Prover-V1.5-RL，我们采用两类引导提示：一类鼓励在每个证明步骤前进行思维链（CoT）推理，另一类则不做此要求（non-CoT）。详细示例见附录 A。

指标（Metric） 我们使用 pass@K 准确率指标评估定理证明性能，该指标衡量模型在 K 次尝试内生成正确证明的成功率。每个模型部署在单张 A100-40G GPU 上，并使用 vLLM 框架（Kwon 等，2023）进行样本生成。采样参数设为温度 1、top-p 0.95、最大 token 数 2,048。生成的证明随后使用 Lean 4 定理证明器验证。验证时，我们导入 Mathlib4（Mathlib Community，2020）与 Aesop（Limperg 和 From，2023）以访问预定义的前提（premises）与 tactic。验证过程限时 300 秒。

跨训练阶段比较（Comparison across Training Stages）：图 3 展示了各训练阶段在 miniF2F 与 ProofNet 数据集上的对比分析。我们的基座模型 DeepSeek-Prover-V1.5-Base 取得了可观的通过率，在使用 3-shot 提示的情况下解决了 miniF2F 基准测试集中的近三分之一问题。监督微调阶段产生了 DeepSeek-Prover-V1.5-SFT，其表现显著优于基座模型：在 miniF2F 上 Pass@128 准确率提升约三分之二，在 ProofNet 上则翻倍。随后的强化学习阶段进一步提升了模型性能，在 K 的所有取值下都提高了 Pass@K 准确率。自然语言数学领域报告的结果（如 DeepSeekMath（Shao et al., 2024））表明，强化学习主要提升的是从 TopK 候选中选出正确回答的能力；与此不同，我们观察到形式化定理证明的基础能力得到了真正的提升。这种提升不仅在小采样预算下显而易见，而且随着采样预算增加仍保持稳定。4.2 节中后续使用更大采样预算的蒙特卡洛树搜索实验进一步支持了这一结论。

CoT 与非 CoT 的比较（Comparison between CoT and non-CoT）：我们比较了 DeepSeek-Prover-V1.5-SFT 与 DeepSeek-Prover-V1.5-RL 在非 CoT 与 CoT 两种生成模式下的性能。图 3 中的结果表明，在大多数设置下 CoT 模式始终优于非 CoT 模式。具体而言，DeepSeek-Prover-V1.5-RL 利用这些增强的定理证明模式，在两个基准上都取得了更优性能，在 miniF2F 上的平均准确率为 51.6%，在 ProofNet 上为 18.2%。CoT 模式中自然语言推理的整合显著增强了形式化证明写作的规划与执行。有关使用与不使用自然语言思维链的证明策略的详细比较，请参阅附录 A 中提供的示例。

3 面向探索的蒙特卡洛树搜索

3.1 Tactic 级树抽象

为了在整段证明生成设置下实现树搜索方法，我们引入一种证明树抽象来定义专门设计的状态空间与动作空间，并借助截断-恢复（truncate-and-resume）机制。大致遵循 Yao et al. (2023) 的范式，我们首先将不完整的证明分解为一系列与单个证明步骤对应的树节点，然后利用这些树节点中存储的部分内容继续证明生成过程。图 4 展示了从整段证明生成构建证明搜索树的过程。

截断：将证明分解为树节点（Truncate: Proof Decomposition into Tree Nodes）：我们在 tactic 层面构建证明搜索树，其中每条树边表示 tactic 状态的一次转移步骤。最初，我们将模型生成的完整证明提交给 Lean 证明器，将其解析为 tactic。随后，我们在最早的验证错误处截断证明，从而确保后续所有 tactic 代码都能成功应用，推动证明向目标定理推进。tactic 代码被分割为若干代码片段，每个片段包含一段有效的 tactic 代码及其关联的思维链注释，对应一条表示 tactic 状态转移的树边。通过这种抽象，每段 tactic 代码都被转换为一系列树节点，形成一条从根到特定节点的路径。

【图表】图 4 MCTS 扩展步骤中的截断-恢复机制。(a) 选定一个节点后，我们追踪其对应的不完整证明代码前缀，该前缀包含文件头、初始陈述以及从祖先节点成功应用的 tactic。(b) 语言模型随后基于该前缀以及包含当前 tactic 状态的注释块生成后续证明。(c) 合并后的证明代码（前缀与新生成的代码）由 Lean 4 证明器验证：如果未发现错误，树搜索过程即终止；如果检测到错误，我们在第一条错误消息处截断新生成的代码，丢弃其后代码，并将成功的部分解析为 tactic。(d) 每个 tactic 都作为新节点加入搜索树，在所选节点下方延伸出一条后代链。(e) 树更新完成后，下一轮扩展通过选择另一个候选节点开始，且不限于叶节点。该过程持续重复，直到找到正确证明或采样预算耗尽。（原文第 9 页）

恢复：从树节点生成证明（Resume: Proof Generation from a Tree Node）：在 Lean 4 中，不同的 tactic 可能导向相同的 tactic 状态，这意味着我们证明树中的每个节点可以对应多种实现相同结果的 tactic 代码。为处理这一情况，我们在每个节点存储一组这样的等价 tactic 代码。当树搜索智能体扩展某个节点时，它随机选择其中一个 tactic 作为语言模型的提示。该提示包含以所选 tactic 结尾的不完整证明代码，以及来自 Lean 证明器的 tactic 状态信息（以注释块形式给出）。经过微调的模型（见 2.2 节）已被训练来识别并使用这种格式，利用带有 tactic 状态注释的不完整代码来指导后续的证明生成。

3.2 基于蒙特卡洛树搜索的交互式定理证明

我们的证明搜索树采用标准的蒙特卡洛树搜索（Monte-Carlo Tree Search, MCTS）范式（MCTS；Coulom, 2006； Browne et al., 2012）构建，该范式迭代执行四个步骤：选择（Selection）、扩展（Expansion）、模拟（Simulation）与回传（Backpropagation）。我们将模拟步骤并入扩展步骤，因为我们的整段证明生成模型本身就会从被扩展的节点执行一次 rollout。算法工作流的详细设计如下。

选择（Selection）：选择步骤（又称树策略）从根节点开始向下遍历，以确定一个有希望被扩展的节点。该算法步骤的目标是在探索与利用之间进行权衡（Kocsis and Szepesvári, 2006）。树节点 s 上的树策略通过从有效操作集合中选择使价值最大的动作来计算：

$$\text{TreePolicy}(s) = \arg\max_{a \in \text{Children}(s) \cup \{\varnothing\}} Q_UCB(s, a) \quad (1)$$

其中，动作 a 可以是移动到子节点（记作 $a \in \text{Children}(s)$），也可以是扩展当前节点 s（记作特殊 token $a = \varnothing$）。该方法使用了一种称为虚拟节点（virtual node）的技术（Wang et al., 2023），它为每个节点分配一个虚拟子节点，以表示选择当前节点 s 进行扩展。这让树搜索智能体能够持续扩展非叶节点，因为动作空间由一个生成模型支撑，而其输出范围无法由固定数量的尝试确定。在节点 s 上执行动作 a 的价值估计 Q_UCB(s, a) 由两个分量组成：

$\forall a \in \text{Children}(s) \cup \{\oslash\}$，$Q_UCB(s, a) = Q(s, a)$〔利用（Exploitation）〕+ $UCB(s, a)$〔探索（Exploration）〕　（2）

其中，$Q(s, a)$ 表示根据选择历史得到的、基于采样的动作价值估计，作为利用分量，从以往的试验中检索高价值候选；$UCB(s, a)$ 表示由置信上界（UCB；Auer, 2002）计算出的探索奖励，它随着状态-动作对 (s, a) 的重复执行而减小。更具体地说，$Q_UCB(s, a)$ 表示对 $Q(s, a)$ 的乐观估计，能以较高概率作为其上界。关于节点价值与 UCB 奖励的详细设置，我们留到 3.3 节讨论。

扩展（Expansion）：下一步是调用证明生成模型来扩展由选择阶段指定的节点。我们恢复存储在被指定用于扩展的节点上的不完整证明代码，执行整段证明生成以提出一系列后续 tactic，并将生成的证明提交给 Lean 证明器验证。这样一次证明补全的尝试，等价于在标准 MCTS 框架中执行一次模拟 rollout。当验证结果表明证明已完成时，搜索过程便可在找到目标定理的新证明后终止。否则，我们解析验证反馈，并将生成的证明截断到最早验证错误所在的断言处。剩余的 tactic 被转换为一条节点路径，合并到搜索树中（见图 4）。需要强调的是，由于我们采用整段证明生成设置——输出是由一系列 tactic 构成的整段证明，而不仅仅是下一个 tactic——我们的扩展过程在每次迭代中可能向搜索树中插入一条树节点路径。这与为博弈游戏设计的传统 MCTS 不同，后者每次迭代通常只扩展一层子节点（Silver et al., 2016, 2018; Schrittwieser et al., 2020）。

回传（Backpropagation）：每次树搜索迭代的最后阶段，是沿从根到被扩展节点的选择轨迹更新价值统计量，即更新与式（1）中所述树策略相关的价值。设 $\tau = \{(\text{root}, s^{(1)}), (s^{(1)}, s^{(2)}), (s^{(2)}, s^{(3)}), \ldots, (s^{(|\tau|-1)} = s_t, \oslash)\}$ 表示第 t 次迭代的选择轨迹，该轨迹以 s_t 作为扩展节点结束。我们在考虑最新轨迹奖励 $R(\tau)$ 的情况下，对 τ 中的所有 (s, a) 更新 $Q_UCB(s, a)$（细节参见式（7））。外在奖励来源于编译器反馈，具体而言，对于已完成的证明赋予奖励 $R_extrinsic(\tau) = 1$，对于未解决的证明赋予 $R_extrinsic(\tau) = 0$。在 3.3 节中，我们将引入一种内在奖励机制来增广奖励分配，从而增强智能体的探索动机。

3.3 蒙特卡洛树搜索的内在奖励

在形式化定理证明的搜索问题中，外在奖励极为稀疏，即搜索智能体只有在证明被完全解决时才能获得非零奖励。更具体地说，证明搜索过程形成树状结构，只有一小部分叶节点能带来非零奖励，这与统计强化学习文献中著名的困难探索（hard-exploration）案例（Krishnamurthy et al., 2016）相符。为在稀疏奖励的序贯决策中促进探索，一种经典范式是构建内在奖励（Schmidhuber, 2010），鼓励智能体不仅优化外在奖励，还获取关于交互环境的一般信息（Bellemare et al., 2016; Houthooft et al., 2016;

Pathak et al., 2017; Burda et al., 2019）。在本节中，我们提出由内在奖励驱动的探索算法——应用于树搜索的 RMax（RMax applied to Tree Search, RMaxTS），将无奖励探索引入证明搜索问题。

将 RMax 应用于 MCTS（RMax applied to MCTS）：我们采用经典探索机制 RMax（Brafman and Tennenholtz, 2002）为蒙特卡洛树搜索构建内在奖励。RMax 的核心思想是探索尽可能广泛的状态空间覆盖：智能体在到达未见状态时给自己赋予最大额度的奖励。在证明搜索的语境下，证明完成之前不会提供任何外在奖励，因此我们的算法流程类似于 ZeroRMax（Jin et al., 2020），其中智能体的探索完全由内在奖励驱动，即设置 $R(\tau) = R_intrinsic(\tau)$。树扩展步骤的内在奖励由是否向搜索树中加入新节点决定：

$$R_intrinsic(\tau) = I[\text{至少有一个新节点被加入搜索树}] \quad (3)$$

其中，τ 表示需要对回传进行奖励分配的最新的选择轨迹。该探索策略优先扩展那些证明器模型所生成的 tactic 能导向多样化 tactic 状态的节点。由于多段 Lean 代码可能产生相同的中间状态转移，这一启发式方法有可能减少冗余生成并提高样本效率。

非平稳奖励下的 UCB（UCB for Non-stationary Rewards）：蒙特卡洛树搜索中 UCB 探索奖励的常见设置是使用 UCB1（Auer et al., 2002）：

$$Q_UCB1(s, a) = W(s, a) / N(s, a) + \sqrt{[\ (2 \ln \Sigma_\{a'\}\ N(s, a')) / N(s, a)\ ]} \quad (4)$$

$$W(s, a) = \Sigma_\{\tau \in \Gamma(s, a)\}\ R(\tau) \quad (5)$$

$$N(s, a) = |\Gamma(s, a)| \quad (6)$$

其中，$\Gamma(s, a) = \{\tau \mid (s, a) \in \tau\}$ 表示包含 (s, a) 作为中间选择步骤的树策略轨迹 τ 的列表。为便于讨论，我们将列表 $\Gamma(s, a) = \{\tau_1, \tau_2, \ldots\}$ 组织成这样的顺序：新收集的轨迹具有更大的下标。在本工作中，我们提出使用 UCB 方法的另一种变体。注意，式（3）推导出的内在奖励是一种非平稳奖励信号，其期望值随着探索的推进而衰减。这是因为随着搜索树通过复杂探索不断扩展，发现具有未见 tactic 状态的新节点必然会变得越来越困难。为处理这种非平稳性，我们考虑折扣置信上界（discounted upper confidence bounds, DUCB；Garivier and Moulines, 2011），它使用折扣因子 $\gamma \in (0, 1)$ 来平滑地丢弃过时的反馈记录：

$$Q_DUCB(s, a) = W_\gamma(s, a) / N_\gamma(s, a) + \sqrt{[\ (2 \ln \Sigma_\{a'\}\ N_\gamma(s, a')) / N_\gamma(s, a)\ ]} \quad (7)$$

$$W_\gamma(s, a) = \Sigma_\{t=1\}^\{N(s, a)\}\ \gamma^\{N(s, a)-t\}\ R(\tau_t) \quad (8)$$

$$N_\gamma(s, a) = \Sigma_\{t=0\}^\{N(s, a)-1\}\ \gamma^t \quad (9)$$

其中，新收到的反馈在价值估计中会被赋予更大的权重。在实践中，我们设置 $\gamma = 0.99$。注意，折扣因子 γ 在 DUCB 中的作用不同于其在无限时域 MDP 值迭代中的作用：这里的折扣应用于树搜索迭代，而不是单条轨迹内的动作步时域。

3.4 蒙特卡洛树搜索的并行化

为提升蒙特卡洛树搜索（MCTS）的效率，我们实现了 Chaslot et al. (2008) 所述的若干成熟并行化技术。

- **根并行（Root Parallelization）**：我们在每个节点上部署 256 个 MCTS 运行器，每块 GPU 上运行一个语言模型，证明生成的批大小为 512。Lean 证明器通过 REPL 调用，并运行在一个拥有数千个 CPU 核心的集群上，其中每个证明验证任务由单独的进程处理，该进程在沙箱中创建与终止。语言模型的证明生成与 Lean 证明器的验证均异步处理。这一设置使 MCTS 运行器能够并发执行树搜索操作，显著加速了整个流程。
- **树并行（Tree Parallelization）**：我们用 32 个线程工作单元管理每棵搜索树，以并行化树迭代步骤。该方法有效地调度并平衡了证明生成与 Lean 验证任务。每个线程工作单元迭代执行树搜索循环：选择候选节点进行扩展、调用语言模型生成证明、用 Lean 证明器验证生成的证明，并执行回传。
- **虚拟损失（Virtual Loss）**：为鼓励并发线程工作单元选择不同的节点，我们为正在进行的迭代赋予虚拟奖励 $R(\tau) = 0$。这包括临时回传奖励 0，并在完成时将其更新为真实奖励。该策略促进了对不同节点的扩展探索，从而提升整体搜索效率。

3.5 与现有方法的比较

在本节中，我们将所提出的证明树搜索方法（它引入了用于整段证明生成的新颖截断-恢复机制）与现有方法进行比较。当前将语言模型用于形式化数学证明搜索的方法通常分为两大策略：

- **多遍证明步骤生成（Multi-pass proof-step generation）**：该策略将证明过程分解为多个 tactic 生成与验证的回合，通常遵循树搜索模式。它一次生成并验证一个 tactic，对下一个 tactic 重复该过程，直到不再有证明目标。代表性方法包括 GPT-f（Polu and Sutskever, 2020; Polu et al., 2022）、Thor（Jiang et al., 2022）、ReProver（Yang et al., 2023）、Hypertree Proof Search（Lample et al., 2022）与 InternLM2-StepProver（Wu et al., 2024）。

- **单遍整段证明生成（Single-pass whole-proof generation）**：该方法在一次尝试中生成并验证整个证明。如果证明不正确，模型会在下一次尝试中生成新证明。这一类方法包括 DSP（Jiang et al., 2022）、Subgoal-Prover Zhao et al. (2023)、LEGO-Prover（Wang et al., 2023）、Lyra（Zheng et al., 2023）与 miniCTX（Hu et al., 2024）。

我们的证明树搜索方法独特地连接了这两类策略，提供了一种新颖的混合方法。它以整段证明生成开始，类似于单遍方法，但通过实现精巧的截断-恢复机制对其进行了扩展。该过程包括将生成的证明截断至成功的初始片段、将该片段解析为各个 tactic，并从该点恢复树搜索。这一迭代过程有效地实现了蒙特卡洛树搜索，将单遍整段证明生成与多遍证明步骤生成无缝结合。因此，我们可以用几乎相同的目标训练单个模型，以同时支持两种策略。我们的实验结果表明，这种统一方法在两种设置下都取得了更优性能。通过结合现有方法的优势并引入创新技术，我们的方法为形式化数学证明搜索提供了一种更通用、更有效的解决方案，有望为该领域的未来进展铺平道路。

4 实验结果

在本节中，我们使用两个不同的基准来评估 DeepSeek-Prover-V1.5 的定理证明能力：miniF2F 涵盖高中水平的习题与竞赛题，ProofNet 则涉及本科水平的定理。我们分别给出完整段证明生成与蒙特卡洛树搜索两种方法的结果，并使用与 2.4 节相同的训练模型与推理配置以确保一致性。

4.1 主要结果

我们对 DeepSeek-Prover-V1.5 与先前最先进的语言模型进行比较分析，突出其性能与进展。

- **通用模型（General-purpose Models）**：GPT-3.5 与 GPT-4（OpenAI, 2023）是 OpenAI 开发的先进生成式 AI 模型，以在包括代码生成在内的多种任务上的有效性而著称。尽管并非专为定理证明设计，其庞大的参数规模提供了显著的能力。这些模型在形式化定理证明中的评估通过 COPRA（Thakur et al., 2023）进行，这是一个利用此类大语言模型提出 tactic 应用的上下文学习智能体。此外，我们还考察了 Llemma（Azerbayev et al., 2024），它是一系列……

……在大量通用数学语料上训练的语言模型，它们通常被用作形式化定理证明的基座模型。

- **面向形式化数学的专用模型**：GPT-f（Polu 与 Sutskever，2020；Polu 等，2022）是将 Transformer（Vaswani 等，2017）应用于定理证明任务中证明步骤生成的早期尝试，它利用最佳优先搜索（best-first search）模块构建完整证明。随后的进展包括 ReProver（Yang 等，2023）、LLMStep（Welleck 与 Saha，2023）与 Lean-STaR（Lin 等，2024）。Hypertree Proof Search

（Lample 等，2022）探索了在 Lean 形式化定理证明中使用蒙特卡洛树搜索；同期的工作 InternLM2-Math（Ying 等，2024）与 InternLM2-StepProver（Wu 等，2024）也展现出卓越的性能。

指标：我们使用 pass@K 准确率指标，将 DeepSeek-Prover-V1.5 与当前最先进的模型进行比较。该指标评估模型在 K 次尝试内生成正确证明的能力。为了在不同生成方案之间对齐计算预算，我们按照以下规则给出采样预算 K。

- 对于单遍采样方法，我们将采样预算 K 定义为所生成证明的总数；为便于与树搜索方法比较，较大的 K 值以乘积形式给出。
- 对于最佳优先搜索方法，遵循 Azerbayev 等（2024）的记号，我们给出 $K = N \times S \times T$，其中 N 表示最佳优先搜索的尝试次数，S 表示每次扩展所生成的策略（tactic）数量，T 表示扩展迭代次数。
- 对于树搜索方法，例如 RMaxTS 与 HTPS（Lample 等，2022），我们给出 $K = N \times T$，其中 N 表示树搜索的尝试次数，T 表示树扩展中所调用的模型生成次数。

miniF2F 上的结果：表 1 对 miniF2F-test 数据集上各种定理证明方法进行了对比分析。在单遍完整证明生成设置下，DeepSeek-Prover-V1.5-RL 取得了 60.2% 的最高通过率，相较 DeepSeek-Prover-V1 的 50.0% 显著提升了 10.2 个百分点。在采样预算限制为 128 次尝试时，DeepSeek-Prover-V1.5-RL 证明了 51.6% 的问题，显著优于其他完整证明生成方法，并与领先的树搜索方法相当。在树搜索方法类别中，DeepSeek-Prover-V1.5-RL + RMaxTS 以 62.7% 的通过率领先，确立了新的最先进水平，并与现有方法拉开了显著差距。值得注意的是，DeepSeek-Prover-V1.5-RL 仅需 3200 次完整证明生成采样即可达到 54.9% 的通过率，超过了此前最先进的结果 InternLM2-StepProver——后者执行 64×3200 次树搜索才达到 54.5%。

【表格】表 1 在 miniF2F-test 数据集上与最先进方法的比较。记号 $\mu \pm \sigma$ 表示平均准确率 μ 与标准差 σ。除非另有说明，DeepSeek-Prover-V1.5-Base 的结果基于 3-shot 提示，而 DeepSeek-Prover-V1.5-SFT 与 RL 采用 CoT 模式提示。符号 † 表示使用两个引导提示的混合策略所得到的性能（详见第 4.2 节）（原文第 15 页）

ProofNet 上的结果：表 2 对 ProofNet 数据集上各种定理证明方法进行了对比分析。在单遍完整证明生成设置下以及借助 RMaxTS 增强后，DeepSeek-Prover-V1.5-RL 在整个 ProofNet 数据集上分别取得了 22.6% 与 25.3% 的通过率。这些结果超过了现有最先进方法 ReProver（13.8%）与 InternLM2-

StepProver（18.1%）。当完整证明生成尝试次数限制为 3200 时，DeepSeek-Prover-V1.5 也证明了 21.7% 的定理，相较此前最先进水平 InternLM2-StepProver 提升了 3.6%。

【表格】表 2 在 ProofNet 数据集上与最先进方法的比较。‡ 注意，ProofNet 的验证集被用于在监督微调中执行专家迭代（原文第 16 页）

4.2 重新审视训练策略在大规模采样下的有效性

我们在大规模采样设置下重新审视若干训练模块的效果，同时关注单遍完整证明生成与蒙特卡洛树搜索。结果表明，第 2.4 节中给出的观察与发现可以推广到样本量很大的采样场景。

强化学习的普遍增益：为支持“基于验证反馈的在线强化学习能够普遍增强模型能力”这一主张，我们在较大的采样预算下将最终模型与仅做 SFT 的版本进行比较。比较结果以表 3 中的两列呈现。无论是否采用思维链（chain-of-thought）策略，DeepSeek-Prover-V1.5-RL 在所有生成设置下都稳定优于 SFT 模型。结果还表明，通过在线 RL 获得的提升与通过 RMaxTS 获得的提升是正交的，二者可以进一步结合以提升性能。通过同时整合 CoT 提示与 RMaxTS，DeepSeek-Prover-V1.5-RL 在 miniF2F-test 上取得了 62.7% 的通过率。这一性能较 SFT 模型显著提升了 3.7%，凸显了强化学习在提升证明补全模型整体有效性方面的关键作用。

CoT、非 CoT 与混合策略：我们在 miniF2F-test 数据集上比较了两种生成模式（即非 CoT 与 CoT）的性能。如表 3 所示，结果表明随着采样预算增大，CoT 相对于非 CoT 模式的优势会被放大。这表明引入自然语言思维链可以使定理证明的规划路径多样化，从而可能带来更广泛的推理策略与更具创新性的解法。结果还显示，这两种模式在不同问题上具有互补优势。在 CoT 模式下，模型的定理证明策略在数学思维上更加系统、更具主动性；而在非 CoT 模式下，模型能够高效地使用 Lean 的高层策略（tactic）来解决那些可由 Lean 自动化机制处理的计算问题。为利用这些优势，我们考虑一种混合策略（在表 3 中记为 non-CoT & CoT），将一半采样预算分配给 CoT 模式，其余分配给非 CoT 模式。两个引导提示的这种简单组合在进一步提升证明补全模型性能方面展现出很大潜力，在 miniF2F-test 上取得了 63.5% 的通过率。附录 B 中给出了示例问题，用以说明两种生成模式的不同优势。

【表格】表 3 一项大规模消融研究，用于考察若干算法设计在模型训练中的有效性。结果在 miniF2F-test 数据集上评估（原文第 17 页）

4.3 RMaxTS 的消融研究

内在奖励与折扣 UCB：我们考察 RMaxTS 的两个核心组成部分的有效性，即公式（3）定义的内在奖励（intrinsic rewards）与公式（7）表述的折扣上置信界（discounted upper confidence bound）。我们以一个实现标准 UCT 算法（Kocsis 与 Szepesvári，2006）且不含内在奖励的基线为起点，其中探索完全由 UCB 奖励项驱动。注意，由于该基线未提供任何非零奖励，UCB 公式的所有变体都变得等价，因为节点选择仅由访问次数决定。图 5 中的实验结果表明，在缺少内在奖励的情况下，UCT（without R_intrinsic）的性能退化到与无搜索方法相当的水平。此外，我们考察了用标准 UCB1（参见公式（4））替代折扣 UCB 的 RMaxTS，记为 RMaxTS（DUCB → UCB1）。结果表明，采用 UCB1 奖励项的 RMaxTS 性能同样一般，与 UCT（without R_intrinsic）相当。这是因为 UCB1 旨在通过穷尽式探索来保证渐近性能（Auer 等，2002），其前提是样本量足够大。相比之下，折扣 UCB 能够加速非平稳内在奖励的价值传播，防止 R_intrinsic 的引导被访问次数的影响所主导。这表明折扣 UCB 机制是内在奖励驱动探索的重要补充。

【图表】图 5 一项考察 RMaxTS 算法设计的模块化消融研究。实验在 miniF2F-test 数据集上使用 CoT 模式的 DeepSeek-Prover-V1.5-RL 进行。左侧面板展示了 6400 个生成样本内的 Pass@K 准确率曲线；右侧面板给出了更大样本量下的结果（原文第 18 页）

策略状态信息的引导：在扩展树节点时，我们将中间策略状态（tactic state）信息作为注释块拼接到不完整代码上，以引导证明补全。借助所提供的辅助信息，证明补全模型可以增强其对策略状态的内部表示，从而为长时程规划提供中间引导。为证明这一优势，我们在 RMaxTS 上进行了实验：该实验直接从原始的不完整代码进行代码补全，而不访问策略状态信息，在图 5 中记为 RMaxTS（without tactic state）。结果表明，在缺少策略状态信息时，应用树搜索带来的性能增益变得有限，尤其是在处理需要大量样本的困难问题时。这凸显了编译器信息的整合是树搜索算法不可或缺的组成部分，能够提升其整体有效性与样本效率。

5 结论、局限与未来工作

我们提出 DeepSeek-Prover-V1.5，这是一个拥有 70 亿参数的语言模型，在 Lean 4 的形式化定理证明中优于所有开源模型。DeepSeek-Prover-V1.5 以 DeepSeek-Prover-V1.5-Base 为初始化，后者在 DeepSeekMath-Base 7B 的预训练基础上，使用面向形式化数学推理的专用语料进行了扩展。我们在一

个全面的 Lean 4 代码补全数据集上进行监督微调，该数据集涵盖了来自各种数学领域的广泛形式化定理。随后，我们采用 GRPO 通过在线强化学习增强完整证明生成。在开发出 DeepSeek-Prover-V1.5 模型之后，我们引入 RMaxTS——蒙特卡洛树搜索的一种变体——通过带有广泛探索的大规模搜索来提升问题求解能力。这些组件构成了训练基于 LLM 的证明助手的完整流水线，使 DeepSeek-Prover-V1.5 相较于 DeepSeek-Prover-V1 取得了显著提升。

DeepSeek-Prover-V1.5 的框架旨在为形式化定理证明建立一条类 AlphaZero 的流水线。专家迭代与合成数据的使用呼应了强化学习的核心试错循环，而编译器预言机（oracle）则充当提供环境监督的世界模型。在强化学习范式中，集成的树搜索模块已被证明能够在多个领域推动超越人类的性能（Silver 等，2016；Fawzi 等，2022；Lutz 等，2023）。在开发 DeepSeek-Prover-V1.5 的过程中，我们关注 RL 的探索（exploration）方面，引入 RMaxTS 以使证明步骤的生成多样化。然而，利用（exploitation）方面——特别是证明搜索问题——仍未被探索。一个有前景的未来方向是训练一个评论家（critic）模型来评估不完整的证明并剪枝搜索分支。这样的部分证明评论家模型将隐式地执行时序信用分配（temporal credit assignment）（Sutton，1984），把证明级别的反馈分解为步骤级别的价值差异（Arjona-Medina 等，2019）。为评估长规划路径并提供引导奖励而开发评论家模型，是一个关键且具有挑战性的问题（Ng 与 Russell，2000；Sorg 等，2010），值得进一步研究。

最后，近期的工作已经从证明单个定理发展到处理复杂的、包含多个定理的 Lean 文件中的真实理论证明（Hu 等，2024）。这一转变是我们完整证明生成方法的自然延伸。我们的观察表明，当前模型已经具备一定的文件级上下文理解能力。展望未来，我们将专注于增强这一方面，以我们的语言模型进展来支持前沿的 Lean 数学形式化开发者。

附录

附录 A 非 CoT 与 CoT 提示在证明补全中的示例

附录 B MiniF2F-test 问题的示例解答

附录 C 非 CoT 优于 CoT 的问题

附录 D CoT 优于非 CoT 的问题

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] Prover-V1.5, we focus on the exploration aspect of RL, introducing RMaxTS to diversify the generation of proof steps. However, the exploitation aspect, specifically the problem of proof search, remains unexplored. A promising future direction is training a critic model to assess incomplete proofs and prune search branches. Such a partial-proof critic model would implicitly perform temporal credit assignment (Sutton, 1984), decomposing proof-level feedback into stepwise value differences (Arjona-Medina et al., 2019). Developing critic models for assessing long planning paths and providing guidance rewards presents a crucial and challenging problem (Ng and Russell, 2000; Sorg et al., 2010) that warrants further investigation.

[2] Finally, recent work has progressed beyond proving individual theorems to addressing real-world theory proving within complex, multi-theorem Lean files (Hu et al., 2024). This shift is a natural extension of our whole-proof generation approach. Our observations indicate that the current model already possesses some understanding of file-level context. Moving forward, we will focus on enhancing this aspect to support cutting-edge Lean mathematical formalization developers with our language model advancements.

[3] [1] J. A. Arjona-Medina, M. Gillhofer, M. Widrich, T. Unterthiner, J. Brandstetter, and S. Hochreiter. Rudder: Return decomposition for delayed rewards. In Proceedings of the 33rd International Conference on Neural Information Processing Systems, pages 13566–13577, 2019.

[4] [2] P . Auer. Using confidence bounds for exploitation-exploration trade-offs. Journal of Machine Learning Research, 3(Nov):397–422, 2002.

[5] [3] P . Auer, N. Cesa-Bianchi, and P . Fischer. Finite-time analysis of the multiarmed bandit problem. Machine learning, 47:235–256, 2002.

[6] [4] Z. Azerbayev, B. Piotrowski, H. Schoelkopf, E. W. Ayers, D. Radev, and J. Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. arXiv preprint arXiv:2302.12433, 2023.

[7] [5] Z. Azerbayev, H. Schoelkopf, K. Paster, M. Dos Santos, S. M. McAleer, A. Q. Jiang, J. Deng, S. Biderman, and S. Welleck. Llemma: An open language model for mathematics. In The Twelfth International Conference on Learning Representations, 2024.

[8] [6] M. G. Bellemare, S. Srinivasan, G. Ostrovski, T. Schaul, D. Saxton, and R. Munos. Unifying count-based exploration and intrinsic motivation. In Proceedings of the 30th International Conference on Neural Information Processing Systems, pages 1479–1487, 2016.

[9] [7] R. I. Brafman and M. Tennenholtz. R-max-a general polynomial time algorithm for nearoptimal reinforcement learning. Journal of Machine Learning Research, 3(Oct):213–231, 2002.

[10] [8] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P . I. Cowling, P . Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. A survey of monte carlo tree search methods. IEEE Transactions on Computational Intelligence and AI in games, 4(1):1–43, 2012.

[11] [9] Y. Burda, H. Edwards, A. Storkey, and O. Klimov. Exploration by random network distillation. In Seventh International Conference on Learning Representations, pages 1–17, 2019. [10] G. M. B. Chaslot, M. H. Winands, and H. J. van Den Herik. Parallel monte-carlo tree search. In Computers and Games: 6th International Conference, CG 2008, Beijing, China, September 29-October 1, 2008. Proceedings 6, pages 60–71. Springer, 2008.

[12] [11] R. Coulom. Efficient selectivity and backup operators in Monte-Carlo tree search. In International conference on computers and games, pages 72–83. Springer, 2006.

[13] [12] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R Ruiz, J. Schrittwieser, G. Swirszcz, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. Nature, 610(7930):47–53, 2022.

[14] [13] G. Feng, B. Zhang, Y. Gu, H. Ye, D. He, and L. Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In Thirty-seventh Conference on Neural Information Processing Systems, volume 36, 2023.

[15] [14] A. Garivier and E. Moulines. On upper-confidence bound policies for switching bandit problems. In International conference on algorithmic learning theory, pages 174–188. Springer, 2011.

[16] [15] R. Houthooft, X. Chen, Y. Duan, J. Schulman, F. De Turck, and P . Abbeel. Vime: variational information maximizing exploration. In Proceedings of the 30th International Conference on Neural Information Processing Systems, pages 1117–1125, 2016.

[17] [16] J. Hu, T. Zhu, and S. Welleck. minictx: Neural theorem proving with (long-)contexts, 2024.

[18] [17] A. Q. Jiang, W. Li, S. Tworkowski, K. Czechowski, T. Odrzygó´ zd´ z, P . Miło´ s, Y. Wu, and M. Jamnik. Thor: wielding hammers to integrate language models and automated theorem provers. In Proceedings of the 36th International Conference on Neural Information Processing Systems, pages 8360–8373, 2022.

[19] [18] A. Q. Jiang, S. Welleck, J. P . Zhou, T. Lacroix, J. Liu, W. Li, M. Jamnik, G. Lample, and Y. Wu. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In The Eleventh International Conference on Learning Representations, 2022.

[20] [19] C. Jin, A. Krishnamurthy, M. Simchowitz, and T. Yu. Reward-free exploration for reinforcement learning. In International Conference on Machine Learning, pages 4870–4879. PMLR, 2020.

[21] European conference [20] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In on machine learning, pages 282–293. Springer, 2006.

[22] [21] A. Krishnamurthy, A. Agarwal, and J. Langford. PAC reinforcement learning with rich observations. In Proceedings of the 30th International Conference on Neural Information Processing Systems, pages 1848–1856, 2016.

[23] [22] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 2023.

[24] [23] G. Lample, M.-A. Lachaux, T. Lavril, X. Martinet, A. Hayat, G. Ebner, A. Rodriguez, and T. Lacroix. Hypertree proof search for neural theorem proving. In Proceedings of the 36th International Conference on Neural Information Processing Systems, pages 26337–26349, 2022. [24] Leanprover Community. A read-eval-print-loop for Lean 4. https://github.com/lea nprover-community/repl , 2023.

[25] [25] J. Limperg and A. H. From. Aesop: White-box best-first proof search for lean. In Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, pages 253–266, 2023.

[26] [26] H. Lin, Z. Sun, Y. Yang, and S. Welleck. Lean-star: Learning to interleave thinking and proving. arXiv preprint arXiv:2407.10040, 2024.

[27] [27] I. D. Lutz, S. Wang, C. Norn, A. Courbet, A. J. Borst, Y. T. Zhao, A. Dosey, L. Cao, J. Xu, E. M. Leaf, et al. Top-down design of protein architectures with reinforcement learning. Science, 380(6642):266–273, 2023.

[28] [28] Mathlib Community. The Lean mathematical library. In Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, page 367–381. Association for Computing Machinery, 2020.

[29] [29] L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In Automated Deduction–CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings 28, pages 625–635. Springer, 2021.

[30] [30] A. Y. Ng and S. J. Russell. Algorithms for inverse reinforcement learning. In Proceedings of the Seventeenth International Conference on Machine Learning, pages 663–670, 2000.

[31] [31] OpenAI. Gpt-4 technical report. arXiv preprint arXiv:2303.08774, 2023.

[32] [32] D. Pathak, P . Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven exploration by selfsupervised prediction. In International conference on machine learning, pages 2778–2787. PMLR, 2017.

[33] [33] L. C. Paulson. Isabelle a Generic Theorem Prover. Springer Verlag, 1994.

[34] [34] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. arXiv preprint arXiv:2009.03393, 2020.

[35] [35] S. Polu, J. M. Han, K. Zheng, M. Baksys, I. Babuschkin, and I. Sutskever. Formal mathematics statement curriculum learning. arXiv preprint arXiv:2202.01344, 2022.

[36] [36] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In Proceedings of the fourteenth international conference on artificial intelligence and statistics, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.

[37] IEEE [37] J. Schmidhuber. Formal theory of creativity, fun, and intrinsic motivation (1990–2010). transactions on autonomous mental development, 2(3):230–247, 2010.

[38] [38] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. Nature, 588(7839):604–609, 2020.

[39] [39] J. Schulman, F. Wolski, P . Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017. [40] Z. Shao, P . Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[40] [41] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V . Panneershelvam, M. Lanctot, et al. Mastering the game of go with deep neural networks and tree search. nature, 529(7587):484–489, 2016.

[41] [42] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. Science, 362(6419):1140–1144, 2018.

[42] [43] J. Sorg, S. Singh, and R. L. Lewis. Reward design via online gradient ascent. In Proceedings of the 23rd International Conference on Neural Information Processing Systems-Volume 2, pages 2190–2198, 2010.

[43] [44] R. S. Sutton. Temporal credit assignment in reinforcement learning. Phd thesis, University of Massachusetts, 1984.

[44] [45] A. Thakur, Y. Wen, and S. Chaudhuri. A language-agent approach to formal theoremproving. In The 3rd Workshop on Mathematical Reasoning and AI at NeurIPS, 2023.

[45] [46] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[46] [47] H. Wang, H. Xin, C. Zheng, Z. Liu, Q. Cao, Y. Huang, J. Xiong, H. Shi, E. Xie, J. Yin, et al. Lego-prover: Neural theorem proving with growing libraries. In The Twelfth International Conference on Learning Representations, 2023.

[47] [48] H. Wang, Y. Yuan, Z. Liu, J. Shen, Y. Yin, J. Xiong, E. Xie, H. Shi, Y. Li, L. Li, et al. DT-solver: Automated theorem proving with dynamic-tree sampling guided by proof-level value function. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 12632–12646, 2023.

[48] [49] R. Wang, J. Zhang, Y. Jia, R. Pan, S. Diao, R. Pi, and T. Zhang. Theoremllama: Transforming general-purpose llms into lean4 experts. arXiv preprint arXiv:2407.03203, 2024.

[49] [50] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V . Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In Proceedings of the 36th International Conference on Neural Information Processing Systems, pages 24824–24837, 2022.

[50] [51] S. Welleck and R. Saha. llmstep: Llm proofstep suggestions in lean. In The 3rd Workshop on Mathematical Reasoning and AI at NeurIPS'23, 2023.

[51] [52] Z. Wu, J. Wang, D. Lin, and K. Chen. Lean-github: Compiling github lean repositories for a versatile lean prover. arXiv preprint arXiv:2407.17227, 2024.

[52] [53] H. Xin, D. Guo, Z. Shao, Z. Ren, Q. Zhu, B. Liu, C. Ruan, W. Li, and X. Liang. Deepseekprover: Advancing theorem proving in llms through large-scale synthetic data. arXiv preprint arXiv:2405.14333, 2024.

[53] [54] K. Yang. minif2f-lean4. https://github.com/yangky11/miniF2F-lean4 , 2023. [55] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P . Song, S. Yu, S. Godil, R. Prenger, and A. Anandkumar. Leandojo: theorem proving with retrieval-augmented language models. In Proceedings of the 37th International Conference on Neural Information Processing Systems, pages 21573–21612, 2023.

[54] [56] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: deliberate problem solving with large language models. In Proceedings of the 37th International Conference on Neural Information Processing Systems, pages 11809– 11822, 2023.

[55] [57] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen. Lean workbook: A largescale lean problem set formalized from natural language math problems. arXiv preprint arXiv:2406.03847, 2024.

[56] [58] H. Ying, S. Zhang, L. Li, Z. Zhou, Y. Shao, Z. Fei, Y. Ma, J. Hong, K. Liu, Z. Wang, et al. Internlm-math: Open math large language models toward verifiable reasoning. arXiv preprint arXiv:2402.06332, 2024.

[57] [59] X. Zhao, W. Li, and L. Kong. Decomposing the enigma: Subgoal-based demonstration learning for formal theorem proving. arXiv preprint arXiv:2305.16366, 2023.

[58] [60] C. Zheng, H. Wang, E. Xie, Z. Liu, J. Sun, H. Xin, J. Shen, Z. Li, and Y. Li. Lyra: Orchestrating dual correction in automated theorem proving. arXiv preprint arXiv:2309.15806, 2023.

[59] [61] K. Zheng, J. M. Han, and S. Polu. miniF2F: a cross-system benchmark for formal olympiadlevel mathematics. In International Conference on Learning Representations, 2022.

[60] [62] Q. Zhu, D. Guo, Z. Shao, D. Yang, P . Wang, R. Xu, Y. Wu, Y. Li, H. Gao, S. Ma, et al. DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence. arXiv preprint arXiv:2406.11931, 2024.

DeepSeek-VL2：面向高级多模态理解的混合专家视觉语言模型

DeepSeek-VL2: Mixture-of-Experts Vision-Language Models for Advanced Multimodal Understanding

Zhiyu Wu ∗, Xiaokang Chen ∗, Zizheng Pan ∗, Xingchao Liu ∗, Wen Liu ∗,†,

DeepSeek-AI research@deepseek.com

arXiv:2412.10302 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-VL2，这是一系列先进的大规模混合专家（MoE）视觉语言模型，通过两项关键升级显著改进了其前代 DeepSeek-VL。在视觉组件方面，我们引入了动态平铺（dynamic tiling）视觉编码策略，专为处理具有不同宽高比的高分辨率图像而设计。在语言组件方面，我们采用结合多头潜在注意力（MLA）机制的 DeepSeekMoE 模型，该机制将键值缓存压缩为潜在向量，从而实现高效推理与高吞吐。在改进的视觉语言数据集上训练后，DeepSeek-VL2 在多种任务上展现出卓越能力，包括但不限于视觉问答、光学字符识别、文档/表格/图表理解以及视觉定位。我们的模型系列由三个变体组成：DeepSeek-VL2-Tiny、DeepSeek-VL2-Small 和 DeepSeek-VL2，激活参数分别为 1.0B、2.8B 和 4.5B。与现有的开源稠密模型和基于 MoE 的模型相比，DeepSeek-VL2 在激活参数相当或更少的情况下取得了相当或最先进的性能。代码与预训练模型已在 https://github.com/deepseek-ai/DeepSeek-VL2 公开。

【图表】图 1 不同开源模型的平均性能随激活参数量的变化。我们平均了 MMBench v1.1、MMStar、MMMU（Val）、MathVista（TestMini）、AI2D（Test）与 OCRBench 的准确率。OCRBench 的分数被除以 10，以缩放到 [0, 100] 区间（原文第 1 页）

1 引言

大视觉-语言模型（Large Vision-Language Model, VLM）已成为人工智能领域的一股变革性力量[15, 54, 59, 63, 83, 88, 94]，将大语言模型（LLM）的强大能力扩展到可同时无缝处理视觉与文本信息。这一进展大幅拓展了 AI 系统应对需要多模态理解的复杂真实世界应用的潜力。

在本技术报告中，我们提出 DeepSeek-VL2——一系列新的开源视觉-语言模型。它利用混合专家（Mixture-of-Experts, MoE）架构，相较前代 DeepSeek-VL [59] 在性能与效率上均实现了大幅提升。我们的改进集中在三个关键方面：（1）提升视觉理解的动态高分辨率视觉编码策略；（2）显著改善训练与推理效率的优化语言模型架构；（3）改进的视觉-语言数据构建流水线，它不仅提升整体性能，还将模型能力扩展到精确视觉定位等新领域。

在视觉部分，我们引入动态平铺视觉编码策略，能够高效处理不同宽高比的高分辨率图像。相比 DeepSeek-VL 的混合视觉编码器（它从 384×384 与 1024×1024 两种固定分辨率的图像中提取特征），该方法有所改进。我们的方法避免了旧式固定尺寸编码器的局限，在需要超高分辨率的任务中表现出色，包括视觉定位、文档/表格/图表分析以及细节特征提取，同时将视觉 token 数量保持在可控范围。受已有切片平铺（slicing-tile）方法的启发，我们的系统将高分辨率输入动态分割为局部图块，通过共享的视觉 Transformer 处理每个图块，并在语言模型内无缝整合提取出的特征。这种设计保留了局部注意力视觉 Transformer 的优势，能够实现丰富的特征提取，而不会出现通常随图像分辨率提高而带来的二次方计算开销。

在语言部分，我们采用 DeepSeek 语言模型 [20, 53]，其特点是多头潜在注意力（Multi-head Latent Attention, MLA）机制。MLA 通过将键值（Key-Value, KV）缓存压缩为潜在向量，显著降低计算成本，从而实现更快的推理与更高的吞吐容量。我们还通过采用稀疏计算技术的 DeepSeekMoE 框架[20, 86] 进一步提升效率。我们的模型系列采用三种 MoE 变体：3B、16B 与 27B；这些 LLM 分别具有 0.57B、2.4B 与 4.1B 激活参数。

我们还从质量、数量与多样性三个方面大幅增强了视觉-语言训练数据。这一全面的数据集使模型能够在广泛任务上获得更好的泛化与性能，包括视觉问答（Visual Question Answering, VQA）、光学字符识别（Optical Character Recognition, OCR）、文档/表格/图表理解、视觉推理以及通用对话应用。改进后的训练数据还使视觉定位与图形用户界面（Graphical User Interface, GUI）感知等新能力成为可能。

总之，DeepSeek-VL2 标志着大规模混合专家视觉-语言建模的一次重大跃升。通过新的视觉处理策略与优化的语言模型，我们开发了一系列在性能与效率之间取得平衡的模型。通过开源预训练模型，我们希望加速该领域的进展并促进协作研究的发展。

【图表】图 2 DeepSeek-VL2 概览。整体结构为 llava 风格的架构，包含一个视觉编码器、一个视觉-语言适配器以及一个基于 MoE 的大语言模型（原文第 4 页）

2 模型架构

DeepSeek-VL2 由三个核心模块组成：（1）视觉编码器；（2）视觉-语言适配器；（3）混合专家语言模型。DeepSeek-VL2 在前代仅解码器（decoder-only）的 LLaVA 风格 [54] 架构之上，引入两项重大改进：动态平铺策略，以及配备多头潜在注意力 [53] 的 DeepSeekMoE [20, 86] 语言模型。这些创新使模型能够更高效地处理高分辨率视觉输入与文本数据。

动态平铺策略。原始 DeepSeek-VL 采用混合视觉编码器，将 SigLIP [106]（在 384×384 分辨率下进行粗粒度特征提取）与 SAM-B [35]（在 1024×1024 分辨率下进行细粒度特征提取）相结合。尽管这种融合方式能够生成适合各类视觉-语言任务的丰富视觉表示，但它受限于固定的 1024×1024 分辨率约束。这一限制在处理分辨率更高、宽高比极端的图像时尤为困难，例如 InfographicVQA [67]、密集 OCR 与细粒度视觉定位任务中的图像。

受 VLM 领域近期进展 [16, 21, 55] 的启发，我们通过将高分辨率图像切分为图块（tile）来实现动态平铺策略。该方法使用单个 SigLIP-SO400M-384 视觉编码器 [106]，即可高效处理宽高比各异的不同高分辨率图像。预训练的 SigLIP 以 384×384 为基础分辨率运行。为适配不同宽高比，我们定义一组候选分辨率：$C_R = \{(m\cdot 384, n\cdot 384) \mid m \in N, n \in N, 1 \le m, n, mn \le 9\}$，其中 m:n 表示宽高比。对于尺寸为 (H, W) 的输入图像，我们计算将其缩放（resize）到 C_R 中每个候选分辨率所需的填充区域（具体做法是：先把原始图像缩放至长边与目标分辨率一致，再在保持原始宽高比的前提下填充另一维度）。我们选择使填充区域最小的分辨率 $(m_i\cdot 384, n_i\cdot 384)$。随后，缩放后的图像被划分为 $m_i\times n_i$ 个 384×384 像素的局部图块，另加一个全局缩略图图块。SigLIP-SO400M-384 视觉编码器处理全部 $(1+m_i\times n_i)$ 个图块，每个图块产生 27×27 = 729 个 1152 维视觉嵌入。为提升计算效率并管理上下文长度，当处理多张（>2）图像时，我们禁用动态平铺策略。

【图表】图 3 动态平铺策略示意图。通过将图像划分为多个图块，DeepSeek-VL2 相比 DeepSeek-VL 获得了更强的细

粒度理解能力（原文第 5 页）

【表格】表 1 DeepSeek-VL2 的架构配置。我们列出了架构的超参数以及混合专家训练相关的细节（原文第 5 页）

视觉-语言适配器。在完成视觉图块处理后，我们执行 2×2 像素重排（pixel shuffle）操作，将每个图块的视觉 token 从 27×27 压缩到 14×14 = 196 个。随后，在处理 (1+m_i×n_i) 个图块时，我们引入三种特殊 token。对于全局缩略图图块（14×14），我们在每行末尾添加 14 个 <tile_newline> token，共得到 14×15 = 210 个 token。对于 m_i×n_i 个局部图块（它们以形状为 (m_i·14, n_i·14) 的二维网格排列），我们在最后一列末尾追加 m_i·14 个 <tile_newline> token，以标示所有局部图块某一行结束。此外，我们在全局缩略图图块与局部图块之间插入一个 <view_separator> token。完整的视觉序列包含 210 + 1 + m_i·14 × (n_i·14 + 1) 个视觉 token，随后通过两层多层感知机（multilayer perceptron, MLP）投影到语言模型的嵌入空间。图 3 给出了动态平铺策略的可视化示意。

DeepSeekMoE 语言模型。我们的语言模型基于 DeepSeekMoE [20, 86]，并引入多头潜在注意力机制 [53]。MLA 通过将键值缓存压缩为潜在向量来提升推理效率，从而提高吞吐容量。该模型还采用了 MoE 架构 [20]，通过稀疏计算实现高效推理。在 MoE 训练中，我们为每个专家引入一个全局偏置项 [86]，以经济有效的方式改善专家之间的负载均衡。DeepSeek-VL2 提供三种模型规模的变体：1.0B、2.8B 与 4.5B。完整的架构规格见表 1。

3 数据构建

我们为 DeepSeek-VL2 从多样化的来源构建了一个全面的视觉-语言数据集。训练过程分为三个不同阶段：（1）视觉-语言对齐；（2）视觉-语言预训练；（3）监督微调（SFT）。下面我们分别描述各阶段所使用的数据。

3.1 视觉-语言对齐数据

对齐阶段聚焦于训练 MLP 连接器，以衔接预训练的视觉编码器与 LLM。在这一初始预热阶段，我们使用 ShareGPT4V [12]——一个包含约 120 万条描述与对话样本的数据集。

3.2 视觉-语言预训练数据

遵循 DeepSeek-VL [59] 的做法，我们的预训练数据将视觉-语言（VL）数据与纯文本数据相结合，以在 VL 能力与纯文本性能之间保持平衡。对于 DeepSeek-VL2，我们保持约 70% VL 数据与 30% 纯文本数据的比例，后者直接取自基座 LLM 的预训练语料。下面我们将 VL 数据划分为若干组并描述其细节。

图文交错数据。我们的数据收集始于若干开源数据集，包括 WIT [79]、WikiHow [38] 以及从 OBELICS [41] 中随机抽取的 30% 样本。这一特定混合比例是通过在 DeepSeek-VL2-Tiny 上的初步实验确定的。为增强多语言能力，我们用在 Wanjuan [29] 中提取的中文内容补充了以英文为主的数据集。此外，我们还构建了一个内部数据集，以扩大对通用真实世界知识的覆盖。

图像描述数据。图像描述（caption）是 VLM 训练中的基础数据，为视觉与文本信息之间提供直接对齐。我们最初利用了多种开源数据集 [8, 25, 28, 36, 37, 39, 40, 48, 50, 51, 73, 78, 80, 82]。然而，我们的初步分析发现这些数据集的质量差异很大：既有先进 VLM 生成的密集而准确的描述，也有描述过于简短、图文不匹配或存在明显幻觉等问题样本。为解决这些质量不一致问题，我们开发了一套全面的图像描述流水线，它考虑：（1）OCR 提示；（2）元信息（例如位置、相机设置）；（3）相关的原始描述作为提示。我们使用内部的描述生成器，按照与 PixelProse [78] 类似的提示策略对图像重新生成描述（recaption），并采用多样化的指令引导 VLM 生成描述。

尽管描述质量整体有所提升，我们仍在大规模标注流水线中观察到重复问题。为缓解这一问题，我们实现了一套质量控制流水线，使用 DeepSeek Chat [53] 仅根据写作质量对所有描述进行打分。在实践中，该方法既高效又能有效过滤低质量描述。

光学字符识别数据。为培养 OCR 能力，我们使用了包括 LaTeX OCR [7] 与 12M RenderedText [93] 在内的开源数据集，并将这些数据集与覆盖多种文档类型的、规模庞大的内部 OCR 数据集相结合。目前，我们的内部数据集主要聚焦于英文与中文字符识别。我们计划在未来工作中扩展到其他语言。

视觉问答（QA）数据。在早期探索中，我们发现通用问答数据明显有益于模型预训练。因此，我们构建了一个全面的视觉问答数据集，包含以下类别：

- **通用 VQA。**我们沿用 DeepSeek-VL 的通用 VQA 数据。更多细节请参见 [59]。
- **表格、图表与文档理解。**我们采用 PubTabNet [112]、FinTabNet [111] 与 Docmatix [42] 来增强文档理解能力。

- **网页到代码与图表到 Python 代码生成。**遵循 DeepSeek-VL 的做法，我们利用 Websight [44] 提升网页到代码的能力，并使用从公开 Jupyter notebook 中获取的 Python 图表。我们通过使用 DeepSeek V2.5 复现一部分 Websight 来增强该数据集。我们还利用 DeepSeek V2.5 生成的 Python 绘图代码，以缓解图表到代码数据中的噪声。
- **带视觉提示的问答。**我们遵循 [9] 的做法，通过在取自 [9, 89, 90] 的图像上叠加各种视觉指示（箭头、方框、圆圈与涂鸦）来构建视觉提示理解数据。随后，我们围绕这些视觉提示所突出显示的对象创建问答对。

视觉定位数据。我们基于 [71, 75] 构建视觉定位数据集。对于每张图像的目标检测标注，我们将数据组织如下：

- 提示（Prompt）：Locate <|ref|><query><|/ref|> in the given image.
- 回答（Response）：<|ref|><query><|/ref|><|det|>[[x1, y1, x2, y2], ...]<|/det|>

在训练期间，问题提示从候选池中随机采样。<|ref|>、<|/ref|>、<|det|>、<|/det|> 是特殊 token。<query> 是占位符，可为类别名称（例如 "car"）或对象描述（例如 "the leftmost person"）。[[x1, y1, x2, y2], ...] 是边界框列表，其中每个边界框对应一个对象的位置。坐标 x1、y1 与 x2、y2 分别指定左上角与右下角，并根据图像分辨率归一化为 0 到 999 之间的值。我们还构建了查询对象被有意从图像中移除的负样本，以增强模型的稳健性。

定位对话数据。我们从 [71] 派生出定位对话（grounded conversation）数据集，其组织格式如下：

- 提示（Prompt）：<|grounding|>Can you describe the content of the image?
- 回答（Response）：Two <|ref|>dogs<|/ref|><|det|>[[x1, y1, x2, y2], ...]<|/det|> are running on the grass.

与其他视觉定位数据一样，<|grounding|>、<|ref|>、<|/ref|>、<|det|>、<|/det|> 是特殊 token，且 x1、y1、x2、y2 遵循相同的归一化方案。

3.3 监督微调数据

我们的 SFT 数据由多样化的开源数据集与高质量的内部问答对组合而成。下面我们详细介绍为提升 SFT 数据集质量所做的努力。

通用视觉问答 尽管公开的视觉问答数据集十分多样 [9, 10, 27, 31, 43, 47, 74]，但它们通常存在三个主要局限：（1）回复过短；（2）OCR 质量较差；（3）内容存在幻觉。为解决这些问题，我们通过联合考虑原始问题、图像与 OCR 信息来重新生成回复。实验表明，该方法能产生更全面、更准确的结

果。在开发过程中，我们观察到早期版本的 DeepSeek-VL2（尤其是 Tiny 版本）偶尔会在中文回复中不恰当地插入英文单词。这一问题在我们的较大模型中并不存在，表明它源于模型容量有限以及视觉-语言预训练阶段中英文数据不均衡。为在较小模型中解决这一局限，我们构建了一个内部中文问答数据集，包含多样化的图像描述与单轮/多轮对话。该数据集有助于缓解语言混杂问题。此外，我们还构建了一个额外的内部数据集，以补充真实世界与文化视觉知识，涵盖动漫、表情包、美食与艺术。

OCR 与文档理解 得益于我们先进的图像描述（image captioning）流程，DeepSeek-VL2 已展现出优于其他最先进 VLM 的 OCR 能力。因此，我们并未在 SFT 阶段进一步增强 OCR 性能，而是专注于清洗现有开源数据集 [24, 31, 43, 66, 67, 77, 92, 104]，移除 OCR 质量较差的样本。对于文档理解，我们从内部数据中整理了多样化的文档页面子集，然后针对文档理解生成多轮对话问答对。早期结果表明，该方法提升了基于文档的交互效果。

表格与图表理解 我们基于原始问题为所有公开数据集 [14, 49] 重新生成回复，以增强表格问答数据，但 Cauldron [43] 除外，因为它已经具有高质量。与我们在视觉-语言预训练阶段发展出的 OCR 能力类似，我们的模型在图表理解上也表现出强劲性能，无需额外的专门投入。

推理、逻辑与数学 我们对公开的推理类数据集 [17, 43, 61, 76, 102, 109] 补充更详细的推理过程，并统一回复格式，将最终答案放在回复末尾。我们观察到，在训练较小的 VLM 时，详细回复的效果较差。在探索中，DeepSeek-VL2-Tiny 使用更简洁的回复表现更好。

教材与学术问题 我们构建了一个内部数据集，聚焦于文档集合中的教材内容。该数据集主要强调多个学科方向的大学水平内容。

网页到代码与图表到 Python 代码生成 我们扩展了内部数据集，用于网页代码与 Python 绘图代码生成，规模超出预训练阶段所用的数据。对于开源数据集，我们通过重新生成答案来提升其质量。

视觉定位 我们使用 [2, 23, 64, 85, 101, 110] 中的数据构建视觉定位数据集。为增强模型能力，我们将查询短语翻译为中文，并创建额外的负样本。我们还加入了上下文视觉定位（in-context visual grounding）数据，其任务是在给定参考图像中用矩形或椭圆标出的参考物体的条件下，在多张图像中定位同一类别的物体。数据格式如下：

- 提示（Prompt）：<|grounding|>The first image shows <object>. Please identify the object of the same category in the second image.
- 回复（Response）：<|ref|><description><|/ref|><|det|>[[x1, y1, x2, y2]]<|/det|>

在该格式中，<|grounding|>、<|ref|>、<|/ref|>、<|det|>、<|/det|> 为特殊 token。<object> 占位符表示诸如“红色边界框中的物体”之类的短语，而 <description> 是模型对所检测物体的描述（例如“cat”）。

定位对话 我们使用 [62, 72] 构建定位对话（grounded conversation）数据，以进一步增强模型在预训练阶段所建立的能力。

纯文本数据集 为保持模型的语言能力，我们在 SFT 阶段也使用纯文本指令微调数据集 [4, 6, 18, 19, 68, 70, 84, 91, 98]。

4 训练方法

4.1 训练流程

DeepSeek-VL2 通过三阶段流程训练：（1）初始阶段：固定语言模型，仅训练视觉编码器与视觉-语言适配器 MLP，使用第 3.1 节详述的图像-文本配对数据；（2）预训练阶段：使用第 3.2 节所述数据开展视觉-语言预训练；（3）微调阶段：使用第 3.3 节概述的数据进行监督微调。在预训练与微调阶段，视觉编码器、视觉-语言适配器与语言模型等所有模型参数均被解冻并同时训练。在所有阶段中，我们都强调视觉理解能力，并仅在文本 token 上计算下一 token 预测损失。

视觉-语言对齐 在预训练语言模型（DeepSeekMoE 3B/16B/27B）的基础上，我们的首要目标是建立视觉特征与语言特征之间的稳健连接。这种对齐使预训练语言模型能够有效处理视觉输入。与以往保持预训练视觉编码器与语言模型固定的方法 [54, 59] 不同，我们让固定分辨率的视觉编码器适配动态高分辨率图像。在这一阶段，我们优化视觉编码器与视觉-语言适配器，同时冻结语言模型。

视觉-语言预训练 在嵌入空间中建立视觉-语言对齐之后，我们将大部分计算资源投入到视觉-语言预训练中。该阶段专注于在多样化任务上发展全面的联合视觉-语言知识。我们解冻包括视觉编码器、视觉-语言适配器 MLP 与 DeepSeekMoE LLM 在内的所有参数，以实现全模型优化。该阶段使用约 800B 图像-文本 token（第 3.2 节），显著增强了模型的多模态理解能力，同时保持其大部分语言能力。

【表格】表 2 训练 DeepSeek-VL2 的超参数。Step LR 调度器在总训练步数的 50% 与 75% 处将学习率除以 $\sqrt{10}$

（原文第 10 页）

监督微调 在最后阶段，我们通过监督微调增强预训练模型的指令遵循与对话能力。我们使用内部视觉-语言 SFT 数据，优化所有参数，但仅在答案与特殊 token 上计算监督信号，并对系统提示与用户提示进行掩码。为增强对话理解能力，我们将多模态数据与来自 DeepSeek-V2 [53] 的纯文本对话数据相结合。这一方法确保模型在多样化的视觉-语言任务上具有稳健性能，包括密集图像描述、通用 VQA、OCR、表格/图表/文档/图形理解、视觉到代码、视觉推理、视觉定位与语言理解等。

4.2 超参数与基础设施

DeepSeek-VL2 训练的详细超参数见表 2。我们使用 HAI-LLM [30]（一个为大型模型设计的高效轻量平台）进行训练与评测。我们的流水线并行策略面临的一个重大挑战，来自视觉编码器相对 LLM 模块的独特计算特性。作为模型流水线中的第一个组件，视觉编码器需要仔细的跨 GPU 负载均衡，以避免流水线气泡（pipeline bubble）并优化 GPU 利用率。为此，我们在流水线并行策略中对视觉编码器实现了细粒度的层切分。此外，我们在前向与反向过程中跨不同数据并行 rank 执行图像瓦片（tile）负载均衡，以缓解动态分辨率策略导致的图像瓦片数量不均衡。我们的训练过程还结合了张量并行与专家并行方法，以达到最高效率。由于部分数据批次仅含文本数据而另一些包含图像数据，我们为不同数据类型引入了两种不同的流水线策略，并按需在两种策略之间切换。DeepSeek-VL2 的训练分别使用 16/33/42 个节点、在 7/10/14 天内完成，每个节点配备 8 块 NVIDIA A100 GPU。

【表格】表 3 与最先进模型在 OCR 相关多模态基准上的对比。†：MoE 模型的激活参数（原文第 11 页）

5 评测

5.1 多模态性能

基准 我们在一系列常用基准上对 DeepSeek-VL2 进行整体评测，包括 DocVQA [66]、ChartQA [65]、InfoVQA² [67]、TextVQA [77]、RealWorldQA [95]、OCRBench [57]、AI2D [34]、MMMU [105]、MMStar [13]、MathVista [60]、MME [26]、MMBench、MMBench-V1.1 [58] 与 MMT-Bench [100]。这些基准涵盖从文档理解、图表解读到真实世界问题求解的多样任务，可全面评估模型的能力。为评估模型的定位能力，我们在 RefCOCO、RefCOCO+ 与 RefCOCOg 基准 [33, 64] 上测试 DeepSeek-VL2。

[2] 由于 InfoVQA 包含极端长宽比的图像与尺寸过大的图像，评测时我们将候选分辨率扩大为 CR = {(m·384, n·384) | m ∈ N, n ∈ N, 1 ≤ m, n, mn ≤ 18}。

【表格】表 4 与最先进模型在通用问答与数学相关多模态基准上的对比。†：MoE 模型的激活参数；*：在不同设置下评测（原文第 12 页）

与最先进方法的对比 在多模态理解基准上，我们将 DeepSeek-VL2 与最先进模型进行比较，包括 LLaVA-OV [45]、InternVL2 [15]、DeepSeek-VL [59]、Qwen2-VL [88]、Phi-3.5-Vision [1]、Molmo [22]、Pixtral [3]、MM1.5 [107] 与 Aria-MoE [46]。结果报告在表 3 与表 4 中。得益于我们的 MoE 架构，DeepSeek-VL2 以更少的激活参数取得相当或更好的性能。在定位基准上，我们将 DeepSeek-VL2 与 Grounding DINO [56]、UNINEXT [52]、ONE-PEACE [87]、mPLUG-2 [97]、Florence-2 [96]、InternVL2 [16]、Shikra [11]、TextHawk2 [103]、Ferret-v2 [108]、MM1.5 [107] 与 Qwen2 [88] 进行比较。我们的模型在相近规模下优于其他 VLM。

5.2 定性研究

本节展示 DeepSeek-VL2 的不同能力，涵盖从通用问答到视觉叙事与视觉定位。

通用视觉问答 得益于我们新的视觉-语言预训练数据集与多样化的 SFT 数据，DeepSeek-VL2 在通用视觉问答上的能力显著提升，如图 4 所示。总体而言，该模型擅长密集图像描述，能够识别常见地标、通用视觉知识以及中英文的丰富文本。它在图表理解上也表现良好，能准确识别属性。此外，我们在图 5 中展示了 DeepSeek-VL2 改进的表情包理解能力，它能够描述正确的语境，并借助有意义的文化背景解释其中的幽默。

【图表】图 4 DeepSeek-VL2 的通用问答能力。我们的模型具备多方面能力，包括地标识别、基于图像的诗歌创作、回答常识问题、理解图表、识别文本等（原文第 13 页）

【图表】图 5 DeepSeek-VL2 的梗图理解能力。我们的模型能够理解梗图中的幽默并给出解释（原文第 14 页）

【图表】图 6 DeepSeek-VL2 的多图对话能力。我们的模型能够理解多张图像之间的关系，并基于这些图像进行一定的推理（原文第 15 页）

【图表】图 7 DeepSeek-VL2 的视觉故事讲述能力。我们的模型可以接受多张图像作为输入，并基于这些图像用中文

或英文讲述一个故事（原文第 16 页）

【表格】表 5 在视觉定位（visual grounding）基准上与最先进模型的对比。我们不同规模的模型在同等规模的 MLLM 中均取得了最佳结果（原文第 17 页）

多图对话。DeepSeek-VL2 在多图对话方面的能力有所提升，如图 6 所示。我们的模型能够分析多张图像之间的关联与差异，同时还能通过整合若干图像的内容进行简单推理。例如，它可以基于某些食材的图像思考如何做一道菜。

视觉故事讲述。在图 7 中，我们展示了 DeepSeek-VL2 能够根据少量图像写出富有创意的故事。故事写作能力得益于其强大的通用视觉能力，例如地标识别与 OCR——这些内容在图中以绿色文字高亮显示。此外，由于故事写作能力源自纯文本的 DeepSeek Chat 模型，而该模型已经具备良好的安全对齐，我们在内部测试中未观察到 DeepSeek-VL2 产生明显有害或 NSFW 的输出。然而值得注意的是，真实世界场景中的创意故事讲述需要更多样的体裁（例如恐怖、喜剧、动作）以及更丰富的情节类型（例如圆满或悲剧结局），这可能与 LLM/VLM 研究中的安全要求存在内在冲突。我们希望在考虑这些挑战的同时，探索扩大故事讲述范围的解决方案。

视觉定位。视觉定位是我们为 DeepSeek-VL2 新引入的一项能力。在图 8 中，我们展示了 DeepSeek-VL2 的通用定位能力。有趣的是，尽管我们训练集中的绝大多数图像来自自然场景，且指代表达式多为物体类别名称或物体的具体描述，我们发现模型能够泛化到其他场景（例如梗图与动漫），并具备识别某些名人与抽象概念的能力。此外，我们在图 10 中展示了 DeepSeek-VL2 具备上下文内（in-context）视觉定位能力。给定第一张图像（其中一个物体由视觉提示指代），模型能够在第二张图像中定位到同一类别的物体。我们还观察到模型展现出涌现能力：给定一张图像与文本描述，模型能够结合图像与文本信息，在第二张图像中识别出对应的物体。示例见图 10 的第二行与第三行。

【图表】图 8 DeepSeek-VL2 的视觉定位能力。我们的模型能够依据物体的类别名称、描述或某些抽象概念来定位物体（原文第 18 页）

【图表】图 9 与 DeepSeek-VL2 的带定位对话。我们的模型能够在图像上进行推理，同时识别相关物体的位置，从而使与真实世界交互成为可能（原文第 19 页）

【图表】图 10 DeepSeek-VL2 的上下文内视觉定位。给定一张图像（无论是否带视觉提示），DeepSeek-VL2 都能够

在另一张图像中找到相关物体（原文第 19 页）

带定位对话（Grounded conversation）。借助特殊 token<|grounding|>，DeepSeek-VL2 能够释放其进行带定位对话的能力：它可以在回复中指出关键物体并给出准确位置，如图 9 所示。这使模型能够更好地与真实世界交互，从而有机会在具身 AI 以及电脑/手机智能体（agent）等领域发挥更大的作用。

6 结论

在本技术报告中，我们提出了 DeepSeek-VL2——基于 MoE 的视觉-语言模型的增强版本，提供 3B、16B 与 27B 三种总参数规模，对应的激活参数量分别为 1.0B、2.8B 与 4.5B。这一配置使训练与推理阶段的计算开销都保持高效。值得注意的是，我们的 3B、16B 与 27B 模型可分别部署在显存为 10 GB、40GB 与 80GB 的单张 GPU 上。我们采用动态分块（dynamic tiling）视觉编码策略，以高效处理不同宽高比的高分辨率图像。通过公开代码与预训练模型，我们希望推动视觉与语言交叉领域的进一步研究与应用。

局限与未来工作。尽管 DeepSeek-VL2 在各类任务上展现出强大的能力，仍有若干方面有待未来改进。目前，DeepSeek-VL2 的上下文窗口仅允许每次会话处理少量图像。我们计划在下一个版本中扩展上下文窗口，以实现更丰富的多图交互。此外，与其他现有 VLM 一样，模型偶尔会在处理模糊图像或未见过的物体时遇到困难，这为未来版本提升鲁棒性提供了机会。最后，尽管 DeepSeek-VL2 在视觉感知与识别任务上表现出色，我们希望进一步加强其推理能力。这些已识别的方面指引着我们持续的研究方向，我们将继续提升模型的能力。

附录

附录：补充材料（如有）见英文原文

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] [1] M. Abdin, J. Aneja, H. Awadalla, A. Awadallah, A. A. Awan, N. Bach, A. Bahree, A. Bakhtiari, J. Bao, H. Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. arXiv preprint arXiv:2404.14219, 2024.

[2] [2] agentsea. Wave-ui 25k. https://huggingface.co/datasets/agentsea/wave-u i-25k , 2024.

[3] [3] P . Agrawal, S. Antoniak, E. B. Hanna, B. Bout, D. Chaplot, J. Chudnovsky, D. Costa, B. De Monicault, S. Garg, T. Gervet, et al. Pixtral 12b. arXiv preprint arXiv:2410.07073, 2024.

[4] [4] A. Amini, S. Gabriel, P . Lin, R. Koncel-Kedziorski, Y. Choi, and H. Hajishirzi. Mathqa: Towards interpretable math word problem solving with operation-based formalisms. arXiv preprint arXiv:1905.13319, 2019.

[5] [5] Anthropic. Claude 3.5 sonnet. https://www.anthropic.com/news/claude-3-5-s onnet , 2024.

[6] [6] Y. Bai, X. Du, Y. Liang, Y. Jin, Z. Liu, J. Zhou, T. Zheng, X. Zhang, N. Ma, Z. Wang, et al. Coig-cqia: Quality is all you need for chinese instruction fine-tuning. arXiv preprint arXiv:2403.18058, 2024.

[7] [7] L. Blecher. Latex-ocr — a tool to convert images of latex equations into latex code. https://github.com/lukas-blecher/LaTeX-OCR , 2023. Accessed: 2023-10-17.

[8] [8] O. B. Bohan and H. Face. Megalith 10m dataset. https://huggingface.co/dataset s/madebyollin/megalith-10m , 2024.

[9] [9] M. Cai, H. Liu, S. K. Mustikovela, G. P . Meyer, Y. Chai, D. Park, and Y. J. Lee. Vip-llava: Making large multimodal models understand arbitrary visual prompts. In CVPR, pages 12914–12923. IEEE, 2024.

[10] [10] G. H. Chen, S. Chen, R. Zhang, J. Chen, X. Wu, Z. Zhang, Z. Chen, J. Li, X. Wan, and B. Wang. Allava: Harnessing gpt4v-synthesized data for a lite vision-language model. arXiv preprint arXiv:2402.11684, 2024.

[11] [11] K. Chen, Z. Zhang, W. Zeng, R. Zhang, F. Zhu, and R. Zhao. Shikra: Unleashing multimodal llm's referential dialogue magic. arXiv preprint arXiv:2306.15195, 2023.

[12] [12] L. Chen, J. Li, X. Dong, P . Zhang, C. He, J. Wang, F. Zhao, and D. Lin. Sharegpt4v: Improving large multi-modal models with better captions. ECCV, 2023.

[13] [13] L. Chen, J. Li, X. Dong, P . Zhang, Y. Zang, Z. Chen, H. Duan, J. Wang, Y. Qiao, D. Lin, et al. Are we on the right way for evaluating large vision-language models? arXiv preprint arXiv:2403.20330, 2024.

[14] [14] W. Chen, H. Wang, J. Chen, Y. Zhang, H. Wang, S. Li, X. Zhou, and W. Y. Wang. Tabfact: A large-scale dataset for table-based fact verification. In International Conference on Learning Representations.

[15] [15] Z. Chen, J. Wu, W. Wang, W. Su, G. Chen, S. Xing, M. Zhong, Q. Zhang, X. Zhu, L. Lu, B. Li, P . Luo, T. Lu, Y. Qiao, and J. Dai. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. arXiv preprint arXiv:2312.14238, 2023. [16] Z. Chen, W. Wang, H. Tian, S. Ye, Z. Gao, E. Cui, W. Tong, K. Hu, J. Luo, Z. Ma, et al. Internvl2: Better than the best—expanding performance boundaries of open-source multimodal models with the progressive scaling strategy, 2024.

[16] [17] A. Cherian, K.-C. Peng, S. Lohit, K. Smith, and J. B. Tenenbaum. Are deep neural networks smarter than second graders? arXiv preprint arXiv:2212.09993, 2022.

[17] [18] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[18] [19] M. Conover, M. Hayes, A. Mathur, J. Xie, J. Wan, S. Shah, A. Ghodsi, P . Wendell, M. Zaharia, and R. Xin. Free dolly: Introducing the world's first truly open instruction-tuned llm, 2023. URL https://www.databricks.com/blog/2023/04/12/dolly-first-ope n-commercially-viable-instruction-tuned-llm .

[19] [20] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. arXiv preprint arXiv:2401.06066, 2024.

[20] [21] W. Dai, N. Lee, B. Wang, Z. Yang, Z. Liu, J. Barker, T. Rintamaki, M. Shoeybi, B. Catanzaro, and W. Ping. Nvlm: Open frontier-class multimodal llms. arXiv preprint, 2024.

[21] [22] M. Deitke, C. Clark, S. Lee, R. Tripathi, Y. Yang, J. S. Park, M. Salehi, N. Muennighoff, K. Lo, L. Soldaini, et al. Molmo and pixmo: Open weights and open data for state-of-the-art multimodal models. arXiv preprint arXiv:2409.17146, 2024.

[22] [23] X. Deng, Y. Gu, B. Zheng, S. Chen, S. Stevens, B. Wang, H. Sun, and Y. Su. Mind2web: Towards a generalist agent for the web. Advances in Neural Information Processing Systems, 36, 2024.

[23] [24] M. Diem, S. Fiel, F. Kleber, R. Sablatnig, J. M. Saavedra, D. Contreras, J. M. Barrios, and L. S. Oliveira. Icfhr 2014 competition on handwritten digit string recognition in challenging datasets (hdsrc 2014). In 2014 14th International Conference on Frontiers in Handwriting Recognition, pages 779–784. IEEE, 2014.

[24] [25] B. Egan, A. Redden, XWAVE, and SilentAntagonist. Dalle3 1 Million+ High Quality Captions, May 2024. URL https://huggingface.co/datasets/ProGamerGov/sy nthetic-dataset-1m-dalle3-high-quality-captions .

[25] [26] C. Fu, P . Chen, Y. Shen, Y. Qin, M. Zhang, X. Lin, J. Yang, X. Zheng, K. Li, X. Sun, Y. Wu, and R. Ji. Mme: A comprehensive evaluation benchmark for multimodal large language models, 2024. URL https://arxiv.org/abs/2306.13394 .

[26] [27] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh. Making the V in VQA matter: Elevating the role of image understanding in Visual Question Answering. In Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[27] [28] J. Gu, X. Meng, G. Lu, L. Hou, N. Minzhe, X. Liang, L. Yao, R. Huang, W. Zhang, X. Jiang, C. Xu, and H. Xu. Wukong: A 100 million large-scale chinese cross-modal pre-training benchmark. In NeurIPS, 2022.

[28] [29] C. He, Z. Jin, C. Xu, J. Qiu, B. Wang, W. Li, H. Yan, J. Wang, and D. Lin. Wanjuan: A comprehensive multimodal dataset for advancing english and chinese large models. arXiv preprint arXiv:2308.10755, 2023. [30] High-flyer. HAI-LLM: Efficient and lightweight training tool for large models, 2023. URL https://www.high-flyer.cn/en/blog/hai-llm .

[29] [31] D. A. Hudson and C. D. Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 6700–6709, 2019.

[30] [32] A. Hurst, A. Lerer, A. P . Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford, et al. Gpt-4v(ision) system card. 2023.

[31] [33] S. Kazemzadeh, V . Ordonez, M. Matten, and T. Berg. Referitgame: Referring to objects in photographs of natural scenes. In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pages 787–798, 2014.

[32] [34] A. Kembhavi, M. Salvato, E. Kolve, M. Seo, H. Hajishirzi, and A. Farhadi. A diagram is worth a dozen images. In Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14, pages 235– 251. Springer, 2016.

[33] [35] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al. Segment anything. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 4015–4026, 2023.

[34] [36] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al. Segment anything. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 4015–4026, 2023.

[35] [37] Y. Kirstain, A. Polyak, U. Singer, S. Matiana, J. Penna, and O. Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. In NeurIPS, 2023.

[36] [38] M. Koupaee and W. Y. Wang. Wikihow: A large scale text summarization dataset. arXiv preprint arXiv:1810.09305, 2018.

[37] [39] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Malloci, A. Kolesnikov, T. Duerig, and V . Ferrari. The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale. IJCV, 2020.

[38] [40] LAION. Laion-aesthetics, 2023. URL https://laion.ai/blog/laion-aesthetics . Accessed: 2023-10-27.

[39] [41] H. Laurençon, L. Saulnier, L. Tronchon, S. Bekman, A. Singh, A. Lozhkov, T. Wang, S. Karamcheti, A. M. Rush, D. Kiela, M. Cord, and V . Sanh. OBELICS: an open web-scale filtered dataset of interleaved image-text documents. In NeurIPS, 2023.

[40] [42] H. Laurençon, A. Marafioti, V . Sanh, and L. Tronchon. Building and better understanding vision-language models: insights and future directions., 2024.

[41] [43] H. Laurençon, L. Tronchon, M. Cord, and V . Sanh. What matters when building visionlanguage models?, 2024.

[42] [44] H. Laurençon, L. Tronchon, and V . Sanh. Unlocking the conversion of web screenshots into html code with the websight dataset, 2024.

[43] [45] B. Li, Y. Zhang, D. Guo, R. Zhang, F. Li, H. Zhang, K. Zhang, P . Zhang, Y. Li, Z. Liu, et al. Llava-onevision: Easy visual task transfer. arXiv preprint arXiv:2408.03326, 2024. [46] D. Li, Y. Liu, H. Wu, Y. Wang, Z. Shen, B. Qu, X. Niu, G. Wang, B. Chen, and J. Li. Aria: An open multimodal native mixture-of-experts model. arXiv preprint arXiv:2410.05993, 2024.

[44] [47] F. Li, R. Zhang, H. Zhang, Y. Zhang, B. Li, W. Li, Z. Ma, and C. Li. Llava-nextinterleave: Tackling multi-image, video, and 3d in large multimodal models. arXiv preprint arXiv:2407.07895, 2024.

[45] [48] L. Li, Y. Wang, R. Xu, P . Wang, X. Feng, L. Kong, and Q. Liu. Multimodal ArXiv: A dataset for improving scientific comprehension of large vision-language models. In ACL, 2024.

[46] [49] L. Li, Y. Wang, R. Xu, P . Wang, X. Feng, L. Kong, and Q. Liu. Multimodal arxiv: A dataset for improving scientific comprehension of large vision-language models. arXiv preprint arXiv:2403.00231, 2024.

[47] [50] X. Li, F. Zhang, H. Diao, Y. Wang, X. Wang, and L.-Y. Duan. Densefusion-1m: Merging vision experts for comprehensive multimodal perception. arXiv preprint arXiv:2407.08303, 2024.

[48] [51] Z. Li, X. Yang, K. Choi, W. Zhu, R. Hsieh, H. Kim, J. H. Lim, S. Ji, B. Lee, X. Yan, et al. Mmsci: A dataset for graduate-level multi-discipline multimodal scientific understanding. arXiv preprint arXiv:2407.04903, 2024.

[49] [52] F. Lin, J. Yuan, S. Wu, F. Wang, and Z. Wang. Uninext: Exploring a unified architecture for vision recognition. In Proceedings of the 31st ACM International Conference on Multimedia, pages 3200–3208, 2023.

[50] [53] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024.

[51] [54] H. Liu, C. Li, Q. Wu, and Y. J. Lee. Visual instruction tuning. Advances in neural information processing systems, 36, 2023.

[52] [55] H. Liu, C. Li, Y. Li, B. Li, Y. Zhang, S. Shen, and Y. J. Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024. URL https://llava-vl.github.io/blog /2024-01-30-llava-next/ .

[53] [56] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In European Conference on Computer Vision, pages 38–55. Springer, 2025.

[54] [57] Y. Liu, Z. Li, B. Yang, C. Li, X. Yin, C.-l. Liu, L. Jin, and X. Bai. On the hidden mystery of ocr in large multimodal models. arXiv preprint arXiv:2305.07895, 2023.

[55] [58] Y. Liu, H. Duan, Y. Zhang, B. Li, S. Zhang, W. Zhao, Y. Yuan, J. Wang, C. He, Z. Liu, et al. Mmbench: Is your multi-modal model an all-around player? In European Conference on Computer Vision, pages 216–233. Springer, 2025.

[56] [59] H. Lu, W. Liu, B. Zhang, B. Wang, K. Dong, B. Liu, J. Sun, T. Ren, Z. Li, H. Yang, et al. Deepseek-vl: towards real-world vision-language understanding. arXiv preprint arXiv:2403.05525, 2024. [60] P . Lu, H. Bansal, T. Xia, J. Liu, C. Li, H. Hajishirzi, H. Cheng, K.-W. Chang, M. Galley, and J. Gao. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. In The Twelfth International Conference on Learning Representations.

[57] [61] P . Lu, L. Qiu, J. Chen, T. Xia, Y. Zhao, W. Zhang, Z. Yu, X. Liang, and S.-C. Zhu. Iconqa: A new benchmark for abstract diagram understanding and visual language reasoning. arXiv preprint arXiv:2110.13214, 2021.

[58] [62] C. Ma, Y. Jiang, J. Wu, Z. Yuan, and X. Qi. Groma: Localized visual tokenization for grounding multimodal large language models. In European Conference on Computer Vision, pages 417–435. Springer, 2025.

[59] [63] Y. Ma, X. Liu, X. Chen, W. Liu, C. Wu, Z. Wu, Z. Pan, Z. Xie, H. Zhang, L. Zhao, et al. Janusflow: Harmonizing autoregression and rectified flow for unified multimodal understanding and generation. arXiv preprint arXiv:2411.07975, 2024.

[60] [64] J. Mao, J. Huang, A. Toshev, O. Camburu, A. L. Yuille, and K. Murphy. Generation and comprehension of unambiguous object descriptions. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 11–20, 2016.

[61] [65] A. Masry, D. X. Long, J. Q. Tan, S. Joty, and E. Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. arXiv preprint arXiv:2203.10244, 2022.

[62] [66] M. Mathew, D. Karatzas, and C. Jawahar. Docvqa: A dataset for vqa on document images. In Proceedings of the IEEE/CVF winter conference on applications of computer vision, pages 2200–2209, 2021.

[63] [67] M. Mathew, V . Bagal, R. Tito, D. Karatzas, E. Valveny, and C. Jawahar. Infographicvqa. In Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pages 1697–1706, 2022.

[64] [68] A. Mitra, H. Khanpour, C. Rosset, and A. Awadallah. Orca-math: Unlocking the potential of slms in grade school math, 2024.

[65] [69] OpenAI. Gpt-4v(ision) system card. https://openai.com/research/gpt-4v-sys tem-card , 2023.

[66] [70] B. Peng, C. Li, P . He, M. Galley, and J. Gao. Instruction tuning with gpt-4. arXiv preprint arXiv:2304.03277, 2023.

[67] [71] Z. Peng, W. Wang, L. Dong, Y. Hao, S. Huang, S. Ma, and F. Wei. Kosmos-2: Grounding multimodal large language models to the world. arXiv preprint arXiv:2306.14824, 2023.

[68] [72] B. A. Plummer, L. Wang, C. M. Cervantes, J. C. Caicedo, J. Hockenmaier, and S. Lazebnik. Flickr30k entities: Collecting region-to-phrase correspondences for richer imageto-sentence models. In Proceedings of the IEEE international conference on computer vision, pages 2641–2649, 2015.

[69] [73] B. Saleh and A. Elgammal. Large-scale classification of fine-art paintings: Learning the right metric on the right feature. arXiv preprint arXiv:1505.00855, 2015.

[70] [74] S. Shah, A. Mishra, N. Yadati, and P . P . Talukdar. Kvqa: Knowledge-aware visual question answering. In Proceedings of the AAAI conference on artificial intelligence, volume 33, pages 8876–8884, 2019. [75] S. Shao, Z. Li, T. Zhang, C. Peng, G. Yu, X. Zhang, J. Li, and J. Sun. Objects365: A large-scale, high-quality dataset for object detection. In Proceedings of the IEEE/CVF international conference on computer vision, pages 8430–8439, 2019.

[71] [76] W. Shi, Z. Hu, Y. Bin, J. Liu, Y. Yang, S.-K. Ng, L. Bing, and R. K.-W. Lee. Math-llava: Bootstrapping mathematical reasoning for multimodal large language models. arXiv preprint arXiv:2406.17294, 2024.

[72] [77] A. Singh, V . Natarajan, M. Shah, Y. Jiang, X. Chen, D. Batra, D. Parikh, and M. Rohrbach. Towards vqa models that can read. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 8317–8326, 2019.

[73] [78] V . Singla, K. Yue, S. Paul, R. Shirkavand, M. Jayawardhana, A. Ganjdanesh, H. Huang, A. Bhatele, G. Somepalli, and T. Goldstein. From pixels to prose: A large dataset of dense image captions. CoRR, abs/2406.10328, 2024.

[74] [79] K. Srinivasan, K. Raman, J. Chen, M. Bendersky, and M. Najork. Wit: Wikipedia-based image text dataset for multimodal multilingual machine learning. In SIGIR, page 2443–2449, 2021.

[75] [80] K. Sun, J. Pan, Y. Ge, H. Li, H. Duan, X. Wu, R. Zhang, A. Zhou, Z. Qin, Y. Wang, J. Dai, Y. Qiao, L. Wang, and H. Li. Journeydb: A benchmark for generative image understanding. In NeurIPS, 2023.

[76] [81] G. Team, P . Georgiev, V . I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530, 2024.

[77] [82] B. Thomee, D. A. Shamma, G. Friedland, B. Elizalde, K. Ni, D. Poland, D. Borth, and L.-J. Li. Yfcc100m: The new data in multimedia research. Communications of the ACM, 59(2): 64–73, 2016.

[78] [83] S. Tong, E. L. Brown II, P . Wu, S. Woo, A. J. IYER, S. C. Akula, S. Yang, J. Yang, M. Middepogu, Z. Wang, et al. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. In The Thirty-eighth Annual Conference on Neural Information Processing Systems.

[79] [84] S. Toshniwal, W. Du, I. Moshkov, B. Kisacanin, A. Ayrapetyan, and I. Gitman. Openmathinstruct-2: Accelerating ai for math with massive open-source instruction data. arXiv preprint arXiv:2410.01560, 2024.

[80] [85] J. Wang, P . Zhang, T. Chu, Y. Cao, Y. Zhou, T. Wu, B. Wang, C. He, and D. Lin. V3det: Vast vocabulary visual detection dataset. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 19844–19854, 2023.

[81] [86] L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. CoRR, abs/2408.15664, 2024. doi: 10.48550/ARXIV.2408.15664. URL https://doi.org/10.48550/arXiv.2408.15664 .

[82] [87] P . Wang, S. Wang, J. Lin, S. Bai, X. Zhou, J. Zhou, X. Wang, and C. Zhou. One-peace: Exploring one general representation model toward unlimited modalities. arXiv preprint arXiv:2305.11172, 2023. [88] P . Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. arXiv preprint arXiv:2409.12191, 2024.

[83] [89] W. Wang, M. Shi, Q. Li, W. Wang, Z. Huang, L. Xing, Z. Chen, H. Li, X. Zhu, Z. Cao, et al. The all-seeing project: Towards panoptic visual recognition and understanding of the open world. arXiv preprint arXiv:2308.01907, 2023.

[84] [90] W. Wang, Y. Ren, H. Luo, T. Li, C. Yan, Z. Chen, W. Wang, Q. Li, L. Lu, X. Zhu, et al. The all-seeing project v2: Towards general relation comprehension of the open world. In European Conference on Computer Vision, pages 471–490. Springer, 2025.

[85] [91] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang. Magicoder: Empowering code generation with OSS-instruct. In Proceedings of the 41st International Conference on Machine Learning, volume 235 of Proceedings of Machine Learning Research, pages 52632–52657. PMLR, 21–27 Jul 2024. URL https://proceedings.mlr.press/v235/wei24h.html .

[86] [93] C. Wendler. Renderedtext dataset. https://huggingface.co/datasets/wendlerc /RenderedText , 2023. Accessed: 2023-10-17.

[87] [94] C. Wu, X. Chen, Z. Wu, Y. Ma, X. Liu, Z. Pan, W. Liu, Z. Xie, X. Yu, C. Ruan, et al. Janus: Decoupling visual encoding for unified multimodal understanding and generation. arXiv preprint arXiv:2410.13848, 2024.

[88] [96] B. Xiao, H. Wu, W. Xu, X. Dai, H. Hu, Y. Lu, M. Zeng, C. Liu, and L. Yuan. Florence-2: Advancing a unified representation for a variety of vision tasks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 4818 –4829, 2024.

[89] [97] H. Xu, Q. Ye, M. Yan, Y. Shi, J. Ye, Y. Xu, C. Li, B. Bi, Q. Qian, W. Wang, et al. mplug-2: A modularized multi-modal foundation model across text, image and video. In International Conference on Machine Learning, pages 38728–38748. PMLR, 2023.

[90] [98] Z. Xu, F. Jiang, L. Niu, Y. Deng, R. Poovendran, Y. Choi, and B. Y. Lin. Magpie: Alignment data synthesis from scratch by prompting aligned llms with nothing. arXiv preprint arXiv:2406.08464, 2024.

[91] [99] Y. Yao, T. Yu, A. Zhang, C. Wang, J. Cui, H. Zhu, T. Cai, H. Li, W. Zhao, Z. He, et al. Minicpm-v: A gpt-4v level mllm on your phone. arXiv preprint arXiv:2408.01800, 2024.

[92] [100] K. Ying, F. Meng, J. Wang, Z. Li, H. Lin, Y. Yang, H. Zhang, W. Zhang, Y. Lin, S. Liu, et al. Mmt-bench: A comprehensive multimodal benchmark for evaluating large visionlanguage models towards multitask agi. In Forty-first International Conference on Machine Learning.

[93] [101] L. Yu, P . Poirson, S. Yang, A. C. Berg, and T. L. Berg. Modeling context in referring expressions. In Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part II 14, pages 69–85. Springer, 2016. [102] L. Yu, W. Jiang, H. Shi, J. Yu, Z. Liu, Y. Zhang, J. T. Kwok, Z. Li, A. Weller, and W. Liu. Metamath: Bootstrap your own mathematical questions for large language models. arXiv preprint arXiv:2309.12284, 2023.

[94] [103] Y.-Q. Yu, M. Liao, J. Zhang, and J. Wu. Texthawk2: A large vision-language model excels in bilingual ocr and grounding with 16x fewer tokens. arXiv preprint arXiv:2410.05261, 2024.

[95] [104] Y. Yuan, X. Liu, W. Dikubab, H. Liu, Z. Ji, Z. Wu, and X. Bai. Syntax-aware network for handwritten mathematical expression recognition. arXiv preprint arXiv:2203.01601, 2022.

[96] [105] X. Yue, Y. Ni, K. Zhang, T. Zheng, R. Liu, G. Zhang, S. Stevens, D. Jiang, W. Ren, Y. Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 9556–9567, 2024.

[97] [106] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pretraining. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 11975–11986, 2023.

[98] [107] H. Zhang, M. Gao, Z. Gan, P . Dufter, N. Wenzel, F. Huang, D. Shah, X. Du, B. Zhang, Y. Li, et al. Mm1.5: Methods, analysis & insights from multimodal llm fine-tuning. arXiv preprint arXiv:2409.20566, 2024.

[99] [108] H. Zhang, H. You, P . Dufter, B. Zhang, C. Chen, H.-Y. Chen, T.-J. Fu, W. Y. Wang, S.-F. Chang, Z. Gan, et al. Ferret-v2: An improved baseline for referring and grounding with large language models. arXiv preprint arXiv:2404.07973, 2024.

[100] [109] R. Zhang, X. Wei, D. Jiang, Y. Zhang, Z. Guo, C. Tong, J. Liu, A. Zhou, B. Wei, S. Zhang, P . Gao, and H. Li. Mavis: Mathematical visual instruction tuning, 2024. URL https: //arxiv.org/abs/2407.08739 .

[101] [110] B. Zheng, B. Gou, J. Kil, H. Sun, and Y. Su. Gpt-4v(ision) is a generalist web agent, if grounded. 2024. URL https://openreview.net/forum?id=piecKJ2DlB .

[102] [111] X. Zheng, D. Burdick, L. Popa, P . Zhong, and N. X. R. Wang. Global table extractor (gte): A framework for joint table identification and cell structure recognition using visual context. Winter Conference for Applications in Computer Vision (WACV), 2021.

[103] [112] X. Zhong, E. ShafieiBavani, and A. Jimeno-Yepes. Image-based table recognition: Data, model, and evaluation. In ECCV, volume 12366, pages 564–580, 2020.

DeepSeek-V3 技术报告

DeepSeek-V3 Technical Report

DeepSeek-AI research@deepseek.com

arXiv:2412.19437 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-V3，这是一个强大的混合专家（MoE）语言模型，总参数量为 671B，每个 token 激活 37B 参数。为实现高效推理与高性价比训练，DeepSeek-V3 采用了多头潜在注意力（MLA）与 DeepSeekMoE 架构，这两者在 DeepSeek-V2 中已得到充分验证。此外，DeepSeek-V3 首创了无辅助损失的负载均衡策略，并设定了多 token 预测的训练目标，以获得更强的性能。我们在 14.8 万亿个多样且高质量的 token 上对 DeepSeek-V3 进行预训练，随后经过监督微调（SFT）与强化学习（RL）阶段，以充分释放其能力。全面的评估表明，DeepSeek-V3 优于其他开源模型，并取得了与领先闭源模型相当的性能。尽管性能优异，DeepSeek-V3 完成全部训练仅需 2.788M 个 H800 GPU 小时。此外，其训练过程非常稳定：在整个训练过程中，我们没有出现任何不可恢复的损失尖峰，也没有执行任何回滚。模型检查点可在 https://github.com/deepseek-ai/DeepSeek-V3 获取。

1 引言

近年来，大语言模型（LLM）经历了快速的迭代与演进（Anthropic，2024；Google，2024；OpenAI，2024a），与通用人工智能（AGI）的差距正逐步缩小。除闭源模型之外，包括 DeepSeek 系列（DeepSeek-AI，2024a,b,c；Guo 等，2024）、LLaMA 系列（AI@Meta，2024a,b；Touvron 等，2023a,b）、Qwen 系列（Qwen，2023，2024a,b）与 Mistral 系列（Jiang 等，2023；Mistral，2024）在内的开源模型也在取得重大进展，努力缩小与闭源模型的差距。为进一步拓展开源模型能力的边界，我们扩大了模型规模，推出 DeepSeek-V3——一个总参数 671B 的大型混合专家（MoE）模型，其中每个 token 激活 37B 参数。

我们始终以前瞻性的视角，致力于兼顾强大的模型性能与经济成本。因此，在架构方面，DeepSeek-V3 仍采用多头潜在注意力（MLA）（DeepSeek-AI，2024c）以实现高效推理，并采用 DeepSeekMoE（Dai 等，2024）以实现高性价比训练。这两种架构已在 DeepSeek-V2（DeepSeek-AI，2024c）中得到验证，展现出在保持稳健模型性能的同时实现高效训练与推理的能力。在基础架构之外，我们额外实施了两项策略，以进一步增强模型能力。首先，DeepSeek-V3 首创了一种无辅助损失（auxiliary-loss-free）的负载均衡策略（Wang 等，2024a），旨在尽量减少为鼓励负载均衡而对模型性能造成的不利影响。其次，DeepSeek-V3 采用了多 token 预测训练目标，我们观察到该目标能提升在评估基准上的整体性能。

为实现高效训练，我们支持 FP8 混合精度训练，并对训练框架进行了全面优化。低精度训练已成为高效训练的一种有前景的方案（Dettmers 等，2022；Kalamkar 等，2019；Narang 等，2017；Peng 等，2023b），其发展与硬件能力的进步密切相关（Luo 等，2024；Micikevicius 等，2022；Rouhani 等，2023a）。在本工作中，我们提出了一个 FP8 混合精度训练框架，并首次在超大规模模型上验证了其有效性。通过对 FP8 计算与存储的支持，我们同时实现了训练加速与 GPU 内存占用的降低。在训练框架方面，我们设计了 DualPipe 算法以实现高效的流水线并行，它减少了流水线气泡，并通过计算-通信重叠隐藏了训练中的大部分通信。这种重叠确保：随着模型进一步扩展，只要保持计算与通信的比值不变，我们仍可以跨节点使用细粒度专家，同时实现接近零的全对全（all-to-all）通信开销。此外，我们还开发了高效的跨节点全对全通信内核，以充分利用 InfiniBand（IB）与 NVLink 带宽。进一步地，我们精细优化了内存占用，使得无需使用昂贵的张量并行（TP）即可训练 DeepSeek-V3。综合这些努力，我们实现了较高的训练效率。

在预训练期间，我们在 14.8T 高质量且多样化的 token 上训练 DeepSeek-V3。预训练过程非常稳定：在整个训练过程中，我们没有遇到任何不可恢复的损失尖峰，也无需回滚。接下来，我们对 DeepSeek-V3 进行两阶段上下文长度扩展：第一阶段将最大上下文长度扩展到 32K，第二阶段进一步扩展到 128K。随后，我们在 DeepSeek-V3 的基座模型上进行后训练，包括监督微调（SFT）与强化学习（RL），以使其与人类偏好对齐并进一步释放其潜力。在后训练阶段，我们从 DeepSeek-R1 系列模型中蒸馏推理能力，同时谨慎地保持模型准确率与生成长度之间的平衡。

我们在全面的基准集合上评估 DeepSeek-V3。尽管训练成本经济，全面的评估表明 DeepSeek-V3-Base 已成为当前可获得的最强开源基座模型，尤其在代码与数学方面。其对话版本也优于其他开源模型，并在系列标准与开放式基准上取得了与 GPT-4o、Claude-3.5-Sonnet 等领先闭源模型相当的性能。

最后，我们再次强调 DeepSeek-V3 经济的训练成本（汇总于表 1），这得益于我们在算法、框架与硬件层面的优化协同设计。在预训练阶段，每训练一万亿 token 仅需 180K 个 H800 GPU 小时，即在我们拥有 2048 块 H800 GPU 的集群上耗时 3.7 天。因此，我们的预训练阶段在不到两个月内完成，耗费 2664K GPU 小时。加上上下文长度扩展的 119K GPU 小时与后训练的 5K GPU 小时，DeepSeek-V3 完整训练仅需 2.788M GPU 小时。假设 H800 GPU 的租赁价格为每 GPU 小时 2 美元，我们的总训练成本仅为 5.576M 美元。注意，上述成本仅包括 DeepSeek-V3 的正式训练，不包括针对架构、算法或数据的前期研究与消融实验成本。

表 1：DeepSeek-V3 的训练成本（假设 H800 的租赁价格为每 GPU 小时 2 美元）。（原文第 5 页）

训练成本	预训练	上下文扩展	后训练	总计
H800 GPU 小时	2664K	119K	5K	2788K
美元（USD）	$5.328M	$0.238M	$0.01M	$5.576M

我们的主要贡献包括：

架构：创新的负载均衡策略与训练目标

- 在 DeepSeek-V2 的高效架构之上，我们首创了无辅助损失的负载均衡策略，最大限度地减少了为鼓励负载均衡而导致的性能下降。
- 我们研究了多 token 预测（MTP）目标，并证明其对模型性能有益。该目标还可用于投机解码，以加速推理。

预训练：迈向极致的训练效率

- 我们设计了 FP8 混合精度训练框架，并首次在超大规模模型上验证了 FP8 训练的可行性与有效性。
- 通过算法、框架与硬件的协同设计，我们克服了跨节点 MoE 训练中的通信瓶颈，实现了近乎完全的计算-通信重叠。这显著提升了训练效率并降低了训练成本，使我们能够在没有额外开销的情况下进一步扩展模型规模。
- 仅以 2.664M H800 GPU 小时的经济成本，我们完成了 DeepSeek-V3 在 14.8T token 上的预训练，产出了当前最强的开源基座模型。预训练之后的后续训练阶段仅需 0.1M GPU 小时。

后训练：从 DeepSeek-R1 蒸馏知识

- 我们提出了一种创新方法，将长思维链（CoT）模型（具体为 DeepSeek R1 系列中的一个模型）的推理能力蒸馏到标准 LLM（特别是 DeepSeek-V3）中。我们的流程优雅地将 R1 的验证与反思模式融入 DeepSeek-V3，并显著提升其推理性能。同时，我们还保持对 DeepSeek-V3 输出风格与长度的控制。

核心评估结果摘要

- **知识：**（1）在 MMLU、MMLU-Pro 与 GPQA 等教育类基准上，DeepSeek-V3 优于所有其他开源模型，在 MMLU 上取得 88.5，在 MMLU-Pro 上取得 75.9，在 GPQA 上取得 59.1，性能与 GPT-4o、Claude-Sonnet-3.5 等领先闭源模型相当，缩小了该领域开源与闭源模型之间的差距。（2）在事实性基准方面，DeepSeek-V3 在 SimpleQA 与中文 SimpleQA 上均展现出开源模型中的优越性能。虽然在英文事实知识（SimpleQA）上落后于 GPT-4o 与 Claude-Sonnet-3.5，但在中文事实知识（中文 SimpleQA）上超过了这些模型，凸显了其在中文事实知识方面的优势。
- **代码、数学与推理：**（1）在所有非长 CoT 的开源与闭源模型中，DeepSeek-V3 在数学相关基准上取得了最先进的性能。值得注意的是，它甚至在 MATH-500 等特定基准上超过了 o1-preview，展现出强大的数学推理能力。（2）在编码相关任务上，DeepSeek-V3 成为 LiveCodeBench 等编码竞赛基准上表现最佳的模型，巩固了其在该领域的领先地位。在工程相关任务上，虽然 DeepSeek-V3 的表现略低于 Claude-Sonnet-3.5，但仍以明显幅度领先其他所有模型，展现出其在多样化技术基准上的竞争力。

在本文余下部分，我们首先详细阐述 DeepSeek-V3 的模型架构（第 2 节）。随后介绍我们的基础设施，包括计算集群、训练框架、对 FP8 训练的支持、推理部署策略，以及我们对未来硬件设计的建议。接着，我们描述预训练过程，包括训练数据构建、超参数设置、长上下文扩展技术、相关评估以及一些讨论（第 4 节）。之后，我们讨论后训练工作，包括监督微调（SFT）、强化学习（RL）、相应评估与讨论（第 5 节）。最后，我们总结本工作，讨论 DeepSeek-V3 的现有局限，并提出未来研究的可能方向（第 6 节）。

2 架构

我们首先介绍 DeepSeek-V3 的基础架构，其特点是采用多头潜在注意力（MLA）（DeepSeek-AI，2024c）以实现高效推理，并采用 DeepSeekMoE（Dai 等，2024）以实现经济训练。然后，我们介绍多 token 预测（MTP）训练目标，我们观察到它能提升评估基准上的整体性能。对于未明确提及的其他细节，DeepSeek-V3 遵循 DeepSeek-V2（DeepSeek-AI，2024c）的设置。

2.1 基础架构

DeepSeek-V3 的基础架构仍处于 Transformer（Vaswani 等，2017）框架之内。为实现高效推理与经济训练，DeepSeek-V3 同样采用已被 DeepSeek-V2 充分验证的 MLA 与 DeepSeekMoE。与 DeepSeek-V2 相比，一个不同之处在于我们额外为 DeepSeekMoE 引入了无辅助损失的负载均衡策略（Wang 等，2024a），以缓解为保障负载均衡而带来的性能退化。图 2 展示了 DeepSeek-V3 的基础架构，本节我们将简要回顾 MLA 与 DeepSeekMoE 的细节。

【图表】图 2 DeepSeek-V3 的基础架构示意图。遵循 DeepSeek-V2，我们采用 MLA 与 DeepSeekMoE 以实现高效推理与经济训练（原文第 7 页）

2.1.1 多头潜在注意力（MLA）

在注意力方面，DeepSeek-V3 采用 MLA 架构。设 d 表示嵌入维度，n_h 表示注意力头数，d_h 表示每个头的维度，h_t ∈ R^d 表示给定注意力层中第 t 个 token 的注意力输入。MLA 的核心是对注意力的键（key）与值（value）进行低秩联合压缩，以减少推理期间的键值（KV）缓存：

$$c^{KV}_t = W^{DKV} \cdot h_t \quad (1)$$

$$[k^C_{t,1}; k^C_{t,2}; \ldots; k^C_{t,n_h}] = k^C_t = W^{UK} \cdot c^{KV}_t \quad (2)$$

$$k^R_t = \mathrm{RoPE}(W^{KR} \cdot h_t) \quad (3)$$

$$k_{t,i} = [k^C_{t,i}; k^R_t] \quad (4)$$

$$[v^C_{t,1}; v^C_{t,2}; \ldots; v^C_{t,n_h}] = v^C_t = W^{UV} \cdot c^{KV}_t \quad (5)$$

其中 $c^{KV}_t \in R^{d_c}$ 是键与值的压缩潜在向量；d_c（$\ll d_h \cdot n_h$）表示 KV 压缩维度；$W^{DKV} \in R^{d_c \times d}$ 表示下投影矩阵；W^{UK}、$W^{UV} \in R^{d_h \cdot n_h \times d_c}$ 分别是键与值的上投影矩阵；$W^{KR} \in R^{d^R_h \times d}$ 是用于生成携带旋转位置编码（RoPE）（Su 等，2024）的解耦键的矩阵；RoPE(·) 表示施加 RoPE 矩阵的操作；[·;·] 表示拼接。注意对于 MLA，生成过程中仅需缓存蓝色方框中的向量（即 c^{KV}_t 与 k^R_t），这显著减少了 KV 缓存，同时保持与标准多头注意力（MHA）（Vaswani 等，2017）相当的性能。

对于注意力查询（query），我们也进行低秩压缩，这可以减少训练时的激活内存：

$$c^Q_t = W^{DQ} \cdot h_t \quad (6)$$

$$[q^C_{t,1}; q^C_{t,2}; \ldots; q^C_{t,n_h}] = q^C_t = W^{UQ} \cdot c^Q_t \quad (7)$$

$$[q^R_{t,1}; q^R_{t,2}; \ldots; q^R_{t,n_h}] = q^R_t = \mathrm{RoPE}(W^{QR} \cdot c^Q_t) \quad (8)$$

$$q_{t,i} = [q^C_{t,i}; q^R_{t,i}] \quad (9)$$

其中 $c^Q_t \in R^{d'_c}$ 是查询的压缩潜在向量；d'_c（$\ll d_h \cdot n_h$）表示查询压缩维度；$W^{DQ} \in R^{d'_c \times d}$、$W^{UQ} \in R^{d_h \cdot n_h \times d'_c}$ 分别是查询的下投影与上投影矩阵；$W^{QR} \in R^{d^R_h \cdot n_h \times d'_c}$ 是用于生成携带 RoPE 的解耦查询的矩阵。

最终，注意力查询（$q_{t,i}$）、键（$k_{j,i}$）与值（$v^C_{j,i}$）相结合，得到最终注意力输出 u_t：

$$o_{t,i} = \sum_{j=1}^{t} \mathrm{Softmax}_j\left(q^T_{t,i} \cdot k_{j,i} / \sqrt{d_h + d^R_h} \right) \cdot v^C_{j,i} \quad (10)$$

$$u_t = W^O \cdot [o_{t,1}; o_{t,2}; \ldots; o_{t,n_h}] \quad (11)$$

其中 $W^O \in R^{d \times d_h \cdot n_h}$ 表示输出投影矩阵。

2.1.2 采用无辅助损失负载均衡的 DeepSeekMoE

DeepSeekMoE 的基础架构：对于前馈网络（FFN），DeepSeek-V3 采用 DeepSeekMoE 架构（Dai 等，2024）。与 GShard（Lepikhin 等，2021）等传统 MoE 架构相比，DeepSeekMoE 使用更细粒度的专家，并隔离出部分专家作为共享专家。设 u_t 表示第 t 个 token 的 FFN 输入，我们如下计算 FFN 输出 h'_t：

$$h'_t = u_t + \sum_{i=1}^{N_s} \mathrm{FFN}^{(s)}_i(u_t) + \sum_{i=1}^{N_r} g_{i,t} \cdot \mathrm{FFN}^{(r)}_i(u_t) \quad (12)$$

$$g_{i,t} = g'_{i,t} / \sum_{j=1}^{N_r} g'_{j,t} \quad (13)$$

$$g'_{i,t} = s_{i,t}，若\ s_{i,t} \in \mathrm{Topk}(\{s_{j,t} \mid 1 \le j \le N_r\}, K_r)；否则为\ 0 \quad (14)$$

$$s_{i,t} = \mathrm{Sigmoid}(u_t^T \cdot e_i) \quad (15)$$

其中 N_s 与 N_r 分别表示共享专家与路由专家的数量；$\mathrm{FFN}^{(s)}_i(\cdot)$ 与 $\mathrm{FFN}^{(r)}_i(\cdot)$ 分别表示第 i 个共享专家与第 i 个路由专家；K_r 表示被激活的路由专家数量；$g_{i,t}$ 是第 i 个专家的门控值；$s_{i,t}$ 是 token 到专家的亲和度；e_i 是第 i 个路由专家的质心向量；$\mathrm{Topk}(\cdot,K)$ 表示由第 t 个 token 对所有路由专家计算出的亲和度分数中最高的 K 个分数构成的集合。与 DeepSeek-V2 略有不同，DeepSeek-V3 使用 sigmoid 函数计算亲和度分数，并对所有被选中专家的亲和度分数进行归一化以产生门控值。

无辅助损失的负载均衡：对于 MoE 模型，专家负载不均衡会导致路由崩溃（Shazeer 等，2017），并在专家并行（EP）场景中降低计算效率。传统解决方案通常依赖辅助损失（Fedus 等，2021；Lepikhin 等，2021）来避免负载不均衡，但过大的辅助损失会损害模型性能（Wang 等，

2024a）。为在负载均衡与模型性能之间取得更好的权衡，我们首创了一种无辅助损失的负载均衡策略（Wang 等，2024a）来确保负载均衡。具体而言，我们为每个专家引入一个偏置项 b_i，并将其加到相应的亲和度分数 s_{i,t} 上以确定 top-K 路由：

g'_{i,t} = s_{i,t}，若 s_{i,t} + b_i ∈ Topk({s_{j,t} + b_j | 1 ≤ j ≤ N_r}, K_r)；否则为 0　　（16）

注意，偏置项仅用于路由。将与 FFN 输出相乘的门控值仍然来自原始亲和度分数 s_{i,t}。在训练过程中，我们持续监控每个训练步整个批次上的专家负载。在每个训练步结束时，如果某个专家过载，我们将其偏置项减少 γ；如果欠载，则增加 γ，其中 γ 是一个称为偏置更新速度（bias update speed）的超参数。通过动态调整，DeepSeek-V3 在训练过程中保持了专家负载均衡，并取得了优于通过纯辅助损失鼓励负载均衡的模型的性能。

补充的序列级辅助损失：尽管 DeepSeek-V3 主要依靠无辅助损失策略实现负载均衡，但为防止任何单条序列内出现极端不均衡，我们还采用了一种补充的序列级均衡损失：

L_Bal = α · Σ_{i=1}^{N_r} f_i · P_i　　（17）

f_i = (N_r / (K_r · T)) · Σ_{t=1}^{T} 1(s_{i,t} ∈ Topk({s_{j,t} | 1 ≤ j ≤ N_r}, K_r))　　（18）

s'_{i,t} = s_{i,t} / Σ_{j=1}^{N_r} s_{j,t}　　（19）

P_i = (1/T) · Σ_{t=1}^{T} s'_{i,t}　　（20）

其中均衡因子 α 是超参数，在 DeepSeek-V3 中会被赋予极小的值；1(·) 表示指示函数；T 表示一条序列中的 token 数。序列级均衡损失鼓励每条序列上的专家负载保持均衡。

节点受限路由：与 DeepSeek-V2 使用的设备受限路由类似，DeepSeek-V3 也采用受限路由机制来限制训练期间的通信成本。简而言之，我们确保每个 token 至多被发送到 M 个节点，这些节点根据分布在每个节点上的专家中最高 K_r·M 个亲和度分数之和来选择。在此约束下，我们的 MoE 训练框架几乎可以实现完全的计算-通信重叠。

不丢弃 token：得益于有效的负载均衡策略，DeepSeek-V3 在整个训练过程中保持了良好的负载均衡。因此，DeepSeek-V3 在训练期间不丢弃任何 token。此外，我们还实现了特定的部署策略以确保推理负载均衡，因此 DeepSeek-V3 在推理期间同样不丢弃 token。

2.2 多 token 预测

受 Gloeckle 等（2024）启发，我们研究并为 DeepSeek-V3 设定了多 token 预测（MTP）目标，将每个位置的预测范围扩展到多个未来 token。一方面，MTP 目标会加密训练信号，可能提升数据效率；另一方面，MTP 可能使模型能够预先规划其表示，以更好地预测未来 token。图 3 展示了我们的

MTP 实现。与 Gloeckle 等（2024）并行地使用独立输出头预测 D 个额外 token 不同，我们顺序地预测额外 token，并在每个预测深度保持完整的因果链。本节介绍我们 MTP 实现的细节。

【图表】图 3 我们的多 token 预测（MTP）实现示意图。我们在每个深度上为每个 token 的预测保留完整的因果链（原文第 10 页）

MTP 模块：具体而言，我们的 MTP 实现使用 D 个顺序模块来预测 D 个额外 token。第 k 个 MTP 模块由一个共享嵌入层 Emb(·)、一个共享输出头 OutHead(·)、一个 Transformer 块 TRM_k(·) 以及一个投影矩阵 M_k ∈ R^{d×2d} 组成。对于第 i 个输入 token t_i，在第 k 个预测深度，我们首先将第 i 个 token 在第 (k−1) 个深度的表示 h^{k−1}_i ∈ R^d 与第 (i+k) 个 token 的嵌入 Emb(t_{i+k}) ∈ R^d 通过线性投影组合：

h'^k_i = M_k · [RMSNorm(h^{k−1}_i); RMSNorm(Emb(t_{i+k}))] （21）

其中 [·;·] 表示拼接。特别地，当 k = 1 时，h^{k−1}_i 指主模型给出的表示。注意对于每个 MTP 模块，其嵌入层与主模型共享。组合后的 h'^k_i 作为第 k 个深度 Transformer 块的输入，以产生当前深度的输出表示 h^k_i：

h^k_{1:T−k} = TRM_k(h'^k_{1:T−k}) （22）

其中 T 表示输入序列长度，i:j 表示切片操作（包含左右边界）。最后，以 h^k_i 为输入，共享输出头将计算第 k 个额外预测 token 的概率分布 P^k_{i+k+1} ∈ R^V，其中 V 是词表大小：

P^k_{i+k+1} = OutHead(h^k_i) （23）

输出头 OutHead(·) 将表示线性映射为 logits，随后应用 Softmax(·) 函数计算第 k 个额外 token 的预测概率。此外，对于每个 MTP 模块，其输出头与主模型共享。我们保持预测因果链的原则与 EAGLE（Li 等，2024b）类似，但其主要目标是投机解码（Leviathan 等，2023；Xia 等，2023），而我们利用 MTP 来改进训练。

MTP 训练目标：对于每个预测深度，我们计算交叉熵损失 L^k_MTP：

L^k_MTP = CrossEntropy(P^k_{2+k:T+1}, t_{2+k:T+1}) = −(1/T) · Σ_{i=2+k}^{T+1} log P^k_i[t_i] （24）

其中 T 表示输入序列长度，t_i 表示第 i 个位置的真实 token，P^k_i[t_i] 表示第 k 个 MTP 模块给出的 t_i 的相应预测概率。最后，我们对所有深度的 MTP 损失求平均，并乘以一个权重因子 λ 得到总体 MTP 损失 L_MTP，作为 DeepSeek-V3 的附加训练目标：

$$L_{MTP} = (\lambda/D) \cdot \Sigma_{k=1}^{D} L^{k}_{MTP} \quad (25)$$

推理中的 MTP：我们的 MTP 策略主要旨在提升主模型的性能，因此在推理期间，我们可以直接丢弃 MTP 模块，主模型可以独立正常地运行。此外，我们也可以将这些 MTP 模块复用于投机解码，以进一步改善生成延迟。

3 基础设施

3.1 计算集群

DeepSeek-V3 在配备 2048 块 NVIDIA H800 GPU 的集群上训练。H800 集群中的每个节点包含 8 块通过节点内 NVLink 与 NVSwitch 连接的 GPU。不同节点之间使用 InfiniBand（IB）互连以促进通信。

3.2 训练框架

DeepSeek-V3 的训练由 HAI-LLM 框架支持，这是我们工程师从零打造的高效、轻量训练框架。总体而言，DeepSeek-V3 采用 16 路流水线并行（PP）（Qi 等，2023a）、跨越 8 个节点的 64 路专家并行（EP）（Lepikhin 等，2021）以及 ZeRO-1 数据并行（DP）（Rajbhandari 等，2020）。

为促进 DeepSeek-V3 的高效训练，我们实施了细致的工程优化。首先，我们设计了 DualPipe 算法以实现高效的流水线并行。与现有 PP 方法相比，DualPipe 的流水线气泡更少。更重要的是，它在正向与反向过程中重叠了计算与通信阶段，从而解决了跨节点专家并行带来的沉重通信开销问题。其次，我们开发了高效的跨节点全对全通信内核，以充分利用 IB 与 NVLink 带宽，并节省专用于通信的流式多处理器（SM）。最后，我们细致优化了训练期间的内存占用，从而无需使用昂贵的张量并行（TP）即可训练 DeepSeek-V3。

3.2.1 DualPipe 与计算-通信重叠

对于 DeepSeek-V3，跨节点专家并行带来的通信开销导致计算与通信之比约为 1:1，效率低下。为应对这一挑战，我们设计了一种创新的流水线并行算法 DualPipe，它不仅通过有效重叠正向与反向的计算-通信阶段来加速模型训练，还减少了流水线气泡。

DualPipe 的核心思想是在一对独立的正向与反向数据块（chunk）内重叠计算与通信。具体而言，我们将每个数据块分为四个组件：注意力、全对全分发（all-to-all dispatch）、MLP 与全对全合并（all-

to-all combine）。特别地，对于反向数据块，注意力与 MLP 都进一步拆分为两部分：“反向传播输入”（backward for input）与“反向传播权重”（backward for weights），这与 ZeroBubble（Qi 等，2023b）类似。此外，我们还有一个 PP 通信组件。如图 4 所示，对于一对正向与反向数据块，我们重新排列这些组件，并手动调整专用于通信与计算的 GPU SM 比例。在这种重叠策略下，我们可以确保全对全通信与 PP 通信在执行期间都能被完全隐藏。鉴于这一高效的重叠策略，完整的 DualPipe 调度如图 5 所示。它采用双向流水线调度，从流水线两端同时送入微批次，很大一部分通信可以被完全重叠。这种重叠还确保：随着模型进一步扩展，只要保持计算与通信的比值不变，我们仍然可以跨节点使用细粒度专家，同时实现接近零的全对全通信开销。

【图表】图 4 一对独立正向与反向数据块的重叠策略（Transformer 块的边界未对齐）。橙色表示正向，绿色表示“反向传播输入”，蓝色表示“反向传播权重”，紫色表示 PP 通信，红色表示屏障。全对全通信与 PP 通信都可以被完全隐藏（原文第 12 页）

【图表】图 5 针对 8 个 PP rank、两个方向各 20 个 micro-batch 的 DualPipe 调度示例。反方向的 micro-batch 与正方向对称，因此为图示简洁我们省略其 batch ID。由同一黑色边框围住的两个单元格表示计算与通信相互重叠（原文第 13 页）

此外，即使在没有沉重通信负担的更一般场景中，DualPipe 仍然展现出效率优势。表 2 总结了不同 PP 方法下的流水线气泡与内存占用。如表所示，与 ZB1P（Qi et al., 2023b）和 1F1B（Harlap et al., 2018）相比，DualPipe 显著减少了流水线气泡，同时仅将峰值激活内存增加 $1/PP$。尽管 DualPipe 需要保留两份模型参数副本，但由于训练中我们使用较大的 EP 规模，这并不会显著增加内存消耗。与 Chimera（Li and Hoefler, 2021）相比，DualPipe 只要求流水线阶段数与 micro-batch 数可被 2 整除，而不要求 micro-batch 数可被流水线阶段数整除。此外，对于 DualPipe 而言，气泡与激活内存都不会随 micro-batch 数量的增长而增加。

表 2：不同流水线并行方法的流水线气泡与内存占用对比。F 表示前向 chunk 的执行时间，B 表示完整反向 chunk 的执行时间，W 表示“权重反向”chunk 的执行时间，$F\&B$ 表示两个相互重叠的前向与反向 chunk 的执行时间（原文第 13 页）。

方法	气泡	参数	激活
1F1B	$(PP-1)(F+B)$	1×	PP
ZB1P	$(PP-1)(F+B-2W)$	1×	PP
DualPipe（Ours）	$(PP^2-1)(F\&B+B-3W)$	2×	$PP+1$

3.2.2 跨节点全对全通信的高效实现

为确保 DualPipe 拥有足够的计算性能，我们定制了高效的跨节点全对全通信 kernel（包括 dispatching 与 combining），以节省专用于通信的 SM 数量。这些 kernel 的实现与 MoE 门控算法以及我们集群的网络拓扑协同设计。具体而言，在我们的集群中，跨节点 GPU 通过 IB 全互连，节点内通信则通过 NVLink 处理。NVLink 提供 160 GB/s 的带宽，约为 IB（50 GB/s）的 3.2 倍。为有效利用 IB 与 NVLink 的不同带宽，我们限制每个 token 最多被分派到 4 个节点，从而降低 IB 流量。对于每个 token，当其路由决策确定后，它首先通过 IB 传输到目标节点上具有相同节点内索引的 GPU。一旦到达目标节点，我们会尽可能确保它立即通过 NVLink 转发到托管其目标专家的具体 GPU，而不会被随后到达的 token 阻塞。这样，IB 与 NVLink 上的通信完全重叠，每个 token 可以高效地平均选择每个节点上的 3.2 个专家，而不会引入来自 NVLink 的额外开销。这意味着，尽管 DeepSeek-V3 在实践中只选择 8 个路由专家，但它可以将该数量扩展到最多 13 个专家（4 节点 × 3.2 专家/节点），同时保持相同的通信成本。总体而言，在这种通信策略下，仅需 20 个 SM 就足以充分利用 IB 与 NVLink 的带宽。

具体而言，我们采用 warp specialization 技术（Bauer et al., 2014），将这 20 个 SM 划分为 10 个通信通道。在 dispatching 过程中，（1）IB 发送、（2）IB 到 NVLink 转发以及（3）NVLink 接收分别由不同的 warp 处理；分配给各通信任务的 warp 数量会根据所有 SM 上的实际负载动态调整。类似地，在 combining 过程中，（1）NVLink 发送、（2）NVLink 到 IB 转发与累加以及（3）IB 接收与累加同样由动态调整的 warp 处理。此外，dispatching 与 combining kernel 都与计算流重叠，因此我们也考虑它们对其他 SM 计算 kernel 的影响。具体而言，我们采用定制的 PTX（Parallel Thread Execution）指令并自动调优通信 chunk 大小，这显著减少了 L2 cache 的使用以及其他 SM 受到的干扰。

3.2.3 以最小开销实现极致省内存

为降低训练期间的内存占用，我们采用以下技术。

RMSNorm 与 MLA 上投影的重计算（Recomputation of RMSNorm and MLA Up-Projection）：我们在反向传播期间重新计算所有 RMSNorm 操作与 MLA 上投影，从而无需持续存储其输出激活。该策略以较小的额外开销显著降低了存储激活所需的内存。

CPU 中的指数移动平均（Exponential Moving Average in CPU）：训练期间，我们保留模型参数的指数移动平均（EMA），用于在学习率衰减后提前估计模型性能。EMA 参数存储在 CPU 内存中，并在每个训练步之后异步更新。该方法使我们能够维护 EMA 参数，而不引入额外的内存或时间开销。

多 token 预测的共享嵌入与输出头（Shared Embedding and Output Head for Multi-Token Prediction）：借助 DualPipe 策略，我们将模型最浅的层（包括嵌入层）与最深的层（包括输出头）部署在同一个 PP rank 上。这种安排使得共享嵌入与输出头的参数和梯度能够在 MTP 模块与主模型之间实现物理共享。这种物理共享机制进一步提升了我们的内存效率。

3.3 FP8 训练

受低精度训练近期进展（Dettmers et al., 2022; Noune et al., 2022; Peng et al., 2023b）的启发，我们提出了一种利用 FP8 数据格式训练 DeepSeek-V3 的细粒度混合精度框架。尽管低精度训练前景广阔，但它通常受限于激活、权重与梯度中存在的离群值（Fishman et al., 2024; He et al.; Sun et al., 2024）。虽然推理量化领域已经取得了显著进展（Frantar et al., 2022; Xiao et al., 2023），但能够证明低精度技术在大规模语言模型预训练中成功应用的研究相对较少（Fishman et al., 2024）。为应对这一挑战并有效扩展 FP8 格式的动态范围，我们引入了一种细粒度量化策略：以 $1\times NC$ 个元素为单位的 tile 级分组，或以 $NC\times NC$ 个元素为单位的 block 级分组。相关的反量化开销在我们的高精度累加过程中被大幅缓解，而这是实现精确 FP8 通用矩阵乘法（GEMM）的关键。此外，为进一步降低 MoE 训练中的内存与通信开销，我们以 FP8 缓存和分派激活，同时以 BF16 存储低精度优化器状态。我们在两个与 DeepSeek-V2-Lite 和 DeepSeek-V2 规模相近的模型上验证了所提出的 FP8 混合精度框架，训练约 1 万亿 token（更多细节见附录 B.1）。值得注意的是，与 BF16 基线相比，我们 FP8 训练模型的相对损失误差始终低于 0.25%，完全处于训练随机性可接受的范围内。

【图表】图 6 采用 FP8 数据格式的整体混合精度框架。为清晰起见，图中仅展示 Linear 算子（原文第 15 页）

3.3.1 混合精度框架

在低精度训练中广泛采用的技术（Kalamkar et al., 2019; Narang et al., 2017）基础上，我们提出了一个用于 FP8 训练的混合精度框架。在该框架中，大多数计算密集型操作以 FP8 执行，而少数关键操作则策略性地保持其原始数据格式，以平衡训练效率与数值稳定性。整体框架如图 6 所示。

首先，为加速模型训练，大多数核心计算 kernel（即 GEMM 操作）以 FP8 精度实现。这些 GEMM 操作接受 FP8 张量作为输入，并产生 BF16 或 FP32 的输出。如图 6 所示，与 Linear 算子相关的全部三个 GEMM，即 Fprop（前向传播）、Dgrad（激活反向传播）与 Wgrad（权重反向传播），均以 FP8 执行。这种设计与原始 BF16 方法相比在理论上将计算速度提高一倍。此外，FP8 Wgrad GEMM 允许将激活以 FP8 存储，以供反向传播使用，这显著降低了内存消耗。

尽管 FP8 格式具有效率优势，但某些算子由于对低精度计算敏感，仍需要更高的精度。此外，一些低成本算子也可以使用更高精度，而对整体训练成本的开销可以忽略不计。因此，经过仔细研究，我们为以下组件保持原始精度（例如 BF16 或 FP32）：嵌入模块、输出头、MoE 门控模块、归一化算子以及注意力算子。这些有针对性的高精度保留确保了 DeepSeek-V3 训练动态的稳定。为进一步保证数值稳定性，我们以更高精度存储主权重、权重梯度与优化器状态。尽管这些高精度组件会带来一定的内存开销，但通过在我们的分布式训练系统中跨多个 DP rank 进行高效分片，其影响可以被最小化。

【图表】图 7 （a）我们提出一种细粒度量化方法，以缓解特征离群值导致的量化误差；为图示简洁，仅展示 Fprop。（b）结合我们的量化策略，我们以 $NC = 128$ 个元素为间隔将 MMA 提升至 CUDA Core 进行高精度累加，从而提升 FP8 GEMM 精度（原文第 16 页）

3.3.2 通过量化与乘法提升精度

基于我们的混合精度 FP8 框架，我们引入了若干策略来提升低精度训练准确率，重点关注量化方法与乘法过程。

细粒度量化（Fine-Grained Quantization）：在低精度训练框架中，上溢与下溢是常见挑战，因为 FP8 格式的动态范围有限，受其较少的指数位约束。一种标准做法是：将输入张量的最大绝对值缩放到 FP8 的最大可表示值，从而使输入分布对齐到 FP8 的可表示范围（Narang et al., 2017）。该方法使低精度训练对激活离群值高度敏感，而激活离群值会严重降低量化精度。为解决这一问题，我们提出一种在更细粒度上应用缩放的细粒度量化方法。如图 7（a）所示，（1）对于激活，我们以 1×128 的 tile 为单位对元素分组并缩放（即每个 token 每 128 个通道）；（2）对于权重，我们以 128×128 的 block 为单位对元素分组并缩放（即每 128 个输入通道 × 128 个输出通道）。该方法通过根据更小的元素组调整缩放，使量化过程能够更好地适应离群值。在附录 B.2 中，我们进一步讨论了以与权重量化相同的方式在 block 级对激活分组并缩放时所出现的训练不稳定问题。

我们方法中的一个关键改动是在 GEMM 操作的内维度上引入逐组 scaling factor。标准 FP8 GEMM 并不直接支持这一功能。然而，结合我们精确的 FP32 累加策略，它可以被高效实现。

值得注意的是，我们的细粒度量化策略与 microscaling 格式（Rouhani et al., 2023b）的思想高度一致，而 NVIDIA 下一代 GPU（Blackwell 系列）的 Tensor Core 已宣布支持具有更小量化粒度的 microscaling 格式（NVIDIA, 2024a）。我们希望我们的设计能够为未来工作提供参考，以跟上最新 GPU 架构的步伐。

提升累加精度（Increasing Accumulation Precision）：低精度 GEMM 操作常常遭遇下溢问题，其精度在很大程度上取决于高精度累加，而累加通常以 FP32 精度进行（Kalamkar et al., 2019; Narang et al., 2017）。然而，我们观察到，在 NVIDIA H800 GPU 上，FP8 GEMM 的累加精度仅能保留约 14 位，显著低于 FP32 的累加精度。当内维度 K 较大时（Wortsman et al., 2023），该问题会更加明显，而这在大规模模型训练中（批大小与模型宽度增大时）是典型场景。以两个随机矩阵、K = 4096 的 GEMM 操作为例，在我们的初步测试中，Tensor Core 有限的累加精度导致最大相对误差接近 2%。尽管存在这些问题，有限的累加精度仍是一些 FP8 框架（NVIDIA, 2024b）的默认选项，严重制约了训练精度。

为解决这一问题，我们采用提升到 CUDA Core 以获得更高精度的策略（Thakkar et al., 2023）。该过程如图 7（b）所示。具体而言，在 Tensor Core 上执行 MMA（矩阵乘累加）期间，中间结果以有限的位宽进行累加。一旦达到 NC 的间隔，这些部分结果就会被复制到 CUDA Core 上的 FP32 寄存器，在那里执行全精度 FP32 累加。如前所述，我们的细粒度量化沿内维度 K 应用逐组 scaling factor。这些 scaling factor 可以在 CUDA Core 上作为反量化过程被高效相乘，额外计算成本极小。

值得注意的是，这一改动降低了单个 warpgroup 的 WGMMA（Warpgroup 级矩阵乘累加）指令发射率。然而，在 H800 架构上，通常会有两个 WGMMA 并发驻留：当一个 warpgroup 执行提升操作时，另一个能够执行 MMA 操作。这种设计使两种操作得以重叠，从而保持 Tensor Core 的高利用率。根据我们的实验，设置 NC = 128 个元素（相当于 4 个 WGMMA）是能够在显著提升精度且不引入可观开销前提下的最小累加间隔。

尾数优先于指数（Mantissa over Exponents）：与先前工作（NVIDIA, 2024b; Peng et al., 2023b; Sun et al., 2019b）采用的混合 FP8 格式不同——该格式在 Fprop 中使用 E4M3（4 位指数、3 位尾数），在 Dgrad 与 Wgrad 中使用 E5M2（5 位指数、2 位尾数）——我们在所有张量上采用 E4M3 格式以获得更高精度。我们将该方法的可行性归因于我们的细粒度量化策略，即 tile 级与 block 级缩

放。通过作用于更小的元素组，我们的方法有效地在这些分组的元素之间共享指数位，从而缓解了动态范围有限带来的影响。

在线量化（Online Quantization）：tensor 级量化框架（NVIDIA, 2024b; Peng et al., 2023b）采用延迟量化，即维护先前迭代中最大绝对值的历史，以推断当前值。为确保缩放因子准确并简化框架，我们针对每个 1×128 的激活 tile 或 128×128 的权重 block 在线计算最大绝对值；在此基础上推导 scaling factor，然后在线将激活或权重量化为 FP8 格式。

3.3.3 低精度存储与通信

结合我们的 FP8 训练框架，我们进一步通过将缓存的激活与优化器状态压缩到更低精度的格式，来降低内存消耗与通信开销。

低精度优化器状态（Low-Precision Optimizer States）：我们采用 BF16 数据格式而非 FP32 来跟踪 AdamW（Loshchilov and Hutter, 2017）优化器中的一阶与二阶矩，而未观察到可感知的性能下降。然而，主权重（由优化器存储）与梯度（用于批大小累加）仍然保留 FP32，以确保整个训练过程中的数值稳定性。

低精度激活（Low-Precision Activation）：如图 6 所示，Wgrad 操作以 FP8 执行。为降低内存消耗，将激活以 FP8 格式缓存供 Linear 算子的反向传播使用是一种自然的选择。然而，为了低成本地实现高精度训练，我们对若干算子采取了特殊处理：

- （1）注意力算子之后 Linear 的输入。这些激活也会在注意力算子的反向传播中使用，因此对精度敏感。我们专门为这些激活采用了定制的 E5M6 数据格式。此外，这些激活在反向传播中会从 1×128 的量化 tile 转换为 128×1 的 tile。为避免引入额外的量化误差，所有 scaling factor 都采用 round scaling，即取 2 的整数次幂。
- （2）MoE 中 SwiGLU 算子的输入。为进一步降低内存开销，我们缓存 SwiGLU 算子的输入并在反向传播中重新计算其输出。这些激活同样以 FP8 存储并采用我们的细粒度量化方法，从而在内存效率与计算精度之间取得平衡。

低精度通信（Low-Precision Communication）：通信带宽是 MoE 模型训练中的关键瓶颈。为缓解这一挑战，我们在 MoE 上投影之前将激活量化为 FP8，然后应用 dispatch 组件，这与 MoE 上投影中的 FP8 Fprop 兼容。与注意力算子之后 Linear 的输入类似，该激活的 scaling factor 为 2 的整数次幂。类似的策略也应用于 MoE 下投影之前的激活梯度。对于前向与反向的 combine 组件，我们将其保留为 BF16，以在训练流程的关键部分保持训练精度。

3.4 推理与部署

我们将 DeepSeek-V3 部署在 H800 集群上，其中每个节点内的 GPU 通过 NVLink 互连，跨集群的所有 GPU 通过 IB 全互连。为同时保证在线服务的服务等级目标（SLO）与高吞吐，我们采用以下将预填充（prefilling）与解码（decoding）阶段分离的部署策略。

3.4.1 预填充

预填充阶段的最小部署单元由 4 个节点、32 块 GPU 组成。注意力部分采用 4 路张量并行（TP4）结合序列并行（SP），并配合 8 路数据并行（DP8）。其较小的 TP 规模（4）限制了 TP 通信开销。对于 MoE 部分，我们使用 32 路专家并行（EP32），这确保每个专家处理足够大的批大小，从而提升计算效率。对于 MoE 的全对全通信，我们采用与训练相同的方法：先通过 IB 跨节点传输 token，再通过 NVLink 在节点内 GPU 之间转发。特别地，我们对浅层中的稠密 MLP 采用 1 路张量并行，以节省 TP 通信。

为实现 MoE 部分不同专家之间的负载均衡，我们需要确保每块 GPU 处理大致相同数量的 token。为此，我们引入冗余专家（redundant experts）部署策略：复制高负载专家并冗余部署。高负载专家基于在线部署期间收集的统计信息检测，并定期调整（例如每 10 分钟一次）。在确定冗余专家集合后，我们根据观测到的负载，在节点内的 GPU 之间仔细重新安排专家，力求在不增加跨节点全对全通信开销的前提下尽可能平衡各 GPU 的负载。对于 DeepSeek-V3 的部署，我们为预填充阶段设置了 32 个冗余专家。对每块 GPU 而言，除了原本托管的 8 个专家之外，它还会额外托管一个冗余专家。

此外，在预填充阶段，为提高吞吐并隐藏全对全通信与 TP 通信的开销，我们同时处理两个计算负载相近的 micro-batch，使一个 micro-batch 的注意力与 MoE 和另一个 micro-batch 的 dispatch 与 combine 相互重叠。

最后，我们正在探索一种专家动态冗余策略：每块 GPU 托管更多专家（例如 16 个专家），但在每个推理步中只有 9 个会被激活。在每一层的全对全操作开始之前，我们即时计算全局最优的路由方案。考虑到预填充阶段涉及大量计算，计算该路由方案的开销几乎可以忽略不计。

3.4.2 解码

在解码期间，我们将共享专家视为一个路由专家。从这个角度看，每个 token 在路由时会选择 9 个专家，其中共享专家被视为一个总会被选中的高负载专家。解码阶段的最小部署单元由 40 个节点、320 块 GPU 组成。注意力部分采用 TP4 结合 SP，并配合 DP80，而 MoE 部分使用 EP320。对于 MoE 部分，每块 GPU 只托管一个专家，另有 64 块 GPU 负责托管冗余专家与共享专家。dispatch 与

combine 部分的全对全通信通过 IB 上的直接点对点传输完成，以实现低延迟。此外，我们利用 IBGDA（NVIDIA, 2022）技术进一步最小化延迟并提升通信效率。

与预填充类似，我们基于在线服务中的专家负载统计，按一定间隔定期确定冗余专家集合。然而，由于每块 GPU 只托管一个专家，我们无需重新安排专家。我们也在探索解码阶段的动态冗余策略。不过，这需要更仔细地优化计算全局最优路由方案的算法，并与 dispatch kernel 融合以降低开销。

此外，为提高吞吐并隐藏全对全通信的开销，我们也在探索在解码阶段同时处理两个计算负载相近的 micro-batch。与预填充不同，注意力在解码阶段占据更大比例的时间。因此，我们将一个 micro-batch 的注意力与另一个 micro-batch 的 dispatch+MoE+combine 重叠。在解码阶段，每个专家的批大小相对较小（通常在 256 个 token 以内），瓶颈在于内存访问而非计算。由于 MoE 部分只需加载一个专家的参数，内存访问开销极小，因此使用较少的 SM 不会显著影响整体性能。所以，为避免影响注意力部分的计算速度，我们可以只分配一小部分 SM 给 dispatch+MoE+combine。

3.5 硬件设计建议

基于我们在全对全通信与 FP8 训练方案上的实现，我们向 AI 硬件厂商提出以下芯片设计建议。

3.5.1 通信硬件

在 DeepSeek-V3 中，我们实现了计算与通信的重叠，以在计算期间隐藏通信延迟。与串行执行计算和通信相比，这显著降低了对通信带宽的依赖。然而，当前的通信实现依赖于昂贵的 SM（例如，我们从 H800 GPU 可用的 132 个 SM 中分配 20 个用于通信），这将限制计算吞吐。此外，使用 SM 进行通信会造成显著的低效，因为 Tensor Core 完全未被充分利用。

目前，SM 在 all-to-all 通信中主要执行以下任务：

- 在 IB（InfiniBand）与 NVLink 域之间转发数据，同时将来自单个 GPU、发往同一节点内多个 GPU 的 IB 流量进行聚合。
- 在 RDMA 缓冲区（已注册的 GPU 内存区域）与输入/输出缓冲区之间传输数据。
- 为 all-to-all combine 执行 reduce 操作。
- 在跨 IB 与 NVLink 域向多个专家分块传输数据时，管理细粒度的内存布局。

我们希望看到未来厂商开发硬件，将这些通信任务从宝贵的计算单元 SM 上卸载，作为 GPU 协处理器或类似 NVIDIA SHARP（Graham et al., 2016）的网络协处理器。此外，为降低应用程序编程复

杂度，我们希望该硬件能够从计算单元的视角统一 IB（scale-out）与 NVLink（scale-up）网络。借助这一统一接口，计算单元可以通过基于简单原语提交通信请求，轻松完成整个 IB-NVLink 统一域内的 read、write、multicast 与 reduce 等操作。

3.5.2 计算硬件

在 Tensor Core 中提高 FP8 GEMM 累加精度：在 NVIDIA Hopper 架构当前的 Tensor Core 实现中，FP8 GEMM 受限于有限的累加精度。在根据最大指数对 32 个尾数乘积进行右移对齐之后，Tensor Core 仅使用每个尾数乘积的最高 14 位进行加法，并截断超出该范围的位。加法结果累加到寄存器同样采用 14 位精度。我们的实现通过在 CUDA core 中以 FP32 精度将 128 次 FP8×FP8 乘法的加法结果累加到寄存器，部分缓解了这一限制。尽管这有助于实现成功的 FP8 训练，但它只是 Hopper 架构在 FP8 GEMM 累加精度上硬件缺陷的折衷方案。未来的芯片需要采用更高的精度。

支持 Tile 级与 Block 级量化：当前 GPU 仅支持 per-tensor 量化，缺乏对我们这种 tile 级与 block 级细粒度量化的原生支持。在当前实现中，当达到 NC 间隔时，部分结果会从 Tensor Core 复制到 CUDA core，乘以 scaling factor，并加到 CUDA core 上的 FP32 寄存器中。尽管结合我们精确的 FP32 累加策略，反量化开销已被显著缓解，但 Tensor Core 与 CUDA core 之间频繁的数据移动仍然限制了计算效率。因此，我们建议未来的芯片通过使 Tensor Core 能够接收 scaling factor 并执行带分组缩放的 MMA 来支持细粒度量化。这样，整个部分和累加与反量化都可以直接在 Tensor Core 内部完成，直到产生最终结果，从而避免频繁的数据移动。

支持在线量化：尽管在线量化在我们的研究中已被证明有效，但当前实现难以有效支持它。在现有流程中，我们需要从 HBM（高带宽内存）读取 128 个 BF16 激活值（前一次计算的输出）用于量化，然后将量化后的 FP8 值写回 HBM，之后又需再次读取以进行 MMA。为解决这一低效问题，我们建议未来的芯片将 FP8 cast 与 TMA（Tensor Memory Accelerator）访问集成为单个融合操作，使量化能够在激活从全局内存传输到共享内存的过程中完成，从而避免频繁的内存读写。我们还建议支持 warp 级 cast 指令以加速，这进一步有利于层归一化与 FP8 cast 的更好融合。或者，可以采用近内存计算（near-memory computing）方案，将计算逻辑放置在 HBM 附近。在这种情况下，BF16 元素在从 HBM 读入 GPU 时即可直接转换为 FP8，从而将片外内存访问减少约 50%。

支持转置 GEMM 操作：当前架构使矩阵转置与 GEMM 操作的融合变得繁琐。在我们的工作流程中，前向传播期间的激活被量化为 1×128 的 FP8 tile 并存储。在反向传播期间，矩阵需要被读出、反量化、转置、重新量化为 128×1 的 tile，然后存入 HBM。为减少内存操作，我们建议未来的芯片

在 MMA 操作之前，支持对训练与推理都需要的那些精度直接从共享内存中进行矩阵的转置读取。结合 FP8 格式转换与 TMA 访问的融合，这一增强将显著简化量化工作流程。

4 预训练

4.1 数据构建

与 DeepSeek-V2 相比，我们通过提高数学与编程样本的比例来优化预训练语料，同时将多语言覆盖范围扩展到英语与中文之外。此外，我们改进了数据处理流程，在保持语料多样性的同时最小化冗余。受 Ding et al. (2024) 的启发，我们采用 document packing 方法以保证数据完整性，但在训练中不引入跨样本的注意力掩蔽。最终，DeepSeek-V3 的训练语料由 14.8T 高质量、多样化的 token 组成（按我们的 tokenizer 计）。

在 DeepSeek-Coder-V2（DeepSeek-AI, 2024a）的训练过程中，我们观察到 Fill-in-Middle（FIM）策略不会损害下一 token 预测能力，同时使模型能够根据上下文线索准确预测中间文本。与 DeepSeek-Coder-V2 一致，我们也在 DeepSeek-V3 的预训练中引入了 FIM 策略。具体而言，我们采用 Prefix-Suffix-Middle（PSM）框架，将数据结构化为：<|fim_begin|> $f$pre <|fim_hole|> $f$suf <|fim_end|> $f$middle <|eos_token|>。该结构在文档级应用，作为 pre-packing 过程的一部分。FIM 策略以 0.1 的比例应用，与 PSM 框架一致。

DeepSeek-V3 的 tokenizer 采用 Byte-level BPE（Shibata et al., 1999），并将词表扩展到 128K 个 token。我们修改了 tokenizer 的预分词器（pretokenizer）与训练数据，以优化多语言压缩效率。此外，与 DeepSeek-V2 相比，新的预分词器引入了将标点与换行组合在一起的 token。然而，当模型处理没有末尾换行的多行 prompt（尤其是 few-shot 评估 prompt）时，这一技巧可能引入 token 边界偏差（token boundary bias；Lundberg, 2023）。为解决该问题，我们在训练期间按一定比例随机拆分这类组合 token，使模型暴露于更广泛的特例之中，从而缓解该偏差。

4.2 超参数

模型超参数：我们将 Transformer 层数设为 61，隐藏维度设为 7168。所有可学习参数均以标准差 0.006 随机初始化。在 MLA 中，我们将注意力头数 $n\hbar$ 设为 128，每头维度 $d\hbar$ 设为 128。KV 压缩维度 dc 设为 512，查询压缩维度 d' c 设为 1536。对于解耦的 query 与 key，我们将每头维度 dRh 设为 64。除前 3 层之外，我们将所有 FFN 替换为 MoE 层。每个 MoE 层由 1 个共享专家与 256 个

路由专家组成，每个专家的中间隐藏维度为 2048。在路由专家中，每个 token 会激活 8 个专家，并确保每个 token 最多被发送到 4 个节点。多 token 预测深度 D 设为 1，即除了精确的下一个 token 之外，每个 token 还会额外预测一个 token。与 DeepSeek-V2 相同，DeepSeek-V3 也在压缩的潜向量之后采用额外的 RMSNorm 层，并在宽度瓶颈处乘以额外的 scaling factor。在此配置下，DeepSeek-V3 共有 671B 总参数，其中每个 token 激活 37B 参数。

训练超参数：我们采用 AdamW 优化器（Loshchilov and Hutter, 2017），超参数设为 $\beta1 = 0.9$、$\beta2 = 0.95$、weight_decay = 0.1。预训练期间最大序列长度设为 4K，DeepSeek-V3 在 14.8T token 上进行预训练。在学习率调度方面，我们首先在前 2K 步内将其从 0 线性增加到 2.2×10^{-4}；随后，在模型消耗 10T 训练 token 之前，保持 2.2×10^{-4} 的恒定学习率；接着，按照余弦衰减曲线，在 4.3T token 内将学习率逐渐衰减到 2.2×10^{-5}。在最后 500B token 的训练中，我们在前 333B token 内保持 2.2×10^{-5} 的恒定学习率，并在剩余的 167B token 中切换到另一个恒定学习率 7.3×10^{-6}。梯度裁剪范数设为 1.0。我们采用批大小调度策略：在前 469B token 的训练中，批大小从 3072 逐渐增加到 15360，随后在剩余训练中保持 15360。我们利用流水线并行将模型的不同层部署到不同的 GPU 上，对于每一层，路由专家将均匀部署在属于 8 个节点的 64 块 GPU 上。在节点受限路由（node-limited routing）方面，每个 token 最多被发送到 4 个节点（即 $M = 4$）。对于无辅助损失负载均衡（auxiliary-loss-free load balancing），我们在前 14.3T token 中将偏置更新速度 γ 设为 0.001，在剩余的 500B token 中设为 0.0。对于平衡损失，我们将 α 设为 0.0001，仅用于避免任何单条序列内出现极端不平衡。MTP 损失权重 λ 在前 10T token 中设为 0.3，在剩余的 4.8T token 中设为 0.1。

【图表】图 8 “大海捞针”（Needle In A Haystack, NIAH）测试的评估结果。DeepSeek-V3 在所有最长至 128K 的上下文窗口长度上均表现良好（原文第 23 页）

4.3 长上下文扩展

我们采用与 DeepSeek-V2（DeepSeek-AI, 2024c）类似的方法为 DeepSeek-V3 赋予长上下文能力。在预训练阶段之后，我们应用 YaRN（Peng et al., 2023a）进行上下文扩展，并执行两个额外的训练阶段（每个阶段包含 1000 步），以逐步将上下文窗口从 4K 扩展到 32K，再扩展到 128K。YaRN 配置与 DeepSeek-V2 中使用的配置一致，仅应用于解耦的共享 key kRt。两个阶段的超参数保持相同，其中 scale $s = 40$、$\alpha = 1$、$\beta = 32$，缩放因子 $\sqrt{t} = 0.1 \ln s + 1$。在第一阶段，序列长度设为

32K，批大小为 1920；在第二阶段，序列长度增加到 128K，批大小降至 480。两个阶段的学习率均设为 7.3×10^{-6}，与预训练阶段的最终学习率一致。

通过这两个阶段的扩展训练，DeepSeek-V3 能够处理最长 128K 的输入，同时保持强劲性能。图 8 表明，经过监督微调后的 DeepSeek-V3 在“大海捞针”（Needle In A Haystack, NIAH）测试中取得了显著性能，在最长至 128K 的上下文窗口长度上表现出一致的稳健性。

4.4 评估

4.4.1 评估基准

DeepSeek-V3 的基座模型在一个以英文与中文为主的多语种语料上预训练，因此我们主要在一系列英文与中文基准上评估其性能，同时也在一个多语言基准上进行评估。我们的评估基于集成在 HAI-LLM 框架中的内部评估框架。所考虑的基准分类列出如下，其中带下划线的基准为中文基准，带双下划线的基准为多语言基准：

多学科多项选择题数据集包括 MMLU（Hendrycks 等，2020）、MMLU-Redux（Gema 等，2024）、MMLU-Pro（Wang 等，2024b）、MMMLU（OpenAI，2024b）、C-Eval（Huang 等，2023）与 CMMLU（Li 等，2023）。

语言理解与推理数据集包括 HellaSwag（Zellers 等，2019）、PIQA（Bisk 等，2020）、ARC（Clark 等，2018）与 BigBench Hard（BBH）（Suzgun 等，2022）。

闭卷问答数据集包括 TriviaQA（Joshi 等，2017）与 NaturalQuestions（Kwiatkowski 等，2019）。

阅读理解数据集包括 RACE（Lai 等，2017）、DROP（Dua 等，2019）、C3（Sun 等，2019a）与 CMRC（Cui 等，2019）。

指代消歧数据集包括 CLUEWSC（Xu 等，2020）与 WinoGrande（Sakaguchi 等，2019）。

语言建模数据集包括 Pile（Gao 等，2020）。

中文理解与文化数据集包括 CCPM（Li 等，2021）。

数学数据集包括 GSM8K（Cobbe 等，2021）、MATH（Hendrycks 等，2021）、MGSM（Shi 等，2023）与 CMath（Wei 等，2023）。

代码数据集包括 HumanEval（Chen 等，2021）、LiveCodeBench-Base（0801-1101）（Jain 等，2024）、MBPP（Austin 等，2021）与 CRUXEval（Gu 等，2024）。

标准化考试包括 AGIEval（Zhong 等，2023）。注意，AGIEval 同时包含英文与中文子集。

遵循我们此前的工作（DeepSeek-AI，2024b,c），我们对 HellaSwag、PIQA、WinoGrande、RACE-Middle、RACE-High、MMLU、MMLU-Redux、MMLU-Pro、MMMLU、ARC-Easy、ARC-Challenge、C-Eval、CMMLU、C3 与 CCPM 等数据集采用基于困惑度（perplexity）的评估；对 TriviaQA、NaturalQuestions、DROP、MATH、GSM8K、MGSM、HumanEval、MBPP、LiveCodeBench-Base、CRUXEval、BBH、AGIEval、CLUEWSC、CMRC 与 CMath 采用基于生成的评估。此外，我们对 Pile-test 采用基于语言建模的评估，并使用每字节比特数（Bits-Per-Byte, BPB）作为指标，以保证使用不同分词器的模型之间可以公平比较。

4.4.2 评估结果

我们在表 3 中将 DeepSeek-V3 的基座模型与最先进的开源基座模型进行比较，包括 DeepSeek-V2-Base（DeepSeek-AI，2024c）（我们此前发布的版本）、Qwen2.5 72B Base（Qwen，2024b）与 LLaMA-3.1 405B Base（AI@Meta，2024b）。我们用内部评估框架评估所有这些模型，并确保它们共享相同的评估设置。需要注意的是，由于过去几个月我们的评估框架发生了变化，DeepSeek-V2-Base 的性能与我们此前报告的结果存在轻微差异。总体而言，DeepSeek-V3-Base 全面优于 DeepSeek-V2-Base 与 Qwen2.5 72B Base，并在多数基准上超过 LLaMA-3.1 405B Base，基本成为最强的开源模型。

【表格】表 3 DeepSeek-V3-Base 与其他代表性开源基座模型之间的比较。所有模型均在我们的内部框架中评估，并共享相同的评估设置。差距不超过 0.3 的分数被视为同一水平。DeepSeek-V3-Base 在大多数基准上取得最佳性能，尤其是在数学与代码任务上（原文第 25 页）

从更细致的角度看，我们逐一将 DeepSeek-V3-Base 与其他开源基座模型进行比较。（1）与 DeepSeek-V2-Base 相比，得益于模型架构的改进、模型规模与训练 token 的扩大以及数据质量的提升，DeepSeek-V3-Base 如预期般取得了显著更好的性能。（2）与 Qwen2.5 72B Base 这一最先进的中文开源模型相比，DeepSeek-V3-Base 仅用其一半的激活参数，便展现出显著优势，尤其是在英文、多语言、代码与数学基准上。在中文基准方面，除了中文多学科多项选择题任务 CMMLU 之外，DeepSeek-V3-Base 的表现也优于 Qwen2.5 72B。（3）与 LLaMA-3.1 405B Base 这一激活参数为其 11 倍的最大开源模型相比，DeepSeek-V3-Base 在多语言、代码与数学基准上同样表现出好得多的性能。在英文与中文语言基准方面，DeepSeek-V3-Base 表现出相当或更好的性能，尤其在 BBH、MMLU 系列、DROP、C-Eval、CMMLU 与 CCPM 上表现突出。

得益于我们高效的架构与全面的工程优化，DeepSeek-V3 实现了极高的训练效率。在我们的训练框架与基础设施下，DeepSeek-V3 每训练一万亿 token 仅需 180K H800 GPU 小时，这比训练 72B 或 405B 稠密模型便宜得多。

4.5 讨论

4.5.1 多 token 预测的消融研究

在表 4 中，我们给出 MTP 策略的消融结果。具体而言，我们在两个不同规模的基线模型上验证 MTP 策略。在小规模上，我们在 1.33T token 上训练一个总参数量为 15.7B 的基线 MoE 模型；在大规模上，我们在 540B token 上训练一个总参数量为 228.7B 的基线 MoE 模型。在此基础上，保持训练数据与其他架构不变，我们为它们附加一个 1 层深度的 MTP 模块，并用 MTP 策略训练两个模型以进行比较。需要注意的是，在推理期间我们直接丢弃 MTP 模块，因此所比较模型的推理开销完全相同。从表中我们可以观察到，MTP 策略在大多数评估基准上持续提升模型性能。

【表格】表 4 MTP 策略的消融结果。MTP 策略在大多数评估基准上持续提升模型性能（原文第 26 页）

4.5.2 无辅助损失平衡策略的消融研究

在表 5 中，我们给出无辅助损失平衡策略的消融结果。我们在两个不同规模的基线模型上验证该策略。在小规模上，我们在 1.33T token 上训练一个总参数量为 15.7B 的基线 MoE 模型；在大规模上，我们在 578B token 上训练一个总参数量为 228.7B 的基线 MoE 模型。

这两个基线模型都纯粹使用辅助损失来鼓励负载均衡，并使用带有 top-K 亲和度归一化的 sigmoid 门控函数。它们用于控制辅助损失强度的超参数分别与 DeepSeek-V2-Lite 和 DeepSeek-V2 相同。在这两个基线模型的基础上，保持训练数据与其他架构不变，我们移除所有辅助损失并引入无辅助损失平衡策略以进行比较。从表中我们可以观察到，无辅助损失策略在大多数评估基准上持续取得更好的模型性能。

【表格】表 5 无辅助损失平衡策略的消融结果。与纯粹的基于辅助损失的方法相比，无辅助损失策略在大多数评估基准上持续取得更好的模型性能（原文第 27 页）

4.5.3 批次级负载均衡 VS. 序列级负载均衡

无辅助损失平衡与序列级辅助损失的关键区别在于其平衡范围：批次级与序列级。与序列级辅助损失相比，批次级平衡施加了更灵活的约束，因为它不要求在每个序列内部实现域内平衡。这种灵活性使专家能够更好地针对不同领域进行专业化。为验证这一点，我们记录并分析了 16B 基于辅助损失的基线模型与 16B 无辅助损失模型在 Pile 测试集不同领域上的专家负载。如图 9 所示，我们观察到无辅助损失模型如预期般展现出更强的专家专业化模式。

【图表】图 9 无辅助损失模型与基于辅助损失的模型在 Pile 测试集三个领域上的专家负载。无辅助损失模型比基于辅助损失的模型展现出更强的专家专业化模式。相对专家负载表示实际专家负载与理论均衡专家负载之间的比值。由于篇幅限制，我们仅展示两层的实验结果作为示例，所有层的结果见附录 C（原文第 28 页）

为了进一步研究这种灵活性与模型性能优势之间的关联，我们额外设计并验证了一种批次级辅助损失，它在每个训练批次而非每个序列上鼓励负载均衡。实验结果表明，在达到相近的批次级负载均衡水平时，批次级辅助损失也能取得与无辅助损失方法相似的模型性能。具体而言，在我们使用 1B MoE 模型的实验中，验证损失分别为：2.258（使用序列级辅助损失）、2.253（使用无辅助损失方法）与 2.253（使用批次级辅助损失）。我们在 3B MoE 模型上也观察到类似结果：使用序列级辅助损失的模型取得 2.085 的验证损失，而使用无辅助损失方法或批次级辅助损失的模型取得相同的 2.080 验证损失。

此外，尽管批次级负载均衡方法展现出一致的性能优势，它们在效率上也面临两个潜在挑战：（1）某些序列或小批次内部的负载不均衡；（2）推理期间由领域偏移引起的负载不均衡。第一个挑战由我们使用大规模专家并行与数据并行的训练框架自然解决，该框架保证每个 micro-batch 的规模足够大。对于第二个挑战，我们还设计并实现了一个带有冗余专家部署的高效推理框架（如第 3.4 节所述）来克服它。

5 后训练

5.1 监督微调

我们精心构建指令微调数据集，使其包含跨多个领域的 150 万条实例，每个领域都采用针对其特定需求定制的不同数据创建方法。

推理数据：对于与推理相关的数据集（包括专注于数学、代码竞赛问题与逻辑谜题的数据集），我们借助内部的 DeepSeek-R1 模型生成数据。具体而言，尽管 R1 生成的数据具有很高的准确性，但它存在过度思考、格式不佳与篇幅过长等问题。我们的目标是在 R1 生成推理数据的高准确性与常规格式推理数据的清晰简洁之间取得平衡。

为确立我们的方法，我们首先使用监督微调（SFT）与强化学习（RL）相结合的训练流水线，开发一个针对特定领域（如代码、数学或通用推理）的专家模型。该专家模型作为最终模型的数据生成器。训练过程会为每条实例生成两种不同的 SFT 样本：第一种以 <问题, 原始回复> 的格式将问题与其原始回复配对；第二种则同时包含系统提示、问题与 R1 回复，格式为 <系统提示, 问题, R1 回复>。

系统提示经过精心设计，包含引导模型产出带有反思与验证机制的回复的指令。在 RL 阶段，模型利用高温采样生成融合 R1 生成数据与原始数据模式的回复，即使在没有显式系统提示时也是如此。经过数百步 RL 之后，中间 RL 模型学会了融入 R1 的模式，从而在策略上提升整体性能。

在完成 RL 训练阶段后，我们采用拒绝采样为最终模型筛选高质量的 SFT 数据，其中专家模型被用作数据生成来源。该方法确保最终训练数据保留 DeepSeek-R1 的优势，同时生成简洁有效的回复。

非推理数据：对于非推理数据（如创意写作、角色扮演与简单问答），我们利用 DeepSeek-V2.5 生成回复，并聘请人工标注员核验数据的准确性与正确性。

SFT 设置：我们使用 SFT 数据集对 DeepSeek-V3-Base 微调两个 epoch，采用余弦衰减学习率调度，学习率从 5×10^−6 起始并逐渐降至 1×10^−6。训练时，每个序列由多个样本打包而成。不过，我们采用样本掩码策略，确保这些样本彼此隔离、互不可见。

5.2 强化学习

5.2.1 奖励模型

在我们的 RL 过程中，我们同时使用基于规则的奖励模型（Reward Model, RM）与基于模型的 RM。

基于规则的 RM：对于可以使用特定规则验证的问题，我们采用基于规则的奖励系统来确定反馈。例如，某些数学问题具有确定的结果，我们要求模型以指定格式（例如放在一个方框中）给出最终答案，从而可以应用规则来验证其正确性。同样，对于 LeetCode 问题，我们可以利用编译器根据测试用例生成反馈。尽可能利用基于规则的验证，可以确保更高的可靠性，因为这种方法不易被操纵或利用。

基于模型的 RM：对于具有自由形式标准答案的问题，我们依赖奖励模型判断回复是否与预期标准答案匹配。相反，对于没有确定标准答案的问题（如涉及创意写作的问题），奖励模型的任务是根据作为输入的问题与相应回复来提供反馈。该奖励模型基于 DeepSeek-V3 的 SFT 检查点训练。为增强其可靠性，我们构建的偏好数据不仅提供最终奖励，还包含通向该奖励的思维链。这种方法有助于降低特定任务中奖励被攻击（reward hacking）的风险。

5.2.2 组相对策略优化

与 DeepSeek-V2（DeepSeek-AI，2024c）类似，我们采用组相对策略优化（Group Relative Policy Optimization, GRPO）（Shao 等，2024），它舍弃了通常与策略模型规模相同的评论家（critic）模型，转而根据组内得分估计基线。具体而言，对于每个问题 q，GRPO 从旧策略模型 π_θold 中采样一组输出 {o_1, o_2, ⋯, o_G}，然后通过最大化以下目标来优化策略模型 π_θ：

J_GRPO(θ) = E[q ∼ P(Q), {o_i}_{i=1}^{G} ∼ π_θold(O|q)] (1/G) Σ_{i=1}^{G} [min((π_θ(o_i|q)/π_θold(o_i|q)) A_i, clip((π_θ(o_i|q)/π_θold(o_i|q)), 1−ε, 1+ε) A_i) − β D_KL(π_θ ‖ π_ref)] （26）

D_KL(π_θ ‖ π_ref) = (π_ref(o_i|q)/π_θ(o_i|q)) − log(π_ref(o_i|q)/π_θ(o_i|q)) − 1 （27）

其中 ε 与 β 为超参数；π_ref 为参考模型；A_i 为优势值，由组内各输出对应的奖励 {r_1, r_2, ..., r_G} 导出：

A_i = (r_i − mean({r_1, r_2, ⋯, r_G})) / std({r_1, r_2, ⋯, r_G}) （28）

在 RL 过程中，我们纳入了来自多种领域（如编程、数学、写作、角色扮演与问答）的提示。这种方法不仅使模型更贴近人类偏好，还提升了基准上的性能，尤其是在可用 SFT 数据有限的场景中。

5.3 评估

5.3.1 评估设置

评估基准：除了用于基座模型测试的基准之外，我们还在 IFEval（Zhou 等，2023）、FRAMES（Krishna 等，2024）、LongBench v2（Bai 等，2024）、GPQA（Rein 等，2023）、SimpleQA（OpenAI，2024c）、C-SimpleQA（He 等，2024）、SWE-Bench Verified（OpenAI，2024d）、Aider[1]、LiveCodeBench（Jain 等，2024）（2024 年 8 月至 2024 年 11 月的问题）、Codeforces[2]、中国全国高中数学联赛（CNMO 2024）[3] 以及 2024 年美国数学邀请赛（AIME 2024）（MAA，2024）上进一步评估指令模型。

对比基线：我们对我们的 chat 模型与若干强基线进行全面评估，包括 DeepSeek-V2-0506、DeepSeek-V2.5-0905 、 Qwen2.5 72B Instruct 、 LLaMA-3.1 405B Instruct 、 Claude-Sonnet-3.5-1022 与 GPT-4o-0513。对于 DeepSeek-V2 模型系列，我们选择最具代表性的变体进行比较。对于闭源模型，评估通过其各自的 API 进行。

详细评估配置：对于 MMLU、DROP、GPQA 与 SimpleQA 等标准基准，我们采用 simple-evals 框架[4]中的评估提示。对于 MMLU-Redux，我们在零样本设置下使用 Zero-Eval 提示格式（Lin，2024）。对于其他数据集，我们遵循其原始评估协议，并使用数据集创建者提供的默认提示。对于代码与数学基准，HumanEval-Mul 数据集共包含 8 种主流编程语言（Python、Java、Cpp、C#、JavaScript、TypeScript、PHP 与 Bash）。我们使用 CoT 与非 CoT 方法评估模型在 LiveCodeBench 上的性能，该数据采集于 2024 年 8 月至 2024 年 11 月。Codeforces 数据集使用参赛者的百分位来衡量。SWE-Bench verified 使用 agentless 框架（Xia 等，2024）评估。我们使用“diff”格式评估 Aider 相关基准。对于数学评估，AIME 与 CNMO 2024 以 0.7 的温度评估，并对 16 次运行的结果取平均，而 MATH-500 采用贪心解码。我们允许所有模型在每个基准上最多输出 8192 个 token。

脚注 1：https://aider.chat

脚注 2：https://codeforces.com

脚注 3：https://www.cms.org.cn/Home/comp/comp/cid/12.html

脚注 4：https://github.com/openai/simple-evals

5.3.2 标准评估

表 6 给出了评估结果，表明 DeepSeek-V3 是表现最好的开源模型。此外，它还能与 GPT-4o 与 Claude-3.5-Sonnet 等前沿闭源模型竞争。

【表格】表 6 DeepSeek-V3 与其他代表性 chat 模型之间的比较。所有模型均在将输出长度限制为 8K 的配置下评估。样本数少于 1000 的基准使用不同的温度设置多次测试，以得到稳健的最终结果。DeepSeek-V3 是表现最好的开源模型，并且与前沿闭源模型相比也展现出有竞争力的性能（原文第 31 页）

英文基准：MMLU 是一个广泛认可的基准，旨在评估大语言模型在多样化的知识领域与任务上的表现。DeepSeek-V3 展现出有竞争力的性能，与 LLaMA-3.1-405B、GPT-4o 和 Claude-Sonnet 3.5 等顶级模型并驾齐驱，同时显著优于 Qwen2.5 72B。此外，DeepSeek-V3 在更具挑战性的教育知识基准 MMLU-Pro 上表现出色，紧随 Claude-Sonnet 3.5 之后。在标签经过修正的 MMLU 改进版 MMLU-

Redux 上，DeepSeek-V3 超越了同类模型。此外，在博士级评估测试床 GPQA-Diamond 上，DeepSeek-V3 取得了出色的结果，仅次于 Claude 3.5 Sonnet，并以明显优势优于其他所有竞争者。

在 DROP、LongBench v2 与 FRAMES 等长上下文理解基准上，DeepSeek-V3 继续展现其顶级模型的地位。它在 DROP 的 3-shot 设置下取得了令人印象深刻的 91.6 F1 分数，优于该类别中的所有其他模型。在需要在超过 100k token 的上下文中进行问答的基准 FRAMES 上，DeepSeek-V3 紧随 GPT-4o 之后，并以显著优势优于其他所有模型。这展现了 DeepSeek-V3 处理极长上下文任务的强大能力。DeepSeek-V3 的长上下文能力还通过其在 LongBench v2 上的同类最佳表现得到进一步验证——该数据集在 DeepSeek-V3 发布前几周才发布。在事实知识基准 SimpleQA 上，DeepSeek-V3 落后于 GPT-4o 与 Claude-Sonnet，这主要归因于其设计侧重与资源分配。DeepSeek-V3 将更多训练 token 分配用于学习中文知识，因此在 C-SimpleQA 上表现优异。在指令遵循基准上，DeepSeek-V3 显著优于其前代 DeepSeek-V2 系列，凸显了其在理解并遵循用户自定义格式约束方面能力的提升。

代码与数学基准：编码对大语言模型而言是一项具有挑战性且实用的任务，既包括 SWE-Bench-Verified 与 Aider 等偏工程的任务，也包括 HumanEval 与 LiveCodeBench 等算法任务。在工程任务中，DeepSeek-V3 落后于 Claude-Sonnet-3.5-1022，但显著优于开源模型。开源的 DeepSeek-V3 有望推动编码相关工程任务的进步。通过开放其强大的能力，DeepSeek-V3 可以推动软件工程与算法开发等领域的创新与改进，赋能开发者与研究者拓展开源模型在编码任务上所能达到的边界。在算法任务中，DeepSeek-V3 展现出更优的性能，在 HumanEval-Mul 与 LiveCodeBench 等基准上优于所有基线。这一成功可归因于其先进的知识蒸馏技术，该技术有效增强了其在以算法为中心的任务中的代码生成与问题求解能力。

在数学基准上，DeepSeek-V3 展现出卓越的性能，显著超越基线，并为非 o1 类模型树立了新的最先进水平。具体而言，在 AIME、MATH-500 与 CNMO 2024 上，DeepSeek-V3 在绝对分数上比第二好的模型 Qwen2.5 72B 高出约 10%，对于如此具有挑战性的基准而言这是相当大的差距。这一出色能力凸显了从 DeepSeek-R1 蒸馏技术的有效性，该技术已被证明对非 o1 类模型大有裨益。

中文基准：Qwen 与 DeepSeek 是两个对中英文都提供稳健支持的代表性模型系列。在事实性基准 Chinese SimpleQA 上，DeepSeek-V3 以 16.4 分的优势超过 Qwen2.5-72B，尽管 Qwen2.5 是在包含 18T token 的更大语料上训练的——这比 DeepSeek-V3 预训练所用的 14.8T token 还多 20%。

在中文教育知识评估的代表性基准 C-Eval 与 CLUEWSC（中文 Winograd Schema Challenge）上，DeepSeek-V3 与 Qwen2.5-72B 表现出相近的性能水平，表明这两个模型都针对具有挑战性的中文推理与教育任务进行了良好优化。

5.3.3 开放式评估

除标准基准外，我们还使用 LLM 作为评判者在开放式生成任务上评估模型，结果如表 7 所示。具体而言，我们遵循 AlpacaEval 2.0（Dubois 等，2024）与 Arena-Hard（Li 等，2024a）的原始配置，二者都利用 GPT-4-Turbo-1106 作为评判者进行成对比较。在 Arena-Hard 上，DeepSeek-V3 相对于基线 GPT-4-0314 取得了超过 86% 的令人印象深刻的胜率，与 Claude-Sonnet-3.5-1022 等顶级模型表现相当。这凸显了 DeepSeek-V3 的稳健能力，尤其是在处理包括编码与调试任务在内的复杂提示时。此外，DeepSeek-V3 取得了开创性的里程碑：成为首个在 Arena-Hard 基准上超过 85% 的开源模型。这一成就显著缩小了开源与闭源模型之间的性能差距，为开源模型在具有挑战性的领域所能取得的成就树立了新的标准。同样，DeepSeek-V3 在 AlpacaEval 2.0 上也展现出卓越性能，优于闭源与开源模型。这展现了其在写作任务与处理直接问答场景方面的出色能力。值得注意的是，它以 20% 的显著优势超过 DeepSeek-V2.5-0905，凸显了其在处理简单任务上的大幅改进，也展示了其各项改进的有效性。

【表格】表 7 英文开放式对话评估。对于 AlpacaEval 2.0，我们使用长度控制的胜率作为指标（原文第 33 页）

5.3.4 DeepSeek-V3 作为生成式奖励模型

我们将 DeepSeek-V3 的评判能力与最先进模型（即 GPT-4o 与 Claude-3.5）进行比较。表 8 给出了这些模型在 RewardBench（Lambert 等，2024）上的表现。DeepSeek-V3 取得了与 GPT-4o-0806 和 Claude-3.5-Sonnet-1022 最佳版本相当的性能，同时超过其他版本。此外，DeepSeek-V3 的评判能力还可以通过投票技术进一步增强。因此，我们采用 DeepSeek-V3 结合投票，对开放式问题提供自我反馈，从而提升对齐过程的有效性与稳健性。

【表格】表 8 GPT-4o、Claude-3.5-sonnet 与 DeepSeek-V3 在 RewardBench 上的表现（原文第 34 页）

5.4 讨论

5.4.1 从 DeepSeek-R1 蒸馏

我们在 DeepSeek-V2.5 的基础上消融从 DeepSeek-R1 蒸馏的贡献。基线模型在短 CoT 数据上训练，而其对照模型使用上述专家检查点生成的数据。

表 9 证明了蒸馏数据的有效性，在 LiveCodeBench 与 MATH-500 基准上均有显著提升。我们的实验揭示了一个有趣的权衡：蒸馏带来更好的性能，但也大幅增加了平均回复长度。为了在模型准确性与计算效率之间保持平衡，我们在蒸馏过程中为 DeepSeek-V3 仔细选择了最优设置。

表 9：从 DeepSeek-R1 蒸馏的贡献。LiveCodeBench 与 MATH-500 的评估设置与表 6 相同。（原文第 34 页）

模型	LiveCodeBench-CoT Pass@1	LiveCodeBench-CoT Length	MATH-500 Pass@1	MATH-500 Length
DeepSeek-V2.5 Baseline	31.1	718	74.6	769
DeepSeek-V2.5 + R1 Distill	37.4	783	83.2	1510

我们的研究表明，从推理模型进行知识蒸馏是后训练优化的一个有前景的方向。虽然我们当前的工作聚焦于从数学与编程领域蒸馏数据，但该方法在各类任务领域中展现出更广泛应用的潜力。在这些特定领域所展示的有效性表明，长 CoT 蒸馏可能有助于提升其他需要复杂推理的认知任务中的模型性能。在不同领域进一步探索该方法仍是未来研究的重要方向。

5.4.2 自我奖励

奖励在 RL 中起着关键作用，引导着优化过程。在通过外部工具即可直接验证的领域（例如某些编程或数学场景），RL 展现出卓越的功效。然而，在更一般的场景中，通过硬编码构建反馈机制并不现实。在 DeepSeek-V3 的开发过程中，针对这些更广泛的情境，我们采用宪法式 AI（constitutional AI）方法（Bai 等，2022），利用 DeepSeek-V3 自身的投票评估结果作为反馈来源。该方法产生了显著的对齐效果，大幅提升了 DeepSeek-V3 在主观评估中的表现。通过整合额外的宪法式输入，DeepSeek-V3 可以朝着宪法式方向优化。我们认为，这种将补充信息与作为反馈来源的 LLM 相结合的范式至关重要。LLM 充当一个多功能的处理器，能够将来自不同场景的非结构化信息转化为奖励，最终促进 LLM 的自我改进。除自我奖励之外，我们还致力于发掘其他通用且可扩展的奖励方法，以在一般场景中持续提升模型能力。

5.4.3 多 token 预测评估

DeepSeek-V3 通过 MTP 技术预测接下来的 2 个 token，而不仅仅是下一个单 token。结合推测解码（speculative decoding）框架（Leviathan 等，2023；Xia 等，2023），它可以显著加速模型的解码速

度。一个自然而然的问题是关于额外预测 token 的接受率。根据我们的评估，在不同生成主题下，第二个 token 预测的接受率介于 85% 与 90% 之间，表现出稳定的可靠性。这一高接受率使 DeepSeek-V3 能够显著提升解码速度，达到 1.8 倍的 TPS（每秒 token 数）。

6 结论、局限与未来方向

本文提出 DeepSeek-V3，这是一个总参数量为 671B、激活参数量为 37B 的大型 MoE 语言模型，在 14.8T token 上训练。除 MLA 与 DeepSeekMoE 架构之外，它还开创了用于负载均衡的无辅助损失策略，并设定了多 token 预测训练目标以获得更强的性能。得益于 FP8 训练的支持与细致的工程优化，DeepSeek-V3 的训练具有成本效益。后训练也成功地从 DeepSeek-R1 系列模型中蒸馏出推理能力。全面评估表明，DeepSeek-V3 已成为当前可用的最强开源模型，并取得了与 GPT-4o 和 Claude-3.5-Sonnet 等领先闭源模型相当的性能。尽管性能强劲，它仍保持经济的训练成本。其完整训练（包括预训练、上下文长度扩展与后训练）仅需 2.788M H800 GPU 小时。

在肯定其强劲性能与成本效益的同时，我们也认识到 DeepSeek-V3 存在一些局限，尤其是在部署方面。首先，为确保高效推理，DeepSeek-V3 推荐的部署单元相对较大，这可能给小型团队带来负担。其次，尽管我们为 DeepSeek-V3 设计的部署策略已经实现了超过 DeepSeek-V2 两倍的端到端生成速度，但仍有进一步提升的潜力。幸运的是，随着更先进硬件的发展，这些局限有望自然得到解决。

DeepSeek 始终坚持以长期主义为指引的开源模型路线，致力于稳步迈向通用人工智能（Artificial General Intelligence, AGI）的终极目标。未来，我们计划在以下方向上战略性地投入研究。

- 我们将持续研究与改进模型架构，进一步提升训练与推理效率，努力实现对无限上下文长度的高效支持。此外，我们将尝试突破 Transformer 的架构局限，从而拓展其建模能力的边界。
- 我们将持续迭代训练数据的数量与质量，并探索引入额外的训练信号来源，以在更全面的维度上推动数据扩展。
- 我们将持续探索并迭代模型的深度思考能力，通过扩展推理的长度与深度来增强其智能与问题求解能力。
- 我们将探索更全面、多维度的模型评估方法，以避免研究过程中过度优化固定基准集的倾向——这种倾向可能对模型能力造成误导性的印象，并影响我们的基础性评估。

附录

附录 A 贡献与致谢

A.1 低精度训练的消融研究

A.2 FP8 与 BF16 训练对比

A.3 关于分块量化的讨论

A.4 16B 辅助损失与无辅助损失模型的专家专业化模式

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] • We will continuously iterate on the quantity and quality of our training data, and explore the incorporation of additional training signal sources, aiming to drive data scaling across a more comprehensive range of dimensions. • We will consistently explore and iterate on the deep thinking capabilities of our models, aiming to enhance their intelligence and problem-solving abilities by expanding their reasoning length and depth. • We will explore more comprehensive and multi-dimensional model evaluation methods to prevent the tendency towards optimizing a fixed set of benchmarks during research, which may create a misleading impression of the model capabilities and affect our foundational assessment.

[2] AI@Meta. Llama 3 model card, 2024a. URL https://github.com/meta-llama/llama3/bl ob/main/MODEL_CARD.md .

[3] AI@Meta. Llama 3.1 model card, 2024b. URL https://github.com/meta-llama/llama-m odels/blob/main/models/llama3_1/MODEL_CARD.md .

[4] Anthropic. Claude 3.5 sonnet, 2024. URL https://www.anthropic.com/news/claude-3 -5-sonnet .

[5] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

[6] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. arXiv preprint arXiv:2212.08073, 2022.

[7] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. arXiv preprint arXiv:2412.15204, 2024.

[8] M. Bauer, S. Treichler, and A. Aiken. Singe: leveraging warp specialization for high performance on GPUs. In Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPoPP '14, page 119–130, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450326568. doi: 10.1145/2555243.2555258. URL https://doi.org/10.1145/2555243.2555258 .

[9] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi. PIQA: reasoning about physical commonsense in natural language. In The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020, pages 7432–7439. AAAI Press, 2020. doi: 10.1609/aaai.v34i05.6239. URL https://doi.org/10.1609/aaai.v34i05.6239 .

[10] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374 .

[11] P . Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. CoRR, abs/1803.05457, 2018. URL http://arxiv.org/abs/1803.05457 .

[12] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[13] Y. Cui, T. Liu, W. Che, L. Xiao, Z. Chen, W. Ma, S. Wang, and G. Hu. A span-extraction dataset for Chinese machine reading comprehension. In K. Inui, J. Jiang, V . Ng, and X. Wan, editors, Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 5883–5889, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1600. URL https://aclanthology.org/D19-1 600 .

[14] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P . Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL https://doi.org/10.48550/arXiv.2401.06066 .

[15] DeepSeek-AI. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. CoRR, abs/2406.11931, 2024a. URL https://doi.org/10.48550/arXiv.2406.11 931 .

[16] DeepSeek-AI. Deepseek LLM: scaling open-source language models with longtermism. CoRR, abs/2401.02954, 2024b. URL https://doi.org/10.48550/arXiv.2401.02954 .

[17] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024c. URL https://doi.org/10.48550/arXiv.2405. 04434 .

[18] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. Advances in Neural Information Processing Systems, 35:30318– 30332, 2022.

[19] H. Ding, Z. Wang, G. Paolini, V . Kumar, A. Deoras, D. Roth, and S. Soatto. Fewer truncations improve language modeling. arXiv preprint arXiv:2404.10830, 2024.

[20] D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368– 2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246 .

[21] Y. Dubois, B. Galambosi, P . Liang, and T. B. Hashimoto. Length-controlled alpacaeval: A simple way to debias automatic evaluators. arXiv preprint arXiv:2404.04475, 2024. W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. CoRR, abs/2101.03961, 2021. URL https://arxiv.org/ abs/2101.03961 .

[22] M. Fishman, B. Chmiel, R. Banner, and D. Soudry. Scaling FP8 training to trillion-token llms. arXiv preprint arXiv:2409.12517, 2024.

[23] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. arXiv preprint arXiv:2210.17323, 2022.

[24] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

[25] A. P . Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P . Minervini. Are we done with mmlu? CoRR, abs/2406.04127, 2024. URL https://doi.or g/10.48550/arXiv.2406.04127 .

[26] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=pEWAcejiU2 .

[27] Google. Our next-generation model: Gemini 1.5, 2024. URL https://blog.google/techno logy/ai/google-gemini-next-generation-model-february-2024 .

[28] R. L. Graham, D. Bureddy, P . Lui, H. Rosenstock, G. Shainer, G. Bloch, D. Goldenerg, M. Dubman, S. Kotchubievsky, V . Koushnir, et al. Scalable hierarchical aggregation protocol (SHArP): A hardware architecture for efficient data reduction. In 2016 First International Workshop on Communication Optimizations in HPC (COMHPC), pages 1–10. IEEE, 2016.

[29] A. Gu, B. Rozière, H. Leather, A. Solar-Lezama, G. Synnaeve, and S. I. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution, 2024.

[30] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang. Deepseek-coder: When the large language model meets programming - the rise of code intelligence. CoRR, abs/2401.14196, 2024. URL https://doi.org/10.485 50/arXiv.2401.14196 .

[31] A. Harlap, D. Narayanan, A. Phanishayee, V . Seshadri, N. Devanur, G. Ganger, and P . Gibbons. Pipedream: Fast and efficient pipeline parallel dnn training, 2018. URL https://arxiv.or g/abs/1806.03377 .

[32] B. He, L. Noci, D. Paliotta, I. Schlag, and T. Hofmann. Understanding and minimising outlier features in transformer training. In The Thirty-eighth Annual Conference on Neural Information Processing Systems.

[33] Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. arXiv preprint arXiv:2411.07140, 2024.

[34] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020. D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[35] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

[36] N. Jain, K. Han, A. Gu, W. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. CoRR, abs/2403.07974, 2024. URL https://doi.org/10.48550/arXiv.2403.07974 .

[37] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, et al. Mistral 7b. arXiv preprint arXiv:2310.06825, 2023.

[38] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147 .

[39] D. Kalamkar, D. Mudigere, N. Mellempudi, D. Das, K. Banerjee, S. Avancha, D. T. Vooturi, N. Jammalamadaka, J. Huang, H. Yuen, et al. A study of bfloat16 for deep learning training. arXiv preprint arXiv:1905.12322, 2019.

[40] S. Krishna, K. Krishna, A. Mohananey, S. Schwarcz, A. Stambler, S. Upadhyay, and M. Faruqui. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. CoRR, abs/2409.12941, 2024. doi: 10.48550/ARXIV.2409.12941. URL https://doi.org/10.485 50/arXiv.2409.12941 .

[41] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. P . Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov. Natural questions: a benchmark for question answering research. Trans. Assoc. Comput. Linguistics, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://doi.org/10.1162/tacl_a_00276 .

[42] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. H. Hovy. RACE: large-scale reading comprehension dataset from examinations. In M. Palmer, R. Hwa, and S. Riedel, editors, Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017, Copenhagen, Denmark, September 9-11, 2017, pages 785–794. Association for Computational Linguistics, 2017. doi: 10.18653/V1/D17-1082. URL https://doi.org/10.18653/v1/d1 7-1082 .

[43] N. Lambert, V . Pyatkin, J. Morrison, L. Miranda, B. Y. Lin, K. Chandu, N. Dziri, S. Kumar, T. Zick, Y. Choi, et al. Rewardbench: Evaluating reward models for language modeling. arXiv preprint arXiv:2403.13787, 2024.

[44] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In 9th International Conference on Learning Representations, ICLR 2021. OpenReview.net, 2021. URL https://openreview.net/forum?id=qrwe7XHTmYb . Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA, volume 202 of Proceedings of Machine Learning Research, pages 19274–19286. PMLR, 2023. URL https://proceedings.mlr.press/v202/leviathan23 a.html .

[45] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.

[46] S. Li and T. Hoefler. Chimera: efficiently training large-scale neural networks with bidirectional pipelines. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21, page 1–14. ACM, Nov. 2021. doi: 10.1145/345881 7.3476145. URL http://dx.doi.org/10.1145/3458817.3476145 .

[47] T. Li, W.-L. Chiang, E. Frick, L. Dunlap, T. Wu, B. Zhu, J. E. Gonzalez, and I. Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. arXiv preprint arXiv:2406.11939, 2024a.

[48] W. Li, F. Qi, M. Sun, X. Yi, and J. Zhang. Ccpm: A chinese classical poetry matching dataset, 2021.

[49] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024b. URL https://openreview.net /forum?id=1NdN7eXyb4 .

[50] B. Y. Lin. ZeroEval: A Unified Framework for Evaluating Language Models, July 2024. URL https://github.com/WildEval/ZeroEval .

[51] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

[52] S. Lundberg. The art of prompt design: Prompt boundaries and token healing, 2023. URL https://towardsdatascience.com/the-art-of-prompt-design-prompt-bound aries-and-token-healing-3b2448b0be38 .

[53] Y. Luo, Z. Zhang, R. Wu, H. Liu, Y. Jin, K. Zheng, M. Wang, Z. He, G. Hu, L. Chen, et al. Ascend HiFloat8 format for deep learning. arXiv preprint arXiv:2409.16626, 2024.

[54] MAA. American invitational mathematics examination - aime. In American Invitational Mathematics Examination - AIME 2024, February 2024. URL https://maa.org/math -competitions/american-invitational-mathematics-examination-aime .

[55] P . Micikevicius, D. Stosic, N. Burgess, M. Cornea, P . Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P . Judd, J. Kamalu, et al. FP8 formats for deep learning. arXiv preprint arXiv:2209.05433, 2022.

[56] Mistral. Cheaper, better, faster, stronger: Continuing to push the frontier of ai and making it accessible to all, 2024. URL https://mistral.ai/news/mixtral-8x22b .

[57] S. Narang, G. Diamos, E. Elsen, P . Micikevicius, J. Alben, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, et al. Mixed precision training. In Int. Conf. on Learning Representation, 2017. B. Noune, P . Jones, D. Justus, D. Masters, and C. Luschi. 8-bit numerical formats for deep neural networks. arXiv preprint arXiv:2206.02915, 2022.

[58] NVIDIA. Improving network performance of HPC systems using NVIDIA Magnum IO NVSH- MEM and GPUDirect Async. https://developer.nvidia.com/blog/improving-net work-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async , 2022.

[59] NVIDIA. Blackwell architecture. https://www.nvidia.com/en-us/data-center/tech nologies/blackwell-architecture/ , 2024a.

[60] NVIDIA. TransformerEngine, 2024b. URL https://github.com/NVIDIA/TransformerE ngine . Accessed: 2024-11-19.

[61] OpenAI. Hello GPT-4o, 2024a. URL https://openai.com/index/hello-gpt-4o/ .

[62] OpenAI. Multilingual massive multitask language understanding (mmmlu), 2024b. URL https://huggingface.co/datasets/openai/MMMLU .

[63] OpenAI. Introducing SimpleQA, 2024c. URL https://openai.com/index/introducing -simpleqa/ .

[64] OpenAI. Introducing SWE-bench verified we're releasing a human-validated subset of swebench that more, 2024d. URL https://openai.com/index/introducing-swe-bench -verified/ .

[65] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models. arXiv preprint arXiv:2309.00071, 2023a.

[66] H. Peng, K. Wu, Y. Wei, G. Zhao, Y. Yang, Z. Liu, Y. Xiong, Z. Yang, B. Ni, J. Hu, et al. FP8-LM: Training FP8 large language models. arXiv preprint arXiv:2310.18313, 2023b.

[67] P . Qi, X. Wan, G. Huang, and M. Lin. Zero bubble pipeline parallelism. arXiv preprint arXiv:2401.10241, 2023a.

[68] P . Qi, X. Wan, G. Huang, and M. Lin. Zero bubble pipeline parallelism, 2023b. URL https: //arxiv.org/abs/2401.10241 .

[69] Qwen. Qwen technical report. arXiv preprint arXiv:2309.16609, 2023.

[70] Qwen. Introducing Qwen1.5, 2024a. URL https://qwenlm.github.io/blog/qwen1.5 .

[71] Qwen. Qwen2.5: A party of foundation models, 2024b. URL https://qwenlm.github.io/b log/qwen2.5 .

[72] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.

[73] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

[74] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, et al. Microscaling data formats for deep learning. arXiv preprint arXiv:2310.10537, 2023a. B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, et al. Microscaling data formats for deep learning. arXiv preprint arXiv:2310.10537, 2023b.

[75] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.

[76] Z. Shao, P . Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[77] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. V . Le, G. E. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In 5th International Conference on Learning Representations, ICLR 2017. OpenReview.net, 2017. URL https: //openreview.net/forum?id=B1ckMDqlg .

[78] F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?id=fR3wGCk-IXp .

[79] Y. Shibata, T. Kida, S. Fukamachi, M. Takeda, A. Shinohara, T. Shinohara, and S. Arikawa. Byte pair encoding: A text compression scheme that accelerates pattern matching. 1999.

[80] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.

[81] K. Sun, D. Yu, D. Yu, and C. Cardie. Investigating prior knowledge for challenging chinese machine reading comprehension, 2019a.

[82] M. Sun, X. Chen, J. Z. Kolter, and Z. Liu. Massive activations in large language models. arXiv preprint arXiv:2402.17762, 2024.

[83] X. Sun, J. Choi, C.-Y. Chen, N. Wang, S. Venkataramani, V . V . Srinivasan, X. Cui, W. Zhang, and K. Gopalakrishnan. Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks. Advances in neural information processing systems, 32, 2019b.

[84] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[85] V . Thakkar, P . Ramani, C. Cecka, A. Shivam, H. Lu, E. Yan, J. Kosaian, M. Hoemmen, H. Wu, A. Kerr, M. Nicely, D. Merrill, D. Blasig, F. Qiao, P . Majcher, P . Springer, M. Hohnerbach, J. Wang, and M. Gupta. CUTLASS, Jan. 2023. URL https://github.com/NVIDIA/cutlas s .

[86] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023a.

[87] H. Touvron, L. Martin, K. Stone, P . Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P . Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. Canton-Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V . Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V . Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P . S. Koura, M. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P . Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P . Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T.

Scialom. Llama 2: Open foundation and fine-tuned chat models. CoRR, abs/2307.09288, 2023b. doi: 10.48550/arXiv.2307.09288. URL https://doi.org/10.48550/arXiv.2307. 09288 .

[88] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[89] L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. CoRR, abs/2408.15664, 2024a. URL https://doi.org/10.48550/arX iv.2408.15664 .

[90] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. CoRR, abs/2406.01574, 2024b. URL https://doi.org/10.48550/arXiv.2406.01574 .

[91] T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.

[92] M. Wortsman, T. Dettmers, L. Zettlemoyer, A. Morcos, A. Farhadi, and L. Schmidt. Stable and low-precision training for large-scale vision-language models. Advances in Neural Information Processing Systems, 36:10271–10298, 2023.

[93] H. Xi, C. Li, J. Chen, and J. Zhu. Training transformers with 4-bit integers. Advances in Neural Information Processing Systems, 36:49146–49168, 2023.

[94] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying llm-based software engineering agents. arXiv preprint, 2024.

[95] H. Xia, T. Ge, P . Wang, S. Chen, F. Wei, and Z. Sui. Speculative decoding: Exploiting speculative execution for accelerating seq2seq generation. In Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023, pages 3909–3925. Association for Computational Linguistics, 2023. URL https://doi.org/10.18653/v1/ 2023.findings-emnlp.257 .

[96] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In International Conference on Machine Learning, pages 38087–38099. PMLR, 2023.

[97] L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL https://doi.org/10.18653/v1/2020.coling-main.419 . R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[98] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364 .

[99] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models. arXiv preprint arXiv:2311.07911, 2023.

DeepSeek-R1：通过强化学习激励 LLM 的推理能力

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI research@deepseek.com

arXiv:2501.12948 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

通用推理是人工智能中长期存在且极其艰巨的挑战。近期的突破，例如大语言模型（LLM）与思维链（CoT）提示，已在基础推理任务上取得可观的成功。然而，这些成功在很大程度上依赖于大量人工标注的示例，且模型的能力对于更复杂的问题仍然不足。我们在此表明，LLM 的推理能力可以通过纯强化学习（RL）来激励，而无需人工标注的推理轨迹。所提出的 RL 框架促进了高级推理模式的涌现发展，例如自我反思、验证与动态策略调整。因此，训练得到的模型在数学、编程竞赛与 STEM 等可验证任务上取得了优异表现，超过了通过传统监督学习在人类示例上训练的同类模型。此外，这些大规模模型所展现的涌现推理模式可以被系统地加以利用，以指导和增强更小模型的推理能力。

1 引言

推理能力是人类智能的基石，使人能够完成从数学问题求解到逻辑推演、编程等复杂认知任务。人工智能的近期进展表明，当规模足够大时，大语言模型（LLM）可以展现出包括推理能力在内的涌现行为（Kaplan 等，2020；Wei 等，2022a）。然而，在预训练中获得此类能力通常需要耗费大量计算资源。与此同时，另一条互补的研究路线表明，大语言模型可以通过思维链（CoT）提示得到有效增强。该技术要么提供精心设计的少样本示例，要么使用诸如“Let's think step by step”这样的极简提示（Kojima 等，2022；Wei 等，2022b），从而使模型能够生成中间推理步骤，显著提升其在复杂任务上的表现。类似地，当模型在后训练阶段学习高质量的多步推理轨迹时，也可以观察到进一步的性能提

升（Chung 等，2024；OpenAI，2023）。尽管这些方法行之有效，但它们存在明显的局限。它们对人工标注推理轨迹的依赖阻碍了可扩展性，并引入了认知偏见。此外，通过将模型约束为复现人类的思维过程，其性能天然地受限于人类，提供的示例，这阻碍了对更优的、非人类式推理路径的探索。

为应对这些问题，我们旨在探索大语言模型（LLM）在 RL 框架中通过自我进化发展推理能力的潜力，并尽量减少对人工标注的依赖。具体而言，我们以 DeepSeek-V3-Base（DeepSeek-AI，2024b）为基础，并采用组相对策略优化（GRPO）（Shao 等，2024）作为 RL 框架。奖励信号仅基于最终预测相对于标准答案的正确性，不对推理过程本身施加约束。值得注意的是，我们在 RL 训练之前跳过了传统的监督微调（SFT）阶段。这一设计选择源于我们的假设：人类定义的推理模式可能会限制模型探索，而无约束的 RL 训练更能激励 LLM 中涌现新的推理能力。通过这一过程（详见第 2 节），我们的模型（称为 DeepSeek-R1-Zero）自然发展出多样且复杂的推理行为。在求解推理问题时，模型表现出生成更长回复的倾向，并在每条回复中融入验证、反思以及对替代方法的探索。尽管我们没有明确教模型如何推理，但它通过强化学习成功学到了更优的推理策略。

尽管 DeepSeek-R1-Zero 展现出卓越的推理能力，但它仍面临可读性差与语言混杂等问题，偶尔会在单条思维链回复中混用英文与中文。此外，DeepSeek-R1-Zero 基于规则的 RL 训练阶段仅聚焦于推理任务，导致其在写作、开放域问答等更广泛领域中的表现有限。为应对这些挑战，我们提出 DeepSeek-R1——一个通过多阶段学习框架训练得到的模型，该框架整合了拒绝采样、强化学习与监督微调（详见第 3 节）。这一训练流程使 DeepSeek-R1 既能继承其前身 DeepSeek-R1-Zero 的推理能力，又能借助额外的非推理数据使模型行为与人类偏好对齐。

为以更低的能耗让更多人获得强大的 AI 能力，我们蒸馏了若干较小的模型并向公众开放。这些蒸馏模型展现出强大的推理能力，超越了其原始指令微调对应版本的表现。我们相信，这些指令微调版本还将为研究社区作出重要贡献：它们为理解长思维链（CoT）推理模型背后的机制提供了宝贵资源，并将促进更强大推理模型的发展。我们在 https://huggingface.co/deepseek-ai 公开发布 DeepSeek-R1 系列模型。

2 DeepSeek-R1-Zero

我们首先详细阐述 DeepSeek-R1-Zero 的训练，该模型完全依赖强化学习，不使用监督微调。为提升大规模 RL 的效率，我们采用组相对策略优化（GRPO）（Shao 等，2024）。

2.1 组相对策略优化

GRPO（Shao 等，2024）是我们用于训练 DeepSeek-R1-Zero 与 DeepSeek-R1 的强化学习算法。它最初被提出用于简化训练过程并降低近端策略优化（PPO）（Schulman 等，2017）的资源消耗，后者广泛用于 LLM 的 RL 阶段（Ouyang 等，2022）。

对于每个问题 q，GRPO 从旧策略 π_θold 中采样一组输出 {o_1, o_2, ⋯, o_G}，然后通过最大化以下目标来优化策略模型 π_θ：

J_GRPO(θ) = E[q∼P(Q), {o_i}_{i=1}^G∼π_θold(O|q)] [(1/G) Σ_{i=1}^G (min((π_θ(o_i|q)/π_θold(o_i|q)) A_i, clip(π_θ(o_i|q)/π_θold(o_i|q), 1−ε, 1+ε) A_i) − β D_KL(π_θ ‖ π_ref))]　　（1）

D_KL(π_θ ‖ π_ref) = π_ref(o_i|q)/π_θ(o_i|q) − log(π_ref(o_i|q)/π_θ(o_i|q)) − 1　　（2）

其中 π_ref 为参考策略，ε 与 β 为超参数，A_i 为优势（advantage），由每组内输出对应的一组奖励 {r_1, r_2, ..., r_G} 计算得到：

A_i = (r_i − mean({r_1, r_2, ⋯, r_G})) / std({r_1, r_2, ⋯, r_G})　　（3）

我们在补充材料 A.3 中给出 GRPO 与 PPO 的对比。为训练 DeepSeek-R1-Zero，我们将学习率设为 3e-6，KL 系数设为 0.001，rollout 的采样温度设为 1。对于每个问题，在 8.2k 步之前我们采样 16 个输出、最大长度为 32,768 token，之后最大长度为 65,536 token。因此，DeepSeek-R1-Zero 的性能与回复长度在 8.2k 步时都出现显著跃升；训练总共持续 10,400 步，对应 1.6 个训练 epoch。每个训练步包含 32 个不同的问题，因此训练批大小为 512。每 400 步，我们用最新的策略模型替换参考模型。为加速训练，每次 rollout 生成 8,192 个输出，随机切分为 16 个 mini-batch，且仅训练一个内部 epoch。

表 1：DeepSeek-R1-Zero 的模板。训练时，prompt 将被替换为具体的推理问题。（原文第 3 页）

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <think>...</think> and <answer>...</answer> tags, respectively, i.e., <think> reasoning process here </think> <answer>answer here</answer>. User: prompt. Assistant:

我们的高性能 RL 基础设施见补充材料 B.1，它保证了可扩展且高效的训练。

2.2 奖励设计

奖励是训练信号的来源，它决定了 RL 优化的方向。对于 DeepSeek-R1-Zero，我们采用基于规则的奖励，为数学、编码与逻辑推理领域的数据提供精确反馈。我们的基于规则的奖励系统主要由两类奖励组成：准确性奖励与格式奖励。

准确性奖励 准确性奖励评估回复是否正确。例如，对于结果确定的数学问题，模型需要以指定格式提供最终答案（例如放在一个方框内），从而能够基于规则可靠地验证正确性。类似地，对于代码竞赛提示，可以利用编译器，根据一组预定义的测试用例评测模型的回复，从而对正确性给出客观反馈。

【图表】图 1 （a）DeepSeek-R1-Zero 训练期间的 AIME 准确率。AIME 以数学问题为输入、以数字为输出，示例如表 32 所示。Pass@1 与 Cons@16 的说明见补充材料 D.1。基线为人类参赛者在 AIME 竞赛中取得的平均分。（b）训练过程中 DeepSeek-R1-Zero 在训练集上的平均回复长度。DeepSeek-R1-Zero 自然学会用更多思考时间来解决推理任务。注意，一个训练步指一次策略更新操作。（原文第 4 页）

格式奖励 格式奖励通过强制特定的格式要求，对准确性奖励模型形成补充。具体而言，模型被激励将其推理过程封装在指定的标签内，即 '<think>' 与 '</think>'。这确保模型的思考过程被明确划分，从而增强可解释性并便于后续分析。

Reward_rule = Reward_acc + Reward_format （4）

准确性奖励与格式奖励以相同权重组合。值得注意的是，我们不将神经奖励模型（无论是基于结果的还是基于过程的）应用于推理任务。这一决定源于我们的观察：神经奖励模型在大规模强化学习过程中容易受到奖励攻击（reward hacking）。此外，重新训练此类模型需要大量计算资源，并会给训练流程引入额外复杂性，从而使整体优化过程更加复杂。

2.3 激励 LLM 的推理能力

具体而言，我们在 DeepSeek-V3 基座模型上应用 RL 技术来训练 DeepSeek-R1-Zero。训练期间，我们设计了一个简洁的模板，要求 DeepSeek-R1-Zero 先产生推理过程，再给出最终答案。我们有意将约束限制在这一结构格式上，避免任何与内容相关的偏见，以确保能够准确观察模型在 RL 过程中的自然演进。

图 1(a) 描绘了 DeepSeek-R1-Zero 在 RL 训练全过程中于 AIME 2024 基准上的性能轨迹，其中 AIME 2024 的平均 pass@1 分数显著上升，从最初的 15.6% 跃升至 77.9%。此外，借助自洽性解码

（Wang 等，2023c），模型性能可进一步提升，达到 86.7% 的准确率。这一表现显著超过了所有人类参赛者的平均表现。除数学竞赛之外，如图 10 所示，DeepSeek-R1-Zero 在编程竞赛以及研究生水平的生物、物理与化学问题上同样表现出色。这些结果彰显了 RL 在增强大语言模型推理能力方面的有效性。

【表格】表 2 DeepSeek-R1-Zero 中间版本中一个有趣的“顿悟时刻”。模型学会以拟人化的口吻重新思考。这对我们而言也是一个顿悟时刻，让我们得以见证强化学习的力量与美感（原文第 5 页）

DeepSeek-R1-Zero 的自我进化例证了 RL 如何能够自主增强模型的推理能力。

如图 1(b) 所示，DeepSeek-R1-Zero 在整个训练过程中思考时间稳步增加，这完全由内在适应驱动，而非外部修改。借助长 CoT，模型逐步改进其推理，生成数百到数千个 token 来探索并改进其问题求解策略。

思考时间的增加促进了复杂行为的自主发展。具体而言，DeepSeek-R1-Zero 日益表现出反思性推理与系统性探索替代解法等高级推理策略（详见补充材料 C.2 的图 9(a)），显著提升了其在数学与编码等可验证任务上的表现。值得注意的是，在训练过程中，DeepSeek-R1-Zero 出现了“顿悟时刻（aha moment）”（表 2），其特征是反思过程中“wait”一词的使用陡然增加（详见补充材料 C.2 的图 9(b)）。这一时刻标志着推理模式的明显变化，清晰展示了 DeepSeek-R1-Zero 的自我进化过程。

DeepSeek-R1-Zero 的自我进化凸显了 RL 的力量与美感：我们并非明确教模型如何解决问题，而只是为其提供正确的激励，它便自主发展出高级的问题求解策略。这也提醒我们 RL 在释放 LLM 更高层次能力方面的潜力，并为未来更自主、更具适应性的模型铺平了道路。

【图表】图 2 DeepSeek-R1 的多阶段流程。关于 DeepSeek-V3 Base 与 DeepSeek-V3 的详细背景见补充材料 A.1。模型 DeepSeek-R1 Dev1、Dev2 与 Dev3 表示该流程中的中间检查点。（原文第 6 页）

3 DeepSeek-R1

尽管 DeepSeek-R1-Zero 展现出强大的推理能力，但它仍存在若干问题。DeepSeek-R1-Zero 面临可读性差与语言混杂等挑战，因为 DeepSeek-V3-Base 是在多种语言（尤其是英文与中文）上训练的。为解决这些问题，我们开发了 DeepSeek-R1，其流程如图 2 所示。

在初始阶段，我们收集数千条冷启动数据，这些数据展现出对话式、与人类对齐的思考过程。随后应用 RL 训练，以提升模型在对话式思考过程与语言一致性方面的表现。接下来，我们再次应用拒

绝采样与 SFT。该阶段将推理数据集与非推理数据集都纳入 SFT 过程，使模型不仅在推理任务上表现出色，还能展现出高级的写作能力。为进一步使模型与人类偏好对齐，我们实施第二个 RL 阶段，旨在提升模型的有用性与无害性，同时改进其推理能力。

本节其余部分详述该流程的关键组件：第 3.1 节介绍我们 RL 阶段所使用的奖励模型，第 3.2 节详细说明具体训练方法与实现细节。此阶段所用数据详见补充材料 B.3。

3.1 基于模型的奖励

对于通用数据，我们借助奖励模型来捕捉复杂而细微场景中的人类偏好。我们在 DeepSeek-V3 流程的基础上构建，并采用相似的偏好对与训练提示分布。对于有用性，我们仅关注最终总结，确保评估强调回复对用户的有用性与相关性，同时尽量减少对底层推理过程的干扰。对于无害性，我们评估模型的完整回复（包括推理过程与总结），以识别并缓解生成过程中可能出现的任何潜在风险、偏见或有害内容。

有用性奖励模型 关于有用性奖励模型的训练，我们首先使用 arena-hard 提示格式（见补充材料 B.2）提示 DeepSeek-V3 生成偏好对，其中每一对包含一个用户查询与两个候选回复。对于每个偏好对，我们向 DeepSeek-V3 查询四次，并随机将回复分配为 Response A 或 Response B，以缓解位置偏差。最终偏好分数由四次独立判断的平均值确定，仅保留分数差（Δ）超过 1 的偏好对，以确保差异有意义。此外，为尽量减少与长度相关的偏差，我们确保整个数据集中被选中与被拒绝的回复长度相当。我们总共构建了 66,000 个数据对用于训练奖励模型。该数据集使用的提示均为非推理问题，来源于公开的开源数据集，或来自明确同意分享其数据用于模型改进的用户。我们奖励模型的架构与 DeepSeek-R1 一致，只是额外增加了一个用于预测标量偏好分数的奖励头。

Reward_helpful = RM_helpful(Response A, Response B)　　（5）

有用性奖励模型以批大小 256、学习率 6e-6 在训练数据集上训练一个 epoch。训练时的最大序列长度设为 8192 token，而在奖励模型推理时未施加显式限制。

安全性奖励模型 为评估并提升模型安全性，我们构建了一个包含 106,000 条提示的数据集，其中的模型生成回复依据预定义的安全准则被标注为“安全”或“不安全”。与有用性奖励模型采用的成对损失不同，安全性奖励模型采用逐点（point-wise）方法训练，以区分安全与不安全的回复。训练超参数与有用性奖励模型相同。

Reward_safety = RM_safety(Response)　　（6）

对于通用查询，每个实例被归类为属于安全性数据集或有用性数据集。分配给每个查询的通用奖励 Reward_General 与相应数据集中定义的奖励相对应。

3.2 训练细节

3.2.1 第一 RL 阶段的训练细节

在 RL 的第一个阶段，我们将学习率设为 3e-6，KL 系数设为 0.001，GRPO 裁剪比例 ε 设为 10，rollout 的采样温度设为 1。对于每个问题，我们采样 16 个输出，最大长度为 32,768。每个训练步包含 32 个不同的问题，因此每步训练批大小为 512。每 400 步，我们用最新的策略模型替换参考模型。为加速训练，每次 rollout 生成 8,192 个输出，随机切分为 16 个 mini-batch，且仅训练一个内部 epoch。然而，为缓解语言混杂问题，我们在 RL 训练中引入语言一致性奖励，其计算方式为 CoT 中目标语言词所占的比例。

Reward_language = Num(Words_target) / Num(Words)　　（7）

尽管补充材料 B.6 中的消融实验表明，这种对齐会导致模型性能略有下降，但该奖励与人类偏好一致，使回复更具可读性。我们直接将语言一致性奖励加到最终奖励中，从而将其应用于推理数据与非推理数据。

请注意，裁剪比例在训练中起着关键作用。较低的值会导致大量 token 的梯度被截断，从而降低模型性能；而较高的值可能导致训练不稳定。

3.2.2 第二 RL 阶段的训练细节

具体而言，我们使用奖励信号与多样化提示分布的组合来训练模型。对于推理数据，我们遵循 DeepSeek-R1-Zero 所述的方法，采用基于规则的奖励来引导数学、编码与逻辑推理领域的学习。在训练过程中，我们观察到 CoT 经常出现语言混杂，尤其是当 RL 提示涉及多种语言时。对于通用数据，我们利用奖励模型来引导训练。最终，将奖励信号与多样化数据分布相结合，使我们能够开发出一个不仅在推理上表现出色，而且兼顾有用性与无害性的模型。给定一批数据，奖励可表述为

Reward = Reward_reasoning + Reward_general + Reward_language　　（8）

其中，Reward_reasoning = Reward_rule　　（9）

Reward_general = Reward_reward_model + Reward_format　　（10）

第二 RL 阶段保留了第一阶段的大部分参数，关键区别是温度降低至 0.7，因为我们发现该阶段较高的温度会导致生成不连贯。该阶段共包含 1,700 个训练步，其中通用指令数据与基于偏好的奖励仅

在最后 400 步引入。我们发现，在基于模型的偏好奖励信号下进行更多训练步可能导致奖励攻击，这一点记录在补充材料 B.5 中。总训练成本列于补充材料 B.4.4。

4 实验

我们在 MMLU（Hendrycks 等，2021）、MMLU-Redux（Gema 等，2025）、MMLU-Pro（Wang 等，2024）、C-Eval（Huang 等，2023）与 CMMLU（Li 等，2024）、IFEval（Zhou 等，2023b）、FRAMES（Krishna 等，2024）、GPQA Diamond（Rein 等，2023）、SimpleQA（OpenAI，2024a）、C-SimpleQA（He 等，2024）、SWE-Bench Verified（OpenAI，2024b）、Aider（Gauthier，2025）、LiveCodeBench（Jain 等，2024）（2024-08 至 2025-01）、Codeforces（Mirzayanov，2025）、中国全国高中数学奥林匹克（CNMO 2024）（CMS，2024）以及 2024 年美国数学邀请赛（AIME 2024）（MAA，2024）上评测我们的模型。

这些基准的详细信息列于补充材料 D。

表 3 汇总了 DeepSeek-R1 在多个发展阶段（如图 2 所示）的性能。DeepSeek-R1-Zero 与 DeepSeek-R1 Dev1 的对比显示，指令遵循能力有大幅提升，这体现在 IF-Eval 与 ArenaHard 基准上更高的分数。然而，由于冷启动数据集规模有限，Dev1 的推理性能相比 DeepSeek-R1-Zero 出现部分退化，最明显的是 AIME 基准。相比之下，DeepSeek-R1 Dev2 表现出，在需要高级推理技能的基准上取得显著性能提升，包括代码生成、数学问题求解与 STEM 相关任务；而面向通用任务的基准（如 AlpacaEval 2.0）仅表现出边际提升。这些结果表明，面向推理的强化学习能够大幅增强推理能力，同时对以用户偏好为导向的基准影响有限。

【表格】表 3 DeepSeek-R1 各阶段的实验结果。加粗数字表示性能在统计上显著（t 检验，$p < 0.01$）（原文第 9 页）

DeepSeek-R1 Dev3 将推理与非推理数据集同时整合进 SFT 流程，从而提升了模型在推理任务与通用语言生成任务上的能力。与 Dev2 相比，DeepSeek-R1 Dev3 在 AlpacaEval 2.0 与 Aider-Polyglot 上取得了显著提升，这归因于大规模非推理语料与代码工程数据集的引入。最后，在 DeepSeek-R1 Dev3 上使用推理与通用混合数据开展全面 RL 训练，得到最终的 DeepSeek-R1。由于前序阶段已完成大量针对推理的 RL，代码与数学基准上的提升较为有限。最终 DeepSeek-R1 的主要进步体现在通用指令遵循与用户偏好基准上，其中 AlpacaEval 2.0 提升 25%，ArenaHard 提升 17%。

此外，我们在附录 D.2 中将 DeepSeek-R1 与其他模型进行比较，模型安全性评估见附录 D.3。附录 E 提供了全面分析，包括与 DeepSeek-V3 的对比、在两个全新测试集上的性能评估、按类别划分的数学能力分析，以及对测试时扩展行为的考察。附录 F 表明，强大的推理能力可以迁移到更小的模型上。

5 伦理与安全声明

随着 DeepSeek-R1 推理能力的提升，我们深刻认识到潜在的伦理风险。例如，R1 可能遭受越狱攻击，从而生成爆炸物制作方案等危险内容；而增强的推理能力又使模型能够给出在操作上更具可行性与可执行性的方案。此外，公开模型也容易受到进一步微调的影响，从而危及固有的安全防护。

在附录 D.3 中，我们从多个角度给出全面的安全报告，包括在开源与内部安全评估基准上的表现，以及跨多种语言、抵御越狱攻击方面的安全水平。这些全面的安全分析表明，与其他最先进模型相比，DeepSeek-R1 模型的固有安全水平总体处于中等水平（与 GPT-4o（2024-05-13）相当）。此外，当配合风险控制系统使用时，模型的安全水平可提升至更优标准。

6 结论、局限与未来工作

我们提出了 DeepSeek-R1-Zero 与 DeepSeek-R1，二者依靠大规模 RL 来激励模型的推理行为。我们的结果表明，预训练检查点本身就蕴含处理复杂推理任务的巨大潜力。我们相信，释放这一潜力的关键不在于大规模人工标注，而在于提供困难的推理问题、可靠的验证器，以及充足的计算资源用于强化学习。自我验证与反思等复杂的推理行为，似乎在强化学习过程中自发涌现。

即便 DeepSeek-R1 在推理基准上取得了前沿结果，它仍面临若干能力局限，概述如下：

结构化输出与工具使用：目前，DeepSeek-R1 的结构化输出能力相较现有模型仍不够理想。此外，DeepSeek-R1 无法利用搜索引擎、计算器等工具来提升输出性能。不过，由于为结构化输出与工具使用构建 RL 环境并不困难，我们相信该问题将在下一版本中得到解决。

token 效率：与多数投票或蒙特卡洛树搜索（MCTS）等传统的测试时计算扩展方法不同，DeepSeek-R1 会在推理时根据问题当前的复杂度动态分配计算资源。具体而言，它在求解简单任务时使用更少的 token，而在复杂任务上生成更多 token。尽管如此，token 效率仍有进一步优化的空间，因为对于较简单的问题，仍能观察到推理过度（表现为过度思考）的情况。

语言混用：DeepSeek-R1 目前针对中文与英文进行优化，在处理其他语言的查询时可能出现语言混用问题。例如，即使查询并非中文或英文，DeepSeek-R1 也可能用英文进行推理与回复。我们计划在未来的更新中解决这一局限。该局限可能与基础检查点 DeepSeek-V3-Base 主要使用中文与英文有关，因此它在两种语言上的推理效果更好。

提示工程：在评估 DeepSeek-R1 时，我们观察到它对提示词较为敏感，少样本提示会持续降低其性能。因此，我们建议用户直接描述问题，并以零样本方式指定输出格式，以获得最佳结果。

软件工程任务：由于评估耗时较长，影响 RL 过程的效率，大规模 RL 尚未在软件工程任务中广泛应用。因此，DeepSeek-R1 在软件工程基准上尚未展现出相对 DeepSeek-V3 的巨大提升。未来版本将通过在软件工程数据上实施拒绝采样，或在 RL 过程中引入异步评估以提升效率来解决这一问题。

除了具体的能力局限之外，纯 RL 方法本身也面临固有挑战：

奖励攻击：纯 RL 的成功依赖于可靠的奖励信号。在本研究中，我们通过推理领域基于规则的奖励模型（RM）来确保奖励的可靠性。然而，对于写作等某些任务，此类可靠的 RM 难以构建。如果奖励信号由模型而非预定义规则给出，它就会随着训练推进而更容易被利用，这意味着策略模型可能找到“捷径”来攻击奖励模型。因此，对于无法通过可靠奖励模型有效评估的复杂任务，扩展纯 RL 方法仍是一个开放挑战。

在本工作中，对于无法获得可靠信号的任务，DeepSeek-R1 使用人工标注来构建监督数据，并且仅进行数百步的 RL。我们希望未来能够获得稳健的奖励模型来解决此类问题。

随着 DeepSeek-R1 等纯 RL 方法的出现，未来在解决任何能够被验证器有效评估的任务方面都具有巨大潜力，无论这些任务对人类而言多么复杂。配备此类先进 RL 技术的机器有望在这些领域超越人类能力，因为它们能够通过试错迭代地优化性能。然而，对于构建可靠奖励模型本身就很困难的任务，挑战依然存在。在这种情况下，缺乏稳健的反馈机制可能阻碍进展，这表明未来研究应聚焦于开发创新方法，为这些复杂且难以验证的问题定义并改进奖励结构。

此外，在推理过程中利用工具具有重大前景。无论是利用编译器或搜索引擎等工具来检索或计算必要信息，还是采用生物或化学试剂等外部工具在真实世界中验证最终结果，这种工具增强的推理集成都可能极大拓展机器驱动解决方案的范围与准确性。

7 作者列表

作者列表按贡献角色组织，每个类别内按名字首字母排序。标注星号（*）的作者已不再隶属我们的团队。

核心贡献者：Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z.F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao,

核心作者的贡献：Peiyi Wang 与 Daya Guo 共同证明了基于结果的 RL 能够促使 LLM 涌现出长思维链模式，从而实现推理能力的突破。他们在 R1-Zero 的创建中做出了同等贡献，其工作为 R1 奠定了基础。Daya Guo 还为 MoE 模型的 RL 训练稳定性做出了贡献。Junxiao Song 提出了 GRPO 算法，实现了初始版本，并为数学任务引入了基于规则的奖励；GRPO 算法随后由 Peiyi Wang 与 Runxin Xu 进一步改进。Zhibin Gou 提出了大幅 PPO 裁剪策略以提升 GRPO 性能，并与 Zhihong Shao、Junxiao Song 共同证明了其重要性。在数据迭代、奖励设计与评估方面，各专项团队主导了不同领域的工作：Qihao Zhu、Z.F. Wu 与 Dejian Yang 负责代码任务；Zhihong Shao、Zhibin Gou 与 Junxiao Song 负责数学任务；Peiyi Wang、Ruoyu Zhang、Runxin Xu 与 Yu Wu 负责其他推理与通用任务。此外，Qihao Zhu 与 Zhihong Shao 为 RL 训练的数据选择策略做出了贡献，Zhuoshu Li 与 Yu Wu 共同主导了整个项目的数据标注工作。在系统方面，Xiao Bi、Xingkai Yu、Shirong Ma、Xiaokang Zhang、Haowei Zhang 与 Ziyi Gao 实现了 RL 流水线，优化系统效率并解决大规模训练中的稳定性问题。最后，Zhibin Gou、Daya Guo 与 Ruoyu Zhang 负责最终训练阶段并监控模型训练动态；Zhibin Gou 主导了 R1-distill 系列的研发。

贡献者：Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo*, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J.L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu*, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R.J. Chen, R.L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S.S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu*, Wentao Zhang, W.L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng,

Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X.Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y.K. Li, Y.Q. Wang, Y.X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma*, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y.X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z.Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu*, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, Zhen Zhang,

附录

A 背景

A.1 DeepSeek-V3

DeepSeek V3（DeepSeek-AI，2024b）是 DeepSeek 开发的一款先进开源 LLM，于 2024 年 12 月发布，代表了 AI 创新的重大飞跃，旨在与 OpenAI 的 GPT-4、Meta 的 Llama 3.1 等领先模型竞争，同时保持显著的成本效率与性能。DeepSeek V3 基于混合专家（MoE）架构构建，总参数量为 6710 亿，每个 token 激活 370 亿，兼顾效率与能力。它在 14.8 万亿高质量、多样化 token 组成的大规模数据集上预训练，随后经过监督微调与强化学习，以增强其在各领域的能力。该模型采用了多项创新特性：用于高效推理的多头潜在注意力（MLA）（DeepSeek-AI，2024a）、无辅助损失的负载均衡策略，以及用于提升性能（尤其是在数学与编程等任务上）的多 token 预测（MTP）（Gloeckle 等，2024）。

对于 DeepSeek-V3-Base 的训练数据，我们仅使用普通网页与电子书，未纳入任何合成数据。然而，我们观察到部分网页包含大量由 OpenAI 模型生成的答案，这可能使基座模型间接地从其他强大模型中获取知识。不过，我们并未有意将 OpenAI 生成的合成数据纳入预训练冷却阶段；该阶段使用的所有数据均为自然出现并通过网页抓取收集的数据。预训练数据集包含大量数学与代码相关内容，表明 DeepSeek-V3-Base 已接触过大量推理轨迹数据。这种广泛接触使模型具备了生成合理解法候选的能力，强化学习可据此有效识别并优化高质量输出。我们按附录 D.1 所述开展了预训练数据污染分析。DeepSeek-V3 基座模型的训练数据主要是中文与英文，这可能是 DeepSeek-R1-Zero 在缺少语言一致性奖励时出现语言混用的原因。

在本文中，我们使用 DeepSeek-V3-Base 这一记号表示基座模型，DeepSeek-V3 表示指令模型。值得注意的是，DeepSeek-R1 与 DeepSeek-R1-Zero 均在 DeepSeek-V3-Base 之上训练，且 DeepSeek-R1 利用了来自 DeepSeek-V3 SFT 数据的非推理数据。DeepSeek-R1-Dev1、DeepSeek-R1-Dev2、DeepSeek-R1-Dev3 是 DeepSeek-R1 的中间检查点。

A.2 传统的后训练范式

后训练已成为精炼预训练 LLM、使其满足特定性能目标并与人类期望对齐的重要步骤。一种被广泛采用的两阶段后训练框架是 SFT 后接 RL（Ouyang 等，2022）。

监督微调通过在一套针对特定任务精心策划的输入-输出对数据集上训练，来精炼预训练 LLM。该过程采用监督学习目标，通常最小化模型预测与标注真值之间的交叉熵损失（Brown 等，2020）。例如，在对话应用中，SFT 可能使用明确给出期望回复的对话数据集，使模型能够将其输出调整到预定义标准（Radford 等，2019）。SFT 有几项显著优势。第一，它利用高质量示例实现精确的任务对齐，使模型在客户支持或技术文档等领域表现出色（Radford 等，2019）。第二，它依赖预训练权重，保证了计算效率，所需资源少于从头训练。最后，显式的输入-输出映射提升了可解释性，因为模型的学习过程直接与可观察数据绑定，降低了行为异常的风险（Ouyang 等，2022）。尽管有其优势，SFT 的性能取决于训练数据集的质量与多样性；狭窄或有偏的数据会损害模型向新情境泛化的能力（Brown 等，2020）。此外，SFT 的静态特性——针对固定输出进行优化——可能无法捕捉不断演变的人类偏好或细微目标。策划高质量数据集的过程劳动密集，进一步增加了其可扩展性的难度，因为数据中的错误或不一致会传导到模型行为中（Ouyang 等，2022）。

在 SFT 之后，强化学习通过依据奖励信号优化输出，进一步精炼 LLM。在此阶段，模型与环境交互——环境通常是一个基于人类反馈训练的奖励模型——并调整其行为以最大化累积奖励。该方法的一个突出实例是基于人类反馈的强化学习（RLHF），其中奖励函数编码了人类偏好（Christiano 等，2017）。因此，RL 将重点从静态监督转向动态优化。值得注意的是，RL 减少了对大量标注资源的需求：SFT 要求为每个输入-输出对提供完整标注的数据集，而 RL 只需少量人类评估或一个训练好的奖励模型（甚至是基于规则的奖励模型）即可运行，显著降低了标注负担。

SFT 与 RL 的依次应用结合了二者互补的优势。SFT 通过将模型扎根于精心策划的示例，建立起稳健的、任务特定的基线；而 RL 则精炼这一基础，使其与更广泛、以人为中心的目标对齐（Ouyang 等，2022）。例如，SFT 可能确保对话系统在语法上的准确性，而 RL 则优化参与度与简洁性，正如 InstructGPT 的开发所示（Ouyang 等，2022）。这种混合方法已被证明能够有效产出既精确又具适应性的模型。

在本研究中，我们证明 SFT 阶段可能阻碍模型探索并发展有效的推理策略。这一局限源于作为 SFT 目标的人类提供回复并不总是最优的学习目标：它们往往缺少显式反思与验证等关键推理要素。为此，DeepSeek-R1-Zero 使模型能够独立于人类先验，直接探索推理模式。通过这种自我探索发现的推理轨迹随后被蒸馏并用于训练其他模型，从而促进获得更稳健、更可泛化的推理能力。

A.3 GRPO 与 PPO 的比较

组相对策略优化（GRPO）（Shao 等，2024）是我们用于训练 DeepSeek-R1-Zero 与 DeepSeek-R1 的强化学习算法。它最初的提出是为了简化训练过程并降低近端策略优化（PPO）（Schulman 等，2017）的资源消耗，后者被广泛用于 LLM 的 RL 阶段（Ouyang 等，2022）。GRPO 与 PPO 的总体对比见图 3。

对于每个问题 q，GRPO 从旧策略 π_θold 中采样一组输出 {o_1, o_2, …, o_G}，然后通过最大化如下目标来优化策略模型 π_θ：

【图表】图 3 PPO 与本文 GRPO 的对比示意。GRPO 舍弃了价值模型，转而从组内得分估计优势值（原文第 15 页）

$$J_{GRPO}(\theta) = \mathbb{E}[q\sim P(Q), \{o_i\}_{i=1}^{G}\sim\pi_{\theta old}(O|q)] \frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta old}(o_i|q)}\cdot A_i,\ \mathrm{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta old}(o_i|q)},\ 1-\varepsilon,\ 1+\varepsilon\right)\cdot A_i\right) - \beta\cdot D_{KL}[\pi_\theta\|\pi_{ref}]\right) \quad (11)$$

$$D_{KL}[\pi_\theta\|\pi_{ref}] = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\left(\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)}\right) - 1 \quad (12)$$

其中 π_ref 是参考策略，ε 与 β 是超参数，A_i 是优势值，使用与组内各输出对应的奖励 {r_1, r_2, …, r_G} 计算：

$$A_i = \left(r_i - \mathrm{mean}(\{r_1, r_2, \ldots, r_G\})\right) / \mathrm{std}(\{r_1, r_2, \ldots, r_G\}) \quad (13)$$

相比之下，在 PPO 中，优势值通常通过应用广义优势估计（GAE）（Schulman 等，2015）计算，它不仅基于奖励，还基于一个学习到的价值模型。由于价值模型通常与策略模型规模相当，它会带来显著的内存与计算开销。此外，价值模型的目标是根据从开头到当前位置已生成的 token，预测从当前位置往后的期望累积奖励。这本身就很困难，尤其是在只有最终结果奖励可用时。在训练长思维链推理模型时，这一挑战更为突出。随着输出长度增加，模型在生成过程中更可能进行反思与修正等行为，这意味着最初生成的内容可能在之后被修改或否定，从而使得基于部分回复预测最终奖励更加不可行。

GRPO 与 PPO 的另一个关键区别在于，训练策略与参考策略之间的 Kullback–Leibler（KL）散度如何纳入训练过程。在 GRPO 中，KL 散度的无偏估计（Schulman，2020）被直接加入损失，如式 11 所示；而在 PPO 中，逐 token 的 KL 惩罚作为稠密奖励在每个 token 处加入（Ouyang 等，2022）。由于强化学习的优化目标是最大化累积奖励。

PPO 的做法惩罚累积 KL 散度，这可能隐含地惩罚响应的长度，从而阻止模型响应长度的增加。此外，在训练长思维链推理模型的场景下，我们可能要训练数千步，训练后的策略可能与初始参考策略显著偏离。为了平衡训练策略可探索的范围与训练的稳定性，我们在实际训练过程中定期将参考策略更新为最新策略。

图 4 比较了 PPO 与 GRPO 在 MATH 任务上的表现，实验使用 DeepSeek-Coder-V2-Lite（16B MoE，激活参数 2.4B）。与 GRPO 不同，PPO 需要额外的超参数调优——尤其是 GAE 中的 λ 系数——并且对该参数高度敏感。当 λ 设为 0.95（多数开源 PPO 实现的默认值）时，PPO 的表现明显逊于 GRPO。然而，经过仔细调优（将 λ 设为 1.0）后，PPO 的性能大幅提升，接近 GRPO 的水平。

【图表】图 4 PPO 与 GRPO 在 MATH 任务上的表现（原文第 16 页）

尽管 PPO 在适当调优后可以达到相当的性能，但它需要额外的计算成本来进行超参数优化。此外，考虑到训练额外的价值模型（value model）所带来的内存与计算开销，GRPO 是更实用的替代方案，尤其是在资源受限的条件下训练大规模模型时。

B 训练细节

B.1 强化学习基础设施

在大模型上开展强化学习训练对基础设施提出了很高要求。我们的 RL 框架采用解耦且可扩展的结构设计，以便无缝集成多种模型与算法。在该框架内，我们同时引入了模块内与模块间的优化技术，以确保训练效率与可扩展性。

具体而言，如图 5 所示，该框架被划分为四个独立模块，每个模块对应 RL 流水线中的一个特定阶段：

【图表】图 5 我们的强化学习框架概览（原文第 17 页）

- **Rollout 模块**：提示从训练数据集中加载，并均匀分发到多个 vLLM（Kwon 等，2023）worker，每个 worker 都配备了 actor 模型，用于采样多条响应。针对 DeepSeek-V3 的 MoE 架构，我们实现了跨节点的专家并行（expert parallelism）策略以减少内存访问开销，并部署热点专家的冗余副本，以平衡不同专家之间的计算负载。此外，我们还利用多 token 预测（Multi-Token Prediction，MTP）组件进行自推测解码（self-speculative decoding），显著加快解码速度，并有效缩短最长样本的完成时间。
- **Inference 模块**：该模块加载奖励模型与参考模型，对 rollout 阶段生成的样本执行前向计算，从而获得基于模型的奖励以及其他必要信息。
- **基于规则的奖励模块**：该模块为模型生成的响应计算基于规则的奖励。我们设计了统一接口，以适配多种实现（例如代码执行器（code executor）、答案匹配器（answer matcher）、格式检查器（format checker）等）。尽管该模块无需将模型加载到 GPU 显存中，但其执行往往较为耗时。为解决这一问题，我们采用异步调度方式，使其执行与 Rollout 模块和 Inference 模块重叠，从而有效隐藏相关延迟。
- **训练模块**：该模块加载 actor 模型与 critic 模型（如有需要），以计算损失并更新模型参数。它灵活支持多种 RL 算法（例如 PPO、GRPO、DPO 等）。为尽量减少序列填充（padding）造成的计算浪费并平衡各设备间的工作负载，我们设计了如下数据打包策略：首先，将全局批次（global batch）中的所有数据按长度排序，并分发到数据并行组内的各个进程；随后，在每个进程内应用 Best-Fit 策略，将数据打包成固定长度的块，使填充最少；最后，调整块的数量，使其在所有进程之间相等。此外，我们集成了 DeepSeek-V3 训练中使用的 DualPipe 算法，以实现高效的流水线并行。

值得注意的是，每个模块（基于规则的奖励模块除外）执行完毕后，该阶段使用的模型实例会自动从显存（VRAM）卸载到系统内存或磁盘存储，从而为后续阶段释放显存。

B.2 奖励模型提示

请扮演一名公正的评判者，评估两个 AI 助手针对下方用户提示所给出的响应质量。你将获得助手 A 的答案与助手 B 的答案，你的任务是评判哪位助手的答案更好。

在评估开始时，请先针对该提示生成你自己的答案。你必须在评判任何答案之前给出自己的答案。

在评估助手答案时，请将两个助手的答案与你的答案进行比较。你必须指出并纠正其中的任何错误或不准确的信息。

然后考虑助手的答案是否有用、相关且简洁。有用是指答案正确回应了提示或遵循了指令。请注意，当用户提示存在歧义或多种解释时，向用户请求澄清或提供更多信息，比基于假设给出答案更有用、更恰当。相关是指响应的所有部分都与所问内容紧密相关或恰如其分。简洁是指响应清晰，不冗长、不过度。

随后，在需要时考虑助手答案的创造性与新颖性。最后，指出助手答案中遗漏的任何重要信息，这些信息若在回应用户提示时包含进来会有所帮助。

在给出解释之后，你必须只输出以下选项之一作为最终评判，并附上标签：

1. 助手 A 明显更好：[[A≫B]]2. 助手 A 稍好：[[A>B]]3. 平局，二者大致相同：[[A=B]]4. 助手 B 稍好：[[B>A]]5. 助手 B 明显更好：[[B≫A]]

示例输出：“My final verdict is tie: [[A=B]]”（我的最终评判是平局：[[A=B]]）。

B.3 数据配方

B.3.1 RL 数据

推理类 RL 数据包括四个类别：数学、代码、STEM 与逻辑问题。此外，我们还在 DeepSeek-R1 的训练中加入了通用 RL 数据，以提升模型的有用性与无害性。所有问题均为中文或英文。RL 数据的说明见表 4，下面我们将逐一描述各类数据的细节：

【表格】表 4 RL 数据与任务说明（原文第 19 页）

- **数学数据集**由 26k 道定量推理题组成，包括数学考试题与竞赛题，提示的平均 token 数为 122。该数据集覆盖代数、微积分、概率与几何等多个数学领域，题目难度从地区性竞赛到国际奥赛不等。对于每道题，模型需要产出逐步推理过程，并最终给出答案——答案可以是数值（如“5”）、数学表达式（如“$x^2 + 3x - 2$”）或方程（如“$y = 2x + 1$”）。数学证明类题目被排除在外，因为其正确性难以判定。为强化学习之用，我们通过将预测答案与参考答案进行匹配来计算推理过程的奖励：若答案与参考答案一致，奖励赋值为 1，否则为 0。
- **代码数据集**包含 17k 道算法竞赛题以及 8k 道缺陷修复（bug fixing）问题。算法竞赛题类似于 Codeforces 或 LeetCode 等平台上的题目，每道题通常包含详细的题目描述、约束条件以及多个输

入输出样例。任务是编写一个完整的函数或程序，能够正确且高效地求解该问题，并通过一整套隐藏测试用例，这些用例同时考察正确性与性能。此类问题考察动态规划、图论、字符串处理与数据结构使用等算法技能。缺陷修复问题从真实的 GitHub issue 中提取。每个任务提供 issue 描述、存在缺陷的源代码版本，以及一组部分或全部失败的单元测试。目标是理解该 issue 的意图，定位并修复代码中的缺陷，并确保修正后的版本通过全部单元测试。

- **STEM 数据集**由 22k 道选择题组成，涵盖物理、化学与生物等主题。STEM 任务中的每道题给出一个学科特定问题，并附 4 到 8 个备选答案，模型需要依据给定上下文与领域知识选出最符合科学事实的答案。提示的平均 token 数为 161。具体而言，该数据集包含 15.5% 物理、30.7% 生物、46.5% 化学，以及 7.3% 的健康与医学等其他主题。由于所有 STEM 题目均为选择题，奖励根据是否匹配正确选项以二值方式给出。
- **逻辑数据集**包含 15k 道题目，旨在评估模型在广泛的逻辑挑战上的推理能力。该数据集既包含真实世界问题，也包含合成生成的问题。所有问题都支持自动评估，提示的平均长度约为 420 token。数据集的真实世界部分由从网络采集的多样化问题构成，包括脑筋急转弯、经典逻辑谜题与知识密集型问题；这些问题以选择题形式呈现，以确保评估客观且一致。合成部分主要包含两类：代码-IO（code-IO）问题与谜题任务。代码-IO 问题使用 Li 等（2025）提出的数据流水线生成，该流水线将竞赛编程问题及其对应的输入输出测试用例转换为可验证的逻辑推理问题。谜题任务包括旨在考察特定推理能力的问题。例如，密码学谜题用于评估模型识别并应用密码方案模式或执行字符串操作的能力；逻辑谜题侧重于在复杂约束下的演绎推理，例如从一组固定前提中推断有效结论（如斑马谜题）；算术谜题则考察模型的数值推理能力（如概率问题与 24 点游戏）。
- **通用数据集**由 66k 道问题组成，旨在评估有用性，涵盖创意写作、编辑、事实问答与角色扮演等多个类别。此外，该数据集还包含 12,000 道聚焦于评估无害性的问题。为确保稳健的验证，我们使用两个奖励模型，它们分别基于针对有用性与无害性的、由模型生成的排序响应数据集训练而成。在训练阶段，我们对有用性奖励模型训练了一个 epoch，最大序列长度为 8192 token。然而，在部署该模型以生成奖励信号时，我们并未对被评估的输入序列施加任何显式长度约束。

B.3.2 DeepSeek-R1 冷启动

对于 DeepSeek-R1，我们构建并收集了少量长思维链（CoT）数据来微调模型，作为 RL 的初始 actor。这一动机主要是产品驱动的，重点在于提升用户体验。当推理过程与第一人称视角的思维模式一致时，用户往往觉得响应更直观、更有吸引力。例如，DeepSeek-R1-Zero 在解题过程中更可能使用

代词“我们”，或完全避免使用第一人称代词，而 DeepSeek-R1 则更频繁地使用“我”。此外，我们认识到这类模式可能引发用户不当的信任。在此我们想强调，所观察到的生动推理模式主要反映的是 DeepSeek 工程化的启发式设计，而非表明模型本身已获得类人的智能或自主解题能力。

在冷启动数据构建中，我们偏好那种先理解问题、随后进行包含反思与验证的详细推理的思考过程，整个思考过程以第一人称视角呈现。此外，保持语言一致性对最佳用户体验至关重要。若缺乏适当控制，无论查询使用何种语言，模型响应都可能混杂多种语言；这种不一致会干扰理解并降低用户满意度。因此，必须仔细精炼，以确保响应保持连贯并与用户意图一致。尽管如此，我们认识到 DeepSeek-R1-Zero 产生的原始思维链（Chain-of-Thought，CoT）推理可能具有超出当前人类先验局限的潜力。具体而言，我们首先让人类标注员将推理轨迹改写为更自然、更接近人类对话风格的表达。修改后的数据对随后被用作示例，提示 LLM 以类似风格改写更多数据。所有 LLM 生成的输出随后都要经过第二轮人工校验，以确保质量与一致性。

清单 1 | 生成人类可读解答的提示

问题

{question}

思考过程

{thought_process}

---基于上述思考过程，为该问题提供一个清晰、易于理解且格式良好的解答。使用与问题相同的语言。

解答必须严格遵循以下要求：

- 与给定的思考过程保持忠实一致。不要添加原思考过程中未出现的新推理步骤或结论。
- 用清晰、格式良好的 LaTeX 展示通向最终答案的关键步骤。
- 最终答案使用 \boxed{} 的格式。
- 保持干净简洁。避免口语化表达。不要在解答中使用“thought process”（思考过程）之类的措辞。

你的响应应直接从解答开始，不要包含任何其他内容。你的任务只是根据所提供的思考过程撰写解答。不要试图自己求解该问题。

具体而言，我们首先收集数千个高质量、多样化的推理提示。对每个提示，我们使用 DeepSeek-R1-Zero 以相对较高的温度 1.0 生成多条推理轨迹。接着，我们对这些生成结果进行筛选，只保留最终答案正确且格式可读的样本。对于数学输出，我们使用 sympy（https://www.sympy.org/）进行解析与

表达式比较；对于格式，我们采用重复检测、语言混杂过滤等规则。最后，我们提示 DeepSeek-V3 对推理过程与总结进行精炼，以确保格式规范且表达符合人类习惯。特别是为解决语言混杂问题，我们指示 DeepSeek-V3 “Translate the thinking process to the same language as the question.”（将思考过程翻译为与问题相同的语言）。由于 DeepSeek-R1-Zero 的总结仅提供最终答案，我们使用清单 1 中的总结提示来生成简洁、人类可读的解答，既概述推理步骤也给出最终结果。

对于代码数据，我们收集了大量竞赛编程题目。具体而言，我们从多个在线评测（online judge，OJ）平台汇编了一个规模可观的竞赛编程题目集合，其中来自 Codeforces 的题目有 5151 道，来自 AtCoder 的有 2504 道。由于这些平台的原始测试用例并不公开，我们开发了一套方法为每道题构造可靠的测试用例。

我们的做法是使用 DeepSeek-V2.5 生成候选测试用例，随后进行严格的验证。具体而言，我们提示 DeepSeek-V2.5 编写 Python 程序，针对每道题的要求生成测试用例，如清单 2 所示。

在获得大量候选测试用例后，我们实施了一个两阶段过滤流程。首先，我们使用正确的提交来剔除那些会产生错误输出的无效测试用例。然后，我们策略性地挑选那些能够成功识别出错误提交中缺陷的测试用例子集。这一过程确保我们最终的测试用例能够正确区分每道题的正确解与错误解。

清单 2 | 为代码问题生成测试用例的提示

`<USER>`

问题：

给定一个字符串 word 和一个整数 k。

word 的子串 s 是“完整”的，当且仅当：

- s 中的每个字符恰好出现 k 次。
- 任意两个相邻字符之差至多为 2，即对于 s 中任意两个相邻字符 c1 和 c2，它们在字母表中位置的绝对差至多为 2。

返回 word 的完整子串数量。

子串是字符串中一段非空的连续字符序列。

输入

第一行包含一个整数 t，表示测试用例数。

每个测试用例的第一行包含一个字符串 word。

每个测试用例的第二行包含一个整数 k。

保证 word 仅由小写英文字母组成，且所有测试用例中 |word| 之和不超过 10^5。

输出

对每个测试用例，输出一个整数，表示 word 的完整子串数量。

示例

输入

```
2
igigee
2
aaabbbccc
3
```

输出

```
3
6
```

构造若干随机输入生成器，生成多样且对抗性的输入，其规模要足够大，以使错误代码超出时间限制。

`</USER>`

`<ASSISTANT>`

首先为大数据构造一个随机输入生成器，包含 10 个长度相同的测试用例。

```
import random
import string

def generate_random_string(l_min, l_max):
    length = random.randint(l_min, l_max)
    seq = ''.join(random.choices(string.ascii_lowercase, k=length))
    k = random.randint(1, length)
```

附录

附录正文（A 背景、B 训练细节等）已随正文译出；其余补充材料（第 23–78 页提示词与数据表）见英文原文

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] AI@Meta. Llama 3.1 model card, 2024. URL https://github.com/meta-llama/llama-models/blob/main/models/llama3_1/MODEL_CARD.md .

[2] E. Akyürek, M. Damani, L. Qiu, H. Guo, Y. Kim, and J. Andreas. The surprising effectiveness of test-time training for abstract reasoning. arXiv preprint arXiv:2411.07279, 2024.

[3] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. arXiv preprint arXiv:2204.05862, 2022.

[4] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V . Le, C. Ré, and A. Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. arXiv preprint arXiv:2407.21787, 2024.

[5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P . Dhariwal, A. Neelakantan, P . Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, editors, Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4 967418bfb8ac142f64a-Abstract.html .

[6] D. Busbridge, A. Shidani, F. Weers, J. Ramapuram, E. Littwin, and R. Webb. Distillation scaling laws. arXiv preprint arXiv:2502.08606, 2025.

[7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374 .

[8] Z. Chen, Y. Min, B. Zhang, J. Chen, J. Jiang, D. Cheng, W. X. Zhao, Z. Liu, X. Miao, Y. Lu, et al. An empirical study on eliciting and improving r1-like reasoning models. arXiv preprint arXiv:2503.04548, 2025.

[9] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez, et al. Chatbot arena: An open platform for evaluating llms by human preference. arXiv preprint arXiv:2403.04132, 2024.

[10] P . F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In I. Guyon, U. von Luxburg, S. Bengio, H. M. Wallach, R. Fergus, S. V . N. Vishwanathan, and R. Garnett, editors, Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA, pages 4299–4307, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/d5e2c0adad503c91f91df240d0cd4e49-Abstract.html . H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, A. Webson, S. S. Gu, Z. Dai, M. Suzgun, X. Chen, A. Chowdhery, A. Castro-Ros, M. Pellat, K. Robinson, D. Valter, S. Narang, G. Mishra, A. Yu, V . Y. Zhao, Y. Huang, A. M. Dai, H. Yu, S. Petrov, E. H. Chi, J. Dean, J. Devlin, A. Roberts, D. Zhou, Q. V . Le, and J. Wei. Scaling instruction-finetuned language models. J. Mach. Learn. Res., 25:70:1–70:53, 2024. URL https://jmlr.org/papers/v25/23-0870.html .

[11] CMS. Chinese national high school mathematics olympiad, 2024. URL https://www.cms.or g.cn/Home/comp/comp/cid/12.html .

[12] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[13] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024a. URL https://doi.org/10.48550/arXiv.2405. 04434 .

[14] DeepSeek-AI. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024b.

[15] H. Face. Open r1: A fully open reproduction of deepseek-r1, January 2025. URL https: //github.com/huggingface/open-r1 .

[16] X. Feng, Z. Wan, M. Wen, S. M. McAleer, Y. Wen, W. Zhang, and J. Wang. Alphazero-like tree-search can guide large language model decoding and training, 2024. URL https: //arxiv.org/abs/2309.17179 .

[17] D. Ganguli, L. Lovitt, J. Kernion, A. Askell, Y. Bai, S. Kadavath, B. Mann, E. Perez, N. Schiefer, K. Ndousse, A. Jones, S. Bowman, A. Chen, T. Conerly, N. DasSarma, D. Drain, N. Elhage, S. E. Showk, S. Fort, Z. Hatfield-Dodds, T. Henighan, D. Hernandez, T. Hume, J. Jacobson, S. Johnston, S. Kravec, C. Olsson, S. Ringer, E. Tran-Johnson, D. Amodei, T. Brown, N. Joseph, S. McCandlish, C. Olah, J. Kaplan, and J. Clark. Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned. CoRR, abs/2209.07858, 2022.

[18] L. Gao, J. Schulman, and J. Hilton. Scaling laws for reward model overoptimization, 2022. URL https://arxiv.org/abs/2210.10760 .

[19] P . Gauthier. Aider LLM leaderboard, 2025. URL https://aider.chat/docs/leaderboar ds/ .

[20] J. Geiping, S. McLeish, N. Jain, J. Kirchenbauer, S. Singh, B. R. Bartoldson, B. Kailkhura, A. Bhatele, and T. Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. arXiv preprint arXiv:2502.05171, 2025.

[21] A. P . Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P . Minervini. Are we done with mmlu? In L. Chiruzzo, A. Ritter, and L. Wang, editors, Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025, pages 5069–5096. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025. naacl-long.262/ . F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=pEWAcejiU2 .

[22] Z. Gou, Z. Shao, Y. Gong, yelong shen, Y. Yang, N. Duan, and W. Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In The Twelfth International Conference on Learning Representations, 2024a. URL https://openreview.net/forum ?id=Sx038qxjek .

[23] Z. Gou, Z. Shao, Y. Gong, yelong shen, Y. Yang, M. Huang, N. Duan, and W. Chen. ToRA: A toolintegrated reasoning agent for mathematical problem solving. In The Twelfth International Conference on Learning Representations, 2024b. URL https://openreview.net/forum ?id=Ep0TtjVoap .

[24] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu. Reasoning with language model is planning with world model. In The 2023 Conference on Empirical Methods in Natural Language Processing, 2023. URL https://openreview.net/forum?id=VTWWvYtF1R .

[25] Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. arXiv preprint arXiv:2411.07140, 2024.

[26] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021. OpenReview.net, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ .

[27] G. E. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. CoRR, abs/1503.02531, 2015. URL http://arxiv.org/abs/1503.02531 .

[28] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, Y. Fu, M. Sun, and J. He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/c6ec1844bec 96d6d32ae95ae694e23d8-Abstract-Datasets_and_Benchmarks.html .

[29] N. Jain, K. Han, A. Gu, W. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. CoRR, abs/2403.07974, 2024. URL https://doi.org/10.48550/arXiv.2403.07974 .

[30] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361, 2020.

[31] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. In A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, editors, Advances in Neural Information Processing Systems, 2022. URL https://openreview.net/forum?id=e2TB b5y0yFf . S. Krishna, K. Krishna, A. Mohananey, S. Schwarcz, A. Stambler, S. Upadhyay, and M. Faruqui. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. CoRR, abs/2409.12941, 2024. doi: 10.48550/ARXIV.2409.12941. URL https://doi.org/10.485 50/arXiv.2409.12941 .

[32] A. Kumar, V . Zhuang, R. Agarwal, Y. Su, J. D. Co-Reyes, A. Singh, K. Baumli, S. Iqbal, C. Bishop, R. Roelofs, et al. Training language models to self-correct via reinforcement learning. arXiv preprint arXiv:2409.12917, 2024.

[33] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles, 2023.

[34] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: measuring massive multitask language understanding in chinese. In L. Ku, A. Martins, and V . Srikumar, editors, Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024, pages 11260–11285. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.FINDINGS-ACL.671. URL https://doi.org/10.18653/v1/2024.findings-acl.671 .

[35] J. Li, D. Guo, D. Yang, R. Xu, Y. Wu, and J. He. Codei/o: Condensing reasoning patterns via code input-output prediction. arXiv preprint arXiv:2502.07316, 2025.

[36] H. Lightman, V . Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=v8L0pN6EOi .

[37] B. Y. Lin. ZeroEval: A Unified Framework for Evaluating Language Models, July 2024. URL https://github.com/WildEval/ZeroEval .

[38] Z. Liu, C. Chen, W. Li, T. Pang, C. Du, and M. Lin. There may not be aha moment in r1-zero-like training — a pilot study. https://oatllm.notion.site/oat-zero , 2025. Notion Blog.

[39] MAA. American invitational mathematics examination - aime. In American Invitational Mathematics Examination - AIME 2024, February 2024. URL https://maa.org/math -competitions/american-invitational-mathematics-examination-aime .

[40] A. Madaan, N. Tandon, P . Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P . Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P . Clark. Self-refine: Iterative refinement with self-feedback. In Thirty-seventh Conference on Neural Information Processing Systems, 2023. URL https://openreview.net/forum?id=S37hOerQLB .

[41] M. Mazeika, L. Phan, X. Yin, A. Zou, Z. Wang, N. Mu, E. Sakhaee, N. Li, S. Basart, B. Li, D. A. Forsyth, and D. Hendrycks. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024.

[42] M. Mirzayanov. Codeforces, 2025. URL https://codeforces.com/ .

[43] N. Muennighoff, A. M. Rush, B. Barak, T. L. Scao, N. Tazi, A. Piktus, S. Pyysalo, T. Wolf, and C. Raffel. Scaling data-constrained language models. In Thirty-seventh Conference on Neural Information Processing Systems, 2023. URL https://openreview.net/forum?id=j5Bu TrEj35 . R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V . Kosaraju, W. Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332, 2021.

[44] OpenAI. GPT4 technical report. arXiv preprint arXiv:2303.08774, 2023.

[45] OpenAI. Introducing SimpleQA, 2024a. URL https://openai.com/index/introducing -simpleqa/ .

[46] OpenAI. Introducing SWE-bench verified we're releasing a human-validated subset of swebench that more, 2024b. URL https://openai.com/index/introducing-swe-bench -verified/ .

[47] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P . Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P . Welinder, P . F. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022. URL http://papers.nips.cc/paper_files/paper/2022/ha sh/b1efde53be364a73914f58805a001731-Abstract-Conference.html .

[48] J. Pan, J. Zhang, X. Wang, L. Yuan, H. Peng, and A. Suhr. Tinyzero. https://github.com/Jiayi- Pan/TinyZero, 2025. Accessed: 2025-01-24.

[49] A. Parrish, A. Chen, N. Nangia, V . Padmakumar, J. Phang, J. Thompson, P . M. Htut, and S. R. Bowman. BBQ: A hand-built bias benchmark for question answering. In Findings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, May 22-27, 2022, pages 2086–2105. Association for Computational Linguistics, 2022.

[50] Qwen. Qwq: Reflect deeply on the boundaries of the unknown, 2024a. URL https://qwenlm .github.io/blog/qwq-32b-preview/ .

[51] Qwen. Qwen2.5: A party of foundation models, 2024b. URL https://qwenlm.github.io/b log/qwen2.5 .

[52] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. OpenAI blog, 1(8):9, 2019.

[53] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023, 2023. URL http://papers .nips.cc/paper_files/paper/2023/hash/a85b405ed65c6477a4fe8302b5e06ce 7-Abstract-Conference.html .

[54] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

[55] P . Röttger, H. Kirk, B. Vidgen, G. Attanasio, F. Bianchi, and D. Hovy. XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models. In Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024, Mexico City, Mexico, June 16-21, 2024, pages 5377–5400. Association for Computational Linguistics, 2024.

[56] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In Thirty-seventh Conference on Neural Information Processing Systems, 2023. URL https://openreview.net/forum?id=Yacmpz84TH .

[57] J. Schulman. Approximating kl divergence, 2020. URL http://joschu.net/blog/kl-app rox.html .

[58] J. Schulman, P . Moritz, S. Levine, M. Jordan, and P . Abbeel. High-dimensional continuous control using generalized advantage estimation. arXiv preprint arXiv:1506.02438, 2015.

[59] J. Schulman, F. Wolski, P . Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

[60] Z. Shao, P . Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[61] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. P . Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. CoRR, abs/1712.01815, 2017a. URL http://arxiv.org/abs/1712.01815 .

[62] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. P . Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of go without human knowledge. Nat., 550(7676):354–359, 2017b. doi: 10.1038/NATURE24270. URL https://doi.org/10.1038/nature24270 .

[63] A. Singh, J. D. Co-Reyes, R. Agarwal, A. Anand, P . Patil, X. Garcia, P . J. Liu, J. Harrison, J. Lee, K. Xu, A. T. Parisi, A. Kumar, A. A. Alemi, A. Rizkowsky, A. Nova, B. Adlam, B. Bohnet, G. F. Elsayed, H. Sedghi, I. Mordatch, I. Simpson, I. Gur, J. Snoek, J. Pennington, J. Hron, K. Kenealy, K. Swersky, K. Mahajan, L. A. Culp, L. Xiao, M. Bileschi, N. Constant, R. Novak, R. Liu, T. Warkentin, Y. Bansal, E. Dyer, B. Neyshabur, J. Sohl-Dickstein, and N. Fiedel. Beyond human data: Scaling self-training for problem-solving with language models. Transactions on Machine Learning Research, 2024. ISSN 2835-8856. URL https://openreview.net/f orum?id=lNAyUngGFK . Expert Certification.

[64] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL https://arxiv.org/abs/2408.033 14 .

[65] C. V . Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=4FWAwZ td2n .

[66] Y. Sun, X. Wang, Z. Liu, J. Miller, A. Efros, and M. Hardt. Test-time training with self-supervision for generalization under distribution shifts. In International conference on machine learning, pages 9229–9248. PMLR, 2020. M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. Le, E. Chi, D. Zhou, and J. Wei. Challenging BIG-bench tasks and whether chain-of-thought can solve them. In A. Rogers, J. Boyd-Graber, and N. Okazaki, editors, Findings of the Association for Computational Linguistics: ACL 2023, pages 13003–13051, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.824. URL https://aclanthology.org/2023.findings-acl.824/ .

[67] H. Touvron, L. Martin, K. Stone, P . Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P . Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023.

[68] T. Trinh, Y. Wu, Q. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. Nature, 2024. doi: 10.1038/s41586-023-06747-5.

[69] J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins. Solving math word problems with process-and outcome-based feedback. arXiv preprint arXiv:2211.14275, 2022.

[70] B. Vidgen, H. R. Kirk, R. Qian, N. Scherrer, A. Kannappan, S. A. Hale, and P . Röttger. SimpleSafetyTests: a Test Suite for Identifying Critical Safety Risks in Large Language Models. CoRR, abs/2311.08370, 2023.

[71] P . Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui. Math-shepherd: A labelfree step-by-step verifier for llms in mathematical reasoning. arXiv preprint arXiv:2312.08935, 2023a.

[72] X. Wang, J. Wei, D. Schuurmans, Q. V . Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023b. URL https://openreview.net/forum?id=1PL1NIMMrw .

[73] X. Wang, J. Wei, D. Schuurmans, Q. V . Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023c. URL https://openreview.net/forum?id=1PL1NIMMrw .

[74] Y. Wang, H. Li, X. Han, P . Nakov, and T. Baldwin. Do-Not-Answer: A Dataset for Evaluating Safeguards in LLMs. CoRR, abs/2308.13387, 2023d.

[75] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. In A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, editors, Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/ad236edc564f3e3156e1b2feafb99a24-Abstract-Datasets_and_Benchmarks_Track.html .

[76] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P . Liang, J. Dean, and W. Fedus. Emergent abilities of large language models. Trans. Mach. Learn. Res., 2022, 2022a. URL https: //openreview.net/forum?id=yzkSU5zdwD . J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V . Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022, 2022b. URL http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f 15af0f7b31abca4-Abstract-Conference.html .

[77] S. Welleck, X. Lu, P . West, F. Brahman, T. Shen, D. Khashabi, and Y. Choi. Generating sequences by learning to self-correct. In The Eleventh International Conference on Learning Representations, 2023. URL https://openreview.net/forum?id=hH36JeQZDaO .

[78] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying llm-based software engineering agents. arXiv preprint, 2024.

[79] H. Xin, Z. Z. Ren, J. Song, Z. Shao, W. Zhao, H. Wang, B. Liu, L. Zhang, X. Lu, Q. Du, W. Gao, Q. Zhu, D. Yang, Z. Gou, Z. F. Wu, F. Luo, and C. Ruan. Deepseek-prover-v1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search, 2024. URL https://arxiv.org/abs/2408.08152 .

[80] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. R. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In Thirty-seventh Conference on Neural Information Processing Systems, 2023a. URL https://openreview.net/forum?i d=5Xc1ecxO1h .

[81] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In The Eleventh International Conference on Learning Representations, 2023b. URL https://openreview.net/forum?id=WE_vluYU L-X .

[82] Z. Yuan, H. Yuan, C. Li, G. Dong, K. Lu, C. Tan, C. Zhou, and J. Zhou. Scaling relationship on learning mathematical reasoning with large language models. arXiv preprint arXiv:2308.01825, 2023.

[83] E. Zelikman, Y. Wu, J. Mu, and N. Goodman. STar: Bootstrapping reasoning with reasoning. In A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, editors, Advances in Neural Information Processing Systems, 2022. URL https://openreview.net/forum?id=_3ELRdg2sgI .

[84] E. Zelikman, G. R. Harik, Y. Shao, V . Jayasiri, N. Haber, and N. Goodman. Quiet-STar: Language models can teach themselves to think before speaking. In First Conference on Language Modeling, 2024. URL https://openreview.net/forum?id=oRXPiSOGH9 .

[85] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, C. Cui, O. Bousquet, Q. V . Le, and E. H. Chi. Least-to-most prompting enables complex reasoning in large language models. In The Eleventh International Conference on Learning Representations, 2023a. URL https://openreview.net/forum?id=WZH7099tgfM .

[86] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou. Instruction-following evaluation for large language models. arXiv preprint arXiv:2311.07911, 2023b.

DEEPSEEK 官方论文 · 中文全文版

原生稀疏注意力：硬件对齐且可原生训练的稀疏注意力

Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention Jingyang Yuan ∗1,2, Huazuo Gao1,

DeepSeek-AI research@deepseek.com

arXiv:2502.11089 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

长上下文建模对下一代语言模型至关重要，但标准注意力机制的高计算成本带来了显著的计算挑战。稀疏注意力为在保持模型能力的同时提升效率提供了一个有前景的方向。我们提出 NSA，这是一种可原生训练的稀疏注意力（Natively trainable Sparse Attention）机制，将算法创新与硬件对齐优化相结合，以实现高效的长上下文建模。NSA 采用动态分层稀疏策略，将粗粒度的 token 压缩与细粒度的 token 选择相结合，从而同时保持全局上下文感知与局部精度。我们的方法通过两项关键创新推进了稀疏注意力设计：（1）通过算术强度均衡的算法设计，并针对现代硬件进行实现优化，实现了显著加速；（2）实现了端到端训练，在不牺牲模型性能的前提下降低预训练计算量。如图 1 所示，实验表明使用 NSA 预训练的模型在通用基准、长上下文任务与基于指令的推理上保持或超过了全注意力（Full Attention）模型。同时，NSA 在 64k 长度序列上于解码、前向传播与反向传播中相较全注意力实现了显著加速，验证了其在整个模型生命周期中的效率。

【图表】图 1 Full Attention 模型与我们的 NSA 在性能与效率上的对比。左图：尽管 NSA 是稀疏的，但它在通用基准、长上下文任务与推理评测上的平均表现仍优于 Full Attention 基线。右图：在处理 64k 长度的序列时，NSA 在解码、前向传播与反向传播等所有阶段相较 Full Attention 均实现了显著的计算加速（原文第 2 页）

延迟瓶颈，且随序列长度的增加而愈发严重。理论估计表明，在解码 64k 长度上下文时，基于 softmax 架构的注意力计算约占总延迟的 70–80%，这凸显了对更高效注意力机制的迫切需求。

一种自然的高效长上下文建模方法是利用 softmax 注意力固有的稀疏性（Ge et al., 2023; Jiang et al., 2023）：有选择地计算关键的查询-键对，可以在保持性能的同时显著降低计算开销。近期的进展通过多种策略展示了这一潜力：KV 缓存淘汰方法（Li et al., 2024; Zhang et al., 2023b; Zhou et al., 2024）、分块 KV 缓存选择方法（Gao et al., 2024; Tang et al., 2024; Xiao et al., 2024a），以及基于采样、聚类或哈希的选择方法（Chen et al., 2024b; Desai et al., 2024; Liu et al., 2024）。尽管这些策略前景可观，现有的稀疏注意力方法在实际部署中往往难以奏效。许多方法未能取得与其理论收益相称的加速；此外，大多数方法缺乏有效的训练期支持，无法充分利用注意力的稀疏模式。

为应对这些局限，有效稀疏注意力的部署必须解决两个关键挑战：（1）**硬件对齐的推理加速**：将理论上的计算量削减转化为实际的速度提升，需要在预填充（prefilling）与解码阶段都进行硬件友好的算法设计，以缓解内存访问与硬件调度瓶颈；（2）**训练感知的算法设计**：通过可训练的算子实现端到端计算，在保持模型性能的同时降低训练成本。这些要求对于真实世界应用实现快速的长上下文推理或训练至关重要。综合这两方面来看，现有方法仍有明显差距。

为实现更有效、更高效的稀疏注意力，我们提出 NSA——一种融合分层 token 建模的可原生训练稀疏注意力（Natively trainable Sparse Attention）架构。如图 2 所示，NSA 将键和值组织为时序块，并经由三条注意力路径处理，以此降低每个查询的计算量：压缩的粗粒度 token、选择性保留的细粒度 token，以及用于局部上下文信息的滑动窗口。随后，我们实现了专用 kernel，以最大化其实际效率。针对上述关键要求，NSA 引入了两项核心创新：（1）硬件对齐的系统：针对 Tensor Core 利用率与内存访问优化分块稀疏注意力，确保算术强度均衡；（2）训练感知的设计：通过高效算法与反向算子实现稳定的端到端训练。这些优化使 NSA 既能高效部署，又能端到端训练。

【图表】图 2 NSA 架构概览。左图：该框架通过三个并行的注意力分支处理输入序列：对于给定的查询，其前面的键与值被分别处理为面向粗粒度模式的压缩注意力（compressed attention）、面向重要 token 块的选择注意力（selected attention），以及面向局部上下文的滑动注意力（sliding attention）。右图：各分支所产生的不同注意力模式的可视化。绿色区域表示需要计算注意力分数的区域，白色区域表示可以跳过的区域（原文第 3 页）

我们在真实语言语料上通过全面的实验来评估 NSA。我们在一个 27B 参数的 Transformer 骨干上使用 260B token 进行预训练，并在通用语言评测、长上下文评测与思维链推理评测中评估 NSA 的表现。我们还在 A100 GPU 上将其 kernel 速度与优化的 Triton（Tillet et al., 2019）实现进行比较。实验

结果表明，NSA 取得了与 Full Attention 基线相当或更优的性能，同时优于现有的稀疏注意力方法。此外，与 Full Attention 相比，NSA 在解码、前向与反向阶段都带来了显著的加速，且加速比随序列长度增加而增大。这些结果验证了我们的分层稀疏注意力设计有效地平衡了模型能力与计算效率。

2 重新审视稀疏注意力方法

现代稀疏注意力方法在降低 Transformer 模型的理论计算复杂度方面取得了显著进展。然而，大多数方法主要在推理阶段施加稀疏性，同时保留预训练的 Full Attention 骨干，这可能会引入架构偏差，限制其充分发挥稀疏注意力优势的能力。在介绍我们的原生稀疏架构之前，我们从两个关键视角系统地分析这些局限。

2.1 高效推理的幻觉

尽管在注意力计算上实现了稀疏性，许多方法却未能带来相应的推理延迟降低，主要原因有两个：

阶段受限的稀疏性。诸如 H2O（Zhang et al., 2023b）之类的方法在自回归解码阶段施加稀疏性，但在预填充阶段需要进行计算密集的预处理（例如注意力图计算、索引构建）。相反， MInference（Jiang et al., 2024）等方法仅关注预填充阶段的稀疏性。这些方法无法在所有推理阶段都实现加速，因为至少有一个阶段的计算成本仍与 Full Attention 相当。这种阶段专门化削弱了这些方法在预填充为主的工作负载（如书籍摘要与代码补全）或解码为主的工作负载（如长思维链（Wei et al., 2022）推理）中的加速能力。

与先进注意力架构不兼容。一些稀疏注意力方法无法适配现代高效解码架构，例如多查询注意力（Multi-Query Attention, MQA）（Shazeer, 2019）与分组查询注意力（Grouped-Query Attention, GQA）（Ainslie et al., 2023）——这些架构通过在同一组查询头之间共享 KV，显著降低了解码期间的内存访问瓶颈。例如，在 Quest（Tang et al., 2024）等方法中，每个注意力头独立选择其 KV 缓存子集。尽管这在多头注意力（Multi-Head Attention, MHA）模型中表现出计算稀疏性与内存访问稀疏性的一致，但在基于 GQA 等架构的模型中情况有所不同：KV 缓存的内存访问量对应于同一 GQA 组内所有查询头所选子集的并集。这一架构特性意味着，虽然这些方法可以减少计算操作，但所需的 KV 缓存内存访问量仍然相对较高。这一限制迫使人们做出关键取舍：一些稀疏注意力方法虽然减少了计算量，但其分散的内存访问模式与先进架构的高效内存访问设计相冲突。

这些局限之所以出现，是因为许多现有稀疏注意力方法聚焦于减少 KV 缓存或降低理论计算量，却难以在先进框架或后端中实现显著的延迟降低。这促使我们开发同时结合先进架构设计与硬件高效实现的算法，以充分利用稀疏性来提升模型效率。

2.2 可训练稀疏性的迷思

我们对原生可训练稀疏注意力的追求，源于分析仅推理方法时得到的两点关键洞见：（1）**性能退化**：事后施加稀疏性迫使模型偏离其预训练的优化轨迹。正如 Chen et al. (2024b) 所展示的，前 20% 的注意力只能覆盖总注意力分数的 70%，这使得预训练模型中的检索头等结构在推理剪枝时十分脆弱。（2）**训练效率需求**：高效处理长序列训练对现代 LLM 开发至关重要，这既包括在更长文档上预训练以增强模型容量，也包括长上下文微调与强化学习等后续适配阶段。然而，现有稀疏注意力方法主要面向推理，训练中的计算挑战在很大程度上仍未得到解决。这一局限阻碍了通过高效训练开发能力更强的长上下文模型。此外，将现有稀疏注意力适配到训练的努力也暴露出一些挑战：

不可训练的组件。ClusterKV（Liu et al., 2024）（包含 k-means 聚类）与 MagicPIG（Chen et al., 2024b）（包含基于 SimHash 的选择）等方法中的离散操作会在计算图中造成不连续。这些不可训练的组件阻止了梯度通过 token 选择过程流动，限制了模型学习最优稀疏模式的能力。

低效的反向传播。一些在理论上可训练的稀疏注意力方法存在实际的训练低效问题。HashAttention（Desai et al., 2024）等方法采用的 token 粒度选择策略，导致在注意力计算期间需要从 KV 缓存中加载大量单个 token。这种非连续的内存访问阻碍了 FlashAttention 等快速注意力技术的有效适配——后者依赖连续的内存访问与分块计算来实现高吞吐。结果，相关实现被迫退回低硬件利用率，显著降低了训练效率。

2.3 原生稀疏性势在必行

上述推理效率与训练可行性方面的局限，促使我们从根本上重新设计稀疏注意力机制。我们提出 NSA——一个原生稀疏的注意力框架，同时应对计算效率与训练需求。在接下来的小节中，我们详述 NSA 的算法设计与算子实现。

3 方法

我们的技术方案涵盖算法设计与 kernel 优化。在以下小节中，我们首先介绍方法的背景，然后给出 NSA 的整体框架及其关键算法组件，最后详述我们为最大化实际效率而设计的硬件优化 kernel。

3.1 背景

注意力机制广泛用于语言建模，其中每个查询 token q_t 会针对所有先前的键 k_:t 计算相关性分数，以生成值 v_:t 的加权和。形式化地，对于长度为 t 的输入序列，注意力操作定义为：

o_t = Attn(q_t, k_:t, v_:t)　　（1）

其中 Attn 表示注意力函数：

Attn(q_t, k_:t, v_:t) = (Σ_{i=1}^{t} α_{t,i} v_i) / (Σ_{j=1}^{t} α_{t,j})，α_{t,i} = exp(q_t^T k_i / √d_k)　　（2）

这里，α_{t,i} 表示 q_t 与 k_i 之间的注意力权重，d_k 是键的特征维度。随着序列长度增加，注意力计算在整体计算成本中日益占据主导地位，给长上下文处理带来了重大挑战。

算术强度（Arithmetic Intensity）是计算操作数与内存访问量之比。它从根本上塑造了算法在硬件上的优化方式。每块 GPU 都有一个由其峰值算力与内存带宽决定的临界算术强度，其取值为这两个硬件上限的比值。对于计算任务而言，算术强度高于该临界阈值时任务为计算受限（受 GPU FLOPS 限制），低于该阈值时则为内存受限（受内存带宽限制）。

具体而言，对于因果自注意力机制，在训练与预填充阶段，批量矩阵乘法与注意力计算表现出较高的算术强度，使这些阶段在现代加速器上属于计算受限；相比之下，自回归解码则受内存带宽约束，因为它每次前向传播只生成一个 token，却需要加载整个键值缓存，导致算术强度很低。这带来了不同的优化目标——在训练与预填充阶段降低计算成本，而在解码阶段降低内存访问。

3.2 整体框架

为发挥注意力天然稀疏模式的潜力，我们提出：对每个查询 q_t，用一组更紧凑、信息更密集的表示键值对 K̃_t、Ṽ_t 替换公式（1）中的原始键值对 k_:t、v_:t。具体而言，我们将优化后的注意力输出形式化定义为：

K̃_t = f_K(q_t, k_:t, v_:t)，Ṽ_t = f_V(q_t, k_:t, v_:t)　　（3）

o*_t = Attn(q_t, K̃_t, Ṽ_t)　　（4）

其中 K̃_t、Ṽ_t 基于当前查询 q_t 与上下文记忆 k_:t、v_:t 动态构建。我们可以设计多种映射策略，以得到不同类别的 K̃_t^c、Ṽ_t^c，并将它们组合如下：

o*_t = Σ_{c∈C} g_t^c · Attn(q_t, K̃_t^c, Ṽ_t^c)　　（5）

如图 2 所示，NSA 采用三种映射策略 C = {cmp, slc, win}，分别表示对键和值的压缩（compression）、选择（selection）与滑动窗口（sliding window）。g_t^c ∈ [0, 1] 是对应策略 c 的门控分数，由输入特征经 MLP 与 sigmoid 激活得到。令 N_t 表示重映射后的键/值总数：

$$N_t = \Sigma_{c\in C} \text{size}[\tilde{K}_t^c] \qquad (6)$$

我们通过确保 $N_t \ll t$ 来维持较高的稀疏率。

3.3 算法设计

在本小节中，我们介绍重映射策略 f_K 与 f_V 的设计：token 压缩、token 选择与滑动窗口。

3.3.1 Token 压缩

通过将连续的键或值块聚合为块级表示，我们得到能够捕捉整个块信息的压缩键与压缩值。形式化地，压缩键表示定义为：

$$\tilde{K}_t^{cmp} = f_K^{cmp}(k_{:t}) = \{ \varphi(k_{id+1:id+l}) \mid 0 \le i \le \lfloor (t-l)/d \rfloor \} \qquad (7)$$

其中 l 是块长度，d 是相邻块之间的滑动步长，φ 是一个可学习的 MLP（带有块内位置编码），用于将块内的键映射为单个压缩键。$\tilde{K}_t^{cmp} \in R^{d_k \times \lfloor (t-l)/d \rfloor}$ 是由压缩键构成的张量。通常我们取 $d < l$，以缓解信息碎片化。压缩值表示 $\tilde{V}_t^{cmp}$ 有类似的形式。压缩表示捕捉更粗粒度的高层语义信息，并降低注意力的计算负担。

3.3.2 Token 选择

仅使用压缩后的 key、value 可能会丢失重要的细粒度信息，这促使我们有选择地保留单个 key、value。下面我们介绍高效的 token 选择机制，它能够以较低的计算开销识别并保留最相关的 token。

块级选择（Blockwise Selection）：我们的选择策略以空间上连续的块为单位处理 key 与 value 序列，这基于两个关键因素：硬件效率方面的考虑，以及注意力分数固有的分布模式。块级选择对于在现代 GPU 上实现高效计算至关重要，因为与基于随机索引的读取相比，现代 GPU 架构对连续块访问的吞吐量显著更高。此外，块级计算能够最优地利用 Tensor Core。这一架构特性使块级内存访问与计算成为高性能注意力实现中的基本原则，FlashAttention 基于块的设计即是一例。块级选择遵循注意力分数固有的分布模式。先前工作（Jiang et al., 2024）表明，注意力分数往往呈现出空间连续性，即相邻的 key 倾向于具有相似的重要性水平。我们在第 6.2 节的可视化也展示了这种空间连续模式。

为实现块级选择，我们首先将 key、value 序列划分为选择块。为识别对注意力计算最重要的块，我们需要为每个块分配重要性分数。下面我们介绍计算这些块级重要性分数的方法。

重要性分数计算（Importance Score Computation）：计算块重要性分数可能引入显著开销。幸运的是，压缩 token 的注意力计算会产生中间注意力分数，我们可以利用它们来推导选择块的重要性分数，公式如下：

p_t^cmp = Softmax(q_t^T K̃_t^cmp)　　（8）

其中 p_t^cmp ∈ R^{⌊(t−l)/d⌋+1} 是 q_t 与压缩 key K̃_t^cmp 之间的注意力分数。令 l' 表示选择块大小。当压缩块与选择块采用相同的分块方案，即 $l' = l = d$ 时，我们可以直接由 p_t^slc = p_t^cmp 得到选择块的重要性分数 p_t^slc。对于分块方案不同的情况，我们根据它们的空间关系推导选择块的重要性分数。给定 $l \leqslant l'$ 、$d \mid l$ 且 $d \mid l'$，有：

p_t^slc[j] = Σ_{m=0}^{l′/d−1} Σ_{n=0}^{l/d−1} p_t^cmp[(l′/d)·j − m − n]　　（9）

其中 [·] 表示访问向量元素的索引算子。对于采用 GQA 或 MQA 且 key-value cache 在 query head 之间共享的模型，必须确保这些 head 之间一致的块选择，以最小化解码期间的 KV cache 加载。一个组内各 head 共享的重要性分数形式化定义为：

p_t^{slc′} = Σ_{h=1}^{H} p_t^{slc,(h)}　　（10）

其中上标中的 (h) 表示 head 索引，H 是每个组内的 query head 数。这一聚合确保了同一组内各 head 之间一致的块选择。

Top-n 块选择（Top-n Block Selection）：在获得选择块的重要性分数后，我们保留按块重要性分数排名前 n 的稀疏块内的 token，公式为：

I_t = { i | rank(p_t^{slc′}[i]) ≤ n }　　（11）

K̃_t^slc = Cat({ k_{i·l′+1:(i+1)·l′} | i ∈ I_t })　　（12）

其中 rank(·) 表示降序排列中的名次，rank = 1 对应最高分数，I_t 是所选块的索引集合，Cat 表示拼接操作。K̃_t^slc ∈ R^{d_k × n·l′} 是由压缩 key 组成的张量。细粒度 value Ṽ_t^slc 也有类似的公式。随后，选中的 key 与 value 与 q_t 一起参与公式（5）中定义的注意力计算。

3.3.3 滑动窗口

在注意力机制中，局部模式通常适应得更快，并可能主导学习过程，从而可能阻碍模型有效地从压缩 token 与选择 token 中学习。为解决这一问题，我们引入一个专门的滑动窗口分支来显式处理局部上下文，使其他分支（压缩与选择）能够专注于学习各自的特征，而不被局部模式走捷径（shortcut）所干扰。具体而言，我们在窗口 w 内保留最近的 token K̃_t^win = k_{t−w:t}、Ṽ_t^win = v_{t−w:t}，并将不同信息来源（压缩 token、选择 token、滑动窗口）的注意力计算隔离到独立的分支

中。这些分支的输出随后通过一个学习到的门控机制进行聚合。为了以极小的计算开销进一步防止注意力分支之间出现捷径学习，我们为三个分支提供独立的 key 与 value。这一架构设计通过防止局部模式识别与长程模式识别之间的梯度干扰来实现稳定学习，同时只引入极小的开销。

在获得全部三类 key 与 value（K̃_t^cmp、Ṽ_t^cmp；K̃_t^slc、Ṽ_t^slc；以及 K̃_t^win、Ṽ_t^win）之后，我们按照公式（5）计算最终的注意力输出。结合上述压缩、选择与滑动窗口机制，这构成了 NSA 完整的算法框架。

3.4 Kernel 设计

为在训练与预填充期间实现 FlashAttention 级别的加速，我们在 Triton 之上实现了硬件对齐的稀疏注意力 kernel。鉴于 MHA 是内存密集型的且对解码而言效率不高，我们跟随当前最先进 LLM 的做法，聚焦于 GQA 与 MQA 这类共享 KV cache 的架构。压缩与滑动窗口注意力计算可以直接兼容现有的 FlashAttention-2 kernel，而针对稀疏选择注意力，我们引入了专门的 kernel 设计。如果按照 FlashAttention 的策略将时间上连续的 query 块加载到 SRAM，将会导致低效的内存访问，因为一个块内的 query 可能需要不相交的 KV 块。为解决这一问题，我们的关键优化在于一种不同的 query 分组策略：对于 query 序列上的每个位置，我们将一个 GQA 组内的所有 query head（它们共享相同的稀疏 KV 块）加载到 SRAM。图 3 展示了我们的前向传播实现。所提出的 kernel 架构具有以下关键特性：

- 1. **分组为中心的数据加载（Group-Centric Data Loading）**：在每次内层循环中，加载位置 t 处该组内所有 head 的 query $Q \in$ R^{[h, d_k]} 以及它们共享的稀疏 key/value 块索引 I_t。
- 2. **共享 KV 获取（Shared KV Fetching）**：在内层循环中，按 I_t 索引的顺序将连续的 key/value 块加载到 SRAM，即 $K \in$ R^{[B_k, d_k]}、$V \in$ R^{[B_k, d_v]}，以最小化内存加载量，其中 B_k 是满足 B_k | l′ 的 kernel 块大小。
- 3. **网格上的外层循环（Outer Loop on Grid）**：由于内层循环长度（与所选块数量 n 成正比）对于不同 query 块几乎相同，我们将 query/output 循环放入 Triton 的 grid 调度器中，以简化和优化 kernel。

这一设计通过以下两点实现了接近最优的算术强度：（1）通过按组共享消除冗余的 KV 传输；（2）在 GPU 流式多处理器之间平衡计算负载。

【图表】图 3 NSA 的 kernel 设计。该 kernel 按 GQA 分组加载 query（Grid Loop），获取对应的稀疏 KV 块（Inner Loop），并在 SRAM 上执行注意力计算。绿色块表示 SRAM 上的数据，蓝色表示 HBM 上的数据（原文第 9 页）

4 实验

我们从三个视角评估 NSA：（1）通用基准性能，（2）长上下文基准性能，（3）思维链推理性能，并与 Full Attention 基线以及最先进的稀疏注意力方法进行比较。我们将稀疏计算范式的效率分析放到第 5 节，在那里详细讨论训练与推理速度。

4.1 预训练设置

遵循最先进 LLM 的常见实践，我们的实验采用结合分组查询注意力（GQA）与混合专家（MoE）的主干架构，具有 27B 总参数与 3B 激活参数。模型由 30 层组成，隐藏维度为 2560。对于 GQA，我们将组数设为 4，注意力头总数为 64。对于每个 head，query、key 与 value 的隐藏维度分别配置为 $dq = dk = 192$ 与 $dv = 128$。对于 MoE，我们采用 DeepSeekMoE（Dai et al., 2024; DeepSeek-AI, 2024）结构，包含 72 个路由专家与 2 个共享专家，并将 top-k 专家设为 6。为确保训练稳定性，第一层的 MoE 被替换为 SwiGLU 形式的 MLP。

【图表】图 4 27B 参数模型上 Full Attention 与我们的 NSA 在预训练损失上的对比。两个模型都表现出稳定的收敛，且 NSA 取得了更低的损失值（原文第 10 页）

表 1：Full Attention 基线与 NSA 在通用基准上的预训练性能对比，覆盖知识类（MMLU、MMLU-PRO、CMMLU）、推理类（BBH、GSM8K、MATH、DROP）与编程类（MBPP、HumanEval）任务。尽管稀疏度很高，NSA 在大多数基准上取得了更优的性能，平均表现更佳（原文第 10 页）。

模型	MMLU（5-shot Acc.）	MMLU-PRO（5-shot Acc.）	CMMLU（5-shot Acc.）	BBH（3-shot Acc.）	GSM8K（8-shot Acc.）	MATH（4-shot Acc.）	DROP（1-shot F1）	MBPP（3-shot Pass@1）	HumanEval（0-shot Pass@1）	平均
Full Attn	0.567	0.279	0.576	0.497	0.486	0.263	0.503	0.482	0.335	0.443
NSA	0.565	0.286	0.587	0.521	0.520	0.264	0.545	0.466	0.348	0.456

所提出的架构在计算成本与模型性能之间实现了有效的权衡。对于 NSA，我们设置压缩块大小 $l = 32$、滑动步长 $d = 16$、所选块大小 $l' = 64$、所选块数量 $n = 16$（其中包括固定激活 1 个初始块与 2 个局部块），以及滑动窗口大小 $w = 512$。Full Attention 与稀疏注意力模型都在 8k 长度文本的 270B token 上进行预训练，随后使用 YaRN（Peng et al., 2024）在 32k 长度文本上进行继续训练与监督微调，以实现长上下文适配。两个模型都训练至完全收敛，以确保公平比较。如图 4 所示，我们的 NSA 与 Full Attention 基线的预训练损失曲线均呈现稳定平滑的下降，且 NSA 始终优于 Full Attention 模型。

4.2 基线方法

除了与 Full Attention 比较之外，我们还评估了若干最先进的推理阶段稀疏注意力方法：H2O（Zhang et al., 2023b）、infLLM（Xiao et al., 2024a）、Quest（Tang et al., 2024），以及 Exact-Top——它首先计算完整的注意力分数，为每个 query 选出 top-n 分数对应的 key，然后在这些位置上计算注意力。这些方法涵盖了多种稀疏注意力范式，包括 KV-cache 驱逐、query 感知选择以及精确 top-n 稀疏选择。

【表格】表 2 NSA 与各基线在 LongBench 上的性能对比，包括单文档问答、多文档问答、合成与代码任务类别的子集。NSA 的表现优于包括 Full Attention 在内的大多数基线（原文第 11 页）

对于通用评估，由于大多数样本的长度都在稀疏注意力基线的局部上下文窗口之内，这些方法实际上等价于 Full Attention。因此，在这种设置下我们只展示 NSA 与 Full Attention 基线的对比结果。在长上下文评估中，我们与所有基线方法进行比较，并将所有稀疏注意力方法的稀疏度设为相同，以确保公平比较。对于思维链推理评估，由于它需要长文本的监督微调，而稀疏注意力基线不支持训练，我们将比较范围限制为 Full Attention。

4.3 性能对比

通用评估（General Evaluation）：我们在涵盖知识、推理与编程能力的综合基准套件上评估了预训练的 NSA 与 Full Attention 基线，包括 MMLU（Hendrycks et al., 2020）、MMLU-PRO（Wang et al., 2024）、CMMLU（Li et al., 2023）、BBH（Suzgun et al., 2022）、GSM8K（Cobbe et al., 2021）、MATH（Hendrycks et al., 2020）、DROP（Dua et al., 2019）、MBPP（Austin et al., 2021）与 HumanEval（Chen et al., 2021）。结果如表 1 所示。尽管具有稀疏性，NSA 仍取得了更优的总体性能，在 9 项指标中的 7 项上优于包括 Full Attention 在内的所有基线。这表明，尽管 NSA 在较短序列

上可能无法充分发挥其效率优势，但它展现出了强劲的性能。值得注意的是，NSA 在推理相关基准上取得了显著提升（DROP：+0.042，GSM8K：+0.034），这表明我们的预训练有助于模型发展出专门的注意力机制。这种稀疏注意力预训练机制迫使模型聚焦于最重要的信息，可能通过过滤掉无关注意力通路带来的噪声而提升性能。在多样化评估中的一致表现也验证了 NSA 作为通用架构的稳健性。

长上下文评估（Long-Context Evaluation）：如图 5 所示，在 64k 上下文的“大海捞针”（needle-in-a-haystack；Kamradt, 2023）测试中，NSA 在所有位置上均取得了完美的检索准确率。这一性能源于我们分层的稀疏注意力设计：它结合了用于高效全局上下文扫描的压缩 token，以及用于精确定位局部信息的选择 token。粗粒度的压缩以较低计算成本识别出相关的上下文块，而对选中 token 的 token 级注意力则确保关键细粒度信息得以保留。这一设计使 NSA 能够同时保持全局感知与局部精确性。我们还在 LongBench（Bai et al., 2023）上评估 NSA，与最先进的稀疏注意力方法以及 Full Attention 基线。

为确保稀疏性一致，我们将所有稀疏注意力基线中每个查询所激活的 token 数设为 2560，这对应于 NSA 在处理 32k 序列长度时平均激活的 token 数。参照 StreamLLM（Xiao et al., 2023），该 token 预算包含开头的 128 个 token 与 512 个局部 token。由于某些 LongBench 子集在所有模型上的得分都很低、可能无法提供有意义的比较，我们将这些子集排除在外。如表 2 所示，NSA 取得了最高的平均分 0.469，优于所有基线（相比 Full Attention 提升 0.032，相比 Exact-Top 提升 0.046）。这一提升源于两项关键创新：（1）我们的原生稀疏注意力设计能够在预训练期间对稀疏模式进行端到端优化，促进稀疏注意力模块与其他模型组件之间的同步适应；（2）分层稀疏注意力机制在局部与全局信息处理之间取得了平衡。

【图表】图 5 在 64k 上下文长度下，不同上下文位置上的 Needle-in-a-Haystack 检索准确率。NSA 凭借其分层稀疏注意力设计实现了完美准确率。（原文第 12 页）

值得注意的是，NSA 在需要长上下文复杂推理的任务上表现尤为出色：在多跳问答任务（HPQ 与 2Wiki）上相对 Full Attention 分别提升 0.087 与 0.051，在代码理解（LCC）上超过基线（+0.069），在段落检索（PassR-en）上优于其他方法（+0.075）。这些结果验证了 NSA 处理多样化长上下文挑战的能力，其原生预训练的稀疏注意力在学习任务最优模式方面提供了额外收益。

思维链推理评估（Chain-of-Thought Reasoning Evaluation）：为评估 NSA 与先进下游训练范式的兼容性，我们研究了其通过后训练获得思维链数学推理能力的情况。鉴于强化学习在较小规模模型上的效果有限，我们采用来自 DeepSeek-R1 的知识蒸馏，使用 100 亿（10B）token 的 32k 长度数学

推理轨迹进行监督微调（SFT）。这产生了两个可比较的模型：Full Attention-R（Full Attention 基线）与 NSA-R（我们的稀疏变体）。我们在具有挑战性的美国数学邀请赛（AIME 24）基准上评估这两个模型。我们使用采样温度 0.7 与 top-p 值 0.95，为每道题生成 16 个回答并取平均分。为验证推理深度的影响，我们在两种生成上下文限制下进行实验：8k 与 16k token，衡量更长的推理链是否能提升准确率。模型预测的示例对比见附录 A。

生成 token 上限	8192	16384
Full Attention-R	0.046	0.092
NSA-R	0.121	0.146

表 3 监督微调后基于指令的 AIME 评测。我们的 NSA-R 在 8k 与 16k 序列长度下均表现出优于 Full Attention-R 的性能。

如表 3 所示，NSA-R 在 8k 上下文设置下取得了显著高于 Full Attention-R 的准确率（+0.075），这一优势在 16k 上下文中依然保持（+0.054）。这些结果验证了原生稀疏注意力的两项关键优势：（1）预训练的稀疏注意力模式能够高效捕捉对复杂数学推导至关重要的长程逻辑依赖；（2）我们架构的硬件对齐设计保持了足够的上下文密度，以支持推理深度的增长而不发生灾难性遗忘。跨上下文长度的一致优势证实了在原生集成到训练流程时，稀疏注意力对高级推理任务的可行性。

5 效率分析

我们在 8 块 GPU 的 A100 系统上评估 NSA 相对 Full Attention 的计算效率。在效率分析中，我们还将模型配置为 GQA 分组 $g = 4$、每组头数 $h = 16$、查询/键维度 $d_k = 192$、值维度 $d_v = 128$。沿用第 4 节的相同设置，我们将 NSA 的压缩块大小设为 $l = 32$、滑动步长 $d = 16$、选中块大小 $l' = 64$、选中块数 $n = 16$、滑动窗口大小 $w = 512$。

5.1 训练速度

我们比较 NSA 注意力与 Full Attention 的 Triton 实现以及基于 Triton 的 FlashAttention-2，以确保在相同后端下进行公平的速度比较。如图 6 所示，随着上下文长度增加，我们的 NSA 取得了逐步增大的加速比，在 64k 上下文长度下前向加速最高达 9.0×，反向加速最高达 6.0×。值得注意的是，序列越长，速度优势越明显。这一加速源于我们为最大化稀疏注意力架构效率而进行的硬件对齐算法设

计：（1）分块（Blockwise）内存访问模式通过合并加载最大化 Tensor Core 利用率；（2）内核中精细的循环调度消除了冗余的 KV 传输。

【图表】图 6 基于 Triton 的 NSA 内核与基于 Triton 的 FlashAttention-2 内核的比较。我们的实现显著降低了所有上下文长度下的延迟，且随着输入长度增加，改进愈发明显。（原文第 13 页）

5.2 解码速度

Attention 的解码速度主要取决于内存访问瓶颈，而后者与 KV cache 的加载量密切相关。在每次解码步骤中，我们的 NSA 最多只需加载 $\lfloor (s-l)/d \rfloor$ 个压缩 token、$n \cdot l'$ 个被选中的 token 以及 w 个邻居 token，其中 s 为已缓存的序列长度。如表 4 所示，随着解码长度增加，我们的方法显著降低了延迟，在 64k 上下文长度下最高实现 11.6× 加速。这一内存访问效率上的优势也随着序列变长而放大。

上下文长度	8192	16384	32768	65536
Full Attention	8192	16384	32768	65536
NSA	2048	2560	3584	5632
预期加速比	4×	6.4×	9.1×	11.6×

表 4 解码期间每次注意力操作的内存访问量（以等效 token 数计）。由于解码的算术强度低且受内存带宽限制，预期加速比与内存访问量近似成线性关系。

6 讨论

在本节中，我们回顾 NSA 的开发过程，并讨论在探索不同稀疏注意力策略时获得的关键洞见。尽管我们的方法展现出令人期待的结果，但理解其他策略所遇到的挑战并分析注意力模式，可为未来的研究方向提供有价值的背景。我们首先考察促使我们做出这些设计选择的替代 token 选择策略所面临的挑战，随后给出有助于洞察注意力分布模式的可视化结果。

6.1 替代 Token 选择策略的挑战

在设计 NSA 之前，我们探索了将现有稀疏注意力方法适配到训练阶段。然而，这些尝试遇到了各种挑战，促使我们设计一种不同的稀疏注意力架构：

基于 Key 聚类的策略（Key-Clustering Based Strategies）：我们考察了 ClusterKV（Liu et al., 2024）等基于聚类的策略。这类方法将来自同一聚类的 Key 与 Value 存储在连续的内存区域中。虽然在训练与推理中理论上可行，但它们面临三个重大挑战：（1）动态聚类机制引入的不可忽视的计算开销；（2）簇间不平衡加剧的算子优化困难，尤其是在混合专家（MoE）系统中——倾斜的专家并行（EP）组执行时间会导致持续的负载不均衡；（3）强制周期性重新聚类与分块顺序训练协议所带来的实现约束。这些因素共同造成显著瓶颈，严重限制了它们在实际部署中的有效性。

其他分块选择策略（Other Blockwise Selection Strategies）：我们还考虑了不同于 NSA 的分块 key、value 选择策略，例如 Quest（Tang et al., 2024）与 InfLLM（Xiao et al., 2024a）。这些方法依赖为每个 KV 块计算重要性分数，并基于其与 q_t 的相似性选择 top-n 个块。然而，现有方法面临两个关键问题：（1）由于选择操作不可微，基于神经网络的重要性分数计算依赖辅助损失，这增加了算子开销，并常常降低模型性能；（2）无参数的启发式重要性分数计算策略的召回率低，导致性能次优。我们在一个架构类似的 3B 参数模型上评估了这两种方法，并将其损失曲线与 NSA 和 Full Attention 进行比较。对于基于辅助损失的选择方法，我们为每个 token 引入额外的查询，并为每个块引入代表性 key 以估计块重要性分数。我们通过对每个 key 块内的注意力分数进行平均池化来计算块级监督信号，并使用 KL 散度监督块重要性预测。为适应高效解码，我们保持单个查询的粒度，而非使用块平均查询。这种基于辅助损失的重要性估计与 SeerAttention（Gao et al., 2024）在概念上相似。对于无参数的启发式选择方法，遵循 Quest 的策略，我们使用查询与 key 块按坐标的 min-max 之间的乘积来实现直接选择，而不引入额外参数。我们还探索了一种冷启动训练方法，即在前 1000 步使用 Full Attention，然后再过渡到启发式分块选择。如图 7 所示，两种方法的损失都更差。

【图表】图 7 在 3B 参数模型上比较 Full Attention 与不同 token 选择策略的训练损失。我们的 NSA 取得了更好的性能。（原文第 15 页）

6.2 可视化

为了探索 Transformer 注意力分布中的潜在模式并为我们的设计寻找灵感，我们在图 8 中可视化了预训练的 27B Full Attention 模型的注意力图。可视化揭示了一些有趣的模式：注意力分数倾向于呈现分块聚类特征，相邻的 key 往往具有相似的注意力分数。这一观察启发了我们对 NSA 的设计，表明基于空间连续性选择 key 块可能是一种有前景的方法。分块聚类现象表明，序列中相邻的 token 可能与查询 token 共享某些语义关系，尽管这些关系的确切性质仍需进一步研究。这一观察促使我们探索

一种作用于连续 token 块而非单个 token 的稀疏注意力机制，旨在提升计算效率并保留高注意力模式。

【图表】图 8 Full Attention Transformer 上注意力图的可视化。浅色区域表示更高的注意力值。如图所示，注意力分数呈现分块聚类分布。（原文第 15 页）

7 相关工作

我们回顾了通过稀疏注意力提升注意力计算效率的现有方法。这些方法根据其核心策略大致可分为三类：（1）固定稀疏模式；（2）动态 token 剪枝；（3）查询感知的选择。我们介绍每一类中的若干代表性工作。

7.1 固定稀疏模式

SlidingWindow 是一种常用方法，它允许查询仅在固定窗口内计算注意力。StreamingLLM（Xiao et al., 2023）将注意力汇聚点（attention sinks）与局部窗口相结合，以处理连续的文本流。MoA（Fu et al., 2024a）与 DuoAttention（Xiao et al., 2024b）也采用类似的局部与汇聚信息进行长序列建模。Longformer（Beltagy et al., 2020）将局部窗口注意力与全局 token 交错，以处理长序列。与它们相比，我们的 NSA 不依赖预定义的稀疏模式，而是自动学习模式，从而释放利用完整上下文的潜力。

7.2 动态 Token 剪枝

一些方法旨在通过动态 KV-cache 剪枝来降低推理时的内存与计算成本。H2O（Zhang et al., 2023b）、BUZZ（Zhao et al., 2024）与 SepLLM（Chen et al., 2024a）实现了自适应方法来降低解码期间 KV-cache 的内存占用。这些方法动态剔除被认为对未来预测不太重要的 token。FastGen（Ge et al., 2023）与 HeadKV（Fu et al., 2024b）通过为各个注意力头分配不同策略来优化计算。SnapKV（Li et al., 2024）引入了一种 token 剪枝策略，通过选择性保留最关键的特征来减少 KV-cache，从而实现高效的内存使用。与这些面向推理的方法不同，我们的 NSA 在训练阶段就原生地引入了稀疏性。

7.3 查询感知的选择

其他工作专注于与查询相关的 token 选择方法，以在保持注意力质量的同时减少注意力计算。Quest（Tang et al., 2024）采用分块选择策略，其中每个块的重要性由查询与 key 块按坐标的 min-max

之间的乘积估计。InfLLM（Xiao et al., 2024a）通过维护注意力汇聚点、局部上下文与可检索块，将固定模式与检索相结合。该方法从每个块中选择代表性 key 来估计块重要性。HashAttention（Desai et al., 2024）通过使用学习到的函数将查询与 key 映射到汉明空间，将关键 token 识别建模为推荐问题。ClusterKV（Liu et al., 2024）首先对 key 进行聚类，然后基于查询-聚类相似度选择最相关的聚类进行注意力计算，从而实现稀疏性。MInference（Jiang et al., 2024）与 TokenSelect（Wu et al., 2024）基于 token 级重要性评分选择 KV 对进行计算。SeerAttention（Gao et al., 2024）将查询与 key 分离为空间块，并执行分块选择以实现高效计算。与这些方法相比，我们的 NSA 在完整模型生命周期（包括训练、预填充与解码）中实现了硬件对齐的稀疏注意力计算。

8 结论

我们提出了 NSA，一种面向高效长上下文建模的硬件对齐稀疏注意力架构。通过在可训练架构中整合分层 token 压缩与分块 token 选择，我们的架构在保持 Full Attention 性能的同时实现了训练与推理加速。NSA 将最先进水平向前推进：其通用基准性能可匹配 full-attention 基线，在长上下文评估中超越了后者的建模能力，并具备更强的推理能力，同时伴随着可度量的计算延迟降低与显著加速。

附录

附录：AIME 结果示例（见英文原文）

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] achieves hardware-aligned sparse attention computation throughout the full model lifecycle, including training, prefilling, and decoding.

[2] We present NSA, a hardware-aligned sparse attention architecture for efficient long-context modeling. By integrating hierarchical token compression with blockwise token selection within a trainable architecture, our architecture achieves accelerated training and inference while maintaining Full Attention performance. NSA advances the state-of-the-art by demonstrating general benchmark performance matches full-attention baselines, exceeding modeling capability in long-context evaluations, and enhanced reasoning ability, all accompanied by measurable reductions in computational latency and achieving significant speedup.

[3] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

[4] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

[5] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. arXiv preprint arXiv:2308.14508, 2023.

[6] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150, 2020.

[7] G. Chen, H. Shi, J. Li, Y. Gao, X. Ren, Y. Chen, X. Jiang, Z. Li, W. Liu, and C. Huang. Sepllm: Accelerate large language models by compressing one segment into one separator. arXiv preprint arXiv:2412.12094, 2024a.

[8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

[9] Z. Chen, R. Sadhukhan, Z. Ye, Y. Zhou, J. Zhang, N. Nolte, Y. Tian, M. Douze, L. Bottou, Z. Jia, et al. Magicpig: Lsh sampling for efficient llm generation. arXiv preprint arXiv:2410.16179, 2024b.

[10] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems, 2021. URL https://arxiv. org/abs/2110.14168, 2021.

[11] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. arXiv preprint arXiv:2401.06066, 2024.

[12] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. 2024. URL https://arxiv.org/abs/2405.04434 . DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948 .

[13] A. Desai, S. Yang, A. Cuadron, A. Klimovic, M. Zaharia, J. E. Gonzalez, and I. Stoica. Hashattention: Semantic sparsity for faster inference. arXiv preprint arXiv:2412.14468, 2024.

[14] D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. arXiv preprint arXiv:1903.00161, 2019.

[15] T. Fu, H. Huang, X. Ning, G. Zhang, B. Chen, T. Wu, H. Wang, Z. Huang, S. Li, S. Yan, et al. Moa: Mixture of sparse attention for automatic large language model compression. arXiv preprint arXiv:2406.14909, 2024a.

[16] Y. Fu, Z. Cai, A. Asi, W. Xiong, Y. Dong, and W. Xiao. Not all heads matter: A head-level kv cache compression method with integrated retrieval and reasoning. arXiv preprint arXiv:2410.19258, 2024b.

[17] Y. Gao, Z. Zeng, D. Du, S. Cao, H. K.-H. So, T. Cao, F. Yang, and M. Yang. Seerattention: Learning intrinsic sparse attention in your llms. arXiv preprint arXiv:2410.13276, 2024.

[18] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao. Model tells you what to discard: Adaptive kv cache compression for llms. arXiv preprint arXiv:2310.01801, 2023.

[19] G. T. Google, P . Georgiev, V . I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530, 2024.

[20] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[21] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. arXiv preprint arXiv:2310.05736, 2023.

[22] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. arXiv preprint arXiv:2407.02490, 2024.

[23] G. Kamradt. LLMTest NeedleInAHaystack. GitHub repository, 2023. URL https://github.com/gkamradt/LLMTest_NeedleInAHaystack . Accessed: [Insert Access Date Here].

[24] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese. arXiv preprint arXiv:2306.09212, 2023.

[25] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P . Lewis, and D. Chen. Snapkv: Llm knows what you are looking for before generation. arXiv preprint arXiv:2404.14469, 2024.

[26] G. Liu, C. Li, J. Zhao, C. Zhang, and M. Guo. Clusterkv: Manipulating llm kv cache in semantic space for recallable compression. arXiv preprint arXiv:2412.03213, 2024.

[27] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P . Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In S. Follmer, J. Han, J. Steimle, and N. H. Riche, editors, Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 2023, San Francisco, CA, USA, 29 October 2023– 1 November 2023, pages 2:1–2:22. ACM, 2023. B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models. In ICLR. OpenReview.net, 2024.

[28] N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019.

[29] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[30] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context llm inference. arXiv preprint arXiv:2406.10774, 2024.

[31] P . Tillet, H.-T. Kung, and D. Cox. Triton: an intermediate language and compiler for tiled neural network computations. In Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, pages 10–19, 2019.

[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. Advances in Neural Information Processing Systems, 2017.

[33] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. arXiv preprint arXiv:2406.01574, 2024.

[34] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V . Le, D. Zhou, et al. Chainof-thought prompting elicits reasoning in large language models. Advances in neural information processing systems, 35:24824–24837, 2022.

[35] W. Wu, Z. Pan, C. Wang, L. Chen, Y. Bai, K. Fu, Z. Wang, and H. Xiong. Tokenselect: Efficient long-context inference and length extrapolation for llms via dynamic token-level kv cache selection. arXiv preprint arXiv:2411.02886, 2024.

[36] C. Xiao, P . Zhang, X. Han, G. Xiao, Y. Lin, Z. Zhang, Z. Liu, and M. Sun. Infllm: Training-free long-context extrapolation for llms with an efficient context memory. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024a.

[37] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. arXiv preprint arXiv:2309.17453, 2023.

[38] G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han. Duoattention: Efficient longcontext llm inference with retrieval and streaming heads. arXiv preprint arXiv:2410.10819, 2024b.

[39] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P . Pham, A. Ravula, Q. Wang, L. Yang, et al. Big bird: Transformers for longer sequences. Advances in neural information processing systems, 33:17283–17297, 2020.

[40] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. Star: Bootstrapping reasoning with reasoning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 – December 9, 2022, 2022. F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J. Lou, and W. Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. In H. Bouamor, J. Pino, and K. Bali, editors, Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6–10, 2023, pages 2471– 2484. Association for Computational Linguistics, 2023a.

[41] K. Zhang, J. Li, G. Li, X. Shi, and Z. Jin. Codeagent: Enhancing code generation with toolintegrated agent systems for real-world repo-level coding challenges. In L. Ku, A. Martins, and V . Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11–16, 2024, pages 13643–13658.

[42] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. Advances in Neural Information Processing Systems, 36:34661–34710, 2023b.

[43] J. Zhao, Z. Fang, S. Li, S. Yang, and S. He. Buzz: Beehive-structured sparse kv cache with segmented heavy hitters for efficient llm inference. arXiv preprint arXiv:2410.23079, 2024.

[44] Z. Zhou, C. Li, X. Chen, S. Wang, Y. Chao, Z. Li, H. Wang, R. An, Q. Shi, Z. Tan, et al. Llm × mapreduce: Simplified long-sequence processing using large language models. arXiv preprint arXiv:2410.09342, 2024. A. Example of AIME Results

[45] Prompt: "|User|Let x , y and z be positive real numbers that satisfy the following system of equa-

[46] tions: x 1 log = yz 2 1 y = log xz 3 z 1 log = xy 4

[47] m 4 3 2 Then the value of log (x y z) is where m and n are relatively prime positive integers. 2 n Find $m + n$. Please reason step by step, and put your final answer within .|Assistant|”

[48] Given the system of equations: y 1 1 z 1 x = 2. log = 3. log = 1. log 2 2 2 yz 2 xz 3 xy 4

[49] We convert these logarithmic equations to their exponential forms: √ y x z 1 / 3 1 / 4 1. 2 2. = = 2 3. = 2 yz xz xy Expressing each variable in terms of x , y , and z : √ 1 / 3 1 / 4 2 · yz 2. y = 2 · xz 3. z = 2 · xy 1. x =

[50] Substituting these expressions into each other, we find: √ √ 1 / 3 1 / 3 - From x = 2 · yz and y = 2 · xz , we get x = 2 ·(2 · xz)· z , leading to − 5 / 12 − 3 / 8 z = 2 . - Substituting z into the equation for y , we get y = 2 . - Substituting − 7 / 24 z and y into the equation for x , we get x = 2 . 4 3 2 Next, we compute x y z : 4 − 7 / 24 4 − 7 / 6 3 − 3 / 8 3 − 9 / 8 2 − 5 / 12 2 − 5 / 6 - x = (2) = 2 - y = (2) = 2 - z = (2) = 2

[51] 4 3 2 − 7 / 6 − 9 / 8 − 5 / 6 − 75 / 24 − 25 / 8 x y z = 2 · 2 · 2 = 2 = 2

[52] 4 3 2 | log (x y z)| = 25 / 8 Thus, m = 25 and n = 8, giving $m + n$ = 33.

[53] Given the system of logarithmic equations:

[54] 1 x = log yz 2 y 1 log = xz 3 1 z = log xy 4

[55] We convert these equations to exponential form:

[56] √ √ x = ⇒ x = 2 = 2 · yz yz 1 1 y 3 3 = = ⇒ y = 2 · xz 2 xz 1 1 z 4 4 = · xy 2 = ⇒ z = 2 xy

[57] Substituting these into each other, we find: √ 1 1 3 4 1. x = 2 · yz 2. y = 2 · xz 3. z = 2 · xy

[58] By solving these equations step-by-step, we find: − 5 / 12 − 2 / 3 − 1 / 24 1 / 24 - z = 2 - xy = 2 - y = 2 - x = 2

[59] However, upon verifying the third equation, we realize the system might not have a solution. Therefore, we use matrix inversion to solve the system:

[60] 1 log (x)− log (y)− log (z) = 2 2 2 2 log (y)− log (x)− log (z) = 2 2 2 3 log (z)− log (x)− log (y) = 2 2 2 4

[61] Let A = log (x) , B = log (y) , C = log (z) . The system becomes: 2 2 2

[62] $B - A - C$ = $C - A - B$ = We solve this system using Cramer’s rule and matrix inversion. The determinant of the matrix is -2, and the solutions are:

[63] 61 / 48 13 / 16 − 1 / 6 x = 2 , y = 2 , z = 2

[64] 61 13 1 4 log x + 3 log y + 2 log z = 4 · + 3 · + 2 · − 2 2 2 48 16 6

[65] 61 16 244 13 117 1 = − − 4 · = · = · 48 48 , 3 16 48 , 2 6 48

[66] 244 117 16 345 115 + − = = 48 48 48 48 16

[67] The fraction is in simplest form, so m = 115 and n = 16. Therefore, $m + n$ = 115 + 16 = 131.

[68] x 4 y “|User |Real numbers x and y with x , y > 1 satisfy log (y) = log (x) = 10. x y What is the value of xy ? . |Assistant|” Please reason step by step, and put your final answer within

[69] Given the equations: x 1. log (y) = 10 x 4 y 2. log (x) = 10 y

[70] We start by applying the logarithm power rule to both equations.

[71] Let z = log (x) . Then, log (y) = . Substituting this into the equation, we get: y x z

[72] y · 4 log (x) = 10 = ⇒ 4 y log (x) = 10 y y

[73] Since log (x) = z , this simplifies to: y

[74] We now have two expressions: 1. x = 10 z 5 / 2 2. y = z

[75] 5 / 2 Substituting x = 10 z and y = into the product xy , we get: z 5 / 2 5 5 xy = (10 z)· = 10 z · = 10 · = 25 z 2 z 2

[76] Given the problem where real numbers x and y with x , y > 1 satisfy the equa- x 4 tions log (y) = 10 and log (x y) = 10, we need to find the value of xy . x y

[77] First, we convert the logarithmic equations to their exponential forms using the change of base formula. x 1. log (y) = 10 x

[78] x log (y) = 10 x ln y 9 Using the change of base formula, log (y) = . Since log x = , we have x y 4 ln x log y = . Thus, x 9

[79] 4 10 · 9 90 45 x · = 10 = ⇒ x = = = 9 4 4 2

[80] 4 log x + 1 = 10 = ⇒ 4 log x = 9 = ⇒ log x = y y y

[81] Again, using the change of base formula, log y = . x 9

[82] Next, we express x and y in terms of each other:

[83] 4 / 9 1 + 4 / 9 13 / 9 45 45 45 45 · = = xy = 2 2 2 2

[84] a However, upon re-evaluating and using another approach by setting x = 2 and b y = 2 , we find:

[85] x log y 4 45 x x log (y) = 10 = ⇒ = 10 = ⇒ x · = 10 = ⇒ x = x log y 9 2 x 4 log x + 1 9 y log (x y) = 10 = ⇒ = 10 = ⇒ 4 · + 1 = 10 = ⇒ 9 + 1 = 10 consistent y log x 4 y

[86] Finally, we find that the product xy simplifies to: xy =

DeepSeek-Prover-V2：通过面向子目标分解的强化学习推进形式化数学推理

DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition Z.Z. Ren, Zhihong Shao*, Junxiao Song*, Huajian Xin†, Haocheng Wan*

DeepSeek-AI research@deepseek.com

arXiv:2504.21801 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-Prover-V2，这是一个面向 Lean 4 形式化定理证明的开源大语言模型，其初始化数据由 DeepSeek-V3 驱动的递归定理证明流程收集而来。冷启动训练过程首先提示 DeepSeek-V3 将复杂问题分解为一系列子目标。被解决子目标的证明被合成为一个思维链过程，并与 DeepSeek-V3 的逐步推理相结合，从而为强化学习构建初始冷启动。这一过程使我们能够将非形式化推理与形式化数学推理集成到单一统一模型中。由此得到的模型 DeepSeek-Prover-V2-671B 在神经定理证明上取得了最先进的性能，在 MiniF2F-test 上达到 88.9% 的通过率，并解决了 PutnamBench 中 658 道题目中的 47 道。除标准基准外，我们引入 ProverBench——一个包含 325 道形式化问题的集合，以丰富我们的评估，其中包括从近期 AIME 竞赛（2024–2025 年）中精选的 15 道题。对这 15 道 AIME 题目的进一步评估显示，该模型成功解决了其中 6 道。作为对比，DeepSeek-V3 使用多数投票解决了其中 8 道，这表明大语言模型在形式化与非形式化数学推理之间的差距正在显著缩小。

1 引言

大语言模型（LLM）中推理能力的出现，彻底改变了人工智能的诸多领域，尤其是数学问题求解领域（DeepSeek-AI，2025）。这些进展在很大程度上得益于推理时扩展（inference-time scaling）范式，其中最典型的是自然语言思维链推理（Jaech 等，2024）。LLM 不再仅依赖单次前向传播得出答

案，而是可以对中间推理步骤进行反思，从而同时提升准确性与可解释性。尽管自然语言推理在求解竞赛级数学问题上取得了成功，但其在形式化定理证明中的应用仍从根本上具有挑战性。LLM 以本质上非形式化的方式进行自然语言推理，依赖启发式、近似以及数据驱动的猜测模式，而这些往往缺乏形式化验证系统所要求的严谨结构。相比之下，Lean（Moura 与 Ullrich，2021）、Isabelle（Paulson，1994）与 Coq（Barras 等，1999）等证明助手运行在严格的逻辑基础之上，其中每个证明步骤都必须被显式构造并形式化验证。这些系统不允许任何歧义、隐式假设或细节省略。弥合非形式化的高层推理与形式化验证系统的句法严谨性之间的鸿沟，一直是神经定理证明领域的长期研究挑战（Yang 等，2024）。

为借助非形式化数学推理的优势来支撑形式化定理证明，一种经典做法是依据自然语言证明梗概的引导，对形式化证明进行层次化分解。Jiang 等（2023）提出了名为 Draft, Sketch, and Prove（DSP）的框架，利用大语言模型生成自然语言证明梗概，随后将其翻译为形式化证明步骤。这种从非形式到形式的定理证明范式与层次化强化学习中的子目标概念高度相似（Barto 与 Mahadevan，2003；Nachum 等，2018；Eppe 等，2022）：复杂任务被分解为层次化的更简单子任务，这些子任务可被独立求解，从而逐步达成总体目标。在形式化定理证明中，子目标通常是某个有助于证明更大定理的中间命题或引理（Zhao 等，2023，2024）。这种层次化分解与人类的问题求解策略一致，并支持模块化、可复用性以及更高效的证明搜索（Wang 等，2024b；Zheng 等，2024）。近期研究进一步扩展了这一范式：采用多层层次结构进行结构化证明生成（Wang 等，2024a），以及利用强化学习技术优化将复杂定理分解为可管理子目标的过程（Dong 等，2024）。

在本文中，我们开发了一个用于子目标分解的推理模型，借助一套合成冷启动数据与大规模强化学习来提升其性能。为构建冷启动数据集，我们开发了一个简洁而有效的递归定理证明流程，利用 DeepSeek-V3（DeepSeek-AI，2024）作为子目标分解与形式化的统一工具。我们提示 DeepSeek-V3 将定理分解为高层证明梗概，同时将这些证明步骤在 Lean 4 中形式化，从而得到一系列子目标。由于子目标分解由大型通用模型驱动，我们使用较小的 7B 模型来处理每个子目标的证明搜索，从而降低相应的计算负担。此外，我们引入课程学习框架，利用分解出的子目标生成猜想定理，逐步提高训练任务的难度，以更好地引导模型的学习过程。当一道难题的各个分解步骤都被解决后，我们将完整的逐步形式化证明与 DeepSeek-V3 相应的思维链配对，以创建冷启动推理数据。基于这些冷启动数据，我们进一步应用强化学习阶段，以进一步增强非形式化数学推理与形式化证明构造之间的联系。实验表明，从任务分解中非形式化推理的冷启动出发进行强化学习，能够显著增强模型的形式化定理证明能力。所得 DeepSeek-Prover-V2-671B 模型在多个基准上树立了神经定理证明的新最先进水平。在 MiniF2F-test 上，它在 Pass@32 下达到 82.4% 的准确率，在 Pass@8192 下提升至 88.9%。该模型对

大学水平定理证明展现出强大的泛化能力：在 Pass@1024 下求解了 ProofNet-test 中 37.1% 的问题，并解决了 PutnamBench 658 道难题中的 47 道。此外，我们贡献了 ProverBench——一个包含 325 道形式化问题的基准数据集，以推进神经定理证明研究，其中 15 道来自著名的 AIME 竞赛（24–25 年度）。DeepSeek-Prover-V2-671B 成功求解了这 15 道高难度 AIME 问题中的 6 道，进一步展示了其精深的数学推理能力。

【图表】图 2 DeepSeek-Prover-V2 所采用的冷启动数据收集流程概览。我们首先提示 DeepSeek-V3 生成自然语言证明梗概，同时将其形式化为带有 sorry 占位符的 Lean 语句，以省略证明细节。随后，一个 7B 证明模型递归求解分解出的子目标。通过组合这些子目标证明，我们为原始复杂问题构造出完整的形式化证明。该组合证明被附加到 DeepSeek-V3 的原始思维链之后，从而创建用于形式化数学推理的高质量冷启动训练数据。（原文第 3 页）

2 方法

2.1 基于子目标分解的递归证明搜索

将复杂定理的证明分解为一系列可作为垫脚石的小引理，是人类数学家常用的一种有效策略。此前已有多项研究在神经定理证明的背景下探索过这种层次化策略，旨在借助现代 LLM 的非形式化推理能力来提升证明搜索效率（Jiang 等，2023；Zhao 等，2023；Wang 等，2024a；Dong 等，2024）。在本文中，我们开发了一个简洁而有效的流程，利用 DeepSeek-V3（DeepSeek-AI，2024）作为形式化定理证明中子目标分解的统一工具。

【图表】图 3 我们将分解后的子目标转换为一系列引理语句的示例说明。我们首先（a）替换原始目标状态，然后（b）将先前的子目标作为前提纳入。语句类型（b）用于复杂问题的递归求解，而类型（a）与（b）都会被纳入课程学习过程。（原文第 4 页）

从自然语言推理勾勒形式化证明 大语言模型的近期进展带来了非形式化推理能力的重大突破。为弥合形式化推理与非形式化推理之间的鸿沟，我们利用前沿的通用 LLM（它们以强大的数学推理与指令遵循能力著称）来建立定理证明系统的基础框架。我们的发现表明，现成的模型（如 DeepSeek-V3（DeepSeek-AI，2024））能够分解证明步骤并用形式化语言表达它们。为证明给定的形式化定理陈述，我们提示 DeepSeek-V3 首先用自然语言分析数学问题，然后将证明分解为更小的步骤，并将每一步翻译为相应的 Lean 形式化语句。由于已知通用模型难以产出完整的 Lean 证明，我们指示 DeepSeek-V3 只生成省略细节的高层证明梗概。由此得到的思维链最终形成由一系列 have 语句构成的

Lean 定理，每个 have 语句均以 sorry 占位符结尾，表示一个待求解的子目标。这一做法与人类构造证明的风格相似：复杂定理被逐步归约为一系列更易处理的引理。

子目标的递归求解 利用 DeepSeek-V3 生成的子目标，我们采用递归求解策略来系统地解决每个中间证明步骤。我们从 have 语句中提取子目标表达式，用以替换给定问题中的原始目标（见图 3(a)），然后将先前的子目标作为前提纳入（见图 3(b)）。这种构造使后续子目标能够利用先前步骤的中间结果来求解，从而形成更局部化的依赖结构，并便于发展出更简单的引理。为降低大规模证明搜索的计算开销，我们采用一个专门针对分解后引理处理而优化的较小 7B 证明模型。当所有分解步骤都被成功求解后，即可自动导出原始定理的完整证明。

面向子目标定理证明的课程学习 证明模型的训练需要大规模的形式化语言问题集，这些数据集通常通过形式化现有的自然语言数学语料而得到（Xin 等，2024a；Ying 等，2024；Lin 等，2025）。尽管对人类撰写的文本进行形式化能提供高质量且多样的形式化内容，但由此为证明模型产生的训练信号往往很稀疏，因为很大一部分计算尝试都无法得到成功的证明，因而提供不了正向奖励信号。为产生更密集的训练信号，Dong 与 Ma（2025）提出了一种自博弈方法：通过生成与原定理陈述密切相关的、可求解的猜想，来丰富训练问题集，从而更高效地利用训练算力。在本文中，我们实现了一种直接的方法，利用子目标来扩展用于模型训练的形式化语句范围。我们生成两类子目标定理：一类将先前的子目标作为前提纳入，另一类则不纳入，分别对应图 3(b) 与图 3(a)。两类定理都被整合进专家迭代阶段（Polu 与 Sutskever，2020），从而建立一套课程，逐步引导证明模型系统地处理经过筛选的难题子集。该流程与 AlphaProof 的测试时强化学习（DeepMind，2024）基于相同的底层原理：生成目标问题的变体，以增强模型求解高难度 IMO 级问题的能力。

2.2 统一非形式推理与证明形式化

上述算法框架分两个阶段运行，利用两个互补的模型：DeepSeek-V3 负责引理分解，7B 证明模型负责补全相应的形式化证明细节。这一流程提供了一种自然且可扩展的机制，通过将语言模型的高层推理与精确的形式化验证相结合来合成形式化推理数据。通过这种方式，我们在单一模型内统一了非形式化数学推理与证明形式化的能力。

通过合成数据实现冷启动 我们筛选出一部分难题：7B 证明模型无法端到端地直接求解它们，但它们的全部子目标都已被成功求解。通过组合所有子目标的证明，我们为原始问题构造出完整的形式化证明。随后，该证明被附加到 DeepSeek-V3 概述相应引理分解的思维链之后，从而形成非形式化推理与后续形式化的连贯综合体。这使我们能够收集数百条高质量合成冷启动数据，作为训练 DeepSeek-Prover-V2 的基础。这种冷启动数据集生成策略与并行的形式化推理模型工作 Kimina-Prover

（Wang 等，2025）不同。具体而言，我们通过将自然语言证明直接形式化为结构化的形式化证明梗概来合成数据；相比之下，Kimina-Prover 采用反向流程：先收集完整形式化证明及其对应的非形式化证明，然后使用通用推理模型将中间的自然语言推理步骤逆向合成为连贯的思考块。

面向推理的强化学习 在合成冷启动数据上微调证明模型之后，我们执行强化学习阶段，进一步增强其衔接非形式化推理与形式化证明构造的能力。遵循面向推理模型的标准训练目标（DeepSeek-AI，2025），我们使用二值的正确/错误反馈作为奖励监督的主要形式。在训练过程中，我们观察到生成的证明结构经常偏离由思维链引导给出的引理分解。为解决这一问题，我们在训练的早期步骤中引入一致性奖励，对结构不对齐进行惩罚，从而显式地强制最终证明包含所有分解出的 have 结构引理。在实践中，强制这种对齐提升了证明准确率，尤其是在需要多步推理的复杂定理上。

2.3 DeepSeek-Prover-V2 的训练细节

两阶段训练：DeepSeek-Prover-V2 通过两阶段训练流程开发，建立了两种互补的证明生成模式：

- **高效非思维链（non-CoT）模式**：该模式针对快速生成 Lean 形式证明代码进行优化，专注于产出不含显式中间推理步骤的简洁证明。
- **高精度思维链（CoT）模式**：该模式在构造最终形式证明之前，系统地阐述中间推理步骤，强调透明度与逻辑递进。

与 DeepSeek-Prover-V1.5（Xin 等，2024b）一致，这两种生成模式由两种不同的引导提示控制（示例见附录 A）。在第一阶段，我们在课程学习框架内采用专家迭代来训练 non-CoT 证明器模型，同时通过基于子目标的递归证明为难题合成证明。选择 non-CoT 生成模式是为了加速迭代训练与数据收集过程，因为它的推理与验证周期明显更快。在此基础上，第二阶段利用冷启动思维链（CoT）数据，这些数据通过将 DeepSeek-V3 复杂的数学推理模式与我们的合成形式证明相结合而生成。CoT 模式通过进一步的强化学习阶段得到增强，遵循推理模型常用的标准训练流程。

专家迭代：DeepSeek-Prover-V2 的 non-CoT 模式训练过程遵循专家迭代（Polu 与 Sutskever，2020）范式，这是开发形式定理证明器时广泛采用的框架。在每一轮训练迭代中，使用当前最优的证明器策略为那些在先前迭代中仍未解决的难题生成证明尝试。那些通过 Lean 证明助手验证的成功尝试会被纳入 SFT 数据集，用于训练改进后的模型。这一迭代循环确保模型不仅从初始示例数据集中学习，还能蒸馏自身成功的推理轨迹，逐步提升求解更难问题的能力。整体训练流程与 DeepSeek-Prover-V1（Xin 等，2024a）和 DeepSeek-Prover-V1.5（Xin 等，2024b）基本一致，仅对训练问题分布做了两处修改。第一，我们纳入来自自动形式化以及多个开源数据集（Ying 等，2024；Dong 与 Ma，

2025；Lin 等，2025）的额外问题，扩展了训练问题领域的覆盖范围。第二，我们用通过子目标分解生成的问题来扩充数据集，目标是求解 MiniF2F 基准（Zheng 等，2022）valid 划分中更具挑战性的实例。

监督微调：我们在 DeepSeek-V3-Base-671B（DeepSeek-AI，2024）上进行监督微调，使用 5e-6 的恒定学习率，上下文窗口为 16,384 个 token。我们的训练语料由两个互补来源组成：（1）通过专家迭代收集的 non-CoT 数据，生成不含中间推理步骤的 Lean 代码；（2）第 2.2 节所述的冷启动 CoT 数据，将 DeepSeek-V3 的高级数学推理过程蒸馏为结构化的证明路径。non-CoT 部分强调 Lean 定理证明器生态中的形式验证技能，而 CoT 示例则显式建模了将数学直觉转化为形式证明结构的认知过程。

强化学习：我们采用组相对策略优化（GRPO；Shao 等，2024）作为强化学习算法，它在推理任务中已展现出更优的有效性与效率（DeepSeek-AI，2025）。与 PPO（Schulman 等，2017）不同，GRPO 无需单独的评论家模型，而是为每个定理提示采样一组候选证明，并基于它们的相对奖励优化策略。训练使用二值奖励：每个生成的 Lean 证明若被验证正确则获得奖励 1，否则为 0。为确保有效学习，我们精心筛选训练提示，仅包含足够有挑战性但仍可由监督微调模型解决的问题。在每轮迭代中，我们采样 256 个不同问题，为每个定理生成 32 个候选证明，最大序列长度为 32,768 个 token。

蒸馏：我们将 DeepSeek-Prover-V1.5-Base-7B（Xin 等，2024b）的最大上下文长度从 4,096 扩展到 32,768 个 token，并使用在 DeepSeek-Prover-V2-671B 强化学习阶段收集的 rollout 数据对该扩展上下文模型进行微调。除 CoT 推理模式外，我们还纳入专家迭代期间收集的 non-CoT 证明数据，以通过小规模模型提供一种成本高效的证明选项，产出简洁的形式化输出。此外，我们执行与 671B 模型训练中相同的强化学习阶段，以提升 DeepSeek-Prover-V2-7B 的性能。

3 实验结果

本节中，我们在形式定理证明的多个基准数据集上系统评估 DeepSeek-Prover-V2，涵盖高中竞赛问题与本科水平数学。DeepSeek-Prover-V2 的所有实验结果均使用 Lean 4.9.0-rc2、在 DeepSeek-Prover-V1.5（Xin 等，2024b）相同的测试环境下得到。若无特别说明，基线评估结果来自各自的原始论文。

3.1 MiniF2F 基准上的结果

MiniF2F（Zheng 等，2022）由 488 个形式化问题陈述组成，取材于包括 AIME、AMC 与 IMO 竞赛在内的多种数学材料，以及 MATH 数据集（Hendrycks 等，2021）中的精选问题。该基准包含奥赛级问题，覆盖初等数学的核心领域，包括代数、数论与归纳法。这些问题被划分为两个规模相等的子集，分别记为 miniF2F-valid 与 miniF2F-test，各包含 244 道题，且各学科领域的分布相同。我们保留 miniF2F-test 集专用于评估模型性能，而将 miniF2F-valid 问题纳入带子目标分解的课程学习。我们采用 Wang 等（2025）发布的修订版 miniF2F，并进一步对 miniF2F-valid 引入两处修订、对 miniF2F-test 引入一处修订（见附录 D）。

与最先进模型的比较：表 1 汇总了在 miniF2F-test 数据集上评估的最先进形式定理证明模型的比较。实验结果表明，DeepSeek-Prover-V2-671B 在 miniF2F-test 基准上确立了新的最先进性能：采用 CoT 生成策略时，仅用 32 个样本就达到了前所未有的 82.4% 准确率。值得注意的是，参数效率更高的 7B 版本 DeepSeek-Prover-V2-7B 也展现出有竞争力的性能，超过了文献中所有现有开源定理证明器。

【表格】表 1 与最先进模型在 miniF2F-test 数据集上的比较。记号 $\mu\pm\sigma$ 表示平均准确率 μ 与标准差 σ；CoT 与 non-CoT 标签指同一模型在两种不同提示引导下的生成模式（原文第 8 页）

对比分析进一步揭示了一个引人注目的扩展规律：随着样本预算从 1 增加到 8192，7B 与 671B 变体之间的性能差距显著拉大，更大的模型展现出更优的样本效率与更陡峭的提升轨迹。

通过子目标引导的课程学习求解难题：表 2 给出了 DeepSeek-Prover-V2 在 miniF2F 基准上所解决问题的详细分解，其在验证集上取得 90.6% 的通过率、在测试集上取得 88.9% 的通过率，整体表现强劲。值得注意的是，我们的子目标引导课程学习框架将通用模型 DeepSeek-V3 与轻量级专用 7B 证明器相结合，在 miniF2F-valid 上达到 89.8% 的成功率，几乎与 DeepSeek-Prover-V2-671B 的性能相当。这些发现凸显了最先进通用 LLM 的潜力：它们能够超越自然语言理解，有效支持复杂的形式推理任务。通过策略性的子目标分解，模型能够将难题拆解为一系列可处理的步骤，从而成为连接非形式化推理与形式证明构造的有效桥梁。

【表格】表 2 DeepSeek-Prover-V2-671B 在 miniF2F 基准上解决的问题。miniF2F-valid 的结果在整个课程学习过程中收集，其余问题则由 DeepSeek-Prover-V2-671B 以 Pass@8192 进一步求解（原文第 8 页）

CoT 与 non-CoT：表 1 的实验结果表明，在形式数学推理中，CoT 推理模式相较 non-CoT 模式具有显著性能优势。这进一步印证了 CoT 提示的有效性——它鼓励将复杂问题分解为中间步骤——并进一步证实推理时扩展（inference-time scaling）在形式定理证明领域同样成立。作为这些发现的补充，表 3 给出了 DeepSeek-Prover-V2 在不同推理模式下生成 token 数量的统计。正如预期，CoT 模式产生明显更长的输出，反映了其复杂的推理过程。有趣的是，在 non-CoT 设置下，671B 模型平均生成的输出比 7B 模型更长。进一步观察发现，尽管 non-CoT 模式并未提示显式推理，更大的模型仍经常在证明代码中插入简短的英文自然语言注释，这些注释类似于隐式的推理步骤（见附录 A）。这表明高容量模型可能隐含地内化并外化中间推理，即使没有显式的 CoT 提示。

表 3：DeepSeek-Prover-V2 在 miniF2F-test 上生成的平均 token 数量（#output tokens）。（原文第 9 页）

模型	non-CoT	CoT
7B	442.6	4488.5
671B	761.8	6751.9

3.2 本科水平基准上的结果

ProofNet（Azerbayev 等，2023）由 371 道 Lean 3 问题组成，取材于一系列流行的本科纯数学教材，涵盖实分析与复分析、线性代数、抽象代数与拓扑学等主题。我们使用 Xin 等（2024b）提供的 ProofNet 的 Lean 4 翻译版本，该版本进一步划分为两个划分：ProofNet-valid 与 ProofNet-test，分别包含 185 与 186 道题。ProofNet 的测试划分专用于模型评估，因为 ProofNet-valid 问题的变体被纳入 Dong 与 Ma（2025）提供的公开合成数据集，而该数据集用于我们的监督微调。如表 4 所示，结果表明，与 non-CoT 设置相比，DeepSeek-Prover-V2 在采用 CoT 推理时通过率有显著提升。值得注意的是，尽管训练数据主要来自高中水平的数学；

该模型对更高级别的、大学水平的数学问题也展现出很强的泛化能力，凸显了其稳健的形式化推理能力。

【表格】表 4 ProofNet-test 与 PutnamBench 上的实验结果。Goedel-Prover-SFT 与 STP 在 PutnamBench 上的分数取自其原始论文，这些论文在一个包含 644 道题的早期版本 PutnamBench 上进行了评测。（原文第 10 页）

PutnamBench（Tsoukalas 等，2024）是一个持续更新的基准，收录 1962 至 2023 年 William Lowell Putnam 数学竞赛的竞赛数学题。Putnam 竞赛是一项极具声望的年度数学竞赛，面向全美与加拿

大的本科生，涵盖分析、线性代数、抽象代数、组合数学、概率与集合论等多种大学水平领域。我们在 PutnamBench 的最新版本上评估模型，该版本包含 658 道以 Lean 4 形式化的题目。我们排除了与 Lean 4.9.0 不兼容的题目，在剩余的 649 道题上评估模型。在每道题 1024 的采样预算下，我们的初始运行解出了 49 道题。在将证明提交给基准维护者后，我们因陈述表述有误而排除了两道题。最终结果为成功解出 47 道题（如表 4 所示），显著优于其 non-CoT 版本。这些结果进一步凸显了 CoT 推理方法在处理具有挑战性的大学水平数学问题上的有效性。

强化学习中的奖励攻击（Reward Hacking） 我们的初始报告曾宣称一个意外发现：DeepSeek-Prover-V2-7B 在 PutnamBench 上成功解出了 13 道连其更大的 671B 版本都未能解出的题目。我们感谢 Lean 社区帮助查明这一异常结果的原因——该问题可追溯到 Lean 4.9.0 中的一个用户界面缺陷。具体而言，apply? tactic 在某些边界情形下未能生成 sorry 声明。在仔细检查模型输出后，我们在其推理方式中发现了一个独特模式：7B 模型频繁使用 Cardinal.toNat 与 Cardinal.natCast_inj 来利用这一用户界面缺陷（示例见附录 B），而 671B 版本生成的输出中明显没有这种模式。

3.3 组合问题的结果

【表格】表 5 CombiBench 的评测结果（with-solution 设置）（原文第 11 页）

CombiBench（Liu 等，2025）是一个综合性基准，包含 100 道以 Lean 4 形式化的组合竞赛题，每道题都配有对应的自然语言陈述。我们在该基准的 with-solution 设置下评估 DeepSeek-Prover-V2，即正确答案已嵌入 Lean 陈述之中，使评估可以专注于证明生成。在过滤掉与 Lean 4.9.0 不兼容的题目以及包含多个 sorry 占位符的题目后，我们在该基准的 77 道题上进行评估。我们的初始运行报告成功解出 12 道题。在与基准维护者合作并确认两道题的陈述表述有误后，最终结果为解出 10 道题。这些结果表明，尽管证明器模型主要在数论与代数方面训练，它在组合问题上也展现出可观的泛化能力，尽管该类问题本身持续困难。一个有趣的发现是，DeepSeek-Prover-V2-671B 的思维链推理能够有效识别表述错误，并相应调整其证明策略。我们在附录 C 中给出一个示例：模型在表述有误的陈述中检测到矛盾，并通过 exfalso 推导出 False 来完成目标。

3.4 FormalMATH 上的结果

【表格】表 6 FormalMATH-All/Lite 的评测结果。DeepSeek-Prover-V2 的性能以 CoT 生成模式评测。（原文第 11 页）

FormalMATH（Yu 等，2025）是一个大规模形式化定理证明基准，包含 5560 道题，从高中数学奥林匹克题目到本科水平的定理，覆盖代数、应用数学、微积分、数论与离散数学等广泛领域。FormalMATH-Lite 是从完整语料中精心挑选的、规模适中的 425 道题的子集（包含 359 道高中水平题目与 66 道本科水平题目），以应对在完整数据集上评估不切实际的问题，同时仍能系统评估跨各数学领域的测试时扩展（test-time scaling）。表 6 给出了各类定理证明器在 FormalMATH-All 与 FormalMATH-Lite 两个基准上的性能对比分析。DeepSeek-Prover-V2-671B 在所有评估指标上均表现更优：在 32 个样本下，它在完整数据集中成功解出 28.31% 的题目，在 Lite 子集中解出 56.00% 的题目；当计算预算扩大到 3200 个样本时，Lite 子集上的性能显著提升至 61.88%。DeepSeek-Prover-V2 的 7B 参数版本同样表现出显著的有效性，优于此前的基线，包括 Goedel-Prover-SFT（Lin 等，2025）、STP（Dong 和 Ma，2025）以及 Kimina-Prover-Preview-Distill-7B（Wang 等，2025）。

【表格】表 7 ProverBench 上的实验结果。All 类别表示由 325 道题构成的完整评测集，AIME 24&25 表示由近期 AIME 竞赛形式化而来的 15 道题的子集。STP（Dong 和 Ma，2025）的结果使用开源模型权重评测。（原文第 12 页）

3.5 ProverBench：AIME 与教材问题的形式化

为增强现有基准并推动形式化定理证明研究，我们提出了一个包含 325 道题目的基准数据集。其中 15 道由近期 AIME 竞赛（AIME 24 与 AIME 25）中的数论与代数题目形式化而来，提供真实的高中竞赛级挑战。其余 310 道题取自精心挑选的教材例题与教学教程，构成一个多样且具有教学基础的、形式化数学问题集合。该基准旨在实现对高中竞赛题与本科水平数学的更全面评估。

竞赛	题目
AIME 24I	P2、P7、P13
AIME 24II	P4、P7、P13、P14
AIME 25I	P1、P8、P9、P11
AIME 25II	P2、P4、P13、P15

表 8 | 为形式化而选取的 AIME 24&25 题目。题号以下划线加粗标注者已被 DeepSeek-Prover-V2 解出；以灰色背景高亮者已被 DeepSeek-V3-0324 在 Maj@16 下解出。

AIME 形式化 美国数学邀请赛（American Invitational Mathematics Examination，AIME）是一项年度数学竞赛，旨在挑战并表彰在数学方面表现出卓越才华的高中生。AIME 24&25 的题目已成为评估

大语言模型推理能力的标准基准。为打通形式化与非形式化数学推理之间模型性能的评估，我们从 AIME 24&25 中挑选并形式化了一部分题目。为确保形式化更干净，我们过滤掉了几何、组合与计数类题目，因为它们在 Lean 中的表示可能较为繁琐。这最终得到 15 道选定的题目，涵盖初等数论与代数方面的竞赛级主题。我们使用标准的“求解答案”（find-answer）任务，在自然语言数学推理设置下，对选定的题目评估 DeepSeek-V3-0324。在 16 个采样响应上进行多数投票后，该模型成功解出 15 道中的 8 道。相比之下，DeepSeek-Prover-V2-671B 在给定正确答案的形式化证明生成设置下，能够为 15 道题中的 6 道构造出有效的形式化证明。这一对比凸显出：非形式化数学推理与形式化定理证明之间的性能差距正在显著缩小，表明先进语言模型在语言理解与形式化逻辑严谨性之间日益对齐。

【表格】表 9 ProverBench 中各数学领域的分布。（原文第 13 页）

教材问题形式化 除 AIME 24&25 之外，我们还用从高中竞赛与本科课程所用教材中精心挑选的题目来扩充基准，以加强对特定数学领域的覆盖。这一整理过程确保在难度层次与主题领域上都有全面的代表性。最终，我们形式化了 310 道题，涵盖从竞赛水平的初等数学到本科阶段通常接触的高等主题的广泛范围。这一综合基准覆盖数论、初等代数、线性代数、抽象代数、微积分、实分析、复分析、泛函分析与概率。刻意纳入这一多样的数学领域，使得能够全面评估模型在不同抽象层次与推理风格下的能力。数论与代数题目考察模型处理离散结构与方程的能力，而面向分析的题目则评估对极限、连续性与微积分的理解。抽象代数与泛函分析部分要求模型对抽象结构与空间进行推理，需要复杂的形式化推理能力。评估结果见表 7。结果显示，采用 CoT 推理的 DeepSeek-Prover-V2-671B 始终优于所有基线，进一步印证了在其他基准评估中观察到的趋势。

4 结论

在本工作中，我们提出了一套用于合成冷启动推理数据的综合流水线，以推进形式化定理证明。我们的数据构建过程基于一个递归定理证明框架：在该框架中，DeepSeek-V3 作为统一模型，在 Lean 4 证明助手中同时承担子目标分解与引理形式化两项任务。我们的方法将高层证明草图与形式化步骤相结合，把证明拆解为一系列可管理的子目标，这些子目标可以用更小的 7B 模型高效求解，从而显著降低计算需求。我们开发的课程学习框架利用这些分解后的子目标生成难度递增的训练任务，形成更有效的学习进阶。通过将完整的形式化证明与 DeepSeek-V3 的思维链推理相配对，我们建立了宝贵的冷启动推理数据，将非形式化数学思考与形式化证明结构连接起来。随后的强化学习阶段显著增强了这一连接，使形式化定理证明能力得到大幅提升。由此得到的模型 DeepSeek-Prover-V2-671B 在涵盖

高中竞赛题与本科水平数学的一系列基准上始终优于所有基线。未来的工作将聚焦于将这一范式扩展到类似 AlphaProof 的系统，最终目标是攻克代表自动定理证明挑战前沿的 IMO 级数学问题。

附录

附录 A.1 非 CoT 生成

附录 A.2 CoT 生成策略

附录 B 触发 BlueViolet 界面 bug 的 Buggy 证明示例

附录 C 使用 BlueViolet.exfalso 证明空真命题的示例

附录 D MiniF2F 修订

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] Y. Li, D. Du, L. Song, C. Li, W. Wang, T. Yang, and H. Mi. Hunyuanprover: A scalable data synthesis framework and guided tree search for automated theorem proving. arXiv preprint arXiv:2412.20735, 2024.

[2] Y. Lin, S. Tang, B. Lyu, J. Wu, H. Lin, K. Yang, J. Li, M. Xia, D. Chen, S. Arora, et al. Goedel- Prover: A frontier model for open-source automated theorem proving. arXiv preprint arXiv:2502.07640, 2025.

[3] J. Liu, X. Lin, J. Bayer, Y. Dillies, W. Jiang, X. Liang, R. Soletskyi, H. Wang, Y. Xie, B. Xiong, et al. CombiBench: Benchmarking llm capability for combinatorial mathematics. arXiv preprint arXiv:2505.03171, 2025.

[4] L. d. Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In Automated Deduction–CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings 28, pages 625–635. Springer, 2021.

[5] O. Nachum, S. S. Gu, H. Lee, and S. Levine. Data-efficient hierarchical reinforcement learning. Advances in neural information processing systems, 31, 2018.

[6] Isabelle a Generic Theorem Prover. Springer Verlag, 1994. L. C. Paulson.

[7] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. arXiv preprint arXiv:2009.03393, 2020.

[8] J. Schulman, F. Wolski, P . Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

[9] Z. Shao, P . Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Li, Y. Wu, and D. Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300, 2024.

[10] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. PutnamBench: Evaluating neural theorem-provers on the putnam mathematical competition. In The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2024.

[11] H. Wang, H. Xin, Z. Liu, W. Li, Y. Huang, J. Lu, Y. Zhicheng, J. Tang, J. Yin, Z. Li, et al. Proving theorems recursively. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024a.

[12] H. Wang, H. Xin, C. Zheng, Z. Liu, Q. Cao, Y. Huang, J. Xiong, H. Shi, E. Xie, J. Yin, et al. Lego-prover: Neural theorem proving with growing libraries. In The Twelfth International Conference on Learning Representations, 2024b.

[13] H. Wang, M. Unsal, X. Lin, M. Baksys, J. Liu, M. D. Santos, F. Sung, M. Vinyes, Z. Ying, Z. Zhu, et al. Kimina-Prover Preview: Towards large formal reasoning models with reinforcement learning. arXiv preprint arXiv:2504.11354, 2025.

[14] Z. Wu, S. Huang, Z. Zhou, H. Ying, J. Wang, D. Lin, and K. Chen. Internlm2. 5-stepprover: Advancing automated theorem proving via expert iteration on large-scale lean problems. arXiv preprint arXiv:2410.15700, 2024.

[15] H. Xin, D. Guo, Z. Shao, Z. Ren, Q. Zhu, B. Liu, C. Ruan, W. Li, and X. Liang. DeepSeek- Prover: Advancing theorem proving in llms through large-scale synthetic data. arXiv preprint arXiv:2405.14333, 2024a. H. Xin, Z. Ren, J. Song, Z. Shao, W. Zhao, H. Wang, B. Liu, L. Zhang, X. Lu, Q. Du, et al. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and monte-carlo tree search. arXiv preprint arXiv:2408.08152, 2024b.

[16] R. Xin, C. Xi, J. Yang, F. Chen, H. Wu, X. Xiao, Y. Sun, S. Zheng, and K. Shen. BFS-Prover: Scalable best-first tree search for llm-based automatic theorem proving. arXiv preprint arXiv:2502.03438, 2025.

[17] K. Yang, G. Poesia, J. He, W. Li, K. Lauter, S. Chaudhuri, and D. Song. Formal mathematical reasoning: A new frontier in AI. arXiv preprint arXiv:2412.16075, 2024.

[18] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen. Lean workbook: A large-scale lean problem set formalized from natural language math problems. In The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2024.

[19] Z. Yu, R. Peng, K. Ding, Y. Li, Z. Peng, M. Liu, Y. Zhang, Z. Yuan, H. Xin, W. Huang, et al. Formalmath: Benchmarking formal mathematical reasoning of large language models. arXiv preprint arXiv:2505.02735, 2025.

[20] J. Zhang, Q. Wang, X. Ji, Y. Liu, Y. Yue, F. Zhang, D. Zhang, G. Zhou, and K. Gai. Leanabell-prover: Posttraining scaling in formal reasoning. arXiv preprint arXiv:2504.06122, 2025.

[21] X. Zhao, W. Li, and L. Kong. Decomposing the enigma: Subgoal-based demonstration learning for formal theorem proving. arXiv preprint arXiv:2305.16366, 2023.

[22] X. Zhao, L. Zheng, H. Bo, C. Hu, U. Thakker, and L. Kong. Subgoalxl: Subgoal-based expert learning for theorem proving. arXiv preprint arXiv:2408.11172, 2024.

[23] C. Zheng, H. Wang, E. Xie, Z. Liu, J. Sun, H. Xin, J. Shen, Z. Li, and Y. Li. Lyra: Orchestrating dual correction in automated theorem proving. Transactions on Machine Learning Research, 2024.

[24] K. Zheng, J. M. Han, and S. Polu. miniF2F: a cross-system benchmark for formal olympiad-level mathematics. In International Conference on Learning Representations, 2022.

深入理解 DeepSeek-V3：扩展挑战与面向 AI 架构的硬件思考

Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures

DeepSeek-AI research@deepseek.com

arXiv:2505.09343 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

大语言模型（LLM）的快速扩展揭示了当前硬件架构中的关键局限，包括内存容量、计算效率与互连带宽方面的约束。在 2,048 块 NVIDIA H800 GPU 上训练的 DeepSeek-V3 表明，硬件感知的模型协同设计能够有效应对这些挑战，从而在规模化场景下实现高性价比的训练与推理。本文深入分析了 DeepSeek-V3/R1 的模型架构及其 AI 基础设施，重点介绍了多项关键创新：提升内存效率的多头潜在注意力（MLA）、优化计算-通信权衡的混合专家（MoE）架构、充分释放硬件能力潜力的 FP8 混合精度训练，以及旨在最大限度降低……的多平面网络拓扑（Multi-Plane Network Topology）。

1 引言

1.1 背景

大语言模型（LLM）近年来经历了快速演进，其驱动力来自模型设计、算力与数据可得性的持续进步。2024 年，GPT4o [59]、LLaMa-3 [3]、Claude 3.5 Sonnet [8]、Grok-2 [74]、Qwen2.5 [71]、Gemini-2 [37] 以及我们的 DeepSeek-V3 [26] 等开创性模型展现出显著进展，进一步缩小了与通用人工智能（AGI）之间的差距。正如 Scaling Laws [45] 所示，增大模型规模、训练数据与计算资源会带来模型性能的大幅提升，凸显了扩展在推进 AI 能力中的关键作用。总体而言，这些发展开启了一个新时代：扩展模型规模与算力被视为解锁更高层次智能的关键。

近期，OpenAI 的 o1/o3 系列模型 [60, 61]、DeepSeek-R1 [28]、Claude-3.7 Sonnet [9]、Gemini 2.5 Pro [38]、Seed1.5-Thinking [68] 与 Qwen3 [72] 等推理模型的进展表明：大规模架构不仅带来了收益，也说明提升推理效率十分必要，尤其是在处理更长上下文与获得更深推理深度方面。这些进展凸显了对更快、更高效推理的需求，进而对计算资源提出了越来越高的要求。

为应对这些挑战，Alibaba、ByteDance、Google、xAI 与 Meta 等行业领军者已部署了庞大的训练集群 [33, 42, 43, 56, 62, 75]，其 GPU 或 TPU 数量达到数万甚至数十万。虽然如此庞大的基础设施支撑了最先进模型的研发，但其高昂成本为较小的研究团队与组织设置了显著的门槛。尽管存在这些门槛，DeepSeek [23–26, 28] 与 Mistral [41, 55] 等开源初创公司也在努力开发最先进的模型。其中，DeepSeek 尤其证明了有效的软硬件协同设计能够实现大模型的低成本训练，从而缩小了小型团队与大型机构之间的差距。

承接这一传统，DeepSeek-V3 [26] 代表了低成本训练的一个新里程碑。仅使用 2,048 块 NVIDIA H800 GPU，DeepSeek-V3 就达到了最先进的性能。这一成果与通过实用且可扩展的方案推进 AI 的承诺相一致，正如此前 Fire-Flyer AI-HPC [7] 的低成本架构所展示的那样。DeepSeek-V3 的实践与洞见展示了如何最大限度地挖掘现有硬件资源的潜力，为更广泛的 AI 与 HPC 社区提供了宝贵的经验。

1.2 目标

本文无意重复 DeepSeek-V3 的详细架构与算法细节，这些内容已在其技术报告 [26] 中详尽记录。相反，本文采取硬件架构与模型设计的双重视角，探索二者在实现低成本大规模训练与推理中的复杂相互作用。通过考察这种协同关系，我们旨在为在不牺牲性能与可及性的前提下高效扩展 LLM 提供可操作的洞见。

具体而言，本文聚焦于：

- **硬件驱动的模型设计（Hardware-Driven Model Design）**：分析 FP8 低精度计算、scale-up/scale-out 网络特性等硬件特征如何影响 DeepSeek-V3 的架构选择。
- **硬件与模型之间的相互依赖（Mutual Dependencies Between Hardware and Models）**：考察硬件能力如何塑造模型创新，以及 LLM 不断演变的需求如何驱动对下一代硬件的需要。
- **硬件开发的未来方向（Future Directions for Hardware Development）**：从 DeepSeek-V3 中提炼可操作的洞见，以指导未来硬件与模型架构的协同设计，为可扩展、低成本的 AI 系统铺平道路。

1.3 本文结构

本文其余部分组织如下。第 2 节探讨支撑 DeepSeek-V3 模型架构的设计原则，重点介绍 Multi-head Latent Attention、Mixture-of-Experts 优化与 Multi-Token Prediction 模块等关键创新。第 3 节说明我们的模型架构如何追求低精度计算与通信。第 4 节包含 scale-up 互联优化，讨论 scale-up/scale-out 的融合，并探索硬件特性如何影响并行与专家选择策略。第 5 节聚焦 scale-out 网络优化，包括多平面网络协同设计与低延迟互联。除第 3～5 节提到的现有局限与未来建议之外，第 6 节进一步阐述从 DeepSeek-V3 中得到的更关键洞见，并指出未来硬件与模型协同设计的方向。

2 DeepSeek 模型的设计原则

DeepSeek-V3 的研发体现了面向硬件的 LLM 扩展方法：每一项设计决策都与硬件约束仔细对齐，以优化性能与成本效益。

如图 1 所示，DeepSeek-V3 采用了 **DeepSeekMoE** [27] 与 **Multi-head Latent Attention（MLA）** [25] 架构，这些架构已在 DeepSeek-V2 [25] 中被证明有效。DeepSeekMoE 释放了 MoE 架构的潜力，而 MLA 通过压缩键值（Key-Value, KV）缓存大幅降低内存消耗。此外，**DeepSeek-V3** 引入 **FP8 混合精度训练**，显著降低了计算成本，使大规模训练在保证模型质量的同时更具可行性。为提升推理速度，DeepSeek-V3 基于其 **Multi-Token Prediction 模块**集成了投机解码（speculative decoding），显著提高了生成速度。除模型架构之外，我们还探索了低成本的 AI 基础设施：部署 **多平面（Multi-Plane）** 两层 Fat-Tree 网络以替代传统的三层 Fat-Tree 拓扑，从而降低集群组网成本。

这些创新旨在应对扩展 LLM 的三个核心挑战——**内存效率**、**成本效益**与**推理速度**，以下各小节将逐一深入探讨。

2.1 内存效率

LLM 通常需要大量内存资源，且内存需求每年增长超过 1000%。相比之下，高速内存（如 HBM）容量的增长速度则慢得多，通常每年不到 50% [35]。虽然多节点并行是应对内存限制的可行方案，但从源头优化内存使用仍然是一项关键而有效的策略。

【图表】图 1 DeepSeek-V3 的基础架构。在 DeepSeek-V2 的 MLA 与 DeepSeekMoE 之上，引入了 Multi-Token Prediction 模块与 FP8 混合精度训练，以提升推理与训练效率；图中标注了架构不同部分计算所使用的精度，所有组件的输入与输出均为 BF16（原文第 3 页）

2.1.1 低精度模型

与使用 BF16 权重的模型相比，FP8 将内存消耗显著减少一半，从而有效缓解 AI 内存墙挑战。关于低精度技术的详细讨论见第 3 节“Low-Precision Driven Design”。

2.1.2 用 MLA 减小 KV 缓存

对于 LLM 推理而言，用户请求往往涉及多轮对话。为高效处理这些请求，先前请求的上下文会被缓存起来，即通常所说的 **KV 缓存**。KV 缓存通过缓存已处理 token 的 **Key** 与 **Value** 向量来应对这一挑战，从而无需为后续 token 重新计算它们。在每一步推理中，模型只计算当前 token 的 Key 与 Value 向量，并将它们与历史缓存的 Key-Value 对结合，完成注意力计算。这种增量式计算将生成每个 token 的复杂度降至 O(N)，使其在处理长序列或多轮输入时非常高效。然而，它也带来了内存受限的瓶颈，因为计算从 GEMM 转变为 GEMV，而 GEMV 的计算/内存访问比要低得多。在现代硬件可提供数百 TFLOPS 算力的情况下，GEMV 很快受限于内存带宽，使内存访问成为主要瓶颈。

为解决这一瓶颈，我们采用 **Multi-head Latent Attention（MLA）** [25]，它通过一个与模型联合训练的投影矩阵，将所有注意力头的 KV 表示压缩为一个更小的潜在向量。推理期间只需缓存该潜在向量，相比为所有注意力头存储 KV 缓存，内存消耗显著降低。

除 MLA 之外，还有若干其他方法被提出以减小 KV 缓存的大小。这些方法极具价值，为内存高效注意力机制的进步提供了重要启发：

- **共享 KV（Grouped-Query Attention, GQA；Multi-Query Attention, MQA）**：与其为每个注意力头维护独立的 KV 对，不如让多个头共享同一组 KV 对，从而显著压缩 KV 存储。代表性方法包括 GQA [5] 与 MQA [70]。
- **窗口化 KV（Windowed KV）**：对于长序列，缓存中只保留一个滑动窗口内的 KV 对，丢弃窗口之外的结果。这虽然减少了存储，但会损害长上下文推理能力。代表性方法包括 Longformer [11] 及相关架构。
- **量化压缩（Quantized Compression）**：KV 对以低位表示存储 [40, 44, 52]，进一步降低内存占用。量化能够在对模型性能影响极小的前提下实现显著压缩。

表 1：KV 缓存大小对比（BF16 精度）：相比使用 GQA 的其他模型，DeepSeek-V3（MLA）大幅减小了 KV 缓存大小（原文第 4 页）

模型	每 token KV 缓存	倍数

DeepSeek-V3（MLA）	70.272 KB	1x
Qwen-2.5 72B（GQA）	327.680 KB	4.66x
LLaMA-3.1 405B（GQA）	516.096 KB	7.28x

表 1 比较了 DeepSeek-V3、Qwen-2.5 72B [71] 与 LLaMA-3.1 405B [4] 每 token 的 KV 缓存内存占用。通过采用 MLA，DeepSeek-V3 显著减小了 KV 缓存，每 token 仅需 70 KB，远低于 LLaMA-3.1 405B 的 516 KB 与 Qwen-2.5 72B 的 327 KB。这一降幅凸显了 MLA 相比基于 GQA 的方法在压缩 KV 表示方面的效率。能够实现如此显著的内存消耗降低，使 DeepSeek-V3 特别适合长上下文处理与资源受限环境，从而实现更具可扩展性与成本效益的推理。

2.1.3 资源高效技术的未来方向与展望

尽管减小 KV 缓存大小是提升内存效率的一条有前景的路径，Transformer 自回归解码固有的二次复杂度仍是一项艰巨挑战，对超长上下文尤其如此。近期研究，例如 Mamba-2 [21] 与 Lightning Attention [63]，探索了线性时间的替代方案，为平衡计算成本与模型性能提供了新的可能。此外，稀疏注意力 [76] 等方法试图对注意力的键与值进行压缩与稀疏激活，代表克服注意力相关计算挑战的另一种尝试。我们期待与更广泛的社区合作，在这一领域取得突破。

2.2 MoE 模型的成本效益

在稀疏计算方面，我们开发了 DeepSeekMoE——一种先进的 **Mixture of Experts（MoE）** 架构，如图 1 右下方所示。MoE 模型的优势体现在两个方面。

2.2.1 降低训练的计算需求

MoE 架构的首要优势在于其显著降低训练成本的能力。通过只选择性地激活一部分专家参数，MoE 模型得以让总参数量大幅扩展，同时保持计算需求适中。例如，**DeepSeek-V2** 拥有 236B 参数，但每个 token 仅激活 21B 参数。类似地，**DeepSeek-V3** 扩展到 671B 参数——几乎是 V2 的三倍——而每个 token 的激活参数仅为 37B。相比之下，Qwen2.5-72B 与 LLaMa3.1-405B 等稠密模型在训练时需要激活全部参数。

表 2：训练 MoE 模型与稠密模型的计算成本对比：以每 token 的计算成本度量，假设序列长度为 4096（原文第 4 页）

模型	规模	训练成本
DeepSeek-V2 MoE	236B	155 GFLOPS/Token
DeepSeek-V3 MoE	671B	250 GFLOPS/Token
Qwen-72B Dense	72B	394 GFLOPS/Token
LLaMa-405B Dense	405B	2448 GFLOPS/Token

如表 2 所示，DeepSeek-V3 的总计算成本约为每 token 250 GFLOPS，而 72B 稠密模型需要 394 GFLOPS，405B 稠密模型需要 2448 GFLOPS。这表明 MoE 模型在消耗少一个数量级的计算资源的同时，取得了与稠密模型相当甚至更优的性能。

2.2.2 个人使用与本地部署的优势

在个性化 LLM 智能体 [53] 日益普及的未来，MoE 模型在单请求场景中具有独特优势。由于每次请求只激活一部分参数，内存与计算需求大幅降低。例如，**DeepSeek-V2**（236B 参数）在推理时仅激活 21B 参数。这使搭载 AI SoC 芯片 [6, 10, 58] 的 PC 能够达到接近 20 token 每秒（TPS）的速度，甚至可达其两倍，足以满足个人使用。相比之下，能力相近的稠密模型（例如 70B 参数）在类似硬件上通常只能达到个位数的 TPS。

值得注意的是，日益流行的 KTransformers [39] 推理引擎使完整的 DeepSeek-V3 模型能够在一台配备消费级 GPU 的低成本服务器（约 10,000 美元）上运行，同时仍可达到接近 20 TPS。

这一效率使 MoE 架构适合本地部署与单用户场景——这些场景中硬件资源往往有限。通过最小化内存与计算开销，MoE 模型无需昂贵的基础设施即可提供高质量的推理性能。

2.3 提升推理速度

2.3.1 重叠计算与通信：最大化吞吐

推理速度既包括系统级的最大吞吐，也包括单请求延迟。为最大化吞吐，我们的模型从一开始就设计为利用双 micro-batch 重叠（dual micro-batch overlap）[31, 79]，有意将通信延迟与计算重叠。正如我们的在线推理系统所展示、并得到开源性能剖析数据 [31] 支持的，我们将 MLA 与 MoE 的计算解耦为两个不同阶段。当一个 micro-batch 执行部分 MLA 或 MoE 计算时，另一个 micro-batch 同时执行相应的 dispatch 通信；反之，在第二个 micro-batch 的计算阶段，第一个 micro-batch 执行 combine 通信步骤。这种流水线式方法使 all-to-all 通信与持续进行的计算无缝重叠，确保 GPU 始终得到充分利用。此外，在生产环境中，我们采用 prefill 与 decode 分离（disaggregation）的架构 [81]，将大批量

的 prefill 请求与对延迟敏感的 decode 请求分配给不同的专家并行组规模。这一策略最终在真实服务条件下最大化了系统吞吐。

2.3.2 推理速度上限

本节聚焦 LLM 服务的 decode 输出速度，通常以 **Time Per Output Token（TPOT）** 度量。TPOT 是影响用户体验的关键指标，也直接影响 OpenAI 的 o1/o3 与 DeepSeek-R1 等推理模型的响应能力——这些模型依赖推理长度来增强其智能。

对于 MoE 模型，实现高推理速度依赖于将专家参数高效地部署到各计算设备上。为达到尽可能快的推理速度，理想情况下每台设备只负责一个专家的计算（必要时可由多台设备协作计算单个专家）。然而，**Expert Parallelism（EP）** 需要将 token 路由到相应的设备，这涉及跨网络的 all-to-all 通信。因此，MoE 推理速度的上限由互联带宽决定。

考虑一个系统：每台设备持有一个专家的参数，并一次处理约 32 个 token。这一 token 数量在计算/内存访问比与通信延迟之间取得平衡，同时保证在专家并行时每台设备处理相同的批大小，从而使通信时间易于计算。

对于采用 CX7 400Gbps InfiniBand（IB）网卡互联的系统，EP 中两次 all-to-all 通信所需的时间计算如下：

Comm. Time = (1Byte + 2Bytes) × 32 × 9 × 7K / 50GB/s = 120.96 μs

其中 dispatch 使用 FP8（1 字节），combine 使用 BF16（2 字节），每个 token 的隐藏维度约为 7K。系数 9 表示每个 token 被传输到 8 个路由专家与 1 个共享专家。该计算未包含网络延迟。

如第 2.3.1 节所述，最大化吞吐需要使用双 micro-batch 重叠。在这一策略下，我们的理论最佳情况分析假设计算开销被最小化，因此性能上界由通信延迟决定。然而在实际推理工作负载中，请求上下文往往长得多，MLA 计算通常主导执行时间。因此，这一分析代表双 micro-batch 重叠下的理想化场景。在该假设下，每层总时间可表示为：

Total Time Per Layer = 2 × 120.96 μs = 241.92 μs

DeepSeek-V3 共有 61 层，因此总推理时间为：

Total Inference Time = 61 × 241.92 μs = 14.76 ms

因此，该系统的理论上限约为 **14.76 ms TPOT**，相当于 **67 token 每秒**。然而在实践中，通信开销、延迟、带宽利用不充分以及计算效率损失等因素都会使这一数字下降。

相比之下，若采用 GB200 NVL72 这类高带宽互联（72 块 GPU 之间 900GB/s 单向带宽），每个 EP 步骤的通信时间将降至：

Comm. Time = (1Byte + 2Bytes) × 32 × 9 × 7K / 900GB/s = 6.72 μs

假设计算与通信完全重叠，这将带来 **0.82 ms TPOT** 的理论上限，即约 **1200 token 每秒**。然而，这一数字纯属理论，并未考虑小批大小下 GPU 效率的大幅下降；在实际部署中，真实吞吐会显著更低。尽管如此，这一计算生动地说明了高带宽 scale-up 网络在加速大规模模型推理方面的变革潜力。

虽然 MoE 模型展现出良好的可扩展性，但仅靠增加硬件资源来实现高推理速度在经济上难以承受。因此，软件与算法也必须为提升推理效率作出贡献。

2.3.3 多 token 预测

受 Gloeckle et al. [36] 启发，DeepSeek-V3 引入了 **Multi-Token Prediction（MTP）** 框架，它在增强模型性能的同时提升推理速度。推理时，传统的自回归模型在每个解码步骤只生成一个 token，导致串行瓶颈。MTP 通过让模型以更低的成本生成额外的候选 token 并并行验证它们来缓解这一问题，这与以往基于自起草（self-drafting）的投机解码方法 [14, 48] 类似。该框架在不牺牲准确性的前提下显著加速推理。

如图 1 上半部分所示，每个 MTP 模块仅使用一层（远轻量于完整模型）来预测额外 token，从而能够并行验证多个候选 token。尽管这会略微损害吞吐，但该方法显著改善了端到端生成延迟。真实实践数据表明，MTP 模块在预测第二个后续 token 时达到 80% 至 90% 的接受率，与没有 MTP 模块的场景相比，生成 TPS 提升了 1.8 倍。

此外，通过每步预测多个 token，MTP 增大了推理批大小，这对提升 EP 的计算强度与硬件利用率至关重要。此类算法创新对于 DeepSeek-V3 实现快速且低成本的推理不可或缺。

2.3.4 推理模型的高推理速度与测试时扩展

以 OpenAI 的 o1/o3 系列 [60, 61] 为代表，LLM 的测试时扩展（test-time scaling）通过在推理期间动态调整计算资源，在数学推理、编程与通用推理方面取得了显著进展。随后的模型——包括 DeepSeek-R1 [28]、Claude-3.7 Sonnet [9]、Gemini 2.5 Pro [38]、Seed1.5-Thinking [68] 与 Qwen3 [72]——采用了类似策略，并在这些任务上取得了显著提升。

对于这些推理模型而言，高 token 输出速度至关重要。在强化学习（RL）工作流（例如 PPO [67]、DPO [64] 与 GRPO [69]）中，快速生成大量样本的必要性使推理吞吐成为关键瓶颈。同样，过长的推理序列会增加用户等待时间，降低此类模型的实用可用性。因此，通过硬件与软件的协同创新

来优化推理速度，对于提升推理模型的效率不可或缺。然而，加速推理与加快 RL 训练的有效策略仍是活跃的研究领域，如第 2.1.3 节所述。我们鼓励更广泛的社区共同探索并开发应对这些持续挑战的新方案。

2.4 技术验证方法

每项加速技术都要经过严格的实证验证，以评估其对精度的影响，包括 MLA、FP8 混合精度计算以及网络协同设计的 MoE 门控路由。鉴于在全规模模型上进行穷尽式消融实验的成本过高，我们采用一种分层且资源高效的验证流程：每项技术首先在小规模模型上充分验证，随后进行最小限度的大规模调优，最后集成到一次全面的训练运行中。例如，在最终集成之前，我们首先在 16B 与 230B 的 DeepSeek-V2 模型上进行了细粒度 FP8 训练的消融研究。在这些受控设置下，相对于 BF16 的相对精度损失保持在 0.25% 以下，这得益于我们采用的高精度累加与细粒度量化策略。

3 低精度驱动的设计

3.1 FP8 混合精度训练

GPTQ [32] 与 AWQ [51] 等量化技术已被广泛用于将位宽降至 8 位、4 位甚至更低，从而显著降低内存需求。然而，这些技术主要在推理阶段用于节省内存，而非用于训练阶段。NVIDIA 的 Transformer Engine 支持 FP8 混合精度训练已有一段时间，但在 DeepSeek-V3 之前，尚无开源大模型利用 FP8 进行训练。通过我们的基础设施团队与算法团队的深度协作，并经过大量实验与创新，我们为 MoE 模型开发了一套兼容 FP8 的训练框架。图 1 展示了训练流程中使用 FP8 精度前向与反向过程的计算组件。我们应用了细粒度量化，即对激活采用 tile 级 1×128 量化，对模型权重采用 block 级 128×128 量化。关于我们 FP8 框架的更多技术细节记录在 DeepSeek-V3 技术报告 [26] 中，我们的细粒度 FP8 GEMM 实现已在 DeepGEMM [78] 中开源。

3.1.1 局限性（Limitations）：尽管 FP8 在加速训练方面潜力巨大，但要充分发挥其能力，仍需解决若干硬件限制：

- **FP8 累加精度（FP8 Accumulation Precision）**：FP8 在 Tensor Core 中使用受限的累加精度，影响大模型训练的稳定性，在 NVIDIA Hopper GPU 上尤为如此。在根据最大指数对 32 个尾数乘积进行右移对齐之后，Tensor Core 仅保留它们最高的 13 位尾数位用于加法，并截断超出该范围的

位。加法结果累加到 FP22 寄存器（1 位符号、8 位指数、13 位尾数）[77]。“FP22”这一术语沿用被引工作中的命名，而 Hopper GPU 上的 FP8 精度问题在 2024 年初也已被业界广泛观察到。

- **细粒度量化的挑战（Fine-Grained Quantization Challenges）**：tile 级与 block 级等细粒度量化在将部分结果从 Tensor Core 传输到 CUDA Core 以乘以 scaling factor 时会引入较大的反量化开销。这会造成频繁的数据移动，降低计算效率并使硬件利用变得复杂。

3.1.2 建议（Suggestions）：为解决现有硬件的局限性，我们对未来设计提出以下建议：

- **提高累加精度（Increased Accumulation Precision）**：硬件应将累加寄存器精度提高到合适的值（例如 FP32），或支持可配置的累加精度，从而针对不同模型在训练与推理上的不同需求，在性能与精度之间进行权衡。
- **原生支持细粒度量化（Native Support for Fine-Grained Quantization）**：硬件应原生支持细粒度量化，使 Tensor Core 能够接收 scaling factor 并执行带分组缩放的矩阵乘法。这样，整个部分和累加与反量化都可以直接在 Tensor Core 内部完成，直到产生最终结果，从而避免频繁的数据移动，降低反量化开销。该方法一个值得注意的工业实现是 NVIDIA Blackwell 对 microscaling 数据格式 [66] 的支持，它体现了原生量化在规模化应用中的实际收益。

3.2 LogFMT：通信压缩

在当前的 DeepSeek-V3 架构中，我们对网络通信采用低精度压缩。在 EP 并行期间，token 使用细粒度 FP8 量化进行分派，与 BF16 相比将通信量降低了 50%，显著缩短了通信时间。由于精度要求，combine 阶段仍使用更高精度（例如 BF16），但我们正在积极测试 FP8、自定义精度格式（例如 E5M6）以及 FP8-BF16 混合，以进一步降低通信量。

除这些传统浮点格式之外，我们还尝试了一种新的数据类型，称为对数浮点格式（Logarithmic Floating-Point Formats，LogFMT-nBit），其中 n 是比特位数，最高 1 位作为符号位 S。通过将激活从原始线性空间映射到对数空间，激活的分布更加均匀。具体而言，给定一个元素 tile $[x1, \cdots, xm]$（在我们的实现中为 1×128），我们取所有元素的绝对值并计算其对数，找出最小值 $\min = \log(\text{abs}(xi))$ 与最大值 $\max = \log(\text{abs}(xj))$。最小值被编码为 S.00⋯01，最大值被编码为 S.11⋯11，相邻表示值之间的间隔为 $\text{Step} = (\max - \min) / (2^{n-1} - 2)$。零值则特殊地用 S.00⋯00 表示。其余数值被舍入到最接近的 Step 的整数倍（记为 K）。解码过程很简单：将符号位与 $\exp(\min + \text{Step} \times (K - 1))$ 组合即可。通过在局部计算 min 与 Step，该数据类型支持针对不同块的动态表示范围，与静态浮点格式相比，可以覆盖更大的范围或提供更高的精度。此外，我们发现，为了实现无偏的激活量化，必须在原始线性空间而

非对数空间中进行舍入。我们还约束 min 大于 $\max - \log(2^{32})$，这意味着最大表示范围与 E5（一种具有 5 位指数的浮点格式）相似。我们在大约 70 亿参数的稠密语言模型上验证了 LogFMT-nBit：对残差分支的输出进行量化，以模拟 MoE 模型中的 combine 阶段。当设置 $n = 8$、与 FP8 共享相同位数时，LogFMT-8Bit 相比 E4M3 或 E5M2 展现出更优的训练精度。将 n 提高到 10 位后，我们发现其结果与 BF16 的 combine 阶段相近。

3.2.1 局限性（Limitations）：使用 LogFMT 的初衷是将其应用于传输过程中的激活或激活函数附近，因为它在相同位宽下能提供比 FP8 更高的精度。然而，后续计算需要重新转换为 BF16 或 FP8，以适配 Hopper GPU Tensor Core 的数据类型。由于 GPU 在 log/exp 操作上的带宽不足，以及编码/解码过程中过高的寄存器压力，如果将编码/解码操作与全对全通信融合，开销可能相当大（50%～100%）。因此，尽管实验结果验证了该格式的有效性，我们最终并未采用它。

3.2.2 建议（Suggestions）：为 FP8 或自定义精度格式提供原生的压缩与解压缩单元支持，是未来硬件的一条可行路径。这有助于最小化带宽需求并简化通信流程。降低的通信开销对 MoE 训练这类带宽密集型任务尤为有益。

4 互连驱动的设计

4.1 当前硬件架构

我们目前使用的 NVIDIA H800 GPU SXM 架构如图 2 所示，它构建于 Hopper 架构之上，与 H100 GPU 类似。然而，出于监管合规要求，其 FP64 计算性能与 NVLink 带宽有所降低。具体而言，H800 SXM 节点中的 NVLink 带宽从 900 GB/s 降至 400 GB/s。节点内 scale-up 带宽的大幅降低给高性能工作负载带来了挑战。为弥补这一不足，每个节点配备八块 400G Infiniband（IB）CX7 NIC，以增强 scale-out 能力，缓解带宽缺口。

为应对这些硬件约束，DeepSeek-V3 模型在设计中引入了若干与硬件优势及局限相匹配的考虑。

4.2 硬件感知的并行

为适配 H800 架构的约束，我们考虑了以下并行策略来优化 DeepSeek-V3 的性能：

- **避免张量并行（Avoidance of Tensor Parallelism, TP）：**由于在有限的 NVLink 带宽下效率不高，训练期间避免使用张量并行。然而在推理期间，仍可选择性地使用 TP 以改善 TTFT 与 TPOT 性能。

- **增强流水线并行（Enhanced Pipeline Parallelism, PP）**：采用 DualPipe [29] 将注意力与 MoE 计算同 MoE 通信重叠。这还减少了流水线气泡并平衡了各 GPU 的内存使用，从而提升整体吞吐。更多细节见技术报告 [26]。
- **加速专家并行（Accelerated Expert Parallelism, EP）**：借助八块 400Gbps InfiniBand（IB）NIC，系统可以以超过 40GB/s 的速度实现全对全通信。值得注意的是，我们的全对全 EP 实现 DeepEP [79] 已经开源，从而实现了下节所讨论的高效专家并行。

【图表】图 2 H800 节点互连（原文第 7 页）

4.3 模型协同设计：节点受限路由

H800 架构中 scale-up（节点内）与 scale-out（节点间）通信之间的带宽差距约为 4:1。具体而言，NVLink 提供 200GB/s 带宽（其中实际可达约 160GB/s），而每块 400Gbps IB NIC 仅提供 50GB/s 带宽（考虑小消息与延迟的影响，有效带宽按 40GB/s 计）。为平衡并充分利用更高的节点内带宽，模型架构与硬件进行了协同设计，尤其是在 TopK 专家选择策略上。

考虑一个包含 8 个节点（共 64 块 GPU）与 256 个路由专家（每块 GPU 4 个专家）的部署。对于 DeepSeek-V3，每个 token 被路由到 1 个共享专家与 8 个路由专家。如果它的 8 个目标专家分布在全部 8 个节点上，则通过 IB 的通信时间将是 8t，其中 t 表示通过 IB 发送一个 token 的时间。然而，借助更高的 NVLink 带宽，路由到同一节点的 token 只需通过 IB 发送一次，然后通过 NVLink 转发到节点内的其他 GPU。NVLink 转发实现了 IB 流量的去重。当给定 token 的目标专家分布在 M 个节点上时，去重后的 IB 通信成本将降至 Mt（M<8）。由于 IB 流量仅取决于 M，DeepSeek-V3 在 TopK 专家选择策略中引入了节点受限路由（Node-Limited Routing）。具体而言，我们将 256 个路由专家分为 8 组，每组 32 个专家，并将每组部署在单个节点上。在此部署之上，我们在算法上确保每个 token 最多被路由到 4 个节点。这一方法缓解了 IB 通信瓶颈，并提升了训练期间的有效通信带宽。

4.4 Scale-Up 与 Scale-Out 的融合

4.4.1 当前实现的局限性（Limitations of Current Implementations）：尽管节点受限路由策略降低了通信带宽需求，但由于节点内（NVLink）与节点间（IB）互连之间的带宽差异，它使通信流水线 kernel 的实现变得复杂。在实践中，GPU 流式多处理器（SM）线程既用于网络消息处理（例如填充

QP 与 WQE），也用于通过 NVLink 转发数据，从而消耗计算资源。例如，在训练期间，H800 GPU 上最多有 20 个 SM 被分配用于通信相关操作，留给实际计算的资源因此减少。为最大化在线推理的吞吐，我们完全通过 NIC RDMA 执行 EP 全对全通信，避免 SM 资源争用并提升计算效率。这凸显了 RDMA 异步通信模型在重叠计算与通信方面的优势。

以下是 SM 目前在 EP 通信中执行的关键任务，尤其是 combine 阶段的 reduce 操作与数据类型转换。将这些任务卸载到专用的通信硬件可以释放 SM 用于计算 kernel，从而显著提升整体效率：

- **数据转发（Forwarding Data）**：在 IB 与 NVLink 域之间聚合发往同一节点内多个 GPU 的 IB 流量。
- **数据传输（Data Transport）**：在 RDMA 缓冲区（已注册的 GPU 内存区域）与输入/输出缓冲区之间移动数据。
- **Reduce 操作（Reduce Operations）**：执行 EP 全对全 combine 通信所需的 reduce 操作。
- **管理内存布局（Managing Memory Layouts）**：处理跨 IB 与 NVLink 域的分块数据传输所需的细粒度内存布局。
- **数据类型转换（Data Type Cast）**：在全对全通信前后转换数据类型。

4.4.2 建议（Suggestions）：为解决这些低效问题，我们强烈建议未来硬件将节点内（scale-up）与节点间（scale-out）通信集成到统一框架中。通过引入用于网络流量管理与 NVLink 和 IB 域之间无缝转发的专用协处理器，此类设计可以降低软件复杂度并最大化带宽利用率。例如，DeepSeek-V3 中采用的节点受限路由策略，可以通过硬件支持动态流量去重而得到进一步优化。

我们也注意到一些新兴的互连协议，例如 Ultra Ethernet Consortium（UEC）[17, 18]、Ultra Accelerator Link（UALink）[16]，二者都有望推动 scale-up 与 scale-out 通信的进步。最近，Unified Bus（UB）[49] 提出了一种实现 scale-up 与 scale-out 融合的新思路。第 6 节将进一步探讨 UEC 与 UALink 提出的若干技术创新。不过在本节中，我们主要关注在编程框架层面实现 scale-up 与 scale-out 的融合：

- （1）**统一网络适配器（Unified Network Adapter）**：设计连接到统一 scale-up 与 scale-out 网络的 NIC（网络接口卡）或 I/O Die。这些适配器还应支持基本的交换功能，例如将来自 scale-out 网络的报文转发到 scale-up 网络内的特定 GPU。这可以通过使用单个 LID（本地标识符）或 IP 地址并配合基于策略的路由来实现。
- （2）**专用通信协处理器（Dedicated Communication Co-Processor）**：引入专用的协处理器或可编程组件（例如 I/O die）来处理网络流量。该组件应从 GPU SM 卸载报文处理，提供硬件加速的

内存拷贝以实现高效的缓冲区管理，并且关键在于，以类似 TMA（Tensor Memory Accelerator）的方式加速内存 load/store 操作，从而以极少的资源消耗使带宽饱和。

- （3）**灵活的转发、广播与 Reduce 机制（Flexible Forwarding, Broadcast and Reduce Mechanisms）**：硬件应支持跨 scale-up 与 scale-out 网络的灵活转发、广播操作（用于 EP dispatch）与 reduce 操作（用于 EP combine），以对应我们当前基于 GPU SM 的实现。这不仅能提升有效带宽，还能降低网络特定操作的计算复杂度。
- （4）**硬件同步原语（Hardware Synchronization Primitives）**：提供细粒度的硬件同步指令，在硬件层面处理内存一致性问题或报文乱序到达问题。这将无需依赖基于软件的同步机制（如 RDMA 完成事件），后者会引入额外延迟并增加编程复杂度。带有 acquire/release 机制的内存语义通信是一种有前景的实现方式。

通过落实这些建议，未来的硬件设计可以显著提升大规模分布式 AI 系统的效率，同时简化软件开发。

4.5 带宽争用与延迟

4.5.1 局限性（Limitations）：此外，当前硬件缺乏在 NVLink 与 PCIe 上针对不同类型流量动态分配带宽的灵活性。例如，在推理期间，将 KV cache 数据从 CPU 内存传输到 GPU 可能消耗数十 GB/s，从而占满 PCIe 带宽。如果 GPU 同时使用 IB 进行 EP 通信，KV cache 传输与 EP 通信之间的这种争用会降低整体性能并导致延迟尖峰。

4.5.2 建议（Suggestions）：

- **NVLink/PCIe 流量动态优先级（Dynamic NVLink/PCIe Traffic Prioritization）**：硬件应支持根据流量类型动态调整优先级。例如，与 EP、TP 以及 KV cache 传输相关的流量应被赋予不同的优先级，以最大化互连效率。对于 PCIe，将流量类别（TC）暴露给用户级编程即可满足需求。
- **I/O Die Chiplet 集成（I/O Die Chiplet Integration）**：将 NIC 直接集成到 I/O die 中，并在同一封装内将其连接到计算 die，而不是通过传统 PCIe 连接，将大幅降低通信延迟并缓解 PCIe 带宽争用。
- **Scale-Up 域内的 CPU–GPU 互连（CPU–GPU Interconnects within the Scale-Up Domain）**：为进一步优化节点内通信，CPU 与 GPU 之间应使用 NVLink 或类似的专用高带宽互连结构，而不是仅依赖 PCIe。与将 NIC 集成到 I/O die 所带来的收益类似，这一做法可以显著改善训练与推理期间在 GPU 与 CPU 内存之间卸载参数或 KV cache 等场景。

【图表】图 3 八平面两层 fat-tree scale-out 网络：每对 GPU 与 IB NIC 属于一个网络平面。跨平面流量必须使用另一块 NIC，并通过 PCIe 或 NVLink 进行节点内转发（原文第 8 页）

【图表】图 4 理想的多平面网络：每块 NIC 配备多个物理端口，每个端口连接到一个不同的网络平面。单个队列对（QP）可以同时利用所有可用端口收发报文，这要求 NIC 原生支持乱序放置（原文第 9 页）

5 大规模网络驱动的设计

5.1 网络协同设计：多平面 Fat-Tree

在 DeepSeek-V3 的训练中，我们部署了一个多平面 Fat-Tree（Multi-Plane Fat-Tree，MPFT）scale-out 网络，如图 3 所示。每个节点配备八块 GPU 与八块 IB NIC，每对 GPU–NIC 被分配到不同的网络平面。此外，每个节点还有一块 400 Gbps 以太网 RoCE NIC 连接到独立的存储网络平面，用于访问 3FS [30] 分布式文件系统。在 scale-out 网络中，我们使用 64 端口 400G IB 交换机，使该拓扑在理论上最多可支持 16,384 块 GPU，同时保留两层网络的成本与延迟优势。然而，受政策与监管限制，最终部署的 GPU 数量仅为两千多块。

此外，由于 IB ConnectX-7 目前的限制，我们部署的 MPFT 网络并未完全实现设想中的架构。理想情况下，如图 4 所示，每块 NIC 应具备多个物理端口，每个端口连接到一个独立的网络平面，但通过端口绑定（port bonding）对用户呈现为单个逻辑接口。从用户角度看，单个队列对（Queue Pair，QP）可以无缝地在所有可用端口上收发报文，类似于 packet spraying。相应地，来自同一 QP 的报文可能经过不同的网络路径，并以乱序方式到达接收端，因此需要 NIC 原生支持乱序放置，以保证消息一致性并维持正确的顺序语义。例如，InfiniBand ConnectX-8 原生支持四个平面。未来的 NIC 若能完全支持先进的多平面能力，将使两层 fat-tree 网络能够有效扩展到规模大得多的 AI 集群。总体而言，多平面架构在故障隔离、鲁棒性、负载均衡与大规模系统可扩展性方面具有显著优势。

【表格】表 3 网络拓扑对比。成本估算基于 Slim Fly（SF）论文 [12] 中的方法。DF 表示经典的 dragonfly 拓扑 [22, 46, 65]（原文第 9 页）

5.1.1 多平面 Fat-Tree 网络的优势（Advantages of Multi-Plane Fat-Tree Network）：

- **多轨 Fat-Tree 的子集（Subset of Multi-Rail Fat-Tree, MRFT）**：MPFT 拓扑构成了更广泛的 MRFT 架构的一个特定子集。因此，NVIDIA 与 NCCL 为 Multi-Rail 网络开发的现有优化可以在 Multi-Plane 网络部署中无缝利用。此外，NCCL 对 PXN [54] 技术的支持解决了平面间隔离的固有挑战，即使平面之间缺乏直接互连，也能实现高效通信。
- **成本效率（Cost Efficiency）**：如表 3 所示，多平面网络使用两层 fat-tree（FT2）拓扑即可支持超过 1 万个端点，与三层 fat tree（FT3）相比显著降低了网络成本。每个端点的成本甚至略优于成本高效的 Slim Fly（SF）拓扑 [12]。
- **流量隔离（Traffic Isolation）**：每个平面独立运行，确保一个平面的拥塞不会影响其他平面。这种隔离提升了整体网络稳定性，并防止级联的性能下降。
- **延迟降低（Latency Reduction）**：正如我们的实验所示，两层拓扑实现了比三层 fat tree 更低的延迟。这使其特别适合对延迟敏感的应用，例如基于 MoE 的训练与推理。
- **鲁棒性（Robustness）**：如图 4 所示，多端口 NIC 提供多条上行链路，因此单端口故障不会中断连接，并且可以实现快速、透明的故障恢复。

需要指出的是，由于当前 400G NDR InfiniBand 的限制，跨平面通信需要在节点内进行转发，这会在推理过程中引入额外延迟。如果未来的硬件能够实现前文所述的 scale-up 与 scale-out 网络融合，这一延迟将可以显著降低，从而进一步提升多平面网络的可行性。

5.1.2 性能分析

为验证多平面网络设计的有效性，我们在自己的集群上开展了真实环境实验，修改集群的网络拓扑，以比较多平面两层胖树（Multi-Plane Two-Layer Fat Tree, MPFT）与单平面多轨胖树（Single-Plane Multi-Rail Fat Tree, MRFT）的性能。以下是实验得到的关键发现：

1. 全对全通信与 EP 场景（All-to-All Communication and EP Scenarios）：如图 5 所示，多平面网络的全对全性能与单平面多轨网络非常接近。这种性能持平可归因于 NCCL 的 PXN [54] 机制，该机制在多轨拓扑中通过 NVLink 优化流量转发。多平面拓扑同样受益于该机制。如图 6 所示，在 16 块 GPU 上进行的全对全通信测试结果表明，MPFT 与 MRFT 拓扑之间的延迟差异可以忽略不计。为评估 MPFT 在实际训练场景中全对全通信的性能，我们测试了训练中常用的 EP 通信模式。如图 7 所示，在多平面网络中，每块 GPU 都能实现超过 40GB/s 的高带宽，提供了满足训练需求的可靠性能。

【图表】图 5 32 至 128 块 GPU 下 MRFT 与 MPFT 网络的 NCCL all-to-all 性能。（原文第 10 页）

【图表】图 6 不同消息大小下 MPFT 与 MRFT 网络在 NCCL all-to-all 测试中的延迟比较，表明两者性能几乎相同。（原文第 10 页）

【图表】图 7 DeepEP 在 MPFT 上的性能：EP dispatch 与 combine 内核通过 all-to-all 在 16 至 128 块 GPU 之间通信。每块 GPU 处理 4096 个 token。观测到的吞吐几乎跑满 400Gbps 的 NIC 带宽。（原文第 10 页）

2. DeepSeek-V3 模型的训练吞吐（Training Throughput for DeepSeek-V3 Model）：我们还在表 4 中比较了 MPFT 与 MRFT 下 DeepSeek-V3 模型的训练指标。MFU（Model Flops Utilization，模型 FLOPs 利用率）基于 BF16 峰值性能计算。因果 MFU 只计入注意力矩阵下三角部分的 flops（与 FlashAttention[19, 20] 一致），而非因果 MFU 则包含整个注意力矩阵的 flops（与 Megatron [47] 一致）。1F、1B 与 1W 分别表示前向时间、输入反向时间与权重反向时间。在 2048 块 GPU 上训练 V3 模型时，MPFT 的性能与 MRFT 几乎相同，观测到的差异处于正常波动与测量误差范围内。

【表格】表 4 MPFT 与 MRFT 网络的训练指标比较。（原文第 10 页）

5.2 低延迟网络

在我们的模型推理中，大规模 EP 高度依赖 all-to-all 通信，而后者对带宽与延迟都极为敏感。考虑 2.3.2 节讨论的典型场景：在网络带宽为 50GB/s 时，数据传输理想上应耗时约 120 μs。因此，微秒量级的固有网络延迟会严重影响系统性能，其影响不可忽视。

5.2.1 IB 还是 RoCE

链路层	同叶交换机	跨叶交换机
RoCE	3.6us	5.6us
InfiniBand	2.8us	3.7us
NVLink	3.33us	-

表 5 64B 数据传输下 IB、RoCE 与节点内 NVLink 的 CPU 侧端到端延迟比较。

如表 5 所示，IB 始终取得更低的延迟，使其成为分布式训练与推理等延迟敏感型工作负载的首选。尽管 IB 相比融合以太网上的 RDMA（RDMA over Converged Ethernet, RoCE）具有更优的延迟表现，但它也存在若干局限：

- **成本（Cost）**：IB 硬件的价格显著高于 RoCE 方案，这限制了其广泛采用。
- **可扩展性（Scalability）**：IB 交换机通常每个交换机仅支持 64 个端口，而 RoCE 交换机通常提供 128 个端口。这限制了基于 IB 的集群的可扩展性，尤其是在大规模部署中。

5.2.2 改进 RoCE 的建议

尽管 RoCE 有潜力成为 IB 的高性价比替代方案，但其当前在延迟与可扩展性方面的局限使其无法完全满足大规模 AI 系统的需求。下面我们列出改进 RoCE 的具体建议：

（1）专用低延迟 RoCE 交换机（Specialized Low-Latency RoCE Switches）：我们建议以太网厂商开发专门针对 RDMA 工作负载优化的 RoCE 交换机，去除不必要的以太网特性。Slingshot 架构 [22] 展示了基于以太网的设计如何达到与 IB 相当的延迟性能。类似地，Broadcom [13] 近期的创新——包括 AI Forwarding Header（AIFH）与即将推出的低延迟以太网交换机——证明了为 AI 量身定制的高性能以太网架构的可行性。我们期待这一方向持续创新。

（2）优化的路由策略（Optimized Route Policy）：如图 8 所示，RoCE 中默认的等价多路径（Equal-Cost Multi-Path, ECMP）路由策略难以在互连之间高效分发流量，导致在 NCCL 集合通信测试中出现严重的拥塞性能退化。LLM 训练流量（例如数据并行（Data Parallelism, DP）中的流量）往往缺乏随机性，导致多条流汇聚到同一条互连链路上。相比之下，自适应路由（Adaptive Routing, AR）[34] 通过在多条路径上动态喷洒数据包，可以显著提升网络性能。虽然基于人工配置路由表的静态路由可以为特定目的地避免链路冲突，但它缺乏灵活性。对于大规模 all-to-all 通信，自适应路由提供了更优的性能与可扩展性。

【图表】图 8 不同路由方法（ECMP、AR、静态路由）与 TP 维度下 AllGather 与 ReduceScatter 通信原语的 RoCE 网络带宽。（原文第 11 页）

（3）改进的流量隔离或拥塞控制机制（Improved Traffic Isolation or Congestion Control Mechanisms）：当前 RoCE 交换机仅支持有限数量的优先级队列，不足以应对涉及并发通信模式（如 EP 的 all-to-all 与 DP 的 all-reduce）的复杂 AI 工作负载。在这类混合工作负载中，all-to-all 流量可能因突发的多对一传输而引发 incast 拥塞，可能降低整体网络性能。为缓解 incast 对其他流量的影响，一种方法是采用虚拟输出队列（virtual output queuing, VOQ），为每个 QP 分配专用虚拟队列以隔离流量。另一种方法是采用更有效的拥塞控制（congestion control, CC）机制，例如基于 RTT 的 CC（RTTCC）或用户可编程 CC（PCC），从而实现网卡-交换机协同优化，在动态流量条件下维持低延迟与高吞吐。

5.2.3 InfiniBand GPUDirect Async（IBGDA）

我们利用 IBGDA [2, 57] 降低网络通信延迟。传统上，网络通信需要创建一个 CPU 代理线程：GPU 准备好数据后，必须通知 CPU 代理；CPU 代理随后填充工作请求（work request, WR）的控制信息，并通过 doorbell 机制向 NIC 发出信号以启动数据传输。该过程引入了额外的通信开销。

IBGDA 通过允许 GPU 直接填写 WR 内容并写入 RDMA doorbell 的 MMIO 地址来解决这一问题。通过在 GPU 内部管理整个控制平面，IBGDA 消除了与 GPU-CPU 通信相关的显著延迟开销。此外，在发送大量小数据包时，控制平面处理器很容易成为瓶颈。由于 GPU 拥有多个并行线程，发送方可以利用这些线程分摊工作负载，从而避免此类瓶颈。包括我们的 DeepEP [79] 在内的一系列工作已经利用了 IBGDA 并报告了显著的性能提升 [1, 15, 80]。因此，我们倡导在各类加速器设备中广泛支持这类能力。

6 讨论及对未来硬件架构设计的洞见

在前文的基础上，我们总结关键的架构洞见，并勾勒针对大规模 AI 工作负载量身定制的硬件设计的未来方向。2.3.2 节强调了大规模 scale-up 网络对加速模型推理的重要性。第 3 节讨论了高效支持低精度计算与通信的必要性。第 4 节探讨了 scale-up 与 scale-out 架构的融合以及若干提出的增强方案。第 5 节聚焦多平面网络拓扑，并指出了基于以太网的互连所需的关键改进。这些章节共同识别了具体应用场景中的硬件局限，并给出了相应建议。在此基础上，本节将讨论扩展到更广泛的考量，并提出未来硬件架构设计的前瞻性方向。

6.1 鲁棒性挑战

6.1.1 局限

- **互连故障（Interconnect Failures）**：高性能互连（如 IB 与 NVLink）容易出现间歇性断连，可能中断节点间通信。这在 EP 等通信密集型工作负载中尤其有害——即使是短暂的中断也可能导致显著的性能下降或作业失败。
- **单点硬件故障（Single Hardware Failures）**：节点崩溃、GPU 故障或 ECC（Error-Correcting Code）内存错误可能危及长时间运行的训练任务，通常需要付出高昂的重启代价。这类故障的影响在大规模部署中会加剧，因为单点故障的概率随系统规模成比例增加。

- **静默数据损坏（Silent Data Corruption）**：ECC 机制未能检测到的错误（如多位内存翻转或计算不准确）会对模型质量构成重大风险。这些错误在长时间运行的任务中尤为隐蔽，因为它们可能在未被发现的情况下传播并破坏下游计算。当前的缓解策略依赖应用层启发式方法，不足以确保系统级鲁棒性。

6.1.2 先进错误检测与纠正的建议

为缓解与静默损坏相关的风险，硬件必须引入超越传统 ECC 的先进错误检测机制。基于校验和的验证或硬件加速的冗余校验等技术可为大规模部署提供更高的可靠性。

此外，硬件厂商应向终端用户提供全面的诊断工具包，使其能够严格验证系统完整性，并主动识别任何潜在的静默数据损坏。这类工具包作为标准硬件包的一部分内置时，能够促进透明度，并在整个运行生命周期内实现持续验证，从而增强整体系统的可信度。

6.2 CPU 瓶颈与互连

尽管加速器设计常常占据中心位置，但 CPU 在协调计算、管理 I/O 与维持系统吞吐方面仍然不可或缺。然而，当前架构面临若干关键瓶颈：

首先，如 4.5 节所述，CPU 与 GPU 之间的 PCIe 接口常常成为带宽瓶颈，尤其是在大规模参数、梯度或 KV cache 传输期间。为缓解这一问题，未来系统应采用直接的 CPU–GPU 互连——例如 NVLink 或 Infinity Fabric——或者将 CPU 与 GPU 都整合到 scale-up 域中，从而消除节点内瓶颈。

除 PCIe 的限制外，维持如此高的数据传输速率还需要异常高的内存带宽。例如，要跑满 160 条 PCIe 5.0 通道，每个节点需要超过 640 GB/s 的带宽，折算为每个节点约 1 TB/s 的内存带宽需求——这对传统 DRAM 架构构成了重大挑战。

最后，内核启动与网络处理等延迟敏感型任务需要较高的 CPU 单核性能，通常要求基准频率高于 4 GHz。此外，现代 AI 工作负载要求每块 GPU 配备足够的 CPU 核心，以防止控制侧瓶颈。对于基于 chiplet 的架构，还需要额外的核心来支持缓存感知的工作负载划分与隔离。

6.3 面向 AI 的智能网络

为满足延迟敏感型工作负载的需求，未来的互连必须同时重视低延迟与网络智能化：

- **共封装光学（Co-Packaged Optics）**：引入硅光子技术可实现可扩展的更高带宽与更高的能效，这两者对于大规模分布式系统都至关重要。

- **无损网络（Lossless Network）**：基于信用的流量控制（Credit-Based Flow Control, CBFC）机制可确保无损数据传输，但简单地触发流量控制可能引发严重的队头阻塞。因此，必须部署先进的、由端点驱动的拥塞控制（CC）算法，主动调节注入速率并避免病态拥塞场景。
- **自适应路由（Adaptive Routing）**：正如 5.2.2 节所强调的，未来网络应标准化采用动态路由方案——例如数据包喷洒与拥塞感知的路径选择——持续监控实时网络状况并智能地重新分配流量。这些自适应策略在缓解热点与减轻集合通信工作负载（包括 all-to-all 与 reduce-scatter 操作）中的瓶颈方面尤为有效。
- **高效容错协议（Efficient Fault-Tolerant Protocols）**：通过部署自愈协议、冗余端口与快速故障切换技术，可以显著增强对故障的鲁棒性。例如，链路层重试机制与选择性重传协议在将可靠性扩展到大型网络方面不可或缺，可最大限度地减少停机时间，并在间歇性故障下确保无缝运行。
- **动态资源管理（Dynamic Resource Management）**：为有效处理混合工作负载，未来硬件应支持动态带宽分配与流量优先级划分。例如，在统一集群中应将推理任务与训练流量隔离，以确保延迟敏感型应用的响应性。

6.4 关于内存语义通信与顺序性问题的讨论

使用 load/store 内存语义进行节点间通信既高效又对程序员友好，但当前实现受内存顺序性挑战的阻碍。例如，在写入数据之后，发送方必须先发出显式的内存屏障（fence），然后才能更新标志以通知接收方，从而确保数据一致性。这种严格顺序会引入额外的往返时间（RTT）延迟，并可能阻塞发起线程，妨碍在途的存储操作并降低吞吐。类似的乱序同步问题也出现在消息语义 RDMA 中；例如，在 InfiniBand 或 NVIDIA BlueField-3 上，于常规 RDMA 写操作之后使用数据包喷洒执行 RDMA 原子加操作，会带来额外的 RTT 延迟。

为解决这些问题，我们倡导硬件提供内置的顺序性保证以支持内存语义通信。这种一致性既应在编程层面强制实施（例如通过 acquire/release 语义），也应由接收方硬件保证，从而在无额外开销的情况下实现按序交付。

有若干可行方法。例如，接收方可以缓存原子消息并使用数据包序列号（packet sequence number, PSN）来确保按序处理；该方法直接且能有效保证正确性。另一种方法是基于区域的 acquire/release（region-based acquire/release, RAR）机制，它尤其有吸引力，因为它能为远程内存访问提供真正的 acquire/release 语义，并为各种工作负载提供更大的灵活性。在该方法中，接收方硬件维护轻量级元数据——例如位图或基于区域的计数器——以跟踪内存区域的状态。Acquire 与 release 操作的作用范围

限定在特定地址区间，从而无需发送方显式屏障即可实现高效的、由硬件强制执行的顺序性。值得注意的是，这两种方法都适合在 NIC 或 I/O die 上实现，并且同时适用于内存语义与消息语义的 RDMA 原语，从而拓宽了它们的实际用途。

6.5 网内计算与压缩

EP 涉及两个关键的 all-to-all 阶段——dispatch 与 combine——它们为网内优化提供了重要机会。dispatch 阶段类似于小规模多播操作，其中单条消息必须被转发到多个目标设备。一种能够在硬件层面自动完成数据包复制并转发到多个目的地的协议，可以大幅降低通信开销并提升效率。combine 阶段相当于小规模归约操作，可以从网内聚合技术中受益。然而，由于 EP combine 的归约范围小且工作负载不均衡，以灵活方式实现网内聚合具有挑战性。

此外，正如 3.2 节所强调的，LogFMT 能够在对模型性能影响极小的情况下实现低精度 token 传输。在网络硬件中原生集成 LogFMT 可以进一步提升熵密度并降低带宽占用，从而优化通信。硬件加速的压缩与解压缩将使 LogFMT 无缝集成到分布式系统中，提升整体吞吐。

6.6 以内存为中心的创新

6.6.1 内存带宽的局限

模型规模的指数级增长已经超过了高带宽内存（high-bandwidth memory, HBM）技术的进步速度。这种差距造成了内存瓶颈，在 Transformer 等注意力密集型架构中尤为突出。

6.6.2 建议

- **DRAM 堆叠加速器（DRAM-Stacked Accelerators）：**利用先进的 3D 堆叠技术，DRAM 芯片可以垂直集成在逻辑芯片之上，从而实现极高的内存带宽、超低延迟以及实用的内存容量（尽管受堆叠层数限制）。这一架构范式对于 MoE 模型中的超快速推理尤为有利，因为在此类场景中内存吞吐是关键瓶颈。SeDRAM[73] 等架构展示了这一方法的潜力，为内存受限型工作负载提供了前所未有的性能。
- **晶圆级系统（System-on-Wafer, SoW）：**晶圆级集成（wafer-scale integration）[50] 可以最大化计算密度与内存带宽，满足超大规模模型的需求。

7 结论

DeepSeek-V3 体现了软硬件协同设计在提升大规模 AI 系统可扩展性、效率与鲁棒性方面的变革性潜力。通过应对当前硬件架构的局限并提出可落地的建议，本文为下一代面向 AI 优化的硬件提供了路线图。随着 AI 工作负载在复杂性与规模上持续增长，这些创新将至关重要，推动智能系统的未来。

附录

附录：补充材料见英文原文

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures ISCA ’25, June 21–25, 2025, Tokyo, Japan

[2] after regular RDMA writes on InfiniBand or NVIDIA BlueField-3 memory throughput is a critical bottleneck. Architectures such can incur additional RTT latency. as SeDRAM[73] exemplify the potential of this approach, deliver- To address these, we advocate for hardware support that offers ing unprecedented performance for memory-bound workloads. built-in ordering guarantees for memory-semantic communication. • System-on-Wafer (SoW): Wafer-scale integration [50] can max- Such consistency should be enforced both at the programming level imize computational density and memory bandwidth, addressing (e.g., via acquire/release semantics) and by hardware at the receiver, the needs of ultra-large-scale models. enabling in-order delivery without added overhead. Several approaches are possible. For example, the receiver can 7 Conclusion buffer atomic messages and use packet sequence numbers (PSN) DeepSeek-V3 exemplifies the transformative potential of hardwareto ensure in-order processing; this method is straightforward and software co-design in advancing the scalability, efficiency, and roeffective for maintaining correctness. Alternatively, a region-based bustness of large-scale AI systems. By addressing the limitations of acquire/release (RAR) mechanism is particularly attractive, as it encurrent hardware architectures and proposing actionable recomables true acquire/release semantics for remote memory access and mendations, this paper provides a roadmap for the next generation offers greater flexibility for a range of workloads. In this approach, of AI-optimized hardware. These innovations will be critical as AI the receiver hardware maintains lightweight metadata—such as workloads continue to grow in complexity and scale, driving the bitmaps or region-based counters—to track the state of memory refuture of intelligent systems. gions. Acquire and release operations are scoped to specific address ranges, enabling efficient, hardware-enforced ordering without ex- References plicit sender-side fences. Notably, both approaches are amenable [1] Elena Agostini, Davide Rossetti, and Sreeram Potluri. 2017. Offloading Commu- to implementation on the NIC or I/O die and are applicable to nication Control Logic in GPU Accelerated Applications. In 2017 17th IEEE/ACM both memory-semantic and message-semantic RDMA primitives, International Symposium on Cluster, Cloud and Grid Computing (CCGRID) . 248– thereby broadening their practical utility. 257. https://doi . org/10 . 1109/CCGRID . 2017 . 29 [2] E. Agostini, D. Rossetti, and S. Potluri. 2018. GPUDirect Async: Exploring GPU synchronous communication techniques for InfiniBand clusters. J. Parallel and 6.5 In-Network Computation and Compression Distrib. Comput. 114 (2018), 28–45. https://doi . org/10 . 1016/j . jpdc . 2017 . 12 . 007 [3] AI@Meta. 2024. Llama 3 Model Card. https://github . com/meta-llama/llama3/ EP involves two critical all-to-all stages— dispatch and comblob/main/MODEL_CARD . md bine —that present significant opportunities for in-network opti- [4] AI@Meta. 2024. Llama 3.1 Model Card. https://github . com/meta-llama/llamamodels/blob/main/models/llama3_1/MODEL_CARD . md mization. The dispatch stage resembles a small-scale multicast [5] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico operation, where a single message must be forwarded to multi- Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transple target devices. A hardware-level protocol enabling automatic former Models from Multi-Head Checkpoints. arXiv preprint arXiv:2305.13245 (2023). packet replication and forwarding to multiple destinations could [6] AMD. 2025. AMD Ryzen AI Max+ PRO 395: Designed to power drastically reduce communication overhead and improve efficiency. a new generation of compact Copilot+ PC workstations. https: //www . amd . com/en/products/processors/laptop/ryzen-pro/ai-max-pro- The combine stage, acting as a small-scale reduction operation, 300-series/amd-ryzen-ai-max-plus-pro-395 . html could benefit from in-network aggregation techniques. However, [7] Wei An, Xiao Bi, Guanting Chen, Shanhuang Chen, Chengqi Deng, Honghui due to the small reduction scope and imbalanced workload in EP Ding, Kai Dong, Qiushi Du, Wenjun Gao, Kang Guan, Jianzhong Guo, Yongqiang Guo, Zhe Fu, Ying He, Panpan Huang, Jiashi Li, Wenfeng Liang, Xiaodong Liu, combine, implementing in-network aggregation in a flexible man- Xin Liu, Yiyuan Liu, Yuxuan Liu, Shanghao Lu, Xuan Lu, Xiaotao Nie, Tian Pei, ner is challenging. Junjie Qiu, Hui Qu, Zehui Ren, Zhangli Sha, Xuecheng Su, Xiaowen Sun, Yixuan Moreover, as highlighted in Section 3.2, LogFMT enables low- Tan, Minghui Tang, Shiyu Wang, Yaohui Wang, Yongji Wang, Ziwei Xie, Yiliang Xiong, Yanhong Xu, Shengfeng Ye, Shuiping Yu, Yukun Zha, Liyue Zhang, Haowei precision token transmission with minimal impact on model per- Zhang, Mingchuan Zhang, Wentao Zhang, Yichao Zhang, Chenggang Zhao, Yao formance. Incorporating LogFMT natively within network hard- Zhao, Shangyan Zhou, Shunfeng Zhou, and Yuheng Zou. 2024. Fire-Flyer AI- HPC: A Cost-Effective Software-Hardware Co-Design for Deep Learning. In SC24: ware could further optimize communication by increasing entropy International Conference for High Performance Computing, Networking, Storage density and reducing bandwidth usage. Hardware-accelerated comand Analysis . 1–23. https://doi . org/10 . 1109/SC41406 . 2024 . 00089 pression and decompression would allow seamless integration of [8] Anthropic. 2024. Claude 3.5 Sonnet. https://www . anthropic . com/news/claude- 3-5-sonnet LogFMT into distributed systems, enhancing overall throughput. [9] Anthropic. 2025. Claude 3.7 Sonnet and Claude Code. https:// www . anthropic . com/news/claude-3-7-sonnet 6.6 Memory-Centric Innovations [10] Apple. 2024. Apple introduces M4 Pro and M4 Max. https://www . apple . com/ newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/ 6.6.1 Limitations of Memory Bandwidth. The exponential growth [11] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-

Document Transformer. arXiv:2004.05150 (2020). in model sizes has outpaced advancements in high-bandwidth mem- [12] Nils Blach, Maciej Besta, Daniele De Sensi, Jens Domke, Hussein Harake, Shigang ory (HBM) technology. This disparity creates a memory bottleneck, Li, Patrick Iff, Marek Konieczny, Kartik Lakhotia, Ales Kubicek, Marcel Ferrari, particularly in attention-heavy architectures like Transformers. Fabrizio Petrini, and Torsten Hoefler. 2025. A high-performance design, implementation, deployment, and evaluation of the slim fly network. In Proceedings of 6.6.2 Suggestions: the 21st USENIX Symposium on Networked Systems Design and Implementation (Santa Clara, CA, USA) (NSDI'24) . USENIX Association, USA, Article 57, 20 pages. • DRAM-Stacked Accelerators: Leveraging advanced 3D stack- [13] Broadcom. 2025. Scale Up Ethernet Framework. https://docs . broadcom . com/ ing technologies, DRAM dies can be vertically integrated atop doc/scale-up-ethernet-framework a logic die, thereby enabling exceptionally high memory band- [14] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework width, ultra-low latency, and a practical memory capacity (though with Multiple Decoding Heads. In Forty-first International Conference on Machine stack-limited). This architectural paradigm proves remarkably Learning, ICML 2024, Vienna, Austria, July 21-27, 2024 . OpenReview.net. https: advantageous for ultra-fast inference in MoE models, where //openreview . net/forum?id = PEpbUobfJv Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Huazuo Gao, Jiashi Li, Liyue Zhang, Panpan Huang, Shangyan Zhou, Shirong Ma, Wenfeng Liang, Ying He, Yuqing ISCA '25, June 21–25, 2025, Tokyo, Japan Wang, Yuxuan Liu, and Y.X. Wei

[3] [15] Shaoyuan Chen, Wencong Xiao, Yutong Lin, Mingxing Zhang, Yingdi Shan, ai/ktransformers Jinlei Jiang, Kang Chen, and Yongwei Wu. 2025. Efficient Heteroge- [40] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, neous Large Language Model Decoding with Model-Attention Disaggregation. Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. KVQuant: Towards arXiv:2405.01814 [cs.LG] https://arxiv . org/abs/2405 . 01814 10 Million Context Length LLM Inference with KV Cache Quantization. arXiv [16] ULTRA ACCELERATOR LINK CONSORTIUM. 2025. Introducing UALink 200G preprint arXiv:2401.18079 (2024). 1.0 Specification. https://ualinkconsortium . org/wp-content/uploads/2025/04/ [41] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, De- UALink-1 . 0-White_Paper_FINAL . pdf vendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, [17] Ultra Ethernet Consortium. 2023. Overview of and Motivation for the Forth- Guillaume Lample, Lucile Saulnier, et al . 2023. Mistral 7B. arXiv preprint coming Ultra Ethernet Consortium Specification. https://ultraethernet . org/wp- arXiv:2310.06825 (2023). content/uploads/sites/20/2023/10/23 . 07 . 12-UEC-1 . 0-Overview-FINAL-WITH- [42] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, LOGO . pdf Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin [18] Ultra Ethernet Consortium. 2024. UEC Progresses Towards v1.0 Set of Spec- Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, ifications. https://ultraethernet . org/uec-progresses-towards-v1-0-set-of- Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang specifications/ Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. [19] Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. Work Partitioning. http://arxiv . org/abs/2402 . 15627 arXiv:2402.15627 [cs]. [20] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. [43] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, In Advances in Neural Information Processing Systems . Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically [21] Tri Dao and Albert Gu. 2024. Transformers are SSMs: generalized models and Reconfigurable Supercomputer for Machine Learning with Hardware Support efficient algorithms through structured state space duality. In Proceedings of the for Embeddings. In Proceedings of the 50th Annual International Symposium on 41st International Conference on Machine Learning (Vienna, Austria) (ICML'24) . Computer Architecture (Orlando, FL, USA) (ISCA '23) . Association for Computing JMLR.org, Article 399, 31 pages. Machinery, New York, NY, USA, Article 82, 14 pages. https://doi . org/10 . 1145/ [22] Daniele De Sensi, Salvatore Di Girolamo, Kim H. McMahon, Duncan Roweth, 3579371 . 3589350 and Torsten Hoefler. 2020. An In-Depth Analysis of the Slingshot Interconnect. [44] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar In SC20: International Conference for High Performance Computing, Networking, Krishna, and Tuo Zhao. 2024. GEAR: An Efficient KV Cache Compression Recipe Storage and Analysis . 1–14. https://doi . org/10 . 1109/SC41405 . 2020 . 00039 for Near-Lossless Generative Inference of LLM. arXiv:2403.05527 [cs.LG] [23] DeepSeek-AI. 2024. DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source [45] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Models in Code Intelligence. CoRR abs/2406.11931 (2024). https://doi . org/ Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 10 . 48550/arXiv . 2406 . 11931 2020. Scaling Laws for Neural Language Models. CoRR abs/2001.08361 (2020). [24] DeepSeek-AI. 2024. DeepSeek LLM: Scaling Open-Source Language Models arXiv:2001.08361 https://arxiv . org/abs/2001 . 08361 with Longtermism. CoRR abs/2401.02954 (2024). https://doi . org/10 . 48550/ [46] John Kim, Wiliam J. Dally, Steve Scott, and Dennis Abts. 2008. TechnologyarXiv . 2401 . 02954 Driven, Highly-Scalable Dragonfly Topology. In 2008 International Symposium [25] DeepSeek-AI. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of- on Computer Architecture . 77–88. https://doi . org/10 . 1109/ISCA . 2008 . 19 Experts Language Model. CoRR abs/2405.04434 (2024). https://doi . org/10 . 48550/ [47] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael arXiv . 2405 . 04434 Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. Proceedings of Machine Learning and [26] DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. (2024). arXiv:2412.19437 [cs.CL] https://arxiv . org/abs/2412 . 19437 Systems 5 (2023). [27] DeepSeek-AI. 2024. DeepSeekMoE: Towards Ultimate Expert Specialization [48] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang.

2024. EAGLE: in Mixture-of-Experts Language Models. CoRR abs/2401.06066 (2024). https: Speculative Sampling Requires Rethinking Feature Uncertainty. In Forty-first //doi . org/10 . 48550/arXiv . 2401 . 06066 International Conference on Machine Learning, ICML 2024, Vienna, Austria, July [28] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs 21-27, 2024 . OpenReview.net. https://openreview . net/forum?id = 1NdN7eXyb4 via Reinforcement Learning. arXiv:2501.12948 [cs.CL] https://arxiv . org/abs/ [49] Heng Liao, Bingyang Liu, Xianping Chen, Zhigang Guo, Chuanning Cheng, 2501 . 12948 Jianbing Wang, Xiangyu Chen, Peng Dong, Rui Meng, Wenjie Liu, Zhe Zhou, [29] DeepSeek-AI. 2025. DualPipe: A bidirectional pipeline parallelism algorithm Ziyang Zhang, Yuhang Gai, Cunle Qian, Yi Xiong, Zhongwu Cheng, Jing Xia, Yuli for computation-communication overlap in V3/R1 training. https://github . com/ Ma, Xi Chen, Wenhua Du, Shizhong Xiao, Chungang Li, Yong Qin, Liudong Xiong, deepseek-ai/dualpipe. Zhou Yu, Lv Chen, Lei Chen, Buyun Wang, Pei Wu, Junen Gao, Xiaochu Li, Jian [30] DeepSeek-AI. 2025. Fire-Flyer File System. https://github . com/deepseek-ai/3FS He, Shizhuan Yan, and Bill McColl. 2025. UB-Mesh: a Hierarchically Localized [31] DeepSeek-AI. 2025. Profiling Data in DeepSeek Infra. https://github . com/ nD-FullMesh Datacenter Network Architecture. arXiv:2503.20377 [cs.AR] https: deepseek-ai/profile-data?tab = readme-ov-file#inference //arxiv . org/abs/2503 . 20377 [32] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: [50] Sean Lie. 2022. Cerebras Architecture Deep Dive: First Look Inside the HW/SW Accurate post-training quantization for generative pre-trained transformers. Co-Design for Deep Learning : Cerebras Systems. In 2022 IEEE Hot Chips 34 arXiv preprint arXiv:2210.17323 (2022). Symposium (HCS) . 1–34. https://doi . org/10 . 1109/HCS55958 . 2022 . 9895479 [33] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, [51] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Activation-aware Weight Quantization for LLM Compression and Acceleration. Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In MLSys . In Proceedings of the ACM SIGCOMM 2024 Conference (Sydney, NSW, Australia) [52] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir (ACM SIGCOMM '24) . Association for Computing Machinery, New York, NY, Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit USA, 57–70. https://doi . org/10 . 1145/3651890 . 3672233 Quantization for KV Cache. arXiv preprint arXiv:2402.02750 (2024). [34] Patrick Geoffray and Torsten Hoefler. 2008. Adaptive Routing Strategies for [53] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Modern High Performance Networks. In 2008 16th IEEE Symposium on High Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Performance Interconnects . 165–172. https://doi . org/10 . 1109/HOTI . 2008 . 21 Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, [35] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, and Kurt Keutzer. 2024. AI and Memory Wall . IEEE Micro 44, 03 (May 2024), Philip S. Yu, and Ming Zhang. 2025. Large Language Model Agent: A Survey 33–39. https://doi . org/10 . 1109/MM . 2024 . 3373763 on Methodology, Applications and Challenges. arXiv preprint arXiv:2503.21460 [36] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and (2025). Gabriel Synnaeve. 2024. Better & Faster Large Language Models via Multi-token [54] Karthik Mandakolathur and Sylvain Jeaugey. 2022. Doubling all2all Prediction. In Forty-first International Conference on Machine Learning, ICML Performance with NVIDIA Collective Communication Library 2.12. 2024, Vienna, Austria, July 21-27, 2024 . OpenReview.net. https://openreview . net/ https://developer . nvidia . com/blog/doubling-all2all-performance-withforum?id = pEWAcejiU2 nvidia-collective-communication-library-2-12/ [37] Google. 2024. Introducing Gemini 2.0: our new AI model for the agen- [55] Mistral. 2024. Cheaper, Better, Faster, Stronger: Continuing to push the frontier tic era. https://blog . google/technology/google-deepmind/google-gemini-ai- of AI and making it accessible to all. https://mistral . ai/news/mixtral-8x22b update-december-2024 [56] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, [38] Google. 2025. Gemini 2.5: Our most intelligent AI model. https://blog . google/ Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, technology/google-deepmind/gemini-model-thinking-updates-march-2025/ Jie Amy Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyan Yang, [39] MADSys group and Approaching.AI. 2025. A Flexible Framework for Experienc- Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, ing Cutting-edge LLM Inference Optimizations. https://github . com/kvcache- Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie Insights into DeepSeek-V3: Scaling Challenges and Reflections on Hardware for AI Architectures ISCA '25, June 21–25, 2025, Tokyo, Japan

[4] Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dimitry Melts, Krishna [77] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. Dhulipala, K. R. Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi 2025. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnaku- Per-thread INT4 Quantization. arXiv:2411.10958 [cs.LG] https://arxiv . org/abs/ mar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr 2411 . 10958 Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, [78] Chenggang Zhao, Liang Zhao, Jiashi Li, and Zhean Xu. 2025. DeepGEMM: clean and Vijay Rao. 2023. Software-Hardware Co-design for Fast and Scalable Training and efficient FP8 GEMM kernels with fine-grained scaling. https://github . com/ of Deep Learning Recommendation Models. http://arxiv . org/abs/2104 . 05158 deepseek-ai/DeepGEMM. arXiv:2104.05158 [cs]. [79] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. 2025. DeepEP: an efficient [57] NVIDIA. 2022. Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async. expert-parallel communication library.

https://github . com/deepseek-ai/DeepEP. https://developer . nvidia . com/blog/improving-network-performance-of- [80] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/ Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels [58] NVIDIA. 2025. NVIDIA DGX Spark: A Grace Blackwell AI supercomputer on your desk. https://www . nvidia . com/en-us/products/workstations/dgx-spark/ using Tile-Centric Primitives. arXiv:2503.20313 [cs.DC] https://arxiv . org/abs/ [59] OpenAI. 2024. Hello GPT-4o. https://openai . com/index/hello-gpt-4o/ 2503 . 20313 [60] OpenAI. 2024. Introducing OpenAI o1. https://openai . com/o1/ [81] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin [61] OpenAI. 2025. Introducing OpenAI o3 and o4-mini. https://openai . com/index/ Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for introducing-o3-and-o4-mini/. Goodput-optimized Large Language Model Serving. In 18th USENIX Symposium [62] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang on Operating Systems Design and Implementation (OSDI 24) . USENIX Associa- Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng tion, Santa Clara, CA, 193–210. https://www . usenix . org/conference/osdi24/ Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. presentation/zhong-yinmin 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In Proceedings of the ACM SIGCOMM 2024 Conference (Sydney, NSW, Australia) (ACM SIGCOMM '24) . Association for Computing Machinery, New York, NY, USA, 691–706. https://doi . org/10 . 1145/3651890 . 3672265 [63] Zhen Qin, Weigao Sun, Dong Li, Xuyang Shen, Weixuan Sun, and Yiran Zhong. 2024. Various lengths, constant speed: efficient language modeling with lightning attention. In Proceedings of the 41st International Conference on Machine Learning (Vienna, Austria) (ICML'24) . JMLR.org, Article 1688, 19 pages. [64] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290 [cs.LG] https://arxiv . org/ abs/2305 . 18290 [65] Md Shafayat Rahman, Saptarshi Bhowmik, Yevgeniy Ryasnianskiy, Xin Yuan, and Michael Lang. 2019. Topology-custom UGAL routing on dragonfly. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (Denver, Colorado) (SC '19) . Association for Computing Machinery, New York, NY, USA, Article 17, 15 pages. https://doi . org/10 . 1145/3295500 . 3356208 [66] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, Stosic Dusan, Venmugil Elango, Maximilian Golub, Alexander Heinecke, Phil James-Roxby, Dharmesh Jani, Gaurav Kolhe, Martin Langhammer, Ada Li, Levi Melnick, Maral Mesmakhosroshahi, Andres Rodriguez, Michael Schulte, Rasoul Shafipour, Lei Shao, Michael Siu, Pradeep Dubey, Paulius Micikevicius, Maxim Naumov, Colin Verrilli, Ralph Wittig, Doug Burger, and Eric Chung. 2023. Microscaling Data Formats for Deep Learning. arXiv:2310.10537 [cs.LG] https://arxiv . org/abs/2310 . 10537 [67] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] https://arxiv . org/abs/1707 . 06347 [68] ByteDance Seed. 2025. Seed1.5-Thinking: Advancing Superb Reasoning Models with Reinforcement Learning. arXiv:2504.13914 [cs.CL] https://arxiv . org/abs/ 2504 . 13914 [69] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeek-Math: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] https://arxiv . org/abs/2402 . 03300 [70] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. CoRR abs/1911.02150 (2019). http://arxiv . org/abs/1911 . 02150 [71] Qwen Team. 2024. Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115 (2024). [72] Qwen Team. 2025. Qwen3: Think Deeper, Act Faster. https://github . com/ QwenLM/Qwen3 [73] Song Wang, Bing Yu, Wenwu Xiao, Fujun Bai, Xiaodong Long, Liang Bai, Xuerong Jia, Fengguo Zuo, Jie Tan, Yixin Guo, Peng Sun, Jun Zhou, Qiong Zhan, Sheng Hu, Yu Zhou, Yi Kang, Qiwei Ren, and Xiping Jiang. 2023. A 135 GBps/Gbit 0.66 pJ/bit Stacked Embedded DRAM with Multilayer Arrays by Fine Pitch Hybrid Bonding and Mini-TSV. In 2023 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits) . 1–2. https://doi . org/10 . 23919/ VLSITechnologyandCir57934 . 2023 . 10185427 [74] xAI. 2024. Grok-2 Beta Release. https://x . ai/news/grok-2. [75] xAI. 2024. Our Gigafactory of Compute:Colossus. https://x . ai/colossus. [76] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. 2025. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. https: //arxiv . org/abs/2502 . 11089

DeepSeek-OCR：上下文的光学压缩

DeepSeek-OCR: Contexts Optical Compression Haoran Wei, Yaofeng Sun, Yukun Li

DeepSeek-AI research@deepseek.com

arXiv:2510.18234 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-OCR，作为对通过光学二维映射压缩长上下文的可行性的初步探索。DeepSeek-OCR 包含两个组件：DeepEncoder 以及作为解码器的 DeepSeek3B-MoE-A570M。具体而言，DeepEncoder 是核心引擎，其设计目标是在高分辨率输入下保持低激活，同时实现高压缩比，以确保视觉 token 数量最优且可控。实验表明，当文本 token 数量不超过视觉 token 数量的 10 倍（即压缩比 ＜10×）时，模型可达到 97% 的解码（OCR）精度；即使在 20× 的压缩比下，OCR 准确率仍保持在约 60%。这表明其在历史长上下文压缩、LLM 记忆遗忘机制等研究方向上具有可观的前景。除此之外，DeepSeek-OCR 还展现出很高的实用价值。在 OmniDocBench 上，它仅用 100 个视觉 token 就超过了 GOT-OCR2.0（每页 256 个 token），并在使用少于 800 个视觉 token 的情况下优于 MinerU2.0（平均每页 6000+ 个 token）。在生产环境中，DeepSeek-OCR 每天可生成 20 万页以上规模的 LLM/VLM 训练数据（单块 A100-40G）。代码与模型权重已在 http://github.com/deepseek-ai/DeepSeek-OCR 公开。

【图表】图 1 (a) 展示了在 Fox [21] 基准上测试的压缩比（真值中的文本 token 数 / 模型使用的视觉 token 数）；(b) 展示了在 OmniDocBench [27] 上的性能对比。DeepSeek-OCR 在端到端模型中取得了最先进的性能，同时使用的视觉 token 最少（原文第 1 页）

1 引言

当前的大语言模型（LLM）在处理长文本内容时面临显著的计算挑战，因为计算量随序列长度呈二次方增长。我们探索一种潜在解决方案：利用视觉模态作为文本信息的高效压缩媒介。一张包含文档文字的图像可以用远少于等价数字文本的 token 表达丰富信息，这表明通过视觉 token 进行光学压缩有望实现更高的压缩比。

这一洞见促使我们从以 LLM 为中心的视角重新审视视觉-语言模型（VLM），聚焦于视觉编码器如何增强 LLM 处理文本信息的效率，而非人类本就擅长的基础 VQA [12, 16, 24, 32, 41]。OCR 任务作为连接视觉与语言的中间模态，为这一视觉-文本压缩范式提供了理想的试验平台，因为它在视觉表示与文本表示之间建立了自然的压缩-解压映射，同时提供了可量化的评测指标。

据此，我们提出 DeepSeek-OCR——一个旨在作为高效视觉-文本压缩初步概念验证（proof-of-concept）的 VLM。我们的工作有三项主要贡献：

第一，我们对视觉-文本 token 压缩比进行了全面的定量分析。在包含多种文档版式的 Fox [21] 基准上，我们的方法在 9–10× 文本压缩下取得 96% 以上的 OCR 解码精度，在 10–12× 压缩下约为 90%，在 20× 压缩下约为 60%（若计入输出与真值之间的格式差异，实际准确率还会更高），如图 1(a) 所示。结果表明，紧凑的语言模型能够有效学习解码压缩后的视觉表示，这意味着更大的 LLM 通过恰当的预训练设计也能容易地获得类似能力。

第二，我们提出 DeepEncoder——一种新颖的架构，即使在高分辨率输入下也能保持较低的激活内存与极少的视觉 token。它通过一个 16× 卷积压缩器，将窗口注意力编码器组件与全局注意力编码器组件串联起来。这种设计确保窗口注意力组件处理大量视觉 token，而压缩器在视觉 token 进入密集全局注意力组件之前对其进行缩减，从而实现有效的内存与 token 压缩。

第三，我们基于 DeepEncoder 与 DeepSeek3B-MoE [19, 20] 开发了 DeepSeek-OCR。如图 1(b) 所示，它在 OmniDocBench 上取得了端到端模型中的最先进性能，同时使用的视觉 token 最少。此外，我们还赋予模型解析图表、化学公式、简单几何图形与自然图像的能力，以进一步增强其实用性。在生产环境中，DeepSeek-OCR 使用 20 个节点（每节点 8 块 A100-40G GPU）每天可为 LLM 或 VLM 生成 3300 万页数据。

总之，本文初步探索了将视觉模态用作 LLM 中文本信息处理的高效压缩媒介。通过 DeepSeek-OCR，我们证明视觉-文本压缩能够在不同的历史上下文阶段实现显著的 token 缩减（7–20×），为解决大语言模型的长上下文挑战提供了一个有前景的方向。我们的定量分析为 VLM 的 token 分配优化

提供了经验性指导，而所提出的 DeepEncoder 架构则以真实部署能力展示了实际可行性。尽管本文以 OCR 作为概念验证，但这一范式为重新思考视觉与语言模态如何协同结合、以提升大规模文本处理与智能体系统中的计算效率开启了新的可能。

【图表】图 2 主流 VLM 中典型的视觉编码器。这里展示了当前开源 VLM 常用的三类编码器，它们各自都存在缺陷（原文第 4 页）

2 相关工作

2.1 VLM 中的典型视觉编码器

当前的开源 VLM 采用三类主要的视觉编码器，如图 2 所示。第一类是以 Vary [36] 为代表的双塔（dual-tower）架构，它利用并行的 SAM [17] 编码器来增加视觉词表参数，以处理高分辨率图像。尽管该方案提供了可控的参数量与激活内存，但它存在显著缺陷：它需要进行两次图像预处理，这不仅使部署复杂化，还使训练期间的编码器流水线并行变得困难。第二类是以 InternVL2.0 [8] 为代表的基于图块（tile）的方法，它将图像切分为小图块并行计算，从而降低高分辨率设置下的激活内存。尽管该方法能够处理极高的分辨率，但由于其原生编码器分辨率通常较低（低于 512×512），它存在明显局限：大图像被过度切分，产生大量视觉 token。第三类是以 Qwen2-VL [35] 为代表的自适应分辨率编码，它采用 NaViT [10] 范式，通过基于 patch 的分割直接处理完整图像，而不进行图块并行化。尽管这种编码器能够灵活处理多种分辨率，但在处理大图像时面临巨大挑战，原因是其激活内存消耗巨大，可能导致 GPU 显存溢出；同时序列打包在训练期间需要极长的序列长度。过长的视觉 token 会拖慢推理的预填充（prefill）与生成（generation）两个阶段。

2.2 端到端 OCR 模型

OCR，尤其是文档解析任务，一直是图像到文本领域高度活跃的研究主题。随着 VLM 的进步，大量端到端 OCR 模型涌现出来，它们通过简化 OCR 系统，从根本上改变了传统流水线架构（传统架构需要独立的检测与识别专家模型）。Nougat [6] 首次采用端到端框架对 arXiv 上的学术论文进行 OCR，展示了模型处理密集感知任务的潜力。GOT-OCR2.0 [38] 将 OCR2.0 的范围扩展到更多合成图像解析任务，并设计了兼顾性能与效率的 OCR 模型，进一步凸显了端到端 OCR 研究的潜力。此外，Qwen-VL 系列 [35]、InternVL 系列 [8] 等通用视觉模型及其众多衍生模型不断增强其文档 OCR 能

力，以探索密集视觉感知的边界。然而，当前模型尚未解决的一个关键研究问题是：对于一份包含 1000 个词的文档，解码至少需要多少个视觉 token？这一问题对"一图胜千言"原理的研究具有重要意义。

【图表】图 3 DeepSeek-OCR 的架构。DeepSeek-OCR 由一个 DeepEncoder 与一个 DeepSeek-3B-MoE 解码器组成。DeepEncoder 是 DeepSeek-OCR 的核心，包含三个组件：以窗口注意力为主的、负责感知的 SAM [17]；具有密集全局注意力的、负责知识的 CLIP [29]；以及连接二者的 16× token 压缩器（原文第 5 页）

3 方法

3.1 架构

如图 3 所示，DeepSeek-OCR 采用统一的端到端 VLM 架构，由编码器与解码器组成。编码器（即 DeepEncoder）负责提取图像特征，并对视觉表示进行 token 化与压缩。解码器则根据图像 token 与提示生成所需结果。DeepEncoder 的参数量约为 380M，主要由串联的 80M SAM-base [17] 与 300M CLIP-large [29] 组成。解码器采用 3B MoE [19, 20] 架构，激活参数为 570M。在接下来的段落中，我们将深入介绍模型组件、数据工程与训练技巧。

3.2 DeepEncoder

为探索上下文光学压缩的可行性，我们需要一个具备以下特性的视觉编码器：（1）能够处理高分辨率；（2）在高分辨率下激活内存低；（3）视觉 token 少；（4）支持多种分辨率输入；（5）参数量适中。然而，如第 2.1 节所述，当前的开源编码器无法完全满足所有这些条件。因此，我们自行设计了一种新颖的视觉编码器，命名为 DeepEncoder。

3.2.1 DeepEncoder 的架构

DeepEncoder 主要包含两个组件：以窗口注意力为主的视觉感知特征提取组件，以及具有密集全局注意力的视觉知识特征提取组件。为受益于先前工作的预训练积累，我们分别采用 SAM-base（patch size 为 16）与 CLIP-large 作为这两个组件的主干架构。对于 CLIP，我们移除了第一个 patch 嵌入层，因为其输入已不再是图像，而是前一流程输出的 token。在两个组件之间，我们借鉴 Vary [36]，使用一个 2 层卷积模块对视觉 token 进行 16× 下采样。每个卷积层的卷积核大小为 3、步长为 2、填充为 1，通道数从 256 增加到 1024。假设输入一张 1024×1024 的图像，DeepEncoder 会将其分割为

1024/16×1024/16=4096 个 patch token。由于编码器的前半部分以窗口注意力为主且仅有 80M，激活内存是可接受的。在进入全局注意力之前，这 4096 个 token 会经过压缩模块，token 数量变为 4096/16=256，从而使整体激活内存可控。

【图表】图 4 为测试不同压缩比（需要不同数量的视觉 token）下的模型性能并增强 DeepSeek-OCR 的实用性，我们为其配置了多种分辨率模式（原文第 6 页）

表 1：DeepEncoder 的多分辨率支持。为兼顾研究与实际应用，我们为 DeepEncoder 设计了多样化的原生分辨率与动态分辨率模式（其中 Tiny、Small、Base、Large 为原生分辨率模式，Gundam 与 Gundam-M 为动态分辨率模式）。（原文第 6 页）

模式	Tiny	Small	Base	Large	Gundam	Gundam-M
原生分辨率	512	640	1024	1280	640+1024	1024+1280
视觉 token 数	64	100	256	400	n×100+256	n×256+400
处理方式	resize	resize	padding	padding	resize + padding	resize + padding

3.2.2 多分辨率支持

假设我们有一张包含 1000 个光学字符的图像，想要测试解码需要多少个视觉 token。这要求模型支持可变数量的视觉 token，也就是说，DeepEncoder 需要支持多种分辨率。

我们通过位置编码的动态插值来满足上述需求，并设计了多种分辨率模式用于模型的同步训练，从而使单个 DeepSeek-OCR 模型具备支持多种分辨率的能力。如图 4 所示，DeepEncoder 主要支持两种输入模式：原生分辨率与动态分辨率；每种模式又包含多个子模式。

原生分辨率支持四个子模式：Tiny、Small、Base 与 Large，其对应的分辨率与 token 数分别为 512×512（64）、640×640（100）、1024×1024（256）与 1280×1280（400）。由于 Tiny 与 Small 模式的分辨率相对较小，为避免浪费视觉 token，图像会直接按原始形状缩放（resize）处理。对于 Base 与 Large 模式，为保持原始图像的宽高比，图像会被填充（pad）到对应尺寸。填充后，有效视觉 token 数少于实际视觉 token 数，计算公式为：

$$N_{valid} = \lceil N_{actual} \times [1 - ((\max(w,h) - \min(w,h)) / \max(w,h))] \rceil \quad (1)$$

其中 w 与 h 分别表示原始输入图像的宽度与高度。

动态分辨率可由两种原生分辨率组合而成。例如，Gundam 模式由 n×640×640 的图块（局部视图）加上一个 1024×1024 的全局视图组成，其切块方法遵循 InternVL2.0 [8]。支持动态分辨率主要是出于应用层面的考虑，尤其是针对超高分辨率输入（例如报纸图像）。切块（tiling）是一种次级窗口注意力的形式，能够进一步有效降低激活内存。值得注意的是，由于我们的原生分辨率相对较大，图像在动态分辨率下不会被过度切分（图块数量控制在 2 到 9 的范围内）。在 Gundam 模式下，DeepEncoder 输出的视觉 token 数为 n×100+256，其中 n 为图块数量。对于宽度与高度均小于 640 的图像，n 被设为 0，即 Gundam 模式会退化为 Base 模式。

Gundam 模式与四种原生分辨率模式一起训练，以实现单个模型支持多种分辨率的目标。需要注意的是，Gundam-master 模式（1024×1024 的局部视图 + 1280×1280 的全局视图）是在已训练好的 DeepSeek-OCR 模型上继续训练得到的。这主要是为了负载均衡，因为 Gundam-master 的分辨率过大，将其一并训练会拖慢整体训练速度。

3.3 MoE 解码器

我们的解码器采用 DeepSeekMoE [19, 20]，具体为 DeepSeek-3B-MoE。推理时，模型从 64 个路由专家中激活 6 个，并激活 2 个共享专家，激活参数约为 570M。3B 的 DeepSeekMoE 非常适合以领域为中心（对我们而言是 OCR）的 VLM 研究，因为它既具备 3B 模型的表达能力，又享有 500M 小模型的推理效率。

解码器从 DeepEncoder 输出的压缩潜在视觉 token 中重建原始文本表示，如下式所示：

f_dec: R^{n×d_latent} → R^{N×d_text}；$\hat{X}$ = f_dec(Z)，其中 n ≤ N　　（2）

其中 Z ∈ R^{n×d_latent} 是 DeepEncoder 输出的压缩潜在（视觉）token，$\hat{X}$ ∈ R^{N×d_text} 是重建的文本表示。函数 f_dec 表示一种非线性映射，紧凑的语言模型可以通过 OCR 风格训练有效学习该映射。可以合理推测，LLM 通过专门的预训练优化，将能更自然地整合此类能力。

3.4 数据引擎

我们为 DeepSeek-OCR 构建了复杂多样的训练数据，包括：OCR 1.0 数据，主要由场景图像 OCR 与文档 OCR 等传统 OCR 任务组成；OCR 2.0 数据，主要包含复杂人造图像的解析任务，例如常见图表、化学公式与平面几何的解析数据；通用视觉数据，主要用于向 DeepSeek-OCR 注入一定的通用图像理解能力，并保留通用视觉接口。

3.4.1 OCR 1.0 数据

文档数据是 DeepSeek-OCR 的首要任务。我们从互联网收集了 3000 万页、覆盖约 100 种语言的多样化 PDF 数据，其中中文与英文合计约 2500 万页，其他语言约 500 万页。对于这些数据，我们创建两类真值（ground truth）：粗标注（coarse annotations）与精标注（fine annotations）。粗标注的提取方式为，直接使用 fitz 从完整数据集中提取，旨在教会模型识别光学文本，尤其是小语种文本。细标注包括中文与英文各 200 万页，使用先进的布局模型（如 PP-DocLayout [33]）与 OCR 模型（如 MinerU [34] 与 GOT-OCR2.0 [38]）进行标注，以构建检测与识别交错的数据。对于小语种，在检测部分，我们发现布局模型具有一定的泛化能力。在识别部分，我们使用 fitz 创建小块（patch）数据来训练一个 GOT-OCR2.0，然后使用训练后的模型对布局处理后的小块进行标注，并借助模型飞轮（model flywheel）创建 60 万条数据样本。在 DeepSeek-OCR 的训练过程中，粗标注与细标注通过不同的提示加以区分。细标注图文对的地面真值可参见图 5。我们还收集了 300 万条 Word 数据，通过直接提取内容构建无布局的高质量图文对。这些数据主要对公式与 HTML 格式的表格有益。此外，我们还选取了一些开源数据 [28, 37] 作为补充。

【图表】图 5 OCR 1.0 细标注展示。我们将地面真值格式化为布局与文本交错（interleaved）的形式，其中每一段文本之前都带有其在原图中的坐标与标签。所有坐标均归一化到 1000 个区间（bins）（原文第 8 页）

对于自然场景 OCR，我们的模型主要支持中文与英文。图像数据来源为 LAION [31] 与 Wukong [13]，使用 PaddleOCR [9] 进行标注，中文与英文各 1000 万条数据样本。与文档 OCR 类似，自然场景 OCR 也可以通过提示控制是否输出检测框。

3.4.2 OCR 2.0 数据

遵循 GOT-OCR2.0 [38]，我们将图表、化学式与平面几何解析数据称为 OCR 2.0 数据。对于图表数据，遵循 OneChart [7] 的做法，我们使用 pyecharts 与 matplotlib 渲染 1000 万张图像，主要包括常用的折线图、柱状图、饼图与复合图表。我们将图表解析定义为图像到 HTML 表格的转换任务，如图 6(a) 所示。对于化学式，我们采用 PubChem 的 SMILES 格式作为数据来源，并使用 RDKit 将其渲染为图像，构建 500 万对图文数据。对于平面几何图像，我们遵循 Slow Perception [39] 进行生成。具体而言，我们使用大小为 4 的感知标尺（perception-ruler）对每条线段建模。为增加渲染数据的多样性，我们引入了几何平移不变的数据增强：同一几何图像在原图中平移后，对应于绘制在坐标系中心位置处的同一地面真值。基于此，我们总共构建 100 万条平面几何解析数据，如图 6(b) 所示。

【图表】图 6 对于图表，我们没有采用 OneChart [7] 的字典格式，而是使用 HTML 表格格式作为标签，这可以节省一定数量的 token。对于平面几何，我们将地面真值转换为字典格式，字典包含线段、端点坐标、线段类型等键，以便更好地阅读。每条线段均使用 Slow Perception [39] 的方式编码（原文第 9 页）

3.4.3 通用视觉数据

DeepEncoder 可以受益于 CLIP 的预训练收益，并且拥有足够的参数来纳入通用视觉知识。因此，我们也为 DeepSeek-OCR 准备了相应的数据。遵循 DeepSeek-VL2 [40]，我们为 caption、检测与 grounding 等任务生成相关数据。需要注意的是，DeepSeek-OCR 并非通用 VLM 模型，这部分数据仅占总数据的 20%。我们引入此类数据主要是为了保留通用视觉接口，以便对我们模型与通用视觉任务感兴趣的研究者能在未来方便地推进他们的工作。

3.4.4 纯文本数据

为确保模型的语言能力，我们引入了 10% 的内部纯文本预训练数据，所有数据均被处理为 8192 个 token 的长度，这也是 DeepSeek-OCR 的序列长度。综上，在训练 DeepSeek-OCR 时，OCR 数据占 70%，通用视觉数据占 20%，纯文本数据占 10%。

3.5 训练流水线

我们的训练流水线非常简单，主要由两个阶段组成：a) 独立训练 DeepEncoder；b) 训练 DeepSeek-OCR。注意，Gundam-master 模式是在已预训练的 DeepSeek-OCR 模型上使用 600 万条采样数据继续训练得到的。由于训练协议与其他模式相同，我们在后文省略详细描述。

3.5.1 训练 DeepEncoder

遵循 Vary [36]，我们利用一个紧凑的语言模型 [15]，并使用下一 token 预测框架来训练 DeepEncoder。在此阶段，我们使用前述所有 OCR 1.0 与 2.0 数据，以及从 LAION [31] 数据集中采样的 1 亿条通用数据。所有数据训练 2 个 epoch，批大小为 1280，使用 AdamW [23] 优化器、余弦退火调度器 [22] 与 5e-5 的学习率。训练序列长度为 4096。

3.5.2 训练 DeepSeek-OCR

在 DeepEncoder 准备就绪后，我们使用第 3.4 节所述的数据训练 DeepSeek-OCR，整个训练过程在 HAI-LLM [14] 平台上进行。整个模型采用流水线并行（pipeline parallelism, PP），被划分为 4 个部

分，其中 DeepEncoder 占两个部分，解码器占两个部分。对于 DeepEncoder，我们将 SAM 与压缩器视为视觉分词器，将它们放在 PP0 并冻结其参数；同时将 CLIP 部分视为输入嵌入层，放在 PP1 并保持权重不冻结以进行训练。对于语言模型部分，由于 DeepSeek3B-MoE 有 12 层，我们在 PP2 与 PP3 上各放置 6 层。我们使用 20 个节点（每个节点 8 张 A100-40G GPU）进行训练，数据并行度（data parallelism, DP）为 40，全局批大小为 640。我们使用 AdamW 优化器与基于步数的调度器，初始学习率为 3e-5。对于纯文本数据，训练速度为每天 90B token；对于多模态数据，训练速度为每天 70B token。

4 评估

4.1 视觉-文本压缩研究

我们选择 Fox [21] 基准来验证 DeepSeek-OCR 对富含文本文档的压缩-解压能力，以初步探索上下文光学压缩的可行性与边界。我们使用 Fox 的英文文档部分，用 DeepSeek-OCR 的分词器（词表大小约 129k）对地面真值文本分词，并选取 600–1300 个 token 的文档进行测试，恰好共 100 页。由于文本 token 数不大，我们只需测试 Tiny 与 Small 模式下的性能，其中 Tiny 模式对应 64 个 token，Small 模式对应 100 个 token。我们使用不带布局的提示 “<image>\nFree OCR.”来控制模型的输出格式。尽管如此，输出格式仍无法完全匹配 Fox 基准，因此实际性能会比测试结果略高一些。

如表 2 所示，在 10 倍压缩率以内，模型的解码精度可以达到约 97%，这是一个非常有前景的结果。未来，或许有可能通过文本到图像的方法实现接近 10 倍的无损上下文压缩。当压缩率超过 10 倍时，性能开始下降，这可能有两个原因：其一是长文档的布局变得更加复杂；另一个原因可能是长文本在 512×512 或 640×640 分辨率下变得模糊。第一个问题可以通过将文本渲染到单个布局页面来解决；而我们认为第二个问题将成为遗忘机制（forgetting mechanism）的一个特性。当把 token 压缩近 20 倍时，我们发现精度仍能接近 60%。这些结果表明，光学上下文压缩是一个非常有前景且值得研究的方向，而且该方法不会带来任何额外开销，因为它可以利用 VLM 基础设施——多模态系统本身就需要一个额外的视觉编码器。

【表格】表 2 我们使用 Fox [21] 基准中所有 600–1300 个 token 的英文文档来测试 DeepSeek-OCR 的视觉-文本压缩率。文本 token 表示使用 DeepSeek-OCR 的分词器对地面真值文本分词后的 token 数。Vision Tokens=64 或 100 分别表示将输入图像调整为 512×512 与 640×640 后 DeepEncoder 输出的视觉 token 数（原文第 10 页）

4.2 OCR 实用性能

DeepSeek-OCR 不仅是一个实验性模型；它具有强大的实用能力，可以为 LLM/VLM 预训练构建数据。为了量化 OCR 性能，我们在 OmniDocBench [27] 上测试 DeepSeek-OCR，结果如表 3 所示。仅需 100 个视觉 token（640×640 分辨率），DeepSeek-OCR 就超过了使用 256 个 token 的 GOT-OCR2.0 [38]；使用 400 个 token（285 个有效 token，1280×1280 分辨率）时，它在该基准上取得了与最先进方法相当的性能。使用少于 800 个 token（Gundam 模式）时，DeepSeek-OCR 优于需要近 7000 个视觉 token 的 MinerU2.0 [34]。这些结果表明，我们的 DeepSeek-OCR 模型在实际应用中很强大，并且由于更高的 token 压缩率，它具有更高的研究上限。

【表格】表 3 我们使用 OmniDocBench [27] 测试 DeepSeek-OCR 在真实文档解析任务上的性能。表中的所有指标均为编辑距离，数值越小表示性能越好。“Tokens”表示每页平均使用的视觉 token 数，“†200dpi”表示使用 fitz 将原始图像插值到 200dpi。对于 DeepSeek-OCR 模型，“Tokens”列中括号内的数值表示有效视觉 token 数，按公式 1 计算（原文第 11 页）

如表 4 所示，某些类别的文档只需很少的 token 即可取得令人满意的性能，例如幻灯片仅需 64 个视觉 token。对于书籍与报告类文档，DeepSeek-OCR 仅用 100 个视觉 token 即可取得良好性能。结合第 4.1 节的分析，这可能是因为这些文档类别中的大多数文本 token 都在 1000 以内，意味着视觉 token 压缩率不超过 10 倍。对于报纸，则需要 Gundam 甚至 Gundam-master 模式才能达到可接受的编辑距离，因为报纸中的文本 token 有 4000–5000 个，远超其他模式的 10 倍压缩。这些实验结果进一步展示了上下文光学压缩的边界，可为 VLM 中视觉 token 优化以及 LLM 中上下文压缩、遗忘机制的研究提供有效参考。

【表格】表 4 OmniDocBench 中不同类别文档的编辑距离。结果表明，某些类型的文档仅用 64 或 100 个视觉 token 即可取得良好性能，而另一些则需 Gundam 模式（原文第 12 页）

4.3 定性研究

4.3.1 深度解析

DeepSeek-OCR 同时具备布局与 OCR 2.0 能力，使其能够通过二次模型调用来进一步解析文档内的图像，我们将这一特性称为“深度解析”（deep parsing）。如图 7、8、9、10 所示，我们的模型可以对图表、几何、化学式乃至自然图像进行深度解析，且只需一个统一的提示。

【图表】图 7 在金融研究报告领域，DeepSeek-OCR 的深度解析模式可用于获取文档内图表的结构化结果。图表是金融与科学领域中至关重要的数据表示形式，而图表结构化提取是未来 OCR 模型不可或缺的能力（原文第 13 页）

【图表】图 8 对于书籍与文章，深度解析模式可以为文档中的自然图像输出稠密描述。仅需一个提示，模型即可自动识别图像类型并输出所需结果。（原文第 14 页）

【图表】图 9 深度解析模式下的 DeepSeek-OCR 还能识别化学文档中的化学式，并将其转换为 SMILES 格式。未来，OCR 1.0+2.0 技术可能在 STEM 领域的 VLM/LLM 发展中发挥重要作用。（原文第 15 页）

【图表】图 10 DeepSeek-OCR 还具备复制（并结构化）简单平面几何图形的能力。由于几何图形中各线段之间存在错综复杂的相互依赖关系，几何解析任务极具挑战，仍有很长的路要走。（原文第 16 页）

4.3.2 多语言识别

互联网上的 PDF 数据不仅包含中文与英文，还包含大量多语言数据，这些数据在训练 LLM 时同样至关重要。对于 PDF 文档，DeepSeek-OCR 可以处理近 100 种语言。与中文、英文文档一样，多语言数据也同时支持带版面与不带版面的 OCR 格式。可视化结果如图 11 所示，我们选取阿拉伯语与僧伽罗语来展示结果。

【图表】图 11 为具备处理广泛爬取的 PDF（多语言数据）的能力，我们训练模型掌握近 100 种语言的 OCR 能力。小语种文档同样可以通过不同提示支持带版面与不带版面的输出。（原文第 17 页）

4.3.3 通用视觉理解

我们还赋予 DeepSeek-OCR 一定程度的通用图像理解能力。相关可视化结果如图 12 所示。

【图表】图 12 我们保留了 DeepSeek-OCR 的通用视觉理解能力，主要包括图像描述、目标检测、指代定位（grounding）等。同时，由于加入了纯文本数据，DeepSeek-OCR 的语言能力也得以保留。请注意，由于我们没有包含 SFT（监督微调）阶段，该模型并非聊天机器人，部分能力需要补全式提示才能激活。（原文第 18 页）

5 讨论

我们的工作是对视觉-文本压缩边界的一次初步探索，研究解码 N 个文本 token 需要多少个视觉 token。初步结果令人鼓舞：DeepSeek-OCR 在大约 10× 的压缩比下实现了近乎无损的 OCR 压缩，而在 20× 压缩下仍保持 60% 的准确率。这些发现为未来应用指明了有前景的方向，例如对多轮对话中超过 k 轮的历史对话实施光学处理，以达到 10× 的压缩效率。

【图表】图 13 遗忘机制是人类记忆最基本的特征之一。上下文光学压缩方法可以模拟这一机制：将此前若干轮的历史文本渲染为图像以完成初步压缩，然后逐步缩小较旧的图像以实现多级压缩，在此过程中 token 数量逐渐减少、文字逐渐模糊，从而实现文本遗忘。（原文第 19 页）

对于更早的上下文，我们可以逐步缩小已渲染的图像，以进一步降低 token 消耗。这一设想的灵感来自人类记忆随时间衰减与视觉感知随空间距离退化之间的自然类比——两者都呈现出相似的渐进式信息损失模式，如图 13 所示。通过结合这些机制，上下文光学压缩方法实现了一种与生物遗忘曲线相仿的记忆衰减：近期信息保持高保真，而久远的记忆则因压缩比提高而自然淡出。

尽管我们的初步探索显示了可扩展的超长上下文处理的潜力——近期上下文保持高分辨率，较早的上下文消耗更少资源——但我们承认这仍是早期工作，需要进一步研究。该方法为理论上的无限上下文架构指明了一条路径，可在信息保留与计算约束之间取得平衡；不过，此类视觉-文本压缩系统的实际影响与局限仍需在未来研究中深入考察。

6 结论

在本技术报告中，我们提出 DeepSeek-OCR，并通过该模型初步验证了上下文光学压缩的可行性，证明该模型能够从少量视觉 token 中有效解码出数量超过其 10 倍的文本 token。我们相信这一发现将促进未来 VLM 与 LLM 的发展。此外，DeepSeek-OCR 是一个极具实用价值的模型，能够支持大规模预训练数据生产，是 LLM 不可或缺的助手。当然，仅凭 OCR 不足以完全验证真正的上下文光学压缩，我们未来将开展数字-光学文本交错预训练、大海捞针（needle-in-a-haystack）测试以及其他评测。从另一个角度看，光学上下文压缩仍有很大的研究与改进空间，代表着一个有前景的新方向。

附录

附录：补充材料见英文原文

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] [1] Marker. URL https://github.com/datalab-to/marker .

[2] [3] Ocrflux, 2025. URL https://github.com/chatdoc-com/OCRFlux .

[3] [4] G. AI. Gemini 2.5-pro, 2025. URL https://gemini.google.com/ .

[4] [5] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P . Wang, S. Wang, J. Tang, H. Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, P . Wang, W. Ding, Z. Fu, Y. Xu, J. Ye, X. Zhang, T. Xie, Z. Cheng, H. Zhang, Z. Yang, H. Xu, and J. Lin. Qwen2.5-vl technical report. arXiv preprint arXiv:2502.13923, 2025.

[5] [6] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic. Nougat: Neural optical understanding for academic documents. arXiv preprint arXiv:2308.13418, 2023.

[6] [7] J. Chen, L. Kong, H. Wei, C. Liu, Z. Ge, L. Zhao, J. Sun, C. Han, and X. Zhang. Onechart: Purify the chart structural extraction via one auxiliary token. In Proceedings of the 32nd ACM International Conference on Multimedia, pages 147–155, 2024.

[7] [8] Z. Chen, W. Wang, H. Tian, S. Ye, Z. Gao, E. Cui, W. Tong, K. Hu, J. Luo, Z. Ma, et al. How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites. arXiv preprint arXiv:2404.16821, 2024.

[8] [9] C. Cui, T. Sun, M. Lin, T. Gao, Y. Zhang, J. Liu, X. Wang, Z. Zhang, C. Zhou, H. Liu, et al. Paddleocr 3.0 technical report. arXiv preprint arXiv:2507.05595, 2025.

[9] [10] M. Dehghani, J. Djolonga, B. Mustafa, P . Padlewski, J. Heek, J. Gilmer, A. Steiner, M. Caron, R. Geirhos, I. Alabdulmohsin, et al. Patch n' pack: Navit, a vision transformer for any aspect ratio and resolution. Advances in Neural Information Processing Systems, 36:3632–3656, 2023.

[10] [11] H. Feng, S. Wei, X. Fei, W. Shi, Y. Han, L. Liao, J. Lu, B. Wu, Q. Liu, C. Lin, et al. Dolphin: Document image parsing via heterogeneous anchor prompting. arXiv preprint arXiv:2505.14059, 2025.

[11] [12] Y. Goyal, T. Khot, D. Summers-Stay, D. Batra, and D. Parikh. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 6904–6913, 2017.

[12] [13] J. Gu, X. Meng, G. Lu, L. Hou, N. Minzhe, X. Liang, L. Yao, R. Huang, W. Zhang, X. Jiang, et al. Wukong: A 100 million large-scale chinese cross-modal pre-training benchmark. Advances in Neural Information Processing Systems, 35:26418–26431, 2022.

[13] [14] High-flyer. HAI-LLM: Efficient and lightweight training tool for large models, 2023. URL https://www.high-flyer.cn/en/blog/hai-llm .

[14] [15] S. Iyer, X. V . Lin, R. Pasunuru, T. Mihaylov, D. Simig, P . Yu, K. Shuster, T. Wang, Q. Liu, P . S. Koura, et al. Opt-iml: Scaling language model instruction meta learning through the lens of generalization. arXiv preprint arXiv:2212.12017, 2022.

[15] [16] S. Kazemzadeh, V . Ordonez, M. Matten, and T. Berg. Referitgame: Referring to objects in photographs of natural scenes. In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pages 787–798, 2014. [17] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al. Segment anything. arXiv preprint arXiv:2304.02643, 2023.

[16] [18] Z. Li, Y. Liu, Q. Liu, Z. Ma, Z. Zhang, S. Zhang, Z. Guo, J. Zhang, X. Wang, and X. Bai. Monkeyocr: Document parsing with a structure-recognition-relation triplet paradigm. arXiv preprint arXiv:2506.05218, 2025.

[17] [19] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024.

[18] [20] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.

[19] [21] C. Liu, H. Wei, J. Chen, L. Kong, Z. Ge, Z. Zhu, L. Zhao, J. Sun, C. Han, and X. Zhang. Focus anywhere for fine-grained multi-page document understanding. arXiv preprint arXiv:2405.14295, 2024.

[20] [22] I. Loshchilov and F. Hutter. Sgdr: Stochastic gradient descent with warm restarts. arXiv preprint arXiv:1608.03983, 2016.

[21] ICLR, 2019. [23] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In

[22] [24] A. Masry, D. X. Long, J. Q. Tan, S. Joty, and E. Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. arXiv preprint arXiv:2203.10244, 2022.

[23] [25] A. Nassar, A. Marafioti, M. Omenetti, M. Lysak, N. Livathinos, C. Auer, L. Morin, R. T. de Lima, Y. Kim, A. S. Gurbuz, et al. Smoldocling: An ultra-compact vision-language model for end-to-end multi-modal document conversion. arXiv preprint arXiv:2503.11576, 2025.

[24] [26] OpenAI. Gpt-4 technical report, 2023.

[25] [27] L. Ouyang, Y. Qu, H. Zhou, J. Zhu, R. Zhang, Q. Lin, B. Wang, Z. Zhao, M. Jiang, X. Zhao, et al. Omnidocbench: Benchmarking diverse pdf document parsing with comprehensive annotations. In Proceedings of the Computer Vision and Pattern Recognition Conference, pages 24838–24848, 2025.

[26] [28] J. Poznanski, A. Rangapur, J. Borchardt, J. Dunkelberger, R. Huff, D. Lin, C. Wilhelm, K. Lo, and L. Soldaini. olmocr: Unlocking trillions of tokens in pdfs with vision language models. arXiv preprint arXiv:2502.18443, 2025.

[27] [29] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P . Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In International conference on machine learning, pages 8748–8763. PMLR, 2021.

[28] [30] Rednote. dots.ocr, 2025. URL https://github.com/rednote-hilab/dots.ocr .

[29] [31] C. Schuhmann, R. Vencu, R. Beaumont, R. Kaczmarczyk, C. Mullis, A. Katta, T. Coombes, J. Jitsev, and A. Komatsuzaki. Laion-400m: Open dataset of clip-filtered 400 million imagetext pairs. arXiv preprint arXiv:2111.02114, 2021. [32] A. Singh, V . Natarajan, M. Shah, Y. Jiang, X. Chen, D. Batra, D. Parikh, and M. Rohrbach. Towards vqa models that can read. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 8317–8326, 2019.

[30] [33] T. Sun, C. Cui, Y. Du, and Y. Liu. Pp-doclayout: A unified document layout detection model to accelerate large-scale data construction. arXiv preprint arXiv:2503.17213, 2025.

[31] [34] B. Wang, C. Xu, X. Zhao, L. Ouyang, F. Wu, Z. Zhao, R. Xu, K. Liu, Y. Qu, F. Shang, et al. Mineru: An open-source solution for precise document content extraction. arXiv preprint arXiv:2409.18839, 2024.

[32] [35] P . Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. arXiv preprint arXiv:2409.12191, 2024.

[33] [36] H. Wei, L. Kong, J. Chen, L. Zhao, Z. Ge, J. Yang, J. Sun, C. Han, and X. Zhang. Vary: Scaling up the vision vocabulary for large vision-language model. In European Conference on Computer Vision, pages 408–424. Springer, 2024.

[34] [37] H. Wei, L. Kong, J. Chen, L. Zhao, Z. Ge, E. Yu, J. Sun, C. Han, and X. Zhang. Small language model meets with reinforced vision vocabulary. arXiv preprint arXiv:2401.12503, 2024.

[35] [38] H. Wei, C. Liu, J. Chen, J. Wang, L. Kong, Y. Xu, Z. Ge, L. Zhao, J. Sun, Y. Peng, et al. General ocr theory: Towards ocr-2.0 via a unified end-to-end model. arXiv preprint arXiv:2409.01704, 2024.

[36] [39] H. Wei, Y. Yin, Y. Li, J. Wang, L. Zhao, J. Sun, Z. Ge, X. Zhang, and D. Jiang. Slow perception: Let's perceive geometric figures step-by-step. arXiv preprint arXiv:2412.20631, 2024.

[37] [40] Z. Wu, X. Chen, Z. Pan, X. Liu, W. Liu, D. Dai, H. Gao, Y. Ma, C. Wu, B. Wang, et al. Deepseek-vl2: Mixture-of-experts vision-language models for advanced multimodal understanding. arXiv preprint arXiv:2412.10302, 2024.

[38] [41] W. Yu, Z. Yang, L. Li, J. Wang, K. Lin, Z. Liu, X. Wang, and L. Wang. Mm-vet: Evaluating large multimodal models for integrated capabilities. arXiv preprint arXiv:2308.02490, 2023.

[39] [42] J. Zhu, W. Wang, Z. Chen, Z. Liu, S. Ye, L. Gu, H. Tian, Y. Duan, W. Su, J. Shao, et al. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. arXiv preprint arXiv:2504.10479, 2025.

DeepSeekMath-V2：迈向可自我验证的数学推理

DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning Zhihong Shao, Yuxiang Luo*, Chengda Lu*†, Z.Z. Ren* Jiewen Hu, Tian Ye, Zhibin Gou, Shirong Ma, Xiaokang Zhang*

DeepSeek-AI research@deepseek.com

arXiv:2511.22570 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

大语言模型在数学推理方面已取得显著进展，数学推理既是 AI 的重要试验场，若进一步发展还可能影响科学研究。通过强化学习奖励正确的最终答案来扩展推理，大语言模型在一年之内便从表现不佳提升到在 AIME、HMMT 等定量推理竞赛上趋于饱和。然而，这一方法面临根本性局限。一味追求更高的最终答案准确率并不能解决一个关键问题：答案正确并不保证推理正确。此外，像定理证明这样的许多数学任务需要严谨的逐步推导，而不是数值答案，这使得最终答案奖励无法适用。为突破深度推理的极限，我们认为有必要验证数学推理的全面性与严谨性。自我验证对于扩展测试时计算尤为重要，尤其是面对没有已知解答的开放问题。朝着可自我验证的数学推理这一目标，我们研究了如何训练一个准确且忠实的、基于 LLM 的定理证明验证器。随后，我们以该验证器作为奖励模型训练一个证明生成器，并激励生成器在最终定稿之前尽可能多地识别并解决自己证明中的问题。为了在生成器变强的同时维持生成与验证之间的差距，我们提出扩展验证计算，自动标注新的难以验证的证明，从而产生训练数据以进一步改进验证器。我们由此得到的模型 DeepSeekMath-V2，……证明了强大的定理证明能力：在扩展测试时计算后，于 IMO 2025 与 CMO 2024 取得金牌水平，并在 Putnam 2024 上取得近乎满分（118/120）。尽管仍有大量工作要做，这些结果表明可自我验证的数学推理是一条可行的研究方向，有望帮助构建能力更强的数学 AI 系统。

1 引言

面向数学推理的强化学习（RL）的常规做法是：根据大语言模型（LLM）对定量推理问题预测的最终答案是否与标准答案一致来给予奖励（Guo 等，2025）。这一方法足以让前沿 LLM 在主要评估最终答案的数学竞赛（如 AIME 与 HMMT）中达到饱和。然而，这种奖励机制存在两个根本局限。第一，它是推理正确性的不可靠代理——模型可能通过有缺陷的逻辑或侥幸的错误得出正确答案。第二，它，不适用于定理证明任务，因为这类任务可能并不要求给出数值形式的最终答案，而严格的推导才是首要目标。

因此，用此类最终答案奖励在定量推理问题上训练的 LLM，仍经常产出在数学上无效或逻辑不一致的自然语言证明。此外，这种训练方式并不能自然地培养模型验证证明有效性的能力——它们表现出很高的假阳性率，即使证明包含明显的逻辑缺陷，也常常声称其有效。

自然语言定理证明中缺乏生成-验证差距，这阻碍了进一步的提升。为解决这一问题，我们提出在 LLM 中发展证明验证能力。我们的方法基于以下关键观察：

- 即使没有参考答案，人类也能识别证明中的问题——这在攻克开放问题时是一项关键能力。
- 当在扩大验证投入后仍无法识别出问题时，证明更可能是有效的。
- 识别有效问题所需的努力程度可以作为证明质量的代理指标，从而可用于优化证明生成。

我们相信，LLM 可以被训练为在没有参考答案的情况下识别证明中的问题。这样的验证器将带来一个迭代改进循环：（1）利用验证反馈优化证明生成；（2）扩大验证计算量，以自动标注难以验证的新证明，从而生成用于改进验证器自身的数据；（3）利用增强后的验证器进一步优化证明生成。此外，可靠的证明验证器使我们能够教会证明生成器像验证器那样评估证明。这使证明生成器能够迭代地精炼其证明，直到它无法再识别或解决任何问题。本质上，我们让模型明确意识到自身的奖励函数，并使其通过审慎推理而非盲目试错来最大化这一奖励。

基于 DeepSeek-V3.2-Exp-Base（DeepSeek-AI，2025），我们开发了 DeepSeekMath-V2——一个针对自然语言定理证明进行优化、展现出可自我验证数学推理能力的大语言模型。我们的模型能够评估并迭代改进自己的证明，在 IMO 2025 与 CMO 2024 等顶级高中数学竞赛中达到金牌水平。在 Putnam 2024 本科生竞赛中，它获得 118/120 分，超过了人类参赛者取得的最高的 90 分（https://kskedlaya.org/putnam-archive/putnam2024stats.html）。

2 方法

2.1 证明验证

2.1.1 训练验证器以识别问题并给证明评分

我们为证明评估设计了高层评分细则（rubrics）I_v（见附录 A.2），目标是训练验证器依据这些细则评估证明，以模仿数学专家的评估过程。具体而言，给定问题 X 与证明 Y，验证器 π_φ(·|X, Y, I_v) 被设计为产出一份证明分析：先总结识别出的问题（若有），然后基于三个等级给出分数——1 表示完整且严谨、所有逻辑步骤都得到清晰论证的证明；0.5 表示整体逻辑合理但存在小错误或遗漏细节的证明；0 表示存在致命逻辑错误或关键缺口的根本性缺陷证明。

构建冷启动 RL 数据：我们通过以下流程构建初始训练数据：

- 我们从 Art of Problem Solving（AoPS）竞赛（https://artofproblemsolving.com/community/c13_contest_collections）中爬取问题，优先选择数学奥林匹克、队伍选拔测试以及 2010 年之后明确要求给出证明的问题，共计 17,503 道。该问题集记为 D_p。
- 我们使用 DeepSeek-V3.2-Exp-Thinking 的一个变体生成候选证明。由于该模型并未针对定理证明进行优化，且倾向于产出简洁但易错的输出，我们提示它在多轮中迭代精炼证明，以提升全面性与严谨性。
- 我们在不同问题类型（如代数与数论）中随机采样证明，并让数学专家依据上述评估细则为每份证明打分。

该过程得到了初始 RL 数据集 D_v = {(X_i, Y_i, s_i)}，其中每一项由问题 X_i、证明 Y_i 与总体证明分数 s_i ∈ {0, 0.5, 1} 组成。

RL 目标：我们在一个经过监督微调、训练数据为数学与代码相关推理数据的 DeepSeek-V3.2-Exp-SFT 版本基础上，使用强化学习训练模型产出证明分析，并采用两个奖励组成：

- **格式奖励 R_format**：一个指示函数，通过检查最终回复是否包含关键短语 “Here is my evaluation of the solution:” 以及在 “Based on my evaluation, the final overall score should be:” 之后是否在 \boxed{} 中给出分数，来约束模型同时生成问题总结与证明分数。
- **分数奖励 R_score**：基于预测分数 s'_i 与标注分数 s_i 之间的接近程度进行奖励：

R_score(s'_i, s_i) = 1 − |s'_i − s_i|　　（1）

训练验证器的 RL 目标为：

$$\max_{\pi_\varphi} \mathbb{E}_{(X_i,Y_i,s_i)\sim D_v,\ (V'_i,s'_i)\sim\pi_\varphi(\cdot|X_i,Y_i)} \left[R_{\text{format}}(V'_i) \cdot R_{\text{score}}(s'_i, s_i) \right] \quad (2)$$

其中 V'_i 表示验证器的最终回复，s'_i 是从中提取的证明分数。

2.1.2 引入元验证以审阅证明分析

第 2.1.1 节所述方法通过 RL 训练证明验证，使预测的证明分数与专家标注对齐，但没有对识别出的问题本身提供直接监督。这造成了一个关键漏洞：在训练中评估有缺陷的证明（$s_i < 1$）时，验证器可以通过预测正确的分数但捏造并不存在的问题来获得全额奖励，从而损害其可信度。

为解决这一问题，我们引入元验证（meta-verification）：一种次级评估过程，用于评估验证器所识别的问题是否确实存在，以及这些问题是否依据评估细则 I_v 在逻辑上足以支撑所预测的证明分数。完整的元验证细则 I_{mv} 详见附录 A.3。

我们使用 RL 训练了一个专门的元验证器来执行这一评估。通过将元验证器的反馈纳入验证器训练，我们可以提升验证器问题识别的忠实度。

元验证器训练流程

- 我们按照第 2.1.1 节获得初始验证器 π_φ。
- 数学专家依据 I_{mv} 对验证器回复的质量打分，构建数据集 $D_{mv} = \{(X_i, Y_i, V_i, ms_i)\}$，其中 V_i 是对证明 Y_i 的分析，$ms_i \in \{0, 0.5, 1\}$ 是专家标注的质量分数。
- 我们训练一个元验证器 $\pi_\eta(\cdot|X, Y, V, I_{mv})$ 来分析验证器的证明分析 V。元验证器先总结在分析本身中发现的问题，然后给出一个质量分数，衡量验证器分析的正确性与论证充分程度。其 RL 目标与验证器训练的结构相同，采用格式奖励与分数奖励。

利用训练好的元验证器 π_η，我们通过将元验证反馈整合进奖励函数来增强验证器训练：

$$R_V = R_{\text{format}} \cdot R_{\text{score}} \cdot R_{\text{meta}} \quad (3)$$

其中 R_{meta} 是来自元验证器的质量分数。

我们同时在验证数据集 D_v 与元验证数据集 D_{mv} 上训练增强后的验证器，并在 D_{mv} 上采用与训练元验证器相同的奖励机制。由此得到的模型可以同时执行证明验证与元验证任务。

在 D_v 的一个验证划分上，由元验证器评估的验证器证明分析平均质量分数从 0.85 提升到 0.96，同时保持证明分数预测的准确率不变。

2.2 证明生成

2.2.1 训练面向定理证明的生成器

以验证器 π_φ 作为生成式奖励模型，我们通过如下 RL 目标训练证明生成器 π_θ(·|X)：

max_{π_θ} E_{X_i~D_p, Y_i~π_θ(·|X_i)} [R_Y]　　（4）

其中 R_Y 是由 π_φ(·|X_i, Y_i, I_v) 给出的证明分数。

2.2.2 通过自我验证增强推理

当证明生成器无法一次性产出完全正确的证明时——这在 IMO、CMO 等竞赛的难题中很常见——迭代验证与精炼可以改善结果。这包括用外部验证器分析证明，并提示生成器解决识别出的问题。

然而，我们观察到一个关键局限：当要求生成器一次性同时生成并分析自己的证明时，它倾向于声称证明正确，即使外部验证器……

容易地识别出缺陷。换言之，生成器虽然能够基于外部反馈改进证明，却无法以专职验证器那样的严谨程度来评估自己的工作。

这一观察促使我们赋予证明生成器真正的验证能力。在训练中，我们提示生成器 π_θ 先生成证明 Y，再生成自我分析 Z，其格式与评分细则 I_v 均与验证器保持一致（见附录 A.1）。我们将自我分析中预测的证明得分记为 s′。

为确保自我评估的忠实性，我们使用验证器 π_φ 对这两个部分进行评估：证明 Y 获得得分 R_Y = s，自我分析 Z 获得元验证得分 R_meta(Z) = m_s。奖励函数将这两项评估结合起来：

R = R_format(Y, Z) · (α · R_Y + β · R_Z)　（5）

R_Z = R_score(s′, s) · R_meta(Z)　（6）

其中 R_format(Y, Z) 用于校验证明与自我分析是否都遵循指定的格式，R_score(s′, s) 用于奖励准确的自我评估。我们设置 α = 0.76、β = 0.24。这一奖励结构产生了以下激励：

- 对错误予以忠实的承认，其回报高于虚假地声称正确。
- 最高的奖励来自生成正确的证明并准确认识到其严谨性。
- 对证明生成器而言，获得高奖励的一个好策略是在最终定稿之前尽可能多地识别并解决问题。

2.3 证明验证与生成之间的协同

证明验证器与生成器形成了一个协同循环：验证器改进生成器，而随着生成器变强，它会产生新的证明，挑战验证器当前的能力。这些具有挑战性的案例——验证器在单次尝试中可能无法识别其中的问题——会成为增强验证器本身的宝贵训练数据。

为了重新训练并改进验证器，我们需要为新生成的证明提供正确性标注数据。人工标注虽然直接，但随着问题变难、错误变得更细微，其耗时程度会不断增加。为提高标注效率，我们为每个证明生成多份验证器分析，以把潜在问题呈现给人工审核。从这一 AI 辅助的标注过程中，我们认识到两个事实，它们使进一步推进自动化水平成为可能：

1. 增加验证器的采样数量，会提高在错误证明中捕捉到真实问题的概率。2. 审查验证器所识别出的问题，本质上正是元验证，而这比从零开始识别问题更容易；元验证对 LLM 而言也更省样本、更易掌握。

基于这些观察，我们开发了以下自动标注流程：

1. 对每个证明，生成 n 份独立的验证分析；2. 对于报告存在问题的分析（得分为 0 或 0.5），生成 m 份元验证评估以确认所发现的问题。若多数元评估确认了该分析的发现，则该分析被视为有效；3. 对每个证明，我们检查给出最低得分的那些分析。若至少有 k 份此类分析被判定为有效，则将该证明标注为该最低得分；若在所有验证尝试中都没有发现任何合理的问题，则该证明被标注为 1；否则，该证明被丢弃，或转交给人类专家标注。

在我们的最后两次训练迭代中，这条完全自动化的流水线完全取代了人工标注。质量检查确认，自动标注与专家判断高度一致。

3 实验

【图表】图 1 按类别与模型划分的 CNML 级问题的平均证明得分（由我们的验证器评估）（原文第 6 页）

3.1 训练设置

我们采用组相对策略优化（Group Relative Policy Optimization，GRPO）（Shao 等，2024）进行强化学习，按第 2 节所述迭代式地优化证明验证与生成能力。在每一轮迭代中，我们首先优化证明验证；随后，证明生成器从验证器检查点初始化，并针对证明生成进行优化。从第二轮迭代开始，证明

验证器使用一个检查点初始化，该检查点通过拒绝采样微调（rejection fine-tuning）整合了上一轮迭代中的验证与生成能力。

3.2 评测基准

我们在以下定理证明基准上评估最终的证明生成器：

- **内部 CNML 级问题（In-House CNML-Level Problems）**：91 道定理证明题，涵盖代数（13）、几何（24）、数论（19）、组合（24）与不等式（11），难度与中国全国高中数学联赛（CNML）的题目相当

竞赛题（Competition Problems）

- **IMO 2025（6 题）**：国际数学奥林匹克，面向大学前学生的全球顶级数学竞赛
- **CMO 2024（6 题）**：中国数学奥林匹克，中国的全国锦标赛
- **Putnam 2024（12 题）**：William Lowell Putnam 竞赛，北美面向本科生的最负盛名的数学竞赛
- **ISL 2024（31 题）**：IMO 短名单（IMO Shortlist），由参赛国提出、并由选题委员会审议是否可能纳入 IMO 2024 的题目集合
- **IMO-ProofBench（60 题）**：由开发 DeepThink IMO-Gold 的 DeepMind 团队构建（Luong 和 Lockhart，2025），该基准（Luong 等，2025）分为基础集（30 题，难度从 IMO 前到 IMO 中等）与进阶集（30 道模拟完整 IMO 考试的挑战性题目，最高达 IMO 困难级）

3.3 评测结果

3.3.1 单次生成

我们首先评估模型在不进行迭代精炼的情况下生成正确证明的能力。在内部问题上，我们为每个被评估模型、每道题生成 8 个证明样本。证明正确性通过由我们最终验证器给出的 8 份验证分析进行多数投票来衡量。如图 1 所示，在 CNML 级问题的所有类别——代数、几何、数论、组合与不等式——中，DeepSeekMath-V2 都持续优于 GPT-5-Thinking-High（OpenAI，2025）与 Gemini 2.5-Pro（DeepMind，2025），展现出跨领域更强的定理证明能力。

3.3.2 结合自我验证的序贯精炼

【图表】图 2 证明质量随最大序贯迭代次数（从 1 次，即不精炼，到 8 次，即初始生成加上最多 7 次基于自我验证的精炼）的变化而提升（原文第 7 页）

对于 IMO、CMO 等竞赛中的难题，模型往往无法在 128K token 的限制内一次性生成全面且严谨的证明。当这种情况发生时，我们的证明生成器会通过自我验证认识到其证明无效，但缺乏足够的上下文长度在一次尝试中解决所有已识别的问题。

为探索扩展上下文与自我验证如何提升证明质量，我们评估了结合自我验证的序贯精炼（sequential refinement with self-verification）。该方法首先生成一个带自我分析的证明，然后迭代地用先前的输出重新提示生成器（精炼提示见附录 A.4），使其能够处理已识别的问题。该过程持续进行，直到生成器给自己打出满分，或达到最大序贯尝试次数。

图 2 展示了在 IMO 短名单 2024 问题上通过序贯精炼带来的证明质量提升。对每道题，我们启动了 32 条独立的精炼线程。证明正确性通过由我们最终验证器给出的 32 份验证分析进行多数投票来衡量。我们在图 2 中报告两项指标：（1）Pass@1——最终，每条线程最终证明的平均分，以及（2）Best@32——每个问题最佳证明的分数，该分数由所有线程中的自评分选出。自选出的最佳证明取得了显著高于线程平均水平的验证分数，表明我们的生成器能够准确评估证明质量。此外，随着最大顺序尝试次数增加，Pass@1 显著提升，说明自我验证有效引导了迭代式改进。这些结果证实，我们的生成器能够可靠地区分高质量证明与有缺陷的证明，并利用这种自我认知系统性地提升其数学推理能力。

【图表】图 3 IMO-ProofBench 的 Basic 与 Advanced 子集上的专家评测结果。除 DeepSeekMath-V2 由我们的专家依据评分指南进行评估之外，其余结果均来自 Luong et al. (2025)（原文第 8 页）

3.3.3 高算力搜索

为求解最具挑战性的问题，我们同时扩展了验证与生成两方面的算力——用大规模验证来识别细微问题，用并行生成来探索多样的证明策略。

我们为每个问题维护一个候选证明池，初始化时包含 64 个证明样本，并为每个样本生成 64 份验证分析。在每轮改进迭代中，我们依据平均验证分数选出得分最高的 64 个证明，并为每个证明随机配对 8 份分析，优先选择那些指出问题的分析（分数为 0 或 0.5）。每一对“证明-分析”用于生成一个改进后的证明，进而更新候选池。该过程最多持续 16 轮迭代，或直到某个证明成功通过全部 64 次验证尝试，这表示对其正确性具有高置信度。所有实验均使用同一个模型——我们的最终证明生成器，它同时执行证明生成与验证。

表 1：灰色表示完全解决的问题，下划线表示获得部分分数的问题（原文第 8 页）

竞赛	题目	得分

IMO 2025	P1, P2, P3, P4, P5	83.3%
CMO 2024	P1, P2, P4, P5, P6	73.8%
Putnam 2024	A1 ~ B4, B5, B6	98.3%

为验证我们的结果，数学专家对得分最高的证明进行了评估。如表 1 所示，我们的方法解出了 IMO 2025 的 6 道题中的 5 道，并解出 CMO 2024 的 4 道题、另有一道获得部分分数，在这两项顶尖高中数学竞赛中均达到金牌水平。在本科生最高级别的数学竞赛 Putnam 2024 中，我们的模型完整解出 12 道题中的 11 道，剩余一题仅有小错误，得分 118/120，超过了人类最高分 90。图 3 展示了在 IMO-ProofBench 上的结果。我们的方法在 Basic 集上优于 DeepMind 的 DeepThink（IMO Gold），在 Advanced 集上与之相当，同时大幅优于所有其他基线。我们注意到，最难的 IMO 级问题对我们的模型而言仍具挑战性。值得注意的是，对于未能完全解决的问题，我们的生成器通常能识别出其证明中的真实问题；而完全解决的问题则能通过全部 64 次验证尝试。这表明我们能够成功训练基于 LLM 的验证器，来评估此前被认为难以自动验证的证明。在验证器引导下扩展测试时计算，我们的模型能够解决人类参赛者需要数小时努力才能解决的问题。

4 相关工作

推理模型（OpenAI, 2024; Guo et al., 2025）在一年之内便已在 AIME 与 HMMT 等定量推理基准上达到饱和。这一快速进展部分归因于其明确界定的评测标准：如果我们只关心最终答案，那么定量推理是易于验证的。然而，这种“最终答案”指标并不适用于定理证明——定理证明往往没有数值答案，而要求严谨的逐步推导。非形式化数学证明长期以来被认为难以自动验证，缺乏评估证明正确性的可靠方法。近期的进展表明这一障碍或许可以跨越。Gemini-2.5 Pro 等模型已展现出一定程度的自我验证能力，能够改进自身解答以提升质量（Huang and Yang, 2025）。更重要的是，DeepMind 内部的 DeepThink 变体（Luong and Lockhart, 2025）凭借纯自然语言推理在 IMO 2025 上取得金牌水平——这构成了一个存在性证明：基于 LLM 对复杂证明的验证是可以实现的。近期研究（Dekoninck et al., 2025; Luong et al., 2025）已开始探索推理模型能否在有参考解答与无参考解答两种情况下评估证明，并显示出令人鼓舞的结果。在本工作中，我们开源了 DeepSeekMath-V2 及其训练方法，作为迈向可自我验证数学推理的具体步骤，展示了模型如何学习验证并改进自身的证明。

Lean（de Moura et al., 2015）与 Isabelle（Paulson, 1994）等证明助手为验证证明提供了可靠途径——证明必须以形式语言书写，但一旦编译通过，正确性即得到保证。AlphaProof（AlphaProof and teams, 2024; Trinh et al., 2024; Chervonyi et al., 2025）是一个专门用于形式化证明搜索的系统，在 IMO 2024 上取得银牌水平，但需要大量计算。尽管利用非形式化推理来引导形式化证明生成已被广泛探索（Jiang et al., 2023），但近期的推理模型大幅提升了非形式化推理的质量，使这种引导远比以往有效。DeepSeek-Prover-V2（Ren et al., 2025）与 Seed-Prover（Chen et al., 2025）等系统如今能在相同的计算预算内产出多得多的有效形式化证明，其中 Seed-Prover 在 IMO 2025 上解出了 6 道题中的 5 道。值得注意的是，这些结果并未针对定理证明任务专门优化非形式化推理组件。我们相信，推进自然语言定理证明将显著有益于形式化推理。我们希望为构建真正可靠的数学推理系统作出贡献——这样的系统同时利用非形式化洞见与形式化保证，以推进数学研究。

5 结论

我们提出了 DeepSeekMath-V2，一个既能生成又能验证数学证明的模型。通过训练模型识别自身推理中的问题，并激励其在最终确定输出之前解决这些问题，我们超越了基于最终答案的奖励的局限，迈向可自我验证的数学推理。我们的迭代训练过程——在提升验证能力与利用这些能力增强生成之间交替——形成了一个可持续的循环，其中每个组件都推动另一个组件前进。我们的关键技术创新包括：（1）训练一个准确且忠实的、基于 LLM 的数学证明验证器；（2）使用元验证大幅减少幻觉出的问题，并确保验证质量；（3）通过自我验证激励证明生成器最大化证明质量；（4）扩展验证计算，以自动标注日益难以验证的证明，从而在无需人工标注的情况下改进验证器。DeepSeekMath-V2 在竞赛数学上展现出强劲性能。在扩展测试时计算后，它在 IMO 2025、CMO 2024 等高中竞赛中取得金牌成绩，并在本科生 Putnam 2024 竞赛中取得近乎满分的成绩。这项工作表明，LLM 能够针对复杂推理任务发展出有意义的自我评估能力。尽管仍有重大挑战，我们希望这一研究方向有助于实现可自我验证的 AI 系统这一目标——这样的系统能够解决研究级别的数学问题。

附录

附录 A 提示词模板

A.1 证明生成提示词

A.2 证明验证提示词

A.3 元验证提示词

A.4 证明改进提示词

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] key technical contributions include: (1) training an accurate and faithful LLM-based verifier for mathematical proofs, (2) using meta-verification to largely reduce hallucinated issues and ensure verification quality, (3) incentivizing the proof generator to maximize proof quality through self-verification, and (4) scaling verification compute to automatically label increasingly hard-to-verify proofs to improve the verifier without human annotation. DeepSeekMath-V2 demonstrates strong performance on competition mathematics. With scaled test-time compute, it achieved gold-medal scores in high-school competitions including IMO 2025 and CMO 2024, and a near-perfect score on the undergraduate Putnam 2024 competition. This work establishes that LLMs can develop meaningful self-evaluation abilities for complex reasoning tasks. While significant challenges remain, we hope this research direction contributes to the goal of creating self-verifiable AI systems that can solve research-level mathematics.

[2] AlphaProof and A. teams. Ai achieves silver-medal standard solving international mathematical olympiad problems, 2024. URL https://deepmind.google/blog/ai-solves-imo-pro blems-at-silver-medal-level .

[3] L. Chen, J. Gu, L. Huang, W. Huang, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, K. Ma, C. Ren, J. Shen, W. Shi, T. Sun, H. Sun, J. Wang, S. Wang, Z. Wang, C. Wei, S. Wei, Y. Wu, Y. Wu, Y. Xia, H. Xin, F. Yang, H. Ying, H. Yuan, Z. Yuan, T. Zhan, C. Zhang, Y. Zhang, G. Zhang, T. Zhao, J. Zhao, Y. Zhou, and T. H. Zhu. Seed-prover: Deep and broad reasoning for automated theorem proving. CoRR, abs/2507.23726, 2025. doi: 10.48550/ARXIV.2507.23726. URL https://doi.org/10.48550/arXiv.2507.23726 .

[4] Y. Chervonyi, T. H. Trinh, M. Olsák, X. Yang, H. Nguyen, M. Menegali, J. Jung, V . Verma, Q. V . Le, and T. Luong. Gold-medalist performance in solving olympiad geometry with alphageometry2. CoRR, abs/2502.03544, 2025. doi: 10.48550/ARXIV.2502.03544. URL https://doi.org/10.48550/arXiv.2502.03544 .

[5] L. M. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer. The lean theorem prover (system description). In A. P . Felty and A. Middeldorp, editors, Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings, volume 9195 of Lecture Notes in Computer Science, pages 378–388. Springer, 2015. doi: 10.1007/978-3-319-21401-6_26. URL https://doi.org/10.1007/978-3-319-21401-6_26 .

[6] G. DeepMind. Gemini 2.5 pro, 2025. URL https://deepmind.google/models/gemini/pr o .

[7] DeepSeek-AI. Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention, 2025.

[8] J. Dekoninck, I. Petrov, K. Minchev, M. Balunovic, M. T. Vechev, M. Marinov, M. Drencheva, L. Konova, M. Shumanov, K. Tsvetkov, N. Drenchev, L. Todorov, K. Nikolova, N. Georgiev, V . Kalinkova, and M. Ismoldayev. The open proof corpus: A large-scale study of llm-generated mathematical proofs. CoRR, abs/2506.21621, 2025. doi: 10.48550/ARXIV.2506.21621. URL https://doi.org/10.48550/arXiv.2506.21621 .

[9] D. Guo, D. Yang, H. Zhang, J. Song, P . Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Ding, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Chen, J. Yuan, J. Tu, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. You, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Zhou, M. Li, M. Wang, M. Li, N. Tian, P . Huang, P . Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. Nat., 645(8081):633–638, 2025. doi: 10.1038/S41586-025-09422-Z. URL https://doi.org/10.1038/s41586-025-09422-z .

[10] Y. Huang and L. F. Yang. Gemini 2.5 pro capable of winning gold at IMO 2025. CoRR, abs/2507.15855, 2025. doi: 10.48550/ARXIV.2507.15855. URL https://doi.org/10 .48550/arXiv.2507.15855 .

[11] A. Q. Jiang, S. Welleck, J. P . Zhou, T. Lacroix, J. Liu, W. Li, M. Jamnik, G. Lample, and Y. Wu. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?id=SMa9EAovKMC .

[12] T. Luong and E. Lockhart. Advanced version of gemini with deep think officially achieves gold-medal standard at the international mathematical olympiad, 2025. URL https://goo. gle/imo-gold .

[13] T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V . Le, and J. Jung. Towards robust mathematical reasoning. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025. URL https://aclanthology.org/2025.emnlp-main.1794/ .

[14] OpenAI. Learning to reason with llms, 2024. URL https://openai.com/index/learnin g-to-reason-with-llms/ .

[15] OpenAI. Introducing gpt-5, 2025. URL https://openai.com/index/introducing-gpt-5 .

[16] L. C. Paulson. Isabelle - A Generic Theorem Prover (with a contribution by T. Nipkow), volume 828 of Lecture Notes in Computer Science. Springer, 1994. ISBN 3-540-58244-4. doi: 10.1007/ BFB0030541. URL https://doi.org/10.1007/BFb0030541 .

[17] Z. Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, Z. F. Wu, Z. Gou, S. Ma, H. Tang, Y. Liu, W. Gao, D. Guo, and C. Ruan. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. CoRR, abs/2504.21801, 2025. doi: 10.48550/ARXIV.2504.21801. URL https://doi.org/10.485 50/arXiv.2504.21801 . Z. Shao, P . Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. CoRR, abs/2402.03300, 2024. doi: 10.48550/ARXIV.2402.03300. URL https://doi.org/10 .48550/arXiv.2402.03300 .

[18] T. H. Trinh, Y. Wu, Q. V . Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. Nat., 625(7995):476–482, 2024. doi: 10.1038/S41586-023-06747-5. URL https://doi.org/10.1038/s41586-023-06747-5 .

DeepSeek-V3.2：推动开源大语言模型的前沿

DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models

DeepSeek-AI research@deepseek.com

arXiv:2512.02556 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-V3.2，该模型将高计算效率与卓越的推理和智能体表现统一起来。DeepSeek-V3.2 的关键技术突破如下：（1）DeepSeek 稀疏注意力（DeepSeek Sparse Attention，DSA）：我们引入 DSA，这是一种高效的注意力机制，在长上下文场景下大幅降低计算复杂度，同时保持模型性能。（2）可扩展的强化学习框架：通过实施稳健的强化学习协议并扩展后训练计算，DeepSeek-V3.2 的表现与 GPT-5 相当。值得注意的是，我们的高计算量变体 DeepSeek-V3.2-Speciale 超越了 GPT-5，其推理水准与 Gemini-3.0-Pro 相当，在 2025 年国际数学奥林匹克（IMO）和国际信息学奥林匹克（IOI）中均取得金牌水准的成绩。（3）大规模智能体任务合成流水线：为将推理融入工具使用场景，我们开发了一条新颖的合成流水线，可在规模上系统地生成训练数据。该方法支撑了可扩展的智能体后训练，在复杂的交互式环境中显著提升泛化性与指令遵循的稳健性。

1 引言

推理模型（DeepSeek-AI，2025；OpenAI，2024a）的发布标志着大语言模型（LLM）演进的一个关键节点，推动了可验证领域整体性能的大幅跃升。自这一里程碑以来，LLM 的能力快速进步。然而，过去数月出现了一个明显的分化：开源社区（MiniMax，2025；MoonShot，2025；Qwen，2025；ZhiPu-AI，2025）仍在持续取得进展，而闭源专有模型（Anthropic，2025b；DeepMind，2025a；OpenAI，2025）的性能提升轨迹则以明显更陡的速率加速。因此，闭源模型与开源模型之间的性能差距非但没有收敛，反而似乎在扩大，专有系统在复杂任务上展现出越来越强的能力。

通过分析，我们识别出限制开源模型在复杂任务上能力的三个关键不足。首先，在架构上，对原始注意力（vanilla attention）（Vaswani 等，2017）机制的主导性依赖严重制约了长序列上的效率。这种低效为可扩展部署与有效的后训练都构成了实质性障碍。其次，在资源分配上，开源模型在后训练阶段的计算投入不足，限制了它们在困难任务上的性能。最后，在 AI 智能体场景中，与专有模型相比，开源模型在泛化与指令遵循能力上明显滞后（EvalSys，2025；Li 等，2025；Luo 等，2025），这阻碍了它们在实际部署中的效果。

为解决这些关键局限，我们首先提出 DSA——一种旨在大幅降低计算复杂度的高效注意力机制。该架构有效解决了效率瓶颈，即使在长上下文场景中也能保持模型性能。其次，我们开发了一套稳定且可扩展的 RL 协议，使得后训练阶段的计算能够大幅扩展。值得注意的是，该框架分配的后训练计算预算超过预训练成本的 10%，从而释放出更高级的能力。第三，我们提出一种新流程，用于在工具使用场景中培养可泛化的推理能力。首先，我们实施一个冷启动阶段，利用 DeepSeek-V3（DeepSeek-AI，2024）的方法论，将推理与工具使用统一到单一轨迹中。随后，我们推进大规模智能体任务合成，生成超过 1,800 个不同环境与 85,000 条复杂提示。这些大量合成数据驱动 RL 过程，显著增强了模型在智能体场景中的泛化与指令遵循能力。

在多个推理基准上，DeepSeek-V3.2 取得了与 Kimi-k2-thinking 和 GPT-5 相近的性能。此外，DeepSeek-V3.2 显著推进了开源模型的智能体能力，在 EvalSys（2025）、Li 等（2025）、Luo 等（2025）所提出的长尾智能体任务上展现出卓越的表现。DeepSeek-V3.2 成为智能体场景中极具成本效益的替代方案，显著缩小了开源模型与前沿闭源模型之间的性能差距，同时成本大幅降低。

值得注意的是，为在推理领域推动开源模型的边界，我们放宽了长度约束以开发 DeepSeek-V3.2-Speciale。最终，DeepSeek-V3.2-Speciale 的性能与领先的闭源系统 Gemini-3.0-Pro（DeepMind，2025b）持平，并在 IOI 2025、ICPC World Final 2025、IMO 2025 与 CMO 2025 中达到金牌水平。

2 DeepSeek-V3.2 架构

2.1 DeepSeek 稀疏注意力

DeepSeek-V3.2 采用与 DeepSeek-V3.2-Exp 完全相同的架构。与 DeepSeek-V3.1 的最后一个版本 DeepSeek-V3.1-Terminus 相比，DeepSeek-V3.2 唯一的架构改动是通过继续训练引入 DeepSeek 稀疏注意力（DSA）。

DSA 原型 DSA 的原型主要由两个组件构成：闪电索引器（lightning indexer）与细粒度 token 选择机制。

闪电索引器计算查询 token h_t ∈ R^d 与前序 token h_s ∈ R^d 之间的索引分数 I_{t,s}，以确定查询 token 将选择哪些 token：

$$I_{t,s} = \Sigma_{j=1}^{H_I} w^I_{t,j} \cdot \mathrm{ReLU}(q^I_{t,j} \cdot k^I_s) \quad (1)$$

其中 H_I 表示索引器头数；q^I_{t,j} ∈ R^{d_I} 与 w^I_{t,j} ∈ R 由查询 token h_t 得到；k^I_s ∈ R^{d_I} 由前序 token h_s 得到。出于吞吐量考虑，我们选择 ReLU 作为激活函数。鉴于闪电索引器的头数较少且可以用 FP8 实现，其计算效率非常出色。

给定每个查询 token h_t 的索引分数 {I_{t,s}}，我们的细粒度 token 选择机制只取回与 top-k 个索引分数对应的键值条目 {c_s}。随后，通过在查询 token h_t 与稀疏选中的键值条目 {c_s} 之间施加注意力机制，计算出注意力输出 u_t：

$$u_t = \mathrm{Attn}(h_t, \{c_s \mid I_{t,s} \in \text{Top-k}(I_{t,:})\}) \quad (2)$$

在 MLA 下实例化 DSA 考虑到需要从 DeepSeek-V3.1-Terminus 继续训练，我们基于 MLA（DeepSeek-AI，2024）为 DeepSeek-V3.2 实例化 DSA。在内核层面，出于计算效率考虑，每个键值条目必须在多个查询之间共享（Yuan 等，2025）。因此，我们基于 MLA[1] 的 MQA（Shazeer，2019）模式实现 DSA：每个潜在向量（即 MLA 的键值条目）将在查询 token 的所有查询头之间共享。基于 MLA 的 DSA 架构如图 2 所示。我们还提供了 DeepSeek-V3.2 的开源实现[2]，以明确相关细节。

[1] 我们在附录 A 中说明 MLA 的 MQA 模式与 MHA 模式的差异。 [2] https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Exp/tree/main/inference

2.1.1 继续预训练

我们以 DeepSeek-V3.1-Terminus 的基座检查点为起点（其上下文长度已扩展至 128K），先进行继续预训练，再进行后训练，从而得到 DeepSeek-V3.2。

DeepSeek-V3.2 的继续预训练包含两个训练阶段。两个阶段的训练数据分布与 DeepSeek-V3.1-Terminus 所用的 128K 长上下文扩展数据完全一致。

【图表】图 2 DeepSeek-V3.2 的注意力架构，其中 DSA 在 MLA 下实例化。绿色部分展示了 DSA 如何依据索引器选择 top-k 键值条目。（原文第 4 页）

稠密预热阶段 我们首先使用一个简短的预热阶段来初始化闪电索引器。在该阶段，我们保持稠密注意力，并冻结除闪电索引器之外的所有模型参数。为使索引器输出与主注意力分布对齐，对于第 t 个查询 token，我们首先通过对所有注意力头求和来聚合主注意力分数；随后沿序列维度对该和进行 L1 归一化，得到目标分布 p_{t,:} ∈ R^t。基于 p_{t,:}，我们设置一个 KL 散度损失作为索引器的训练目标：

$$L_I = \Sigma_t D_{KL}(p_{t,:} \| Softmax(I_{t,:})) \quad (3)$$

在预热阶段，我们使用 10^{-3} 的学习率。我们仅训练索引器 1000 步，每步包含 16 条 128K token 的序列，共计 2.1B token。

稀疏训练阶段 在索引器预热之后，我们引入细粒度 token 选择机制，并优化所有模型参数，使模型适应 DSA 的稀疏模式。在该阶段，我们同样持续将索引器输出对齐到主注意力分布，但只考虑被选中的 token 集合 $S_t = \{s \mid I_{t,s} \in Top\text{-}k(I_{t,:})\}$：

$$L_I = \Sigma_t D_{KL}(p_{t,S_t} \| Softmax(I_{t,S_t})) \quad (4)$$

值得注意的是，我们将索引器输入从计算图中分离（detach），以便单独优化。索引器的训练信号仅来自 L_I，而主模型的优化仅依据语言建模损失。在该稀疏训练阶段，我们使用 7.3×10^{-6} 的学习率，并为每个查询 token 选择 2048 个键值 token。我们同时训练主模型与索引器 15000 步，每步包含 480 条 128K token 的序列，共计 943.7B token。

2.2 对等性评测

标准基准 2025 年 9 月，我们在一套关注多样化能力的基准上评测 DeepSeek-V3.2-Exp，并将其与 DeepSeek-V3.1-Terminus 比较，结果显示二者性能相近。尽管 DeepSeek-V3.2-Exp 显著提升了长序列上的计算效率，但在短上下文与长上下文任务上，我们都没有观察到相对 DeepSeek-V3.1-Terminus 的实质性性能退化。

人类偏好 鉴于直接的人类偏好评估本质上容易受偏差影响，我们采用 ChatbotArena 作为间接评测框架，以近似新开发的基座模型的用户偏好。DeepSeek-V3.1-Terminus 与 DeepSeek-V3.2-Exp 采用完全相同的后训练策略，二者在 2025 年 11 月 10 日评测中获得的 Elo 分数非常接近。这些结果表明，尽管新基座模型引入了稀疏注意力机制，其性能仍与上一代持平。

长上下文评测 DeepSeek-V3.2-Exp 发布后，多项独立的长上下文评测使用了此前未见过的测试集。一个代表性基准是 AA-LCR[3]：在推理模式下，DeepSeek-V3.2-Exp 的得分比 DeepSeek-V3.1-Terminus 高 4 分。在 Fiction.liveBench 评测[4]中，DeepSeek-V3.2-Exp 在多项指标上持续优于

DeepSeek-V3.1-Terminus。这些证据表明，DeepSeek-V3.2-Exp 的基座检查点在长上下文任务上没有出现回退。

[3] https://artificialanalysis.ai/evaluations/artificial-analysis-long-context-reasoning [4] https://fiction.live/stories/Fiction-liveBench-April-6-2025/oQdzQvKHw8JyXbN87

2.3 推理成本

DSA 将主模型核心注意力的复杂度从 $O(L^2)$ 降至 $O(Lk)$，其中 k（$\ll L$）为被选中的 token 数量。尽管闪电索引器仍具有 $O(L^2)$ 的复杂度，但与 DeepSeek-V3.1-Terminus 中的 MLA 相比，其所需计算量要少得多。结合我们优化的实现，DSA 在长上下文场景中实现了显著的端到端加速。图 3 展示了 DeepSeek-V3.1-Terminus 与 DeepSeek-V3.2 的 token 成本如何随序列中的 token 位置变化。这些成本通过对部署在 H800 GPU 上的实际服务进行基准测试估算得到，GPU 租用价格为每 GPU 小时 2 美元。注意，对于短序列预填充，我们专门实现了一种掩码 MHA 模式来模拟 DSA，从而在短上下文条件下获得更高效率。

3 后训练

在继续预训练之后，我们进行后训练以得到最终的 DeepSeek-V3.2。DeepSeek-V3.2 的后训练同样采用与稀疏继续预训练阶段相同的稀疏注意力。对于 DeepSeek-V3.2，我们保持与 DeepSeek-V3.2-Exp 相同的后训练流程，其中包括专家蒸馏与混合 RL 训练。

专家蒸馏 对于每项任务，我们首先开发一个专门针对该领域的专家模型，所有专家模型都从同一个预训练的 DeepSeek-V3.2 基座检查点微调而来。除写作任务与通用问答外，我们的框架涵盖六个专业领域：数学、编程、通用逻辑推理、通用智能体任务、智能体编码与智能体搜索，且所有领域都同时支持思考（thinking）与非思考（non-thinking）模式。每个专家模型都使用大规模强化学习（RL）算力训练。此外，我们使用不同的模型来生成用于长思维链推理（思考模式）与直接回复生成（非思考模式）的训练数据。专家模型准备就绪后，它们被用于为最终检查点生成领域特定数据。实验结果表明，在蒸馏数据上训练的模型，其性能仅略低于领域专家模型，而这一性能差距可通过后续 RL 训练有效消除。

【图表】图 3 DeepSeek-V3.1-Terminus 与 DeepSeek-V3.2 在 H800 集群上的推理成本。（原文第 6 页）

混合 RL 训练 对于 DeepSeek-V3.2，我们仍采用组相对策略优化（GRPO）（DeepSeek-AI，2025；Shao 等，2024）作为 RL 训练算法。与 DeepSeek-V3.2-Exp 一样，我们将推理、智能体与人类对齐训练合并为一个 RL 阶段。该方法有效平衡了不同领域的性能，同时避免了多阶段训练范式常见的灾难性遗忘问题。对于推理与智能体任务，我们采用基于规则的结果奖励、长度惩罚与语言一致性奖励。对于通用任务，我们采用生成式奖励模型，其中每条提示都有各自的评分细则（rubrics）用于评估。

DeepSeek-V3.2 与 DeepSeek-V3.2-Speciale DeepSeek-V3.2 整合了从专家模型蒸馏得到的推理、智能体与人类对齐数据，经过数千步的继续 RL 训练以达到最终检查点。为探究扩展思考的潜力，我们还开发了一个实验性变体 DeepSeek-V3.2-Speciale。该模型仅在推理数据上训练，并在 RL 期间降低了长度惩罚。此外，我们引入了 DeepSeekMath-V2（Shao 等，2025）的数据集与奖励方法，以增强数学证明能力。

我们希望强调两方面的工作：如何在第 3.1 节中建立稳定的配方以扩展 RL 算力，以及如何在第 3.2 节中将思考整合进智能体任务。

3.1 扩展 GRPO

我们首先回顾 GRPO 的目标。对于每个问题 q，GRPO 通过最大化如下目标，在从旧策略 π_old 采样的一组回答 {o_1, …, o_G} 上优化策略模型 π_θ：

J_GRPO(θ) = E_{q∼P(Q), {o_i}_{i=1}^G∼π_old(·|q)} [(1/G) Σ_{i=1}^{G} (1/|o_i|) Σ_{t=1}^{|o_i|} (min(r_{i,t}(θ)·Â_{i,t}, clip(r_{i,t}(θ), 1−ε, 1+ε)·Â_{i,t}) − β·D_KL(π_θ(o_{i,t})‖π_ref(o_{i,t})))] （5）

其中，重要性采样比为

r_{i,t}(θ) = π_θ(o_{i,t}|q, o_{i,<t}) / π_old(o_{i,t}|q, o_{i,<t}) （6）

ε 与 β 分别是控制裁剪范围与 KL 惩罚强度的超参数。Â_{i,t} 是 o_{i,t} 的优势值，通过在每个组内对结果奖励进行归一化来估计。具体而言，使用一组奖励模型为组内每个输出 o_i 给出结果奖励 R_i，相应得到 G 个奖励 R = {R_1, …, R_G}。o_{i,t} 的优势值通过从输出 o_i 的奖励中减去该组的平均奖励计算得到，即 Â_{i,t} = R_i − mean(R)。

下文我们概述若干直接构建在 GRPO 算法之上、用于稳定 RL 扩展的额外策略。

无偏 KL 估计（Unbiased KL Estimate）：给定 o_{i,t} 采样自旧策略 π_old(·|q, o_{i,<t})，我们利用当前策略 π_θ 与旧策略 π_old 之间的重要性采样比，对 K3 估计器（Schulman，2020）进行校正，从而得到无偏的 KL 估计。

$$D_{KL}(\pi_\theta(o_{i,t})\|\pi_{ref}(o_{i,t})) = \frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{old}(o_{i,t}|q,o_{i,<t})} \cdot \left(\frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - \log\left(\frac{\pi_{ref}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} \right) - 1 \right) \quad (7)$$

这一调整的直接结果是，该 KL 估计器的梯度变为无偏，从而消除了系统性估计误差，有利于稳定收敛。这与原始 K3 估计器形成鲜明对比，尤其是当被采样的 token 在当前策略下的概率显著低于参考策略时（即 $\pi_\theta \ll \pi_{ref}$）。在这种情况下，K3 估计器的梯度会给这些 token 赋予不成比例的巨大且无界的权重以最大化其似然，导致梯度更新噪声很大，并在后续迭代中不断累积，损害样本质量并引发不稳定的训练动态。在实践中，我们发现不同领域受益于不同强度的 KL 正则化。对于数学等某些领域，施加相对较弱的 KL 惩罚甚至完全省略它可以带来更好的性能。

离策略序列掩码（Off-Policy Sequence Masking）：为提高 RL 系统的效率，我们通常会生成大批 rollout 数据，然后将其拆分为多个小批次进行若干步梯度更新。这种做法本质上引入了离策略行为。此外，用于高效数据生成的推理框架通常经过高度优化，其实现细节可能与训练框架不同。这种训练-推理不一致进一步加剧了离策略程度。为稳定训练并提高对离策略更新的容忍度，我们掩码掉引入显著策略分歧的负向序列，该分歧以数据采样策略 π_old 与当前策略 π_θ 之间的 KL 散度衡量。更具体地说，我们在 GRPO 损失中引入一个二值掩码 M：

$$J_{GRPO}(\theta) = \mathbb{E}_{q\sim P(Q),\ \{o_i\}_{i=1}^{G}\sim\pi_{old}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min\left(r_{i,t}(\theta)\hat{A}_{i,t},\ \text{clip}(r_{i,t}(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,t} \right) M_{i,t} - \beta D_{KL}(\pi_\theta(o_{i,t})\|\pi_{ref}(o_{i,t})) \right) \right] \quad (8)$$

$$M_{i,t} = \begin{cases} 0, & \text{若 } \hat{A}_{i,t} < 0 \text{ 且 } \frac{1}{|o_i|}\sum_{t=1}^{|o_i|} \log\left(\frac{\pi_{old}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} \right) > \delta \\ 1, & \text{否则为} \end{cases} \quad (9)$$

其中 δ 是控制策略分歧阈值的超参数。注意此处的 π_old 表示由推理框架直接返回的采样概率，因此旧策略与当前策略之间的 KL 散度同时涵盖了上述两类离策略来源。同样值得注意的是，我们只掩码优势值为负的序列。直观上，模型从自身错误中学习获益最大，而高度离策略的负样本可能有害，可能误导或使优化过程不稳定。我们在实验中观察到，这一离策略序列掩码操作在某些原本会表现出不稳定的训练场景中提升了稳定性。

保留路由（Keep Routing）：混合专家（MoE）模型通过在推理时仅激活一部分专家模块来提高计算效率。然而，推理框架与训练框架之间的差异，加上策略更新，可能导致即使输入相同，推理与

训练期间的专家路由也不一致。这种不一致会引起活跃参数子空间的突变，从而破坏优化稳定性并加剧离策略问题。为缓解这一问题，我们在推理框架中保留采样时使用的专家路由路径，并在训练中强制使用相同的路由路径，确保优化的是相同的专家参数。我们发现这一保留路由操作对 MoE 模型的 RL 训练稳定性至关重要，并已在我们的 RL 训练流程中自 DeepSeek-V3-0324 起采用。

保留采样掩码（Keep Sampling Mask）：Top-p 与 top-k 采样是广泛用于提升 LLM 生成回复质量的采样策略。在 RL 训练中采用这些策略同样有益，因为它避免了采样概率极低、会被用作优化目标的 token。虽然这种截断保持了样本质量，但它引入了 π_old 与 π_θ 动作空间之间的不匹配，违反了重要性采样的原则并使训练不稳定。为解决这一问题，我们保留从 π_old 采样时的截断掩码，并在训练时将其应用于 π_θ，确保两个策略共享相同的动作子空间。实验上我们发现，将 top-p 采样与保留采样掩码策略结合，能有效在 RL 训练期间保持语言一致性。

3.2 工具使用中的思考

3.2.1 思考上下文管理

DeepSeek-R1 已表明，引入思考过程能显著增强模型解决复杂问题的能力。基于这一洞见，我们致力于将思考能力集成到工具调用场景中。

我们观察到，沿用 DeepSeek-R1 的策略——在第二轮消息到来时丢弃推理内容——会导致显著的 token 浪费。这种做法迫使模型在后续每次工具调用时重复地对整个问题进行推理。为缓解这一问题，我们设计了一种严格针对工具调用场景的上下文管理机制，如图 4 所示：

- 仅当对话中引入新的用户消息时，才丢弃历史推理内容。如果仅追加与工具相关的消息（例如工具输出），则推理内容会在整个交互过程中保留。
- 移除推理轨迹时，工具调用及其结果的历史记录仍保留在上下文中。

值得注意的是，某些智能体框架（如 Roo Code 或 Terminus）通过用户消息模拟工具交互。由于上述上下文管理规则，这些框架可能无法充分受益于我们增强的推理保留机制。因此，我们建议在这类架构下使用非思考模型以获得最佳性能。

【图表】图 4 工具调用场景中的思考保留机制（原文第 9 页）

3.2.2 冷启动

鉴于已有推理数据（非智能体）与非推理的智能体数据，将这两种能力整合起来的一种直接策略是通过精心设计的提示。

我们假设模型具备足够的能力来准确遵循显式指令，从而能够在推理过程中无缝地融入工具执行。

为演示冷启动机制的运作，我们如附录表 6–8 所示选取训练数据作为示例。需要注意的是，不同的任务提示对应不同的系统提示。表 6–8 给出了一个与竞赛编程提示相对应的示例。表 6 展示了我们推理数据的一个示例，其系统提示明确要求模型在给出最终答案之前进行推理，并使用特殊标签 <think></think> 标注推理路径。表 7 展示了非推理智能体数据的提示，其中系统提示包含工具调用的引导。表 8 展示了我们设计的系统提示，用于指示模型在其推理过程中整合多次工具调用。

通过这种方式，尽管工具使用模式下的推理可能不够稳健，模型偶尔仍能生成期望的轨迹，从而为后续强化学习阶段提供基础。

3.2.3 大规模智能体任务

多样化的 RL 任务对于增强模型鲁棒性至关重要。对于搜索、代码工程与代码解释等任务，我们采用真实世界工具，包括真实网页搜索 API、编码工具与 Jupyter Notebook。虽然这些 RL 环境是真实的，但所用的提示要么抽取自互联网来源，要么是合成生成的，而非来自真实用户交互。对于其他任务，环境与提示均为合成构建。我们使用的智能体任务见表 1。

表 1：不同智能体任务的描述，包括任务数量、环境类型（真实或合成）与提示来源（抽取或合成）。（原文第 10 页）

智能体任务	任务数量	环境	提示来源
代码智能体（code agent）	24667	真实	抽取
搜索智能体（search agent）	50275	真实	合成
通用智能体（general agent）	4417	合成	合成
代码解释器（code interpreter）	5908	真实	抽取

搜索智能体（Search Agent）：我们采用基于 DeepSeek-V3.2 的多智能体流水线来生成多样化、高质量的训练数据。我们首先从大规模网页语料中跨多个领域采样有信息量的长尾实体。随后，一个“问题构建”智能体使用搜索工具，以可配置的深度与广度参数探索每个实体，并将发现的信息整合为问答对。多个配置异构（不同检查点、系统提示等）的答案生成智能体为每个提出的问答对生成多样化的候选回复。一个具备搜索能力的验证智能体通过多轮校验所有答案，仅保留标准答案正确且所有候选回复均可验证为错误的样本。这些数据涵盖多种语言、领域与难度级别。为补充这些可验证样本并更好地反映真实使用情况，我们还用现有 helpful RL 数据集中经筛选的、搜索工具能带来可衡量收益的实例来扩充数据集。然后，我们针对多个质量维度制定详细评估细则，并使用生成式奖励模型依据这些细则为回复打分。这种混合方法使我们能够同时优化事实可靠性与实际有用性。

代码智能体（Code Agent）：我们通过从 GitHub 挖掘数百万个 issue 与 Pull Request（PR）对，为软件问题解决构建了大规模可执行环境。该数据集使用启发式规则与基于 LLM 的判断进行严格过滤以确保高质量，要求每条记录包含合理的问题描述、相关的金标准补丁（gold patch）以及用于验证的测试补丁。我们采用由 DeepSeek-V3.2 驱动的自动化环境搭建智能体为这些数据对构建可执行环境。该智能体负责包安装、依赖解析与测试执行。测试结果以标准 JUnit 格式输出，确保跨编程语言与测试框架的一致解析。仅当应用金标准补丁后产生非零数量的 false-to-positive（F2P）测试用例（表明问题被修复）且 pass-to-fail（P2F）测试用例数量为零（表明无回归）时，环境才被视为成功构建。借助这一流水线，我们成功构建了数万个可复现的问题解决环境，覆盖 Python、Java、JavaScript、TypeScript、C、C++、Go 与 PHP 等多种编程语言。

代码解释器智能体（Code Interpreter Agent）：我们使用 Jupyter Notebook 作为代码解释器来处理复杂推理任务。为此，我们构建了一组涵盖数学、逻辑与数据科学的多样化问题，每道题都要求模型利用代码执行能力来获得解答。

通用智能体（General Agent）：为扩展 RL 中的智能体环境与任务规模，我们采用一个自动环境合成智能体，合成了 1,827 个面向任务的环境。这些任务难以求解但易于验证。合成工作流主要包括环境与工具集构建、任务合成与解生成。具体而言，流程如下。

- 给定一个任务类别（例如规划旅行行程）以及一个配备 bash 与搜索工具的沙箱，该智能体首先使用这些工具从互联网生成或检索相关数据，并将其存入沙箱数据库。
- 随后，该智能体合成一组任务特定的工具，每个工具都以函数的形式实现。
- 为创建既有挑战性又可自动验证的任务，该智能体首先基于当前数据库提出一个简单任务，并给出其用 Python 实现的解函数与验证函数。解函数被限制为只能调用工具函数或执行逻辑计算，不

能调用其他函数或直接访问数据库，从而确保任务只能通过工具接口求解。此外，解函数产生的结果必须通过验证函数校验。如果解未通过校验，智能体将修改解函数或验证函数，直到解的输出通过验证。随后，智能体迭代地提高任务难度，并相应更新解函数与验证函数。在这一迭代过程中，如果当前工具集不足以求解任务，智能体会扩充工具集。

按照这一工作流，我们获得了数千个 <environment, tools, task, verifier> 元组。随后，我们使用 DeepSeek-V3.2 在该数据集上进行 RL，并且仅保留 pass@100 非零的实例，最终得到 1,827 个环境及其对应任务（共 4,417 个）。下文给出一个合成的旅行规划示例。该示例表明，尽管在大规模组合空间中搜索满足所有约束的旅行方案颇具挑战，但检查给定候选解是否满足这些约束则相对直接。

合成任务示例：旅行规划（An Example of Synthesized Task: Trip Planning）

我计划从杭州出发进行为期三天的旅行，需要你帮我制定 2025 年 10 月 1 日至 10 月 3 日的行程。几点重要要求：整个行程中我不想重复任何城市、酒店、景点或餐厅。另外，请确保你推荐的每一家酒店、餐厅和景点都确实位于我当天所在的城市。还有关于第二天的一点——我希望在预算上精打细算。如果我最终预订了一家每晚 800 元或以上的豪华酒店，那么我需要更谨慎地控制其他开销：我在两家餐厅（午餐和晚餐）上的总花费应低于 350 元，两家餐厅的评分都应至少为 4.0 星，且下午景点门票需低于 120 元。如果第二天的酒店处于中高档（500–800 元），那么我的灵活性稍大一些——只需确保至少一家餐厅的评分在 4.0 或以上，且景点门票应低于 180 元。对于更经济实惠的酒店（200–500 元区间），我只需确保至少有一家餐厅的评分在 3.2 或以上。你能帮我整理出这份行程吗？

提交结果格式（Submit Result Format）

```
[
  { "time": "2025-10-01", "city": "cite_name", "hotel": "hotel_name", "afternoon_restaurant": "restaurant_name",
"afternoon_attraction": "attraction_name", "evening_restaurant": "restaurant_name" },
  { "time": "2025-10-02", "city": "cite_name", "hotel": "hotel_name", "afternoon_restaurant": "restaurant_name",
"afternoon_attraction": "attraction_name", "evening_restaurant": "restaurant_name" },
  { "time": "2025-10-03", "city": "cite_name", "hotel": "hotel_name", "afternoon_restaurant": "restaurant_name",
"afternoon_attraction": "attraction_name", "evening_restaurant": "restaurant_name" }
]
```

旅行规划工具集（Tool Set for Trip Planning）

函数名	描述
get_all_attractions_by_city(city)	获取给定城市的全部景点。
get_all_cities()	从数据库中获取全部城市。
get_all_hotels_by_city(city)	获取给定城市的全部酒店。
get_all_restaurants_by_city(city)	获取给定城市的全部餐厅。
get_city_by_attraction(attraction)	获取给定景点名称所属的城市。

get_city_by_hotel(hotel)	获取给定酒店名称所属的城市。
get_city_by_restaurant(restaurant)	获取给定餐厅名称所属的城市。
get_city_transport(city)	获取给定城市的全部市内交通方式。
get_infos_by_attraction(info_keywords, attraction)	获取给定景点的指定信息。
get_infos_by_city(info_keywords, city)	获取给定城市的指定信息。
get_infos_by_hotel(info_keywords, hotel)	获取给定酒店的指定信息。
get_infos_by_restaurant(info_keywords, restaurant)	获取给定餐厅的指定信息。
get_inter_city_transport(from_city, to_city)	获取给定城市对之间的全部交通方式。
get_weather_by_city_date(city, date)	获取给定城市与日期的天气。
submit_result(answer_text)	提交最终答案内容。

4 评测

4.1 主要结果

我们在 MMLU-Pro（Wang 等，2024）、GPQA Diamond（Rein 等，2023）、Human Last Exam（HLE）Text-only（Phan 等，2025）、LiveCodeBench（2024.08–2025.04）、Codeforces、Aider-Polyglot、AIME 2025、HMMT Feb 2025、HMMT Nov 2025（Balunović 等，2025）、IMOAnswerBench（Luong 等，2025）、Terminal Bench 2.0、SWE-Verified（OpenAI，2024b）、SWE Multilingual（Yang 等，2025）、BrowseComp（Wei 等，2025）、BrowseCompZh（Zhou 等，2025）、τ2-bench（Barres 等，2025）、MCP-Universe（Luo 等，2025）、MCP-Mark（EvalSys，2025）与 Tool-Decathlon（Li 等，2025）上评测模型。工具使用类基准使用标准函数调用格式评测，其中模型被配置为思考模式。对于 MCP-Universe（Luo 等，2025）与 MCP-Mark（EvalSys，2025），我们使用内部环境评测所有模型，因为搜索与 playwright 环境可能与官方设置略有差异。我们将温度设为 1.0，上下文窗口设为 128K token。

对于 AIME、HMMT、IMOAnswerBench 与 HLE 等数学相关任务，我们使用如下模板进行评测：“{question}\nPlease reason step by step, and put your final answer within \boxed{}.”在 HLE 的情况下，我们还使用官方模板评估了 DeepSeek-V3.2-Thinking，得分为 23.9。

【表格】表 2 DeepSeek-V3.2 与闭源/开源模型的对比。对于开源模型，我们仅与支持工具使用中思考的模型进行比较。粗体数字表示各模型类别（开源与闭源）中的最佳成绩。τ2-Bench 的结果按各类别的平均值计算。关于 BrowseComp，采用上下文管理技术后的性能以 * 标注。（原文第 13 页）

DeepSeek-V3.2 在推理任务上取得了与 GPT-5-high 相近的表现，但略逊于 Gemini-3.0-Pro。与 K2-Thinking 相比，DeepSeek-V3.2 以显著更少的输出 token 取得了相当的成绩，如表 3 所示。这些性能提升可归因于投入到 RL 训练中的计算资源增加。近几个月来，我们观察到性能随 RL 训练预算的扩大而持续提升，该预算已超过预训练成本的 10%。我们推测，若分配更多计算预算，推理能力还可进一步增强。值得注意的是，本文所呈现的 DeepSeek-V3.2 性能受到长度约束奖励模型的限制；解除该限制后，我们观察到模型性能的进一步提升，详见第 4.2 节。

在代码智能体评测中，DeepSeek-V3.2 在 SWE-bench Verified 与 Terminal Bench 2.0 上均显著优于开源 LLM，展现了其在真实编码工作流中的潜力。关于 Terminal Bench 2.0，如前所述，我们针对“思考模式”的上下文管理策略目前与 Terminus 不兼容；因此所报告的 46.4 分是使用 Claude Code 框架取得的。我们还在非思考模式下用 Terminus 评估了 DeepSeek-V3.2，得分为 39.3。对于 SWE-bench Verified，主要得分使用我们的内部框架获得。在其他设置下的稳健性测试——包括 Claude Code 与 RooCode 框架以及非思考模式——得到了一致的结果，分数在 72 至 74 之间。

对于搜索智能体评测，我们使用标准的商用搜索 API 评估模型。由于 DeepSeek-V3.2 支持的最大上下文长度仅为 128K，约 20% 以上的测试用例超出该限制。为此，我们采用上下文管理方法来得到最终分数。作为参考，不使用上下文管理时的得分为 51.4。更多细节见第 4.4 节。

在工具使用基准上，DeepSeek-V3.2 大幅缩小了开源与闭源 LLM 之间的性能差距，但仍低于前沿模型。对于 τ2-bench，我们使用模型本身作为用户智能体，最终各类别得分为 63.8（航空）、81.1（零售）与 96.2（电信）。对于 MCP 基准，我们采用函数调用格式，并将工具输出放在标记为“tool”角色的消息中，而非“user”角色。在测试中，我们观察到 DeepSeek-V3.2 频繁进行冗余的自我验证，生成过长的轨迹。这一倾向常常使上下文长度超出 128K 限制，在 MCP-Mark GitHub 与 Playwright 评测等任务中尤为明显。因此，这一现象妨碍了 DeepSeek-V3.2 的最终性能。不过，引入上下文管理策略可以进一步提升性能。我们将其视为未来工作的方向，以及用户实践中的一项考虑。即使 DeepSeek-V3.2 受到该问题影响，它仍显著优于现有开源模型。值得注意的是，由于这些基准所使用的环境与工具集在 RL 训练中未曾出现，所观察到的提升表明 DeepSeek-V3.2 具备将其推理策略泛化到域外智能体场景的能力。非思考模型在智能体场景下的评测见附录表 9。

4.2 DeepSeek-V3.2-Speciale 的结果

表 3 表明，DeepSeek-V3.2-Speciale 通过利用更多的推理 token 取得了更优的性能，在多个基准上超越了当前最佳的 Gemini-3.0-Pro。值得注意的是，如表 4 所示，这一通用模型在没有针对性训练的情况下，于 2025 年国际信息学奥林匹克（IOI）与 ICPC 世界总决赛（ICPC WF）中达到金牌水准。此外，通过引入 Shao 等（2025）的技术，该模型在复杂证明任务中表现出色，在 2025 年国际数学奥林匹克（IMO）与中国数学奥林匹克（CMO）中达到金牌门槛（脚注 5）。详细的评测协议见附录 D。

然而，DeepSeek-V3.2-Speciale 的 token 效率仍显著劣于 Gemini-3.0-Pro。为降低部署成本与延迟，我们在正式版 DeepSeek-V3.2 的训练中施加了更严格的 token 约束，以优化性能与成本之间的权衡。我们认为 token 效率仍是未来研究的关键领域。

脚注 5：我们评测的是 CMO 2025 的英文版。IMO 2025 与 CMO 2025 的题目及推理代码见：https://github.com/deepseek-ai/DeepSeek-Math-V2 。

【表格】表 3 推理模型的基准性能与效率。每个基准的单元格给出准确率与输出 token 数（以千计）。各基准中准确率最高者以粗体标出，次高者以下划线标出。（原文第 15 页）

竞赛	P1	P2	P3	P4	P5	P6	总分	奖牌
IMO 2025	7	7	7	7	7	0	35/42	金牌
CMO 2025	18	18	9	21	18	18	102/126	金牌
IOI 2025	100	82	72	100	55	83	492/600	金牌

竞赛	A	B	C	D	E	F	G	H	I	J	K	L	总分	奖牌
ICPC WF 2025	3	-	1	1	2	2	-	1	1	1	1	1	10/12	金牌

表 4 | DeepSeek-V3.2-Speciale 在顶级数学与编程竞赛中的表现。对于 ICPC WF 2025，我们报告各成功解出题目的提交次数。DeepSeek-V3.2-Speciale 在 ICPC WF 2025 中位列第 2，在 IOI 2025 中位列第 10。

4.3 合成智能体任务

本节中，我们开展消融实验，研究合成智能体任务的效果。我们关注两个问题。第一，合成任务对强化学习而言是否足够具有挑战性？第二，这些合成任务的泛化性如何，即它们能否迁移到不同的下游任务或真实环境？

为回答第一个问题，我们从通用合成智能体任务中随机抽取 50 个实例，同时评估用于合成的模型与前沿闭源 LLM。如表 5 所示，DeepSeek-V3.2-Exp 的准确率仅为 12%，而前沿闭源模型最高达到 62%。这些结果表明，合成数据中包含了同时对 DeepSeek-V3.2-Exp 与前沿闭源模型都具有挑战性的智能体任务。

Pass@K	DeepSeek-v3.2-Exp	Sonnet-4.5	Gemini-3.0 Pro	GPT-5-Thinking
1	12%	34%	51%	62%
2	18%	47%	65%	75%
4	26%	62%	74%	82%

表 5 | 通用合成任务在不同模型上的准确率。

为探究在合成数据上进行 RL 能否泛化到不同任务或真实环境，我们对 DeepSeek-V3.2 的 SFT 检查点（记为 DeepSeek-V3.2-SFT）施加 RL。为排除长 CoT 与其他 RL 数据的影响，我们仅在非思考模式下对合成智能体任务进行 RL。随后，我们将该模型与 DeepSeek-V3.2-SFT 及 DeepSeek-V3.2-Exp 进行比较，其中 DeepSeek-V3.2-Exp 仅在搜索与代码环境中用 RL 训练。如图 5 所示，在合成数据上进行大规模 RL，在 Tau2Bench、MCP-Mark 与 MCP-Universe 基准上相对 DeepSeek-V3.2-SFT 带来了显著提升。相比之下，将 RL 限制在代码与搜索场景并不会在这些基准上提升性能，这进一步凸显了合成数据的潜力。

【图表】图 5 仅使用合成的通用智能体数据对 DeepSeek-V3.2-SFT 进行 RL 训练（原文第 16 页）

4.4 搜索智能体的上下文管理

即便拥有 128k 这样的扩展上下文窗口，智能体工作流（尤其是基于搜索的场景）仍经常触及最大长度限制，导致推理过程被提前截断。这一瓶颈阻碍了测试时计算潜力的充分发挥。为此，我们在测试时引入上下文管理，采用简单策略在 token 用量超过上下文窗口长度的 80% 时扩展 token 预算。这些策略包括：（1）Summary，即对溢出的轨迹进行总结并重新发起 rollout；（2）Discard-75%，即丢

弃轨迹中前 75% 的工具调用历史以释放空间；（3）Discard-all，即通过丢弃此前全部工具调用历史来重置上下文（类似于 new context 工具（Anthropic，2025a））。作为对照，我们还实现了一个并行扩展基线 Parallel-fewest-step，它采样 N 条独立轨迹并选择步数最少的那条。

【图表】图 6 采用不同测试时计算扩展策略时 Browsecomp 的准确率（原文第 17 页）

我们在 BrowseComp 基准（Wei 等，2025）上评估这些策略。如图 6 所示，在不同的计算预算下，上下文管理通过让模型扩展测试时计算、为空出更多空间执行额外步骤，带来了显著性能提升。例如，Summary 将平均步数延长至 364，性能提升最高达 60.2。然而，其整体效率相对较低。尽管简单，Discard-all 在效率与可扩展性两方面都表现良好，取得 67.6 的分数，与并行扩展相当，而所用步数显著更少。

总之，测试时计算既可以通过上下文管理串行扩展，也可以并行扩展，两者都能有效扩展模型的问题求解能力。然而，不同策略的效率与可扩展性各不相同。因此，在基准评测模型性能时，必须考虑实际计算成本。同时，寻找串行与并行扩展的最优组合，以同时最大化效率与可扩展性，仍是未来工作的重要方向。

5 结论、局限与未来工作

在本工作中，我们提出了 DeepSeek-V3.2，这一框架有效弥合了计算效率与高级推理能力之间的差距。借助 DSA，我们在不牺牲长上下文性能的前提下解决了关键的计算复杂度问题。通过增加计算预算，DeepSeek-V3.2 在推理基准上取得了与 GPT-5 相当的性能。最后，我们的大规模智能体任务合成流水线的引入显著增强了工具使用能力，为构建基于开源 LLM 的稳健且可泛化的 AI 智能体开辟了新的可能。此外，我们的高计算量变体 DeepSeek-V3.2-Speciale 在 IMO 与 IOI 中的金牌成绩验证了这一点，为开源 LLM 树立了里程碑。

尽管取得了这些成果，我们承认相比 Gemini-3.0-Pro 等前沿闭源模型仍存在一些局限。第一，由于总训练 FLOPs 较少，DeepSeek-V3.2 的世界知识广度仍落后于领先的专有模型。我们计划在未来的迭代中通过扩大预训练计算来弥补这一知识差距。第二，token 效率仍是一大挑战；DeepSeek-V3.2 通常需要更长的生成轨迹（即更多 token）才能达到 Gemini-3.0-Pro 等模型的输出质量。未来工作将聚焦于优化模型推理链的智能密度以提升效率。第三，在解决复杂任务方面仍逊于前沿模型，这促使我们进一步改进基础模型与后训练配方。

附录

附录 A MLA 的 MHA 与 MQA 模式

附录 B 冷启动模板

附录 C 非思考模式 DeepSeek-V3.2 的智能体评测

附录 D IOI、ICPC 总决赛、IMO 与 CMO 的评测方法

附录 E 作者列表

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] models. We plan to address this knowledge gap in future iterations by scaling up the pre-training compute. Second, token efficiency remains a challenge; DeepSeek-V3.2 typically requires longer generation trajectories (i.e., more tokens) to match the output quality of models like Gemini- 3.0-Pro. Future work will focus on optimizing the intelligence density of the model's reasoning chains to improve efficiency. Third, solving complex tasks is still inferior to frontier models, motivating us to further refine our foundation model and post-training recipe.

[2] Anthropic. System card: Claude opus 4.5, 2025a. URL https://assets.anthropic.com/m /64823ba7485345a7/Claude-Opus-4-5-System-Card.pdf .

[3] Anthropic. Introducing claude sonnet 4.5, 2025b. URL https://www.anthropic.com/news /claude-sonnet-4-5l .

[4] M. Balunovi´ c, J. Dekoninck, I. Petrov, N. Jovanovi´ c, and M. Vechev. Matharena: Evaluating llms on uncontaminated math competitions. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmark, 2025.

[5] V . Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ -bench: Evaluating conversational agents in a dual-control environment, 2025. URL https://arxiv.org/abs/2506.07982 .

[6] DeepMind. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. arXiv preprint arXiv:2507.06261, 2025a.

[7] G. DeepMind. Gemini 3 pro model card, 2025b. URL https://storage.googleapis.com /deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf .

[8] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024. doi: 10.48550/ARXIV.2405.04434. URL https: //doi.org/10.48550/arXiv.2405.04434 .

[9] DeepSeek-AI. Deepseek-v3 technical report, 2024. URL https://arxiv.org/abs/2412.194 37 .

[10] DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. Nature, 645(8081):633–638, 2025.

[11] EvalSys. Mcpmark leaderboard, 2025. URL https://mcpmark.ai/leaderboard .

[12] J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. arXiv preprint arXiv:2510.25726, 2025.

[13] Z. Luo, Z. Shen, W. Yang, Z. Zhao, P . Jwalapuram, A. Saha, D. Sahoo, S. Savarese, C. Xiong, and J. Li. Mcp-universe: Benchmarking large language models with real-world model context protocol servers. arXiv preprint arXiv:2508.14704, 2025.

[14] T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V . Le, and J. Jung. Towards robust mathematical reasoning. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025. URL https://aclanthology.org/2025.emnlp-main.1794/ . MiniMax. https://www.minimax.io/news/minimax-m2, 2025. URL https://www.minimax. io/news/minimax-m2 .

[15] MoonShot. Introducing kimi k2 thinking, 2025. URL https://moonshotai.github.io/Kim i-K2/thinking.html .

[16] OpenAI. Learning to reason with llms, 2024a. URL https://openai.com/index/learnin g-to-reason-with-llms/ .

[17] OpenAI. Introducing SWE-bench verified we're releasing a human-validated subset of swebench that more, 2024b. URL https://openai.com/index/introducing-swe-bench -verified/ .

[18] OpenAI. Introducing gpt-5, 2025. URL https://openai.com/index/introducing-gpt-5 / .

[19] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity's last exam. arXiv preprint arXiv:2501.14249, 2025.

[20] Qwen. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388 .

[21] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

[22] J. Schulman. Approximating KL divergence, 2020. URL http://joschu.net/blog/kl-app rox.html .

[23] Z. Shao, P . Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. CoRR, abs/2402.03300, 2024. doi: 10.48550/ARXIV.2402.03300. URL https://doi.org/10 .48550/arXiv.2402.03300 .

[24] Z. Shao, Y. Luo, C. Lu, Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, and X. Zhang. Deepseekmath-v2: Towards self-verifiable mathematical reasoning, 2025.

[25] N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019. URL http://arxiv.org/abs/1911.02150 .

[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. pages 5998–6008, 2017. URL https://proceedings.neur ips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html .

[27] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. CoRR, abs/2406.01574, 2024. URL https://doi.org/10.48550/arXiv.2406.01574 .

[28] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025.

[29] J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents, 2025. URL https: //arxiv.org/abs/2504.21798 . $\boldsymbol{W\ i\ UV}$ c $ %& Input Hidden h ! { q ! , # $ } { v ! , # $ } $\boldsymbol{W\ i\ UK}$ c $ %& c ! %& c ! ’ { q ! , # (} k ! (Multi - Head Attention (Core Attention) concatenate concatenate { [q ! , # $; q ! , # ( ] } { [k ! , # $; k ! ( ] } ⋯ Output Hidden u ! ⋯ ⋯ ⋯ ⋯ ⋯ apply RoPE apply RoPE { k ! , # $ } { o ! , # } $\boldsymbol{W\ i\ UV}$ o $, & ’ $\boldsymbol{W\ i\ UK}$ q $, & ’ ⋯ concatenate Input Hidden h ! { q ! , # $ } c ! %& c ! ’ { q ! , # (} k ! (Multi - Query Attention (Core Attention) concatenate { [q ! , #) ; q ! , # ( ] } { [c ! %& ; k ! ( ] } ⋯ Output Hidden u ! ⋯ ⋯ ⋯ ⋯ ⋯ ⋯ { q ! , #) } ⋯ ⋯ { o ! , # $ } { o ! , # } apply RoPE apply RoPE

[30] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, editors, Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, pages 23078–23097. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.acl-long.1126/ .

[31] ZhiPu-AI. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. arXiv preprint arXiv:2508.06471, 2025.

[32] P . Zhou, B. Leon, X. Ying, C. Zhang, Y. Shao, Q. Ye, D. Chong, Z. Jin, C. Xie, M. Cao, et al. Browsecomp-zh: Benchmarking web browsing ability of large language models in chinese. arXiv preprint arXiv:2504.19314, 2025.

[33] (a) MHA mode of MLA. (b) MQA mode of MLA.

[34] Figure 7 | Illustration of the MHA and MQA modes of MLA. For DeepSeek-V3.1-Terminus, the MHA mode is used for training and prefilling, while the MQA mode is used for decoding.

[35] Figure 7 illustrates two aspects of MLA – the MHA and MQA modes – as well as the transformation between them.

[36] B. Cold Start Template Table 6 | An example of the reasoning data system prompt. The system prompt requires the model to output the reasoning process in the tag <think></think>.

[37] Reasoning You are an expert Python programmer. You will be given a question (problem System specification) and will generate a correct Python program that matches the spec- Prompt ification and passes all tests. Please first reason before giving the final answer. The reasoning process enclosed within <think> </think>. The final answer is output after the </think> tag. Prompt Given a linked list, swap every two adjacent nodes and return its head ... Reasoning <think> Response ... </think> [FINAL ANSWER]

[38] Table 7 | {TOOL-DESCRIPTIONS} and {TOOLCALL-FORMAT} will be replaced with the specific tools and our designed toolcall format.

[39] Agent Use Python interpreter tool to execute Python code. The code will not be shown System to the user. This tool should be used for internal reasoning, but not for code that Prompt is intended to be visible to the user (e.g. when creating plots, tables, or files). When you send a message containing Python code to python, it will be executed in a stateful Jupyter notebook environment. python will respond with the output of the execution or time out after 120.0 seconds. ## Tools You have access to the following tools: {TOOL-DESCRIPTIONS} Important: ALWAYS adhere to this exact format for tool use: {TOOLCALL-FORMAT} Prompt Given a linked list, swap every two adjacent nodes and return its head ... Agent Re- [MULTI-TURN TOOLCALL] sponse [FINAL ANSWER]

[40] Table 8 | The model executes tool calls in thinking process.

[41] Reasoning You are a helpful assistant with access to a Python interpreter. Required - You may use the Python tool **multiple times** during your reasoning, a.k.a in Agent <think></think>, with a maximum of 20 code executions. System - Call the Python tool early in your reasoning to aid in solving the task. Continue Prompt reasoning and invoking tools as needed until you reach the final answer. Once you have the answer, stop reasoning and present your solution using Markdown and LaTeX. - Do NOT invoke any tools in your presented final solution steps. - To improve efficiency and accuracy, you should prefer code execution over language-based reasoning whenever possible. Keep your reasoning succinct; let the code do the heavy lifting. ## Tools You have access to the following tools: {TOOL-DESCRIPTIONS} Important: ALWAYS adhere to this exact format for tool use: {TOOLCALL-FORMAT} Prompt Given a linked list, swap every two adjacent nodes and return its head ... Agent <think> [MULTI-TURN Thinking-Then-TOOLCALL] Response </think> with Thinking [FINAL ANSWER] C. Non-thinking DeepSeek-V3.2 Agentic Evaluation

[42] Table 9 | Comparison between DeepSeek-V3.2 non-thinking and thinking modes. The terminal bench scores are evaluated with the Claude Code framework in the table. Non-thinking score of Terminal Bench 2.0 with Terminus framework is 39.3.

[43] Terminal Bench 2.0 (Acc) 37.1 46.4 Code Agent 72.1 73.1 SWE Verified (Resolved) SWE Multilingual (Resolved) 68.9 70.2

[44] τ -bench (Pass@1) 77.2 80.3 MCP-Universe (Success Rate) 38.6 45.9 ToolUse 26.5 38.0 MCP-Mark (Pass@1) Tool-Decathlon (Pass@1) 25.6 35.2

[45] The performance of non-thinking mode is slightly worse than the thinking mode, but still competitive.

[46] D. Evaluation Method of IOI, ICPC World Final, IMO, and CMO

[47] For all competitions, the model's maximum generation length is set to 128k. No tools or internet access are used, and testing strictly adheres to the contest's time and attempt limits.

[48] For the IOI evaluation, we designed our submission strategy in accordance with the official competition rules, which permit up to 50 submissions per problem and score each submission based on the maximum points achieved across all subtasks. Specifically, we first sampled 500 candidate solutions for each problem, then applied a multi-stage filtering pipeline. In the initial stage, we eliminated invalid submissions that failed to pass the provided sample test cases or exceeded the length constraints. Subsequently, we employed the DeepSeek-V32-Exp model to identify and remove samples in which the model explicitly indicated an inability or refusal to solve the problem. From the remaining valid candidates, we selected the 50 samples with the longest thinking traces for final submission.

[49] For the ICPC evaluation, we adapted the same filtering methodology but with a smaller initial sample size. We generated 32 candidate solutions per problem and applied the identical filtering criteria to select submissions.

[50] In the IMO and CMO tasks, we employ a generate-verify-refine loop. The model iteratively improves its solution until it achieves a perfect self-evaluation or hits the maximum revision cap, identical to the process in Shao et al. (2025). E. Author List

[51] Research & Engineering : Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan*, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Erhang Li, Fangqi Zhou*, Fangyun Lin, Fucong Dai, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoran Wei, Haowei Zhang, Haowen Luo, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, Jialiang Huang, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jingchang Chen, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jun Ran*, Junguang Jiang, Junjie Qiu, Junlong Li*, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexin Huang*, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Liang Zhao, Liangsheng Yin*, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyue Zhang, M.S. Di, M.Y Xu, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S.H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Songyang Zhou, Tao Ni, Tao Yun, Tian Pei, Tian Ye, Tianyuan Yue, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjun Gao, Wentao Zhang, Xi Gao, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyuan Li*, Xu Chen, Xuecheng Su, Xuehai Pan, Xuheng Lin, Xuwei Fu, Y.Q. Wang, Yang Zhang, Yanhong Xu, Yanru Ma, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yiliang Xiong, Ying He, Ying Zhou, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuduan Wang, Yue Gong*, Yuhan Wu, Yuheng Zou, Yukun Li, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z.F. Wu, Z.Z. Ren, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhiyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao

[52] Data Annotation: Bei Feng, Hui Li, J.L. Cai, Jiaqi Ni, Lei Xu, Meng Li, Ning Tian, R.J. Chen, R.L. Jin, S.S. Li, Shuang Zhou, Tianyu Sun, X.Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xinnan Song, Xinyi Zhou, Y.X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Zhen Huang, Zhipeng Xu, Zhongyu Zhang

[53] Business & Compliance: Dongjie Ji, Jian Liang, Jianzhong Guo, Jin Chen, Leyi Xia, Miaojun Wang, Mingming Li, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, Shengfeng Ye, T.Wang, W.L. Xiao, Wei An, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Ying Tang, Yukun Zha, Zekai Zhang, Zhe Ju, Zhen Zhang, Zihua Qu

[54] Authors are listed alphabetically by their first name. Names marked with * denote individuals who have departed from our team.

DEEPSEEK 官方论文 · 中文全文版

mHC：流形约束超连接

mHC: Manifold-Constrained Hyper-Connections Zhenda Xie†, Yixuan Wei*, Huanqi Cao*, Chenggang Zhao, Chengqi Deng, Jiashi Li,*

DeepSeek-AI research@deepseek.com

arXiv:2512.24880 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

近来，以超连接（Hyper-Connections，HC）为代表的研究通过扩展残差流宽度并多样化连接模式，拓展了过去十年间无处不在的残差连接范式。这些工作虽带来显著的性能提升，但这种多样化从根本上损害了残差连接所固有的恒等映射（identity mapping）性质，进而造成严重的训练不稳定与可扩展性受限，并额外带来显著的内存访问开销。为应对这些挑战，我们提出流形约束超连接（Manifold-Constrained Hyper-Connections，mHC），这是一个通用框架：它将 HC 的残差连接空间投影到特定的流形上，以恢复恒等映射性质，同时结合严格的基础设施优化以确保效率。实证实验表明，mHC 在大规模训练中行之有效，带来了切实的性能提升与更优的可扩展性。我们预期，mHC 作为 HC 的一种灵活而实用的扩展，将有助于更深入地理解拓扑架构设计，并为基础模型的演进指明有前景的方向。

1 引言

自 ResNet（He et al., 2016a）提出以来，深度神经网络架构经历了快速演进。如图 1(a) 所示，单层的结构可以形式化如下：

$$x_{l+1} = x_l + F(x_l, W_l) \qquad (1)$$

其中 x_l 与 x_{l+1} 分别表示第 l 层的 C 维输入与输出，F 表示残差函数。尽管过去十年间残差函数 F 已演变为包含卷积、注意力机制与前馈网络等多种操作，但残差连接的范式始终保持其原始形态。伴随 Transformer（Vaswani et al., 2017）架构的发展，这一范式如今已成为大语言模型（LLM）（Brown et al., 2020; Liu et al., 2024b; Touvron et al., 2023）中的基础设计要素。

这一成功主要归功于残差连接简洁的形式。更重要的是，早期研究（He et al., 2016b）揭示出，残差连接的恒等映射性质在大规模训练中维持了稳定性与效率。通过在多层之间递归地延展残差连接，由式（1）可得：

$$x_L = x_l + \Sigma_{i=l}^{L-1} F(x_i, W_i) \quad (2)$$

其中 L 与 l 分别对应更深的层与更浅的层。所谓恒等映射，指的是分量 x_l 本身，它强调了来自较浅层的信号未经任何修改便直接映射到更深层的性质。

近期，以超连接（Hyper-Connections, HC）（Zhu et al., 2024）为代表的研究为残差连接引入了新的维度，并以实证方式展示了其性能潜力。HC 的单层架构如图 1(b) 所示。通过扩展残差流的宽度并增强连接的复杂性，HC 在不改变单个单元在 FLOPs 方面计算开销的前提下，显著提升了拓扑复杂度。形式上，HC 中的单层传播定义为：

$$x_{l+1} = H_l^{res} x_l + H_l^{post\top} F(H_l^{pre} x_l, W_l) \quad (3)$$

其中 x_l 与 x_{l+1} 分别表示第 l 层的输入与输出。与式（1）的表述不同，x_l 与 x_{l+1} 的特征维度从 C 扩展为 n×C，其中 n 为扩展率。$H_l^{res} \in R^{n\times n}$ 表示一个可学习映射，用于混合残差流内的特征。同样作为可学习映射，$H_l^{pre} \in R^{1\times n}$ 将 nC 维流中的特征聚合为 C 维的层输入；反过来，$H_l^{post} \in R^{1\times n}$ 将层输出映射回该流。

然而，随着训练规模增大，HC 引入了潜在的不稳定风险。首要担忧在于，HC 的无约束特性在架构跨越多层时损害了恒等映射性质。在由多条并行流构成的架构中，理想的恒等映射起到一种守恒机制的作用：它确保前向与反向传播过程中，各流之间的平均信号强度保持不变。通过式（3）将 HC 递归延展到多层，可得：

$$x_L = (\Pi_{i=1}^{L-l} H_{L-i}^{res}) x_l + \Sigma_{i=l}^{L-1} (\Pi_{j=1}^{L-1-i} H_{L-j}^{res}) H_i^{post\top} F(H_i^{pre} x_i, W_i) \quad (4)$$

其中 L 与 l 分别表示更深的层与更浅的层。与式（2）不同，HC 中的复合映射 $\Pi_{i=1}^{L-l} H_{L-i}^{res}$ 无法保持特征的全局均值。这一偏差会导致信号无界放大或衰减，从而在大规模训练中引发不稳定。另一点需要考虑的是：尽管 HC 在 FLOPs 方面保持了计算效率，但原始设计并未解决加宽残差流所带来的内存访问成本方面的硬件效率问题。这些因素共同限制了 HC 的实际可扩展性，并阻碍其应用于大规模训练。

为应对这些挑战，我们提出 **流形约束超连接（Manifold-Constrained Hyper-Connections, mHC）**，如图 1(c) 所示。这是一个通用框架：它将 HC 的残差连接空间投影到特定流形上，以恢复恒等映射性质，同时结合严格的基础设施优化以确保效率。具体而言，mHC 利用 Sinkhorn-Knopp 算

法（Sinkhorn and Knopp, 1967），对 H_l^{res} 进行熵正则化投影，使其落入 Birkhoff 多面体。该操作有效地将残差连接矩阵约束在由双随机矩阵构成的流形内。由于这些矩阵的行和与列和均等于 1，$H_l^{res} x_l$ 这一运算表现为输入特征的凸组合。这一特性促成了良态的信号传播：特征均值保持不变，信号范数受到严格正则化，从而有效缓解信号消失或爆炸的风险。此外，由于双随机矩阵对矩阵乘法封闭，复合映射 $\prod_{i=1}^{L-l} H_{L-i}^{res}$ 仍保持这一守恒性质。因此，mHC 能有效维持任意深度之间恒等映射的稳定性。为确保效率，我们采用 kernel 融合，并基于 TileLang（Wang et al., 2025）开发混合精度 kernel。此外，我们通过选择性重计算来降低内存占用，并在 DualPipe 调度（Liu et al., 2024b）中仔细重叠通信。

在语言模型预训练上的大量实验表明，mHC 在保持 HC 性能优势的同时展现出卓越的稳定性与可扩展性。内部大规模训练显示，mHC 支持大规模训练，且在扩展率 n=4 时仅引入 6.7% 的额外时间开销。

2 相关工作

深度学习中的架构进展大致可分为微观设计与宏观设计两类。微观设计关注计算块内部的结构，规定特征如何在空间、时间与通道维度上被处理。相比之下，宏观设计确立块间的拓扑结构，从而决定特征表示如何在不同层之间传播、路由与合并。

2.1 微观设计

受参数共享与平移不变性的驱动，卷积最初主导了结构化信号的处理。尽管此后出现的深度可分离卷积（Chollet, 2017）与分组卷积（Xie et al., 2017）等变体优化了效率，但 Transformer（Vaswani et al., 2017）的出现使注意力与前馈网络（FFNs）成为现代架构的基本构建单元。注意力机制促进了全局信息传播，而 FFN 增强了单个特征的表征能力。为在性能与 LLM 的计算需求之间取得平衡，注意力机制已演进出多查询注意力（Multi-Query Attention, MQA）（Shazeer, 2019）、分组查询注意力（Grouped-Query Attention, GQA）（Ainslie et al., 2023）与多头潜在注意力（Multi-Head Latent Attention, MLA）（Liu et al., 2024a）等高效变体。同时，FFN 通过混合专家（Mixture-of-Experts, MoE）（Fedus et al., 2022; Lepikhin et al., 2020; Shazeer et al., 2017）被推广为稀疏计算范式，使得参数量得以大规模扩展而计算成本不成比例增长。

2.2 宏观设计

宏观设计支配网络的全局拓扑（Srivastava et al., 2015）。在 ResNet（He et al., 2016a）之后，DenseNet（Huang et al., 2017）与 Fractal-Net（Larsson et al., 2016）等架构分别通过稠密连接与多路径结构来提升拓扑复杂度，以期提高性能。深度层聚合（Deep Layer Aggregation, DLA）（Yu et al., 2018）通过在多种深度与分辨率上递归聚合特征，进一步扩展了这一范式。

最近，宏观设计的焦点已转向扩展残差流的宽度（Chai et al., 2020; Fang et al., 2023; Heddes et al., 2025; Mak and Flanigan, 2025; Menghani et al., 2025; Pagliardini et al., 2024; Xiao et al., 2025; Xie et al., 2023; Zhu et al., 2024）。超连接（Hyper-Connections, HC）（Zhu et al., 2024）引入可学习矩阵来调节不同深度上特征之间的连接强度，而残差矩阵 Transformer（Residual Matrix Transformer, RMT）（Mak and Flanigan, 2025）用外积记忆矩阵取代标准残差流，以促进特征存储。类似地，MUDDFormer（Xiao et al., 2025）采用多路动态稠密连接来优化跨层信息流。尽管这些方法潜力可观，但它们损害了残差连接固有的恒等映射性质，从而引入不稳定性并阻碍可扩展性。此外，由于特征宽度扩大，它们还会带来显著的内存访问开销。在 HC 的基础上，本文提出的 mHC 将残差连接空间限制在特定流形上，以恢复恒等映射性质，同时结合严格的基础设施优化以确保效率。该方法在保持扩展连接拓扑优势的同时，增强了稳定性与可扩展性。

3 预备知识

我们首先建立本文所用的记号。在 HC 的表述中，第 l 层的输入 $x_l \in \mathbb{R}^{1\times C}$ 会按因子 n 进行扩展，以构造隐藏矩阵 $x_l = (x_{l,0}^\top, \ldots, x_{l,n-1}^\top)^\top \in \mathbb{R}^{n\times C}$，该矩阵可视为 n 条残差流。这一操作有效地拓宽了残差流的宽度。为管理该流的读出、写入与更新过程，HC 引入三个可学习的线性映射——H_l^{pre}、$H_l^{post} \in \mathbb{R}^{1\times n}$ 以及 $H_l^{res} \in \mathbb{R}^{n\times n}$。这些映射修改了式（1）所示的标准残差连接，得到式（3）中给出的表述。

在 HC 的表述中，可学习映射由两部分系数构成：依赖输入的部分与全局的部分，分别称为动态映射与静态映射。形式上，HC 按如下方式计算这些系数：

$$\tilde{x}_l = \mathrm{RMSNorm}(x_l)$$

$$H_l^{pre} = \alpha_l^{pre} \cdot \tanh(\theta_l^{pre} \tilde{x}_l^\top) + b_l^{pre}$$

$$H_l^{post} = \alpha_l^{post} \cdot \tanh(\theta_l^{post} \tilde{x}_l^\top) + b_l^{post}$$

$$H_l^{res} = \alpha_l^{res} \cdot \tanh(\theta_l^{res} \tilde{x}_l^\top) + b_l^{res} \qquad (5)$$

其中 RMSNorm(·)（Zhang and Sennrich, 2019）作用于最后一维，标量 α_l^pre、α_l^post 与 α_l^res ∈ R 是初始化为小值的可学习门控因子。动态

……映射通过线性投影得到，其参数为 θ^pre_l、θ^post_l ∈ R^{1×C} 与 θ^res_l ∈ R^{n×C}；而静态映射由可学习偏置 b^pre_l、b^post_l ∈ R^{1×n} 与 b^res_l ∈ R^{n×n} 表示。值得注意的是，引入这些映射——H^pre_l、H^post_l 与 H^res_l——所带来的计算开销可以忽略不计，因为典型的扩展率 n（例如 4）远小于输入维度 C。借助这一设计，HC 有效地将残差流的信息容量与该层的输入维度解耦，而后者与模型的计算复杂度（FLOPs）强相关。因此，HC 为通过调整残差流宽度进行扩展提供了一条新途径，补充了预训练 scaling laws（Hoffmann et al., 2022）中所讨论的模型 FLOPs 与训练数据规模这两个传统的扩展维度。

尽管 HC 需要三个映射来处理残差流与层输入之间的维度不匹配，但表 1 中的初步实验表明，残差映射 H^res_l 带来的性能增益最为显著。这一发现凸显了残差流内部有效信息交换的关键重要性。

表 1：HC 各组成部分的消融研究。当某个特定映射（H^pre_l、H^post_l 或 H^res_l）被禁用时，我们采用固定映射来维持维度一致性：H^pre_l 使用 1/n 的均匀权重，H^post_l 使用全 1 的均匀权重，H^res_l 使用单位矩阵（原文第 6 页）。

H^res_l	H^pre_l	H^post_l	绝对损失差
—	—	—	0.0
✓	—	—	−0.022
✓	✓	—	−0.025
✓	✓	✓	−0.027

3.1 数值不稳定性

尽管残差映射 H^res_l 对性能至关重要，但它的逐层顺序应用给数值稳定性带来了显著风险。如公式（4）所述，当 HC 扩展到多层时，从第 l 层到第 L 层的有效信号传播由复合映射 ∏_{i=1}^{L−l} H^res_{L−i} 决定。由于可学习的映射 H^res_l 不受约束，该复合映射不可避免地会偏离恒等映射。因此，无论在前向传播还是反向传播中，信号幅度都容易出现爆炸或消失。这一现象破坏了残差学习的基本前提——即依赖无阻碍的信号流动——从而使更深或更大规模模型的训练过程变得不稳定。

经验证据支持这一分析。如图 2 所示，我们在大规模实验中观察到不稳定的损失行为。以 mHC 为基线，HC 在约 12k 步时出现意外的损失激增，这与梯度范数的不稳定性高度相关。此外，对

H^res_l 的分析验证了这种不稳定性的机制。为了量化复合映射 ∏_{i=1}^{L−l} H^res_{L−i} 如何沿残差流放大信号，我们使用两个指标：第一个基于复合映射行和的最大绝对值，刻画前向传播中最坏情况的扩张；第二个基于列和的最大绝对值，对应反向传播。我们将这两个指标称为复合映射的 Amax 增益幅度（Amax Gain Magnitude）。如图 3（b）所示，Amax 增益幅度出现了峰值达 3000 的极端值，与 1 形成鲜明偏离，这证实了残差流存在爆炸现象。

【图表】图 2 超连接（HC）的训练不稳定性。该图展示了（a）HC 相对于 mHC 的绝对损失差，以及（b）梯度范数的对比。所有结果均基于 27B 模型（原文第 7 页）

【图表】图 3 超连接（HC）的传播不稳定性。该图展示了 27B 模型中（a）单层映射 H^res_l 与（b）复合映射 ∏_{i=1}^{L−l} H^res_{L−i} 的传播动态。层索引 l（x 轴）将每个标准 Transformer 块展开为两个独立的层（注意力与 FFN）。Amax 增益幅度（y 轴）计算为最大绝对行和（用于前向信号）与最大绝对列和（用于反向梯度），并在选定序列的所有 token 上取平均（原文第 7 页）

3.2 系统开销

尽管由于额外映射的线性性质，HC 的计算复杂度尚可接受，但系统层面的开销构成了不可忽视的挑战。具体而言，内存访问（I/O）成本往往是现代模型架构中的主要瓶颈之一，这被广泛称为“内存墙”（memory wall；Dao et al., 2022）。这一瓶颈在架构设计中常被忽视，却决定性地影响运行时效率。聚焦于广泛采用的 pre-norm Transformer（Vaswani et al., 2017）架构，我们分析了 HC 固有的 I/O 模式。表 2 总结了 n 流残差设计在单个残差层中引入的每 token 内存访问开销。分析表明，HC 将内存访问成本增加了一个大致与 n 成正比的因子。若没有融合 kernel 的缓解，这种过度的 I/O 需求会显著降低训练吞吐。此外，由于 H^pre_l、H^post_l 与 H^res_l 涉及可学习参数，它们的中间激活需要保留以用于反向传播。这导致 GPU 显存占用大幅增加，往往需要使用梯度检查点（gradient checkpointing）来维持可行的显存使用。进一步地，HC 在流水线并行中需要 n 倍的通信开销（Qi et al., 2024），导致更大的气泡并降低训练吞吐。

【表格】表 2 每 token 内存访问成本对比。该分析统计了前向传播中残差流维护所引入的开销，不包含层函数 F 的内部 I/O（原文第 8 页）

4 方法

4.1 流形约束超连接

受恒等映射原理（He et al., 2016b）的启发，mHC 的核心前提是将残差映射 H^res_l 约束到一个特定的流形上。原始的恒等映射通过强制 H^res_l = I 来确保稳定性，但它从根本上排除了残差流内部的信息交换，而这对最大化多流架构的潜力至关重要。因此，我们提出将残差映射投影到一个流形上，该流形既能保持跨层信号传播的稳定性，又能促进残差流之间的相互交互，从而保持模型的表达能力。为此，我们将 H^res_l 限制为双随机矩阵（doubly stochastic matrix），即元素非负且行和与列和均为 1 的矩阵。形式化地，令 M^res 表示双随机矩阵的流形（也称为 Birkhoff 多胞形）。我们将 H^res_l 约束为 P_{M^res}(H^res_l)，定义为：

$$P_{M^{res}}(H^{res}_l) \coloneqq \{ H^{res}_l \in R^{n\times n} \mid H^{res}_l 1_n = 1_n, 1_n^T H^{res}_l = 1_n^T, H^{res}_l \geqslant 0 \} \quad (6)$$

其中 1_n 表示 n 维全 1 向量。

值得注意的是，当 n = 1 时，双随机条件退化为标量 1，从而恢复原始的恒等映射。选择双随机性赋予了若干有利于大规模模型训练的严格理论性质：

- **范数保持（Norm Preservation）**：双随机矩阵的谱范数有界于 1（即 $\|H^{res}_l\|_2 \leq 1$）。这意味着可学习映射是非扩张的，能够有效缓解梯度爆炸问题。
- **复合封闭性（Compositional Closure）**：双随机矩阵集合在矩阵乘法下封闭。这确保了跨多层的复合残差映射 $\prod_{i=1}^{L-l} H^{res}_{L-i}$ 仍然是双随机的，从而在整个模型深度上保持稳定性。
- **Birkhoff 多胞形的几何解释（Geometric Interpretation via the Birkhoff Polytope）**：集合 M^res 构成 Birkhoff 多胞形，即置换矩阵集合的凸包。这提供了一个清晰的几何解释：残差映射的作用相当于置换的凸组合。从数学上看，这类矩阵的反复应用倾向于单调增加跨流的信息混合，从而有效地充当一种稳健的特征融合机制。

此外，我们对输入映射 H^pre_l 与输出映射 H^post_l 施加非负性约束。该约束可防止正负系数组合所产生的信号抵消，这也可以视为一种特殊的流形投影。

4.2 参数化与流形投影

本节详细介绍 mHC 中 H^pre_l、H^post_l 与 H^res_l 的计算过程。给定第 l 层的输入隐藏矩阵 x_l ∈ R^{n×C}，我们首先将其展平为向量 fix_l = vec(x_l) ∈ R^{1×nC}，以保留完整的上下文信息。然后，我们沿用原始 HC 的公式得到动态映射与静态映射如下：

$$\text{fix}'_l = \text{RMSNorm}(\text{fix}_l)$$

$$\tilde{H}^{\text{pre}}_l = \alpha^{\text{pre}}_l \cdot (\text{fix}'_l \varphi^{\text{pre}}_l) + b^{\text{pre}}_l$$

$$\tilde{H}^{\text{post}}_l = \alpha^{\text{post}}_l \cdot (\text{fix}'_l \varphi^{\text{post}}_l) + b^{\text{post}}_l$$

$$\tilde{H}^{\text{res}}_l = \alpha^{\text{res}}_l \cdot \text{mat}(\text{fix}'_l \varphi^{\text{res}}_l) + b^{\text{res}}_l \qquad (7)$$

其中 φ^{pre}_l、$\varphi^{\text{post}}_l \in \mathbb{R}^{nC\times n}$ 与 $\varphi^{\text{res}}_l \in \mathbb{R}^{nC\times n^2}$ 是用于动态映射的线性投影，mat(·) 是从 $\mathbb{R}^{1\times n^2}$ 到 $\mathbb{R}^{n\times n}$ 的重塑函数。

随后，最终的约束映射通过下式得到：

$$H^{\text{pre}}_l = \sigma(\tilde{H}^{\text{pre}}_l)$$

$$H^{\text{post}}_l = 2\sigma(\tilde{H}^{\text{post}}_l)$$

$$H^{\text{res}}_l = \text{Sinkhorn-Knopp}(\tilde{H}^{\text{res}}_l) \qquad (8)$$

其中 $\sigma(\cdot)$ 表示 Sigmoid 函数。Sinkhorn-Knopp(·) 算子首先通过指数运算使所有元素为正，然后执行迭代归一化过程，交替地重新缩放行与列使其和为 1。具体而言，给定正矩阵 $M^{(0)} = \exp(\tilde{H}^{\text{res}}_l)$ 作为起点，归一化迭代过程如下：

$$M^{(t)} = T_r(T_c(M^{(t-1)})) \qquad (9)$$

其中 T_r 与 T_c 分别表示行归一化与列归一化。当 $t_{\max} \to \infty$ 时，该过程收敛到双随机矩阵 $H^{\text{res}}_l = M^{(t_{\max})}$。在我们的实验中，我们选择 $t_{\max} = 20$ 作为实用取值。

4.3 高效基础设施设计

本节详细介绍为 mHC 量身定制的基础设施设计。经过严格的优化，我们在大规模模型中实现了 mHC（n = 4），训练开销仅为 6.7% 的边际成本。

4.3.1 Kernel 融合

我们观察到，mHC 中的 RMSNorm 在高维隐藏状态 fix_l ∈ R^{1×nC} 上运算时会带来显著延迟，因此我们重新排列了除以范数的操作的顺序，使其位于

矩阵乘法。这一优化在保持数学等价性的同时提升了效率。此外，我们采用混合精度策略，在不牺牲速度的前提下最大化数值精度，并将共享内存访问的多个操作融合为统一的计算内核，以减少内存带宽瓶颈。基于式（10）至（13）所详述的输入与参数，我们实现三个专门的 mHC 内核来计算 H_l^{pre}、H_l^{post} 与 H_l^{res}。在这些内核中，偏置与线性投影被合并进 b_l 与 φ_l，RMSNorm 权重也被吸收进 φ_l。

- 式（14）至（15）：我们开发了一个统一内核，在 x_l 上融合两次扫描，利用矩阵乘法单元最大化内存带宽利用率。反向传播（包含两次矩阵乘法）同样被合并为单个内核，从而消除对 x_l 的重复加载。两个内核都具有精细调优的流水线（加载、类型转换、计算、存储），以高效处理混合精度。
- 式（16）至（18）：这些针对小系数的轻量级操作被机会性地融合为单个内核，显著降低内核启动开销。
- 式（19）：我们在单个内核中实现 Sinkhorn-Knopp 迭代。对于反向传播，我们推导了自定义的反向内核，在片上重计算中间结果并遍历整个迭代过程。

$$\varphi_l : \text{tfloat32}\ [nC, n^2 + 2n] \quad （10）$$

$$x_l : \text{bfloat16}\ [1, nC] \quad （11）$$

$$\alpha_l^{pre}, \alpha_l^{post}, \alpha_l^{res} : \text{float32 标量} \quad （12）$$

$$b_l : \text{float32}\ [1, n^2 + 2n] \quad （13）$$

$$[\tilde{\tilde{H}}_l^{pre}, \tilde{\tilde{H}}_l^{post}, \tilde{\tilde{H}}_l^{res}] : \text{float32} = x_l \varphi_l \quad （14）$$

$$r : \text{float32} = \|x_l\|_2 / \sqrt{(nC)} \quad （15）$$

$$[\tilde{H}_l^{pre}, \tilde{H}_l^{post}, \tilde{H}_l^{res}] : \text{float32} = (1/r)[\alpha_l^{pre} \tilde{\tilde{H}}_l^{pre}, \alpha_l^{post} \tilde{\tilde{H}}_l^{post}, \alpha_l^{res} \tilde{\tilde{H}}_l^{res}] + b_l \quad （16）$$

$$H_l^{pre} : \text{float32} = \sigma(\tilde{H}_l^{pre}) \quad （17）$$

$$H_l^{post} : \text{float32} = 2\sigma(\tilde{H}_l^{post}) \quad （18）$$

$$H_l^{res} : \text{float32} = \text{Sinkhorn-Knopp}(\tilde{H}_l^{res}) \quad （19）$$

利用上述内核推导出的系数，我们引入两个额外内核来应用这些映射：一个用于 $F_{pre} := H_l^{pre} x_l$，另一个用于 $F_{post,res} := H_l^{res} x_l + H_l^{post\top} F(\cdot,\cdot)$。通过将 H_l^{post} 与 H_l^{res} 的应用与残差合并融合在一起，我们将该内核读取的元素数量从 $(3n+1)C$ 减少到 $(n+1)C$，写入的元素数量从 $3nC$ 减少到 nC。我们使用 TileLang（Wang et al., 2025）高效实现了大多数内核（式（14）至（15）

除外）。该框架简化了具有复杂计算过程的内核实现，并使我们能够以最小的工程投入充分利用内存带宽。

4.3.2 重计算

n-stream 残差设计在训练期间会引入大量内存开销。为缓解这一问题，我们在前向传播后丢弃 mHC 内核的中间激活，并在反向传播中通过重新执行 mHC 内核（不包含沉重的层函数 F）即时重算它们。因此，对于连续 L_r 层组成的块，我们只需存储第一层的输入 x_{l0}。在排除轻量级系数并考虑 F 内部 pre-norm 的情况下，表 3 总结了为反向传播保留的中间激活。

激活值	x_{l0}	F(H_l^pre x_l, W_l)	x_l	H_l^pre x_l	RMSNorm(H_l^pre x_l)
大小（元素数）	nC	C	nC	C	C
存储方式	每 L_r 层	每层	L_r 层内临时存储	L_r 层内临时存储	L_r 层内临时存储

表 3 存储与重计算的中间激活。我们列出了为反向传播保留的每 token 激活，以及在 L_r 个连续层中重新计算的临时激活。层 l_0 表示 L_r 层中的第一层，层 l 处于 $[l_0, l_0+L_r-1]$ 范围内。

由于 mHC 内核的重计算按连续 L_r 层组成的块进行，给定总共 L 层，我们必须为全部 $\lceil L/L_r \rceil$ 个块持久存储第一层输入 x_{l0} 以用于反向传播。除这部分常驻内存外，重计算过程还会为活动块引入 $(n+2)C \times L_r$ 个元素的瞬时内存开销，这决定了反向传播期间的峰值内存占用。因此，我们通过最小化与 L_r 对应的总内存占用来确定最优块大小 L_r^*：

$$L_r^* = \arg\min_{L_r} [nC \times \lceil L/L_r \rceil + (n+2)C \times L_r] \approx \sqrt{(nL/(n+2))} \quad (20)$$

此外，大规模训练中的流水线并行施加了一个约束：重计算块不得跨越流水线阶段边界。我们观察到理论最优值 L_r^* 通常与每个流水线阶段的层数相吻合，因此选择将重计算边界与流水线阶段对齐。

4.3.3 DualPipe 中的通信重叠

在大规模训练中，流水线并行是缓解参数与梯度内存占用的标准做法。具体而言，我们采用 DualPipe 调度（Liu et al., 2024b），它有效重叠了 scale-out 互连通信流量，例如专家并行与流水线并行中的流量。然而，与单流设计相比，mHC 中提出的 n-stream 残差会跨流水线阶段引入大量通信延迟。此外，在阶段边界处，对全部 L_r 层 mHC 内核的重计算会引入不可忽视的计算开销。为解决这些瓶颈，我们扩展了 DualPipe 调度（见图 4），以改善流水线阶段边界处通信与计算的重叠。

值得注意的是，为避免阻塞通信流，我们在专用的高优先级计算流上执行 MLP（即 FFN）层的 F_post,res 内核。我们进一步避免在注意力层的长时间运行操作中使用持久化内核，从而防止长时间的停顿。这一设计使得重叠的注意力计算可以被抢占，从而在保持计算设备处理单元高利用率的同时实现灵活调度。此外，重计算过程与流水线通信依赖解耦，因为每个阶段的初始激活 x_{l0} 已经缓存在本地。

【图表】图 4 mHC 的通信-计算重叠。我们扩展 DualPipe 调度以处理 mHC 引入的开销。每个块的长度仅为示意，不代表实际时长。(F)、(B)、(W) 分别指前向传播、反向传播与权重梯度计算。F^A 与 F^M 分别表示 Attention 与 MLP 对应的内核。（原文第 12 页）

5 实验

5.1 实验设置

我们通过语言模型预训练来验证所提方法，并对比基线、HC 与我们提出的 mHC。我们采用受 DeepSeek-V3（Liu et al., 2024b）启发的 MoE 架构，训练四个不同的模型变体以覆盖不同的评估区间。具体而言，HC 与 mHC 的扩展率 n 均设为 4。我们的主要关注点是一个 27B 模型，其训练数据量与其参数量成比例，该模型作为我们系统级主要结果的实验对象。在此基础上，我们通过引入以成比例数据训练的较小 3B 与 9B 模型来分析计算扩展行为，从而观察不同计算量下的性能趋势。此外，为了专门研究 token 扩展行为，我们在 1 万亿 token 的固定语料上训练了一个单独的 3B 模型。详细的模型配置与训练超参数见附录 A.1。

5.2 主要结果

【图表】图 5 流形约束超连接（mHC）的训练稳定性。该图展示了 (a) mHC 与 HC 相对基线的绝对训练损失差距，以及 (b) 三种方法的梯度范数。所有实验均使用 27B 模型。结果表明，mHC 在损失与梯度范数两方面都表现出更好的稳定性。（原文第 12 页）

我们首先考察 27B 模型的训练稳定性与收敛性。如图 5(a)所示，mHC 有效缓解了 HC 中观察到的训练不稳定性，相比基线实现了 0.021 的最终损失降低。图 5(b) 中的梯度范数分析进一步证实了这种稳定性提升：mHC 的行为显著优于 HC，并保持与基线相当的稳定曲线。

模型	BBH（EM，3-shot）	DROP（F1，3-shot）	GSM8K（EM，8-shot）	HellaSwag（Acc.，10-shot）	MATH（EM，4-shot）	MMLU（Acc.，5-shot）	PIQA（Acc.，0-shot）	TriviaQA（EM，5-shot）
27B Baseline	43.8	47.0	46.7	73.7	22.0	59.0	78.5	54.3
27B w/ HC	48.9	51.6	53.2	74.3	26.4	63.0	79.9	56.3
27B w/ mHC	51.0	53.9	53.8	74.7	26.0	63.4	80.5	57.6

表 4 27B 模型的系统级基准结果。该表比较了基线、HC 与 mHC 在 8 个多样化下游基准上的零样本与少样本性能。mHC 一致优于基线，并在大多数基准上超过 HC，证明了其在大规模预训练中的有效性。

表 4 展示了在一组多样化基准（Bisk et al., 2020; Cobbe et al., 2021; Hendrycks et al., 2020, 2021; Joshi et al., 2017; Zellers et al., 2019）上的下游性能。mHC 带来了全面的提升，一致优于基线，并在大多数任务上超过 HC。值得注意的是，与 HC 相比，mHC 进一步增强了模型的推理能力，在 BBH（Suzgun et al., 2022）上带来 2.1% 的性能提升，在 DROP（Dua et al., 2019）上带来 2.3% 的提升。

5.3 扩展实验

【图表】图 6 mHC 相对基线的扩展性质。(a) 计算扩展曲线。实线描绘不同计算预算下的性能差距。每个点代表模型规模与数据规模的一个特定计算最优配置，从 3B、9B 扩展到 27B 参数。(b) Token 扩展曲线。3B 模型在训练过程中的轨迹。每个点代表模型在不同训练 token 下的性能。详细架构与训练配置见附录 A.1。（原文第 13 页）

为评估我们方法的可扩展性，我们报告 mHC 相对基线在不同规模下的相对损失改进。在图 6(a) 中，我们绘制了跨越 3B、9B 与 27B 参数的计算扩展曲线。该轨迹表明，即使计算预算更高，性能优势仍稳健保持，仅出现轻微衰减。此外，我们在图 6(b) 中考察了运行内动态，该图展示了 3B 模型的 token 扩展曲线。总体而言，这些发现验证了 mHC 在大规模场景中的有效性。这一结论也得到我们内部大规模训练实验的进一步佐证。

5.4 稳定性分析

【图表】图 7 流形约束超连接（mHC）的传播稳定性。该图展示了 27B 模型中 (a) 单层映射 P_{M_res}(H_l^res) 与 (b) 复合映射 ∏_{i=1}^{L−l} P_{M_res}(H_{L−i}^res) 的传播动态。结果表明，与 HC 相比，mHC 显著增强了传播

稳定性。（原文第 14 页）

【图表】图 8 可学习映射的可视化。该图展示了 HC（第一行）与 mHC（第二行）的代表性单层映射与复合映射。每个矩阵通过对选定序列中所有 token 求平均计算得到。沿 y 轴与 x 轴标注的标签分别表示前向信号增益（行和）与反向梯度增益（列和）。（原文第 14 页）

与图 3 类似，图 7 展示了 mHC 的传播稳定性。理想情况下，单层映射满足双随机约束，这意味着前向信号增益与反向梯度增益都应当等于 1。然而，采用 Sinkhorn-Knopp 算法的实际实现必须限制迭代次数以保证计算效率。在我们的设置中，我们使用 20 次迭代来获得近似解。因此，如图 7(a)所示，反向梯度增益略微偏离 1。在图 7(b) 所示的复合情形中，偏差有所增大但仍保持有界，最大值约为 1.6。值得注意的是，与 HC 中接近 3000 的最大增益幅度相比，mHC 将其显著降低了三个数量级。这些结果表明，与 HC 相比，mHC 显著增强了传播稳定性，确保了前向信号与反向梯度流的稳定。此外，图 8 展示了代表性映射。我们观察到，对于 HC，当最大增益较大时，其他数值也往往较大，这表明所有传播路径上存在普遍的不稳定性。相比之下，mHC 始终产生稳定的结果。

6 结论与展望

在本文中，我们发现：虽然如 Hyper-Connections（HC）所提出的那样，扩展残差流的宽度并使连接多样化能够带来性能提升，但这些连接的无约束性质会导致信号发散。这种破坏会损害信号能量在层间的守恒，引发训练不稳定，并阻碍深层网络的可扩展性。为应对这些挑战，我们提出了流形约束超连接（Manifold-Constrained Hyper-Connections, mHC），这是一个将残差连接空间投影到特定流形上的通用框架。通过采用 Sinkhorn-Knopp 算法对残差映射施加双随机约束，mHC 将信号传播转化为特征的凸组合。实证结果证实，mHC 有效恢复了恒等映射性质，实现了稳定的大规模训练，并相比传统 HC 具有更优的可扩展性。关键在于，通过高效的基础设施级优化，mHC 以可忽略的计算开销实现了这些改进。

作为 HC 范式的广义扩展，mHC 为未来研究开辟了若干有前景的方向。尽管本工作使用双随机矩阵来保证稳定性，该框架也支持探索针对特定学习目标定制的多种流形约束。我们预计，对不同几何约束的进一步研究可能会产生新方法，更好地优化可塑性与稳定性之间的权衡。此外，我们希望 mHC 能重新激发社区对宏观架构设计的兴趣。通过加深对拓扑结构如何影响优化与表示学习的理解，mHC 将有助于解决当前的局限，并可能为下一代基础架构的演进照亮新的路径。

附录

附录 A 模型规格与超参数详情

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] In this paper, we identify that while expanding the width of residual stream and diversifying connections yields performance gains as proposed in Hyper-Connections (HC), the unconstrained nature of these connections leads to signal divergence. This disruption compromises the conservation of signal energy across layers, inducing training instability and hindering the scalability of deep networks. To address these challenges, we introduce Manifold-Constrained Hyper-Connections (m HC), a generalized framework that projects the residual connection space onto a specific manifold. By employing the Sinkhorn-Knopp algorithm to enforce a doubly stochastic constraint on residual mappings, m HC transforms signal propagation into a convex combination of features. Empirical results confirm that m HC effectively restores the identity mapping property, enabling stable large-scale training with superior scalability compared to conventional HC. Crucially, through efficient infrastructure-level optimizations, m HC delivers these improvements with negligible computational overhead.

[2] As a generalized extension of the HC paradigm, m HC opens several promising avenues for future research. Although this work utilizes doubly stochastic matrices to ensure stability, the framework accommodates the exploration of diverse manifold constraints tailored to specific learning objectives. We anticipate that further investigation into distinct geometric constraints could yield novel methods that better optimize the trade-off between plasticity and stability. Furthermore, we hope m HC rejuvenates community interest in macro-architecture design. By deepening the understanding of how topological structures influence optimization and representation learning, m HC will help address current limitations and potentially illuminate new pathways for the evolution of next-generation foundational architectures.

[3] J. Ainslie, J. Lee-Thorp, M. De Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

[4] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi. PIQA: reasoning about physical commonsense in natural language. In The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020, pages 7432–7439. AAAI Press, 2020. doi: 10.1609/aaai.v34i05.6239. URL https://doi.org/10.1609/aaai.v34i05.6239 .

[5] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P . Dhariwal, A. Neelakantan, P . Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. Advances in neural information processing systems, 33:1877–1901, 2020.

[6] Y. Chai, S. Jin, and X. Hou. Highway transformer: Self-gating enhanced self-attentive networks. In D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, editors, Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 6887–6900, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.616. URL https://aclanthology.org/2020.acl-main.616/ .

[7] F. Chollet. Xception: Deep learning with depthwise separable convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 1251–1258, 2017. K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In Advances in Neural Information Processing Systems (NeurIPS), 2022.

[9] D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368– 2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246 .

[10] Y. Fang, Y. CAI, J. Chen, J. Zhao, G. Tian, and G. Li. Cross-layer retrospective retrieving via layer attention. In The Eleventh International Conference on Learning Representations, 2023. URL https://openreview.net/forum?id=pvgEL1yS3Ql .

[11] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research, 23(120):1–39, 2022.

[12] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016a.

[13] K. He, X. Zhang, S. Ren, and J. Sun. Identity mappings in deep residual networks. In European conference on computer vision, pages 630–645. Springer, 2016b.

[14] M. Heddes, A. Javanmard, K. Axiotis, G. Fu, M. Bateni, and V . Mirrokni. Deepcrossattention: Supercharging transformer residual connections. In Forty-second International Conference on Machine Learning, 2025. URL https://openreview.net/forum?id=j3JBfFnGYh .

[15] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[16] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[17] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, O. Vinyals, J. Rae, and L. Sifre. An empirical analysis of compute-optimal large language model training. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems, volume 35, pages 30016–30030. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf .

[18] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger. Densely connected convolutional networks. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 4700–4708, 2017. M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147 .

[19] G. Larsson, M. Maire, and G. Shakhnarovich. Fractalnet: Ultra-deep neural networks without residuals. arXiv preprint arXiv:1605.07648, 2016.

[20] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. arXiv preprint arXiv:2006.16668, 2020.

[21] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024a.

[22] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024b.

[23] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

[24] B. Mak and J. Flanigan. Residual matrix transformers: Scaling the size of the residual stream. arXiv preprint arXiv:2506.22696, 2025.

[25] G. Menghani, R. Kumar, and S. Kumar. LAurel: Learned augmented residual layer. In Forty-second International Conference on Machine Learning, 2025. URL https://open review.net/forum?id=rUDRWP9WvZ .

[26] M. Pagliardini, A. Mohtashami, F. Fleuret, and M. Jaggi. Denseformer: Enhancing information flow in transformers via depth weighted averaging. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024. URL https://openreview.net/forum ?id=kMnoh7CXrq .

[27] P . Qi, X. Wan, G. Huang, and M. Lin. Zero bubble (almost) pipeline parallelism. In The Twelfth International Conference on Learning Representations, 2024. URL https://openreview .net/forum?id=tuzTN0eIO5 .

[28] N. Shazeer. Fast transformer decoding: One write-head is all you need. arXiv preprint arXiv:1911.02150, 2019.

[29] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.

[30] R. Sinkhorn and P . Knopp. Concerning nonnegative matrices and doubly stochastic matrices. Pacific Journal of Mathematics, 21(2):343–348, 1967.

[31] R. K. Srivastava, K. Greff, and J. Schmidhuber. Training very deep networks. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, Advances in Neural Information Processing Systems, volume 28. Curran Associates, Inc., 2015. URL https://proceedings. neurips.cc/paper_files/paper/2015/file/215a71a12769b056c3c32e7299f1c5e d-Paper.pdf . J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.

[32] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[33] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971, 2023.

[34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[35] L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. arXiv preprint arXiv:2408.15664, 2024.

[36] L. Wang, Y. Cheng, Y. Shi, Z. Tang, Z. Mo, W. Xie, L. Ma, Y. Xia, J. Xue, F. Yang, et al. Tilelang: A composable tiled programming model for ai systems. arXiv preprint arXiv:2504.17577, 2025.

[37] D. Xiao, Q. Meng, S. Li, and X. Yuan. Muddformer: Breaking residual bottlenecks in transformers via multiway dynamic dense connections. arXiv preprint arXiv:2502.12170, 2025.

[38] S. Xie, R. Girshick, P . Dollár, Z. Tu, and K. He. Aggregated residual transformations for deep neural networks. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 1492–1500, 2017.

[39] S. Xie, H. Zhang, J. Guo, X. Tan, J. Bian, H. H. Awadalla, A. Menezes, T. Qin, and R. Yan. Residual: Transformer with dual residual connections, 2023. URL https://arxiv.org/abs/2304.1 4802 .

[40] F. Yu, D. Wang, E. Shelhamer, and T. Darrell. Deep layer aggregation. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 2403–2412, 2018.

[41] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[42] B. Zhang and R. Sennrich. Root mean square layer normalization. Advances in neural information processing systems, 32, 2019.

[43] D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyper-connections. arXiv preprint arXiv:2409.19606, 2024.

条件记忆：以可扩展查找开辟大语言模型的新稀疏性维度

Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models Xin Cheng1,2 ∗, Rui Tian2 ∗, Wangding Zeng2,

DeepSeek-AI research@deepseek.com

arXiv:2601.07372 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

混合专家（MoE）通过条件计算扩展模型容量，而 Transformer 缺乏原生的知识查找原语，只能通过计算低效地模拟检索。为解决这一问题，我们引入条件记忆（conditional memory）作为一条互补的稀疏性维度，并通过 Engram 加以实现——该模块对经典的 N-gram 嵌入进行现代化改造，实现 O(1) 复杂度的查找。通过形式化稀疏性分配（Sparsity Allocation）问题，我们发现了一条 U 型扩展定律，用于优化神经计算（MoE）与静态记忆（Engram）之间的权衡。在这一规律的指导下，我们将 Engram 扩展到 27B 参数，在参数量与 FLOPs 严格对齐的条件下优于 MoE 基线。最值得注意的是，尽管该记忆模块本被期望有助于知识检索（如 MMLU +3.4；CMMLU +4.0），我们却在通用推理（如 BBH +5.0；ARC-Challenge +3.7）以及代码/数学领域（HumanEval +3.0；MATH +2.4）观察到更大的增益。机理分析表明，Engram 使主干的浅层从静态重建中解放出来，从而有效地加深了网络以进行复杂推理。此外，通过将局部依赖交由查找处理，它释放了注意力容量用于全局上下文，显著提升长上下文检索能力（如 Multi-Query NIAH：84.2 → 97.0）。最后，Engram 建立了“基础设施感知”的效率优势：其确定性寻址支持在运行时从主机内存预取，开销可忽略不计。我们将条件记忆视为下一代稀疏模型不可或缺的建模原语。代码见：https://github.com/deepseek-ai/Engram。

1 引言

稀疏性是智能系统中反复出现的设计原则，从生物神经回路（Lennie，2003；Olshausen 与 Field，1997）到现代大语言模型（LLM）皆然。目前，这一原则主要通过混合专家（Mixture-of-Experts, MoE）（Dai 等，2024；Shazeer 等，2017）实现，它通过条件计算来扩展容量。由于其能够在不按比例增加计算量的情况下大幅增大模型规模，MoE 已成为前沿模型的事实标准（Comanici 等，2025；Guo 等，2025；Kimi，2025）。

尽管这一条件计算范式取得了成功，但语言信号内在的异质性表明，结构层面仍有显著优化空间。具体而言，语言建模包含两类性质不同的子任务：组合推理与知识检索。前者需要深度、动态的计算，而相当一部分文本——例如命名实体与公式化模式——是局部的、静态的且高度程式化的（Constant 等，2017；Erman，2000）。经典 N 元语法（N-gram）模型（Brants 等，2007；Liu 等，2024b；Nguyen，2024）在捕捉此类局部依赖方面的有效性意味着，这些规律可以自然地表示为计算代价低廉的查找。由于标准 Transformer（Vaswani 等，2017）缺乏原生的知识查找原语，当前 LLM 不得不通过计算来模拟检索。例如，解析一个常见的多 token 实体需要消耗多个早期层的注意力与前馈网络（Ghandeharioun 等，2024；Jin 等，2025）（见表 3）。这一过程本质上相当于在运行时昂贵地重建一张静态查找表，把宝贵的串行深度浪费在琐碎操作上，而这些深度本可分配给更高层次的推理。

为使模型架构与这种语言二元性相匹配，我们提出一种互补的稀疏性维度：条件记忆。条件计算通过稀疏激活参数来处理动态逻辑（Bengio 等，2013；Shazeer 等，2017），而条件记忆则依赖稀疏查找操作来检索固定知识的静态嵌入。作为该范式的初步探索，我们重新审视 N-gram 嵌入（Bojanowski 等，2017）这一经典实例：局部上下文作为键，通过常数时间 O(1) 的查找索引一张巨大的嵌入表（Huang 等，2025a；Pagnoni 等，2025；Tito Svenstrup 等，2017；Yu 等，2025）。我们的研究表明，或许出人意料的是，这种静态检索机制可以成为现代 MoE 架构的理想补充——但前提是设计得当。本文提出 Engram，一个以经典 N-gram 结构为基础、并配备分词器压缩、多头哈希、上下文感知门控与多分支集成等现代改造（详见第 2 节）的条件记忆模块。

为量化这两种原语之间的协同效应，我们提出稀疏分配（Sparsity Allocation）问题：在固定的总参数预算下，容量应如何在 MoE 专家与 Engram 记忆之间分配？我们的实验揭示出一条独特的 U 形扩展规律：即便是简单的查找机制，一旦被当作一等公民式的建模原语，也能成为神经计算不可或缺的补充。在这一分配规律的指导下，我们将 Engram 扩展到 27B 参数模型。与严格等参数、等 FLOPs

的 MoE-27B 基线相比，Engram-27B 在多个领域均取得了更高的效率。关键在于，这些收益并不局限于记忆容量直观上更有利的知识密集型任务（例如 MMLU：+3.4；CMMLU：+4.0；MMLU-Pro：+1.8）；我们在通用推理（例如 BBH：+5.0；ARC-Challenge：+3.7；DROP：+3.3）以及代码/数学领域（例如 HumanEval：+3.0；MATH：+2.4；GSM8K：+2.2）观察到更显著的提升。

通过 LogitLens（nostalgebraist，2020）与 CKA（Hendrycks 等，2021a）进行的机制分析揭示了这些收益的来源：Engram 使骨干网络免于在早期层重建静态知识，从而增加了可用于复杂推理的有效深度。此外，通过将局部依赖交由查找处理，Engram 释放了注意力容量，使其能够聚焦全局上下文，从而在长上下文场景中实现卓越性能——在 LongPPL（Fang 等）与 RULER（Hsieh 等）上大幅优于基线（例如 Multi-Query NIAH：97.0 对 84.2；Variable Tracking：89.0 对 77.0）。

最后，我们将“基础设施感知的效率”确立为一等原则。与 MoE 的动态路由不同，Engram 采用确定性的 ID 以实现运行时预取，从而将通信与计算重叠。实证结果表明，将 100B 参数的嵌入表卸载到主机内存带来的额外开销可以忽略不计（< 3%）。这表明 Engram 有效绕开了 GPU 显存约束，从而支持激进的参数扩展。

【图表】图 1 Engram 架构。该模块通过检索静态 N-gram 记忆并借助上下文感知门控将其与动态隐藏状态融合，从而增强骨干网络。该模块仅应用于特定层，以将记忆与计算解耦，同时保持标准输入嵌入与反嵌入模块不变。（原文第 3 页）

2 架构

2.1 概述

如图 1 所示，Engram 是一个条件记忆模块，旨在通过将静态模式存储与动态计算在结构上分离，来增强 Transformer 骨干网络。形式化地，给定输入序列 $X = (x_1, ..., x_T)$ 与第 ℓ 层的隐藏状态 $H^{(\ell)} \in R^{T \times d}$，该模块对每个位置 t 分两个功能阶段进行处理：检索与融合。首先，如第 2.2 节所述，我们提取并压缩后缀 N-gram，通过哈希确定性地检索静态嵌入向量。随后，在第 2.3 节中，这些检索到的嵌入由当前隐藏状态动态调制，并通过一个轻量级卷积进行精炼。最后，我们在第 2.4 节讨论其与多分支架构的集成，在第 2.5 节讨论系统级设计。

2.2 通过哈希 N-gram 的稀疏检索

第一个阶段将局部上下文映射到静态记忆条目，涉及分词器压缩与通过确定性哈希检索嵌入。

分词器压缩 虽然 N-gram 模型通常直接作用于分词器输出，但标准子词分词器优先保证无损重建，常常为语义等价的词分配不相交的 ID（例如 Apple 与 ␣apple）（Kudo 与 Richardson，2018；Li 等，2023b）。为最大化语义密度，我们实现了一个词表投影层。具体而言，我们预计算一个满射函数 $P: V \to V'$，基于归一化后的文本等价性（使用 NFKC（Whistler，2025）、小写化等）将原始 token ID 折叠为规范标识符。实践中，对于 128k 词表的分词器，该过程使有效词表规模减少了 23%（见附录 C）。形式化地，对于位置 t 的 token，我们将其原始 ID x_t 映射为规范 ID $x'_t = P(x_t)$，以构成后缀 N-gram $g_{t,n} = (x'_{t-n+1}, ..., x'_t)$。

多头哈希 直接为所有可能的 N-gram 组合空间参数化是不可行的。遵循 Tito Svenstrup 等（2017）的做法，我们采用基于哈希的方法。为缓解冲突，我们为每个 N-gram 阶数 n 使用 K 个不同的哈希头。每个头 k 通过确定性函数 $\varphi_{n,k}$ 将压缩后的上下文映射为嵌入表 $E_{n,k}$（素数大小 $M_{n,k}$）中的索引：

$$z_{t,n,k} \triangleq \varphi_{n,k}(g_{t,n}),\ e_{t,n,k} = E_{n,k}[z_{t,n,k}] \qquad (1)$$

实践中，$\varphi_{n,k}$ 实现为轻量级的乘法-XOR 哈希。我们通过拼接所有检索到的嵌入来构造最终记忆向量 $e_t \in R^{d_{mem}}$：

$$e_t \triangleq \Big\Vert_{n=2}^{N} \Big\Vert_{k=1}^{K} e_{t,n,k} \qquad (2)$$

2.3 上下文感知门控

检索到的嵌入 e_t 作为上下文无关的先验。然而，由于它们是静态的，本质上缺乏上下文适应性，并且可能因哈希冲突或一词多义而受到噪声影响（Haber 与 Poesio，2024）。为增强表达能力并消除这种歧义，我们采用一种受注意力机制（Bahdanau 等，2015；Vaswani 等，2017）启发的上下文感知门控机制。具体而言，我们利用当前隐藏状态 h_t——它已通过前置注意力层聚合了全局上下文——作为动态 Query，而将检索到的记忆 e_t 作为 Key 与 Value 投影的来源：

$$k_t = W_K e_t,\ v_t = W_V e_t \qquad (3)$$

其中 W_K、W_V 为可学习投影矩阵。为保证梯度稳定性（Dehghani 等，2023），我们在计算标量门 $\alpha_t \in (0, 1)$ 之前对 Query 与 Key 应用 RMSNorm（Zhang 与 Sennrich，2019）：

$$\alpha_t = \sigma\left(\mathrm{RMSNorm}(h_t)^\top \mathrm{RMSNorm}(k_t) / \sqrt{d} \right) \qquad (4)$$

门控后的输出定义为 $\tilde{v}_t = \alpha_t \cdot v_t$。这一设计强制语义对齐：若检索到的记忆 e_t 与当前上下文 h_t 相矛盾，门 α_t 会趋近于零，从而有效抑制噪声。

最后，为扩大感受野并增强模型的非线性，我们引入一个短的深度可分离因果卷积（Gu 等，2022；Peng 等，2023）。令 $\tilde{V} \in \mathbb{R}^{T \times d}$ 表示门控值的序列。使用核大小 w（设为 4）、膨胀率 δ（设为最大 N-gram 阶数）与 SiLU 激活函数（Elfwing 等，2018），最终输出 Y 计算为：

$$Y = \mathrm{SiLU}\left(\mathrm{Conv1D}\left(\mathrm{RMSNorm}(\tilde{V})\right)\right) + \tilde{V} \qquad (5)$$

【图表】图 2 Engram 的系统实现。（a）训练阶段：巨大的嵌入表被分片到可用的 GPU 上，采用 All-to-All 通信原语跨设备检索活跃的嵌入行。（b）推理阶段：Engram 表被卸载到主机内存。利用确定性的检索逻辑，主机异步预取并传输嵌入，使通信与前置 Transformer 模块的设备端计算重叠。（原文第 5 页）

Engram 模块通过残差连接集成到骨干网络中：$H^{(\ell)} \leftarrow H^{(\ell)} + Y$，随后接标准的 Attention 与 MoE。关键在于，Engram 并非应用于每一层；其具体放置位置经过优化，以在建模效果与第 2.5 节详述的系统级延迟约束之间取得平衡。

2.4 与多分支架构的集成

在本工作中，我们并未采用标准的单流连接（He 等，2016），而是采用先进的多分支架构作为默认骨干，原因在于其更优的建模能力（Larsson 等，2017；Szegedy 等，2015；Xie 等，2025；Zhu 等，2025）。该架构的一个决定性特征是：将残差流扩展为 M 个并行分支，其中信息流由可学习的连接权重进行调制。

尽管 Engram 模块本身与拓扑无关，但将其适配到多分支框架仍需结构优化，以平衡效率与表达能力。具体而言，我们实现一种参数共享策略：所有 M 个分支共享单张稀疏嵌入表与 Value 投影矩阵 W_V，而采用 M 个不同的 Key 投影矩阵 $\{W_K^{(m)}\}_{m=1}^{M}$ 以实现分支特定的门控行为。对于第 m 个分支（隐藏状态为 $h_t^{(m)}$），分支特定的门控信号计算为：

$$\alpha_t^{(m)} = \sigma\left(\mathrm{RMSNorm}(h_t^{(m)})^{\top} \mathrm{RMSNorm}(W_K^{(m)} e_t) / \sqrt{d}\right) \qquad (6)$$

随后，检索到的记忆由作用于共享 value 向量的独立门控进行调制：$u_t^{(m)} = \alpha_t^{(m)} \cdot (W_V e_t)$。这一设计使线性投影（一个 W_V 与 M 个不同的 $W_K^{(m)}$）可以融合为单次稠密的 FP8 矩阵乘法，从而最大化现代 GPU 的计算利用率。除非另有说明，所有实验均采用与流形约束超连接（Manifold-Constrained Hyper-Connections，$M = 4$）（Xie 等，2025）的集成。

2.5 系统效率：解耦计算与内存

模型参数的扩展往往受限于 GPU 高带宽显存（HBM）的容量有限。然而，与依赖运行时隐藏状态进行动态路由的 MoE 不同，Engram 采用仅基于输入 token ID 的确定性检索机制，从而自然地将参数存储与计算解耦。这种可预测性为训练与推理带来了专门的优化策略，如图 2 所示。

在训练期间，为容纳大规模嵌入表，我们采用标准的模型并行，将表分片到可用 GPU 上。前向传播中使用 All-to-All 通信原语收集活跃行，反向传播中分派梯度，从而使总内存容量能够随加速器数量线性扩展。

在推理期间，这种确定性使得预取-重叠策略成为可能。由于记忆索引在前向传播之前就已确定，系统可以通过 PCIe 从充裕的主机内存中异步检索嵌入。为有效掩盖通信延迟，Engram 模块被放置在骨干网络的特定层，利用前置层的计算作为缓冲区，以避免 GPU 停顿。这要求一种硬件-算法协同设计策略：虽然将 Engram 放得更深能延长用于隐藏延迟的计算窗口，但我们在第 6.2 节的消融实验表明，建模性能更倾向于早期介入，以卸载局部模式重建。因此，最优放置必须同时满足建模与系统延迟两方面的约束。

此外，自然语言 N-gram 本质上服从 Zipf 分布（Chao 与 Zipf，1950；Piantadosi，2014），即一小部分模式占据了绝大多数记忆访问。这一统计特性催生了多级缓存层次结构：高频访问的嵌入可缓存在更快的存储层（例如 GPU HBM 或主机 DRAM），而长尾的稀有模式则驻留在更慢、容量更大的介质（例如 NVMe SSD）中。这种分层使 Engram 能够扩展到巨大的记忆容量，同时对有效延迟的影响微乎其微。

3 扩展规律与稀疏分配

作为条件记忆的一种实例，Engram 在结构上与 MoE 专家提供的条件计算形成互补。本节研究这种二元性的扩展性质，以及如何最优地分配稀疏容量。具体而言，有两个关键问题驱动我们的研究：

- **有限约束下的分配** 当总参数与训练算力固定时（等参数、等 FLOPs），应如何在 MoE 专家与 Engram 嵌入之间划分稀疏容量？
- **无限记忆情形** 考虑到 Engram 具有不随规模增长的 O(1) 开销，如果记忆预算被放宽或激进扩展，Engram 自身会呈现出怎样的扩展行为？

【图表】图 3 稀疏分配与 Engram 扩展。左：不同分配比例 ρ 下的验证损失，展示了两种算力预算（2e20 与 6e20 FLOPs）。两种情形均呈现 U 形，混合分配优于纯 MoE。右：无限记忆情形下的扩展行为。验证损失随嵌入数量呈

对数线性趋势。（原文第 7 页）

3.1 MoE 与 Engram 之间的最优分配比例

算力匹配的表述 我们使用三个参数指标来分析这一权衡：

- P_tot：总可训练参数，不含词表嵌入与 LM head。
- P_act：每个 token 激活的参数。该数量决定了训练成本（FLOPs）。
- P_sparse ≜ P_tot − P_act：非激活参数，代表可用于扩展模型规模而不产生计算成本的“免费”参数预算（例如未被选中的专家或未被检索的嵌入）。

我们在每个 FLOPs 预算内保持 P_tot 与 P_act 固定，从而使模型具有相同的参数数量与相同的每 token FLOPs。对于 MoE，P_act 由 top-k 选中的专家决定，而未选中专家的参数计入 P_sparse。对于 Engram，每个 token 只检索常数个槽位，因此扩展嵌入槽位数量会增加 P_tot，但不增加每 token FLOPs。

分配比例 我们将分配比例 $\rho \in [0, 1]$ 定义为分配给 MoE 专家容量的非激活参数预算占比：

$$P_{MoE}^{(sparse)} = \rho P_{sparse}, \quad P_{Engram} = (1-\rho) P_{sparse} \qquad (7)$$

直观上：

- $\rho = 1$ 对应纯 MoE 模型（所有非激活参数都是路由专家）。
- $\rho < 1$ 会减少路由专家数量，并将释放出的参数重新分配给 Engram 嵌入槽位。

实验方案 我们在两种算力情形下评估这一权衡，并在两种设置中保持稀疏比 $P_{tot}/P_{act} \approx 10$ 不变：

- $C = 2\times10^{20}$ FLOPs：$P_{tot} \approx 5.7B$，$P_{act} = 568M$。基线（$\rho = 1$）共有 106 个专家。
- $C = 6\times10^{20}$ FLOPs：$P_{tot} \approx 9.9B$，$P_{act} = 993M$。基线（$\rho = 1$）共有 99 个专家。

对于不同的 ρ，我们仅通过调整路由专家数量与 Engram 嵌入槽（embedding slot）数量来构建相应模型。所有实验均使用相同的训练流程与优化超参数。

结果与分析：图 3（左）揭示了验证损失与分配比 ρ 之间一致的 U 形关系。值得注意的是，即使将 MoE 分配比例降至仅 $\rho \approx 40\%$（即 5.7B 模型共 46 个专家、9.9B 模型共 43 个专家），Engram 模型仍能达到与纯 MoE 基线（$\rho = 100\%$）相当的性能。此外，纯 MoE 基线并非最优：将约 20%–25% 的稀疏参数预算重新分配给 Engram 可获得最佳性能。定量而言，在 10B 规模（C = 6×10^20）下，验

证损失从 1.7248（ρ = 100%）改善至 ρ ≈ 80% 最优附近的 1.7109（Δ = 0.0139）。关键在于，该最优点的位置在不同规模下保持稳定（ρ ≈ 75%–80%），表明在所考察的规模范围内（固定稀疏度下）存在稳健的分配偏好。观察到的 U 形关系印证了两个模块之间的结构性互补：

- **MoE 主导（ρ → 100%）**：模型缺乏针对静态模式的专用记忆，不得不通过深度与计算低效地重建这些模式。
- **Engram 主导（ρ → 0%）**：模型失去条件计算能力，损害需要动态、上下文相关推理的任务；在该机制下，记忆无法替代计算。

3.2 Engram 在无限记忆机制下的表现

在第 3.1 节中，我们在固定参数预算下优化了分配。现在我们探讨互补的情形：激进的记忆扩展。这一研究源于 Engram 在第 2.5 节详述的将存储与计算解耦的独特能力。

实验协议：我们使用固定的 MoE 主干，P_tot ≈ 3B、P_act = 568M，训练 100B token 以确保收敛。在该主干之上，我们附加一个 Engram 表，并将槽数 M 从 2.58×10^5 扫描到 1.0×10^7（最多增加约 130 亿参数）。作为基线，我们与 OverEncoding（Huang 等，2025a）进行比较，后者通过平均方式将 N-gram 嵌入与词表嵌入整合。我们注意到，尽管 SCONE（Yu 等，2025）等其他工作也研究大规模嵌入，但它主要面向推理，并包含额外模块（f-gram 模型）与额外的训练 FLOPs，因而与本研究严格的等计算量（iso-compute）约束不相容。

结果：图 3（右）表明，扩展记忆槽数量能带来清晰且一致的验证损失改善。在所考察的范围内，该曲线遵循严格的幂律（在对数空间中呈线性），表明 Engram 提供了一个可预测的扩展旋钮：更大的记忆持续带来收益，而无需额外的计算。关键在于扩展效率：虽然 OverEncoding 的直接平均方法也能从更大的记忆表中获益，但 Engram 能从相同的记忆预算中释放大得多的扩展潜力。结合第 3.1 节的分配规律，这些结果验证了条件记忆是一个独特、可扩展的稀疏容量维度，与 MoE 的条件计算形成互补。

4 大规模预训练

借助所提出的 Engram 架构与经验推导出的分配规律，我们将 Engram 扩展到数十亿参数规模，以验证其在实际语言模型预训练中的有效性。具体而言，我们训练四个模型：（1）Dense-4B（总参数 4.1B）、（2）MoE-27B（总参数 26.7B）、（3）Engram-27B（总参数 26.7B）与（4）Engram-40B

（总参数 39.5B）。所有模型均使用相同的数据课程（相同的 token 预算与顺序）训练，且激活参数量严格对齐。

【表格】表 1 密集、MoE 与 Engram 模型的预训练性能对比。所有模型均训练 262B token，并在激活参数量上对齐（3.8B）。Engram-27B 通过将参数从路由专家（72 → 55）重新分配给一个 5.7B 参数的 Engram 记忆，从而与 MoE-27B 等参数。Engram-40B 在保持激活参数预算固定的同时进一步扩大 Engram 记忆（18.5B 参数）。完整的训练期基准轨迹见附录 B（原文第 9 页）

4.1 实验设置

训练数据与模型配置：所有模型均在 2620 亿 token 的语料上预训练，我们使用 DeepSeek-v3（Liu 等，2024a）的分词器，词表大小为 128k。在建模方面，为确保受控比较，除非另有说明，我们对所有模型采用一致的默认设置。我们使用 30 层 Transformer，隐藏维度为 2560。每个模块将 32 个头的多头潜在注意力（MLA）（DeepSeek-AI，2024）通过 mHC（Xie 等，2025）连接到扩展率为 4 的 FFN。所有模型均使用 Muon（Jordan 等，2024；Liu 等，2025b）优化；详细超参数见附录 A。我们实例化四个不同的模型：

- **Dense-4B** 作为基线模型。它采用上述主干架构，在每个模块中加入标准稠密 FFN。
- **MoE-27B** 用 DeepSeekMoE 模块（Dai 等，2024）替换标准稠密 FFN。该模型配置 72 个路由专家与 2 个共享专家（每个 token 激活 top-k = 6 个路由专家），总参数扩展到 26.7B，同时保持与 Dense-4B 相同的激活参数。
- **Engram-27B** 严格地从 MoE-27B 架构派生，以确保公平比较。我们将路由专家数量从 72 减少到 55，并将释放出的参数重新分配给 5.7B 参数的嵌入模块（$\rho = 74.3\%$），保持总模型规模恒定为 26.7B。在 Engram 配置方面，我们在第 2 层与第 15 层实例化该模块，并将最大 N-gram 大小设为 3、头数设为 8、维度设为 1280。在优化方面，嵌入参数使用 Adam（Kingma，2014）更新，学习率放大 5 倍且不使用权重衰减；同时卷积参数初始化为零，以在训练开始时严格保持恒等映射。
- **Engram-40B** 保持与 Engram-27B 相同的主干与计算预算，但将稀疏嵌入模块扩展到 18.5B 参数（总参数 39.5B）。该模型旨在考察 Engram 的扩展特性。

评估协议：我们在涵盖语言建模、知识、推理、阅读理解与代码/数学的多样化基准集上评估模型。对于每个基准，我们遵循标准的提示协议与评估指标。

- **语言建模**：我们报告 The Pile（Gao 等，2020）测试集上的损失，以及与训练数据同分布的验证集上的损失。
- **知识与推理**：MMLU（Hendrycks 等，2021a）、MMLU-Redux（Gema 等，2025）、MMLU-Pro（Wang 等，2024b）、CMMLU（Li 等，2024）、C-Eval（Huang 等，2023）、AGIEval（Zhong 等，2024）、ARC-Easy/Challenge（Clark 等，2018）、TriviaQA（Joshi 等，2017）、TriviaQA-ZH（内部）、PopQA（Mallen 等，2023）、CCPM（Li 等，2021）、BBH（Suzgun 等，2023）、HellaSwag（Zellers 等，2019）、PIQA（Bisk 等，2020）与 WinoGrande（Sakaguchi 等，2021）。
- **阅读理解**：DROP（Dua 等，2019）、RACE（Middle/High）（Lai 等，2017）与 C3（Sun 等，2020）。
- **代码与数学**：HumanEval（Chen 等，2021）、MBPP（Austin 等，2021）、CruxEval（Gu 等，2024）、GSM8K（Cobbe 等，2021）、MGSM（Shi 等，2023）与 MATH（Hendrycks 等，2021b）。

4.2 实验结果

表 1 汇总了主要结果。首先，与既有文献一致（Borgeaud 等，2022；He，2024；Shazeer 等，2017），稀疏架构相较稠密模型展现出更优的扩展规律。在相同的训练计算预算下，三种稀疏变体（MoE-27B、Engram-27B/40B）在所有基准上均显著优于等 FLOPs 的 Dense-4B 基线。

更重要的是，Engram-27B 相较等参数、等 FLOPs 的 MoE-27B 基线持续提升。有趣的是，这些收益并不局限于知识密集型任务（如 MMLU：+3.0，MMLU-Pro：+1.8，CMMLU：+4.0）——在这些任务上记忆容量直观上更具优势。我们在通用推理领域（如 BBH：+5.0，ARC-Challenge：+3.7，DROP：+3.3）以及代码与数学推理（如 HumanEval：+3.0，MBPP：+1.6，GSM8K：+2.2，MATH：+2.4）中观察到更显著的提升。为降低基准噪声的影响并可视化训练动态，我们在附录 B 中给出预训练期间完整的基准轨迹。这些结果支持我们的假设：引入专用的知识查找原语可提升表示效率，其效果超出将全部稀疏预算分配给条件计算所能达到的水平。

最后，扩展到 Engram-40B 进一步降低了预训练损失，并提升了大多数基准上的性能。尽管它尚未在每个任务上严格优于 Engram-27B，但这很可能是训练不足所致。我们观察到，Engram-40B 与基线之间的训练损失差距在训练末期持续扩大，表明扩展后的记忆容量在当前 token 预算内尚未完全饱和。

5 长上下文训练

通过将局部依赖建模卸载到静态查找，Engram 架构为管理全局上下文保留了宝贵的注意力容量。本节中，我们通过长上下文扩展训练（Gao 等，2025；Peng 等，2024）实证验证这一结构性优势。通过严格的评估协议，将架构贡献与基座模型能力相隔离，我们证明 Engram 在长程检索与推理任务上带来显著收益。

5.1 实验设置

训练细节：为获得长上下文能力，我们采用 DeepSeek-V3（Liu 等，2024a）引入的上下文扩展策略。在预训练阶段之后，我们在 32768 token 的上下文训练阶段应用 YaRN（Peng 等，2024）扩展上下文窗口，训练 5000 步（300 亿 token 的高质量长上下文数据）。超参数为 scale $s = 10$、$\alpha = 1$、$\beta = 32$，缩放因子 $f = 0.707$。

模型配置：我们比较四种不同模型配置的上下文扩展。对于 MoE-27B 与 Engram-27B，我们均使用最终的预训练检查点（50k 步）。此外，为严格考察架构效率，我们为 Engram-27B 选取 41k 与 46k 步的两个中间检查点。尽管初始化阶段不同，所有变体都经历完全相同的上下文扩展训练协议。关键在于，Engram-27B（46k）之所以被选中，是因为它表现出与完全训练的 MoE-27B（50k）相同的预训练损失。这构成了一个受控的“等损失（Iso-Loss）”设置，确保上下文扩展期间的任何性能差异都可归因于架构，而非模型的起始质量。

评估基准：我们使用 LongPPL（Fang 等）与 RULER（Hsieh 等）评估长上下文性能。对于 LongPPL，我们构建了涵盖四类别的评估集：长篇书籍、研究论文、代码仓库与长思维链（CoT）轨迹。对于 RULER，我们在汇总为 8 个类别的 14 个子集上评估：Single（S）、Multi-keys（MK）、Multi-values（MV）与 Multi-queries（MQ）Needle-in-a-Haystack；多跳变量追踪（VT）、常见词抽取（CWE）、高频词抽取（FWE）与问答（QA）。

5.2 实验结果

评估结果汇总于表 2。为准确评估 Engram 架构的贡献，我们的分析分两步进行：首先，将基座模型能力的影响与架构设计解耦；其次，进行受控分析。

表 2：长上下文性能对比。括号中的值（例如（50k, 1.62））表示上下文扩展之前的预训练步数及对应损失。两项关键发现：（1）仅用 82% 的预训练 FLOPs（41k 对 50k），Engram-27B 便在

LongPPL（Fang 等）上达到与基线相当的性能，同时在 RULER（Hsieh 等）上取得显著更高的准确率；（2）在等预训练损失（iso-pretraining-loss，46k）与等预训练 FLOPs（50k）两种设置下，Engram-27B 在所有指标上均大幅优于基线。粗体表示最佳，下划线表示次佳。（原文第 11 页）

模型	Long PPL-Book	Long PPL-Paper	Long PPL-Code	Long PPL-L-CoT	NIAH-S	NIAH-MK	NIAH-MV	NIAH-MQ	NIAH-VT	RULER-CWE	RULER-FWE	RULER-QA
MoE-27B（50k，1.63）	4.38	2.91	2.49	14.16	100.0	88.0	92.7	84.2	77.0	4.5	73.0	34.5
Engram-27B（41k，1.66）	4.37	2.92	2.50	14.26	99.6	88.3	93.0	89.5	83.2	3.8	99.6	44.0
Engram-27B（46k，1.63）	4.19	2.84	2.45	13.59	97.6	89.0	95.5	97.0	87.2	4.3	98.6	37.5
Engram-27B（50k，1.62）	4.14	2.82	2.44	13.41	99.3	89.3	96.5	97.0	89.0	5.9	99.3	40.5

1. 长上下文能力不止取决于注意力机制。 尽管注意力机制与位置编码为上下文处理提供了结构基础（Press 等，2022；Su 等，2024；Xiao 等，2024；Yang 等，2025），但我们的结果表明，长上下文性能并不完全由架构先验决定。观察 Engram（41k → 50k）的轨迹，我们发现即使在上下文扩展阶段控制相同的模型架构与固定的计算预算，长上下文性能仍随预训练进程单调提升。这表明长上下文性能与基座模型的通用建模能力内在耦合。因此，严格的架构比较必须通过对齐基座模型损失而非仅对齐训练步数来控制这一混淆变量。

2. 受控设置下的架构优越性。 遵循上述原则，我们将 Engram 与 MoE 基线进行对比。在控制基座能力后，Engram 模块的效率优势变得明显：

- **等损失设置（46k 对基线）**：该设置严格隔离架构效率。当比较 Engram-27B（46k）与完全训练的 MoE-27B（50k）——两者在预训练损失上对齐——时，Engram 展现出显著收益。具体而言，它在复杂检索任务上胜过基线（如 Multi-Query NIAH：97.0 对 84.2；VT：87.2 对 77.0）。
- **等 FLOPs 设置（50k 对基线）**：在标准的等计算预算下，Engram-27B（50k）进一步拉大这一差距，全面取得最高性能。
- **极端设置（≈ 82% 计算量）**：即使提前停止训练的 Engram-27B（41k），也仍与完全训练的 MoE-27B（50k）保持高度竞争力。它在 LongPPL 上与基线持平，并在 RULER 上超过基线，凸显了 Engram 架构的内在优越性。

6 分析

本节中，我们研究 Engram 的内部机制，包括其有效深度（第 6.1 节）、核心模块设计（第 6.2 节）与参数敏感性（第 6.3 节）。此外，我们评估带卸载（offloading）的推理吞吐（第 6.4 节），并以一个案例研究作结（第 6.5 节）。

6.1 Engram 在功能上是否等价于增加模型深度？

当前 LLM 缺乏专用的知识查找原语，它们依赖计算来模拟记忆召回。如表 3 所示，要识别实体“Diana, Princess of Wales”，LLM 必须消耗多层注意力与 FFN 来逐步组合特征（Ghandeharioun 等，2024；Jin 等，2025；Li 与 Subramani，2025），而这一过程理论上可以通过知识查找操作来识别。

鉴于此，我们假设通过为模型配备显式的知识查找能力，Engram 分担了特征组合的早期阶段，从而有效地模拟了模型深度的增加。为验证这一假设，我们采用两种机制可解释性工具：LogitLens（Belrose 等，2023；nostalgebraist，2020）与中心化核对齐分析（CKA）（Davari 等，2023；Kornblith 等，2019）。

【图表】图 4 表征对齐与收敛速度分析。(a) 通过 LogitLens（nostalgebraist，2020）计算的逐层 KL 散度；早期层中持续更低的散度表明 Engram 加速了预测收敛。(b-c) 由 CKA（Kornblith 等，2019）计算的相似度热力图；高相似度对角线的明显上移表明，Engram 的浅层在功能上等价于 MoE 模型的更深层，从而有效增加了模型深度（原文第 13

页）

【表格】表 3 复现自 Ghandeharioun 等（2024）的实体消解（entity resolution）示例。该表展示了 LLM 如何通过多层注意力与 FFN 逐步整合上下文 token，以构建实体“Diana, Princess of Wales”的内部表征。“Latent State Translation”列给出由 PatchScope（Ghandeharioun 等，2024）为最后一个 token“Wales”自动生成的文本，而“Explanation”列给出原作者提供的人工解释。（原文第 14 页）

6.1.1 加速的预测收敛

我们首先使用 LogitLens（nostalgebraist，2020）分析预测在各层之间的演化。通过用最终的 LM Head 投影每个中间层的隐藏状态，我们计算中间输出分布与模型最终输出分布之间的 Kullback–Leibler 散度（Kullback 和 Leibler，1951）。该指标量化了某个潜在表征距离“可预测（prediction-ready）”状态有多近（Belrose 等，2023；Csordás 等，2025）。

图 4（a）报告了逐层的 KL 散度。与 MoE 基线相比，两种 Engram 变体都呈现出系统性更小的 KL 散度，其中差距最明显的是早期 block。Engram 曲线更陡的下降表明模型更快地完成特征组合。这一观察与我们的假设一致：通过显式访问外部知识，Engram 减少了所需的计算步骤，从而在网络层级中更早地得到高置信度、有效的预测。

6.1.2 表征对齐与有效深度

为进一步考察 Engram 层是否在语义上对应于基线的更深层，我们采用中心核对齐（Centered Kernel Alignment，CKA）——一种被广泛采用、用于比较表征结构的指标（Kornblith 等，2019；Kriegeskorte 等，2008）。给定两组表征 X 与 Y（例如来自不同模型或不同层的激活），CKA 定义为：

$$\mathrm{CKA}(K, L) = \mathrm{HSIC}(K, L) / \sqrt{(\mathrm{HSIC}(K, K) \cdot \mathrm{HSIC}(L, L))} \quad (8)$$

其中 $K = X X^{\mathsf{T}}$ 与 $L = Y Y^{\mathsf{T}}$ 表示 Gram 矩阵（使用线性核），HSIC 是 Hilbert-Schmidt 独立性准则（Hilbert-Schmidt Independence Criterion；Gretton 等，2005）。我们采用带 HSIC 无偏估计的小批量实现（Davari 等，2023），并在 Few-NERD 数据集（Ding 等，2021）上评估，提取命名实体最后一个 token 对应的隐藏状态。

【图表】图 5 架构消融结果。我们在两种设置下将 3B MoE 基线与 Engram 变体进行比较：（1）层敏感性（深蓝色曲线）：扫描单个 Engram 模块的插入深度，证实早期注入（第 2 层）最优，而更深层效果下降；（2）组件消融

（右侧标记）：从参考配置中移除子模块，表明多分支整合、tokenizer 压缩与上下文感知门控的重要性。（原文第 15 页）

为严格量化逐层的对应关系，我们首先计算两两 CKA 相似度矩阵 $S \in [0, 1]^{L\times L}$，其中 L 为层数。随后我们引入软对齐指数 a_j，定义为每个 Engram 层 j 最相似的前 k 个 MoE 层的加权质心：

$$a_j = \left(\sum_{i\in I_j} S_{i,j} \cdot i\right) / \left(\sum_{i\in I_j} S_{i,j}\right)，\text{其中 } I_j = \operatorname{argtop_k}_i (S_{i,j}) \quad （9）$$

这里 $S_{i,j}$ 表示 MoE 层 i 与 Engram 层 j 之间的相似度分数。指数 a_j 作为 Engram 层 j 所对应的“有效 MoE 深度”的稳健代理，其中采用 top-k 过滤（$k = 5$）以减轻低相似度噪声的影响。

图 4（b）–（c）展示了叠加软对齐曲线（白色虚线）的相似度热力图。我们观察到相对对角线的明显上移，即在相当多的层上 $a_j > j$。例如，Engram-27B 在第 5 层形成的表征与 MoE 基线约第 12 层的表征最为接近。

这种一致的非对角偏移与 LogitLens 的结果（第 6.1.1 节）相吻合，证实 Engram 在更早的层就获得了更深的表征。这验证了我们的核心假设：通过显式查找绕过早期阶段的特征组合，Engram 在功能上等价于增加模型的有效深度。

6.2 结构消融与层敏感性

本节中，我们在受控设置下对 Engram 进行消融，以考察各关键模块设计的有效性。除非另有说明，主干为 12 层的 3B MoE 模型（激活参数 0.56B），训练 100B token。图 5 报告验证损失。

橙色虚线表示 3B MoE 基线（验证损失 = 1.808）。

参考配置 我们在主干上增加一个固定的 1.6B 参数 Engram 记忆。我们的参考模型使用 {2, 3}-gram，并将 Engram 插入第 2 层与第 6 层，取得验证损失 = 1.768，相较 MoE 基线有显著提升（$\Delta = 0.04$）。下文所有结构消融均以此参考配置为基准定义。

记忆应注入到哪里？ 为研究深度敏感性，我们保持 Engram 预算固定（1.6B），但把它合并为单个 Engram 模块，并将其插入层从 1 扫描到 12（图 5 中深蓝色的“Layer Sweep”曲线）。该实验揭示了 Engram 放置上固有的权衡。

放置的权衡 早期注入 Engram 使其能够在主干耗费计算深度之前卸载局部模式重建，这与主干自然的层级处理方式相契合（Ghandeharioun 等，2024；Jin 等，2025；Li 和 Subramani，2025；Tenney 等，2019）。然而，这会带来门控精度的代价：早期的隐藏状态尚未通过注意力聚合足够的全局上下

文，且并行分支缺乏细粒度调制所需的表征差异（Xie 等，2025；Zhu 等，2025）。因此，最优放置需要平衡（i）早期卸载静态局部模式与（ii）后期利用更强的上下文查询进行门控。

扫描结果表明，第 2 层取得最佳的单层性能（验证损失 = 1.770），优于第 1 层，并随插入点加深而性能下降。这表明一轮注意力已足以为门控提供有意义的、带上下文的 h_t，同时又足够早，能够替代主干底层的局部聚合。虽然在单次注入约束下第 2 层最优，但我们发现将同样的 1.6B 记忆拆分为两个更小的模块（通过减小嵌入维度 d_mem 实现）并分别放在第 2 层与第 6 层，效果更好（验证损失 = 1.768）。这种分层设计通过将早期干预与丰富的后期上下文门控相结合，调和了这一权衡。更重要的是，分层插入还带来系统层面的实用优势，能够更好地利用内存层次结构，详见第 2.5 节。

哪些组件重要？ 从参考配置出发，我们在保持 Engram 参数预算固定的情况下逐个消融设计选择。结果以图 5 中的标记表示。我们发现三个组件带来最显著的增益：（i）多分支主干中的分支特定融合，（ii）上下文感知门控，以及（iii）tokenizer 压缩。移除其中任何一项都会导致验证损失的最大回退。具体而言，在“w/o multi branch”消融中，我们保留 mHC 主干结构，但将分支特定的门控替换为对 pre-mapping H_pre 之后的隐藏状态施加的单次 Engram 融合（Xie 等，2025）。

其他改动的影响较小：移除轻量的 depthwise 卷积仅带来边缘的性能下降。在固定 1.6B 预算下把容量分配给 4-gram 略次优——很可能是因为它稀释了更频繁的 2/3-gram 模式的容量——不过我们并不排除在更大记忆规模下高阶 N-gram 会变得有益。

【图表】图 6 Engram 消融下的性能保留情况。事实性知识高度依赖 Engram 模块，而阅读理解在很大程度上由主干保留。（原文第 17 页）

6.3 敏感性分析

为刻画 Engram 模块的功能贡献，我们在推理时完全抑制稀疏嵌入输出、同时保持主干不变，以此评估模型。关键在于，这种事后消融会引入训练 - 推理不一致，可能在复杂的混合能力任务中引入噪声。因此，我们优先分析事实性知识（Factual Knowledge）与阅读理解（Reading Comprehension）——敏感性谱系的两端——它们在这一压力测试下具有最高的信噪比。

如图 6 所示，结果呈现出鲜明的功能二分。事实性知识基准出现灾难性崩塌，仅保留原始性能的 29–44%（例如 TriviaQA 为 29%），这证实 Engram 模块是参数化知识的主要存储库。相反，阅读理解任务表现出惊人的韧性，保留 81–93%（例如 C3 为 93%），这表明依赖上下文的任务主要依靠主干的注意力机制，而非 Engram。

6.4 系统效率

Engram 相较基于路由的 MoE 的一个关键系统优势在于：其稀疏激活由显式的静态哈希 ID 寻址。这带来了严格确定性的内存访问模式：一旦 token 序列确定，下一次 Engram 查找的索引就是固定的，并可在相应层执行之前计算出来。

实验设置 我们实现了一套基于 nano-vLLM（脚注 1）的推理测试框架——它是业界标准 vLLM 引擎（Kwon 等，2023）的精简原型。为获得干净的延迟基线、避免 MoE 中专家并行（Expert Parallel）的通信模式干扰，我们在两个稠密主干（Dense-4B 与 Dense-8B）上进行基准测试。我们将一个庞大的 100B 参数 Engram 层插入第二个 Transformer block，整个嵌入表常驻主机 DRAM。推理期间，系统异步预取 Engram 层的嵌入，使 PCIe 传输与第一个 block 的计算重叠。

脚注 1：https://github.com/GeeeekExplorer/nano-vllm

【表格】表 4 端到端推理吞吐量。我们在 100B 参数 Engram 层完全卸载到主机内存的情况下测量推理吞吐量。（原文第 18 页）

结果 如表 4 所示，将 100B 参数的嵌入表卸载到主机内存带来的吞吐损失可以忽略，在 8B 主干上最大仅为 2.8%。这证实了早期稠密 block 的计算强度提供了足够的时间窗口来掩盖检索延迟。关键在于，每步的有效通信量随激活槽（slot）数量而非嵌入表总大小而扩展。

重要的是，该实验是一个保守的基线。虽然第 2.5 节的分层设计利用 Zipfian 局部性将高频项缓存在 HBM 中，但我们的实验设置迫使所有检索都经由主机内存的 PCIe 总线。这一基线检索策略仅带来极小开销，这一事实强烈表明：完全优化且感知局部性的实现将带来可忽略的吞吐损失。

6.5 案例研究：门控可视化

在第 2.3 节中，我们引入了上下文感知门控机制，旨在动态调制从静态记忆检索到的信息融入主干的强度。为实证检验 Engram 是否按预期工作，我们在图 7 中可视化了 Engram-27B 在不同样本上的门控标量 α_t（脚注 2）。

结果展现出鲜明的选择性模式。门控机制在完成局部静态模式时持续激活（以红色显示）。在英文中，我们观察到在多 token 命名实体（如“Alexander the Great”“the Milky Way”）与程式化短语（如“By the way”“Princess of Wales”）上出现强激活。该行为能有效跨语言泛化。在中文例子里，Engram 识别并检索出独特习语表达与历史实体，例如“四大发明”（Four Great Inventions）与“张仲

景”（Zhang Zhongjing）。这些定性结果证实，Engram 成功识别并处理了模式化的语言依赖，从而有效减轻了 Transformer 主干记忆这些静态关联的负担。

脚注 2：如我们的架构设置所述，该模型使用带有 Engram 模块的 mHC（M = 4），Engram 模块插入在第 2 层与第 15 层。因此，对任一给定 token，模型共计算 8 个不同的门控标量。我们观察到并非每个分支都编码可解释的激活模式。为清晰呈现本可视化，我们选择并展示与语义模式匹配相关性最强的门控值。

【图表】图 7 Engram 门控机制的可视化。热力图强度对应门控标量 $\alpha_t \in [0, 1]$ 的大小，红色越深表示激活越强。由于 Engram 作用于后缀 N-gram（此处 N = 3），某个特定 token x_t 上的高激活意味着以该 token 结尾的前置 token 序列（例如在 t 处结束的短语）被识别为可从记忆中有效检索的静态模式。（原文第 19 页）

7 相关工作

N-gram 建模与嵌入扩展 起源于 Shannon 的框架（Shannon，1948），N-gram 模型依赖局部历史来预测 token，传统上采用平滑技术（Katz，1987；Kneser 和 Ney，1995）以缓解数据稀疏问题。尽管范式已转向神经架构（Bengio 等，2003）以捕捉长程依赖，N-gram 查找的计算效率仍在现代表征学习中得到保留，FastText（Bojanowski 等，2017）等开创性工作即为例证。

近来，这一范式以嵌入扩展（embedding scaling）的形式重新兴起。Per-Layer Embeddings（Team，2025）、STEM（Sadhukhan 等，2026）、L3（Tseng 和 Sa，2026）与 DeepEmbed（RWKV Team，2025）等架构通过超大表来扩展容量，而另一条独具特色的开创性研究路线——与我们的方法最为相关——则将组合式 N-gram 结构直接整合进表征空间。N-Grammer（Roy 等，2022）用由离散潜在文本表征构造的 N-gram 直接增强 Transformer 架构，Feng 等（2023）则展示了 N-gram 在语音识别中的有效性。SuperBPE（Liu 等，2025a）与 SCONE（Yu 等，2025）明确针对高频模式：前者将多词表达合并为“superword”token，后者通过一个辅助编码模型实现。与此同时，OverEncoding（Huang 等，2025a）与 Byte Latent Transformer（BLT）（Pagnoni 等，2025）分别采用哈希 N-gram 嵌入来捕捉 token 级与字节级的局部依赖。这些研究共同表明，通过 N-gram 表征扩展参数、且计算开销极小是卓有成效的。尽管这些方法在各自的设定中都有显著收益，我们的工作在两个关键维度上有根本不同。

- 第一，关于建模与评测协议。先前的方法往往将 N-gram 嵌入视为外部增强，而未在严格公平的对比协议下验证其效率。例如，SCONE（Yu 等，2025）以推理为重心，并依赖会带来额外训练

FLOPs 的辅助模块；类似地，OverEncoding（Huang 等，2025a）即便在非等参数（non-isoparametric）设置下也未能在稀疏 MoE 主干上带来有意义的提升。相反，我们把条件记忆视为一等的建模原语，并通过精心设计的 Engram 模块加以实例化。通过在稀疏性分配（Sparsity Allocation）框架内严格评估这一设计，我们证明了它相较严格等参数、等 FLOPs 的 MoE 基线具有明显优势。

- 第二，从系统角度看，我们倡导算法-系统协同设计。现有方法将嵌入严格放在输入层（第 0 层），这本质上使内存访问与计算串行化（Huang 等，2025a；Yu 等，2025）。相反，Engram 策略性地将记忆注入更深的层，以实现通信-计算重叠。此外，通过利用 N-gram 固有的 Zipfian 分布，我们可以最大化硬件内存层次的效用。这种整体设计使 Engram 能够扩展到海量参数，而推理开销可忽略不计。

高基数类别嵌入（High-Cardinality Categorical Embeddings） 一个密切相关的表征问题出现在大规模推荐系统中：模型需要为数百个类别特征学习嵌入，而这些特征的词表可能包含数百万到数十亿个 ID（Coleman 等，2023）。先前工作通过多哈希与组合式表征（Shi 等，2020；Tito Svenstrup 等，2017）、频率感知哈希与冲突管理（Tsang 和 Ahle，2022；Zhang 等，2020，2024）、频率或重要性自适应的容量分配（Ginart 等，2021；Joglekar 等，2020；Liu 等，2021），以及基于量化、张量分解或共享内存布局的结构化压缩（Desai 等，2022；Kang 等，2020；Yin 等，2021）来改善参数-精度与内存访问之间的权衡。其他方法在多个类别字段之间共享表征（Coleman 等，2023）、在不使用显式表的情况下生成嵌入（Kang 等，2021），或用学习到的语义标识符替换任意 item ID（Rajput 等，2023；Singh 等，2024）。Engram 与这一文献共享同一挑战：在高度偏斜的访问分布下表征极大的离散键空间；但不同之处在于，它从有序的文本 N-gram 构造键，并通过上下文感知门控将检索到的表征注入 Transformer 的中间层。

混合专家（Mixture-of-Experts） MoE 架构通过为每个 token 条件性地激活一部分稀疏专家，将模型容量与计算成本解耦，这一范式由 Shazeer 等（2017）提出。随后的创新如 GShard（Lepikhin 等，2020）、BASE（Lewis 等，2021）、Switch Transformer（Fedus 等，2022）与 GLaM（Du 等，2022）实现了超线性的参数扩展，同时保持恒定的推理成本。最近，DeepSeek-MoE（Dai 等，2024）通过细粒度专家分割与共享专家隔离，展示了更优的效率，在激活参数相当时显著优于稠密模型。采用该架构的最先进模型如 DeepSeek-V3（Liu 等，2024a）与 Kimi-k2（Kimi，2025），进一步将总参数推高到数千亿规模。

记忆网络（Memory Network） 关于记忆增强网络的研究旨在扩展模型容量而不成比例地增加计算成本，大体可分为参数化与非参数化两类。参数化记忆方法，如 PKM（Lample 等，2019）、PEER（He，2024）、Selfmem（Cheng 等，2023b）、Memory+（Berges 等，2025）与 Ultra-Mem（Huang 等，2025b,c），将大规模稀疏键值存储直接整合进模型层中，从而在几乎不影响 FLOPs 的情况下显著提升容量。相反，非参数化记忆方法如 REALM（Guu 等，2020）、RETRO（Borgeaud 等，2022；Wang 等，2023）、CoG（Cao 等；Lan 等，2023）与 PlugLM（Cheng 等，2023a）将知识存储与模型处理解耦，把外部记忆视为可编辑、可扩展的键值存储，使模型无需昂贵的重新训练即可适应不断变化的信息。

知识存储机制（Mechanisms of Knowledge Storage） 与容量扩展并行，大量研究审视了 Transformer 编码与检索事实性知识的内部机制。前馈网络（FFN）被广泛假设为键值记忆（Key-Value memories）（Geva 等，2021）。在这一框架下，第一层充当模式检测器（“键”），第二层则将特定信息投射进残差流（“值”）。这种模块性由特定“知识神经元”的识别所佐证，这些神经元负责存储不同的事实（Dai 等，2022）。因果追踪（causal tracing）方法提供了进一步验证，它将事实回忆的信息流映射到特定的 FFN 层（Meng 等，2022）。这些洞见促成了 ROME（Meng 等，2022）与 MEMIT（Meng 等，2023）等精确的模型编辑算法，它们能够在不重新训练的情况下直接更新事实关联。此外，对内部表征的研究——例如 Othello-GPT（Li 等，2023a）中的研究——表明这些存储机制可能促成结构化“世界模型”的涌现，而不仅仅是统计记忆。

8 结论

在本工作中，我们提出条件记忆（conditional memory）作为对主流条件计算范式（MoE）的互补稀疏性维度，旨在解决通过动态计算模拟知识检索的低效问题。我们通过 Engram 实例化这一概念——该模块对经典的 N-gram 嵌入进行现代化改造，从而为静态模式实现可扩展的、常数时间 O(1) 的查找。

通过形式化稀疏性分配（Sparsity Allocation）问题，我们发现了一条 U 型扩展定律，表明在 MoE 专家与 Engram 记忆之间混合分配稀疏容量，严格优于纯 MoE 基线。在这一规律的指导下，我们将 Engram 扩展到 27B 参数，在多个领域取得了更优性能。值得注意的是，尽管记忆模块直觉上有助于知识检索，我们却在通用推理、代码与数学方面观察到更大的增益。

我们的机理分析表明，Engram 通过让浅层摆脱静态重建任务，有效地“加深”了网络，从而释放注意力容量以聚焦全局上下文与复杂推理。这一架构转变转化为长上下文能力的实质提升，LongPPL

与 RULER 上的性能增益即为佐证。最后，Engram 倡导将基础设施感知的效率作为一等设计原则。其确定性寻址允许存储与计算解耦，从而能够将海量参数表卸载到主机内存，而推理开销可忽略不计。我们将条件记忆功能视为下一代稀疏模型不可或缺的建模原语。

附录

附录 A 详细模型架构与超参数

附录 B 完整基准曲线

附录 C Tokenizer 压缩案例研究

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] N. Belrose, Z. Furman, L. Smith, D. Halawi, I. Ostrovsky, L. McKinney, S. Biderman, and J. Steinhardt. Eliciting latent predictions from transformers with the tuned lens. arXiv preprint arXiv:2303.08112, 2023.

[2] Y. Bengio, R. Ducharme, P . Vincent, and C. Janvin. A neural probabilistic language model. J. Mach. Learn. Res., 3:1137–1155, 2003. URL https://jmlr.org/papers/v3/bengio03a .html .

[3] Y. Bengio, N. Léonard, and A. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013. URL https://arxiv.org/abs/1308.3432 .

[4] V . Berges, B. Oguz, D. Haziza, W. Yih, L. Zettlemoyer, and G. Ghosh. Memory layers at scale. In Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025. OpenReview.net, 2025. URL https://openreview.net/forum ?id=ATqGm1WyDj .

[5] X. Bi, D. Chen, G. Chen, S. Chen, D. Dai, C. Deng, H. Ding, K. Dong, Q. Du, Z. Fu, et al. Deepseek llm: Scaling open-source language models with longtermism. arXiv preprint arXiv:2401.02954, 2024.

[6] Y. Bisk, R. Zellers, J. Gao, Y. Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In Proceedings of the AAAI conference on artificial intelligence, volume 34, pages 7432–7439, 2020.

[7] P . Bojanowski, E. Grave, A. Joulin, and T. Mikolov. Enriching word vectors with subword information. Transactions of the association for computational linguistics, 5:135–146, 2017.

[8] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, et al. Improving language models by retrieving from trillions of tokens. In International conference on machine learning, pages 2206–2240. PMLR, 2022.

[9] T. Brants, A. C. Popat, P . Xu, F. J. Och, and J. Dean. Large language models in machine translation. In J. Eisner, editor, Proceedings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL), pages 858–867, Prague, Czech Republic, June 2007. Association for Computational Linguistics. URL https://aclanthology.org/D07-1090/ .

[10] B. Cao, D. Cai, L. Cui, X. Cheng, W. Bi, Y. Zou, and S. Shi. Retrieval is accurate generation. In The Twelfth International Conference on Learning Representations.

[11] Y. R. Chao and G. K. Zipf. Human behavior and the principle of least effort: An introduction to human ecology. Language, 26:394, 1950. URL https://api.semanticscholar.org/Co rpusID:10182796 .

[12] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code, 2021. URL https: //arxiv.org/abs/2107.03374 . X. Cheng, Y. Lin, X. Chen, D. Zhao, and R. Yan. Decouple knowledge from paramters for plug-and-play language modeling. In A. Rogers, J. Boyd-Graber, and N. Okazaki, editors, Findings of the Association for Computational Linguistics: ACL 2023, pages 14288–14308, Toronto, Canada, July 2023a. Association for Computational Linguistics. doi: 10.18653/v1/20 23.findings-acl.901. URL https://aclanthology.org/2023.findings-acl.901/ .

[13] X. Cheng, D. Luo, X. Chen, L. Liu, D. Zhao, and R. Yan. Lift yourself up: Retrieval-augmented text generation with self-memory. Advances in Neural Information Processing Systems, 36: 43780–43799, 2023b.

[14] P . Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. arXiv preprint arXiv:1803.05457, 2018.

[15] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[16] B. Coleman, W.-C. Kang, M. Fahrbach, R. Wang, L. Hong, E. Chi, and D. Cheng. Unified embedding: Battle-tested feature representations for web-scale ml systems. Advances in Neural Information Processing Systems, 36:56234–56255, 2023.

[17] G. Comanici, E. Bieber, M. Schaekermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. arXiv preprint arXiv:2507.06261, 2025.

[18] M. Constant, G. Eryi˘ git, J. Monti, L. Van Der Plas, C. Ramisch, M. Rosner, and A. Todirascu. Survey: multiword expression processing: a survey. Computational Linguistics, 43(4):837–892, 2017.

[19] R. Csordás, C. D. Manning, and C. Potts. Do language models use their depth efficiently? arXiv preprint arXiv:2505.13898, 2025.

[20] D. Dai, L. Dong, Y. Hao, Z. Sui, B. Chang, and F. Wei. Knowledge neurons in pretrained transformers. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 8493–8502, 2022.

[21] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. arXiv preprint arXiv:2401.06066, 2024.

[22] M. Davari, S. Horoi, A. Natik, G. Lajoie, G. Wolf, and E. Belilovsky. Reliability of CKA as a similarity measure in deep learning. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?id=8HRvyxc606 .

[23] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. URL https://arxiv.org/abs/2405.04434 .

[24] M. Dehghani, J. Djolonga, B. Mustafa, P . Padlewski, J. Heek, J. Gilmer, A. P . Steiner, M. Caron, R. Geirhos, I. Alabdulmohsin, R. Jenatton, L. Beyer, M. Tschannen, A. Arnab, X. Wang, C. Riquelme Ruiz, M. Minderer, J. Puigcerver, U. Evci, M. Kumar, S. V . Steenkiste, G. F. Elsayed, A. Mahendran, F. Yu, A. Oliver, F. Huot, J. Bastings, M. Collier, A. A. Gritsenko, V . Birodkar, C. N. Vasconcelos, Y. Tay, T. Mensink, A. Kolesnikov, F. Pavetic, D. Tran, T. Kipf, M. Lucic, X. Zhai, D. Keysers, J. J. Harmsen, and N. Houlsby. Scaling vision transformers to 22 billion parameters. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, editors, Proceedings of the 40th International Conference on Machine Learning, volume 202 of Proceedings of Machine Learning Research, pages 7480–7512. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/dehghani23a.html .

[25] A. Desai, L. Chou, and A. Shrivastava. Random offset block embedding array (robe) for criteotb benchmark mlperf dlrm model : 1000 × compression and 3.1 × faster inference, 2022. URL https://arxiv.org/abs/2108.02191 .

[26] N. Ding, G. Xu, Y. Chen, X. Wang, X. Han, P . Xie, H. Zheng, and Z. Liu. Few-nerd: A fewshot named entity recognition dataset. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), pages 3198–3213, 2021.

[27] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, et al. Glam: Efficient scaling of language models with mixture-of-experts. In International conference on machine learning, pages 5547–5569. PMLR, 2022.

[28] D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368– 2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246 .

[29] S. Elfwing, E. Uchibe, and K. Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. Neural networks, 107:3–11, 2018.

[30] B. Erman. The idiom principle and the open choice principle. Text-Interdisciplinary Journal for the Study of Discourse, 2000.

[31] L. Fang, Y. Wang, Z. Liu, C. Zhang, S. Jegelka, J. Gao, B. Ding, and Y. Wang. What is wrong with perplexity for long-context language modeling? In The Thirteenth International Conference on Learning Representations.

[32] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. Journal of Machine Learning Research, 23(120):1–39, 2022.

[33] Y. Feng, M. Tu, R. Xia, C. Huang, and Y. Wang. Memory augmented lookup dictionary based language modeling for automatic speech recognition. In Proc. Interspeech 2023, pages 481– 485, 2023.

[34] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020.

[35] T. Gao, A. Wettig, H. Yen, and D. Chen. How to train long-context language models (effectively). In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 7376–7399, 2025. A. P . Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, et al. Are we done with mmlu? In Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 5069–5096, 2025.

[36] M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, pages 5484–5495, 2021.

[37] A. Ghandeharioun, A. Caciularu, A. Pearce, L. Dixon, and M. Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models. In International Conference on Machine Learning, pages 15466–15490. PMLR, 2024.

[38] A. A. Ginart, M. Naumov, D. Mudigere, J. Yang, and J. Zou. Mixed dimension embeddings with application to memory-efficient recommendation systems. In 2021 IEEE International symposium on information theory (ISIT), pages 2786–2791. IEEE, 2021.

[39] A. Gretton, O. Bousquet, A. Smola, and B. Schölkopf. Measuring statistical dependence with hilbert-schmidt norms. In International conference on algorithmic learning theory, pages 63–77. Springer, 2005.

[40] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022. OpenReview.net, 2022. URL https://openreview.net/forum?id=uY LFoz1vlAC .

[41] A. Gu, B. Rozière, H. J. Leather, A. Solar-Lezama, G. Synnaeve, and S. Wang. Cruxeval: A benchmark for code reasoning, understanding and execution. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=Ffpg52swvg .

[42] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P . Wang, X. Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025.

[43] K. Guu, K. Lee, Z. Tung, P . Pasupat, and M. Chang. Retrieval augmented language model pre-training. In International conference on machine learning, pages 3929–3938. PMLR, 2020.

[44] J. Haber and M. Poesio. Polysemy—Evidence from linguistics, behavioral science, and contextualized language models. Computational Linguistics, 50(1):351–417, Mar. 2024. doi: 10.1162/coli_a_00500. URL https://aclanthology.org/2024.cl-1.10/ .

[45] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016.

[46] X. O. He. Mixture of a million experts. arXiv preprint arXiv:2407.04153, 2024.

[47] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In 9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021. OpenReview.net, 2021a. URL https://openreview.net/forum?id=d7KBjmI3GmQ . D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. In J. Vanschoren and S. Yeung, editors, Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual, 2021b. URL https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/has h/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html .

[48] C.-P . Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg. Ruler: What's the real context size of your long-context language models? In First Conference on Language Modeling.

[49] H. Huang, D. Zhu, B. Wu, Y. Zeng, Y. Wang, Q. Min, and X. Zhou. Over-tokenized transformer: Vocabulary is generally worth scaling. In Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025. OpenReview.net, 2025a. URL https://openreview.net/forum?id=gbeZKej40m .

[50] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, Y. Fu, et al. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models. Advances in Neural Information Processing Systems, 36:62991–63010, 2023.

[51] Z. Huang, Y. Bao, Q. Min, S. Chen, R. Guo, H. Huang, D. Zhu, Y. Zeng, B. Wu, X. Zhou, et al. Ultramemv2: Memory networks scaling to 120b parameters with superior long-context learning. arXiv preprint arXiv:2508.18756, 2025b.

[52] Z. Huang, Q. Min, H. Huang, Y. Zeng, D. Zhu, R. Guo, and X. Zhou. Ultra-sparse memory network. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025c. URL https://openreview.n et/forum?id=zjeHLSiNv1 .

[53] M. Jin, Q. Yu, J. Huang, Q. Zeng, Z. Wang, W. Hua, H. Zhao, K. Mei, Y. Meng, K. Ding, F. Yang, M. Du, and Y. Zhang. Exploring concept depth: How large language models acquire knowledge and concept at different layers? In O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. D. Eugenio, and S. Schockaert, editors, Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025, pages 558–573. Association for Computational Linguistics, 2025. URL https://acla nthology.org/2025.coling-main.37/ .

[54] M. R. Joglekar, C. Li, M. Chen, T. Xu, X. Wang, J. K. Adams, P . Khaitan, J. Liu, and Q. V . Le. Neural input search for large scale recommendation models. In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pages 2387–2397, 2020.

[55] K. Jordan, Y. Jin, V . Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/p osts/muon/ .

[56] M. Joshi, E. Choi, D. S. Weld, and L. Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017, Vancouver, Canada, July 30 - August 4, Volume 1: Long Papers, pages 1601–1611. Association for Computational Linguistics, 2017. doi: 10.18653/V1/P17-1147. URL https://doi.org/ 10.18653/v1/P17-1147 . W.-C. Kang, D. Z. Cheng, T. Chen, X. Yi, D. Lin, L. Hong, and E. H. Chi. Learning multi-granular quantized embeddings for large-vocab categorical features in recommender systems. In Companion Proceedings of the Web Conference 2020, pages 562–566, 2020.

[57] W.-C. Kang, D. Z. Cheng, T. Yao, X. Yi, T. Chen, L. Hong, and E. H. Chi. Learning to embed categorical features without embedding tables for recommendation. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, pages 840–850, 2021.

[58] S. M. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. IEEE Trans. Acoust. Speech Signal Process., 35(3):400–401, 1987. doi: 10.1109/TASSP .1987.1165125. URL https://doi.org/10.1109/TASSP.1987.1165125 .

[59] Kimi. Kimi K2: open agentic intelligence. CoRR, abs/2507.20534, 2025. doi: 10.48550/ARXIV.2 507.20534. URL https://doi.org/10.48550/arXiv.2507.20534 .

[60] D. P . Kingma. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980, 2014.

[61] R. Kneser and H. Ney. Improved backing-off for m-gram language modeling. In 1995 international conference on acoustics, speech, and signal processing, volume 1, pages 181– 184. IEEE, 1995.

[62] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In International conference on machine learning, pages 3519–3529. PMlR, 2019.

[63] N. Kriegeskorte, M. Mur, and P . A. Bandettini. Representational similarity analysis-connecting the branches of systems neuroscience. Frontiers in systems neuroscience, 2:249, 2008.

[64] T. Kudo and J. Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In E. Blanco and W. Lu, editors, Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, EMNLP 2018: System Demonstrations, Brussels, Belgium, October 31 - November 4, 2018, pages 66–71. Association for Computational Linguistics, 2018. doi: 10.18653/V1/D18-2012. URL https://doi.org/10.18653/v1/d18-2012 .

[65] S. Kullback and R. A. Leibler. On information and sufficiency. The annals of mathematical statistics, 22(1):79–86, 1951.

[66] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In Proceedings of the 29th symposium on operating systems principles, pages 611–626, 2023.

[67] G. Lai, Q. Xie, H. Liu, Y. Yang, and E. H. Hovy. RACE: large-scale reading comprehension dataset from examinations. In M. Palmer, R. Hwa, and S. Riedel, editors, Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017, Copenhagen, Denmark, September 9-11, 2017, pages 785–794. Association for Computational Linguistics, 2017. doi: 10.18653/V1/D17-1082. URL https://doi.org/10.18653/v1/d1 7-1082 .

[68] G. Lample, A. Sablayrolles, M. Ranzato, L. Denoyer, and H. Jégou. Large memory layers with product keys. Advances in Neural Information Processing Systems, 32, 2019.

[69] T. Lan, D. Cai, Y. Wang, H. Huang, and X.-L. Mao. Copy is all you need. arXiv preprint arXiv:2307.06962, 2023. G. Larsson, M. Maire, and G. Shakhnarovich. Fractalnet: Ultra-deep neural networks without residuals. In 5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings. OpenReview.net, 2017. URL https: //openreview.net/forum?id=S1VaB4cex .

[70] P . Lennie. The cost of cortical computation. Current biology, 13(6):493–497, 2003.

[71] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. arXiv preprint arXiv:2006.16668, 2020.

[72] M. Lewis, S. Bhosale, T. Dettmers, N. Goyal, and L. Zettlemoyer. Base layers: Simplifying training of large, sparse models. In M. Meila and T. Zhang, editors, Proceedings of the 38th International Conference on Machine Learning, volume 139 of Proceedings of Machine Learning Research, pages 6265–6274. PMLR, 18–24 Jul 2021. URL https://proceedings. mlr.press/v139/lewis21a.html .

[73] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese. In Findings of the Association for Computational Linguistics: ACL 2024, pages 11260–11285, 2024.

[74] K. Li, A. K. Hopkins, D. Bau, F. B. Viégas, H. Pfister, and M. Wattenberg. Emergent world representations: Exploring a sequence model trained on a synthetic task. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023a. URL https://openreview.net/forum?id=DeG07_TcZvT .

[75] M. Li and N. Subramani. Echoes of bert: Do modern language models rediscover the classical nlp pipeline?, 2025. URL https://arxiv.org/abs/2506.02132 .

[76] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, M. Davaadorj, J. Lamy- Poirier, J. Monteiro, O. Shliazhko, N. Gontier, N. Meade, A. Zebaze, M. Yee, L. K. Umapathi, J. Zhu, B. Lipkin, M. Oblokulov, Z. Wang, R. M. V , J. T. Stillerman, S. S. Patel, D. Abulkhanov, M. Zocca, M. Dey, Z. Zhang, N. Fahmy, U. Bhattacharyya, W. Yu, S. Singh, S. Luccioni, P . Villegas, M. Kunakov, F. Zhdanov, M. Romero, T. Lee, N. Timor, J. Ding, C. Schlesinger, H. Schoelkopf, J. Ebert, T. Dao, M. Mishra, A. Gu, J. Robinson, C. J. Anderson, B. Dolan-Gavitt, D. Contractor, S. Reddy, D. Fried, D. Bahdanau, Y. Jernite, C. M. Ferrandis, S. Hughes, T. Wolf, A. Guha, L. von Werra, and H. de Vries. Starcoder: may the source be with you! Trans. Mach. Learn. Res., 2023, 2023b. URL https://openreview.net/forum?id=KoFOg41haE .

[77] W. Li, F. Qi, M. Sun, X. Yi, and J. Zhang. Ccpm: A chinese classical poetry matching dataset. arXiv preprint arXiv:2106.01979, 2021.

[78] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024a.

[79] A. Liu, J. Hayase, V . Hofmann, S. Oh, N. A. Smith, and Y. Choi. SuperBPE: Space travel for language models. In Second Conference on Language Modeling, 2025a. URL https: //openreview.net/forum?id=lcDRvffeNP .

[80] J. Liu, S. Min, L. Zettlemoyer, Y. Choi, and H. Hajishirzi. Infini-gram: Scaling unbounded n-gram language models to a trillion tokens. In First Conference on Language Modeling, 2024b. URL https://openreview.net/forum?id=u2vAyMeLMm . J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, et al. Muon is scalable for llm training. arXiv preprint arXiv:2502.16982, 2025b.

[81] S. Liu, C. Gao, Y. Chen, D. Jin, and Y. Li. Learnable embedding sizes for recommender systems. arXiv preprint arXiv:2101.07577, 2021.

[82] A. Mallen, A. Asai, V . Zhong, R. Das, D. Khashabi, and H. Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 9802–9822, 2023.

[83] K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. Advances in neural information processing systems, 35:17359–17372, 2022.

[84] K. Meng, A. S. Sharma, A. J. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.n et/forum?id=MkbcAHIYgyS .

[85] T. Nguyen. Understanding transformers via n-gram statistics. Advances in neural information processing systems, 37:98049–98082, 2024.

[86] nostalgebraist. interpreting gpt: the logit lens. LessWrong, 2020. URL https://www.lesswr ong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens .

[87] B. A. Olshausen and D. J. Field. Sparse coding with an overcomplete basis set: A strategy employed by v1? Vision research, 37(23):3311–3325, 1997.

[88] A. Pagnoni, R. Pasunuru, P . Rodriguez, J. Nguyen, B. Muller, M. Li, C. Zhou, L. Yu, J. E. Weston, L. Zettlemoyer, et al. Byte latent transformer: Patches scale better than tokens. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 9238–9258, 2025.

[89] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, S. Biderman, H. Cao, X. Cheng, M. Chung, L. Derczynski, X. Du, M. Grella, K. K. GV , X. He, H. Hou, P . Kazienko, J. Kocon, J. Kong, B. Koptyra, H. Lau, J. Lin, K. S. I. Mantri, F. Mom, A. Saito, G. Song, X. Tang, J. S. Wind, S. Wozniak, Z. Zhang, Q. Zhou, J. Zhu, and R. Zhu. RWKV: reinventing rnns for the transformer era. In H. Bouamor, J. Pino, and K. Bali, editors, Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023, pages 14048– 14077. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.FINDINGS-E MNLP.936. URL https://doi.org/10.18653/v1/2023.findings-emnlp.936 .

[90] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openre view.net/forum?id=wHBfxhZu1u .

[91] S. T. Piantadosi. Zipf's word frequency law in natural language: A critical review and future directions. Psychonomic bulletin & review, 21(5):1112–1130, 2014.

[92] O. Press, N. A. Smith, and M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022. OpenReview.net, 2022. URL https://openreview.net/forum?id=R8sQPpGCv0 . S. Rajput, N. Mehta, A. Singh, R. Hulikal Keshavan, T. Vu, L. Heldt, L. Hong, Y. Tay, V . Tran, J. Samost, et al. Recommender systems with generative retrieval. Advances in Neural Information Processing Systems, 36:10299–10315, 2023.

[93] A. Roy, R. Anil, G. Lai, B. Lee, J. Zhao, S. Zhang, S. Wang, Y. Zhang, S. Wu, R. Swavely, et al. N-grammer: Augmenting transformers with latent n-grams. arXiv preprint arXiv:2207.06366, 2022.

[94] RWKV Team. Rwkv architecture history. https://wiki.rwkv.com/basic/architecture .html , 2025. Section "RWKV-V8's DeepEmbed", accessed 2025-12-09.

[95] R. Sadhukhan, S. Cao, H. Dong, C. Zhao, A. Purpura-Pontoniere, Y. Tian, Z. Liu, and B. Chen. Stem: Scaling transformers with embedding modules. arXiv preprint arXiv:2601.10639, 2026.

[96] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale. Communications of the ACM, 64(9):99–106, 2021.

[97] C. E. Shannon. A mathematical theory of communication. The Bell system technical journal, 27 (3):379–423, 1948.

[98] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538, 2017.

[99] F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning

Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?id=fR3wGCk-IXp .

[100] H.-J. M. Shi, D. Mudigere, M. Naumov, and J. Yang. Compositional embeddings using complementary partitions for memory-efficient recommendation systems. In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pages 165–175, 2020.

[101] A. Singh, T. Vu, N. Mehta, R. Keshavan, M. Sathiamoorthy, Y. Zheng, L. Hong, L. Heldt, L. Wei, D. Tandon, et al. Better generalization with semantic ids: A case study in ranking for recommendations. In Proceedings of the 18th ACM Conference on Recommender Systems, pages 1039–1044, 2024.

[102] J. Su, M. H. M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024. doi: 10.1016/J.NEUCOM.2 023.127063. URL https://doi.org/10.1016/j.neucom.2023.127063 .

[103] K. Sun, D. Yu, D. Yu, and C. Cardie. Investigating prior knowledge for challenging chinese machine reading comprehension. Transactions of the Association for Computational Linguistics, 8:141–155, 2020.

[104] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. Le, E. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In Findings of the Association for Computational Linguistics: ACL 2023, pages 13003–13051, 2023. C. Szegedy, W. Liu, Y. Jia, P . Sermanet, S. Reed, D. Anguelov, D. Erhan, V . Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 1–9, 2015.

[105] G. Team. Gemma 3n. 2025. URL https://ai.google.dev/gemma/docs/gemma-3n .

[106] I. Tenney, D. Das, and E. Pavlick. Bert rediscovers the classical nlp pipeline. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics, 2019.

[107] D. Tito Svenstrup, J. Hansen, and O. Winther. Hash embeddings for efficient word representations. Advances in neural information processing systems, 30, 2017.

[108] H. L.-H. Tsang and T. D. Ahle. Clustering the sketch: A novel approach to embedding table compression. arXiv preprint arXiv:2210.05974, 2022.

[109] A. Tseng and C. D. Sa. L : Large lookup layers, 2026. URL https://arxiv.org/abs/2601.2 1461 .

[110] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[111] B. Wang, W. Ping, P . Xu, L. McAfee, Z. Liu, M. Shoeybi, Y. Dong, O. Kuchaiev, B. Li, C. Xiao, et al. Shall we pretrain autoregressive language models with retrieval? a comprehensive study. In Proceedings of the 2023 conference on empirical methods in natural language processing, pages 7763–7786, 2023.

[112] L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts, 2024a. URL https://arxiv.org/abs/2408.15664 .

[113] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. Advances in Neural Information Processing Systems, 37:95266–95290, 2024b.

[114] K. Whistler. Unicode standard annex #15: Unicode normalization forms. Unicode Standard Annex 15, The Unicode Consortium, July 2025. URL https://www.unicode.org/report s/tr15/tr15-57.html . Version Unicode 17.0.0, Revision 57. Accessed 2026-01-04.

[115] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openreview .net/forum?id=NG7sS51zVF .

[116] Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifold-constrained hyper-connections, 2025. URL https://arxiv.org/abs/2512.24880 .

[117] S. Yang, Y. Shen, K. Wen, S. Tan, M. Mishra, L. Ren, R. Panda, and Y. Kim. Path attention: Position encoding via accumulating householder transformations. arXiv preprint arXiv:2505.16381, 2025.

[118] C. Yin, B. Acun, C.-J. Wu, and X. Liu. Tt-rec: Tensor train compression for deep learning recommendation models. Proceedings of Machine Learning and Systems, 3:448–462, 2021. D. Yu, E. Cohen, B. Ghazi, Y. Huang, P . Kamath, R. Kumar, D. Liu, and C. Zhang. Scaling embedding layers in language models. arXiv preprint arXiv:2502.01637, 2025.

[119] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. Hellaswag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/V1/P19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[120] B. Zhang and R. Sennrich. Root mean square layer normalization. Advances in neural information processing systems, 32, 2019.

[121] C. Zhang, Y. Liu, Y. Xie, S. I. Ktena, A. Tejani, A. Gupta, P . K. Myana, D. Dilipkumar, S. Paul, I. Ihara, et al. Model size reduction using frequency based double hashing for recommender systems. In Proceedings of the 14th ACM Conference on Recommender Systems, pages 521– 526, 2020.

[122] H. Zhang, Z. Liu, B. Chen, Y. Zhao, T. Zhao, T. Yang, and B. Cui. Cafe: Towards compact, adaptive, and fast embedding for large-scale recommendation models. Proceedings of the ACM on Management of Data, 2(1):1–28, 2024.

[123] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. Agieval: A human-centric benchmark for evaluating foundation models. In Findings of the Association for Computational Linguistics: NAACL 2024, pages 2299–2314, 2024.

[124] D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyperconnections. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL https://openreview.net /forum?id=9FqARW7dwB .

DeepSeek-OCR 2：视觉因果流

DeepSeek-OCR 2: Visual Causal Flow Haoran Wei, Yaofeng Sun, Yukun Li

DeepSeek-AI research@deepseek.com

arXiv:2601.20552 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-OCR 2，以探索一种新型编码器——DeepEncoder V2——的可行性，该编码器能够依据图像语义动态地重排视觉 token。传统的视觉语言模型（VLM）在将视觉 token 输入大语言模型时，无一例外地按固定的光栅扫描顺序（从左上到右下）处理，并使用固定的位置编码。然而，这与人类的视觉感知相矛盾：人类视觉遵循由内在逻辑结构驱动的、灵活而语义连贯的扫描模式。尤其对于版式复杂的图像，人类视觉会呈现受因果引导的序贯处理。受这一认知机制启发，DeepEncoder V2 旨在赋予编码器因果推理能力，使其能够在由 LLM 进行内容解读之前智能地重排视觉 token。本工作探索了一种新范式：能否通过两个级联的 1D 因果推理结构有效地实现 2D 图像理解，从而提供一种有望实现真正 2D 推理的新架构路径。代码与模型权重已公开于 http://github.com/deepseek-ai/DeepSeek-OCR-2。

【图表】图 1 我们将 DeepEncoder 中的 CLIP 组件替换为 LLM 风格的架构。通过定制注意力掩码，视觉 token 采用双向注意力，而可学习查询采用因果注意力。因此每个查询 token 都能关注全部视觉 token 及其之前的查询，从而对视觉信息实现渐进式的因果重排（原文第 1 页）

1 引言

人类视觉系统与基于 Transformer 的视觉编码器 [14, 16] 高度相似：中央凹注视（foveal fixations）发挥类似视觉 token 的作用，局部清晰而具备全局感知。然而，与现有编码器严格地从左上到右下扫描 token 不同，人类视觉遵循一种由语义理解引导的、因果驱动的流动。不妨想象沿着螺旋

线描画的过程——我们的眼动遵循内在逻辑，每一次后续注视都因果地依赖于之前的注视。以此类推，模型中的视觉 token 应当被有选择地处理，其顺序应高度取决于视觉语义，而非空间坐标。

这一洞见促使我们从根本上重新思考视觉-语言模型（VLM）的架构设计，尤其是编码器部分。LLM 本质上是在一维序列数据上训练的，而图像是二维结构。按预定义的栅格扫描顺序直接展平图像块，会引入无根据的归纳偏置，从而忽视语义关系。为解决这一问题，我们提出 DeepSeek-OCR 2，并设计了一种新颖的编码器——DeepEncoder V2——以迈向更接近人类的视觉编码。沿用 DeepSeek-OCR [54]，我们以文档阅读作为主要实验平台。文档带来了丰富的挑战，包括复杂的版面顺序、繁复的公式与表格。这些结构化元素本身承载着因果式的视觉逻辑，要求复杂的推理能力，因而使文档 OCR 成为验证我们方法的理想平台。

我们的主要贡献有三方面：

第一，我们提出 DeepEncoder V2，它包含若干关键创新：（1）我们将 DeepEncoder [54] 中的 CLIP [37] 组件替换为紧凑的 LLM [48] 架构（如图 1 所示），以实现视觉因果流；（2）为支持并行化处理，我们引入可学习查询 [10]（称为因果流 token），并将视觉 token 作为前缀置于其前——通过定制的注意力掩码，视觉 token 保持全局感受野，而因果流 token 能够获得对视觉 token 的重排序能力；（3）我们保持因果 token 与视觉 token 数量相等（并加入填充与边界等冗余），从而为重新注视（re-fixation）提供充足的容量；（4）只有因果流 token——即编码器输出的后半部分——被送入 LLM [24] 解码器，从而实现级联的、具备因果感知的视觉理解。

第二，借助 DeepEncoder V2，我们提出 DeepSeek-OCR 2，它在保持 DeepSeek-OCR 的图像压缩率与解码效率的同时取得了显著的性能提升。我们将送入 LLM 的视觉 token 数量限制在 256 至 1120 之间。下界（256）对应 DeepSeek-OCR 对 1024×1024 图像的 token 化结果，上界（1120）则与 Gemini-3 pro [44] 的最大视觉 token 预算相匹配。这一设计使 DeepSeek-OCR 2 既可作为用于研究探索的新型 VLM 架构，也可作为为 LLM 预训练生成高质量训练数据的实用工具。

最后，我们为将语言模型架构用作 VLM 编码器提供了初步验证——这是一条通往统一全模态编码的有前景路径。该框架只需配置各模态专属的可学习查询，即可在多种模态（图像、音频、文本 [28]）上完成特征提取与 token 压缩。关键在于，它天然地能够承接 LLM 社区的先进基础设施优化，包括混合专家（MoE）架构、高效注意力机制 [26] 等。

综上，我们为 DeepSeek-OCR 2 提出 DeepEncoder V2，采用专门的注意力机制来有效建模文档阅读的视觉因果流。相比 DeepSeek-OCR 基线，DeepSeek-OCR 2 在 OmniDocBench v1.5 [34] 上取得 3.73% 的性能提升，并在视觉阅读逻辑方面取得可观的进展。

【图表】图 2 该图展示了两个采用并行化查询的计算机视觉模型：用于目标检测的 DETR 解码器 [10]，以及用于视觉 token 压缩的 BLIP2 Q-former [22]。二者都在查询之间采用双向自注意力（原文第 4 页）

2 相关工作

2.1 解码器中的并行化查询

DETR [10] 率先将 Transformer 架构引入目标检测，从根本上摆脱了传统检测范式 [38, 40]。为克服 Transformer 块中串行解码的效率限制，DETR 引入了预设的并行化可学习查询——一组 100 个目标查询，通过训练编码形状与位置等目标先验。这些查询通过交叉注意力机制与特征图 [18] 交互，同时通过自注意力在彼此之间进行双向信息交换。DETR 奠定了一种基础范式，使 Transformer 能够处理并行化的 token。此后，目标查询设计已成为后续基于 Transformer 的检测方法 [29, 56] 中事实上的标准架构组件。

2.2 投影器中的并行化查询

近年来，视觉-语言模型 [7, 9, 22, 50] 发展迅速，架构逐渐趋同于“编码器-投影器-LLM”范式。投影器将视觉 token 与 LLM 的嵌入空间对齐，是使 LLM 理解视觉内容的关键桥梁。BLIP-2 [22] 中提出的 Q-former 就是一个有效的投影器设计，它利用可学习查询进行视觉 token 压缩。Q-former 采用类 BERT [15] 架构，并从 DETR 的目标查询 [10] 中汲取灵感，使用 32 个可学习查询，通过交叉注意力与数百个 CLIP [37] 视觉 token 交互。这些压缩后的查询表示随后被送入 LLM，实现了从视觉空间到语言空间的有效映射。Q-former 的成功表明，并行化可学习查询不仅对检测任务中的特征解码有效，对多模态对齐中的 token 压缩同样有效。

2.3 基于 LLM 的多模态初始化

在大规模互联网数据上训练的 LLM 已被证明可作为多模态模型的有效初始化。Pang et al. [35] 表明，冻结的 LLM Transformer 层能够增强视觉判别任务。此外，视觉领域的 Fuyu [5] 与 Chameleon [43] 等无编码器或轻量编码器模型，以及语音领域的 VALL-E [47]，进一步验证了 LLM 预训练权重用于多模态初始化的潜力。

【图表】图 3 DeepSeek-OCR 2 采用 DeepEncoder 的视觉 token 压缩机制，使用 80M 参数的图像压缩器将视觉 token

减少 16 倍。DeepEncoder V2 的不同之处在于，用一个紧凑的语言模型架构替换了 DeepEncoder 的 CLIP 模块。通过定制的注意力掩码，这种 LM 风格的视觉编码器获得了 CLIP 的知识压缩能力，同时启动了视觉序列的因果建模（原文第 5 页）

3 方法

3.1 架构

如图 3 所示，DeepSeek-OCR 2 继承了 DeepSeek-OCR 的整体架构，由编码器与解码器组成。编码器将图像离散化为视觉 token，解码器则以这些视觉 token 与文本 prompt 为条件生成输出。关键区别在于编码器：我们将 DeepEncoder 升级为 DeepEncoder V2，它保留了前代的全部能力，同时通过一种全新的架构设计引入因果推理。我们将在后续章节详细阐述 DeepSeek-OCR 2 的细节。

3.2 DeepEncoder V2

标准编码器（vanilla encoder）是一个重要组件，它通过注意力机制提取并压缩图像特征，其中每个 token 关注所有其他 token，从而实现类似人类中央凹与周边视觉的全图感受野。然而，将二维图像 patch 展平为一维序列，会通过面向文本的位置编码（例如 RoPE [42]）强加一种僵化的顺序偏置。这与自然的视觉阅读模式相矛盾，在光学文本、表单与表格中的非线性布局中尤其如此。

3.2.1 视觉 tokenizer

DeepEncoder V2 的第一个组件是视觉 tokenizer。遵循 DeepEncoder 的做法，我们采用一种结合 80M 参数 SAM-base [21] 与两个卷积层 [50] 的架构。最后一个卷积层的输出维度从 DeepEncoder 中的 1024 降至 896，以与后续流程对齐。需要注意的是，这种基于压缩的 tokenizer 并非必需，可以替换为简单的 patch embedding。我们保留它，是因为它以极少的参数通过窗口注意力实现了 16× 的 token 压缩 [19, 46, 51, 52]，显著降低了后续全局注意力模块的计算成本与激活内存。此外，它的参数量（约 80M）与 LLM 中文本输入嵌入通常使用的 100M 参数相当。

【图表】图 4 DeepEncoder V2 的 token 数量计算。DeepEncoder V2 采用包含 0–6 个局部视图的多裁剪策略，每张图像输出 256–1120 个 token。当局部视图数为 0 时，仅全局视图产生 256 个 token；当有 6 个局部视图时，数量达到 1120（6×144+256）（原文第 6 页）

3.2.2 语言模型作为视觉编码器

在 DeepEncoder 中，CLIP ViT 紧随视觉 tokenizer 之后，用于压缩视觉知识。DeepEncoder V2 将该组件重新设计为具有双流注意力机制的 LLM 风格架构。视觉 token 采用双向注意力，以保留 CLIP 的全局建模能力；而新引入的因果流查询（causal flow queries）则采用因果注意力。这些可学习查询作为后缀追加在视觉 token 之后，其中每个查询关注所有视觉 token 以及此前的查询。通过保持查询数量与视觉 token 数量相等，这种设计在不改变 token 数量的前提下，对视觉特征施加语义排序与蒸馏。最终，只有因果查询的输出会被送入 LLM 解码器。

我们使用 Qwen2-0.5B [48] 实例化该架构，其 500M 参数与 CLIP ViT（300M）相当，且不会引入过高的计算开销。采用视觉 token 前缀拼接的 decoder-only 架构被证明至关重要：在 mBART 风格 [30] 的编码器-解码器结构中使用交叉注意力的额外实验无法收敛。我们推测，这一失败源于视觉 token 在被隔离到单独编码器时交互不足。相反，前缀设计使视觉 token 在所有层中都保持活跃，从而与因果查询形成有效的视觉信息交换。

该架构实际上建立了两级级联因果推理：编码器通过可学习查询在语义上重排视觉 token，而 LLM 解码器则对有序序列执行自回归推理。与通过位置编码强加僵化空间顺序的标准编码器不同，我们的因果有序查询能够适应平滑的视觉语义，同时自然地与 LLM 的单向注意力模式对齐。这一设计或许能够弥合二维空间结构与一维因果语言建模之间的鸿沟。

3.2.3 因果流查询

如前所述，因果查询 token 的数量等于视觉 token 的数量，计算方式为 $W\times H/(16^2\times 16)$，其中 W 与 H 分别表示输入编码器的图像宽度与高度。为避免为不同分辨率维护多套查询集合，我们采用多裁剪策略，并在预定义分辨率下使用固定的查询配置。

具体而言，全局视图使用 1024×1024 的分辨率，对应 256 个查询嵌入，记为 query_global。局部裁剪采用 768×768 的分辨率，裁剪数量 k 的取值范围为 0 到 6（当图像的宽度与高度都小于 768 时不进行裁剪）。所有局部视图共享一套统一的 144 个查询嵌入，记为 query_local。因此，送入 LLM 的重排后视觉 token 总数为 k×144+256，范围为 [256, 1120]。这一最大 token 数量（1120）低于 DeepSeek-OCR 的 1156（Gundam 模式），并与 Gemini-3-Pro 的最大视觉 token 预算相当。

【图表】图 5 DeepEncoder V2 的注意力掩码架构。双向掩码（视觉 token，类 ViT）与因果三角掩码（流 token，LLM 解码器风格）的拼接（原文第 7 页）

3.2.4 注意力掩码

为更清晰地说明 DeepEncoder V2 的注意力机制，我们在图 5 中可视化了注意力掩码。该注意力掩码由两个不同的区域组成。左侧区域对原始视觉 token 施加双向注意力（类似 ViT），允许任意 token 之间完全可见。右侧区域对因果流 token 采用因果注意力（三角掩码，与 decoder-only LLM 相同），其中每个 token 只关注此前的 token。这两个部分沿序列维度拼接，构成 DeepEncoder V2 的注意力掩码 M，如下所示：

M = [1_{m×m}　0_{m×n}；1_{n×m}　LowerTri(n)]，其中 n = m　　（1）

其中 n 是因果查询 token 的数量，m 表示标准视觉 token 的数量，LowerTri 表示下三角矩阵（对角线及其下方为 1，上方为 0）。

3.3 DeepSeek-MoE 解码器

由于 DeepSeek-OCR 2 主要聚焦于编码器改进，我们没有升级解码器组件。遵循这一设计原则，我们保留 DeepSeek-OCR 的解码器——一个具有约 500M 激活参数的 3B 参数 MoE 结构。DeepSeek-OCR 2 的核心前向过程可以形式化为：

O = D(π_Q(T_L(E(I) ⊕ Q_0; M)))　　（2）

其中 I ∈ R^{H×W×3} 是输入图像，E 是将图像映射为 m 个视觉 token V ∈ R^{m×d} 的视觉 tokenizer，Q_0 ∈ R^{n×d} 是可学习的因果查询嵌入，⊕ 表示序列拼接，T_L 表示具有掩码注意力的 L 层 Transformer，M ∈ {0,1}^{2n×2n} 是公式（1）中定义的分块因果注意力掩码，π_Q 是提取最后 n 个 token 的投影算子（即 Z = X_{m+1:m+n}），D 是语言解码器，O ∈ R^{n×|V|} 是 LLM 词表上的输出 logits。

4 实验设置

4.1 数据引擎

DeepSeek-OCR 2 采用与 DeepSeek-OCR 相同的数据源，包括 OCR 1.0、OCR 2.0 [11, 27, 53] 与通用视觉数据 [54]，其中 OCR 数据占训练混合的 80%。我们还引入两项修改：（1）对 OCR 1.0 数据采用更均衡的采样策略，按内容类型（文本、公式、表格）对页面进行划分，比例为 3:1:1；（2）通过合并语义相近的类别（例如统一“图注”与“图题”）来精炼版面检测标签。鉴于这些差异很小，我们认为 DeepSeek-OCR 是有效的对比基线。

4.2 训练流程

我们分三个阶段训练 DeepSeek-OCR 2：（1）编码器预训练；（2）查询增强；（3）解码器专门化。第一阶段使视觉分词器与 LLM 式编码器掌握特征提取、token 压缩与 token 重排序等基础能力。第二阶段进一步增强编码器的 token 重排序能力，同时增强视觉知识压缩。第三阶段冻结编码器参数，仅优化解码器，从而在相同 FLOPs 下实现更高的数据吞吐。

4.2.1 训练 DeepEncoder V2

参照 DeepSeek-OCR 与 Vary [50]，我们以语言建模目标训练 DeepEncoder V2，将编码器与一个轻量级解码器 [20] 耦合，通过下一 token 预测进行联合优化。我们使用两个分别处理 768×768 与 1024×1024 分辨率的数据加载器。视觉分词器由 DeepEncoder 初始化，LLM 式编码器由 Qwen2-0.5B-base [48] 初始化。预训练后，仅保留编码器参数用于后续阶段。我们使用 AdamW [32] 优化器，学习率采用从 1e-4 到 1e-6 的余弦衰减，在 160 块 A100 GPU（20 个节点 × 8 块 GPU）上训练，批大小为 640，共 40k 次迭代（序列打包长度为 8K，约 1 亿个图文对样本）。

4.2.2 查询增强

在 DeepEncoder V2 预训练后，我们将其与 DeepSeek-3B-A500M [24, 25] 集成，作为最终流程。我们冻结视觉分词器（SAM-conv 结构），同时联合优化 LLM 编码器与 LLM 解码器，以增强查询表示。在该阶段，我们通过多裁剪策略将两种分辨率统一到单个数据加载器中。我们采用 4 阶段流水线并行：视觉分词器（PP0）、LLM 式编码器（PP1）以及 DeepSeek-LLM 各层（在 PP2-3 上每阶段 6 层）。使用 160 块 GPU（每块 40GB），我们配置 40 个数据并行副本（每个副本 4 块 GPU），以全局批大小 1280 进行训练，使用相同的优化器以及从 5e-5 到 1e-6 的学习率衰减，共训练 15k 次迭代。

4.2.3 继续训练 LLM

为了快速消耗训练数据，我们在该阶段冻结所有 DeepEncoder V2 参数，仅更新 DeepSeek-LLM 参数。该阶段加速了训练（在相同全局批大小下训练速度提升一倍以上），同时帮助 LLM 更好地理解 DeepEncoder V2 重排序后的视觉 token。从第二阶段继续，我们在该阶段再进行一次从 1e-6 到 5e-8 的学习率衰减，训练 20k 次迭代。

5 评估

我们选择 OmniDocBench v1.5 [34] 作为评估的主要基准。该基准包含 1,355 个文档页面，涵盖 9 个主要类别（包括杂志、学术论文、研究报告等），中英文皆有。凭借多样化的测试样本与稳健的评估标准，OmniDocBench 为验证 DeepSeek-OCR 2 的性能、尤其是 DeepEncoder V2 的有效性，提供了有效的框架。

5.1 主要结果

如表 1 所示，DeepSeek-OCR 2 取得了 91.09% 的领先性能，同时使用的视觉 token 上限（V-token_max）最小。与 DeepSeek-OCR 基线相比，在相似的训练数据来源下取得了 3.73% 的提升，验证了我们新设计架构的有效性。除整体提升外，阅读顺序（R-order）的编辑距离（ED）也显著下降（从 0.085 降至 0.057），表明新的 DeepEncoder V2 能够根据图像信息有效地选择与排列初始视觉 token。如表 2 所示，DeepSeek-OCR 2（0.100）在文档解析中取得了比 Gemini-3 Pro（0.115）更低的 ED，且视觉 token 预算相近（1120），进一步表明我们的新模型在保持视觉 token 高压缩率的同时确保了更优性能，具有极高的潜力。

【表格】表 1 在 OmniDocBench v1.5 上对文档阅读的全面评估。V-token_max 表示该基准中每页使用的最大视觉 token 数。R-order 表示阅读顺序。除 DeepSeek-OCR 与 DeepSeek-OCR 2 外，表中其他模型的结果均来自 OmniDocBench 仓库。（原文第 9 页）

模型	V-token_max↓	Text_Edit↓	Formula_Edit↓	Table_Edit↓	R-order_Edit↓	Overall_Edit↓
Gemini-3 pro [44]	1120	-	-	-	-	0.115
Seed-1.8 [41]	5120	-	-	-	-	0.106
DeepSeek-OCR	1156	0.073	0.236	0.123	0.085	0.129
DeepSeek-OCR 2	1120	0.048	0.198	0.096	0.057	0.100

表 2 OmniDocBench v1.5 中不同文档元素类别的编辑距离。V-token_max 表示最低的最大视觉 token 数。

5.2 改进空间

我们对 DeepSeek-OCR 与 DeepSeek-OCR 2 在 9 种文档类型上进行了详细的性能比较，发现 DeepSeek-OCR 2 仍有相当大的改进空间，如表 3 所示。在文本识别编辑距离（ED）方面，DeepSeek-OCR 2 在大多数情况下优于 DeepSeek-OCR，但也存在明显弱点，例如报纸，其表现 >0.13 ED。我们认为主要有两个原因：（1）较低的视觉 token 上限可能影响对文本极其丰富的报纸的识别，未来可以通过增加局部裁剪数量来简单解决；（2）报纸数据不足——我们的训练数据仅包含 25 万个相关样本，不足以针对该类训练 DeepEncoder V2。当然，在阅读顺序（R-order）指标上，DeepSeek-OCR 2 全面优于 DeepSeek-OCR，进一步验证了我们视觉因果流编码器设计的有效性。

模型	指标	PPT	AcademicPaper	Book	ColorfulTextbook	ExamPaper	Magazine	Newspaper	Note	ResearchReport
DS-OCR	Text	0.052	0.028	0.022	0.130	0.074	0.049	0.131	0.145	0.015
DS-OCR	R-order	0.052	0.021	0.040	0.125	0.083	0.101	0.217	0.089	0.016
DS-OCR 2	Text	0.031	0.013	0.033	0.053	0.047	0.026	0.139	0.068	0.008
DS-OCR 2	R-order	0.025	0.013	0.027	0.066	0.048	0.100	0.176	0.035	0.011

表 3 DeepSeek-OCR 2 与 DeepSeek-OCR 在 9 种文档类型上的详细比较。R-order 表示阅读顺序。所有指标均为编辑距离，越低越好。

5.3 实用就绪度

DeepSeek-OCR 服务于两个主要生产用例：为 DeepSeek-LLM 读取图像/文档的在线 OCR 服务，以及执行批量 PDF 处理的预训练数据流水线。我们比较了 DeepSeek-OCR 2 与 DeepSeek-OCR 的生产性能。由于生产环境中无法获得真值，我们主要关注重复率这一关键指标。如表 4 所示，与其前代（DeepSeek-OCR）相比，DeepSeek-OCR 2 显著提升了实用就绪度：在线用户日志图像的重复率从 6.25% 降至 4.17%，PDF 数据生产的重复率从 3.69% 降至 2.88%。这些结果进一步验证了 DeepSeek-OCR 2 架构的有效性，尤其是其逻辑视觉理解能力。

模型	指标	online-user-logs（image）	pretrain-data（PDF）
DeepSeek-OCR	Repeat↓	6.25%	3.69%

DeepSeek-OCR 2	Repeat↓	4.17%（↓2.08%）	2.88%（↓0.81%）

表 4 DeepSeek-OCR 与 DeepSeek-OCR 2 的生产性能比较。对于服务于 LLM 流水线的 OCR 模型，生产环境中无法获取真值。因此，重复率构成主要可观测的质量指标。

6 讨论与未来工作

6.1 迈向真正的 2D 推理

DeepSeek-OCR 2 提出了一种新颖的架构范式：LLM 式编码器级联 LLM 解码器。这种由两个一维因果推理器组成的级联，有望实现真正的 2D 推理：编码器执行阅读逻辑推理（通过查询 token 对视觉信息进行因果重排序），而解码器在这些因果有序的表示上执行视觉任务推理。将 2D 理解分解为两个互补/正交的 1D 因果推理子任务，可能代表着迈向真正 2D 推理的突破。

当然，实现这一目标仍是一段漫长的旅程。例如，为了实现对视觉内容的多次重新检查与多跳重排序，我们可能需要比原始视觉 token 序列长得多的因果流 token。我们将在未来工作中继续改进这一架构，并探索其在通用视觉推理任务上的有效性。

6.2 迈向原生多模态

DeepEncoder V2 初步验证了 LLM 式编码器在视觉任务上的可行性。更重要的是，这一架构有潜力演进为一个统一的 omni-modal 编码器：单个编码器配备共享的 W_k、W_v 投影、注意力机制与 FFN，可以通过模态特定的可学习查询嵌入来处理多种模态。这样的编码器可以在同一参数空间内压缩文本、提取语音特征并重组视觉内容，区别仅在于其查询嵌入所学习到的权重。

DeepSeek-OCR 的光学压缩代表着迈向原生多模态的初步探索，而我们相信 DeepSeek-OCR 2 的 LLM 式编码器架构标志着我们朝这一方向迈出的又一步。未来我们还将继续探索通过这一共享编码器框架整合更多模态。

7 结论

在本技术报告中，我们提出了 DeepSeek-OCR 2——DeepSeek-OCR 的一次重大升级，它在保持高视觉 token 压缩率的同时实现了有意义的性能提升。这一进步由新提出的 DeepEncoder V2 驱动，它通

过整合双向与因果注意力机制，隐式地蒸馏出对视觉世界的因果理解，从而在视觉编码器中带来因果推理能力，并显著提升视觉阅读逻辑。

尽管光学文本阅读（尤其是文档解析）是 LLM 时代最实用的视觉任务之一，但它只是更广泛视觉理解图景的一小部分。展望未来，我们将改进并调整这一架构以适应更多样化的场景，朝着更全面的多模态智能愿景深入探索。

附录

附录：补充材料见英文原文

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] [1] Marker. URL https://github.com/datalab-to/marker .

[2] [2] Nanonets-ocr-s, 2025. URL https://huggingface.co/nanonets/Nanonets-OCR-s .

[3] [3] Ocrverse, 2025. URL https://github.com/DocTron-hub/OCRVerse .

[4] [4] Ocrflux, 2025. URL https://github.com/chatdoc-com/OCRFlux .

[5] [5] Adept. Fuyu-8b. https://huggingface.co/adept/fuyu-8b , 2023.

[6] [6] G. AI. Gemini 2.5-pro, 2025. URL https://gemini.google.com/ .

[7] [7] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P . Wang, J. Lin, C. Zhou, and J. Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond. arXiv preprint arXiv:2308.12966, 2023.

[8] [8] S. Bai, Y. Cai, R. Chen, et al. Qwen3-vl technical report. arXiv preprint arXiv:2511.21631, 2025. URL https://arxiv.org/abs/2511.21631 .

[9] [9] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P . Wang, S. Wang, J. Tang, H. Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, P . Wang, W. Ding, Z. Fu, Y. Xu, J. Ye, X. Zhang, T. Xie, Z. Cheng, H. Zhang, Z. Yang, H. Xu, and J. Lin. Qwen2.5-vl technical report. arXiv preprint arXiv:2502.13923, 2025.

[10] [10] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko. End-to-end object detection with transformers. In European conference on computer vision, pages 213–229. Springer, 2020.

[11] [11] J. Chen, L. Kong, H. Wei, C. Liu, Z. Ge, L. Zhao, J. Sun, C. Han, and X. Zhang. Onechart: Purify the chart structural extraction via one auxiliary token. In Proceedings of the 32nd ACM International Conference on Multimedia, pages 147–155, 2024.

[12] [12] C. Cui, T. Sun, S. Liang, et al. Paddleocr-vl: Boosting multilingual document parsing via a 0.9 b ultra-compact vision-language model. arXiv preprint arXiv:2510.14528, 2025.

[13] [13] C. Cui, T. Sun, M. Lin, T. Gao, Y. Zhang, J. Liu, X. Wang, Z. Zhang, C. Zhou, H. Liu, et al. Paddleocr 3.0 technical report. arXiv preprint arXiv:2507.05595, 2025.

[14] [14] M. Dehghani, J. Djolonga, B. Mustafa, P . Padlewski, J. Heek, J. Gilmer, A. Steiner, M. Caron, R. Geirhos, I. Alabdulmohsin, et al. Patch n' pack: Navit, a vision transformer for any aspect ratio and resolution. Advances in Neural Information Processing Systems, 36:3632–3656, 2023.

[15] [15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), pages 4171–4186, 2019.

[16] [16] A. Dosovitskiy. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929, 2020.

[17] [17] H. Feng, S. Wei, X. Fei, W. Shi, Y. Han, L. Liao, J. Lu, B. Wu, Q. Liu, C. Lin, et al. Dolphin: Document image parsing via heterogeneous anchor prompting. arXiv preprint arXiv:2505.14059, 2025. [18] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 770–778, 2016.

[18] [19] A. Huang, C. Yao, C. Han, F. Wan, H. Guo, H. Lv, H. Zhou, J. Wang, J. Zhou, J. Sun, et al. Step3-vl-10b technical report. arXiv preprint arXiv:2601.09668, 2026.

[19] [20] S. Iyer, X. V . Lin, R. Pasunuru, T. Mihaylov, D. Simig, P . Yu, K. Shuster, T. Wang, Q. Liu, P . S. Koura, et al. Opt-iml: Scaling language model instruction meta learning through the lens of generalization. arXiv preprint arXiv:2212.12017, 2022.

[20] [21] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, et al. Segment anything. arXiv preprint arXiv:2304.02643, 2023.

[21] [22] J. Li, D. Li, S. Savarese, and S. Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In International conference on machine learning, pages 19730–19742. PMLR, 2023.

[22] [23] Z. Li, Y. Liu, Q. Liu, Z. Ma, Z. Zhang, S. Zhang, Z. Guo, J. Zhang, X. Wang, and X. Bai. Monkeyocr: Document parsing with a structure-recognition-relation triplet paradigm. arXiv preprint arXiv:2506.05218, 2025.

[23] [24] A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Dengr, C. Ruan, D. Dai, D. Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434, 2024.

[24] [25] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, et al. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437, 2024.

[25] [26] A. Liu, A. Mei, B. Lin, B. Xue, B. Wang, B. Xu, B. Wu, B. Zhang, C. Lin, C. Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. arXiv preprint arXiv:2512.02556, 2025.

[26] [27] C. Liu, H. Wei, J. Chen, L. Kong, Z. Ge, Z. Zhu, L. Zhao, J. Sun, C. Han, and X. Zhang. Focus anywhere for fine-grained multi-page document understanding. arXiv preprint arXiv:2405.14295, 2024.

[27] [28] F. Liu and H. Qiu. Context cascade compression: Exploring the upper limits of text compression. arXiv preprint arXiv:2511.15244, 2025.

[28] [29] F. Liu, H. Wei, W. Zhao, G. Li, J. Peng, and Z. Li. Wb-detr: Transformer-based detector without backbone. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pages 2979–2987, 2021.

[29] [30] Y. Liu, J. Gu, N. Goyal, X. Li, S. Edunov, M. Ghazvininejad, M. Lewis, and L. Zettlemoyer. Multilingual denoising pre-training for neural machine translation. Transactions of the Association for Computational Linguistics, 8:726–742, 2020.

[30] [31] Y. Liu, Z. Zhao, L. Tian, et al. Points-reader: Distillation-free adaptation of vision-language models for document conversion. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 1576–1601, November 2025.

[31] ICLR, 2019. [32] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In

[32] [33] OpenAI. Gpt-4 technical report, 2023. [34] L. Ouyang, Y. Qu, H. Zhou, J. Zhu, R. Zhang, Q. Lin, B. Wang, Z. Zhao, M. Jiang, X. Zhao, et al. Omnidocbench: Benchmarking diverse pdf document parsing with comprehensive annotations. In Proceedings of the Computer Vision and Pattern Recognition Conference, pages 24838–24848, 2025.

[33] [35] Z. Pang, Z. Xie, Y. Man, and Y.-X. Wang. Frozen transformers in language models are effective visual encoder layers. arXiv preprint arXiv:2310.12973, 2023.

[34] [36] J. Poznanski, A. Rangapur, J. Borchardt, J. Dunkelberger, R. Huff, D. Lin, C. Wilhelm, K. Lo, and L. Soldaini. olmocr: Unlocking trillions of tokens in pdfs with vision language models. arXiv preprint arXiv:2502.18443, 2025.

[35] [37] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P . Mishkin, J. Clark, et al. Learning transferable visual models from natural language supervision. In International conference on machine learning, pages 8748–8763. PMLR, 2021.

[36] [38] J. Redmon and A. Farhadi. Yolo9000: better, faster, stronger. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 7263–7271, 2017.

[37] [39] Rednote. dots.ocr, 2025. URL https://github.com/rednote-hilab/dots.ocr .

[38] [40] S. Ren, K. He, R. Girshick, and J. Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. In Advances in neural information processing systems, pages 91–99, 2015.

[39] [41] B. Seed. Seed1. 8 model card: Towards generalized real-world agency, 2025.

[40] [42] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. arXiv preprint arXiv:2104.09864, 2021.

[41] [43] C. Team. Chameleon: Mixed-modal early-fusion foundation models. arXiv preprint arXiv:2405.09818, 2024.

[42] [44] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, et al. Gemini: a family of highly capable multimodal models. arXiv preprint arXiv:2312.11805, 2023.

[43] [45] B. Wang, C. Xu, X. Zhao, L. Ouyang, F. Wu, Z. Zhao, R. Xu, K. Liu, Y. Qu, F. Shang, et al. Mineru: An open-source solution for precise document content extraction. arXiv preprint arXiv:2409.18839, 2024.

[44] [46] B. Wang, B. Wang, C. Wan, G. Huang, H. Hu, H. Jia, H. Nie, M. Li, N. Chen, S. Chen, et al. Step-3 is large yet affordable: Model-system co-design for cost-effective decoding. arXiv preprint arXiv:2507.19427, 2025.

[45] [47] C. Wang, S. Chen, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li, et al. Neural codec language models are zero-shot text to speech synthesizers. arXiv preprint arXiv:2301.02111, 2023.

[46] [48] P . Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai, K. Chen, X. Liu, J. Wang, W. Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. arXiv preprint arXiv:2409.12191, 2024. [49] W. Wang, Z. Gao, L. Gu, et al. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. arXiv preprint arXiv:2508.18265, 2025.

[47] [50] H. Wei, L. Kong, J. Chen, L. Zhao, Z. Ge, J. Yang, J. Sun, C. Han, and X. Zhang. Vary: Scaling up the vision vocabulary for large vision-language model. In European Conference on Computer Vision, pages 408–424. Springer, 2024.

[48] [51] H. Wei, L. Kong, J. Chen, L. Zhao, Z. Ge, E. Yu, J. Sun, C. Han, and X. Zhang. Small language model meets with reinforced vision vocabulary. arXiv preprint arXiv:2401.12503, 2024.

[49] [52] H. Wei, C. Liu, J. Chen, J. Wang, L. Kong, Y. Xu, Z. Ge, L. Zhao, J. Sun, Y. Peng, et al. General ocr theory: Towards ocr-2.0 via a unified end-to-end model. arXiv preprint arXiv:2409.01704, 2024.

[50] [53] H. Wei, Y. Yin, Y. Li, J. Wang, L. Zhao, J. Sun, Z. Ge, X. Zhang, and D. Jiang. Slow perception: Let's perceive geometric figures step-by-step. arXiv preprint arXiv:2412.20631, 2024.

[51] [54] H. Wei, Y. Sun, and Y. Li. Deepseek-ocr: Contexts optical compression. arXiv preprint arXiv:2510.18234, 2025.

[52] [55] J. Zhu, W. Wang, Z. Chen, Z. Liu, S. Ye, L. Gu, H. Tian, Y. Duan, W. Su, J. Shao, et al. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. arXiv preprint arXiv:2504.10479, 2025.

[53] [56] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai. Deformable detr: Deformable transformers for end-to-end object detection. arXiv preprint arXiv:2010.04159, 2020.

DeepSeek-V4：迈向高效的百万 token 上下文智能

DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI research@deepseek.com

arXiv:2606.19348 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

我们提出 DeepSeek-V4 系列的预览版本，其中包括两个强大的混合专家（MoE）语言模型——拥有 1.6T 参数（激活 49B）的 DeepSeek-V4-Pro 与拥有 284B 参数（激活 13B）的 DeepSeek-V4-Flash——二者均支持一百万 token 的上下文长度。DeepSeek-V4 系列在架构与优化方面引入了若干关键升级：（1）一种混合注意力架构，将压缩稀疏注意力（Compressed Sparse Attention，CSA）与重压缩注意力（Heavily Compressed Attention，HCA）相结合，以提升长上下文效率；（2）流形约束超连接（Manifold-Constrained Hyper-Connections，mHC），用于增强传统残差连接；（3）以及 Muon 优化器，带来更快的收敛与更强的训练稳定性。我们在超过 32T 的多样化高质量 token 上对两个模型进行预训练，随后通过全面的后训练流水线释放并进一步增强其能力。DeepSeek-V4-Pro-Max 是 DeepSeek-V4-Pro 的最大推理强度模式，它重新定义了开源模型的最先进水平（state-of-the-art），在核心任务上超越其前代模型。同时，DeepSeek-V4 系列在长上下文场景中极为高效。在一百万 token 的上下文设置下，与 DeepSeek-V3.2 相比，DeepSeek-V4-Pro 仅需 27% 的单 token 推理 FLOPs 和 10% 的 KV cache。这使我们能够常规性地支持一百万 token 上下文，从而使长程任务与进一步的测试时扩展（test-time scaling）更加可行。模型 checkpoint 见 https://huggingface.co/collections/deepseek-ai/deepseek-v4。

1 引言

推理模型（DeepSeek-AI，2025；OpenAI，2024c）的出现确立了测试时扩展（test-time scaling）的新范式，为 Large Language Model（LLM）带来了显著的性能提升。然而，这一扩展范式从根本上受限于原始注意力机制（Vaswani 等，2017）的平方计算复杂度，这为超长上下文与推理过程造成了难以逾越的瓶颈。与此同时，长程（long-horizon）场景与任务的出现——从复杂的智能体工作流到大规模的跨文档分析——也使对超长上下文的高效支持成为未来进展的关键。尽管近期的开源工作（Bai 等，2025a；DeepSeek-AI，2024；MiniMax，2025；Qwen，2025）推进了通用能力，但在处理超长序列方面的这一核心架构低效仍是关键障碍，既限制了测试时扩展带来的进一步收益，也阻碍了对长程场景与任务的深入探索。

为打破超长上下文中的效率壁垒，我们开发了 DeepSeek-V4 系列，包括预览版本：拥有 1.6T 参数（49B 激活）的 DeepSeek-V4-Pro，以及拥有 284B 参数（13B 激活）的 DeepSeek-V4-Flash。通过架构创新，DeepSeek-V4 系列在处理超长序列时的计算效率实现了跨越式提升。这一突破使高效支持一百万 token 的上下文长度成为可能，为下一代 LLM 开启了百万长度上下文的新时代。我们相信，高效处理超长序列的能力开启了测试时扩展的下一个前沿，为长程任务的深入研究铺平了道路，并为探索在线学习等未来范式奠定了必要基础。

与 DeepSeek-V3 架构（DeepSeek-AI，2024）相比，DeepSeek-V4 系列保留了 DeepSeekMoE 框架（Dai 等，2024）与多 token 预测（MTP）策略，同时在架构与优化上引入了若干关键创新。为提升长上下文效率，我们设计了一种混合注意力机制，结合压缩稀疏注意力（CSA）与重度压缩注意力（HCA）。CSA 沿序列维度压缩 KV 缓存，然后执行 DeepSeek 稀疏注意力（DSA）（DeepSeek-AI，2025）；而 HCA 对 KV 缓存施加更激进的压缩，但保持稠密注意力。为增强建模能力，我们引入流形约束超连接（mHC）（Xie 等，2026），对传统残差连接进行升级。此外，我们将 Muon 优化器（Jordan 等，2024；Liu 等，2025）引入 DeepSeek-V4 系列的训练，从而实现更快的收敛与更好的训练稳定性。

为使 DeepSeek-V4 系列能够高效训练与推理并支持高生产力开发，我们引入了若干基础设施优化。第一，我们为 MoE 模块设计并实现了单一融合内核，将计算、通信与内存访问完全重叠。第二，我们采用 TileLang（Wang 等，2026）这一领域特定语言（DSL），以平衡开发效率与运行时效率。第三，我们提供高效的批次不变（batch-invariant）且确定性的内核库，以确保训练与推理之间的逐位可复现性。第四，在训练框架方面，我们扩展了自动微分框架，引入张量级检查点（tensor-level

checkpointing）以实现细粒度的重计算控制；并通过面向 Muon 优化器的混合 ZeRO 策略、基于重计算与融合内核的低成本 mHC 实现，以及用于管理压缩注意力的两阶段上下文并行，提升训练效率。第五，在推理框架方面，我们设计了异构 KV 缓存结构及磁盘存储策略，以实现高效的共享前缀复用。此外，在后训练阶段，我们对 MoE 专家权重与索引器 QK 路径引入 FP4 量化感知训练，以降低内存与计算开销。

通过采用混合 CSA 与 HCA，以及计算与存储上的精度优化，与 DeepSeek-V3.2 相比，DeepSeek-V4 系列实现了显著更低的推理 FLOPs 与大幅缩减的 KV 缓存，在长上下文场景中尤为明显。图 1 的右半部分展示了 DeepSeek-V3.2 与 DeepSeek-V4 系列估算的每 token 推理 FLOPs 与累积 KV 缓存大小。在 1M token 上下文的场景下，即便是激活参数更多的 DeepSeek-V4-Pro，其每 token FLOPs 也仅为 DeepSeek-V3.2 的 27%（按等效 FP8 FLOPs 计），KV 缓存大小仅为其 10%。此外，激活参数更少的 DeepSeek-V4-Flash 将效率推得更远：在 1M token 上下文设置下，其每 token FLOPs 仅为 DeepSeek-V3.2 的 10%，KV 缓存大小仅为其 7%。另外，DeepSeek-V4 系列的路由专家参数采用 FP4 精度。尽管在现有硬件上 FP4 × FP8 运算的峰值 FLOPs 目前与 FP8 × FP8 相同，但理论上它们在未来硬件上可实现 1/3 的额外效率，这将进一步提升 DeepSeek-V4 系列的效率。

在预训练阶段，我们分别用 32T token 训练 DeepSeek-V4-Flash、用 33T token 训练 DeepSeek-V4-Pro。预训练之后，这两个模型能够原生且高效地支持 1M 长度的上下文。在我们的内部评测中，凭借参数效率更高的设计，DeepSeek-V4-Flash-Base 已在大多数基准上超越 DeepSeek-V3.2-Base。DeepSeek-V4-Pro-Base 进一步扩大了这一优势，在 DeepSeek 基础模型中树立了新的性能标准，在推理、编码、长上下文与常识世界知识任务上实现全面领先。

DeepSeek-V4 系列的后训练流程采用两阶段范式：先独立培养各领域专家模型，再通过在线策略蒸馏（on-policy distillation）（Gu 等，2024；Lu 与 Lab，2025）进行统一模型整合。首先，针对每个目标领域——例如数学、编码、智能体与指令遵循——独立训练一个单独的专家模型。基座模型首先在高质量的领域特定数据上进行监督微调（SFT），以建立基础能力。随后，使用组相对策略优化（GRPO）（DeepSeek-AI，2025）进行强化学习（RL），在针对特定成功标准定制的奖励模型引导下，进一步优化模型的领域对齐行为。这一阶段产生了一组多样化的专家模型，每个专家在其各自领域表现卓越。最后，为整合这些不同的专长，通过在线策略蒸馏训练一个统一模型，其中统一模型作为学生，学习与教师模型一起优化反向 KL 损失。

核心评测结果概览

- **知识**：在广泛世界知识的评估中，DeepSeek-V4-Pro 的最大推理力度模式 DeepSeek-V4-Pro-Max 在 SimpleQA（OpenAI，2024d）与 Chinese-SimpleQA（He 等，2024）基准上显著优于领先的开源模型。在教育知识方面——通过 MMLU-Pro（Wang 等，2024b）、HLE（Phan 等，2025）与 GPQA（Rein 等，2023）评估——DeepSeek-V4-Pro-Max 相对开源同类模型仅有微弱领先。尽管在这些基于知识的评估中仍落后于领先的专有模型 Gemini-3.1-Pro，DeepSeek-V4-Pro-Max 已显著缩小了差距。
- **推理**：通过扩展推理 token，DeepSeek-V4-Pro-Max 在标准推理基准上相对 GPT-5.2 与 Gemini-3.0-Pro 展现出更优性能。然而，其表现仍略逊于 GPT-5.4 与 Gemini-3.1-Pro，表明其发展轨迹落后于最先进的前沿模型约 3 至 6 个月。此外，DeepSeek-V4-Flash-Max 在复杂推理任务上取得了与 GPT-5.2 和 Gemini-3.0-Pro 相当的性能，使其成为一种极具成本效益的复杂推理架构。

【图表】图 2 DeepSeek-V4 系列的整体架构。注意力层采用混合 CSA（压缩稀疏注意力）与 HCA（重度压缩注意力），前馈层采用 DeepSeekMoE，并用 mHC 强化传统残差连接。（原文第 6 页）

- **智能体**：在公开基准上，DeepSeek-V4-Pro-Max 与领先的开源模型（如 Kimi-K2.6 与 GLM-5.1）相当，但略逊于前沿闭源模型。在我们的内部评测中，DeepSeek-V4-Pro-Max 优于 Claude Sonnet 4.5，并接近 Opus 4.5 的水平。
- **长上下文**：DeepSeek-V4-Pro-Max 在 1M token 上下文窗口的合成与真实用例上均取得强劲结果，在学术基准上甚至超过 Gemini-3.1-Pro。
- **DeepSeek-V4-Pro 与 DeepSeek-V4-Flash 对比**：由于参数规模较小，DeepSeek-V4-Flash-Max 在知识评估中表现较低。然而，在分配更大的思考预算时，它在推理任务上取得了相当的结果。在智能体评测中，DeepSeek-V4-Flash-Max 虽在若干基准上达到 DeepSeek-V4-Pro-Max 的性能，但在更复杂、更高难度的任务上仍落后于规模更大的对应版本。

2 架构

总体而言，DeepSeek-V4 系列保留 Transformer（Vaswani 等，2017）架构与多 token 预测（MTP）模块（DeepSeek-AI，2024；Gloeckle 等，2024），同时在 DeepSeek-V3 基础上引入若干关键升级：（1）第一，我们引入流形约束超连接（mHC）（Xie 等，2026）以强化传统残差连接；（2）第二，我们设计混合注意力架构，通过压缩稀疏注意力与重度压缩注意力大幅提升长上下文效率；

（3）第三，我们采用 Muon（Jordan 等，2024；Liu 等，2025）作为优化器。对于混合专家（MoE）组件，我们仍采用 DeepSeekMoE（Dai 等，2024）架构，仅相对 DeepSeek-V3 做小幅调整。多 token 预测（MTP）（DeepSeek-AI，2024；Gloeckle 等，2024；Li 等，2024；Qi 等，2020）的配置与 DeepSeek-V3 完全相同。其余未明确说明的细节均沿用 DeepSeek-V3（DeepSeek-AI，2024）的设置。图 2 展示了 DeepSeek-V4 的整体架构，细节描述如下。

2.1 继承自 DeepSeek-V3 的设计

混合专家 与此前的 DeepSeek 系列模型（DeepSeek-AI，2024；DeepSeek-AI，2024）一样，DeepSeek-V4 系列在前馈网络（FFN）中同样采用 DeepSeekMoE 范式（Dai 等，2024），设置细粒度的路由专家与共享专家。与 DeepSeek-V3 不同，我们将计算亲和分数（affinity score）的激活函数从 Sigmoid(·) 改为 Sqrt(Softplus(·))。在负载均衡方面，我们仍采用无辅助损失（auxiliary-loss-free）策略（DeepSeek-AI，2024；Wang 等，2024a），并辅以轻微的序列级均衡损失，以防止单个序列内出现极端不均衡。对于 DeepSeek-V4，我们取消了对路由目标节点数量的约束，并重新精心设计并行策略以维持训练效率。此外，与 DeepSeek-V3 相比，我们将最初若干个 Transformer 块中的稠密 FFN 层替换为采用哈希路由（Hash routing）（Roller 等，2021）的 MoE 层。哈希路由策略依据输入 token ID 的预定义哈希函数来确定每个 token 的目标专家。

多 token 预测 与 DeepSeek-V3 一样，DeepSeek-V4 系列同样设置 MTP 模块与目标。鉴于 MTP 策略已在 DeepSeek-V3 中得到验证，我们对 DeepSeek-V4 系列采用相同的策略，未作修改。

2.2 流形约束超连接

如图 2 所示，DeepSeek-V4 系列引入流形约束超连接（mHC）（Xie 等，2026），以强化相邻 Transformer 块之间的传统残差连接。与朴素超连接（HC）（Zhu 等，2025）相比，mHC 的核心思想是将残差映射约束到特定流形上，从而在保持模型表达能力的同时增强跨层信号传播的稳定性。本节简要介绍标准 HC，并阐述我们如何设计 mHC 以实现稳定训练。

标准超连接 标准 HC 将残差流的宽度扩大 n_{hc} 倍。具体而言，残差流的形状从 $\mathbb{R}^d$ 扩展为 $\mathbb{R}^{n_{hc}\times d}$，其中 d 是实际层输入的隐藏维度。设 $X_l = [x_{l,1}; \ldots; x_{l,n_{hc}}]^T \in \mathbb{R}^{n_{hc}\times d}$ 为第 l 层之前的残差状态。HC 引入三个线性映射：输入映射 $A_l \in \mathbb{R}^{1\times n_{hc}}$、残差变换 $B_l \in \mathbb{R}^{n_{hc}\times n_{hc}}$ 与输出映射 $C_l \in \mathbb{R}^{n_{hc}\times 1}$。残差状态的更新可表述为：

$$X_{l+1} = B_l X_l + C_l F_l(A_l X_l) \qquad (1)$$

其中 F_l 表示第 l 层（例如 MoE 层），其输入与输出形状均为 R^d。注意，实际层输入 A_l X_l ∈ R^d 也是 d 维的，因此扩展后的残差宽度不会影响内层的设计。HC 将残差宽度与实际隐藏维度解耦，提供了一种计算开销极低的补充扩展维度，因为 n_hc 通常远小于隐藏维度 d。然而，尽管 HC 在提升模型性能方面展现出潜力，我们发现堆叠多层时训练会频繁出现数值不稳定，这阻碍了 HC 的扩展。

流形约束的残差映射 mHC 的核心创新是将残差映射矩阵 B_l 约束到双随机矩阵流形（Birkhoff 多胞形）M 上，从而增强跨层信号传播的稳定性：

B_l ∈ M ∈ ⩾ { M R^{n×n} 2 M 1_n = 1_n, 1_n^T M = 1_n^T, M

该约束确保映射矩阵的谱范数 ‖B_l‖_2 不超过 1，因此残差变换是非扩张的（non-expansive），这提升了前向传播与反向传播过程中的数值稳定性。此外，集合 M 对乘法封闭，这保证了 mHC 深度堆叠场景下的稳定性。另外，输入变换 A_l 与输出变换 C_l 也被约束为非负且有界（通过 Sigmoid 函数），以避免信号相消的风险。

动态参数化 三个线性映射的参数是动态生成的，被分解为动态（依赖输入）分量与静态（不依赖输入）分量。给定输入 X_l ∈ R^{n_hc×d}，首先将其展平并归一化：X̂_l = RMSNorm(vec(X_l)) ∈ R^{1×n_hc d}。然后，我们沿用常规 HC 生成无约束的原始参数 Ã_l ∈ R^{1×n_hc}、B̃_l ∈ R^{n_hc×n_hc} 与 C̃_l ∈ R^{n_hc×1}：

Ã_l = α_pre^l · (X̂_l W_pre^l) + S_pre^l （3）

B̃_l = α_res^l · Mat(X̂_l W_res^l) + S_res^l （4）

C̃_l = α_post^l · (X̂_l W_post^l)^T + S_post^l （5）

其中 W_pre^l、W_post^l ∈ R^{n_hc d×n_hc} 与 W_res^l ∈ R^{n_hc d×n_hc^2} 是用于生成动态分量的可学习参数；Mat(·) 将大小为 1×n_hc^2 的向量重塑为 n_hc×n_hc 的矩阵；S_pre^l ∈ R^{1×n_hc}、S_post^l ∈ R^{n_hc×1} 与 S_res^l ∈ R^{n_hc×n_hc} 是可学习的静态偏置；α_pre^l、α_res^l、α_post^l ∈ R 是初始化为较小值的可学习门控因子。

施加参数约束 在获得无约束的原始参数 Ã_l、B̃_l、C̃_l 之后，我们对它们施加前述约束以增强数值稳定性。具体而言，对于输入与输出映射，我们采用 Sigmoid 函数 σ(·) 以确保其非负性与有界性：

A_l = σ(Ã_l) （6）

C_l = 2σ(C̃_l) （7）

至于残差映射 B̃_l，我们将其投影到双随机矩阵流形 M 上。这通过 Sinkhorn-Knopp 算法实现：该算法首先对 B̃_l 应用指数函数以确保正性，得到 M^(0) = exp(B̃_l)，然后迭代进行列归一化与行归一化：

$$M^{(t)} = T_r(T_c(M^{(t-1)})) \quad (8)$$

其中 T_r 与 T_c 分别表示行归一化与列归一化。该迭代收敛到一个受约束的双随机矩阵 B_l = M^(t_max)。我们取 t_max = 20 作为实用取值。

【图表】图 3 CSA 的核心架构。它将 KV 条目数量压缩至 1/m，然后应用 DeepSeek 稀疏注意力以进一步加速。此外，一小部分滑动窗口 KV 条目与选中的压缩 KV 条目相结合，以增强局部细粒度依赖。（原文第 9 页）

2.3 基于 CSA 与 HCA 的混合注意力

当上下文长度达到极端规模时，注意力机制成为模型中的主要计算瓶颈。对于 DeepSeek-V4，我们设计了两种高效注意力架构——压缩稀疏注意力（CSA）与重度压缩注意力（HCA）——并采用它们的交错混合配置，从而大幅降低长文本场景中注意力的计算成本。CSA 同时集成了压缩与稀疏注意力策略：它首先将每 m 个 token 的键值（KV）缓存压缩为一个条目，然后应用 DeepSeek 稀疏注意力（DSA）（DeepSeek-AI，2025），其中每个查询 token 只关注 k 个压缩 KV 条目。HCA 旨在实现极端压缩，将每 m′（≫m）个 token 的 KV 缓存合并为一个条目。CSA 与 HCA 的混合架构显著提升了 DeepSeek-V4 系列的长上下文效率，使百万 token 上下文在实践中成为可能。本节介绍混合注意力架构的核心技术，我们还提供了开源实现[1]，以明确更多细节。

[1] https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/tree/main/inference

2.3.1 压缩稀疏注意力

CSA 的核心架构如图 3 所示：它首先将每 m 个 token 的 KV 缓存压缩为一个条目，然后应用 DeepSeek 稀疏注意力以进一步加速。

压缩键值条目 令 H ∈ R^{n×d} 为输入隐藏状态序列，其中 n 为序列长度、d 为隐藏维度。CSA 首先计算两组 KV 条目 C_a、C_b ∈ R^{n×c} 及其对应的压缩权重 Z_a、Z_b ∈ R^{n×c}，其中 c 为头维度：

$$C_a = H \cdot W_a^{KV},\ C_b = H \cdot W_b^{KV} \quad (9)$$

$$Z_a = H \cdot W_a^{Z},\ Z_b = H \cdot W_b^{Z} \quad (10)$$

其中 W_a^{KV}、W_b^{KV}、W_a^{Z}、$W_b^{Z} \in \mathbb{R}^{d\times c}$ 为可训练参数。接下来，C_a 与 C_b 中每 m 个 KV 条目将依据其压缩权重与可学习位置偏置 B_a、$B_b \in \mathbb{R}^{m\times c}$ 被压缩为一个条目，得到 $C_{Comp} \in \mathbb{R}^{(n/m)\times c}$。每个压缩条目 $C_{Comp}^i \in \mathbb{R}^c$ 计算如下：

$$[S_a^{mi:m(i+1)-1};\ S_b^{m(i-1):mi-1}] = \mathrm{Softmax_row}([Z_a^{mi:m(i+1)-1} + B_a;\ Z_b^{m(i-1):mi-1} + B_b]) \quad (11)$$

$$C_{Comp}^i = \sum_{j=mi}^{m(i+1)-1} S_a^j \odot C_a^j + \sum_{j=m(i-1)}^{mi-1} S_b^j \odot C_b^j \quad (12)$$

其中 $\odot$ 表示 Hadamard 积；$\mathrm{Softmax_row}(\cdot)$ 表示沿行维度的 softmax 操作，在来自 Z_a 与 Z_b 的共 $2m$ 个元素上进行归一化。当 $i = 0$ 时，$Z_b^{m(i-1):mi-1}$ 以负无穷填充，$C_b^{m(i-1):mi-1}$ 以零填充。注意，每个 C_{Comp}^i 由 $2m$ 个 KV 条目得到，但用于 C_{Comp}^i 的 C_b 索引与用于 C_{Comp}^{i-1} 的 C_a 索引是重叠的。因此，CSA 实际上将序列长度压缩至 $1/m$ 倍。

用于稀疏选择的闪电索引器 在获得压缩 KV 条目 C_{Comp} 之后，CSA 应用 DSA 策略为核心注意力选择 top-k 个压缩 KV 条目。首先，CSA 执行与 C_{Comp} 相同的压缩操作，得到压缩索引器键 $K_{IComp} \in \mathbb{R}^{(n/m)\times c_I}$，其中 c_I 为索引器头维度。然后，对于查询 token t，我们以低秩方式生成索引器查询 $\{q_{t,1}^I; q_{t,2}^I; ...; q_{t,n_{Ih}}^I\}$：

$$c_Q^t = h_t \cdot W_{DQ} \quad (13)$$

$$[q_{t,1}^I; q_{t,2}^I; ...; q_{t,n_{Ih}}^I] = q_t^I = c_Q^t \cdot W_{IUQ} \quad (14)$$

其中 $h_t \in \mathbb{R}^d$ 是查询 token t 的输入隐藏状态；$c_Q^t \in \mathbb{R}^{d_c}$ 是查询的压缩潜在向量；d_c 表示查询压缩维度；n_{Ih} 表示索引器查询头数；$W_{DQ} \in \mathbb{R}^{d\times d_c}$ 与 $W_{IUQ} \in \mathbb{R}^{d_c\times c_I n_{Ih}}$ 分别为索引器查询的下投影与上投影矩阵。接下来，查询 token t 与前序压缩块 s（$s < \mathrm{Floor}(t/m)$）之间的索引分数 $I_{t,s} \in \mathbb{R}$ 计算如下：

$$[w_{t,1}^I; w_{t,2}^I; ...; w_{t,n_{Ih}}^I] = w_t^I = h_t \cdot W_w \quad (15)$$

$$I_{t,s} = \sum_{h=1}^{n_{Ih}} w_{t,h}^I \cdot \mathrm{ReLU}(q_{t,h}^I \cdot K_{IComp}^s) \quad (16)$$

其中 $W_w \in \mathbb{R}^{d\times n_{Ih}}$ 为可学习矩阵；$w_{t,h}^I \in \mathbb{R}$ 是第 h 个索引器头的权重。对于查询 token t，给定其索引分数 $I_{t,:}$，我们采用 top-k 选择器有选择地保留一部分压缩 KV 条目 $C_{SprsComp}^t$ 用于后续核心注意力：

$$C_{SprsComp}^t = \{ C_{Comp}^s \mid I_{t,s} \in \text{Top-k}(I_{t,:}) \} \quad (17)$$

【图表】图 4 HCA 的核心架构。它执行更重的压缩，将 m'（$\gg m$）个 token 的 KV 条目合并为一个。此外，我们额外引入一小部分滑动窗口 KV 条目，以增强局部细粒度依赖。（原文第 11 页）

共享键值 MQA 在选择稀疏 KV 条目之后，CSA 以多查询注意力（MQA）（Shazeer，2019）方式执行核心注意力，其中 C_SprsComp^t 中的每个压缩 KV 条目同时充当注意力键与值。具体而言，对于查询 token t，我们首先从压缩潜在向量 c_Q^t 生成注意力查询 {q_{t,1}; q_{t,2}; ...; q_{t,n_h}}：

[q_{t,1}; q_{t,2}; ...; q_{t,n_h}] = q_t = c_Q^t · W_UQ （18）

其中 n_h 表示查询头数；W_UQ ∈ R^{d_c×c n_h} 为查询的上投影矩阵。注意，潜在查询向量 c_Q^t 与索引器查询所用的向量共享。接下来，我们在 {q_{t,i}} 与 C_SprsComp^t 上执行 MQA：

o_{t,i} = CoreAttn(query=q_{t,i}, key=C_SprsComp^t, value=C_SprsComp^t) （19）

其中 o_{t,i} ∈ R^c 是第 t 个 token 第 i 个头的核心注意力输出；CoreAttn(·) 表示核心注意力操作。

分组输出投影 在 DeepSeek-V4 的配置中，c n_h 相当大。因此，直接将核心注意力操作的输出 [o_{t,1}; o_{t,2}; ...; o_{t,n_h}] = o_t ∈ R^{c n_h} 投影为 d 维隐藏状态会带来可观的计算负担。为缓解这一成本，我们设计了分组输出投影策略。具体而言，我们首先将 n_h 个输出划分为 g 组，然后对每组输出 o_{G,t,i} ∈ R^{c n_h/g}，将其投影为 d_g 维的中间输出 o_{G′,t,i} ∈ R^{d_g}，其中 d_g < c n_h/g。最后，我们将中间输出 [o_{G′,t,1}; o_{G′,t,2}; ...; o_{G′,t,g}] ∈ R^{d_g g} 投影为最终注意力输出 ô_t ∈ R^d。

2.3.2 重度压缩注意力

HCA 的核心架构如图 4 所示：它以更重的方式压缩 KV 缓存，但不采用稀疏注意力。

压缩键值条目 总的来说，HCA 的压缩策略与 CSA 类似，但采用更大的压缩率 m′（≫m），且不执行重叠压缩。令 H ∈ R^{n×d} 为输入隐藏状态序列，HCA 首先计算原始 KV 条目 C ∈ R^{n×c} 及其对应的压缩权重 Z ∈ R^{n×c}：

C = H · W_KV （20）

Z = H · W_Z （21）

其中 W_KV、W_Z ∈ R^{d×c} 为可训练参数。接下来，C 中每 m′ 个 KV 条目将依据压缩权重与可学习位置偏置 B ∈ R^{m′×c} 被压缩为一个条目，得到 C_Comp ∈ R^{(n/m′)×c}。每个压缩条目 C_Comp^i ∈ R^c 计算如下：

S_{m′i:m′(i+1)−1} = Softmax_row(Z_{m′i:m′(i+1)−1} + B) （22）

C_Comp^i = Σ_{j=m′i}^{m′(i+1)−1} S_j ⊙ C_j （23）

通过这一压缩操作，HCA 将序列长度压缩至 1/m′ 倍。

共享键值 MQA 与分组输出投影 HCA 同样采用与 CSA 相同的共享 KV MQA 与分组输出投影策略。在 KV 压缩之后，对于查询 token t，HCA 首先以低秩方式生成注意力查询 {q_{t,1}; q_{t,2}; ...; q_{t,n_h}}：

c_Q^t = h_t · W_DQ （24）

[q_{t,1}; q_{t,2}; ...; q_{t,n_h}] = q_t = c_Q^t · W_UQ （25）

其中 h_t ∈ R^d 是查询 token t 的输入隐藏状态；n_h 表示查询头数；W_DQ ∈ R^{d×d_c} 与 W_UQ ∈ R^{d_c×c n_h} 分别为查询的下投影与上投影矩阵。接下来，我们在 {q_{t,i}} 与 C_Comp 上执行 MQA：

o_{t,i} = CoreAttn(query=q_{t,i}, key=C_Comp, value=C_Comp) （26）

其中 o_{t,i} ∈ R^c 是第 t 个 token 第 i 个头的核心注意力输出。接下来，与 CSA 一样，HCA 将 n_h 个输出划分为 g 组，并对每组输出 o_{G,t,i} ∈ R^{c n_h/g} 投影为 d_g 维的中间输出 o_{G′,t,i} ∈ R^{d_g}，其中 d_g < c n_h/g。最后，HCA 将中间输出 [o_{G′,t,1}; o_{G′,t,2}; ...; o_{G′,t,g}] ∈ R^{d_g g} 投影为最终注意力输出 ô_t ∈ R^d。

2.3.3 其他细节

除上述 CSA 与 HCA 的核心架构外，我们的混合注意力还包含若干其他技术。为行文清晰，我们在上述介绍中省略了这些技术，并在本小节简要描述。此外，本小节仅聚焦其核心思想，为简洁起见可能省略一些微小细节。我们建议读者参阅我们的开源实现，以了解明确无歧义的细节。

查询与键值条目归一化 对于 CSA 与 HCA，我们都在核心注意力操作之前，对查询的每个头以及压缩 KV 条目的唯一头额外执行一次 RMSNorm 操作。这种归一化可避免注意力 logits 爆炸，并可能提升训练稳定性。

部分旋转位置编码：对于 CSA 与 HCA，我们都在注意力查询、KV 条目与核心注意力输出上部分使用旋转位置编码（RoPE）（Su 等，2024）。具体而言，对于 CSA 与 HCA 中使用的每个查询向量与 KV 条目向量，我们对其最后 64 个维度施加 RoPE。由于 KV 条目同时充当注意力键与值，朴素的核心注意力输出 {o_{t,i}} 会携带由 KV 条目加权和导出的绝对位置嵌入。作为对策，我们还对每个 o_{t,i} 的最后 64 个维度施加位置为 −i 的 RoPE。这样，核心注意力的输出也将携带相对位置嵌入——每个 KV 条目对核心注意力输出的贡献也将与查询和该 KV 条目之间的距离相关。

滑动窗口注意力的附加分支：为在 CSA 与 HCA 中严格保持因果性，每个查询只能关注其之前的压缩 KV 块。因此，查询无法访问其自身压缩块内其他 token 的信息。同时，在语言建模中，最近的 token 通常与查询 token 更具相关性。基于这些原因，我们以滑动窗口方式为 CSA 与 HCA 引入一个补充注意力分支，以更好地建模局部依赖。具体而言，对于每个查询 token，我们额外生成 n_win 个未压缩 KV 条目，对应当前最近的 n_win 个 token。在 CSA 与 HCA 的核心注意力中，这些滑动窗口内的 KV 条目将与压缩 KV 条目一起使用。

注意力汇聚（Attention Sink）：在 CSA 与 HCA 的核心注意力中，我们采用注意力汇聚技巧（OpenAI，2025；Xiao 等，2024）。具体而言，我们设置一组可学习的汇聚 logits {z'_1, z'_2, …, z'_{n_h}}。对于第 h 个注意力头，Exp(z'_h) 将被加到注意力分数的分母上：

$$s_{h,i,j} = \mathrm{Exp}(z_{h,i,j}) / \left(\Sigma_{k} \mathrm{Exp}(z_{h,i,k}) + \mathrm{Exp}(z'_h) \right) \qquad (27)$$

其中 $s_{h,i,j}$、$z_{h,i,j} \in R$ 分别表示第 h 个注意力头在第 i 个查询 token 与第 j 个先前 token 或压缩块之间的注意力分数与注意力 logit。该技术使每个查询头能够将其总注意力分数调整为不等于 1，甚至接近 0。

2.3.4 效率讨论

由于采用混合 CSA 与 HCA，并结合低精度计算与存储，DeepSeek-V4 系列的注意力模块在注意力 FLOPs 与 KV 缓存大小两方面都实现了显著效率，尤其是在长上下文场景中。第一，我们对 KV 条目采用混合存储格式：旋转位置编码（RoPE）维度使用 BF16 精度，其余维度使用 FP8 精度。与纯 BF16 存储相比，这种混合表示将 KV 缓存大小减少近一半。第二，闪电索引器（lightning indexer）内的注意力计算以 FP4 精度执行，这加速了超长上下文下的注意力操作。第三，相对 DeepSeek-V3.2，DeepSeek-V4 系列选择了更小的注意力 top-k，从而提升模型在短文本与中等长度文本上的效率。最后，也是最重要的一点，压缩注意力与混合注意力技术大幅降低了 KV 缓存大小与计算 FLOPs。

以 BF16 GQA8（Ainslie 等，2023）、头维度为 128 作为基线——这是 LLM 注意力的常见配置之一——DeepSeek-V4 系列的 KV 缓存大小在 1M 上下文设置下可大幅降至该基线的约 2%。

此外，即使与已经很高效的 DeepSeek-V3.2（DeepSeek-AI，2025）相比，DeepSeek-V4 系列在效率上仍具有显著优势。它们的推理 FLOPs 与 KV 缓存大小对比见图 1 右侧。

2.4 Muon 优化器

我们对 DeepSeek-V4 系列的大多数模块采用 Muon（Jordan 等，2024；Liu 等，2025）优化器，因为它收敛更快且训练稳定性更好。我们完整的 Muon 优化算法总结于算法 1。

算法 1：DeepSeek-V4 的 Muon 优化器

- **输入**：学习率 η、动量 μ、权重衰减 λ、更新重缩放因子 γ
- **1**：for 每个训练步 t do
- **2**：for 每个逻辑独立的权重 W ∈ R^{n×m} do
- **3**：G_t = ∇_W L_t(W_{t−1})（计算梯度）
- **4**：M_t = μM_{t−1} + G_t（累积动量缓冲）
- **5**：O'_t = HybridNewtonSchulz(μM_t + G_t)（Nesterov 技巧与混合 Newton-Schulz）
- **6**：O_t = O'_t · √(max(n,m)) · γ（重缩放更新 RMS）
- **7**：W_t = W_{t−1}·(1−ηλ) − ηO_t（执行权重衰减与更新）
- **8**：end for
- **9**：end for

基本配置：嵌入模块、预测头模块、mHC 模块的静态偏置与门控因子，以及所有 RMSNorm 模块的权重，继续使用 AdamW（Loshchilov 与 Hutter，2017）优化器；其余所有模块均用 Muon 更新。遵循 Liu 等（2025），我们也对 Muon 参数施加权重衰减，使用 Nesterov（Jordan 等，2024；Nesterov，1983）技巧，并重缩放更新矩阵的均方根（RMS），以复用我们的 AdamW 超参数。与他们不同的是，我们使用混合 Newton-Schulz 迭代进行正交化。

混合 Newton-Schulz 迭代：给定矩阵 M，设其奇异值分解（SVD）为 M = UΣV^T。Newton-Schulz 迭代旨在将 M 近似正交化为 UV^T。通常，M 会先被归一化为 M_0 = M/||M||_F，以确保其最大奇异值不超过 1。随后，每次 Newton-Schulz 迭代执行如下操作：

$$M_k = a \cdot M_{k-1} + b \cdot (M_{k-1} M_{k-1}^T) \cdot M_{k-1} + c \cdot (M_{k-1} M_{k-1}^T)^2 \cdot M_{k-1} \quad (28)$$

我们的混合 Newton-Schulz 在两个不同阶段共执行 10 次迭代。前 8 步使用系数 (a, b, c) = (3.4445, −4.7750, 2.0315) 以驱动快速收敛，使奇异值接近 1。最后 2 步切换到系数 (a, b, c) = (2, −1.5, 0.5)，将奇异值精确稳定在 1。

避免注意力 logits 爆炸：DeepSeek-V4 系列的注意力架构允许我们直接在注意力查询与 KV 条目上应用 RMSNorm，从而有效防止注意力 logits 爆炸。因此，我们的 Muon 优化器不采用 QK-Clip 技术（Liu 等，2025）。

3 通用基础设施

3.1 专家并行中的细粒度通信-计算重叠

混合专家（MoE）可通过专家并行（EP）加速。然而，EP 需要复杂的跨节点通信，并对互连带宽与延迟提出很高要求。为缓解 EP 中的通信瓶颈，并在更低的互连带宽需求下实现更高的端到端性能，我们提出一种细粒度的 EP 方案，将通信与计算融合为单个流水线内核，实现通信-计算重叠。

通信延迟可以被隐藏：我们 EP 方案的关键洞见是，MoE 层中的通信延迟可以被有效地隐藏在计算之下。如图 5 所示，在 DeepSeek-V4 系列中，每个 MoE 层主要可分解为四个阶段：两个通信密集型阶段——Dispatch 与 Combine，以及两个计算密集型阶段——Linear-1 与 Linear-2。我们的性能剖析表明，在单个 MoE 层内，通信总时间小于计算总时间。因此，在将通信与计算融合为统一流水线后，计算仍然是主要瓶颈，这意味着系统能够容忍更低的互连带宽而不会降低端到端性能。

【图表】图 5 我们的 EP 方案与相关工作的对比示意。Comet（Zhang 等，2025b）分别将 Dispatch 与 Linear-1、Linear-2 与 Combine 重叠。我们的 EP 方案通过将专家拆分并调度为波次，实现了更细粒度的重叠。理论加速比在 DeepSeek-V4-Flash 架构配置下评估（原文第 15 页）

细粒度 EP 方案：为进一步降低互连带宽需求并放大重叠收益，我们引入更细粒度的专家划分方案。受多项相关工作（Aimuyo 等，2025；Zhang 等，2025b）启发，我们将专家拆分并调度为波次（wave）。每个波次包含一小部分专家。一旦某个波次内的所有专家完成通信，计算便可以立即开始，而无需等待其他专家。在稳态下，当前波次的计算、下一波次的 token 传输以及已完成专家的结果发送都会并发进行，如图 5 所示。这在专家之间形成了细粒度流水线，使整个波次期间的计算与通信都保持连续。基于波次的调度加速了强化学习（RL）rollout 等极端场景下的性能，这类场景通常会遇到长尾小批次。

性能与开源 Mega-Kernel：我们在 NVIDIA GPU 与华为昇腾 NPU 两个平台上验证了细粒度 EP 方案。与强有力的非融合基线相比，它在通用推理负载上实现了 1.50–1.73× 的加速，在 RL rollout 与高速智能体服务等延迟敏感场景中最高可达 1.96×。我们已将基于 CUDA 的 mega-kernel 实现开源，

命名为 MegaMoE（https://github.com/deepseek-ai/DeepGEMM/pull/304），作为 DeepGEMM 的一个组件。

观察与建议：我们分享内核开发中的观察与经验，并向硬件厂商提出一些建议，希望有助于高效的硬件设计并实现更好的软硬件协同设计：

- **计算-通信比**：完全的通信-计算重叠取决于计算-通信比，而不仅仅是带宽。记峰值计算吞吐为 C、互连带宽为 B，当 $C/B \le V_{comp}/V_{comm}$ 时通信可被完全隐藏，其中 V_{comp} 表示计算量、V_{comm} 表示通信量。对于 DeepSeek-V4-Pro，每个 token-专家对需要 $6hd$ FLOPs（SwiGLU 的门、上、下投影），却只需 $3h$ 字节的通信（FP8 Dispatch + BF16 Combine），因此该式简化为：$C/B \le 2d = 6144$ FLOPs/Byte。也就是说，每 GBps 的互连带宽足以隐藏 6.1 TFLOP/s 计算的通信。一旦带宽达到该阈值，它就不再是瓶颈，将额外的芯片面积投入进一步提升带宽会带来递减的回报。我们鼓励未来的硬件设计瞄准此类平衡点，而不是无条件地扩展带宽。
- **功耗预算**：极端的 kernel 融合会使计算、内存与网络同时处于高负载，使功耗节流成为关键的性能限制因素。我们建议未来的硬件设计为此类完全并发的工作负载提供充足的功耗余量。
- **通信原语**：在 dispatch 阶段，我们采用基于拉取（pull）的方式，即每个 GPU 主动从远端 GPU 读取激活值，避免细粒度 push 带来的高通知延迟。未来若硬件具备更低延迟的跨 GPU 信令，push 将变得可行，从而实现更自然的通信模式。
- **激活函数**：我们建议用不涉及指数或除法运算的低成本逐元素激活函数替代 SwiGLU。这直接减少了 GEMM 后处理的开销，防止 GEMM 流水线被激活函数计算阻塞，从而提升整体计算吞吐与资源利用率。

3.2 基于 TileLang 的灵活高效内核开发

在实践中，我们精细的模型架构会产生数百个细粒度的 Torch ATen 算子。我们采用 TileLang（Wang 等，2026）开发了一组融合内核来替代其中绝大多数算子，以最小的工作量交付最优性能。它还使我们能够在验证阶段快速原型化注意力变体等算子。这些内核在模型架构开发、大规模训练以及最终的推理服务生产部署中都发挥着关键作用。作为一种领域特定语言（DSL），TileLang 在开发生产力与运行时效率之间取得平衡，既支持快速开发，也支持在同一代码库内进行深度迭代优化。此外，我们与 TileLang 社区紧密合作，以打造更敏捷、高效、稳定的内核开发工作流。

通过 Host Codegen 降低调用开销：随着加速器性能持续提升，CPU 侧的编排开销日益突出。对于小而高度优化的内核，这类固定的主机端开销很容易限制利用率与吞吐。这种开销的一个常见来源

是，主机端逻辑（如运行时契约检查）通常用 Python 编写以获得灵活性，因而每次调用都会产生固定成本。我们通过 Host Codegen 缓解这一开销，它将大部分主机端逻辑移入生成的主机代码。具体而言，我们首先在 IR（中间表示）层面协同生成设备内核与轻量级主机启动器，并嵌入从语言前端解析出的必要元数据——如数据类型、秩/形状约束与步长/布局假设。随后，启动器被下沉为基于 TVM-FFI（Chen 等，2018）框架构建的主机源代码，其紧凑的调用约定与零拷贝张量互操作共同将主机端开销降至最低。运行时，这段生成的主机代码执行验证与参数编排，将全部逐调用检查移出 Python 执行路径。我们的测量表明，CPU 侧验证开销从数十或数百微秒降至每次调用不足 1 微秒。

SMT 求解器辅助的形式化整数分析：TileLang 内核涉及复杂的张量索引算术，需要强大的形式化整数分析。在布局推断、内存冒险检测与边界分析等编译过程中，编译器必须验证整数表达式是否满足特定性质，以启用相应优化。因此，更强的形式分析能力可以解锁更高级、更复杂的优化机会。为此，我们将 Z3 SMT 求解器（De Moura 与 Bjørner，2008）集成到 TileLang 的代数系统中，为张量程序中的大多数整数表达式提供形式分析能力。我们通过将 TileLang 的整数表达式转换为 Z3 的无量词非线性整数算术（QF_NIA），在计算开销与形式化表达能力之间取得平衡。基于整数线性规划（ILP）求解器，QF_NIA 可无缝求解内核中常见的标准线性整数表达式。此外，其固有的非线性推理能力可有效应对变量张量形状上的向量化等高级挑战。在合理的资源限制下，Z3 提升了整体优化性能，同时将编译时间开销限制在几秒钟。这一影响在向量化、屏障插入与代码简化等多个编译过程中都很显著。

数值精度与按位可复现性：在生产环境中，数值正确性与可复现性同原始吞吐一样关键。因此，我们默认优先考虑准确性：在编译器层面禁用 fast-math 优化，影响精度的近似仅作为显式、可选项的前端算子提供（例如 T.__exp、T.__log 与 T.__sin）。反之，当需要严格的 IEEE-754 语义时，TileLang 提供带显式舍入模式的 IEEE 兼容内建函数（例如 T.ieee_fsqrt、T.ieee_fdiv 与 T.ieee_add），使开发者能够精确指定数值行为。

我们还以按位可复现性为目标，用其对照手写 CUDA 基线验证内核。我们将 TileLang 的代数化简与下沉规则与主流 CUDA 工具链（如 NVCC）对齐，以避免引入非预期位级差异的变换。布局注解（例如 T.annotate_layout）进一步允许用户固定依赖布局的下沉决策，使评估与累加顺序与参考 CUDA 实现保持一致，从而在需要时实现逐位相同的输出。我们的评估表明，这些面向准确性与可复现性的设计选择并未牺牲性能：在保守默认设置下，TileLang 内核仍具竞争力，同时暴露可选择性放宽数值约束以换取更高速度的旋钮。

3.3 高性能、批不变且确定性的内核库

为支持高效训练与推理，我们开发了一套全面的高性能计算内核。除了基本功能与最大化硬件利用率之外，另一个关键设计目标是确保训练可复现性，以及预训练、后训练与推理流水线之间的逐位对齐。因此，我们实现了端到端、逐位批不变且确定性的内核，且性能开销极小。这些内核对调试、稳定性分析与一致的后训练行为很有帮助。

批不变性（Batch Invariance）：批不变性确保任意给定 token 的输出在批内位置变化时仍保持逐位相同。实现批不变性面临以下主要挑战：

- **注意力**：为实现批不变性，我们不能使用 split-KV 方法（Dao 等，2023），该方法将单条序列的注意力计算分散到多个流多处理器（SM）上以平衡 SM 负载。然而，放弃该技术会导致严重的波量化（wave-quantization）问题（https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html#wave-quant），对 GPU 利用率产生不利影响。为此，我们为批不变解码开发了双内核策略。第一个内核在单个 SM 内计算整条序列的注意力输出，确保在波全部占满时保持高吞吐。第二个内核则使用多个 SM 处理单条序列，以最小化最后一个部分填充波的延迟，从而缓解波量化。为在这两个内核之间实现逐位一致，我们精心设计了第二个内核的计算路径，确保其累加顺序与第一个内核相同。此外，第二个内核利用线程块簇（thread-block clusters）内的分布式共享内存（https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html#distributed-shared-memory），实现跨 SM 的高速数据交换。这一双内核方法有效地将批不变解码的开销限制在可忽略的程度。
- **矩阵乘法**：传统 cuBLAS 库（NVIDIA Corporation，2024）无法实现批不变性。因此，我们用 DeepGEMM（Zhao 等，2025）端到端替代它。此外，对于非常小的批大小，传统实现通常采用 split-k（Osama 等，2023）技术来提升性能。遗憾的是，split-k 技术无法保证批不变性，而这是 DeepSeek-V4 的一项关键特性。因此，我们在大多数场景中放弃 split-k，但这可能造成性能下降。为此，我们引入一组优化，使我们的矩阵乘法实现能够在大多数主要场景中达到甚至超过标准 split-k 的性能。

确定性（Determinism）：确定性训练对调试硬件或软件问题非常有益。此外，当训练出现损失尖峰等异常时，确定性使研究者能够更容易定位数值原因并进一步改进模型设计。训练中的非确定性通常源于非确定性的累加顺序，往往由原子加指令造成。该问题主要出现在反向传播中，尤其是在以下部分：

- **注意力反向**：在稀疏注意力反向传播的传统实现中，我们使用 atomicAdd 累加 KV token 的梯度。由于浮点加法不满足结合律，这会引入非确定性。为解决此问题，我们为每个 SM 分配独立的累加缓冲区，随后对所有缓冲区执行全局确定性求和。
- **MoE 反向**：当来自不同 rank 的多个 SM 并发向接收 rank 上的同一缓冲区写入数据时，协商写入位置也会引入非确定性。为解决此问题，我们在每个 rank 内设计 token 顺序预处理机制，并结合跨多个 rank 的缓冲区隔离。该策略同时保证了专家并行发送结果与 MoE 反向传播中累加顺序的确定性。
- **mHC 中的矩阵乘法**：mHC 涉及一个输出维度仅为 24 的矩阵乘法。对于非常小的批大小，我们不得不使用 split-k（Osama 等，2023）算法，其朴素实现会导致非确定性。为此，我们分别输出每个 split 部分，并在后续内核中执行确定性归约，从而同时保持性能与确定性。

3.4 训练框架

我们的训练框架建立在为 DeepSeek-V3（DeepSeek-AI，2024）开发的可扩展高效基础设施之上。在训练 DeepSeek-V4 时，我们继承了这一稳健基础，同时引入若干关键创新，以适配其新颖的架构组件——特别是 Muon 优化器、mHC 与混合注意力机制——同时保持高训练效率与稳定性。

3.4.1 Muon 的高效实现

Muon 优化器需要完整的梯度矩阵来计算参数更新，这与零冗余优化器（ZeRO）（Rajbhandari 等，2020）结合时会产生挑战。传统 ZeRO 是为 AdamW 等逐元素优化器设计的，单个参数矩阵可以跨多个 rank 划分并更新。为解决这一冲突，我们为 Muon 设计了一种 ZeRO 桶分配的混合策略。

对于稠密参数，我们限制 ZeRO 并行的最大规模，并采用背包算法将参数矩阵分配给这些 rank，确保每个 rank 管理大致均衡的负载。每个 rank 上的桶会被填充以匹配所有 rank 中最大桶的大小，从而便于高效的 reduce-scatter 操作。在我们的设置中，每个 rank 管理的参数矩阵不超过五个，这种填充通常带来不到 10% 的内存开销。当数据并行的总体规模超过 ZeRO 的限制时，我们在额外的数据并行组之间冗余计算 Muon 更新，以计算换取了桶总内存的降低。

对于 MoE 参数，我们独立优化每个专家。我们首先展平所有层中所有专家的 SwiGLU（Shazeer，2020）下投影矩阵，随后依次展平上投影矩阵与门控矩阵。然后，我们对展平后的向量进行填充，以确保在不拆分任何逻辑独立矩阵的前提下，能将该向量均匀分配到所有 rank。鉴于专家数量众多，我们不对 MoE 参数施加 ZeRO 并行限制，填充开销也可忽略。

此外，在每个 rank 上，形状相同的连续参数会被自动合并，从而支持 Newton-Schulz 迭代的批处理执行，提升硬件利用率。进一步地，我们观察到 Muon 中的 Newton-Schulz 迭代在使用 BF16 矩阵乘法计算时仍然稳定。利用这一点，我们以随机舍入方式将跨数据并行 rank 同步的 MoE 梯度进一步量化为 BF16 精度，使通信量减半。为避免低精度加法器引入的累积误差，我们用两阶段方法替代传统的树形或环形 reduce-scatter 集合通信：首先通过 all-to-all 操作在各 rank 之间交换本地梯度，然后每个 rank 在 FP32 中执行本地求和。这一设计保持了数值稳健性。

3.4.2 mHC 的高性价比与内存高效实现

与传统的残差连接相比，mHC 的引入同时增加了激活内存消耗与流水线阶段之间的通信量。为缓解这些成本，我们实施了几项优化策略。

首先，我们为训练与推理精心设计并实现了 mHC 的融合内核。其次，我们引入重计算策略，选择性地对中间张量做检查点。具体而言，我们重计算层间的大部分隐藏状态以及所有归一化后的层输入，同时避免重计算计算密集的操作。这在节省内存与计算开销之间取得平衡。第三，我们调整 DualPipe 的 1F1B 重叠方案，以适应增加的流水线通信，并支持 mHC 中部分操作的并发执行。

总体而言，这些优化将 mHC 的墙钟时间开销限制在重叠 1F1B 流水线阶段的仅 6.7%。更多工程优化细节可参见专门的 mHC 论文（Xie 等，2026）。

3.4.3 面向长上下文注意力的上下文并行

传统上下文并行（CP）划分序列维度，每个 rank 维护连续的 s 个 token。这给我们的压缩注意力机制（即 CSA 与 HCA）带来两个挑战。一方面，训练样本由多条序列打包而成，每条序列按因子 m（或 m'）独立压缩，不足 m 的尾部 token 会被丢弃。因此，压缩后的 KV 长度通常小于 s/m，且在各 rank 之间不同。另一方面，压缩需要 m 个连续 KV 条目，这些条目可能跨越两个相邻 CP rank 的边界。

为应对这些挑战，我们设计了一种两阶段通信方法。第一阶段，每个 rank i 将其最后 m 个未压缩 KV 条目发送给 rank i+1。随后，rank i+1 将这些接收到的部分条目与其本地 s 个未压缩 KV 条目一起压缩，产出固定长度为 $s/m + 1$ 的压缩条目，其中包含一些填充条目。第二阶段，通过所有 CP rank 之间的 all-gather 操作收集本地压缩的 KV 条目。然后，一个融合的 select-and-pad 算子将它们重排为完整的压缩 KV 条目集合，总长度为 $\text{cp_size} \cdot s/m$。所有填充条目被放在末尾。对于 HCA 以及 CSA 中的索引器，每个查询 token 可见的压缩 KV 条目范围可由规则预先计算。对于 CSA 中的稀疏注意力，top-k 选择器显式指定每个查询可见压缩 KV 条目的索引。

3.4.4 扩展自动微分以支持灵活的激活检查点

传统激活检查点实现以整个模块为粒度，决定在反向传播期间保留还是重计算其输出激活。这种粗粒度往往导致重计算成本与激活内存占用之间的次优权衡。另一种方法是手动实现整个层的前向与反向逻辑，显式管理张量检查点状态。虽然能实现细粒度控制，但这种方法失去了自动微分框架的便利，显著增加了开发复杂度。

为在不牺牲编程效率的前提下实现细粒度控制，我们实现了一种支持自动微分的张量级激活检查点机制。借助该机制，开发者只需实现前向传播，并选择性地注解单个张量以进行自动检查点与重计算。我们的框架利用 TorchFX（Reed 等，2022）追踪完整计算图。对于每个被注解的张量，它执行反向遍历，识别重计算所需的最小子图。我们将这些最小子图定义为重计算图，并将其插入反向逻辑中相应梯度计算之前。

与手动实现相比，这一设计在训练期间不引入额外开销。该框架中的重计算通过直接释放被注解张量的 GPU 内存并复用重计算张量的存储指针来实现，无需任何 GPU 内存拷贝。此外，由于图追踪会具体执行模型，我们可以追踪每个张量的底层存储指针，从而自动对共享存储的张量（例如 reshape 操作的输入与输出）去重重计算。这使开发者无需在注解重计算时考虑底层内存细节。

3.5 推理框架

我们的推理框架大体继承自 DeepSeek-V3，在 KV 缓存管理上有所不同。

3.5.1 KV 缓存结构与管理

为高效管理 DeepSeek-V4 中混合注意力机制带来的异构 KV 缓存，我们设计了定制的 KV 缓存布局。该布局如图 6 所示，下面我们将详细阐述。

DeepSeek-V4 中的异构 KV 条目：DeepSeek-V4 系列的混合注意力机制引入了多种 KV 条目，它们的 KV 缓存大小与更新规则各不相同。用于稀疏选择的闪电索引器在 KV 缓存中引入了额外的维度，其嵌入大小与主注意力中的不同。CSA 与 HCA 采用的压缩技术分别将序列长度降低 1/m 与 1/m'，从而减小整体 KV 缓存大小。结果是，不同层的 KV 缓存大小各不相同。此外，滑动窗口注意力（SWA）层也以不同的 KV 缓存大小运行，并具有单独的缓存命中与淘汰策略。在压缩分支中，每 m 个 token 生成一个 KV 条目。当剩余 token 数量不足以进行压缩时，所有待处理 token 及其关联隐藏状态都必须保留在缓冲区中，直到可以执行压缩操作。这些被缓冲的 token 代表由位置上下文决定的序列状态，同样在 KV 缓存框架中管理。

【图表】图 6 DeepSeek-V4 的 KV 缓存布局示意。KV 缓存被组织为两个主要部分：用于 CSA/HCA 的经典 KV 缓存，以及用于 SWA 与 CSA/HCA 中尚未可压缩 token 的状态缓存。在状态缓存中，每个请求被分配一个固定大小的缓存块；在该块内，SWA 段存储最近 n_win 个 token 对应的 KV 条目，而 CSA/HCA 段存储尚未可压缩的未压缩尾部状态。在经典 KV 缓存中，我们为每个请求分配多个块；每个缓存块覆盖 lcm(m, m') 个原始 token，产生 k1 = lcm(m, m')/m 个 CSA 压缩 token 与 k2 = lcm(m, m')/m' 个 HCA 压缩 token（原文第 22 页）

管理混合注意力 KV 缓存的挑战：混合注意力机制违背了 PagedAttention 及其变体的基本假设。尽管近期的混合 KV 缓存管理算法（如 Jenga（Zhang 等，2025a）、Hymba（Dong 等，2025））面向通用混合注意力模型或特定结构，但有两个主要障碍阻碍在 PagedAttention 框架下统一所有层的 KV 缓存：

- 多样的缓存策略，例如滑动窗口注意力所使用的策略。
- 高性能注意力内核带来的约束，包括对齐要求。

为实现 DeepSeek-V4 的高效 KV 缓存管理，我们设计了相应策略来克服这两个挑战。

面向 SWA 与未压缩尾部 token 的状态缓存：为应对第一个障碍，我们采用另一种缓存管理机制。由于 SWA 旨在有限的 KV 缓存大小下提升性能，将其与压缩分支中未压缩的尾部 token 一起视为状态空间模型是合理的。相应的 KV 缓存因此可被视为仅取决于当前位置的、特定于序列的状态。据此，我们预分配一个固定且有限大小的状态缓存池，并动态分配给每条序列。

稀疏注意力内核协同设计：关于第二个障碍，传统高性能注意力内核通常假设每块固定 B 个 token 以优化性能，在 CSA 中对应 B·m 个原始 token，在 HCA 中对应 B·m' 个。通过采用高性能稀疏注意力内核，不同层可以在不损失性能的情况下容纳可变的每块 token 数。这需要协同设计 KV 缓存布局与稀疏注意力内核。例如，将块填充以对齐缓存行可以提升性能。因此，对于压缩比为 m 的 CSA 与压缩比为 m' 的 HCA，每块的原始 token 数可以是 lcm(m, m')（两个压缩比的最小公倍数）的任意倍数。

3.5.2 磁盘上的 KV 缓存存储

在服务 DeepSeek-V4 时，我们利用磁盘上的 KV 缓存存储机制来消除共享前缀请求的重复预填充。对于 CSA/HCA 中的压缩 KV 条目与滑动窗口注意力（SWA）中的未压缩 KV 条目，我们设计了各自的存储管理方案。

对于 CSA 与 HCA，我们直接将所有压缩 KV 条目存储到磁盘。当请求命中已存储的前缀时，我们读取并复用与该前缀对应的压缩 KV 条目，直到最后一个完整的压缩块。特别地，对于尾部不完整

块中的前缀 token，我们仍需重计算它们以恢复未压缩 KV 条目，因为 CSA 与 HCA 中的未压缩 KV 条目并未存储。

对于 SWA KV 条目，由于它们未被压缩且存在于每一层，其体量约为压缩 CSA 与 HCA KV 条目的 8 倍。为高效处理这些庞大的 SWA KV 条目，我们提出并实现了三种不同的磁盘 SWA KV 条目管理策略，每种策略在存储开销与计算冗余之间提供不同的权衡：

- **完整 SWA 缓存**：该策略存储所有 token 的完整 SWA KV 条目，确保计算零冗余。在该策略下，命中前缀的 SWA KV 条目只需读取该前缀内最后 n_win 个 token 的磁盘缓存即可重建。尽管计算零冗余，但该策略对现代基于 SSD 的存储系统效率不高——每个命中请求只会访问所存 SWA KV 缓存的一小部分，导致写密集且不均衡的访问模式。
- **周期性检查点**：该策略每 p 个 token 对最后 n_win 个 token 的 SWA KV 条目做一次检查点，其中 p 是可调参数。对于命中的前缀，我们加载最近一次检查点状态，然后重计算剩余的尾部 token。通过调节 p，该策略可在存储与计算之间按需权衡。
- **零 SWA 缓存**：该策略不存储任何 SWA KV 条目。对于命中的前缀，我们需要执行更多重计算来恢复 SWA KV 条目。具体而言，在每个注意力层中，每个 token 的 SWA KV 条目仅依赖上一层最近 n_win 个 token 的 SWA KV 条目。因此，利用已缓存的 CSA 与 HCA KV 条目，重计算最后 n_win·L 个 token 就足以为一个 L 层模型恢复最后 n_win 个 SWA KV 条目。

  根据具体的部署场景，我们选择最合适的策略，以在存储与计算之间实现所需的权衡。

4 预训练

4.1 数据构建

在 DeepSeek-V3 的预训练数据基础上，我们致力于构建更多样、更高质量且具有更长有效上下文的训练语料。我们持续改进数据构建流水线。对于来自网络的数据，我们采用过滤策略去除批量自动生成与模板化的内容，从而降低模型崩塌（model collapse）的风险（Zhu 等，2024）。数学与编程语料仍是训练数据的核心组成部分，我们还在中期训练（mid-training）阶段引入智能体（agentic）数据，以进一步增强 DeepSeek-V4 系列的编码能力。对于多语言数据，我们为 DeepSeek-V4 构建了更大的语料，提升其对不同文化中长尾知识的捕捉能力。对于 DeepSeek-V4，我们特别重视长文档数据的整理，优先选择科学论文、技术报告以及其他体现独特学术价值的材料。综合上述所有来源，我们的预训练语料包含超过 32T token，涵盖数学内容、代码、网页、长文档以及其他高质量类别。

对于预训练数据，我们大体沿用 DeepSeek-V3 的预处理策略。在分词方面，我们在 DeepSeek-V3 分词器的基础上引入了若干用于上下文构建的特殊 token，词表大小仍保持为 128K。我们还继承了 DeepSeek-V3 的 token 切分（token-splitting；DeepSeek-AI，2024）与填充中间（Fill-in-Middle，FIM；DeepSeek-AI，2024）策略。受 Ding 等（2024）启发，我们将来自不同来源的文档打包成合适的序列，以尽量减少样本截断。与 DeepSeek-V3 不同，我们在预训练中采用样本级注意力掩码（sample-level attention masking）。

4.2 预训练设置

4.2.1 模型设置

DeepSeek-V4-Flash 我们将 Transformer 层数设为 43，隐藏维度 d 设为 4096。前两层使用纯滑动窗口注意力。对于后续层，CSA 与 HCA 交替使用。对于 CSA，我们将压缩率 m 设为 4，索引器查询头数 n_Ih 设为 64，索引器头维度 c_I 设为 128，用于稀疏注意力的 KV 条目数（即 attention top-k）设为 512。对于 HCA，我们将压缩率 m′ 设为 128。对于 CSA 与 HCA，我们将查询头数 n_h 设为 64，头维度 c 设为 512，查询压缩维度 d_c 设为 1024。输出投影组数 g 设为 8，每个中间注意力输出的维度 d_g 设为 1024。对于滑动窗口注意力的附加分支，窗口大小 n_win 设为 128。我们在所有 Transformer block 中使用 MoE 层，但对前 3 个 MoE 层采用 Hash 路由策略。每个 MoE 层由 1 个共享专家与 256 个路由专家组成，每个专家的中间隐藏维度为 2048。在路由专家中，每个 token 会激活 6 个专家。多 token 预测（MTP）深度设为 1。对于 mHC，扩展因子 n_hc 设为 4，Sinkhorn-Knopp 迭代次数 t_max 设为 20。在该配置下，DeepSeek-V4-Flash 总参数为 284B，其中每个 token 激活 13B。

DeepSeek-V4-Pro 我们将 Transformer 层数设为 61，隐藏维度 d 设为 7168。前两层使用 HCA。对于后续层，CSA 与 HCA 交替使用。对于 CSA，我们将压缩率 m 设为 4，索引器查询头数 n_Ih 设为 64，索引器头维度 c_I 设为 128，用于稀疏注意力的 KV 条目数（即 attention top-k）设为 1024。对于 HCA，我们将压缩率 m′ 设为 128。对于 CSA 与 HCA，我们将查询头数 n_h 设为 128，头维度 c 设为 512，查询压缩维度 d_c 设为 1536。输出投影组数 g 设为 16，每个中间注意力输出的维度 d_g 设为 1024。对于滑动窗口注意力的附加分支，窗口大小 n_win 设为 128。我们在所有 Transformer block 中使用 MoE 层，但对前 3 个 MoE 层采用 Hash 路由策略。每个 MoE 层由 1 个共享专家与 384 个路由专家组成，每个专家的中间隐藏维度为 3072。在路由专家中，每个 token 会激活

6 个专家。多 token 预测深度设为 1。对于 mHC，扩展因子 n_hc 设为 4，Sinkhorn-Knopp 迭代次数 t_max 设为 20。在该配置下，DeepSeek-V4-Pro 总参数为 1.6T，其中每个 token 激活 49B。

4.2.2 训练设置

DeepSeek-V4-Flash 我们对大多数参数采用 Muon 优化器（Jordan 等，2024；Liu 等，2025），但对嵌入模块、预测头模块以及所有 RMSNorm 模块的权重使用 AdamW 优化器（Loshchilov 和 Hutter，2017）。对于 AdamW，我们将超参数设为 $\beta1 = 0.9$、$\beta2 = 0.95$、$\varepsilon = 10^{-20}$、weight_decay = 0.1。对于 Muon，我们将动量设为 0.95、权重衰减设为 0.1，并将每个更新矩阵的 RMS 重新缩放到 0.18，以复用 AdamW 的学习率。我们在 32T token 上训练 DeepSeek-V4-Flash，并像 DeepSeek-V3 一样采用批大小调度策略：将批大小（以 token 计）从小值逐步增大到 75.5M，并在训练的大部分时间保持在 75.5M。学习率在前 2000 步线性预热，训练的大部分时间保持为 2.7×10^{-4}。接近训练结束时，我们按余弦调度将学习率最终衰减到 2.7×10^{-5}。训练从 4K 的序列长度开始，随后逐步扩展到 16K、64K 与 1M。关于稀疏注意力的设置，我们先用稠密注意力对前 1T token 进行预热，并在 64K 序列长度时引入稀疏注意力，此后训练全程保持稀疏注意力。引入注意力稀疏性时，我们先设置一个短阶段预热 CSA 中的闪电索引器（lightning indexer），然后在训练的大部分时间用稀疏注意力训练模型。对于无辅助损失（auxiliary-loss-free）的负载均衡，我们将偏置更新速度设为 0.001。对于均衡损失，我们将其损失权重设为 0.0001，以避免单条序列内部出现极端不均衡。MTP 损失权重在训练的大部分时间设为 0.3，在开始学习率衰减后设为 0.1。

DeepSeek-V4-Pro 除超参数的具体取值外，DeepSeek-V4-Pro 的训练设置与 DeepSeek-V4-Flash 基本一致。我们对大多数参数采用 Muon 优化器，但对嵌入模块、预测头模块以及所有 RMSNorm 模块的权重使用 AdamW 优化器。AdamW 与 Muon 的超参数与 DeepSeek-V4-Flash 相同。我们在 33T token 上训练 DeepSeek-V4-Pro，同样采用批大小调度策略，最大批大小为 94.4M token。学习率调度策略与 DeepSeek-V4-Flash 基本一致，但峰值学习率设为 2.0×10^{-4}，结束学习率设为 2.0×10^{-5}。训练同样从 4K 序列长度开始，随后逐步扩展到 16K、64K 与 1M。与 DeepSeek-V4-Flash 相比，DeepSeek-V4-Pro 以更长的稠密注意力阶段开始，引入稀疏注意力的策略与 DeepSeek-V4-Flash 相同，遵循两阶段训练方法。对于无辅助损失的负载均衡，我们将偏置更新速度设为 0.001。对于均衡损失，我们将其损失权重设为 0.0001，以避免单条序列内部出现极端不均衡。MTP 损失权重在训练的大部分时间设为 0.3，在开始学习率衰减后设为 0.1。

4.2.3 缓解训练不稳定

训练万亿参数级 MoE 模型会带来显著的稳定性挑战，DeepSeek-V4 系列也不例外。我们在训练中遇到了明显的不稳定问题。虽然简单的回滚（rollback）可以暂时恢复训练状态，但作为长期方案并不充分，因为它们无法防止损失尖峰（loss spike）再次出现。从经验上看，我们发现尖峰的出现始终与 MoE 层中的离群值（outlier）相关，而路由机制本身似乎会加剧这些离群值的产生。因此，我们试图从两个维度解决该问题：打破路由引发的恶性循环，以及直接抑制异常值。幸运的是，我们发现了两种有效维持训练稳定性的实用技术。尽管对其内在机制的完整理论理解目前仍是一个开放问题，我们仍公开分享它们，以促进社区的进一步探索。

预期路由（Anticipatory Routing） 我们发现，将主干网络与路由网络的同步更新解耦能显著提升训练稳定性。因此，在第 t 步，我们使用当前网络参数 θ_t 进行特征计算，但路由索引则使用历史网络参数 $\theta_{t-\Delta t}$ 计算并应用。在实践中，为规避两次加载模型参数的开销，我们在第 $t-\Delta t$ 步提前获取第 t 步的数据，并“预先”计算并缓存将在第 t 步使用的路由索引——这也是我们将该方法命名为 Anticipatory Routing 的原因。我们还在基础设施层面做了大量优化。首先，鉴于预计算路由索引只需对数据进行一次前向计算，我们仔细编排了流水线执行以及计算与专家并行（Expert Parallelism，EP）通信的重叠，成功将 Anticipatory Routing 的额外墙钟时间开销限制在约 20%。其次，我们引入了自动检测机制：仅在出现损失尖峰时触发短暂回滚并启用 Anticipatory Routing；在这种模式下运行一段时间后，系统会恢复标准训练。最终，这种动态应用使我们能够以可忽略的总体额外训练开销避免损失尖峰，且不损害模型性能。

SwiGLU 截断（SwiGLU Clamping） 在既有文献中（Bello 等，2017；Riviere 等，2024），截断（clamping）已被明确用于约束数值范围，从而增强训练稳定性。在实际训练运行中，我们从经验上发现，应用 SwiGLU 截断（OpenAI，2025）能有效消除离群值，并显著帮助稳定训练过程，且不损害性能。在 DeepSeek-V4-Flash 与 DeepSeek-V4-Pro 的整个训练过程中，我们将 SwiGLU 的线性分量截断到 $[-10, 10]$ 区间，同时将门控分量的上界限制为 10。

4.3 评测

4.3.1 评测基准

对于基础模型的评测，我们考虑了覆盖四个关键维度的基准：世界知识、语言理解与推理、编码与数学，以及长上下文处理。

世界知识基准包括 AGIEval（Zhong 等，2023）、C-Eval（Huang 等，2023）、CMMLU（Li 等，2023）、MMLU（Hendrycks 等，2020）、MMLU-Redux（Gema 等，2024）、MMLU-Pro（Wang 等，2024b）、MMMLU（OpenAI，2024a）、MultiLoKo（Hupkes 和 Bogoychev，2025）、Simple-QA verified（Haas 等，2025）、SuperGPQA（Du 等，2025）、FACTS Parametric（Cheng 等，2025）与 TriviaQA（Joshi 等，2017）。

语言理解与推理基准包括 BigBench Hard（BBH）（Suzgun 等，2022）、DROP（Dua 等，2019）、HellaSwag（Zellers 等，2019）、CLUEWSC（Xu 等，2020）与 WinoGrande（Sakaguchi 等，2019）。

编码与数学基准包括 BigCodeBench（Zhuo 等，2025）、HumanEval（Chen 等，2021）、GSM8K（Cobbe 等，2021）、MATH（Hendrycks 等，2021）、MGSM（Shi 等，2023）与 CMath（Wei 等，2023）。

长上下文基准包括 LongBench-V2（Bai 等，2025b）。

4.3.2 评测结果

在表 1 中，我们给出了 DeepSeek-V3.2、DeepSeek-V4-Flash 与 DeepSeek-V4-Pro 基础模型的详细对比，所有模型均在统一的内部框架下、以严格一致的设置进行评测。

将 DeepSeek-V4-Flash-Base 与 DeepSeek-V3.2-Base 对比，可以看到一个颇具说服力的效率故事。尽管激活参数与总参数都显著更少，DeepSeek-V4-Flash-Base 在大量基准上优于 DeepSeek-V3.2-Base。这一优势在世界知识任务以及具有挑战性的长上下文场景中尤为明显。这些结果表明，DeepSeek-V4-Flash-Base 的架构改进、数据质量提升与训练优化，使其即便在更紧凑的参数预算下也能取得更优性能，在多数评测上有效超越了规模更大的 DeepSeek-V3.2-Base。

此外，DeepSeek-V4-Pro-Base 展现出更进一步、具有决定性的能力跃升，对 DeepSeek-V3.2-Base 与 DeepSeek-V4-Flash-Base 形成了近乎全面的领先。DeepSeek-V4-Pro-Base 在几乎所有类别上都有提升，在最严苛的基准上创下 DeepSeek 基础模型的新高。在知识密集型评测中，它带来大幅增益，同时也显著推进了长上下文理解。在多数推理与代码基准上，DeepSeek-V4-Pro-Base 也超过了此前的两个模型。这种全面的提升确认了 DeepSeek-V4-Pro-Base 是 DeepSeek 系列中最强的基础模型，在知识、推理、编码与长上下文能力的整个谱系上均优于其前代。

【表格】表 1 DeepSeek-V3.2-Base、DeepSeek-V4-Flash-Base 与 DeepSeek-V4-Pro-Base 之间的对比。所有模型均在我们的内部框架中评测、共享相同评测设置。差距不超过 0.3 的分数视为同一水平。每行中最高分以粗体标出，次高者

以下划线标出。（原文第 28 页）

5 后训练

5.1 后训练流水线

预训练之后，我们进行了后训练阶段，以得到 DeepSeek-V4 系列的最终模型。尽管训练流水线大体沿用 DeepSeek-V3.2，但做了一项关键的方法替换：混合强化学习（RL）阶段被完全替换为在线策略蒸馏（On-Policy Distillation，OPD；Gu 等，2024；Lu 和 Lab，2025）。

5.1.1 专家训练

领域专家模型的开发通过调整 DeepSeek-V3.2 的训练流水线完成。具体而言，每个模型依次经过初始微调阶段以及随后由领域特定提示与奖励信号引导的强化学习（RL）阶段进行优化。在 RL 阶段，我们实现了组相对策略优化（Group Relative Policy Optimization，GRPO）算法，超参数与我们此前的研究保持高度一致（DeepSeek-AI，2025；DeepSeek-AI，2025）。

推理强度（Reasoning Efforts） 人们普遍认为，模型在推理任务上的表现从根本上取决于所投入的计算量。因此，我们在不同的 RL 配置下训练了不同的专家模型，以形成针对不同推理能力优化的模型。如表 2 所示，DeepSeek-V4-Pro 与 DeepSeek-V4-Flash 均支持三种特定的推理强度模式。对每种模式，我们在 RL 训练中施加不同的长度惩罚与上下文窗口，从而产生不同的推理输出 token 长度。为整合这些不同的推理模式，我们使用由 `<think>` 与 `</think>` token 界定的专用响应格式。此外，对于“Think Max”模式，我们在系统提示开头前置一条特定指令来引导模型的推理过程，如表 3 所示。

推理模式	特点	典型用例	响应格式
Non-think	快速、基于习惯或简单规则的直觉式响应。	日常例行任务、紧急反应、低风险决策。	`</think>` 摘要
Think High	有意识的逻辑分析，较慢但更准确。	复杂问题求解、规划、中等风险决策。	`<think>` 思考 token `</think>` 摘要
Think Max	将推理推到极致。慢但强大。	探索模型推理能力的边界。	1. 开头注入特殊的系统提示；2. `<think>` 思考 token `</think>` 摘要

表 2 | 三种推理模式的对比。

表 3 | 注入到“Think Max”模式的系统提示中的指令。

> 推理强度：绝对最大化，不允许任何捷径。>> 你必须在思考中极为详尽，全面分解问题以解决根本原因，针对所有可能的路径、边界情形与对抗性场景严格压力测试你的逻辑。>> 明确写出你完整的斟酌过程，记录每一个中间步骤、考虑过的替代方案以及被否定的假设，以确保没有任何假设未经检验。

生成式奖励模型（Generative Reward Model） 通常，易于验证的任务可以使用简单的基于规则的验证器或测试用例进行有效优化。相比之下，难以验证的任务传统上依赖基于人类反馈的强化学习（RLHF），这需要大量人工标注来训练标量奖励模型。然而，在 DeepSeek-V4 系列的后训练阶段，我们舍弃了这些传统的基于标量的奖励模型。相反，为处理难以验证的任务，我们策划了以评分细则（rubric）引导的 RL 数据，并采用生成式奖励模型（Generative Reward Model，GRM）来评估策略轨迹。关键在于，我们直接对 GRM 本身施加 RL 优化。在这一范式中，actor 网络本身即充当 GRM，使模型的评估（评判）能力与标准生成能力能够联合优化。通过统一这两类角色，模型内部的推理能力被内在地融入其评估过程，从而得到高度稳健的评分。此外，该方法仅需少量多样的人工标注即可取得优异表现，因为模型会利用自身的逻辑在复杂任务间泛化。

表 4 | DeepSeek-V4 系列的工具调用模式（Tool-call schema）。

```
## Tools
You have access to a set of tools to help answer the user’s question. You can
invoke tools by writing a "<|DSML|tool_calls>" block like the following:
<|DSML|tool_calls>
<|DSML|invoke name="$TOOL_NAME">
<|DSML|parameter name="$PARAMETER_NAME" string="true|false">$PARAMETER_VALUE
</|DSML|parameter>
...
</|DSML|invoke>
<|DSML|invoke name="$TOOL_NAME2">
...
</|DSML|invoke>
</|DSML|tool_calls>
String parameters should be specified as is and set ‘string="true"‘. For all
other types (numbers, booleans, arrays, objects), pass the value in JSON
format and set ‘string="false"‘.
If thinking_mode is enabled (triggered by <think>), you MUST output your
complete reasoning inside <think>...</think> BEFORE any tool calls or
final response.
Otherwise, output directly after </think> with tool calls or final response.
### Available Tool Schemas
{Tool Definition...}
You MUST strictly follow the above definedtool name and parameter schemas to
invoke tool calls.
```

工具调用模式与特殊 token 与此前版本一致，我们使用专用的 `<think></think>` 标签来界定推理路径。在 DeepSeek-V4 系列中，我们引入了新的工具调用模式：采用特殊的“|DSML|”token，并使用基于 XML 的格式进行工具调用，如表 4 所示。我们的实验表明，XML 格式能有效缓解转义失败并减少工具调用错误，为模型与工具的交互提供了更稳健的接口。

【图表】图 7 DeepSeek-V4 系列的思考管理：（a）带工具时的思考；（b）不带工具时的思考。（原文第 31 页）

交错思考（Interleaved Thinking） DeepSeek-V3.2 引入了一种上下文管理策略，在工具结果轮次之间保留推理痕迹，但在新用户消息到来时将其丢弃。该策略虽然有效，但在复杂的智能体工作流中仍造成不必要的 token 浪费——每个新的用户轮次都会清空所有累积的推理内容，迫使模型从头重建其问题求解状态。借助 DeepSeek-V4 系列扩展的 1M token 上下文窗口，我们进一步改进了这一机制，以在智能体环境中最大化交错思考的效果：

- **工具调用场景**：如图 7（a）所示，所有推理内容在整个对话过程中被完整保留。与 DeepSeek-V3.2 在每个新用户轮次丢弃思考痕迹不同，DeepSeek-V4 系列在所有轮次间保留完整的推理历史，包括跨越用户消息边界。这使模型能够在长程智能体任务中维持连贯、累积的思维链。
- **一般对话场景**：如图 7（b）所示，原策略得以保留：当新用户消息到来时丢弃此前轮次的推理内容，从而在持久推理痕迹收益有限的场景中保持上下文简洁。

与 DeepSeek-V3.2 相同，通过用户消息模拟工具交互的智能体框架（例如 Terminus）可能不会触发工具调用的上下文路径，因而可能无法受益于增强的推理持久性。我们仍推荐此类架构使用 non-think 模型。

【表格】表 5 用于辅助任务的 Quick Instruction 特殊 token。（原文第 32 页）

Quick Instruction 在聊天机器人场景中，在生成响应之前必须执行若干辅助任务（例如判断是否触发网页搜索、意图识别等）。传统上，这些任务由单独的小模型处理，由于无法复用已有的 KV cache 而需要冗余的预填充（prefill）。为克服这一限制，我们引入了 Quick Instruction。我们直接在输入序列后附加一组专用特殊 token，每个 token 对应一个特定的辅助任务。通过直接复用已计算好的 KV cache，该机制完全避免了冗余预填充，并允许某些任务（如生成搜索查询、判定权威性与领域）并行执行。因此，该方法显著降低了用户感知的首 token 延迟（time-to-first-token，TTFT），并消除了维护与迭代额外小模型所带来的工程开销。所支持的 Quick Instruction token 汇总于表 5。

5.1.2 在线策略蒸馏

在通过专门的微调与强化学习训练多个领域专家模型之后，我们采用多教师在线策略蒸馏（On-Policy Distillation，OPD；Gu 等，2024；Lu 和 Lab，2025）作为将专家能力合并到最终模型的主要技术。OPD 已成为一种有效的后训练范式，可高效地将领域专家的知识与能力迁移到单一的统一模型。其做法是让学生模型在自己的生成轨迹上学习教师模型的输出分布。形式化地，给定 N 个专家模型组成的集合 {π_E1, π_E2, ..., π_EN}，OPD 的目标函数定义为：

$$L_{OPD}(\theta) = \sum_{i=1}^{N} w_i \cdot D_{KL}(\pi_\theta \| \pi_{Ei}) \quad (29)$$

在该公式中，w_i 表示分配给每个专家的权重，通常由该专家的相对重要性决定。计算反向 KL 损失 D_KL(π_θ ‖ π_Ei) 需要从学生模型 π_θ 采样训练轨迹，以保持在线策略学习。其内在逻辑是确保统一策略 π_θ 有选择地从与当前任务上下文相关的专家那里学习（例如，数学推理任务与数学专家对齐，编程任务与编码专家对齐）。通过这一机制，来自物理上彼此独立的专家权重中的知识，经由 logits 层面的对齐被整合进统一的参数空间，从而在实践中规避了传统权重合并或混合 RL 技术中常见的性能退化。在这一阶段，我们使用覆盖不同领域的十多个教师模型来蒸馏单个学生模型。

在处理上述 OPD 目标时，先前工作通常将全词表 KL 损失简化为每个 token 位置上的 token 级 KL 估计，并复用 RL 框架，将 sg[log(π_Ei(y_t|x, y_<t) / π_θ(y_t|x, y_<t))]（sg 表示停止梯度操作）作为策略损失计算中的逐 token 优势估计。尽管这种方法在资源上高效，但会导致梯度估计的高方差，并常常引起训练不稳定。因此，我们在 OPD 中采用全词表的 logit 蒸馏。在计算反向 KL 损失时保留完整的 logit 分布，可获得更稳定的梯度估计，并确保对教师知识的忠实蒸馏。在下一小节中，我们描述使全词表 OPD 能够规模化可行的工程工作。

5.2 后训练基础设施

我们的后训练基础设施构建在为 DeepSeek-V3.2 开发的可扩展框架之上。具体而言，我们集成了第 3.4 节所述的同一套分布式训练栈，以及此前引入的、用于高效自回归采样的 rollout 引擎。在此基础上，本工作引入了以下主要增强。这些设计使涉及十多个不同教师模型的超长上下文 RL 与 OPD 合并任务得以高效执行，从而显著加快模型发布的迭代周期。

5.2.1 FP4 量化感知训练

为在部署时实现推理加速并减少内存流量，我们在后训练阶段引入了量化感知训练（Quantization-Aware Training，QAT）（Jacob 等，2018），使模型（包括教师模型与参考模型）能够适应量化带来

的精度下降。我们对两个组件应用 FP4（MXFP4）量化（Rouhani 等，2023）：（1）MoE 专家权重，这是 GPU 内存占用的主要来源之一（OpenAI，2025）；（2）CSA 索引器中的 Query-Key（QK）路径——其中 QK 激活完全以 FP4 进行缓存、加载与相乘，从而加速长上下文场景下的注意力分数计算。此外，在该 QAT 过程中，我们进一步将索引分数 I_{:,:} 从 FP32 量化为 BF16。该优化使 top-k 选择器获得 2 倍加速，同时保持 99.7% 的 KV 条目召回率。

对于 MoE 专家权重，遵循 QAT 的常见做法，优化器维护的 FP32 主权重首先被量化为 FP4，随后反量化回 FP8 进行计算。值得注意的是，我们的 FP4→FP8 反量化是无损的。这是因为与 FP4（E2M1）相比，FP8（E4M3）多出 2 个指数位，提供更大的动态范围。因此，只要每个 FP8 量化块（128×128 tile）内 FP4 子块（1×32 tile）的最大与最小缩放因子之比不超过某一阈值，细粒度的缩放信息就能被 FP8 扩展后的动态范围完全吸收。我们通过实验验证当前权重满足这一条件。这使得整个 QAT 流水线无需任何修改即可完全复用现有的 FP8 训练框架。在反向传播中，梯度基于与前向相同的 FP8 权重计算，并直接回传到 FP32 主权重，相当于在量化操作上应用直通估计器（Straight-Through Estimator，STE）。这也避免了对转置权重重新量化的需要。

在 RL 训练的推理与 rollout 阶段（不涉及反向传播），我们直接使用原生 FP4 量化权重，而非模拟量化。这确保采样期间的模型行为与线上部署完全一致，同时减少 kernel 的内存加载以实现实际加速，并显著降低内存消耗。我们对 CSA 索引器中的 QK 路径也做同样处理。

5.2.2 全词表 OPD 的高效教师调度

我们的框架支持全词表在线策略蒸馏（OPD），教师数量实际上不受限制，且每个教师模型都可能包含数万亿参数。为实现这一点，所有教师权重被卸载到集中式分布式存储中，并在教师前向计算期间按需加载，同时采用类 ZeRO 的参数分片以缓解 I/O 与 DRAM 压力。此外，在词表规模 $|V| > 100k$ 的情况下，为所有教师朴素地物化（materialize）logits 是不可行的，即便转存到磁盘也一样。我们的解决方案是：在前向计算期间只将最后一层的教师隐藏状态缓存到集中式缓冲区中。训练时，取出这些缓存状态并通过对应的预测头模块，即时重建完整的 logits。这一设计带来的重计算开销可忽略，同时完全规避了显式物化 logits 所带来的内存负担。为减轻教师预测头对 GPU 内存的占用，我们在数据分发时按教师索引对训练样本排序。这一安排确保每个不同的教师头在每个小批次中只加载一次，且任一时刻设备内存中至多驻留一个教师头。所有参数与隐藏状态的加载/卸载操作都在后台异步进行，不会阻塞关键路径上的计算。最后，教师与学生 logits 之间的精确 KL 散度使用专门的 TileLang kernel 计算，从而加速计算并减少动态内存分配。

5.2.3 可抢占且容错的 rollout 服务

为在最大化 GPU 资源利用率的同时，为高优先级任务提供快速的硬件供给，我们的 GPU 集群采用集群级的抢占式任务调度器，任何正在运行的任务都可能随时被抢占。此外，在大规模 GPU 集群中硬件故障十分普遍。为此，我们实现了一个用于 RL/OPD rollout 的可抢占且容错的 LLM 生成服务。

具体而言，我们为每个生成请求实现了一个 token 粒度的预写日志（Write-Ahead Log，WAL）。每当为某个请求生成新的 token，我们立即将其追加到该请求的 WAL 中。发生抢占时，我们暂停推理引擎并保存未完成请求的 KV cache。恢复时，我们使用持久化的 WAL 与保存的 KV cache 继续解码。即便发生致命的硬件错误，我们也可以利用 WAL 中持久化的 token 重新执行 prefill 阶段，以重建 KV cache。

重要的是，从头重新生成未完成的请求在数学上是不正确的，因为这会引入长度偏差（length bias）。由于较短的响应更可能挺过中断，从头重新生成会使模型更倾向于产生更短的序列，每当发生中断时。

若推理栈是批不变（batch-invariant）且确定性的，这一正确性问题也可以通过使用一致的随机数种子重新生成来解决——该种子用于采样器中的伪随机数生成器。然而，这种做法仍需承担重新运行解码阶段的额外开销，其效率远低于我们的 token 粒度 WAL 方法。

5.2.4 面向百万 token 上下文的 RL 框架扩展

我们针对百万 token 序列上高效的 RL 与 OPD 引入了定向优化。在 rollout 阶段，我们采用可抢占且容错的 rollout 服务，详见第 5.2.3 节。在推理与训练阶段，我们将 rollout 数据格式分解为轻量级元数据与体量较大的逐 token 字段。在数据分发时，可加载整个 rollout 数据的元数据，以执行全局打乱与打包布局计算。体量较大的逐 token 字段通过共享内存数据加载器加载，以消除节点内的数据冗余，并在 mini-batch 粒度上被消费后立即释放，从而大幅降低 CPU 与 GPU 的内存压力。设备端 mini-batch 的数量根据工作负载动态确定，从而在计算吞吐与 I/O 重叠之间实现高效权衡。

5.2.5 面向智能体 AI 的沙箱基础设施

为满足智能体（agentic）AI 在后训练与评测过程中多样化的执行需求，我们构建了一个生产级沙箱平台 **DeepSeek Elastic Compute（DSec）**。DSec 由三个 Rust 组件构成——API 网关（Apiserver）、单机代理（Edge）与集群监控器（Watcher）——它们通过定制的 RPC 协议互联，并基于 3FS 分布式文件系统（DeepSeek-AI, 2025）横向扩展。在生产环境中，单个 DSec 集群可管理数十万个并发沙箱实例。

DSec 的设计源于四点观察：（1）智能体工作负载高度异构，从轻量级函数调用到完整的软件工程流水线，且对操作系统与安全性的要求各不相同；（2）环境镜像数量众多、体积庞大，却必须快速加载并支持迭代式定制；（3）高密度部署要求高效的 CPU 与内存利用率；（4）沙箱生命周期必须与 GPU 训练调度相协调，包括抢占与基于检查点的恢复。基于这些观察，我们将在下文逐一阐述 DSec 的四项核心设计。

四个执行基座，统一接口。DSec 提供单一的 Python SDK（libdsec），抽象出四种执行基座。**Function Call** 将无状态调用分发到预热好的容器池，消除冷启动开销。**Container** 完全兼容 Docker，并利用 EROFS（Gao et al., 2019）的按需加载实现高效的镜像组装。**microVM** 基于 Firecracker（Agache et al., 2020），为安全敏感的高密度部署增加 VM 级隔离。**fullVM** 基于 QEMU（Bellard, 2005），支持任意客户机操作系统。四者共享统一的 API 面——命令执行、文件传输与 TTY 访问——在它们之间切换只需修改一个参数。

通过分层存储实现快速镜像加载。DSec 通过分层、按需加载，在快速启动与庞大且不断增长的环境镜像语料之间取得协调。对于容器，基础镜像与文件系统提交以 3FS 为后端的只读 EROFS 层存储，并直接挂载到 overlay lowerdir 中。挂载时我们将文件元数据保存在本地磁盘上以便随时可用；同时，数据块在请求时才从 3FS 获取。对于 microVM，DSec 采用 overlaybd（Li et al., 2020）磁盘格式：只读基础层位于 3FS 上以供跨实例共享，而写入则进入本地写时复制（copy-on-write）层。此类快照可链式叠加，便于高效版本管理与毫秒级恢复。

大规模并发下的密度优化。为在单集群内容纳数十万个沙箱，DSec 着力缓解两类资源瓶颈。第一，它减少虚拟化环境中重复的页缓存占用，并应用内存回收以支持安全的超配（overcommit）。第二，它缓解容器运行时中的自旋锁竞争，从而降低单个沙箱的 CPU 开销，显著提升单机打包密度。

轨迹日志与抢占安全的恢复。DSec 为每个沙箱维护全局有序的轨迹日志，持久记录每次命令调用及其结果。该轨迹有三方面用途：（1）**客户端快进（client fast-forwarding）**——当训练任务被抢占时，沙箱资源仍被保留；恢复时，DSec 会重放已完成命令的缓存结果，加快任务恢复，同时避免因重新执行非幂等操作而产生错误；（2）**细粒度溯源（fine-grained provenance）**——每次状态变更的来源与相应结果均可追溯；（3）**确定性重放（deterministic replay）**——任意历史会话都可依据其轨迹被忠实复现。

5.3 标准基准评测

5.3.1 评测设置

知识与推理。知识与推理数据集包括 MMLU-Pro（Wang et al., 2024b）、GPQA（Rein et al., 2023）、Human Last Exam（Phan et al., 2025）、SimpleQA Verified（Haas et al., 2025）、Chinese-SimpleQA（He et al., 2024）、LiveCodeBench-v6（Jain et al., 2024）、CodeForces（内部基准）、HMMT 2026 Feb、Apex（Balunović et al., 2025）、Apex Shortlist（Balunović et al., 2025）、IMOAnswerBench（Luong et al., 2025）与 PutnamBench（Tsoukalas et al., 2024）。

在代码方面，我们在 LiveCodeBench-v6 与内部 Codeforces 基准上评测 DeepSeek-V4 系列。对于 Codeforces，我们收集了 14 场 Codeforces Division 1 比赛，共 114 道题（2025 年 5 月至 2025 年 11 月）。Elo 等级分按如下方式计算。对每场比赛，我们为每道题生成 32 个候选解答。对每道题独立地，我们从这 32 个解答中不放回地抽取 10 个，并以随机顺序排列，构成提交序列。每次提交都依据领域专家构建的测试集进行判定。已解出题目的得分遵循 OpenAI（2025）的罚时方案：模型获得与那些以相同次数先前失败尝试解出同一题的人类参赛者得分的中位数。由此得到每个抽样提交序列的比赛总分，再转换为比赛排名，并进一步通过标准 Codeforces 等级分系统转换为估计等级分。比赛级期望等级分定义为：在每道题 10 次提交的所有可能随机选择与排序上的估计等级分的期望。模型的总体等级分为全部 14 场比赛中这些比赛级期望等级分的平均值。

对于推理与知识任务，我们将温度设为 1.0，并将上下文窗口分别设为 8K、128K 与 384K token，对应 Non-think、High 与 Max 模式。对于数学任务（如 HMMT、IMOAnswerBench、Apex 与 HLE），我们使用如下模板进行评测：

```
{question}\nPlease reason step by step, and put your final answer within \boxed{}.
```

对于 DeepSeek-V4-Pro-Max 在数学任务上的评测，我们使用如下模板以激发更深入的推理：

```
Solve the following problem. The problem may ask you to prove a statement, or ask for an answer. If finding an answer is required, you should come up with the answer, and your final solution should also be a rigorous proof of that answer being valid.\n\n{question}
```

对于形式化数学任务，我们在 Lean v4.28.0-rc1（Moura and Ullrich, 2021）上以智能体（agentic）设置进行评测，可访问 Lean 编译器与语义 tactic 搜索引擎，最多执行 500 次工具调用并使用最大推理强度。此外，我们评测了一条计算更密集的流水线：首先生成候选自然语言解答，并通过自验证（Shao et al., 2025）进行筛选，随后将保留的解答作为引导提供给形式化智能体，以证明相应的 Lean

命题。该设计利用非形式化推理来改进探索，同时通过形式化验证保持严格的正确性。只有在两种设置下严格验证器 Comparator 均接受时，提交才被计为正确。

对于 K2.6 与 GLM-5.1，我们留空了一些条目，因为它们各自的 API 过于繁忙，无法对我们的查询返回响应。

百万 token 上下文。由于 DeepSeek-V4 系列支持百万 token 上下文，我们通过选择 OpenAI MRCR（OpenAI, 2024b）与 CorpusQA（Lu et al., 2026）作为基准，来评测模型在长上下文场景中的性能。我们重新评测了 Claude Opus 4.6 与 Gemini 3.1 Pro 在这些任务上的表现，目标是统一所有模型的配置。我们没有评测 GPT-5.4，因为其 API 未能对我们的大部分查询作出响应。

智能体（Agent）。智能体数据集包括 Terminal Bench 2.0（Merrill et al., 2026）、SWE-Verified（OpenAI, 2024e）、SWE Multilingual（Yang et al., 2025）、SWE-Pro（Deng et al., 2025）、BrowseComp（Wei et al., 2025）、MCPAtlas 的公开评测集（Bandi et al., 2026）、GDPval-AA（AA, 2025; Patwardhan et al., 2025）与 Tool-Decathlon（Li et al., 2025）。

对于代码智能体任务（SWE-Verified、Terminal-Bench、SWE-Pro、SWE Multilingual），我们使用内部开发的评测框架评测 DeepSeek-V4 系列。该框架提供最小工具集——一个 bash 工具与一个文件编辑工具。最大交互步数设为 500，最大上下文长度设为 512K token。关于 Terminal-Bench 2.0，我们注意到 GLM-5.1 所指出的环境相关问题。尽管如此，为保持一致性，我们报告在原始 Terminal-Bench 2.0 数据集上的性能。在 Terminal-Bench 2.0 Verified 子集上，DeepSeek-V4-Pro 取得约 72.0 的分数。

对于搜索智能体任务（BrowseComp、HLE w/ tool），我们同样使用内部 harness，配备网页搜索与 Python 工具，并将最大交互步数设为 500、最大上下文长度设为 512K token。对于 BrowseComp，我们采用与 DeepSeek-V3.2（DeepSeek-AI, 2025）相同的 discard-all 上下文管理策略。

5.3.2 评测结果

DeepSeek-V4-Pro-Max 与其他闭源/开源模型的对比见表 6。此外，我们评测了 DeepSeek-V4-Flash 与 DeepSeek-V4-Pro 的不同模式，结果见表 7。

【表格】表 6 DeepSeek-V4-Pro-Max 与闭源/开源模型的对比。“Max”“xHigh”“High”表示推理强度。最佳结果以粗体标出，次佳结果加下划线（原文第 38 页）

知识。在通用世界知识的评测中，DeepSeek-V4-Pro-Max（DeepSeek-V4-Pro 的最大推理强度模式）在开源大语言模型中确立了新的最先进水平。如 SimpleQA-Verified 所示，DeepSeek-V4-Pro-Max

以 20 个绝对百分点的优势显著优于所有现有开源基线。尽管取得这些进展，它目前仍落后于领先的专有模型 Gemini-3.1-Pro。在教育知识与推理领域，DeepSeek-V4-Pro-Max 在 MMLU-Pro、GPQA 与 HLE 基准上小幅优于 Kimi 与 GLM，但仍落后于领先的专有模型。总体而言，DeepSeek-V4-Pro-Max 在提升开源模型世界知识能力方面标志着一个重要里程碑。此外，DeepSeek-V4-Flash 与 DeepSeek-V4-Pro 在知识类任务上存在显著性能差距；这在预期之内，因为更大的参数量有助于在预训练期间保留更多知识。值得注意的是，当分配更高的推理强度时，两个模型在知识基准上均表现更佳。

【表格】表 7 DeepSeek-V4 系列不同规模与模式的对比。“Non-Think”“High”“Max”表示推理强度（原文第 39 页）

推理。DeepSeek-V4-Pro-Max 在推理基准上优于所有此前的开源模型，并在许多指标上匹敌最先进的闭源模型；而较小的 DeepSeek-V4-Flash-Max 在代码与数学推理任务上也超过了此前最佳的开源模型 K2.6-Thinking。同时，DeepSeek-V4-Pro 与 DeepSeek-V4-Flash 在编程竞赛中表现出色。根据我们的评测，它们的性能与 GPT-5.4 相当，这使开源模型首次在该任务上匹敌闭源模型。在 Codeforces 排行榜上，DeepSeek-V4-Pro-Max 目前位列人类参赛者中的第 23 名。DeepSeek-V4 在智能体与计算密集两种设置下的形式化数学任务上同样表现强劲。在智能体设置下，它取得了最先进的结果（见图 8），优于 Seed Prover（Chen et al., 2025）等此前模型。在计算更密集的流水线下，性能进一步提升，超越了 Aristotle（Achim et al., 2025）等系统，并达到该设置下已知的最佳结果。

智能体（Agent）。DeepSeek-V4 系列在评测中展现出强劲的智能体性能。在代码智能体任务上，DeepSeek-V4-Pro 的表现与 K2.6 和 GLM-5.1 相当，但这些开源模型整体仍落后于其闭源对手。DeepSeek-V4-Flash 在编码任务上弱于 DeepSeek-V4-Pro，尤其是在 Terminal Bench 2.0 上。其他智能体评测中也呈现类似趋势。值得注意的是，DeepSeek-V4-Pro 在 MCPAtlas 与 Toolathlon 上表现良好——这两个评测集包含大量工具与 MCP 服务——这表明我们的模型具备出色的泛化能力，而非仅能在内部框架上取得好成绩。

百万 token 上下文。在衡量上下文内检索能力的 MRCR 任务上，DeepSeek-V4-Pro 优于 Gemini-3.1-Pro，但仍落后于 Claude Opus 4.6。如图 9 所示，在 128K 上下文窗口内，检索性能保持高度稳定。虽然性能下降在超过 128K 之后开始显现，但该模型在 1M token 下的检索能力相较闭源与开源对手依然非常强劲。与 MRCR 不同，CorpusQA 更接近真实场景。评测结果同样表明，DeepSeek-V4-Pro 优于 Gemini-3.1-Pro。

【图表】图 8 实用范式与前哨范式下的形式化推理。左图：Putnam-200 Pass@8 按照 Seed-Prover 引入的设置，在 PutnamBench（Tsoukalas et al., 2024）的固定随机子集上评测；所有模型都在同一题目集上测试。我们遵循 Seed-

Prover 协议，但将专有搜索工具替换为开源的 LeanExplore（Asher, 2025），从而得到一个工具最少、采样受限的轻量设置。右图：Putnam-2025 在规模化的形式-非形式混合范式中探索数学推理的前沿，其中非形式化推理与形式化验证相结合，以暴露缺口并提升严谨性；DeepSeek-V4 达到证明完美的 120/120（原文第 40 页）

【图表】图 9 DeepSeek-V4 系列在 MRCR 任务上的性能（原文第 40 页）

推理强度（Reasoning Effort）。如表 7 所示，采用更长上下文并在 RL 中降低长度惩罚的 Max 模式，在最困难的任务上优于 High 模式。图 10 比较了 DeepSeek-V4-Pro、DeepSeek-V4-Flash 与 DeepSeek-V3.2 在代表性推理与智能体任务上的性能与成本。通过扩展测试时计算，DeepSeek-V4 系列相较前代取得了显著提升。此外，在 HLE 等推理任务上，DeepSeek-V4-Pro 展现出比 DeepSeek-V3.2 更高的 token 效率。

【图表】图 10 不同推理强度下的 HLE 与 Terminal Bench 2.0 性能。“None”表示 Non-think 模式，“Speciale”表示 DeepSeek-V3.2-Speciale 模型（原文第 41 页）

5.4 真实世界任务上的表现

标准基准往往难以刻画多样化真实世界任务的复杂性，从而在测试结果与实际用户体验之间造成差距。为弥合这一差距，我们开发了专有的内部指标，优先考虑真实世界的使用模式，而非传统基准。这一做法确保我们的优化能够转化为切实的收益。我们的评测框架专门针对 DeepSeek API 与 Chatbot 的主要用例，使模型性能与实际需求对齐。

5.4.1 中文写作

DeepSeek 的主要用例之一是中文写作。我们针对功能性写作与创意写作开展了严格的评测。表 12 给出了 DeepSeek-V4-Pro 与 Gemini-3.1-Pro 在功能性写作任务上的成对比较。这些任务由常见的日常写作查询构成，提示通常简洁明了。我们选择 Gemini-3.1-Pro 作为基线，因为它是我们评测中中文写作表现最好的外部模型。结果表明，DeepSeek-V4-Pro 以 62.7% 对 34.1% 的总体胜率优于基线；这主要是因为 Gemini 在中文写作场景中偶尔会让其固有的风格偏好凌驾于用户的明确要求之上。

表 13 给出了创意写作的对比，沿指令遵循与写作质量两个维度评估。与 Gemini-3.1-Pro 相比，DeepSeek-V4-Pro 在指令遵循上取得 60.0% 的胜率，在写作质量上取得 77.5% 的胜率，表明指令遵循略有提升、写作质量显著提升。尽管 DeepSeek-V4-Pro 在总体用户案例分析中结果更优，但仅针对最

具挑战性的提示——特别是涉及高复杂度约束或多轮场景的提示——进行的评测显示，Claude Opus 4.5 相较 DeepSeek-V4-Pro 仍保持性能优势。如表 14 所示，Claude Opus 4.5 取得 52.0% 对 45.9% 的胜率。

5.4.2 搜索

搜索增强问答是 DeepSeek Chatbot 的核心能力。在 DeepSeek 网页版与 App 中，“non-think”模式采用检索增强搜索（Retrieval-Augmented Search, RAG），而“thinking”模式采用智能体搜索（agentic search）。

检索增强搜索。我们在客观与主观问答类别上对 DeepSeek-V4-Pro 与 DeepSeek-V3.2 进行了成对评测。如表 11 所示，DeepSeek-V4-Pro 以明显优势优于 DeepSeek-V3.2，在两个类别上均展现出一致优势。最显著的提升出现在单值搜索与规划/策略任务上，表明 DeepSeek-V4-Pro 擅长定位精确的事实性答案，并从检索上下文中综合出结构化方案。然而，DeepSeek-V3.2 在比较与推荐类任务上仍相对有竞争力，这表明 DeepSeek-V4-Pro 在需要对搜索结果进行均衡、多视角推理的场景中仍有改进空间。

智能体搜索。与标准 RAG 不同，智能体搜索让模型能够针对每个查询迭代地调用搜索与抓取工具，从而显著提升整体搜索性能。对于 DeepSeek-Chat 的 thinking 模式，我们优化了智能体搜索功能，以在预定义的“思考预算”内最大化回答准确率。如表 9 所示，智能体搜索持续优于 RAG，尤其是在复杂任务上。此外，其成本依然高效——智能体搜索仅比标准 RAG 略贵（见表 10）。

5.4.3 白领任务

为严格评估模型在复杂企业生产力场景中的效用，我们构建了一套包含 30 项高级中文专业任务的综合套件。这些工作流刻意涵盖高层认知需求，包括深度信息分析、综合文档生成与细致文档编辑，横跨 13 个关键行业（如金融、教育、法律与技术）。评测在配备基础工具（包括 Bash 与网页搜索）的内部智能体 harness 中进行。

鉴于这些任务的开放性，自动化指标通常难以捕捉高质量回答的细微之处。因此，我们开展人工评测，将 DeepSeek-V4-Pro-Max 与 Opus-4.6-Max 进行对比。标注者在四个维度上对模型输出进行盲评：

- **任务完成（Task Completion）：**核心问题是否被成功解决。
- **指令遵循（Instruction Following）：**是否遵守特定约束与指令。
- **内容质量（Content Quality）：**事实准确性、逻辑连贯性与专业语气。
- **排版美观（Formatting Aesthetics）：**版面可读性与视觉呈现。

如图 11 所示，DeepSeek-V4-Pro-Max 在多样化的中文白领任务上优于 Opus-4.6-Max，取得了 63% 的可观非失败率（non-loss rate），并在分析、生成与编辑任务上展现出一致优势。图 12 所示的详细维度得分凸显了模型在任务完成与内容质量上的主要优势。具体而言，DeepSeek-V4-Pro-Max 通过频繁提供补充洞见与自验证步骤，主动预判用户的隐含意图。它在长文生成方面同样出色，能给出深入、连贯的叙述，而不是像 Opus-4.6-Max 那样频繁依赖过于简化的要点列表。此外，模型严格遵守正式的职业惯例，例如规范的中文层级编号。然而，在指令遵循方面，它偶尔会忽略特定的格式约束，略逊于 Opus。此外，模型在将大量文本输入浓缩为简洁摘要方面能力较弱。最后，其排版美观在演示幻灯片的整体视觉设计方面仍有较大提升空间。图 13、14 与 15 展示了若干测试案例；由于部分输出篇幅较长，仅显示部分页面。

【图表】图 11 分析、生成、编辑任务及整体性能的胜率对比（原文第 43 页）

【图表】图 12 详细维度得分，包括任务完成、内容质量、排版美观与指令遵循（原文第 43 页）

【图表】图 13 一个任务的示例输出：要求为某热门奶茶品牌与北京地铁起草一份联合营销方案（原文第 43 页）

5.4.4 代码智能体

为评测我们的编码智能体能力，我们从真实的内部研发工作负载中策划任务。我们从 50 余名内部工程师处收集了约 200 项具有挑战性的任务，涵盖功能开发、缺陷修复、重构与诊断，涉及 PyTorch、CUDA、Rust 与 C++ 等多样技术栈。每项任务都附带其原始代码仓库、相应的执行环境与人工标注的评分细则；经过严格的质量筛选后，保留 30 项任务作为评测集。如表 8 所示，DeepSeek-V4-Pro 显著优于 Claude Sonnet 4.5，并接近 Claude Opus 4.5 的水平。

表 8：研发编码基准上的对比（外部模型仅严格用于评测目的）（原文第 44 页）

模型	Haiku 4.5	Sonnet 4.5	DeepSeek-V4-Pro-Max	Opus 4.5	Opus 4.5 Thinking	Opus 4.6 Thinking
Pass Rate (%)	13	47	67	70	73	80

在一项针对 DeepSeek 开发者与研究者（N = 85）——他们均有在日常工作中使用 DeepSeek-V4-Pro 进行智能体编码的经验——的调查中，当被问及相较其他前沿模型，DeepSeek-V4-Pro 是否已可作为其默认且主要的编码模型时，52% 表示肯定，39% 倾向于肯定，不到 9% 表示否定。受访者认为

DeepSeek-V4-Pro 在大多数任务上交付了令人满意的结果，但也指出存在琐碎错误、对模糊提示的误解，以及偶尔的过度思考。

6 结论、局限与未来方向

在本工作中，我们提出了 DeepSeek-V4 系列的预览版本，目标是突破超长上下文处理效率瓶颈的下一代大语言模型。通过结合融合 CSA 与 HCA 的混合注意力架构，DeepSeek-V4 系列在长序列效率上实现了跨越式提升。这些架构创新，连同大量基础设施优化，使百万 token 上下文的高效原生支持成为可能，并为未来的测试时扩展、长程任务以及在线学习等新兴范式奠定了必要基础。评测结果表明，DeepSeek-V4-Pro-Max（DeepSeek-V4-Pro 的最大推理强度模式）重新定义了开源模型的最先进水平。它在知识基准上大幅优于此前开源模型，取得了接近前沿专有模型的优越推理性能，并具备有竞争力的智能体能力。同时，DeepSeek-V4-Flash-Max 在保持高成本效益架构的同时，取得了与领先闭源模型相当的推理性能。我们相信，DeepSeek-V4 系列为开源模型开启了百万长度上下文的新时代，并为更好的效率、规模与智能铺平了道路。

为追求极致的长上下文效率，DeepSeek-V4 系列采用了大胆的架构设计。为将风险降到最低，我们保留了许多经过初步验证的组件与技巧；这些组件虽然有效，却使架构相对复杂。在未来的迭代中，我们将开展更全面、更有原则的研究，将架构提炼至最本质的设计，使其在不牺牲性能的前提下更加优雅。同时，尽管 Anticipatory Routing 与 SwiGLU Clamping 已被证明能有效缓解训练不稳定，其底层原理仍未得到充分理解。我们将积极研究训练稳定性的基础问题，并加强内部指标监控，目标是形成一种更有原则、更具预测性的稳定大规模训练方法。此外，在 MoE 与稀疏注意力架构之外，我们还将主动探索新的模型稀疏维度——例如更稀疏的嵌入模块（Cheng et al., 2026）——以在不损害能力的前提下进一步提升计算与内存效率。我们还将持续研究低延迟架构与系统技术，使长上下文部署与交互更具响应性。此外，我们认识到长程、多轮智能体任务的重要性与实用价值，并将继续在该方向迭代探索。我们还在努力为模型加入多模态能力。最后，我们致力于开发更好的数据整理与合成策略，以在日益广泛的场景与任务中持续增强模型的智能、鲁棒性与实用可用性。

附录

附录 A 作者列表与致谢

A.1 作者列表

A.2 致谢

附录 B 评测细节

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] In addition, beyond the MoE and sparse attention architecture, we will also proactively explore model sparsity along new dimensions — such as more sparse embedding modules (Cheng et al., 2026) — to further improve computational and memory efficiency without compromising capability. We will also continuously investigate low-latency architectures and system techniques to make long-context deployment and interaction more responsive. Furthermore, we recognize the importance and practical value of long-horizon, multi-round agentic tasks, and will continue to iterate and explore in this direction. We are also working on incorporating multimodal capabilities to our models. Finally, we are committed to developing better data curation and synthesis strategies to consistently enhance model intelligence, robustness, and practical usability across an increasingly broad range of scenarios and tasks.

[2] AA. Gdpval-aa leaderboard, 2025. URL https://artificialanalysis.ai/methodolog y/intelligence-benchmarking#gdpval-aa .

[3] T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner, K. Henningsgard, Y. Kudryashov, A. Meiburg, et al. Aristotle: Imo-level automated theorem proving. arXiv preprint arXiv:2510.01346, 2025.

[4] A. Agache, M. Brooker, A. Florescu, A. Iordache, A. Liguori, R. Neugebauer, P . Piwonka, and D.-M. Popa. Firecracker: lightweight virtualization for serverless applications. In Proceedings of the 17th Usenix Conference on Networked Systems Design and Implementation, NSDI'20, page 419–434, USA, 2020. USENIX Association. ISBN 9781939133137.

[5] O. J. Aimuyo, B. Oh, and R. Singh. Flashmoe: Fast distributed moe in a single kernel. Advances in Neural Information Processing Systems, 2025. URL https://neurips.cc/virtual/2 025/poster/119124 .

[6] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

[7] J. Asher. LeanExplore: A search engine for Lean 4 declarations, 2025. URL https://arxiv.or g/abs/2506.11085 .

[8] Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen, Z. Chen, J. Cui, H. Ding, M. Dong, A. Du, C. Du, D. Du, Y. Du, Y. Fan, Y. Feng, K. Fu, B. Gao, H. Gao, P . Gao, T. Gao, X. Gu, L. Guan, H. Guo, J. Guo, H. Hu, X. Hao, T. He, W. He, W. He, C. Hong, Y. Hu, Z. Hu, W. Huang, Z. Huang, Z. Huang, T. Jiang, Z. Jiang, X. Jin, Y. Kang, G. Lai, C. Li, F. Li, H. Li, M. Li, W. Li, Y. Li, Y. Li, Z. Li, Z. Li, H. Lin, X. Lin, Z. Lin, C. Liu, C. Liu, H. Liu, J. Liu, J. Liu, L. Liu, S. Liu, T. Y. Liu, T. Liu, W. Liu, Y. Liu, Y. Liu, Y. Liu, Y. Liu, Z. Liu, E. Lu, L. Lu, S. Ma, X. Ma, Y. Ma, S. Mao, J. Mei, X. Men, Y. Miao, S. Pan, Y. Peng, R. Qin, B. Qu, Z. Shang, L. Shi, S. Shi, F. Song, J. Su, Z. Su, X. Sun, F. Sung, H. Tang, J. Tao, Q. Teng, C. Wang, D. Wang, F. Wang, and H. Wang. Kimi K2: open agentic intelligence. CoRR, abs/2507.20534, 2025a. URL https://doi.org/10.48550/arXiv.2507.20534 .

[9] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639–3664, 2025b. M. Balunovi´ c, J. Dekoninck, I. Petrov, N. Jovanovi´ c, and M. Vechev. Matharena: Evaluating llms on uncontaminated math competitions. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmark, 2025.

[10] C. Bandi, B. Hertzberg, G. Boo, T. Polakam, J. Da, S. Hassaan, M. Sharma, A. Park, E. Hernandez, D. Rambado, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. arXiv preprint arXiv:2602.00933, 2026.

[11] F. Bellard. Qemu, a fast and portable dynamic translator. In Proceedings of the Annual Conference on USENIX Annual Technical Conference, ATEC '05, page 41, USA, 2005. USENIX Association.

[12] I. Bello, H. Pham, Q. V . Le, M. Norouzi, and S. Bengio. Neural combinatorial optimization with reinforcement learning, 2017. URL https://openreview.net/forum?id=rJY3vK9eg .

[13] J. Chen, W. Chen, J. Du, J. Hu, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, W. Shi, Z. Wang, M. Wang, C. Wei, S. Wei, H. Xin, F. Yang, W. Gao, Z. Yuan, T. Zhan, Z. Zheng, T. Zhou, and T. H. Zhu. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience, 2025. URL https://arxiv.org/abs/2512.17260 .

[14] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374 .

[15] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In 13th USENIX Symposium on Operating Systems

Design and Implementation (OSDI 18), pages 578–594, Carlsbad, CA, Oct. 2018. USENIX Association. ISBN 978-1-939133-08-3. URL https://www.usenix.org/conference/osdi18/prese ntation/chen .

[16] A. Cheng, A. Jacovi, A. Globerson, B. Golan, C. Kwong, C. Alberti, C. Tao, E. Ben-David, G. S. Tomar, L. Haas, et al. The facts leaderboard: A comprehensive benchmark for large language model factuality. arXiv preprint arXiv:2512.10791, 2025.

[17] X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. CoRR, abs/2601.07372, 2026. doi: 10.48550/ARXIV.2601. 07372. URL https://doi.org/10.48550/arXiv.2601.07372 .

[18] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[19] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P . Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL https://doi.org/10.48550/arXiv.2401.06066 . T. Dao, D. Haziza, F. Massa, and G. Sizov. Flash-decoding for long-context inference, 2023. URL https://pytorch.org/blog/flash-decoding/ .

[20] L. De Moura and N. Bjørner. Z3: an efficient smt solver. In Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS'08/ETAPS'08, page 337–340, Berlin, Heidelberg, 2008. Springer-Verlag. ISBN 3540787992.

[21] DeepSeek-AI. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. CoRR, abs/2406.11931, 2024. URL https://doi.org/10.48550/arXiv.2406.11 931 .

[22] DeepSeek-AI. Deepseek-v3 technical report. CoRR, abs/2412.19437, 2024. URL https://doi. org/10.48550/arXiv.2412.19437 .

[23] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024. URL https://doi.org/10.48550/arXiv.2405.04 434 .

[24] DeepSeek-AI. Fire-flyer file system, 2025. URL https://github.com/deepseek-ai/3FS .

[25] DeepSeek-AI. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. Nat., 645(8081):633–638, 2025. URL https://doi.org/10.1038/s41586-025-09422-z .

[26] DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556 .

[27] X. Deng, J. Da, E. Pan, Y. Y. He, C. Ide, K. Garg, N. Lauffer, A. Park, N. Pasari, C. Rane, K. Sampath, M. Krishnan, S. Kundurthy, S. Hendryx, Z. Wang, V . Bharadwaj, J. Holm, R. Aluri, C. B. C. Zhang, N. Jacobson, B. Liu, and B. Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks?, 2025. URL https://arxiv.org/abs/2509.16941 .

[28] H. Ding, Z. Wang, G. Paolini, V . Kumar, A. Deoras, D. Roth, and S. Soatto. Fewer truncations improve language modeling. arXiv preprint arXiv:2404.10830, 2024.

[29] X. Dong, Y. Fu, S. Diao, W. Byeon, Z. CHEN, A. S. Mahabaleshwarkar, S.-Y. Liu, M. V . keirsbilck, M.-H. Chen, Y. Suhara, Y. C. Lin, J. Kautz, and P . Molchanov. Hymba: A hybrid-head architecture for small language models. In The Thirteenth International Conference on Learning Representations, 2025. URL https://openreview.net/forum?id=A1ztozypga .

[30] X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. arXiv preprint arXiv:2502.14739, 2025.

[31] D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368– 2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246 .

[32] X. Gao, M. Dong, X. Miao, W. Du, C. Yu, and H. Chen. Erofs: a compression-friendly readonly file system for resource-scarce devices. In Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference, USENIX ATC '19, page 149–162, USA, 2019. USENIX Association. ISBN 9781939133038. A. P . Gema, J. O. J. Leang, G. Hong, A. Devoto, A. C. M. Mancino, R. Saxena, X. He, Y. Zhao, X. Du, M. R. G. Madani, C. Barale, R. McHardy, J. Harris, J. Kaddour, E. van Krieken, and P . Minervini. Are we done with mmlu? CoRR, abs/2406.04127, 2024. URL https://doi.or g/10.48550/arXiv.2406.04127 .

[33] F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. Better & faster large language models via multi-token prediction. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=pEWAcejiU2 .

[34] Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In The Twelfth International Conference on Learning Representations, 2024.

[35] L. Haas, G. Yona, G. D'Antonio, S. Goldshtein, and D. Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. arXiv preprint arXiv:2509.07968, 2025.

[36] Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models. arXiv preprint arXiv:2411.07140, 2024.

[37] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

[38] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[39] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

[40] D. Hupkes and N. Bogoychev. Multiloko: a multilingual local knowledge benchmark for llms spanning 31 languages. CoRR, abs/2504.10356, 2025. doi: 10.48550/ARXIV.2504.10356. URL https://doi.org/10.48550/arXiv.2504.10356 .

[41] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2018.

[42] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. arXiv preprint arXiv:2403.07974, 2024.

[43] K. Jordan, Y. Jin, V . Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. Cited on, page 10, 2024.

[44] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In R. Barzilay and M.-Y. Kan, editors, Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147 . H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. DADI: Block-Level image service for agile and elastic application deployment. In 2020 USENIX Annual Technical Conference (USENIX ATC 20), pages 727–740. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL https://www.usenix.org/conference/atc20/presentation/li-huiba .

[45] H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. CMMLU: Measuring massive multitask language understanding in Chinese. arXiv preprint arXiv:2306.09212, 2023.

[46] J. Li, W. Zhao, J. Zhao, W. Zeng, H. Wu, X. Wang, R. Ge, Y. Cao, Y. Huang, W. Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. arXiv preprint arXiv:2510.25726, 2025.

[47] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net, 2024. URL https://openreview.net /forum?id=1NdN7eXyb4 .

[48] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon is scalable for LLM training. CoRR, abs/2502.16982, 2025. URL https://doi.org/10.48550/arXiv.2502.16982 .

[49] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.

[50] K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi: 10.64434/tml.20251026. https://thinkingmachines.ai/blog/on-policy-distillation.

[51] Z. Lu, C. Li, Y. Shi, W. Shen, M. Yan, and F. Huang. Corpusqa: A 10 million token benchmark for corpus-level analysis and reasoning. arXiv preprint arXiv:2601.14952, 2026.

[52] T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V . Le, and J. Jung. Towards robust mathematical reasoning. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025. URL https://aclanthology.org/2025.emnlp-main.1794/ .

[53] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint arXiv:2601.11868, 2026.

[54] MiniMax. Meet minimax-m2, 2025. URL https://github.com/MiniMax-AI/MiniMax-M2 .

[55] L. d. Moura and S. Ullrich. The lean 4 theorem prover and programming language. In International Conference on Automated Deduction, pages 625–635. Springer, 2021.

[56] Y. Nesterov. A method of solving a convex programming problem with convergence rate $O(1/k)$. Soviet Mathematics Doklady, 27:372–376, 1983.

[57] NVIDIA Corporation. cublas documentation, 2024. URL https://docs.nvidia.com/cuda /cublas/ . Version 12.4. Accessed: 2024-09-16.

[58] OpenAI. Multilingual massive multitask language understanding (mmmlu), 2024a. URL https://huggingface.co/datasets/openai/MMMLU . OpenAI. Openai mrcr: Long context multiple needle in a haystack benchmark, 2024b. URL https://huggingface.co/datasets/openai/mrcr .

[59] OpenAI. Learning to reason with llms, 2024c. URL https://openai.com/index/learnin g-to-reason-with-llms .

[60] OpenAI. Introducing SimpleQA, 2024d. URL https://openai.com/index/introducing -simpleqa/ .

[61] OpenAI. Introducing SWE-bench verified we're releasing a human-validated subset of swebench that more, 2024e. URL https://openai.com/index/introducing-swe-bench-v erified/ .

[62] OpenAI. gpt-oss-120b & gpt-oss-20b model card. CoRR, abs/2508.10925, 2025. doi: 10.48550/A RXIV.2508.10925. URL https://doi.org/10.48550/arXiv.2508.10925 .

[63] M. Osama, D. Merrill, C. Cecka, M. Garland, and J. D. Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pages 429–431, 2023.

[64] T. Patwardhan, R. Dias, E. Proehl, G. Kim, M. Wang, O. Watkins, S. P . Fishman, M. Aljubeh, P . Thacker, L. Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. arXiv preprint arXiv:2510.04374, 2025.

[65] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity's last exam. arXiv preprint arXiv:2501.14249, 2025.

[66] W. Qi, Y. Yan, Y. Gong, D. Liu, N. Duan, J. Chen, R. Zhang, and M. Zhou. Prophetnet: Predicting future n-gram for sequence-to-sequence pre-training. In T. Cohn, Y. He, and Y. Liu, editors, Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020, volume EMNLP 2020 of Findings of ACL, pages 2401–2410. Association for Computational Linguistics, 2020. URL https://doi.org/10.18653/v1/2020.f indings-emnlp.217 .

[67] Qwen. Qwen3 technical report. CoRR, abs/2505.09388, 2025. doi: 10.48550/ARXIV.2505.09388. URL https://doi.org/10.48550/arXiv.2505.09388 .

[68] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: Memory optimizations toward training trillion parameter models. In SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pages 1–16. IEEE, 2020.

[69] J. K. Reed, Z. DeVito, H. He, A. Ussery, and J. Ansel. Torch.fx: Practical program capture and transformation for deep learning in python, 2022. URL https://arxiv.org/abs/2112.0 8429 .

[70] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

[71] G. T. M. Riviere, S. Pathak, P . G. Sessa, C. Hardin, S. Bhupatiraju, L. Hussenot, T. Mesnard, B. Shahriari, A. Ram'e, J. Ferret, P . Liu, P . D. Tafti, A. Friesen, M. Casbon, S. Ramos, R. Kumar, C. L. Lan, S. Jerome, A. Tsitsulin, N. Vieillard, P . Sta ´ nczyk, S. Girgin, N. Momchev, M. Hoffman, S. Thakoor, J.-B. Grill, B. Neyshabur, A. Walton, A. Severyn, A. Parrish, A. Ahmad, A. Hutchison, A. Abdagic, A. Carl, A. Shen, A. Brock, A. Coenen, A. Laforge, A. Paterson, B. Bastian, B. Piot, B. Wu, B. Royal, C. Chen, C. Kumar, C. Perry, C. A. Welty, C. A. Choquette- Choo, D. Sinopalnikov, D. Weinberger, D. Vijaykumar, D. Rogozi'nska, D. Herbison, E. Bandy, E. Wang, E. Noland, E. Moreira, E. Senter, E. Eltyshev, F. Visin, G. Rasskin, G. Wei, G. Cameron, G. Martins, H. Hashemi, H. Klimczak-Pluci'nska, H. Batra, H. Dhand, I. Nardini, J. Mein, J. Zhou, J. Svensson, J. Stanway, J. Chan, J. Zhou, J. Carrasqueira, J. Iljazi, J. Becker, J. Fernandez, J. R. van Amersfoort, J. Gordon, J. Lipschultz, J. Newlan, J. Ji, K. Mohamed, K. Badola, K. Black, K. Millican, K. McDonell, K. Nguyen, K. Sodhia, K. Greene, L. L. Sjoesund, L. Usui, L. Sifre, L. Heuermann, L. cia Lago, L. McNealus, L. B. Soares, L. Kilpatrick, L. Dixon, L. L. B. Martins, M. Reid, M. Singh, M. Iverson, M. Gorner, M. Velloso, M. Wirth, M. Davidow, M. Miller, M. Rahtz, M. Watson, M. Risdal, M. Kazemi, M. Moynihan, M. Zhang, M. Kahng, M. Park, M. Rahman, M. Khatwani, N. Dao, N. shad Bardoliwalla, N. Devanathan, N. Dumai, N. Chauhan, O. Wahltinez, P . Botarda, P . Barnes, P . Barham, P . Michel, P . chong Jin, P . Georgiev, P . Culliton, P . Kuppala, R. Comanescu, R. Merhej, R. Jana, R. A. Rokni, R. Agarwal, R. Mullins, S. Saadat, S. M. M. Carthy, S. Perrin, S. M. R. Arnold, S. bastian Krause, S. Dai, S. Garg, S. Sheth, S. Ronstrom, S. Chan, T. Jordan, T. Yu, T. Eccles, T. Hennigan, T. Kociský, T. Doshi, V . Jain, V . Yadav, V . Meshram, V . Dharmadhikari, W. Barkley, W. Wei, W. Ye, W. Han, W. Kwon, X. Xu, Z. Shen, Z. Gong, Z. Wei, V . Cotruta, P . Kirk, A. Rao, M. Giang, L. Peran, T. Warkentin, E. Collins, J. Barral, Z. Ghahramani, R. Hadsell, D. Sculley, J. Banks, A. Dragan, S. Petrov, O. Vinyals, J. Dean, D. Hassabis, K. Kavukcuoglu, C. Farabet, E. Buchatskaya, S. Borgeaud, N. Fiedel, A. Joulin, K. Kenealy, R. Dadashi, and A. Andreev. Gemma 2: Improving open language models at a practical size. arXiv preprint arXiv:2408.00118, 2024.

[72] S. Roller, S. Sukhbaatar, A. Szlam, and J. Weston. Hash layers for large sparse models. In M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P . Liang, and J. W. Vaughan, editors, Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual, pages 17555–17566, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/92bf5e6240737 e0326ea59846a83e076-Abstract.html .

[73] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V . Elango, M. Golub, A. Heinecke, P . James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P . Dubey, P . Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. Microscaling data formats for deep learning, 2023.

[74] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.

[75] Z. Shao, Y. Luo, C. Lu, Z. Z. Ren, J. Hu, T. Ye, Z. Gou, S. Ma, and X. Zhang. Deepseekmath-v2: Towards self-verifiable mathematical reasoning, 2025. URL https://arxiv.org/abs/25 11.22570 .

[76] N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019. URL http://arxiv.org/abs/1911.02150 .

[77] N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.

[78] F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?id=fR3wGCk-IXp . J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. Neurocomputing, 568:127063, 2024.

[79] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[80] G. Tsoukalas, J. Lee, J. Jennings, J. Xin, M. Ding, M. Jennings, A. Thakur, and S. Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024. URL https://arxiv.org/abs/2407.11214 .

[81] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[82] L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. CoRR, abs/2408.15664, 2024a. URL https://doi.org/10.48550/arX iv.2408.15664 .

[83] L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, et al. Tilelang: Bridge programmability and performance in modern neural kernels. In The Fourteenth International Conference on Learning Representations, 2026.

[84] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. CoRR, abs/2406.01574, 2024b. URL https://doi.org/10.48550/arXiv.2406.01574 .

[85] J. Wei, Z. Sun, S. Papay, S. McKinney, J. Han, I. Fulford, H. W. Chung, A. T. Passos, W. Fedus, and A. Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516, 2025.

[86] T. Wei, J. Luan, W. Liu, S. Dong, and B. Wang. Cmath: Can your language model pass chinese elementary school math test?, 2023.

[87] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openreview .net/forum?id=NG7sS51zVF .

[88] Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifoldconstrained hyper-connections, 2026. URL https://arxiv.org/abs/2512.24880 .

[89] L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, Y. Tian, Q. Dong, W. Liu, B. Shi, Y. Cui, J. Li, J. Zeng, R. Wang, W. Xie, Y. Li, Y. Patterson, Z. Tian, Y. Zhang, H. Zhou, S. Liu, Z. Zhao, Q. Zhao, C. Yue, X. Zhang, Z. Yang, K. Richardson, and Z. Lan. CLUE: A chinese language understanding evaluation benchmark. In D. Scott, N. Bel, and C. Zong, editors, Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020, pages 4762–4772. International Committee on Computational Linguistics, 2020. doi: 10.18653/V1/2020.COLING-MAIN.419. URL https://doi.org/10.18653/v1/2020.coling-main.419 . J. Yang, K. Lieret, C. E. Jimenez, A. Wettig, K. Khandpur, Y. Zhang, B. Hui, O. Press, L. Schmidt, and D. Yang. Swe-smith: Scaling data for software engineering agents, 2025. URL https: //arxiv.org/abs/2504.21798 .

[90] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[91] C. Zhang, K. Du, S. Liu, W. Kwon, X. Mo, Y. Wang, X. Liu, K. You, Z. Li, M. Long, J. Zhai, J. Gonzalez, and I. Stoica. Jenga: Effective memory management for serving llm with heterogeneity. In Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25, page 446–461, New York, NY, USA, 2025a. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764823. URL https://doi.org/10.1145/3731569.3764823 .

[92] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu. Comet: Fine-grained computation-communication overlapping for mixture-of-experts. 2025b. URL https://arxiv.org/abs/2502.19811 .

[93] C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling. https://github.com/deepseek-ai/DeepGEMM , 2025.

[94] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364 .

[95] D. Zhu, H. Huang, Z. Huang, Y. Zeng, Y. Mao, B. Wu, Q. Min, and X. Zhou. Hyperconnections. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL https://openreview.net /forum?id=9FqARW7dwB .

[96] X. Zhu, D. Cheng, H. Li, K. Zhang, E. Hua, X. Lv, N. Ding, Z. Lin, Z. Zheng, and B. Zhou. How to synthesize text data without model collapse? arXiv preprint arXiv:2412.14689, 2024.

[97] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P . Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL http s://openreview.net/forum?id=YrycTjllL0 .

DeepSeek-V4.1-Flash：突破 KV Cache 压缩的极限

DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression

DeepSeek-AI research@deepseek.com

arXiv:2609.19969 · 中文翻译版（正文全文；图表保留原文页码位置）

摘 要

长程智能体的广泛采用，使模型的工作负载日益呈现“输入密集”的特征。尽管已有工作大幅降低了长上下文计算的开销，预填充（prefill）仍然计算代价高昂，庞大的 KV cache 也持续给 HBM 与 SSD 的容量及数据传输带宽带来压力。这些计算、存储与带宽需求共同构成了进一步降低部署成本的主要瓶颈。为应对这一挑战，我们提出 DeepSeek-V4.1-Flash，这是一个多模态混合专家（MoE）模型，主干参数为 552B，并支持最长一百万 token 的上下文。借助其因果编码器-解码器（Causal Encoder-Decoder，CED）架构，该模型在解码阶段每 token 激活 16B 参数，而在预填充阶段仅激活 8B 参数，显著提升了智能体工作负载的成本效率。为突破 KV cache 压缩的极限，DeepSeek-V4.1-Flash 将压缩稀疏注意力 2（Compressed Sparse Attention 2，CSA2）中的跨层 KV cache 复用与 FP4 KV 缓存相结合。这些设计将其全局 KV cache 占用（始终位于 HBM）降至每 token 890 字节，约为 DeepSeek-V4-Flash 对应占用的 1/4。此外，通过一项名为 SWA Bounded Replay 的专门部署优化，DeepSeek-V4.1-Flash 将其持久化 KV cache 占用（始终位于 SSD 或主机内存）降至 DeepSeek-V4-Flash 的约 1/8。尽管 KV cache 占用大幅缩小，该模型的性能仍显著优于基线。此外，我们精简了 DeepSeek-V4 架构，并引入若干高效架构……扩展。我们使用包含 45T token 的多模态语料预训练 DeepSeek-V4.1-Flash，并进行全面的后训练，在多种基于文本与多模态的智能体场景中均取得强劲表现。模型检查点已发布于：https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash

1 引言

近年来，长时程智能体（long-horizon agents）的应用快速扩展，使超长上下文处理日益成为一种重要的模型工作负载。支撑此类工作负载不仅需要高效的长序列处理，还需要对大型 KV cache 进行持久化存储、复用与传输。因此，KV cache 管理已成为模型部署的一项基础能力，同时在计算、存储与通信方面带来了巨大挑战。此前稀疏注意力领域的进展（DeepSeek-AI, 2025, 2026b）已显著降低了长序列处理的计算成本，使得持久化存储与数据移动日益成为突出的瓶颈。

具体而言，DeepSeek-V4（DeepSeek-AI, 2026b）将覆盖完整上下文的全局注意力分支与局部滑动窗口注意力（Sliding-Window Attention, SWA）相结合。全局分支维护全局 KV，包括 main KV 与 indexer K；SWA 则维护局部 KV 状态。在固定窗口大小下，SWA KV 存储量与序列长度无关且有界。因此，对于足够长的序列，全局 KV 主导了运行时的 KV 占用，而后者受 HBM 容量约束。此外，部分 KV 被持久化以用于前缀复用，称为持久化 KV（persistent KV）cache，其受 SSD 与主机内存容量约束。I/O 与互连带宽也限制了 cache 的迁移与加载。这些约束共同限制了服务吞吐、增加了部署成本，并最终阻碍了智能体在更长任务跨度与更广泛应用场景中的部署与采用。

因此，进一步降低 KV cache 占用对于缓解存储与通信瓶颈、降低长上下文服务成本至关重要。为此，我们研发了 DeepSeek-V4.1-Flash——一个专为更激进的 KV cache 压缩而设计的多模态混合专家（MoE）模型。DeepSeek-V4.1-Flash 拥有 552B 主干参数，原生支持多模态输入，并可处理最长 100 万 token 的上下文。我们采用因果编码器-解码器（Causal Encoder-Decoder, CED）架构，其中解码器的全局 KV 由编码器最终隐藏状态投影而来。该设计使模型在预填充（prefill）期间每个 token 激活 8B 参数、在解码（decode）期间激活 16B 参数，这对输入密集的智能体场景尤具成本效益。尽管规模远大于 DeepSeek-V4-Flash，DeepSeek-V4.1-Flash 在相同序列长度下所需的运行时 KV cache 存储仅约为其 1/4，持久化 KV cache 存储仅约为其 1/8。此外，DeepSeek-V4.1-Flash 的整体性能优于 DeepSeek-V4-Flash。

这一程度的 KV cache 压缩是通过在模型架构、cache 精度与部署策略三个层面进行联合优化实现的。从概念上看，DeepSeek-V4 可视为一个以 SWA 为基础的局部处理主干，并辅以压缩后的全局上下文。这一视角促使我们专注于简化全局分支，同时大体保留局部注意力设计。在架构层面，我们设计了压缩稀疏注意力 2（Compressed Sparse Attention 2, CSA2），对全局 KV（包括 main KV 与 indexer K）以及 Top-K 索引施加跨层复用，从而大幅降低 KV cache 存储。CSA2 具有三种静态分配的模式：Full、Reindex 与 Reuse。Full 模式生成全局 KV 并执行索引；Reindex 模式复用前一层产生的全

局 KV，并使用自身的 indexer Q 对共享的 indexer K 重新打分，选出新的 Top-K 索引；Reuse 模式复用前一层的全局 KV 与 Top-K 索引，直接执行稀疏注意力。在三种模式中，每一层都保留自己的全局 Q 与 SWA KV。共享全局 KV 与 indexer K 减少了重复的 cache 存储。此外，与采用压缩稀疏注意力（CSA）– 重度压缩注意力（HCA）混合架构的 DeepSeek-V4 不同，DeepSeek-V4.1-Flash 使用纯 CSA2。在 cache 精度层面，我们在训练期间使用 FP4 全局 KV cache，性能仅有边际下降。CSA2 与 FP4 KV cache 共同将全局 KV cache 存储降至 DeepSeek-V4-Flash 的约 1/4，如图 1(b) 所示。在部署层面，DeepSeek-V4.1-Flash 与 DeepSeek-V4 一样，在每一层都使用 SWA。在 DeepSeek-V4 中，我们采用混合策略来权衡持久化 SWA KV cache 的存储成本与精确重建所需的计算量。精确重建需要重放最近的 L×n_win 个 token，其中 L 为层数，n_win 为 SWA 窗口大小。在 DeepSeek-V4.1-Flash 中，我们引入了 SWA 有界重放（SWA Bounded Replay），仅通过重放最近的 n_win 个 token 来近似重建所需的 SWA KV 状态。实验表明，这只会造成可忽略的性能下降。这一发现建立了一种新的存储 – 计算权衡，使我们能够避免将 SWA KV cache 持久化到 SSD，同时只需付出少量预填充重计算。借助 SWA 有界重放，持久化 KV cache 占用进一步降至 DeepSeek-V4-Flash 的约 1/8。这些优化共同大幅缓解了 HBM 与 SSD 容量压力，降低了部署成本，并为更大规模的部署铺平了道路。

【图表】图 2 各代 DeepSeek 模型的单 token 解码 FLOPs 随上下文长度的变化。我们通过分别以 1、0.5、0.25 对 BF16、FP8、FP4 运算加权来计入计算精度。DeepSeek-V4.1-Flash 的解码 FLOPs 随上下文长度增加几乎保持恒定，大幅降低了长上下文场景的计算成本（原文第 5 页）

作为对 CED 与 CSA2 的补充，我们进一步精简了原有的 DeepSeek-V4 架构。此外，我们将原有的 mHC（Xie et al., 2026）设计升级为 Single-Pass mHC，并配合 Mega-mHC 部署 kernel，相对于原先需要四个 kernel 的实现，将激活内存流量减半。我们还集成了 Engram（Cheng et al., 2026b）条件记忆模块以增强模型能力。同时，我们引入了 DSpark（Cheng et al., 2026a）推测解码架构，通过半自回归草稿生成与置信度调度的验证来提升解码效率。综合所有架构改进，DeepSeek-V4.1-Flash 的单 token 解码 FLOPs 在不同上下文长度下几乎保持恒定。图 2 表明，将上下文长度扩展 256 倍（从 4K 到 1M），其解码 FLOPs 仅增加 1/4，远小于 DeepSeek-V4-Flash 所观察到的增长。

为充分释放这些架构设计带来的 KV cache 压缩收益并进一步提升训练与推理效率，我们系统性地协同优化了 DeepSeek-V4.1-Flash 的训练基础设施与推理系统，确保高效且可扩展的大规模多模态训练与长上下文部署。训练基础设施支持解耦的视觉编码器执行、面向长序列的均衡图像分片，以及用于注意力复用的跨阶段共享状态管理。推理系统实现了编码器与解码器两条 SWA 有界重放路径。进一

步优化包括通信–计算重叠、分片的 Engram 嵌入表以及推理 kernel 融合。具体而言，每个 CSA2 Reuse 模式层在预填充期间仅需执行 15 个 kernel，在解码期间仅需 11 个 kernel。我们还将长生命周期的全局 KV 存储与短生命周期的编码器 SWA KV 在主机内存中分离，并使用有界重放来近似重建缺失的编码器 SWA 状态。

在预训练方面，我们在包含 45T token 的大规模多模态语料上训练 DeepSeek-V4.1-Flash。稀疏注意力在 64K 序列长度下从零开始训练，不设任何稠密注意力预热阶段。预训练之后，模型具备原生多模态能力，并支持最长 100 万 token 的上下文。在评测中，DeepSeek-V4.1-Flash-Base 的世界知识、推理与编程能力与 DeepSeek-V4-Pro-Base 相当，并在留出评测上带来 5%–10% 的提升，而仅使用 1/3 的总参数与 1/4 的激活参数。这些结果共同凸显了其强大的参数效率，也反映了真实部署场景下训练数据质量的提升。

在此基座模型之上，我们开展后训练以激发其推理与智能体能力。与上述架构创新不同，我们的后训练没有引入任何算法创新：训练配方遵循监督微调（SFT）→强化学习（RL）→在线策略蒸馏（OPD）的标准范式，除沿用 DeepSeek-V4 研发中已有的成熟实践外没有任何改动（DeepSeek-AI, 2026b）。所有实质性变化都在数据流程中。我们开发了用于数据合成与环境构建的大规模自动化流程，并逐步扩展 RL 过程中使用的数据、任务与 rollout 规模，从而将模型能力扩展到文本、多模态与智能体领域。图 1(a) 总结了 DeepSeek-V4.1-Flash 在核心智能体基准上的表现。我们的评测表明，尽管规模紧凑，DeepSeek-V4.1-Flash 仍展现出独特的能力画像：

- **推理（Reasoning）**：该模型具备强大的推理能力，在数学与竞赛编程等推理密集型基准上保持高准确率，性能与 Kimi-K3（Team et al., 2026a）、DeepSeek-V4-Pro 等顶级开源模型相当。
- **智能体（Agent）**：DeepSeek-V4.1-Flash 在 Terminal-Bench 2.1（Merrill et al., 2026）、DeepSWE v1.1（DataCurve, 2026）与 AutomationBench（Shepard and Salimans, 2026）等标准智能体基准上，性能与闭源前沿模型相当。它已被证明完全能够处理日常编程任务与白领工作流。然而，在需要专家级领域知识的科学导向智能体任务（如 Terminal-Bench 4.0；Marten et al., 2026a）上，与巨型模型之间仍存在差距。
- **多模态（Multimodal）**：在多模态领域，该模型在评估视觉推理与专业图表解读的基准上，超越了 Kimi-K3 等顶级开源竞争者。除了正式指标之外，它在真实世界的视觉智能体工作流（如前端开发与办公自动化）中也展现出实用价值，能够利用渲染的屏幕截图进行视觉检查与自我纠正。不过，我们承认，与巨型闭源系统相比，整体上仍存在明显的性能差距。

这些结果表明，DeepSeek-V4.1-Flash 已能在绝大多数基准上匹配闭源前沿模型，并能够完成超过 95% 的真实世界任务。同时，其较小的激活足迹带来了低推理延迟与低服务成本。因此，我们相信 DeepSeek-V4.1-Flash 在能力与效率之间提供了有利的权衡，可作为一款快速、可负担的助手，支撑广大用户的日常工作。总之，DeepSeek-V4.1-Flash 在提升模型智能与推理效率的同时降低了部署成本。它大幅降低了大规模部署长时程智能体的成本门槛，并为其在更广泛场景中的采用创造了新机遇。DeepSeek-V4.1-Flash 也是我们持续推进扩展的新起点。在此基础上，我们将追求模型架构、预训练与后训练的联合扩展，以进一步探索模型智能的前沿。

【图表】图 3 DeepSeek-V4.1-Flash 的总体架构。40 层网络被划分为因果编码器与解码器，各 20 层。所有前馈层均使用标准 DeepSeekMoE。编码器的前两层使用滑动窗口注意力（SWA），其余层使用压缩稀疏注意力 2（CSA2），其中 CSA2(ratio, mode) 指定压缩率与模式。模型还使用 Single-Pass mHC、Engram、DSpark 以及层次化稀疏索引器

（原文第 7 页）

2 架构

2.1 概述

DeepSeek-V4.1-Flash 是一个多模态混合专家（MoE）Transformer，以图像与文本为输入，自回归地生成文本。其语言主干由 40 个因果 Transformer 层组成，组织为 20 层因果编码器后接 20 层解码器。除前两层仅使用滑动窗口注意力（SWA）外，每一层都同时包含全局注意力与 SWA。视觉编码器与 MLP 投影器将图像转换为视觉嵌入，并使其与文本嵌入联合处理，多模态数据从语言模型预训练之初就被纳入。总体而言，DeepSeek-V4.1-Flash 拥有 552B 主干参数与 196B Engram 参数，在预填充期间每个 token 激活 8B 参数，在解码期间激活 16B 参数。图 3 展示了 DeepSeek-V4.1-Flash 的总体架构。

因果编码器–解码器（CED）架构与压缩稀疏注意力 2（CSA2）分别应对长上下文推理中相互补充的开销。CED 由编码器输出构建解码器的全局键值（KV）cache，使大多数 prompt token 可以绕过完整的解码器计算，同时保留层内局部滑动窗口注意力。这几乎将预填充计算减半，降低了在上下文不断增长的智能体工作负载中处理新输入或未缓存输入的成本。CSA2 则跨层共享全局 KV 以降低 cache 存储，并复用稀疏选择以减少索引工作量。在解码器中，层次化稀疏索引器（Hierarchical Sparse

Indexer）将后续索引器限制在由更早的索引器选出的候选池内，进一步减少每个 query 需要打分的条目数量。

我们保留了 DeepSeekMoE（Dai et al., 2024）的共享专家与细粒度路由专家，并为图像与文本 token 引入了模态特定的负载均衡（Wang et al., 2024a）。Single-Pass mHC（Xie et al., 2026）改进了残差流混合，以实现更高效的 kernel 融合；Engram（Cheng et al., 2026b）则增加了稀疏访问的条件记忆。在主干预训练期间，我们省略了 MTP 模块，改用 DSpark（Cheng et al., 2026a）进行推测解码。DSpark 在主干预训练阶段之后单独训练。此外，我们将 main KV cache 压缩到 FP4，以进一步降低存储开销。以下各节将描述这些组件以及相应的优化改动。

2.1.1 多模态架构

多模态输入通路由视觉编码器与 MLP 投影器组成。对于每张输入图像，视觉编码器产生一个空间网格形式的视觉特征。随后，3×3 的 pixel-unshuffle 操作沿通道维度重排每个局部邻域，在 MLP 投影器将特征映射到语言主干的隐藏维度之前降低空间分辨率。最后，得到的视觉嵌入被插入输入嵌入序列中对应的图像 token 位置，并由语言主干与文本嵌入联合处理。

DeepSeek-ViT：我们从零训练了一个名为 DeepSeek-ViT 的视觉编码器，以原生处理不同分辨率的图像。我们基于 Vision Transformer（Dosovitskiy et al., 2021）架构构建 DeepSeek-ViT，并做了若干修改。为适应任意分辨率的输入，我们用 2D-RoPE 替换了标准的绝对位置嵌入。为使 ViT 更贴近 LLM 的设计原则，我们将 patch 嵌入层的卷积替换为线性投影，以确保与 Muon 优化器兼容。我们还采用 RMSNorm（Zhang and Sennrich, 2019）进行归一化，并以 SwiGLU（Shazeer, 2020）作为激活函数。在将视觉特征送入 LLM 之前，我们施加 3×3 下采样的 pixel-unshuffle 操作，将视觉 token 数量减少为原来的九分之一，从而有效支持最高约 1344×1344 像素的输入分辨率。

面向 MoE 的多模态无辅助损失负载均衡：图像 token 与文本 token 表现出不同的表示分布，并可能在 MoE 中引发不同的专家路由偏好。因此，对二者聚合负载进行均衡可能会掩盖模态特定的不平衡。为解决这一问题，我们扩展了无辅助损失负载均衡（Wang et al., 2024a），为文本 token 与图像 token 分别维护逐专家的校正偏置。在路由时，每个 token 使用与其模态对应的校正偏置进行专家选择，同时保留原始路由分数用于对所选专家的输出加权。每次训练步之后，两组偏置分别根据各自对应的专家负载独立更新。这一设计在每个模态内部平衡了专家利用率，有助于稳定高效的多模态训练。

2.2 因果编码器-解码器（CED）

在智能体工作流中，频繁的工具调用会产生大量预填充请求，当 KV cache 未命中时会带来严重的计算开销。为缓解这一预填充瓶颈，我们受 YOCO（Sun et al., 2024）启发，提出了因果编码器-解码器（CED）架构。YOCO 通过允许上半部分层直接共享下半部分层生成的 KV cache 来减少预填充计算。在这一概念基础上，CED 引入了一系列结构性改进，以同时提升整体 KV cache 容量与 KV 生成的计算深度。因此，CED 成功减少了近一半的预填充计算，同时保持与基线相当的性能。

对于全局注意力，CED 将 Transformer 的底部 L/2 层视为因果编码器。对于上半部分层（即解码器，l > L/2），KV 条目并非来自各层自己的隐藏状态 H_l，而是使用与层相关的投影权重（W^KV_l 与 W^Z_l）直接从第 (L/2) 层的隐藏状态 H_{L/2} 投影得到：

C_l = H_{L/2} W^KV_l，Z_l = H_{L/2} W^Z_l，l > L/2　　（1）

其中 C 与 Z 分别表示 KV 条目及其对应的压缩权重。该设计使 CED 在预填充阶段只需计算前一半的层，从而以极小的计算成本获得上层的全局 KV cache。

对于滑动窗口注意力（SWA），CED 在所有层中保持传统的逐层计算。具体而言，对于任意层 l，局部 key 与 value 直接来自当前层的隐藏状态 H_l。该设计有效提升了局部 KV 生成的计算深度。然而，保持这种逐层计算必须引入一个 SWA 重放过程。在预填充阶段，为解码器计算 SWA KV cache 需要额外处理 n_win×L/2 个 token（其中 n_win 表示窗口大小）。对于每轮 prompt 较短的多轮交互，解码器中的这一计算开销变得不可忽略。幸运的是，先前工作（Chen et al., 2025）表明，SWA 的实际有效感受野远小于理论上的 n_win×L/2。受此观察启发，我们引入了解码器 SWA 有界重放（Decoder SWA Bounded Replay），在 SWA 计算中只预填充 prompt 的最后 n_win 个 token，从而显著降低计算成本。更多细节见第 3.2.2 节。

总体而言，对于序列长度 N ≫ n_win，CED 将预填充复杂度从 O(NL) 降至 O(NL/2 + n_win×L/2) ≈ O(NL/2)，有效将整体计算减半。

2.3 压缩稀疏注意力 2（CSA2）

服务长上下文需要同时控制 KV cache 存储与注意力计算。这些成本可以沿三个可相乘的维度降低：其一是条目大小，GQA（Ainslie et al., 2023）减少 KV head 数量，MLA（DeepSeek-AI, 2024）则在各 head 之间共享一个小的潜向量；其二是序列维度，即每 m 个 token 被压缩为一个条目，如 DeepSeek-V4（DeepSeek-AI, 2026b）中的 CSA 与 HCA；其三是层维度，即某些层复用其他层的 cache 与选择结果（Brandon et al., 2024），而不是保留自己的 cache，或整体由更高效的层替代。先前

工作表明，沿层维度的压缩是有效的：IndexCache（Bai et al., 2026）跨层复用 Top-K 索引以减少索引器计算；YOIO（Sun et al., 2026b）只计算一次稀疏路由并在所有层间共享；HySparse（Gao et al., 2026）则让稀疏层复用稠密层的 KV cache。然而，仅复用索引并不能节省 main KV 存储；全网范围的路由共享会限制性能；混合设计仍保留全注意力层；更重要的是，这些方法都没有同时覆盖全部三个可相乘的维度。

CSA2 联合利用这三个维度：它跨层共享 main KV 与 indexer K，并允许各层复用 Top-K 索引，且将 cache 共享与索引复用解耦。它将这些复用策略与简化的压缩器以及层次化稀疏索引器相结合，后者在解码器中收窄后续索引层的搜索范围。

与 CSA 类似，CSA2 包含一个轻量级索引器，使用 indexer Q 与 indexer K 对 main KV 条目打分，并为每个 query 选出 Top-K 条目。每个 Q 关注所选条目以及层内局部滑动窗口 KV（SWA KV）。CSA2 也将未压缩的 main KV 设置作为压缩率为 1 的特例。同时，CSA2 简化了压缩器与索引器。在 CSA 中，压缩率 m 由 2m 个原始 KV cache 条目生成每个 main KV 条目，且相邻压缩条目的源条目存在重叠；它还使用绝对位置嵌入来在压缩时编码这 2m 个条目的位置。CSA2 去除了这种重叠与绝对位置嵌入。此外，CSA2 通过对 main KV 条目投影得到 indexer K，替代了 CSA 中从隐藏状态出发的独立压缩路径。这两种设计都简化了实现并提升了训练效率。

第 2.3.1 节与第 2.3.2 节将分别介绍跨层复用策略与层次化稀疏索引器。

【图表】图 4 CSA2 的三种运行模式。各模式在如何获取 main KV、indexer K 与 Top-K 索引上有所不同。绿色块表示在当前层计算得到的量；黄色块表示从最近的 Full Mode 层复用的 main KV 与 indexer K；红色块表示从最近的产生索引的层（Full 或 Reindex Mode）复用的 Top-K 索引；三种模式都在当前层计算 main Q 与 SWA KV（原文第 10 页）

2.3.1 跨层 KV 与索引复用

每个 CSA2 层被静态分配为 Full、Reindex 或 Reuse 三种模式之一。在三种模式中，该层都计算自己的 query 与 SWA KV，并将它们与选中的 main KV 条目一起用于产生新的注意力输出。不同模式的区别在于获取 main KV、indexer K 与 Top-K 索引的方式。图 4 展示了这三种模式。

Full 模式：该层计算自己的 main KV 与 indexer Q，由该 main KV 投影出 indexer K，并运行索引器产生新的 Top-K 索引。因此它执行完整的 CSA2 计算路径，其组件职责与 DeepSeek-V4 中一个完整的 CSA 层相同。

【图表】图 5 层次化稀疏索引器。每个方块表示一个位置；绿色方块标记被选中的索引，蓝色矩形标记根据其最大索

引器分数选出的块。解码器中处于 Full 模式的第一个 CSA2 层选择自己的 Top-512 索引，并基于所选块构建供后续层使用的共享候选池；处于 Reindex 模式的 CSA2 层随后从该池中选择各自的 Top-512 索引（原文第 11 页）

Reindex 模式：该层复用前一层最近可用的 main KV 及其对应的 indexer K。索引器计算自己的 query，对复用的 key 重新打分，并产生新的 Top-K 索引。这使得稀疏选择能够跨层变化，同时 main KV 与 indexer K 保持共享。

Reuse 模式：该层复用最近可用的 main KV，以及由前一个处于 Full 或 Reindex 模式的层针对该 main KV 计算的最新 Top-K 索引。它使用这一选择执行注意力，而不计算 indexer Q 或评估索引分数。共享 main KV 与 indexer K 降低了 cache 存储，而复用 Top-K 索引避免了额外的索引器计算。Reindex 模式在保持 cache 共享的同时允许所选条目跨层变化。当 CSA2 与 CED 结合时，被分配为 Full 模式的解码器层由第 (L/2) 层（即因果编码器的最后一层）的隐藏状态计算自己的全局 KV，Reindex 与 Reuse 模式保持不变。

2.3.2 层次化稀疏索引器

跨层索引复用减少了索引器评估的次数，但其余索引器仍需对全部因果可见上下文打分。对于极长的上下文，这一成本仍是主要的计算瓶颈。先前工作通过在 token 级索引之前对池化的块表示打分并剪枝，引入了索引器稀疏性（Xu et al., 2026b）。我们发现，在解码器中，来自较浅层索引器的信息可以自然地用于限制更深层索引器所考虑的候选，而无需引入任何额外状态。因此，我们引入了层次化稀疏索引器（Hierarchical Sparse Indexer），它仅在 CED 的解码器中使用，以减少解码期间重复的打分工作。对于每个 query，第一个被分配为 Full 模式的层会构建一个候选池，后续重新索引的层将其作为搜索域。在候选池大小固定的情况下，这使更深层索引器每次查询的开销从随上下文长度线性增长变为常数。

该机制具有训练感知性，并在后训练阶段引入：候选限制在训练与推理时完全一致地应用，因此更深的 indexer 在其推理时使用的相同搜索域下进行优化。图 5 展示了这一过程。

第一个 Full Mode 层会对所有因果可见的 main KV 位置打分，并为其自身注意力生成 Top-K 索引。它还执行分块候选选择：每个块被赋予其各位置中的最大索引分数，得分最高的块被选中。随后，它把被选中块覆盖的位置收集到一个比最终 Top-K 集合更大的候选池中。例如，选择 2,048 个块、每块 8 个位置，就得到 16,384 个候选位置。该池界定了后续 indexer 的搜索范围；最终的 Top-K 选择则决定每一层读取哪些 main KV 条目。

Reindex Mode 下的后续层仅对相应 query 的候选位置打分，并在该池内选择自己的 Top-K 条目。Reuse Mode 下的层不执行新的索引，而是使用针对其所复用的 main KV 计算出的最新 Top-K 索引。因此，候选池在参与索引的各层之间共享，而它们最终的选择可以不同。

在候选池大小固定的情况下，后续每个 indexer 对每个 query 需要打分的位置数有界，且与上下文长度无关。第一个 Full Mode 层仍然扫描整个因果可见范围。因此，分层索引降低了后续 indexer 评估的开销，同时保留了首次的全范围扫描。

2.4 高效的架构扩展

2.4.1 单遍 mHC

在 DeepSeek-V4 中，我们引入了 mHC（Xie et al., 2026），它在相邻 Transformer 块之间维护 n 条残差流。对于每个 token，我们用 $X_l \in R^{n\times d}$ 表示这些流，其中 l 是块索引，d 是隐藏维度。这些流的更新方式如下：

$$X_{l+1} = B_l X_l + C_l F_l(A_l X_l),\quad (A_l, B_l, C_l) = H(X_l) \qquad (2)$$

其中 $A_l \in R^{1\times n}$、$C_l \in R^{n\times 1}$ 与 $B_l \in R^{n\times n}$ 是由 X_l 预测得到的逐 token 系数。系数预测器 H 包含归一化与投影。

理想情况下，两个块之间的残差变换是一个从 (X_{l-1}, Y_{l-1}) 到 $(X_l, \hat{X}_l)$ 的单一映射，其中 $\hat{X}_l = A_l X_l$ 是当前块的输入，$Y_{l-1} = F_{l-1}(\hat{X}_{l-1})$ 是前一个块的输出。这种映射需要 (n+1)d 次读取与 (n+1)d 次写入，因此激活内存流量的下界为 (2n+2)d。在实践中，DeepSeek-V4 采用 (2) 式的多遍实现，包含三个因数据依赖而顺序执行的内核：

$$X_l = B_{l-1} X_{l-1} + C_{l-1} Y_{l-1} \quad \text{残差更新，对 n 收缩} \qquad (3)$$

$$(A_l, B_l, C_l) = H(X_l) \quad \text{系数，对 nd 收缩} \qquad (4)$$

$$\hat{X}_l = A_l X_l \quad \text{输入混合，对 n 收缩} \qquad (5)$$

这三个内核分别读取 (n+1)d、nd 与 nd 个值，总共写入 (n+1)d 个值。若把 F_l 中的 pre-norm 也计算在内，总激活内存流量为 (4n+4)d，是下界的两倍。

在 H 中，归一化权重会离线折叠进投影权重，RMS 除法在投影之后进行。因此，三个阶段中的两个可以共享一次残差遍历：残差更新不需要跨隐藏维度归约，因此 X_l 的每个 tile 都可以计算出来，并立即用于累积投影输出以及计算 RMS 所需的平方和。输入混合无法融合进这一遍历，因为

A_l 要等到对全部隐藏 tile 的归约完成后才可用，所以它需要第二次读取 X_l。这第二遍还可以顺带完成输入 pre-norm。这种两遍实现总共需要 $(3n+2)d$ 次激活读写，比下界多一次对 X_l 的读取。

因此，我们引入单遍 mHC（Single-Pass mHC），将输入混合系数整体平移一个块，即每个块使用前一个块产生的混合系数，从而消除上述依赖：

$$X_{l+1} = B_l X_l + C_l F_l(A_{l-1} X_l),\ (A_l, B_l, C_l) = H(X_l) \quad (6)$$

现在输入混合使用 A_{l-1} 而非 A_l，因此不再依赖由 X_l 计算出的系数。X_l 的每个 tile 因而可以立即同时用于输入混合与系数预测，无需等待完整归约。经验上，这一平移带来的性能退化可以忽略不计。

对于预训练，我们保留现有的多内核实现，因为这一平移只改变每个块所应用的混合系数。对于部署，我们将残差更新、输入混合与系数预测融合为单个内核 Mega-mHC。该内核实现 mHC 时使用 $(3n+2)d$ 次激活读写，实现 Single-Pass mHC 时使用 $(2n+2)d$ 次激活读写。Mega-mHC 沿隐藏维度以 tile 为单位处理 X_l。每个 tile 被用于计算混合输入，并累积预测下一块的 (A_l, B_l, C_l) 所需的量。该内核还集成了输入 pre-norm 与 FP8 转换。由此，残差只需读取一次、写入一次，达到理想映射的 $(n+1)d$ 次读取与 $(n+1)d$ 次写入，并将我们原始实现的激活内存流量减半。

2.4.2 Engram

我们为 DeepSeek-V4.1-Flash 增加了 Engram（Cheng et al., 2026c），这是我们此前工作中提出的条件记忆模块，用于将记忆与计算解耦。我们沿用原始 Engram 设计——分词器压缩、多头哈希、上下文感知门控与多分支集成——并做两处修改。第一，我们省去短因果卷积，因为其性能收益不足以抵消它在推理栈中带来的额外复杂度。第二，我们使用基于动量的更新加 Sinkhorn 平衡来优化 Engram 嵌入，详见 2.5 节。

我们在两个模块之间均匀分配 196B 的 Engram 参数。每个模块使用 N-gram 阶数 $\{2, 3, 4\}$，配备 8 个哈希头，每个阶数的总嵌入维度为 2048。每个头索引一张约 16M 条目的表，表大小选为不同的素数。嵌入表与 key/value 投影均使用 FP8 精度。这些模块被放置在第 1 层与第 14 层（零索引），以平衡各训练流水线阶段的内存使用。在推理时，确定性寻址使嵌入可以通过后台 RDMA 传输从主机内存预取，其中第一个模块的预取与第一个 Transformer 块的计算重叠。Engram 训练与推理的更多实现细节见 3.1.3 节。

2.4.3 DSpark

我们为 DeepSeek-V4.1-Flash 配备了 DSpark（Cheng et al., 2026a），这是一个推测解码模块，结合了半自回归起草与置信度调度的验证。起草器由三个 Transformer 块组成，滑动注意力窗口为 128 个 token。通过这三个块的一次前向传播会并行计算五个起草位置的 base logits，同时一个轻量级 Markov 头对起草 token 之间的依赖建模。置信度头预测每个位置的条件接受概率，这些概率被用于估计前缀存活概率。调度器将这些估计与实测的引擎吞吐曲线相结合，为每个请求动态选择验证长度，目标是在当前系统负载下最大化系统范围内的期望 token 吞吐。

不同于 DeepSeek-V3（DeepSeek-AI, 2024）中贯穿预训练始终与骨干联合训练的 MTP 模块，DSpark 在预训练之后的一个专门阶段引入。在该阶段，我们只训练 DSpark，保持骨干冻结。在后训练期间，我们继续让 DSpark 与骨干一起训练，但不将 DSpark 目标的梯度回传进骨干。这使 DSpark 与不断演化的策略保持一致，从而能够加速在线服务以及 RL 与 OPD 的 rollout 生成。

2.4.4 FP4 主 KV Cache

长上下文智能体工作负载需要每个请求具有较大的 KV cache，这增加了服务成本。DeepSeek-V4 已经使用量化感知训练（QAT）（Jacob et al., 2018）处理 FP4 indexer 的 query 与 key，从而加速索引计算并减小 indexer cache 的大小。我们采用 OCP 标准的 MXFP4 格式（Rouhani et al., 2023），以尽可能支持更多硬件平台，尽管在我们的实验中其他格式精度更高。现在我们将 QAT 扩展到主 KV cache，此时 FP4 主要用于减少存储，而非加速矩阵乘法。在注意力之前对缓存值进行反量化，使我们能够使用更精确的格式，而无需该格式的原生矩阵乘法支持，从而保持跨硬件平台的兼容性。

在评估过的约 4-bit 格式中，我们选择 E2M1，并为每 16 个通道使用一个 E4M3 scale，遵循 NVFP4（Alvarez et al., 2025）但省去其二级全局 scale，以平衡精度与简洁性。省去该 scale 为主 KV cache 留下了充足的动态范围：该格式支持高达 $448 \times 6 = 2688$ 的幅值，远高于该 cache 的幅值上界。在 DeepSeek-V4.1-Flash 中，训练得到的最大 RMSNorm 权重幅值约为 1。经 RMS 归一化后，512 通道 KV latent 的 L2 范数至多约为 $\sqrt{512}$。RoPE 保持该范数不变，因此旋转后各通道的最大绝对值也以约 $\sqrt{512} \approx 22.6$ 为界。此外，训练期间观察到的最大幅值约为 10。因此，省去全局 scale 不会造成可测量的精度下降，并简化了 cache 布局。

为在 DeepSeek-V4.1-Flash 中启用 FP4 主 KV cache 存储，我们在后训练阶段引入 QAT。非 RoPE 与 RoPE 分量使用相同的量化格式。我们在 RoPE 之后量化 cache：在我们的实验中，在 RoPE 之前量化仅带来边际的精度提升，且会在解码期间引入额外开销。由于 SWA KV cache 对量化敏感，

我们对其保留 FP8。与 DeepSeek-V4 的 FP8 主 KV cache 相比，该格式将存储占用几乎减半，无论在 HBM 中还是卸载到 SSD 时都是如此。

2.5 优化

在 DeepSeek-V4 所用优化配置的基础上，我们做了一些新修改，以更好地契合架构设计。

第一，我们采用按头的 Muon（head-wise Muon），即在应用 Muon 更新之前按头拆分 Query 权重。这里我们简要讨论这一设计的动机。将 Muon 视为预条件梯度下降，标准 Muon 对所有头使用同一个预条件器，而 head-wise Muon 为不同头提供不同的预条件器。这一设计能更好地处理注意力头之间的异质性（Zhang et al., 2024; Zhang, 2026, Section 3）。因此，我们观察到 head-wise Muon 优于标准 Muon。head-wise Muon 的经验优势也在 GLM 5（Zeng et al., 2026）与 Kimi-K3（Team et al., 2026a）中得到验证。

第二，对新引入的 Engram 参数应用 Adam 会大幅增加优化器状态的内存占用。为降低训练期间的内存使用，我们改用基于动量的更新加 Sinkhorn 平衡来优化 Engram 嵌入表、token embedding 与预测头。Sinkhorn 平衡此前已在 SinkGD（Scetbon et al., 2025）中应用于线性层权重矩阵；这里我们将其扩展到这些大型参数矩阵。与 Muon 一样，这一方法只需要一个动量缓冲区，而在经验上优于 Adam。

基础配置（Basic Configurations）：我们对归一化层权重以及其他非矩阵参数（包括偏置与缩放因子）保留 AdamW（Loshchilov and Hutter, 2019）。我们对语言模型骨干中线性变换的权重矩阵、Engram 投影层以及视觉-语言投影器使用 Muon（Jordan et al., 2024）。对 Query 与 Key 权重使用 head-wise Muon。我们对 Muon 应用解耦权重衰减与 Nesterov 动量（Nesterov, 1983; Liu et al., 2025）；归一化层权重同样受权重衰减影响，而偏置与缩放因子不受影响。Sinkhorn 平衡更新也使用 Nesterov 动量，但不应用权重衰减。在预训练期间，我们保持视觉编码器冻结，直到学习率衰减阶段，而视觉编码器的最终归一化层与视觉-语言投影器保持可训练。在学习率衰减开始时，我们解冻视觉编码器，并以较小的学习率与 LLM 联合优化。

Engram / Embedding / Prediction Head 的 Sinkhorn 平衡更新：完整流程总结于算法 1。总体而言，它与 Muon 的工作流程相同，只是用 Sinkhorn 平衡替代了 Newton–Schulz 正交化。我们用 m 表示较大的矩阵维度（对应嵌入表与预测头的词表大小），用 n 表示隐藏维度。

算法 1 带 Sinkhorn 平衡的动量更新

输入：权重 $W_t \in R^{m\times n}$，梯度 G_t，动量 M_{t-1}，动量系数 β，基础学习率 η_t，学习率修正 γ，数值常数 ε 与

```
τ，以及奇数个归一化步数 K
M_t ← β M_{t−1} + (1−β) G_t
Ĝ_t ← β M_t + (1−β) G_t        ◁ Nesterov 动量
ρ_i ← ‖Ĝ_{t,i,:}‖₂，且 ρ̄ ← (1/m) Σ_{i=1}^m ρ_i
U^(0) ← Ĝ_t
若 ρ_i ⩽ τ ρ̄，则置 U^(0)_{i,:} ← 0        ◁ 屏蔽接近零的行
for k = 1, …, K do
    if k 为奇数 then
        for all i = 1, …, m do
            U^(k)_{i,:} ← U^(k−1)_{i,:} / (‖U^(k−1)_{i,:}‖₂ + ε)
        end for
    else
        for all j = 1, …, n do
            U^(k)_{:,j} ← U^(k−1)_{:,j} / (‖U^(k−1)_{:,j}‖₂ + ε)
        end for
    end if
end for
Δ_t ← √n U^(K)        ◁ 将单位行 ℓ₂ 范数转换为单位行 RMS
η̃_t ← γ η_t        ◁ 匹配 Adam 的更新幅度
W_{t+1} ← W_t − η̃_t Δ_t
```

给定 Nesterov 动量更新 $\hat{G}_t$，Sinkhorn 平衡寻找对角缩放矩阵 D_r 与 D_c，使得

$$\Delta_t = \sqrt{n}\, U^{(K)} = \sqrt{n}\, D_r \hat{G}_t D_c，\ \frac{1}{n}\sum_{j=1}^{n} (\Delta_t)^2_{ij} \approx 1，\ \frac{1}{m}\sum_{i=1}^{m} (\Delta_t)^2_{ij} \approx 1 \quad (7)$$

因此，该过程近似地使更新矩阵的行 RMS 与列 RMS 相等。这里，一行对应一个 token 索引或 n-gram 身份，一列编码一个隐藏特征。Sinkhorn 平衡利用这种 token–特征结构，同时沿行与列进行归一化。为数值稳定性，满足 $\rho_i \leqslant \tau\cdot\bar{\rho}$ 的行会被屏蔽。因子 $\sqrt{n}$ 将单位行 ℓ_2 范数转换为单位行 RMS。另外，我们调整有效学习率为 $\tilde{\eta}_t = \gamma\eta_t$，以匹配 Adam 的更新幅度。我们设置 $\gamma = 0.18$，接近 Moonlight（Liu et al., 2025）中使用的因子 0.2。

更广泛地说，Sinkhorn 平衡与利用矩阵或张量轴结构的优化器密切相关（Shazeer and Stern, 2018; Zhang et al., 2025a; Wen et al., 2025; Glentis et al., 2025; Deng et al., 2026; Yuan et al., 2026; Xu et al., 2026a）。例如，Adafactor（Shazeer and Stern, 2018）以不同方式执行行与列归一化，Adam-mini（Zhang et al., 2025a）对嵌入表与预测头使用另一种行归一化。这些归一化策略可能在优化性能与通信开销上存在差异。我们将更细致的研究留作未来方向。

3 通用基础设施

3.1 训练基础设施

3.1.1 多模态训练基础设施

对比学习中的通信-计算重叠（Communication-Computation Overlap in Contrastive Learning）：视觉编码器首先以对比目标进行优化，然后再用生成式下一 token 预测损失进行微调。在对比阶段，损失是在整批文本-视觉对上计算的，因此两种模态的特征必须在数据并行 rank 之间 all-gather，带来大量通信。由于文本特征的梯度只依赖于收集到的视觉特征——对称地，视觉特征的梯度只依赖于收集到的文本特征——每次 all-gather 都可以与前向或反向传播重叠，而不是阻塞流水线：

Forward(V) → (Forward(T) ‖ AllGather(V)) → ∇Text → (Backward(T) ‖ AllGather(T)) → ∇Vision → Backward(V)

其中 V 与 T 表示视觉与文本特征，(A ‖ C) 表示计算 A 与通信 C 的重叠，∇ 表示梯度计算。在该调度中，视觉特征在文本前向传播期间被收集，文本特征在文本反向传播期间被收集，因此两次 all-gather 都被完全隐藏在有用的计算之后。

端到端并行（End-to-End Parallelism）：为处理视觉编码器与 LLM 之间的模型与数据异质性（Zhang et al., 2025b），我们采用近期训练系统中所使用的解耦编码器设计（Team et al., 2025, 2026b）。视觉编码器被复制到 LLM 参数树之外，每个训练步被划分为三个阶段：视觉编码器前向、LLM 前向/反向、视觉编码器反向。这种分离避免了视觉编码器与 LLM 计算之间的相互干扰。负载均衡的视觉处理被限制在第一与最后一个阶段，而 LLM 阶段不包含视觉计算，并保持纯文本训练的并行策略。

长序列多模态训练优化（Long-Sequence Multimodal Training Optimization）：DeepSeek-V4.1-Flash 在最长可达 100 万 token 的序列上训练，且预训练与后训练阶段都有相当一部分训练发生在超长序列长度下。在这些序列长度下，多模态样本会造成沉重的 I/O、CPU 与内存瓶颈。

- **均衡图像分片（Balanced image sharding）：**预训练期间，单个超长、图像密集的序列可能在加载时耗尽一台主机的 I/O、CPU 与内存，因此每条序列的图像会在 CP rank 之间进行负载均衡的分片，且每个图像只被加载一次。在图像只读取一次的情况下，只要满足 $N\times\rho/B_IO < N\times C/B_GPU \iff \rho < (B_IO/B_GPU)\cdot C$，加载就能被隐藏在计算之后，其中 N 是 token 数，ρ 是每个 token 的原始字节数，C 是每个 token 的计算量，B_IO 与 B_GPU 分别是文件系统与 GPU 带

宽。由于 N 被消去，该判据只涉及每 token 的量（ρ 与 C），与序列长度和集群规模无关；ρ 由视觉模块配置（例如分辨率上限或空间下采样）决定。因此，只有在每 token 计算量较低的小模型（如消融实验）中，存储吞吐才会成为瓶颈，而生产规模的模型仍处于计算受限状态。

- **增量图像传输（Incremental image transfer）**：除上述均衡分片外，强化学习 rollout 只以增量方式将图像传输到推理引擎，并将引擎在 CPU 侧的解码与预处理输出缓存在分布式文件系统上，以便在多次 rollout 与后续训练中复用。

3.1.2 CSA2 的注意力共享训练

在 2.3 节中，我们介绍 CSA2——一种具有三种模式的注意力共享方法，其中一些模式会在多层之间共享 main KV、indexer K 以及 Top-K 索引中的一个或多个。在大规模分布式训练中支持 CSA2，需要超出注意力计算本身的额外协调。特别是，共享注意力组件的层可能被放置在不同的流水线阶段，这使得直接的模块复用与传统的阶段本地执行不兼容。因此，我们采用若干设计来支持 CSA2 训练。

影子 indexer（Shadow indexers）：在每个参与阶段放置一个轻量级的可执行副本，同时为共享参数保留单一逻辑所有者。所有者仍负责优化与 checkpointing，而参数同步与梯度聚合使影子副本在训练全程保持一致。这一设计保持了原始模型语义，同时无需流水线调度器把共享层当作特殊的执行单元。

流水线载荷扩展（Pipeline payload extensions）：当源层与消费层跨越流水线边界时，为下游消费者提供所需的中间表示与稀疏路由信息。这些状态被纳入现有的点对点通信路径，并与上下文并行一致地划分，避免不必要的复制，同时保持相应的梯度流。

微批次级共享状态管理（Micro-batch-level shared-state management）：跟踪与并发活跃流水线微批次相关的状态，并协调它们在前向执行、激活重计算与反向传播中的生命周期。共享状态会一直保留到其最终消费者完成，然后立即释放以限制额外内存开销。同一运行时抽象还处理阶段放置与源-消费者关系，使注意力实现能够访问共享状态，而无需依赖物理流水线布局。

结合对优化器、checkpointing、warm-up 与计算图追踪工作流的轻量级适配，这些机制使 CSA2 能够在现有分布式训练接口与流水线调度下透明运行。

3.1.3 Engram

Engram 嵌入表在 engram 并行规模的专用进程组之间按行划分。组大小控制每设备内存使用与嵌入查找通信范围之间的权衡。优化器状态进一步在各表分区的副本之间分片。Engram 查找索引只依赖

输入 token 序列。因此，在每个流水线阶段开始处理当前训练步的微批次之前，会为整个本地批次发起嵌入预取，从而尽量减少对流水线执行的干扰。嵌入梯度在反向传播期间被缓冲，并在骨干反向传播之后返回给其所属 rank。为与多模态训练高效集成，嵌入预取与梯度传输被安排为与视觉编码器的前向和反向计算重叠。嵌入以 FP8 存储与获取，取回的值与缩放因子直接传给后续 GEMM。对于 Engram 表更新，Sinkhorn 归一化在迭代之间维护行与列的缩放向量，以避免重复写入完整的归一化矩阵。行归一化与部分列统计量的累积被融合进单个内核，以进一步减少内存流量。在 RL rollout 期间，Engram 嵌入表常驻 GPU 内存。这一放置减轻了主机内存压力，并有助于避免由主机内存碎片导致的 OOM 失败。

3.2 推理系统

DeepSeek-V4.1-Flash 在设计上把推理效率作为一等关注点。尽管其架构在概念上很复杂，但由此得到的推理内核流程却相当简洁。通过合理的核融合，我们将复杂操作封装起来，并在少量融合内核内部保持硬件资源充分流水化——包括 FlashMLA 中的 fused-RoPE-attention-RoPE-cast 内核（Li and Liu, 2025）、DeepGEMM 中的 Mega-Gate、Mega-mHC 与 Mega-MoE 内核（Zhao et al., 2025）、TileKernels 中的内核（Wang et al., 2026a），以及 DeepSelect 中的 TopK 内核（Qian et al., 2026）。因此，绝大多数 Transformer 层——即 CSA2 以 Reuse Mode 运行的那些层——在前填充（prefill）期间仅执行 15 个内核，在解码期间仅执行 11 个内核，从而实现高吞吐与低延迟的推理。

在部署层面，我们采用编码-前填充-解码（Encoder–Prefill–Decode, EPD）解耦，使视觉编码、前填充与解码能够独立扩展并在执行中重叠。

3.2.1 持久化 KV Cache 管理

在相同工作负载下，V4.1 将持久化 KV cache 占用降至 V4 的 1/8。这一降低由两个乘性因子构成：持久化 KV cache 不再存储 SWA KV，使其大小几乎减半；其中保留的全局 KV 又通过架构与精度优化进一步压缩至 V4 占用的 1/4。

在 V4 的部署中，SWA KV 占持久化 KV cache 容量近一半。在该 cache 内，全局 KV 与 SWA KV 独立管理，由 LRU 驱逐策略控制。全局 KV 完整存储，一旦命中就复用完整前缀。相比之下，SWA KV 只在两个特定点缓存——prompt 末尾与输出末尾——以支持重新生成与多轮会话；命中后可从该缓存位置恢复计算。为保持高命中率，我们在 SSD 上配置了足够大的持久化 KV cache，使得在典型工作负载下，两类 KV 都能驻留超过 72 小时。尽管在指定位置只保留 n_win 个 KV 条目，未压缩的 SWA KV cache 仍会带来可观的存储开销，尤其是在短回合的多轮对话中。

持久化存储 SWA KV 既昂贵又低效，因为其访问模式与持久化 KV cache 的长期保留策略不匹配。不同于具有长尾复用特性的全局 KV，SWA KV 只在活跃会话内部一个狭窄的、分钟级窗口内被复用，一旦会话结束或下一回合开始就失效。V4 技术报告提出了 **Zero SWA Caching**，通过重算缺失的 SWA KV 来避免存储开销。然而，精确恢复需要对 L×n_win 个 token 进行完整前向传播，其成本在生产部署中被证明过高。

因此，V4.1 对持久化 KV cache 管理作如下修改：

1. SWA KV 不再缓存在持久化 KV cache 中，而是存储在分布式内存池中，该池由每台机器 10% 的主机 DRAM 提供。尽管该池的总容量小得多，但其较短的 TTL（仅数分钟）使过期条目可以立即回收用于新会话；在真实工作负载下，这种高周转足以服务绝大多数并发活跃会话。全局 KV 仍保留在持久化 KV cache 中，保证至少 72 小时的生命周期。2. 驱逐 SWA KV 不可避免地会造成 miss，而借助轻量级回退机制 **Encoder SWA Bounded Replay**（详见 3.2.2 节），这些 miss 的成本仍然可承受。对于不可避免但罕见的、命中全局 KV 却缺失 SWA KV 的请求，它通过只重算 n_win 个 token 而非完整的 L×n_win token 前向来恢复缺失状态。这种有界回放是该设计的基石：它将灾难性的 miss 转变为优雅、廉价的降级，从而为从持久化 KV cache 中移除 SWA KV 提供了依据。

3.2.2 SWA 有界回放

由于 SWA 依赖跨层累积，精确重建 L 层的 SWA KV 需要回放 L×n_win 个 token。SWA Bounded Replay 只回放最近的 n_win 个 token，并将 SWA 截断到回放片段，接受近似状态：对于从位置 s 开始的回放，位置 i 的 query 关注 SWA keys 的范围为 [max(s, i−W+1), i]。

编码器 SWA 有界回放（Encoder SWA Bounded Replay）：Encoder SWA Bounded Replay 使前缀缓存只依赖全局 KV，从而可以将 SWA KV 从持久化 KV cache 中移除。

当编码器 SWA KV 缺失时，我们回放缓存前缀的最后 n_win 个 token，并将它们与未缓存的后缀一起处理。被回放的 token 只重新生成 SWA KV，复用已缓存的全局 KV 而无需重算或覆写；未缓存的后缀则同时生成全局 KV 与 SWA KV。

按设计，被回放的前缀状态是近似的，因此为未缓存后缀计算的全局 KV 与 SWA KV 依赖于缓存命中位置，在不同位置之间并非数学上完全相同。令人鼓舞的是，我们的实验证据证实，这种有界回放策略几乎不损害回答质量。

解码器 SWA 有界回放（Decoder SWA Bounded Replay）：Decoder SWA Bounded Replay 将解码器前向传播限制在 n_win 个 token，几乎将总前填充计算减半。

在 CED 下，解码器全局 KV 由最终的编码器隐藏状态投影得到。将前填充在编码器处结束的唯一障碍是解码器 SWA KV，它由每个解码器层自身的隐藏状态生成，并且是首批解码步骤所需。由于我们从不会缓存解码器 SWA KV，精确重建它需要在最后 $L_2 \times n_win$ 个 prompt token 上运行 L_2 个解码器层，当短未缓存后缀跟在长已缓存前缀之后时，这一成本很高。因此，我们在这一场景也应用有界回放策略：在每次前填充时，回放 prompt 的最后 n_win 个 token，在相同的 SWA 截断下将其编码器输出送入解码器层，并将得到的解码器 SWA KV 仅用于解码，而不用于前缀缓存。

按设计，重建的解码器 SWA KV 与完整解码器前向传播得到的并不数学等价。同时，我们发现该策略对回答质量仅有可忽略的影响。为增加安全性，我们还在后训练期间模拟相同的回放，以实现训练感知的适配。

4 预训练

4.1 数据构建

文本数据整理（Text Data Curation）：为追求更高的智能，我们超越小规模数据实验所反映的通用、样本级质量，更加关注多样语料之间的整体交互，这些交互能带来独特的信息增益。我们采用更系统、标准化的数据构建流水线，以提高数据质量并优化数据混合。具体而言，基于更全面的评估，我们精心设计了一个跨越模型参数与训练数据的扩展阶梯（scaling ladder），以指导大规模训练。我们过滤掉信息增益有限的模型生成内容，包括能力较弱模型的输出以及低质量的机器翻译文本。我们将此类内容视为隐式重复，因为它们在很大程度上只是对已有信息的重新表述，在长训练周期中可能变得有害。我们还探索了模型在环的数据迭代方法，作为未来大规模合成数据的基础。此外，我们引入更多领域专家来构建细粒度的数据质量评估维度。与上一版本相比，新语料纳入了来自新发布开源仓库、commit、库与新兴框架的更新代码，以覆盖更广泛的编程语言，并更好地反映当代真实软件工程场景。

多模态数据整理（Multimodal Data Curation）：我们的多模态预训练数据集主要由三类数据构成：图文对、交错图文数据与领域特定数据。基于原始网络数据天然提供丰富多模态知识这一前提，我们避免大规模数据合成，而是优先以原生形式清洗并利用数据，以在预训练期间实现最直接、可扩展的视觉知识压缩。在最初的数据收集过程中，我们发现爬取系统过度偏向以文本为中心的网页内容；因此我们从 Common Crawl 重新引导该系统，以改善其对多模态来源的覆盖。对于图文数据，我们从网页中提取图像及其关联的 alt 文本，通过图文相关性阈值进行过滤，并基于图像语义去重。对于

交错数据，我们主要从网页与 PDF 构建该子集。处理大规模多模态语料通常比处理纯文本语料带来更高的 CPU 与磁盘存储成本，这促使我们将交错数据构建组织为开销逐步增加的多个阶段。在图像检索之前，我们应用启发式与统计过滤、去重以及质量模型，以筛选高价值文档。存活的文档随后被组装为交错图文序列，其中以图像感知的方式再次进行过滤与去重。最后，我们使用 SmolVLM（Marafioti et al., 2025）对图文内容进行严格质量评分，从而提取高质量交错数据。在此过程中被过滤掉的文档部分通过筛选与重组回收为额外的图文对。为弥补网络收集数据固有的局限，我们还引入领域特定数据集，以增强模型在细粒度视觉感知（例如视觉定位与指向）、光学字符识别（OCR）以及长尾知识获取方面的能力。我们还收集了大量图像-代码对与计算机使用轨迹，以提升多模态智能体理解能力。

数据整合与去重（Data Integration and Deduplication）：由于我们的纯文本数据与多模态数据经过不同的流水线处理，我们将最终训练语料构建为两类数据源的并集。对于重叠样本，我们用其多模态版本替换纯文本版本，并采用两种配置中较大的 epoch 数。经过这一替换后，所得语料中纯文本与多模态数据的 token 比例为 7:1。我们通过联合预取与分配训练样本，在预训练与上下文扩展期间尽量减少样本重叠。超长文档在混合之前被确定性地预切分，以确保训练 token 在数据分片与训练步之间均匀分布。我们进一步改进了 best-fit packing 算法，使填充率至多为 10^{-4}。

4.2 预训练设置

4.2.1 模型设置

我们将 Transformer 层数设为 40，隐藏维度 d 设为 5120。我们采用因果编码器-解码器（Causal Encoder-Decoder）架构，其中编码器有 20 层，而另外 20 层位于解码器。

前两层中，我们使用纯滑动窗口注意力。其余 18 个编码器层使用压缩率 $m = 2$ 的 CSA2。这些层被划分为三个配置相同的组，每组六层。在每组中，第一层以全量模式（Full Mode）运行，其余五层以复用模式（Reuse Mode）运行。20 个解码器层使用压缩率 $m = 1$ 的 CSA2。这些层被划分为五个组，每组四层。在第一组中，第一层以全量模式运行，其余三层以复用模式运行。其余四组配置相同：第一层以重索引模式（Reindex Mode）运行，其余三层以复用模式运行。对于所有 CSA2 层，我们将索引器查询头数设为 32，索引器头维度设为 128，稀疏注意力所选择的 KV 条目数（即注意力 top-k）设为 512。我们将查询头数设为 64，头维度设为 512，查询压缩维度设为 1280。对于分层稀疏索引器（Hierarchical Sparse Indexer），我们最多选择 2,048 个块、每块 8 个位置，合计至多 16,384 个候选位置。输出投影分组数设为 8，每个中间注意力输出的维度设为 1024。对于滑动窗口注意力的额外分支，窗口大小 n_win 设为 128。我们在所有 Transformer 块中采用 MoE 层，使用带钳制的

SwiGLU 激活函数（OpenAI，2025），阈值设为 10。每个 MoE 层由 1 个共享专家与 384 个路由专家组成，每个专家的中间隐藏维度为 2304。在路由专家中，每个 token 将激活 6 个专家。对于 mHC，扩展因子设为 4，Sinkhorn-Knopp 迭代次数设为 20。对于视觉编码器，我们将其层数设为 32，隐藏维度设为 1024，注意力头数设为 16，图像 patch 大小设为 14。视觉 MLP 投影器有 2 层，隐藏维度为 5120。在此配置下，DeepSeek-V4.1-Flash 包含 552B 骨干参数，预填充（prefill）阶段每个 token 激活 8B，解码（decode）阶段激活 16B。

4.2.2 训练设置

对于线性变换的参数，我们采用 Muon 优化器（Jordan 等，2024；Liu 等，2025）；对于所有 RMSNorm 模块的权重及其他非矩阵参数，使用 AdamW 优化器（Loshchilov 与 Hutter，2019）；对于所有嵌入与预测头，使用 Sinkhorn 平衡更新。对于 AdamW，我们将超参数设为 $\beta1 = 0.9$、$\beta2 = 0.95$、$\varepsilon = 10^{-20}$、weight_decay = 0.1。对于 Muon，我们将动量设为 0.95、权重衰减设为 0.1，并将每个更新矩阵的 RMS 重新缩放到 0.18，以复用 AdamW 的学习率。对于 Sinkhorn 平衡更新，我们使用与 Muon 相同的动量系数与学习率校正因子，并设置 $K = 11$、$\tau = 10^{-3}$、$\varepsilon = 10^{-20}$。遵循（Cheng 等，2026b）的做法，Engram 的学习率被放大 5×。我们在 45T token 的多模态数据上训练 DeepSeek-V4.1-Flash，过程中未出现不稳定现象。整个训练过程中批大小固定为 1.006 亿 token。学习率在前 2000 步内线性预热，随后保持为 2.6×10^{-4} 直至 28T token。在 28T 至 40T token 之间，我们按余弦调度将学习率衰减至 2.6×10^{-5}。从 40T 到 45T token，学习率保持在该值。我们从零开始以稀疏注意力在 64K 序列长度上训练模型，并在 34T token 时将序列长度扩展到 1M。对于无辅助损失的负载均衡，我们将图像与文本 token 的偏置更新速度均设为 0.001，同时保留一个损失权重为 0.0001 的、较小的序列级均衡损失，以避免单个序列内部出现极端不均衡。与 DeepSeek-V4 类似，我们在预训练期间采用样本级注意力掩码。

视觉编码器训练。我们的 DeepSeek-ViT 在与语言骨干整合之前要经历一个单独的训练阶段。该训练流程包含两个阶段：对比预训练与自回归微调。在对比预训练中，我们使用 SigLIP（Zhai 等，2023）提出的 sigmoid 对比损失，在约 47B 图文对上优化模型，这些图文对来自 alt 文本数据。为从如此庞大的数据集中高效学习视觉表示，我们通过缩小较大图像（同时保持其宽高比）将最大输入分辨率限制为 224×224 像素。尽管在这一阶段使用更高分辨率能带来明显增益，但实证结果表明，这些收益对最终模型的贡献很小。由于后续的自回归阶段会专门处理高分辨率外推，因此在对比预训练阶段提高分辨率会显著增加计算开销，而整体提升有限。在自回归微调阶段，我们将视觉编码器连接到一个 4B MoE LLM，并使用下一 token 预测目标在 236B token 上训练，数据包括图像描述、alt 文

本、图表与 OCR 等。该阶段旨在增强编码器对细粒度视觉特征进行建模的能力。因此，我们通过按比例缩放越界图像，将输入分辨率限制在 544×544 至 1344×1344 像素之间。该阶段结束后，我们丢弃该 LLM，只保留优化后的视觉编码器，用于后续的预训练流程，并在其中沿用相同的输入分辨率策略。

4.3 评测

4.3.1 评测基准

我们将 DeepSeek-V4.1-Flash-Base 与其前代模型 DeepSeek-V4-Flash-Base 及 DeepSeek-V4-Pro-Base 进行比较。我们报告的基准覆盖五个关键维度：世界知识、语言理解与推理、代码与数学、长上下文以及多模态能力。

世界知识。基准包括 AGIEval（Zhong 等，2023）、MMLU-Pro（Wang 等，2024b）、C-Eval（Huang 等，2023）、MultiLoKo（Hupkes 与 Bogoychev，2025）、SimpleQA-Verified（Haas 等，2025）与 SuperGPQA（Du 等，2025）。

语言理解与推理。基准包括 BigBench Hard（BBH）（Suzgun 等，2022）、BigBench Extra Hard（BBEH）（Kazemi 等，2025）、DROP（Dua 等，2019）与 HellaSwag（Zellers 等，2019）。

代码与数学。基准包括 BigCodeBench（Zhuo 等，2025）、HumanEval（Chen 等，2021）、GSM8K（Cobbe 等，2021）、MATH（Hendrycks 等，2021）与 MGSM（Shi 等，2023）。

长上下文。基准包括 LongBench-V2（Bai 等，2025）。

多模态。基准包括 MMMU-Pro（Yue 等，2025）、DocVQA（Mathew 等，2021）、CVBench（Tong 等，2024）与 RefCOCO/RefCOCO+/RefCOCO-g（Kazemzadeh 等，2014；Nagaraja 等，2016；Mao 等，2016；Yu 等，2016）。

4.3.2 评测结果

表 1 给出了 DeepSeek-V4-Flash、DeepSeek-V4-Pro 与 DeepSeek-V4.1-Flash 基座模型的详细比较，所有模型均在我们的内部评测框架下、以严格受控且可复现的设置进行评测。与 DeepSeek-V4-Flash-Base 及 DeepSeek-V4-Pro-Base 相比，我们最新的基座模型展现出可观的效率收益。DeepSeek-V4.1-Flash 的激活参数量远少于 DeepSeek-V4-Pro-Base，占用的 KV 缓存也大幅缩减，然而其性能与前代完全持平。这些结果也反映了我们对预训练数据构建流程所做的大幅改进。在本版本中，我们引入了原生多模态训练，并通过相应的多模态评测验证了其有效性。在更多样化、多模态的语料上训练后，DeepSeek-V4.1-Flash 取得了与 DeepSeek-V4-Pro 相当的世界知识与理解能力。在推理与编码基准

上，DeepSeek-V4.1-Flash 表现出持续的进步，在多个基准上达到接近或优于 DeepSeek-V4-Pro 的性能。

【表格】表 1 DeepSeek-V4-Flash-Base、DeepSeek-V4-Pro-Base 与 DeepSeek-V4.1-Flash-Base 之间的比较。所有模型均在我们的内部框架中评测，并采用相同的评测设置。差距不超过 0.3 的分数被视为同一水平。每行最高分以粗体显示，次高分以下划线显示（原文第 24 页）

为在真实研发场景中进一步评估模型能力，我们还在一批专用内部语料上进行了困惑度（perplexity）测试。由于困惑度测试无法通过模型 API 完成，我们主要关注自己预训练的基座模型。在语料选择上，我们基于日常研发收集了一个单独的评测集，包括内部文档、专有代码仓库与学术材料，聚焦于复杂科学问题与前沿研究中的推理、归因与问题求解。结果如图 6 所示，我们报告了不同模型的 bits-per-byte（BPB），数值越低表示性能越好。

【图表】图 6 DeepSeek-V4-Flash-Base、DeepSeek-V4-Pro-Base 与 DeepSeek-V4.1-Flash-Base 在留出评测集上的 bits-per-byte（BPB）对比。DeepSeek-V4.1-Flash-Base 在所有任务上取得最低 BPB，展现出作为强大基座模型的更大潜力（原文第 25 页）

5 后训练

5.1 后训练流程

在本次发布中，我们没有引入新的后训练算法。整体配方遵循标准的监督微调（SFT）→ 强化学习（RL）→ 在线策略蒸馏（on-policy distillation, OPD）（Gu 等，2024；Lu 与 Lab，2025）范式，除成熟实践之外没有算法层面的改动。相反，我们的努力几乎完全集中在"让模型学什么"而非"如何优化模型"上：我们投入大规模自动化流水线进行数据合成与环境构建。具体而言，该流水线（i）合成多样化、可验证的训练任务及其参考解与奖励信号；（ii）以程序化方式构建并扩展交互式智能体环境，使轨迹能够以较低成本被采集与评估；（iii）执行严格的过滤、去重与难度校准，以确保数据质量与课程平衡。我们发现，在固定且平常的优化流程下，合成数据与环境的规模、多样性与可验证性的系统性提升，几乎解释了所有观察到的收益。这一观察呼应了一个更广泛的结论：在当前阶段，工程化数据与环境流水线所带来的边际回报，已显著超过后训练中算法创新的边际回报。

5.1.1 大规模智能体任务合成

任务是智能体学习的基础燃料。然而，构建高质量训练任务历来需要大量人工投入。我们观察到，模型已经开始展现出构建自身训练任务的能力，尽管这一能力仍远未完善。认识到这一潜力后，我们投入了大量精力来增强模型的任务构建与质量验证能力。

我们将每个任务形式化为一个三元组（问题、环境、验证系统），并从两个维度评估其质量：难度——确保任务并非平凡；正确性——保证三个组件中不存在关键缺陷。以难度与正确性作为奖励信号，我们迭代地训练模型构建更好的任务。我们还对 RL 任务的全生命周期进行监控：每当一个任务在新的 RL 运行中被使用，所产生的轨迹都会为质量复审计提供新的证据。在这一总体框架下，我们为两类核心场景构建了专门的训练环境生产流水线：通用智能体与编码智能体。

通用智能体。对于通用智能体，我们鼓励内部员工与外部合作伙伴将我们的最新模型纳入日常工作流，并在自愿基础上回传交互数据与反馈。基于回传数据中观察到的接口，我们构建了大量模拟工具（mocked tools），以复现真实世界工具与系统的接口与行为，包括其输入格式、输出结构、API schema 与行为约束，覆盖常用的 SaaS 与企业应用，以及更专业的业务后端系统。与此同时，我们大规模收集内部员工提交的负面反馈与模型失败案例，并将其纳入流水线，以生成基于真实工作流的单轮与多轮智能体环境。通过重建相关的工具上下文、用户交互模式与失败条件，该流水线能够系统性地重放失败，并针对观察到的模型弱点开展有针对性的强化学习。

编码智能体。编码智能体的训练环境来自两个来源：1）来自内部员工与外部合作伙伴的编码智能体会话，经筛选保留高度复杂的任务或模型表现较差的任务，再按轨迹去重；2）满足 star 数阈值的公开 GitHub 仓库。环境构建由多个专用智能体协作完成。首先，一个智能体判断项目能否在容器内构建并完整运行、能否被自动验证；若满足条件，它选择一个特定的轮次或提交作为任务起点，设计若干足够复杂的实现方向，并产出具体的评测点（包括 fail-to-pass 与 pass-to-pass 测试点）以及一份构建报告，必要时从网络上获取外部资源。接着，另一个智能体在隔离容器中设置依赖、初始工作目录、测试代码与任务描述，执行自测，清除任何可能泄漏任务解法的痕迹，并将环境打包为新的镜像层。随后，多个不同的智能体尝试完成该任务，一个独立的质量检查智能体审查环境以及求解智能体的轨迹，检查环境问题、事实错误、评测点与任务描述不匹配之处以及可被作弊利用的风险。若检查未通过，修复智能体将修复所有已识别的问题、调整过易或过难的评测点，任务重新进入验证。

通过这些流水线，我们可以自动、批量地生产正确、有区分度、且长度与难度可控的 RL 训练数据。无论是通用智能体环境中对真实工作流的忠实重建，还是编码智能体环境中对编码任务的精确构建，最终都汇入统一的训练系统，在持续的质量监控下驱动模型能力的迭代提升。

5.1.2 合成任务中的强化学习

我们在合成任务中使用大规模异步 RL 来提升模型性能，并塑造其在复杂场景中的行为。我们从两个维度扩展 RL 训练：训练算力与脚手架（scaffold）数量。如图 7 与图 8 所示，无论是在单个脚手架内扩展、在同一脚手架的多个变体上联合扩展，还是在异构脚手架之间扩展，性能都随着 RL 累计步数与算力的增加而持续提升。

对于跨多样化脚手架的 RL 训练，我们将智能体的 rollout 执行解耦为一个智能体沙箱与一个 worker 容器。沙箱运行脚手架及其工具，而 worker 提供一个与脚手架无关的控制层，负责编排 rollout、将异构交互归一化为统一的轨迹 schema，并与训练器通信。二者都运行在 DSec（第 5.1.3 节）上，位于可抢占的 GPU 训练池之外，从而将长生命周期的 rollout 与细粒度的训练调度分离。在训练器被抢占期间，rollout 执行可以被挂起并卸载，同时保留其完整状态以便之后恢复，并释放 CPU 与 GPU 资源。这一设计使得在异构脚手架上开展稳定高效的 RL 成为可能，而无需改动底层算法。

【图表】图 7 在 DeepSeek Harness 的 Minimal 模式下，随着 RL 训练规模扩大，各项代码智能体基准上的性能持续提升。进一步将最大上下文长度扩展到 1M token，可继续提升极长程任务（例如 Terminal-Bench v3.0）上的性能

（原文第 27 页）

为将有效 RL 算力扩展到单次训练运行之外，我们使用模型合并来重新初始化后续的 RL 运行。具体而言，我们合并来自不同脚手架或配置的多次运行的检查点，从而整合沿不同优化路径获得的改进。在图 7 与图 8 中，不连续的曲线段反映了模型重新初始化后的连续 RL 运行。这在任务性能与 token 效率两方面都带来进一步收益，为聚合并行 RL 算力并在连续运行中持续扩展提供了一种简单而实用的方法。

5.1.3 大规模运行智能体：DSec

随着我们从 DeepSeek-V3 过渡到 V4，智能体训练环境在数量与多样性上的快速增长促使我们构建了 DeepSeek 弹性计算（DeepSeek Elastic Compute, DSec）——一个面向大规模智能体训练与评测的生产级沙箱平台。其初始设计针对异构执行环境、可扩展的镜像分发、多种隔离后端、高密度资源管理、命令轨迹日志记录以及可安全抢占的恢复能力。

V4.1 的训练进一步将需求提升到数百万个并发沙箱实例，跨越多样化的 harness（运行框架）、平台、代码仓库、软件依赖与任务专用服务。在这一规模下，主要瓶颈转向数据中心可扩展性、工作负载隔离、单节点算力密度，以及如何遏制能力日益增强的智能体的不当行为。下面我们简要描述几个关键设计方面。

【图表】图 8 在多个版本的 Claude Code 上联合训练（左）以及在异构脚手架上联合训练（右，包括 OpenCode、Pi 以及 Standard 与 PTC 模式下的 DeepSeek Harness）时，性能随 RL 累计步数提升。性能在 DeepSWE v1.1 上评测。较浅的曲线显示各个脚手架版本或脚手架的单独评测结果（原文第 28 页）

大规模水平扩展算力。DSec 通过两种互补机制进行扩展：分片，以及以放宽一致性为代价的调度。为容纳大量机器，我们将计算节点划分为多个分片（即所谓的扩展单元）。这种分片还使我们能够通过隔离不同实验的工作负载来缩小影响范围（blast radius），防止单个内存密集型任务耗尽由不相关工作共享的资源。DSec 没有采用 Kubernetes 等现成编排器，而是使用自研的放置引擎来调度大量沙箱，以牺牲强全局一致性来换取可扩展性。这一设计基于一个关键观察：只要每个计算节点强制执行本地安全约束，智能体沙箱的放置就只要求最终一致性。为此，放置引擎以多个独立副本部署，无需同步协调。每个副本根据近期测量预测资源可用性，并做出"足够好"的放置决策。为补偿一致性的损失，每个节点负责验证最终的放置决策，并执行硬性准入约束：若超过本地警告阈值，则拒绝新的放置。整体而言，这一设计使 DSec 能够扩展到数百万个容器，而不会在中心协调上形成瓶颈。

高密度运行沙箱。在节点层面，我们使用硬件支持的 sub-NUMA 分区，并将每个 worker VM 绑定到单独的 NUMA 域。容器在这些 worker VM 内运行，其 CPU 与内存分配被限制在 VM 本地的 NUMA 资源范围内。这一设置在对激进的内存超售与 Linux 内核锁竞争之间取得平衡，同时将内存压力与运行时故障局部化。在可比的工作负载配置下，它使每台物理节点在出现可测量的端到端性能退化之前所支持的并发运行容器密度从约 1,000 个提升到 2,500 个以上。

然而，如此高密度的部署仍可能因后台工作负载的干扰而扭曲对时延敏感的评测。因此 DSec 引入了时延敏感（latency-sensitive, LS）执行类别。我们对非 LS 任务应用 SCHED_IDLE 以最小化其调度优先级，并使用核心调度（core scheduling）确保只有同一优先级类别的任务能在兄弟超线程上同时执行，从而消除干扰。

缓解行为不当的智能体。在 RL 训练期间，我们经常观察到智能体试图进行奖励黑客（reward hacking）或无意中使环境崩溃。在某些尝试中，我们的智能体利用了近期披露的漏洞，包括 XFS 驱动的权限问题、AppArmor 中的非法内存访问、从包镜像服务泄漏答案等。智能体还常常因删除关键二进制文件、破坏系统文件，甚至移除文件系统而恶名昭著。我们使用按沙箱配置的 AppArmor 配置文件与基于 eBPF 的细粒度网络策略来阻止此类尝试。如果智能体使其环境崩溃，我们将该崩溃视为失败的轨迹，并向 RL 框架上报一个"惩戒（repercussion）"信号。

5.1.4 RL 中可控的推理努力

除了模型架构与硬件方面的进展外，输出 token 数量是服务成本的另一项关键决定因素，进而决定了真实应用中的成本-质量权衡。因此，我们在强化学习期间引入一个标量努力等级 b 作为显式的条件信号。该机制同时应用于单轮推理与多轮智能体任务。具体而言，我们在系统提示前拼接如下指令：

Reasoning Effort: {effort} (range 1–100; higher values request more thorough reasoning)

其中 $b \in \{1, ..., 100\}$ 表示所请求的努力等级。

对于每个训练提示 x，我们在每个努力等级 $b \in B$ 上采样 M_b 个回答：

$$z_{b,j} \sim \pi_\theta(\cdot|x, b)，b \in B，j = 1, ..., M_b \qquad （8）$$

其中 j 为在努力等级 b 上采样到的回答的索引。共享同一 (x, b) 的回答构成一个子组，子组内的奖励经均值中心化后用于计算组相对优势。因此，不同努力等级的回答不会被直接比较。取而代之的是，通过让奖励中的长度分量依赖于 b，在每个子组内部诱导出依赖努力等级的行为。具体而言，我们在回答 $z_{b,j}$ 的奖励中加入长度惩罚项 $r^{len}_{b,j}$：

$$r^{len}_{b,j} = -\min(C_{max}, k(b) \cdot \ell_{b,j} / L_{norm}) \qquad （9）$$

其中 $\ell_{b,j}$ 为推理 token 数，L_{norm} 为参考长度，C_{max} 限制施加于一条轨迹的最大扣减量。token 惩罚系数随所请求的努力等级呈指数下降：

$$k(b) = k_0 \cdot \exp(-(b - b_{min}) / \tau)，\tau = \lambda \cdot \Delta b \qquad （10）$$

其中 k_0 是不同努力等级下的基础惩罚系数，b_{min} 是 B 的最小值，Δb 是训练努力等级之间的平均间距，λ 控制惩罚衰减的速率。将 b 增加 τ 会使惩罚系数乘以 e^{-1}。参数 k_0 控制趋向更短推理的总体压力，而更小的 τ 会使惩罚衰减更快，并往往在努力等级之间产生更大的行为差异。附录 C 给出了 k(b) 指数参数化的边际效用动机。

在部署时，标量 b 为模型的推理强度提供了一个灵活的控制接口。通过改变 b，单个模型检查点可以在学习到的成本-质量前沿上于不同运行区间之间移动，以适应不同的时延、token 预算与解答质量要求。尽管训练只使用了有限的一组努力等级，但在部署时可以使用中间值来引出插值后的推理行为，从而为测试时资源分配提供一种细粒度且高效的机制。

在 2026 年 9 月发布的生产部署中，公开 API 提供三个预设的推理努力档位——max、high 与 low——它们直接映射到这一标量接口。如表 2 所示，三个档位分别对应努力值 $b = 100$、$b = 75$ 与 b

= 50，因此 API 用户可以在不改变模型权重或解码配置的情况下，在学习到的成本-质量前沿上选择运行点。

表 2：公开 API 推理努力档位与底层标量努力值 b 的对应关系。（原文第 30 页）

API 档位	努力值 b
max	100
high	75
low	50

5.2 异步后训练基础设施

LLM 强化学习在 rollout 阶段的长尾问题一直是训练效率的主要瓶颈。为解决这一问题，我们扩展了后训练基础设施，使其支持异步生成样本（Zeng 等，2026；Team 等，2026b），通过维持足够高的并发水平，显著缓解了 rollout 阶段的长尾问题。目前，我们几乎所有的 RL 与 OPD 任务都已启用异步训练，rollout 效率得到大幅提升。

5.2.1 整体工作流程

我们将 rollout 与训练部署在同一批物理设备上，并对二者的执行进行分时复用，从而无需手动调优两个阶段之间的资源分配。每个任务都会指定在途样本数量的上界，系统在整个 rollout 阶段维持该上界。

我们评估了三种用于维持目标 rollout 并发的派发粒度。最终方案是样本级派发：一旦新完成的样本数达到分配给下一个提示的 GRPO 组大小，我们就派发该提示，而不论这些完成样本来自哪些组。这有助于在整个训练过程中保持稳定的 rollout 并发。在确定这一派发策略之前，我们尝试了另外两种派发粒度。第一次尝试中，我们在开始时额外派发若干个批次，并在每次训练迭代后补足一个完整批次；然而这导致训练指标出现剧烈振荡，说明批次级粒度过粗。第二次尝试中，我们改用提示级派发，即在一个 GRPO 组完成后派发新提示，但发现它容易在 GRPO 组内的长尾样本上停滞，难以平稳维持目标 rollout 并发。

一旦积累了足够的训练样本，训练就会抢占正在进行的 rollout。

在训练期间，我们使用拼接式路由重放（concatenated routing-replay）：对于跨越多个检查点的样本，我们会将在各 rollout 片段中产生的专家路由拼接起来，而不是丢弃这些路由信息并用新检查点重新计算。

5.2.2 缓解长度偏差与离策略效应

异步生成在有效提升 rollout 效率的同时，也引入了两种可能降低训练质量的副作用。第一，它会产生长度分布偏差，尤其是在训练早期，因为较短的序列往往先完成，从而主导了最初的训练批次。第二，它不可避免地会产生离策略（off-policy）样本，即其 token 部分或全部由较早检查点生成的样本。这两个问题需要不同的处理策略。

为处理长度偏差，我们采用两种机制。第一，调度器可以按数据集限制并发度，这有助于调节每个数据集在稳态训练批次中的比例，通过控制进入样本的来源来间接缓解长度偏斜。第二，我们支持丢弃过早返回的短样本，以平滑进入稳态长度分布的过渡，并防止模型过拟合于过短的序列。

对于离策略问题，我们实现了两种额外机制。第一，通过调整控制样本调度以及训练样本等待条件的逻辑，我们可以限制最大离策略比例，确保训练数据不会过度偏离当前模型。第二，在训练过程中，我们加入了一种损失掩码方案，消除陈旧程度过高的 token 的贡献，从而减轻陈旧样本对梯度更新的不利影响。

5.2.3 性能优化

在我们的异步 RL 框架中，rollout 阶段会被周期性地中断，以切换到更新后的策略检查点。我们力求使这一过程无缝衔接：中断应近乎瞬时完成，被中断的 rollout 应像从未停止过一样恢复。

为了及时停止 rollout，我们支持 token 级中断：生成可以在任意 token 边界处暂停。一旦收集到足够的训练数据，所有正在进行的样本几乎立即停止，使系统能够毫无延迟地进入训练阶段。

为了在检查点切换之间保留 rollout 进度，我们在生成过程中以 token 为粒度持久化 rollout 状态，例如 KV cache 与专家路由。在使用新检查点恢复时，直接复用所持久化的状态，从而消除了重新 prefill 的开销，并让被中断的样本从恰好中断处继续。由于这需要保留所有正在进行样本的状态，我们执行样本粒度的垃圾回收，在每个样本完成时立即释放其状态。

除检查点切换之外，同一套快速中断与无缝恢复机制还使训练任务能够及时响应集群调度的抢占信号而不丢失进度，从而提升集群的整体利用率。

5.2.4 大规模在线策略蒸馏

作为后训练的最后一个阶段，最终的全词表 OPD 任务使用来自所有领域的数据集、基于 40 多个教师模型进行训练。该任务同样采用异步生成以提升 rollout 效率。由于各领域的训练流程不同，每个领域的最佳教师可能来自模型开发的不同阶段。此外，教师模型之间以及教师与学生之间的架构也可

能不同。我们的后训练基础设施可以轻松适配这种设置，支持全词表 OPD 使用数量实际上不受限制、架构异构的教师模型，并以可忽略的成本在它们之间高效切换（DeepSeek-AI，2026b）。

OPD 阶段还需要在训练过程中进行动态重配置。我们持续跟踪模型能力，并可能相应地调整训练配方，包括数据集混合比例、每数据集并发上限与启用的教师模型。在同步训练中，由于 rollout 批次提供了明确的配置边界，这类变更很直接。然而在异步设置下，由不同配置生成的样本可能同时处于进行中。我们的基础设施支持在配置之间进行一致的过渡，而不会打断 rollout 或训练。

5.3 评估

5.3.1 评估设置

我们的后训练评估主要聚焦于推理与智能体（agentic）能力，而知识密集型性能在很大程度上由预训练决定，并在表 1 中报告。

对于推理，我们在 GPQA Diamond（Rein 等，2023）、Humanity's Last Exam（Phan 等，2025）、Codeforces（内部基准）与 MathArena Apex（Dekoninck 等，2025）上评估，温度与 top-p 均设为 1.0。对于智能体能力，我们从以下四类进行评估：

- **代码智能体**：Terminal-Bench 2.1（Merrill 等，2026）、Terminal-Bench 3.0（Marten 等，2026b）、Terminal-Bench 4.0（Marten 等，2026a）、DeepSWE v1.1（DataCurve，2026）、ProgramBench（Yang 等，2026）、NL2Repo-Bench（Ding 等，2025）。
- **网络安全**：SEC-Bench Pro 260505 版本（Lee 等，2026）、CyberGym（Wang 等，2026c）与 ExploitGym（Wang 等，2026b）。
- **通用智能体**：AutomationBench v1.0.6 的公开评估集（Shepard 与 Salimans，2026）、Agents' Last Exam（Sun 等，2026a）（ALE-CLI）。
- **视觉智能体**：Chartography（Garre 等，2026）、BabyVision（Chen 等，2026）、ZeroBench 主集（Roberts 等，2025）。

对于代码智能体，我们使用 DeepSeek Harness 的 Minimal 模式评估 DeepSeek-V4.1-Flash，上下文窗口为 100 万 token，温度设为 1.0，top-p 设为 0.95。为对齐官方设置要求，我们为 DeepSWE v1.1 采用 mini-SWE 脚手架。对于 SEC-Bench Pro，我们特意使用 Claude Code 脚手架，因为其会话压缩设计。对于视觉智能体任务，我们使用 Claude Code 脚手架进行评估，上下文窗口为 512k token，温

度设为 1.0，top-p 设为 0.95。Agents' Last Exam 与 AutomationBench 使用其官方脚手架评估。模型在其他编码脚手架上的性能见表 4。

为减轻编码智能体评估中的奖励攻击（reward hacking），我们限制互联网访问，并从环境中清除 Git 历史。此外，我们在各类环境中自动清除临时构建与包缓存，包括 Go 模块缓存（go/mod）、node modules 依赖产物、编译后的 .jar 文件与 Python 的 pycache 目录。尽管采取了这些防范措施，我们在测试中仍观察到求取漏洞利用的行为——例如在 CyberGym 中反编译 Ubuntu Linux 的核心软件包以寻找漏洞。随着模型能力日益增强，标准评估基础设施（如 Docker 容器与验证脚本）越来越容易被模型钻空子。我们敦促更广泛的研究社区在设计下一代基准时，优先检测并缓解此类行为。

5.3.2 评估结果

【表格】表 3 DeepSeek-V4.1-Flash 与闭源/开源模型的比较。† 表示 HLE 的纯文本子集。最佳结果以粗体标出，次佳结果以下划线标出（原文第 33 页）

如表 3 所示，DeepSeek-V4.1-Flash 在推理与智能体基准上均较其前代 DeepSeek-V4-Flash 实现了显著性能提升，同时与顶级开源及专有模型持平或更优。

在核心推理任务中，DeepSeek-V4.1-Flash 取得 3471 的 Codeforces 评分，超过 DeepSeek-V4-Flash（3289）与 DeepSeek-V4-Pro（3348）。在 MathArena Apex 上，它取得 65.6% 的 Pass@1 准确率，与表现最好的开源基线 Kimi-K3（65.6%）及 DeepSeek-V4-Pro（65.3%）完全持平。此外，其 GPQA Diamond 分数达到 90.9%，较 DeepSeek-V4-Flash（89.9%）稳步提升。

在智能体任务上，性能提升更为显著。值得注意的是，在 DeepSWE v1.1 上，DeepSeek-V4.1-Flash 达到 74.2% 的通过率，较 DeepSeek-V4-Flash（54.4%）大幅跃升，并超过了包括 Opus-5（74.0%）与 GPT-5.6 Sol（73.0%）在内的领先专有模型。在 Terminal-Bench 2.1 上，它取得 90.6%，优于 Opus-5（89.1%）与 GLM-5.3（88.2%）。同样，在 Automation-Bench（54.8%）与 Agents' Last Exam（31.8%）上，DeepSeek-V4.1-Flash 取得了领先分数，超过开源对手与顶级闭源系统。尽管体型紧凑，DeepSeek-V4.1-Flash 展现出最先进的智能体能力，大幅缩小了与前沿闭源模型的差距，同时在开源替代方案中确立了明显优势。

在网络安全任务上，DeepSeek-V4.1-Flash 在开源模型中确立了新的最先进水平。鉴于这些能力具有双重用途（dual-use）性质，我们鼓励社区负责任地使用它们，例如用于防御性安全研究与漏洞修复。

在视觉智能体任务领域，DeepSeek-V4.1-Flash 展现出稳健的能力，尤其是在需要视觉推理与复杂专业图表分析的场景中。虽然它优于领先的开源模型 Kimi-K3，但我们承认，与领先的闭源替代方案相比仍存在可测量的差距。

除峰值性能之外，DeepSeek-V4.1-Flash 还提供了一个推理努力（reasoning-effort）设置，使用户能够以可控方式在推理成本与准确率之间进行权衡。如图 9 所示，无论是推理基准还是智能体基准，准确率与输出长度都随努力程度稳步提升。将努力程度从 25 提升到 100，可使八个推理密集基准的平均 Pass@1 从 67.1% 提升到 76.3%，DeepSWE v1.1 从 66.0% 提升到 74.2%，Terminal-Bench 2.1 从 82.4% 提升到 90.6%，代价是输出 token 增加约 2.5 倍。值得注意的是，在单回复推理上学到的努力程度控制能够忠实地迁移到长时程智能体轨迹中，在那里它控制着跨轮次的探索与验证总量。收益是前置的：60–80 区间已经以不到最大设置一半的 token 预算恢复了最大设置的绝大部分准确率，而最后提升到努力程度 100 会使智能体轨迹延长 1.6–1.8 倍，却只带来边际改进。因此，最高档最适合留给最具挑战性的任务，而中等努力程度则为日常智能体使用提供了有利的成本-性能平衡。

5.3.3 不同推理努力下的性能

图 9 报告了一系列预算取值下的性能与平均回复长度。随着推理努力程度提高，模型生成越来越长的推理轨迹，且在推理密集基准与软件工程任务上准确率单调提升，最大增益集中在低到中等预算区间，此后收益递减。尽管我们的 RL 只涉及有限数量的努力档位，但使用标量努力值可以在特定范围内实现对回复长度的灵活插值控制。这使实践者能够沿着平滑的连续谱在质量与延迟、token 成本之间进行权衡：延迟敏感的应用可以在低努力下运行，仅带来适度的准确率下降；而困难任务可以调用高努力，以恢复模型完整的推理能力。在我们的公开 API 服务中，我们暴露三个预设努力档位映射到该刻度：low、high 与 max 分别对应努力值 50、75 与 100。

【图表】图 9 性能与输出长度随推理努力程度的变化。每个面板绘制了 Pass@1（实线，左轴）与每个回复的平均输出 token 数（虚线，右轴）随推理努力值从 25 到 100 的变化；推理密集基准的结果为八个基准（AIME 2026、Apex 2025 Shortlist、GPQA Diamond、HLE、IMO-AnswerBench、LiveCodeBench、MathArena-Apex、SimpleQA-Verified）的平均值。DeepSWE v1.1 基于 mini-SWE 评估，Terminal-Bench v2.1 基于 DeepSeek Harness（Minimal）评估（原文第 35 页）

5.3.4 跨智能体脚手架的性能

在实践中，模型很少被部署在单一固定的智能体框架内；不同脚手架在系统提示、工具定义、上下文管理策略与交互协议上各不相同，而过度拟合某一特定 harness 的模型在换到另一个 harness 时可

能会显著退化。为评估我们的模型对此类变化的稳健性，我们的比较覆盖了来自六个脚手架家族的八种配置：Claude Code（Anthropic，2026）、Codex（OpenAI，2026）、OpenCode（Anomaly，2026）、Pi（Zechner，2026）、mini-SWE（Yang 等，2024），以及 DeepSeek Harness（DSH）（DeepSeek-AI，2026a）的 Minimal、Standard 与 PTC 模式。对于每个脚手架，我们保持模型检查点、解码配置与任务集完全一致，只改变其外围 harness，包括其原生系统提示、工具 schema 与轮次逻辑。表 4 报告了在最大推理努力（100）下于 DeepSWE v1.1 与 Terminal-Bench v2.1 上的性能。模型的智能体能力能够很好地迁移到具有不同提示与工具接口的脚手架家族，而不是依赖某一特定 harness 的约定。随着外围交互协议与工具抽象的变化，其性能依然稳健，表明其智能体行为并未紧密耦合于单一脚手架设计。这种稳健性与我们合成训练数据（第 5 节）中环境、工具 schema 与交互格式的多样性一致，该数据旨在鼓励跨智能体脚手架的泛化。

表 4：最大推理努力下跨智能体脚手架的性能。（原文第 35 页）

基准（指标）	Claude Code	Codex	OpenCode	Pi	mini-SWE	DeepSeek Harness Minimal	DeepSeek Harness Standard	DeepSeek Harness PTC
DeepSWE v1.1（Resolved）	69.8	65.6	65.5	66.2	74.2	72.6	70.5	67.6
Terminal-Bench v2.1（Pass@1）	88.0	84.1	85.0	86.1	90.3	90.6	85.8	85.8

注：所有脚手架在 DeepSWE v1.1 上每个任务使用 N = 8 个样本，在 Terminal-Bench v2.1 上使用 N = 3 个样本。运行使用 Linux 容器，温度 1.0、top-p 0.95、100 万 token 的上下文窗口，并在两个基准上对每个智能体设置 max_steps=500 个模型生成轮次。Terminal-Bench v2.1 在不访问网络的情况下评估。我们评估了四个 Claude Code 版本，各版本结果及其平均值见附录表 5；本表报告的是 v2.1.251。更多与脚手架相关的配置详见附录 B.1。

5.3.5 多智能体

为了探索复杂任务上的多智能体协作，我们使用 DeepSeek Harness 的 Agent Team 模式对 DeepSeek-V4.1-Flash 进行了初步实验。

多智能体 harness：我们使用 DeepSeek Harness 的 Agent Team 模式，其中 lead 智能体默认可以通过 spawn_teammate 异步创建具名、持久的队友。每个队友接收一个委派任务，并以 fresh 模式（不含 lead 历史）或 fork 模式（包含 lead 已完成轮次的一次性快照）启动。所有智能体共享同一份仓库检出，使彼此的编辑立即可见。

智能体通过持久的对等邮箱进行通信。通过 send_message 发送的消息会在运行中的队友的下一个步骤边界处送达；对空闲队友会开启新一轮；对不活跃队友则将其恢复。在每个队友的任务生命周期中，lead 使用 list_agents 监控运行时状态，并使用 wait_agent 等待状态、邮箱或共享任务的变化。任务归属、依赖关系与建议性写入范围维护在共享任务板上（使用四个 team_task_* 工具，并在更新时进行修订检查）。

当需要干预时，只有 lead 可以通过 interrupt_agent 中断队友当前的回合。所需工作完成后，lead 会审查并测试合并后的改动，并生成最终回复。

训练：我们使用组合了任务性能、鼓励委派与智能体间通信的协作奖励、以及促进高效协调的派生延迟惩罚的 RL 奖励来训练 Agent Team 模式。派生延迟的计算方式是：将执行事件及其协作依赖表示为一个有向无环图（DAG），根据固定 prefill/decode 速率下的 token 数加上实测工具执行时间赋以成本，并取关键路径的长度。这鼓励有用的并行，同时惩罚不必要的串行工作与同步，并降低对服务端批处理与排队延迟的敏感性。

性能：我们构建了 ProgramBench（Yang 等，2026）的一个高置信度子集，只保留参考解在隐藏测试套件上通过率至少为 95% 的任务。该过滤过程留下 172 个“黄金”任务。我们还在 FrontierSWE v2（Kondra 等，2026）上评估，它是 FrontierSWE 的规模更大、更具挑战性的后继版本，采用了大幅改进的方法论。我们通过排除需要 GPU 访问的任务，从当前公开任务中构建了一个无 GPU 子集。我们在显式的每次 rollout 挂钟时间截止期限下，对单智能体与多智能体配置在这两个基准上进行评估。这里报告的结果是初步的：我们将观察到的最强多智能体配置与可用的最强单智能体基线进行比较。在 ProgramBench 上，我们每个任务最多运行三次 rollout，每个配置对应 516 次计划 rollout。我们报告 Almost@1，它衡量的是单次 rollout 得分至少为 0.95 的比例。在 FrontierSWE v2 上，我们报告 Mean@5。ProgramBench 的截止期限从 1 小时到 12 小时，而 FrontierSWE v2 在从 1 小时到 20 小时的截止期限下评估。在每个截止期限，指标均根据截止时刻可获得的输出计算。如图 10 所示，在两个基准的每个截止期限上，多智能体配置都优于对应的单智能体配置。在 ProgramBench 上，多智能体配置的 Almost@1 从 1 小时的 13.59% 升至 8 小时的峰值 30.04%，而单智能体配置为 12.79% 与

20.39%。在 FrontierSWE v2 上，多智能体配置的 Mean@5 从 1 小时的 13.50% 升至 20 小时的 32.90%，而单智能体配置从 10.50% 升至 28.20%。

【图表】图 10 单智能体与多智能体配置在 ProgramBench（Almost@1）与 FrontierSWE v2（Mean@5）上的测试时计算扩展，随每次 rollout 的挂钟时间截止期限变化（原文第 36 页）

6 结论、局限与未来方向

在本工作中，我们提出 DeepSeek-V4.1-Flash，这是一个多模态混合专家（Mixture-of-Experts, MoE）模型，支持最长 100 万 token 的上下文。通过对模型架构、缓存精度与部署策略的联合优化，DeepSeek-V4.1-Flash 突破了 KV cache 压缩的极限。其因果编码器-解码器（Causal Encoder-Decoder, CED）架构使模型在 prefill 阶段每个 token 仅激活 8B 参数，而在 decode 阶段为 16B，提升了输入密集型智能体工作负载的成本效率。在相同序列长度下，Compressed Sparse Attention 2（CSA2）中的跨层 KV cache 复用与 FP4 KV 缓存将其全局 KV cache 占用（始终位于 HBM）降至每 token 890 字节，约为 DeepSeek-V4-Flash 相应占用的 1/4。SWA Bounded Replay 进一步将其持久化 KV cache 占用（始终位于 SSD 或主机内存）降至 DeepSeek-V4-Flash 的约 1/8。这些缩减缓解了 HBM 与 SSD 的容量压力，同时模型整体性能显著优于 DeepSeek-V4-Flash。尽管参数规模远小于 GLM-5.3 与 Kimi-K3 等当代开源模型，DeepSeek-V4.1 在关键基准上取得了相当——在若干任务上甚至更优——的性能。

尽管 DeepSeek-V4.1-Flash 相对 DeepSeek-V4-Flash 大幅简化了若干架构组件，但新引入的架构变更也带来了尚未被充分刻画的稳健性边界。我们的内部评估覆盖了多样化的测试用例与边界条件，在已评估的设置中我们没有观察到模型能力的系统性退化。然而，任何有限的测试套件都无法覆盖所有极端输入与部署条件。CSA2 中潜在的选择错误以及 SWA Bounded Replay 中的近似状态重建，仍可能在未测试的边界情况下导致能力退化。展望未来，我们将继续扩展压力测试与评估体系，特别关注长上下文上的稀疏检索与缓存恢复边界处的 SWA 状态重建。我们还将监控真实工作负载，系统性地刻画潜在失效模式与稳健性边界，并进一步提升模型在极端条件下的稳健性。

随着 AI 模型取得卓越的性能，标准评估基准日益趋于饱和。虽然 DeepSeek-V4.1-Flash 的表现已接近 Fable-5 与 GPT-6 Astra 等顶级模型——在日常应用中提供高度可比的用户体验——但在最具挑战性的任务上仍存在性能差距。尽管基准分数差距很小，这种持平并不意味着模型在复杂、高难度推理与边缘案例上达到了领先闭源系统的前沿能力。

因此，我们将持续更新评估协议，以确保对最先进推理边界的严格评估。在继续努力降低模型成本的同时，我们相信模型智能的进一步进步将取决于数据、模型容量与 RL 的协同扩展。以 DeepSeek-V4.1-Flash 为新的起点，我们将继续探索模型能力的极限，并系统性地解决大规模数据合成与 RL 扩展中的关键挑战。我们还将积极推动模型与 harness 的协同设计（model–harness co-design），使整个系统能够共同演进与优化。通过同步推进成本降低与能力扩展，我们希望让高能力智能体更易获得、更易部署，进一步降低在更广泛行业与场景中采用 AI 技术的门槛。

附录

附录 A 作者列表

附录 B 评测细节

附录 C 脚手架配置

附录 D 跨脚手架的推理努力

附录 E 不同推理努力下推理基准的详细结果

附录 F 推理努力控制中的指数 token 惩罚

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

[2] E. Alvarez, O. Almog, E. Chung, S. Layton, D. Stosic, R. Krashinsky, and K. Aubrey. Introducing nvfp4 for efficient and accurate low-precision inference, 2025. URL https://developer. nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/ .

[3] Anomaly. Opencode. https://github.com/anomalyco/opencode , 2026.

[4] Anthropic. Claude code. https://code.claude.com/docs/en/overview , 2026.

[5] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 3639–3664, 2025.

[6] Y. Bai, Q. Dong, T. Jiang, X. Lv, Z. Du, A. Zeng, J. Tang, and J. Li. Indexcache: Accelerating sparse attention via cross-layer index reuse. arXiv preprint arXiv:2603.12201, 2026.

[7] W. Brandon, M. Mishra, A. Nrusimha, R. Panda, and J. Ragan-Kelley. Reducing transformer keyvalue cache size with cross-layer attention. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, Advances in Neural Information Processing Systems, volume 37, pages 86927–86957. Curran Associates, Inc., 2024. doi: 10.52202/07901 7-2758. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/9 e23d020c18e4c40d81c6a0fc7a46f68-Paper-Conference.pdf .

[8] L. Chen, D. Xu, C. An, X. Wang, Y. Zhang, J. Chen, Z. Liang, F. Wei, J. Liang, Y. Xiao, et al. Powerattention: exponentially scaling of receptive fields for effective sparse attention. arXiv preprint arXiv:2503.03588, 2025.

[9] L. Chen, W. Xie, Y. Liang, H. He, H. Zhao, Z. Yang, Z. Huang, H. Wu, H. Lu, Y. Bao, et al. Babyvision: Visual reasoning beyond language. arXiv preprint arXiv:2601.06521, 2026.

[10] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P . de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P . Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P . Tillet, F. P . Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V . Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P . Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. CoRR, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374 .

[11] X. Cheng, X. Yu, C. Shao, J. Li, Y. Xiong, Y. Qian, J. Zhu, S. Ma, X. Zhang, J. Ye, et al. Dspark: Confidence-scheduled speculative decoding with semi-autoregressive generation. arXiv preprint arXiv:2607.05147, 2026a.

[12] X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, Y. Li, H. Zhang, H. Zhang, D. Zhao, and W. Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models. CoRR, abs/2601.07372, 2026b. doi: 10.48550/ARXIV.2601. 07372. URL https://doi.org/10.48550/arXiv.2601.07372 .

[13] X. Cheng, W. Zeng, D. Dai, Q. Chen, B. Wang, Z. Xie, K. Huang, X. Yu, Z. Hao, H. Zhang, et al. Conditional memory via scalable lookup: A new axis of sparsity for large language models. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 4968–4990, 2026c.

[14] K. Cobbe, V . Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. arXiv preprint arXiv:2110.14168, 2021.

[15] D. Dai, C. Deng, C. Zhao, R. X. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P . Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. CoRR, abs/2401.06066, 2024. URL https://doi.org/10.48550/arXiv.2401.06066 .

[16] DataCurve. Deepswe v1.1, 2026. URL https://deepswe.datacurve.ai/ .

[17] DeepSeek-AI. Deepseek-v3 technical report. CoRR, abs/2412.19437, 2024. URL https://doi. org/10.48550/arXiv.2412.19437 .

[18] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. CoRR, abs/2405.04434, 2024. URL https://doi.org/10.48550/arXiv.2405.04 434 .

[19] DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556 .

[20] DeepSeek-AI. Deepseek harness: Everything is a plugin. https://github.com/deepseek-a i/deepseek-harness , 2026a.

[21] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence. CoRR, abs/2606.19348, 2026b. URL https://doi.org/10.48550/arXiv.2606.19348 .

[22] J. Dekoninck, N. Jovanovi´ c, I. Petrov, and M. Vechev. Matharena apex: Unconquered final-answer problems, 2025. URL https://matharena.ai/apex/ .

[23] S. Deng, Z. Ouyang, T. Pang, Z. Liu, R. Jin, S. Yu, and Y. Yang. Rmnp: Row-momentum normalized preconditioning for scalable matrix-based optimization. arXiv preprint arXiv:2603.20527, 2026.

[24] J. Ding, S. Long, C. Pu, H. Zhou, H. Gao, X. Gao, C. He, Y. Hou, F. Hu, Z. Li, et al. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. arXiv preprint arXiv:2512.12730, 2025.

[25] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In International Conference on Learning Representations, 2021. URL https://openreview.net/forum?id=YicbFdNTTy .

[26] X. Du, Y. Yao, K. Ma, B. Wang, T. Zheng, K. Zhu, M. Liu, Y. Liang, X. Jin, Z. Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines. arXiv preprint arXiv:2502.14739, 2025. D. Dua, Y. Wang, P . Dasigi, G. Stanovsky, S. Singh, and M. Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In J. Burstein, C. Doran, and T. Solorio, editors, Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers), pages 2368– 2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL https://doi.org/10.18653/v1/n19-1246 .

[27] Y. Gao, J. Wei, Q. Zhang, Y. Cheng, S. Chen, Z. Tang, Z. Jiang, Y. Song, H. Zhang, L. Zhao, B. Yang, G. Wang, S. Cao, and F. Luo. Hysparse: A hybrid sparse attention architecture with oracle token selection and kv cache sharing. arXiv preprint arXiv:2602.03560, 2026.

[28] S. Garre, C. Mutty, S. Mehta, and E. Chen. Chartography: A benchmark for professional chart understanding. arXiv preprint arXiv:2608.10677, 2026.

[29] A. Glentis, J. Li, A. Han, and M. Hong. Memory-efficient llm pretraining via minimalist optimizer design. arXiv preprint arXiv:2506.16659, 2025.

[30] Y. Gu, L. Dong, F. Wei, and M. Huang. Minillm: Knowledge distillation of large language models. In The Twelfth International Conference on Learning Representations, 2024.

[31] L. Haas, G. Yona, G. D'Antonio, S. Goldshtein, and D. Das. Simpleqa verified: A reliable factuality benchmark to measure parametric knowledge. arXiv preprint arXiv:2509.07968, 2025.

[32] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset. arXiv preprint arXiv:2103.03874, 2021.

[33] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, et al. C-Eval: A multi-level multi-discipline chinese evaluation suite for foundation models. arXiv preprint arXiv:2305.08322, 2023.

[34] D. Hupkes and N. Bogoychev. Multiloko: a multilingual local knowledge benchmark for llms spanning 31 languages. CoRR, abs/2504.10356, 2025. doi: 10.48550/ARXIV.2504.10356. URL https://doi.org/10.48550/arXiv.2504.10356 .

[35] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), June 2018.

[36] K. Jordan, Y. Jin, V . Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. Cited on, page 10, 2024.

[37] M. Kazemi, B. Fatemi, H. Bansal, J. Palowitch, C. Anastasiou, S. V . Mehta, L. K. Jain, V . Aglietti, D. Jindal, Y. P . Chen, et al. Big-bench extra hard. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 26473–26501, 2025.

[38] S. Kazemzadeh, V . Ordonez, M. Matten, and T. Berg. Referitgame: Referring to objects in photographs of natural scenes. In Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP), pages 787–798, 2014. R. Kondra, S. Mhatre, A. Kumar, E. Chu, B. B. Ahmad, A. Nangia, R. Agarwal, A. Dasgupta, A. Sinha, B. Sridharan, K. Dave, B. Graham, G. Song, A. Rahul, W. H. Lim, A. Thangamuthu, R. Singh, D. Liu, N. Pour, C. Chen, and J. Mattern. Frontierswe v2. Proximal Blog, 2026. https://frontierswe.com/blog/v2.

[39] H. Lee, J. Liu, D. Kim, W. Xia, Z. Zhang, C. S. Xia, and L. Zhang. Sec-bench pro: Can language models solve long-horizon software security tasks? arXiv preprint arXiv:2605.26548, 2026.

[40] J. Li and S. Liu. Flashmla: Efficient multi-head latent attention kernels. https://github.com /deepseek-ai/FlashMLA , 2025.

[41] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang. Muon is scalable for LLM training. CoRR, abs/2502.16982, 2025. URL https://doi.org/10.48550/arXiv.2502.16982 .

[42] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In International Conference on Learning Representations, 2019. URL https://openreview.net/forum?id=Bkg6Ri CqY7 .

[43] K. Lu and T. M. Lab. On-policy distillation. Thinking Machines Lab: Connectionism, 2025. doi: 10.64434/tml.20251026. https://thinkingmachines.ai/blog/on-policy-distillation.

[44] J. Mao, J. Huang, A. Toshev, O. Camburu, A. L. Yuille, and K. Murphy. Generation and comprehension of unambiguous object descriptions. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 11–20, 2016.

[45] A. Marafioti, O. Zohar, M. Farré, M. Noyan, E. Bakouch, P . Cuenca, C. Zakka, L. B. Allal, A. Lozhkov, N. Tazi, V . Srivastav, J. Lochner, H. Larcher, M. Morlon, L. Tunstall, L. von Werra, and T. Wolf. Smolvlm: Redefining small and efficient multimodal models. arXiv preprint arXiv:2504.05299, 2025.

[46] R. Marten, A. Shaw, I. Bercovich, B. Droste, T. Cerruti, S. Dillmann, R. Wang, D. Wahdany, A. Hart, K. Krauth, ScaleAI, Snorkel AI, Turing, gNucleus AI, Boolean AI, N. Carlini, S. Lyu, A. Wei, A. Khatua, B. Plüster, C. Dwivedi, C. Sutcliffe, Yuming, D. Tivris, D. Wang, H. W. Goh, H. Xing, H. Lin, I. Salia, J. Seol, J. Bao, J. Ouyang, J. Park, L. Walsh, L. Kong, M. Ivanov, M. Ubl, M. Liamets, O. Menis, P . Migdal, Q. Bao, R. Movva, R. Ben Chaim, N. Srinath, S. Bogdanik, S. Yadav, S. Benjamin, T. Kung, W. Hughes, X. Lan, H. Gupta, S. Mishra, C. Wang, H. He, J. Tu, K. Montgomery, Z. Tu, A. Naik, D. Mortensen, I. Zhang, Y. Mathur, E. Liu, K. Singh, M. Yu, S. Feng, V . Gangal, Z. Tao, S. Ruan, J. Mueller, J. Cabezas, J. Bauer, K. X. Li, R. Zhang, A. Feller, A. Madayan, L. Chen, B. Feuer, X. Li, B. Li, H. Raj, S. Galler, L. Shi, I. Segal, K. Buchanan, S. P ., R. Desai, A. Schneider, C. Settles, X. Lin, M. Nezhurina, A. Wang, M. Kowalczyk, J.-X. Zhao, S. Satia, J. Hu, S. Atef, K. Chen, S. Vance, G. Segato, J. Jitsev, A. Dimakis, M. Merrill, A. Konwinski, and L. Schmidt. Terminal-Bench, Aug. 2026a. URL https://github.com/harbor-framework/terminal-bench .

[47] R. Marten, A. Shaw, A. Konwinski, and Terminal-Bench Contributors. Terminal-bench 3.0: Harder tasks for better agents. https://www.tbench.ai/news/terminal-bench-3-0 , 2026b. An Open Benchmark for Agent Work in Terminal Environments.

[48] M. Mathew, D. Karatzas, and C. Jawahar. Docvqa: A dataset for vqa on document images. In 2021 IEEE Winter Conference on Applications of Computer Vision (WACV), pages 2199–2208. IEEE, 2021. M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint arXiv:2601.11868, 2026.

[49] V . K. Nagaraja, V . I. Morariu, and L. S. Davis. Modeling context between objects for referring expression understanding. In European conference on computer vision, pages 792–807. Springer, 2016.

[50] Y. Nesterov. A method of solving a convex programming problem with convergence rate $O(1/k)$. Soviet Mathematics Doklady, 27:372–376, 1983.

[51] OpenAI. gpt-oss-120b & gpt-oss-20b model card. CoRR, abs/2508.10925, 2025. doi: 10.48550/A RXIV.2508.10925. URL https://doi.org/10.48550/arXiv.2508.10925 .

[52] OpenAI. Codex. https://github.com/openai/codex , 2026.

[53] L. Phan, A. Gatti, Z. Han, N. Li, J. Hu, H. Zhang, C. B. C. Zhang, M. Shaaban, J. Ling, S. Shi, et al. Humanity's last exam. arXiv preprint arXiv:2501.14249, 2025.

[54] Y. Qian, S. Liu, and Y. Li. Deepselect: High-performance topk kernels for deepseek sparse attention and sampling. https://github.com/deepseek-ai/DeepSelect , 2026.

[55] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. GPQA: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022, 2023.

[56] J. Roberts, M. R. Taesiri, A. Sharma, A. Gupta, S. Roberts, I. Croitoru, S.-V . Bogolin, J. Tang, F. Langer, V . Raina, et al. Zerobench: An impossible visual benchmark for contemporary large multimodal models. arXiv preprint arXiv:2502.09696, 2025.

[57] B. D. Rouhani, R. Zhao, A. More, M. Hall, A. Khodamoradi, S. Deng, D. Choudhary, M. Cornea, E. Dellinger, K. Denolf, S. Dusan, V . Elango, M. Golub, A. Heinecke, P . James-Roxby, D. Jani, G. Kolhe, M. Langhammer, A. Li, L. Melnick, M. Mesmakhosroshahi, A. Rodriguez, M. Schulte, R. Shafipour, L. Shao, M. Siu, P . Dubey, P . Micikevicius, M. Naumov, C. Verrilli, R. Wittig, D. Burger, and E. Chung. Microscaling data formats for deep learning, 2023.

[58] M. Scetbon, C. Ma, W. Gong, and E. Meeds. Gradient multi-normalization for efficient LLM training. In The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2025. URL https://openreview.net/forum?id=oanhUGY6un .

[59] N. Shazeer. Glu variants improve transformer. arXiv preprint arXiv:2002.05202, 2020.

[60] N. Shazeer and M. Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In International conference on machine learning, pages 4596–4604. PMLR, 2018.

[61] D. Shepard and R. Salimans. Automationbench. arXiv preprint arXiv:2604.18934, 2026.

[62] F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, D. Das, and J. Wei. Language models are multilingual chain-of-thought reasoners. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL https://openreview.net/forum?id=fR3wGCk-IXp .

[63] Y. Sun, L. Dong, Y. Zhu, S. Huang, W. Wang, S. Ma, Q. Zhang, J. Wang, and F. Wei. You only cache once: Decoder-decoder architectures for language models. Advances in Neural Information Processing Systems, 37:7339–7361, 2024. Y. Sun, X. Han, W.

Zhang, Y. Pang, T. Wang, Y. Cao, Y. Huang, C. Duroiu, H. Zhang, J. Lin, et al. Agents' last exam. arXiv preprint arXiv:2606.05405, 2026a.

[64] Y. Sun, Y. Zhang, L. Dong, J. Wang, and F. Wei. You only index once: Cross-layer sparse attention with shared routing. arXiv preprint arXiv:2606.06467, 2026b.

[65] M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V . Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

[66] K. Team, T. Bai, Y. Bai, Y. Bao, J. Cai, X. Cai, P . Cao, Y. Cao, Z. Chai, Y. Charles, et al. Kimi k3: Open frontier intelligence. arXiv preprint arXiv:2607.24653, 2026a.

[67] K. Team, T. Bai, Y. Bai, Y. Bao, S. Cai, Y. Cao, Z. Chai, Y. Charles, H. Che, C. Chen, et al. Kimi k2.5: Visual agentic intelligence. arXiv preprint arXiv:2602.02276, 2026b.

[68] M. L. Team, B. Wang, B. Xiao, B. Zhang, B. Rong, B. Chen, C. Wan, C. Zhang, C. Huang, C. Chen, et al. Longcat-flash-omni technical report. arXiv preprint arXiv:2511.00279, 2025.

[69] S. Tong, E. Brown, P . Wu, S. Woo, M. Middepogu, S. C. Akula, J. Yang, S. Yang, A. Iyer, X. Pan, A. Wang, R. Fergus, Y. LeCun, and S. Xie. Cambrian-1: A fully open, vision-centric exploration of multimodal llms, 2024.

[70] L. Wang, H. Gao, C. Zhao, X. Sun, and D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. CoRR, abs/2408.15664, 2024a. URL https://doi.org/10.48550/arX iv.2408.15664 .

[71] X. Wang, C. Xu, H. Cao, R. Tian, W. Zhao, K. Yu, and C. Zhao. Tilekernels. https://github.c om/deepseek-ai/TileKernels , 2026a.

[72] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, T. Li, M. Ku, K. Wang, A. Zhuang, R. Fan, X. Yue, and W. Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. CoRR, abs/2406.01574, 2024b. URL https://doi.org/10.48550/arXiv.2406.01574 .

[73] Z. Wang, N. Schiller, H. Li, S. S. Narayana, M. Nasr, N. Carlini, X. Qi, E. Wallace, E. Bursztein, L. Invernizzi, et al. Exploitgym: Can ai agents turn security vulnerabilities into real attacks? arXiv preprint arXiv:2605.11086, 2026b.

[74] Z. Wang, T. Shi, J. He, M. Cai, J. Zhang, and D. Song. Cybergym: Evaluating ai agents' real-world cybersecurity capabilities at scale. In International Conference on Learning Representations, volume 2026, pages 123341–123386, 2026c.

[75] Z. Wen, Y. Shi, J. Wang, P . Luo, L. Qiao, D. Li, and T. Sun. Sron: State-free llm training via row-wise gradient normalization. 2025.

[76] Z. Xie, Y. Wei, H. Cao, C. Zhao, C. Deng, J. Li, D. Dai, H. Gao, J. Chang, K. Yu, L. Zhao, S. Zhou, Z. Xu, Z. Zhang, W. Zeng, S. Hu, Y. Wang, J. Yuan, L. Wang, and W. Liang. mhc: Manifoldconstrained hyper-connections, 2026. URL https://arxiv.org/abs/2512.24880 .

[77] R. Xu, J. Li, and Y. Lu. On the width scaling of neural optimizers under matrix operator norms i: Row/column normalization and hyperparameter transfer. arXiv preprint arXiv:2603.09952, 2026a. Y. Xu, F. Meng, F. Jiang, Y. Wang, R. Zhou, Z. Wang, J. Wu, Z. Pan, X. Tang, W. Pei, et al. Hisa: Efficient hierarchical indexing for fine-grained sparse attention. arXiv preprint arXiv:2603.28458, 2026b.

[78] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024. URL https://arxi v.org/abs/2405.15793 .

[79] J. Yang, K. Lieret, J. Ma, P . Thakkar, D. Pedchenko, S. Sootla, E. McMilin, P . Yin, R. Hou, G. Synnaeve, D. Yang, and O. Press. Programbench: Can language models rebuild programs from scratch?, 2026. URL https://arxiv.org/abs/2605.03546 .

[80] L. Yu, P . Poirson, S. Yang, A. C. Berg, and T. L. Berg. Modeling context in referring expressions. In European conference on computer vision, pages 69–85. Springer, 2016.

[81] J. Yuan, J. Zou, S. Wang, Y. Liu, and F. Nie. Nora: Normalized orthogonal row alignment for scalable matrix optimizer. arXiv preprint arXiv:2605.03769, 2026.

[82] X. Yue, T. Zheng, Y. Ni, Y. Wang, K. Zhang, S. Tong, Y. Sun, B. Yu, G. Zhang, H. Sun, et al. Mmmupro: A more robust multi-discipline multimodal understanding benchmark. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 15134–15186, 2025.

[83] M. Zechner. Pi: The coding-agent harness you can make your own. https://github.com/e arendil-works/pi , 2026.

[84] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In A. Korhonen, D. R. Traum, and L. Màrquez, editors, Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers, pages 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL https://doi.org/10.18653/v1/p1 9-1472 .

[85] A. Zeng, X. Lv, Z. Hou, Z. Du, Q. Zheng, B. Chen, D. Yin, C. Ge, C. Huang, C. Xie, et al. Glm-5: from vibe coding to agentic engineering. arXiv preprint arXiv:2602.15763, 2026.

[86] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In 2023 IEEE/CVF International Conference on Computer Vision (ICCV), pages 11941–11952. IEEE, 2023.

[87] B. Zhang and R. Sennrich. Root mean square layer normalization. Advances in neural information processing systems, 32, 2019.

[88] Y. Zhang. On the principles behind neural network optimizers. arXiv preprint arXiv:2608.16760, 2026.

[89] Y. Zhang, C. Chen, T. Ding, Z. Li, R. Sun, and Z.-Q. Luo. Why transformers need adam: A hessian perspective. Advances in neural information processing systems, 37:131786–131823, 2024.

[90] Y. Zhang, C. Chen, Z. Li, T. Ding, C. Wu, D. D. Kingma, Y. Ye, Z.-Q. Luo, and R. Sun. Adammini: Use fewer learning rates to gain more. In International Conference on Learning Representations, volume 2025, pages 28033–28063, 2025a. Z. Zhang, Y. Zhong, Y. Jiang, H. Hu, J. Sun, Z. Ge, Y. Zhu, D. Jiang, and X. Jin. Disttrain: Addressing model and data heterogeneity with disaggregated training for multimodal large language models. In Proceedings of the ACM SIGCOMM 2025 Conference, pages 24–38, 2025b.

[91] C. Zhao, L. Zhao, J. Li, Z. Xu, and C. Xu. Deepgemm: clean and efficient fp8 gemm kernels with fine-grained scaling. https://github.com/deepseek-ai/DeepGEMM , 2025.

[92] W. Zhong, R. Cui, Y. Guo, Y. Liang, S. Lu, Y. Wang, A. Saied, W. Chen, and N. Duan. AGIEval: A human-centric benchmark for evaluating foundation models. CoRR, abs/2304.06364, 2023. doi: 10.48550/arXiv.2304.06364. URL https://doi.org/10.48550/arXiv.2304.06364 .

[93] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, J. Hoang, A. R. Zebaze, X. Hong, W. Li, J. Kaddour, M. Xu, Z. Zhang, P . Yadav, and et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net, 2025. URL http s://openreview.net/forum?id=YrycTjllL0 .

DeepSeek 弹性计算（DSec）：面向大规模高效智能体训练的沙箱基础设施

DeepSeek Elastic Compute (DSec): A Sandbox Infrastructure for Effective Agentic Training at Scale

Jialiang Huang†‡, Hongxuan Tang†, Jingchang Chen†, Yuxuan Liu†, Yixiao Chen†, Yuan

DeepSeek-AI　research@deepseek.com

arXiv:2609.22978 · 中文翻译版（正文全文；图表保留原文页码位置）

摘　要

基于大语言模型（LLM）的大规模智能体训练与评测，依赖隔离的、有状态的执行环境，模型在其中查看代码仓库、调用工具、执行命令，并与任务专用服务交互。这类工作负载会大量突发地创建沙箱，其功能与隔离需求跨异构范围，需要在长交互过程中保持状态，并且要从复用率有限的大型镜像语料中取用镜像。因此，支撑这类负载需要的是一个弹性执行平台，而非单一的沙箱运行时。本报告介绍 DeepSeek 弹性计算（DeepSeek Elastic Compute，DSec）——一个生产级沙箱平台，它通过统一 SDK 对外提供 FnCall、容器、microVM 与完整 VM 四类沙箱后端。DSec 在集群范围内协调放置与生命周期管理，从独立版本化的层组合出运行环境，结合内存共享、内存回收与 CPU 调度以实现高密度执行，并按需从 Fire-Flyer 文件系统（3FS）——一个集群级分布式文件系统——加载镜像数据。DSec 与强化学习（RL）框架协同设计，将有状态的 rollout 执行与可被抢占的 GPU 训练解耦，把沙箱生命周期与训练相协调，以在回收空闲资源的同时保留 rollout 状态，并缓解奖励攻击（reward hacking）等智能体不当行为。DSec 的单个生产规模单元约含 160 个节点，每天服务约 300 万个沙箱；在生产环境中，它支持超过 38 万个并发沙箱，并可持续每秒超过 5,000 次的沙箱创建。我们的评测与部署经……验表明，这些机制降低了环境搭建与镜像分发的开销，提高了内存效率，并在高密度超配下保持对延迟敏感的性能。

1 引言

前沿 LLM 的近期进展使智能体工作流变得切实可行并得到广泛采用（Guo 等，2025；Jimenez 等，2024；OpenAI 等，2024）。智能体模型不再只产出单一文本答案，而是与执行环境交互：它可能浏览代码库、调用工具、执行命令、检查失败并修改文件，或在计算机使用（computer-use）任务中通过图形界面操作浏览器与桌面应用（Xie 等，2024；Zhou 等，2024）。在所有这些负载中，模型基于反馈迭代，直到任务被解决。这一执行模型催生了日益壮大的智能体工具与编排框架（harness）生态，例如 DeepSeek Harness（DSH）（Shi 等，2026）、OpenCode（Anomaly，2025）以及多智能体训练框架。训练可靠的智能体需要大规模强化学习（RL），其中模型通过与真实、隔离的执行环境交互来学习，而不仅仅依赖静态的输入-输出样例。

智能体训练流水线涵盖环境与数据构建、RL rollout、奖励计算、策略更新与周期性评测。在这些阶段中，RL rollout 与评测对沙箱平台造成的压力最大，因为它们规模大、并发高，且与训练循环紧密耦合。在 RL 中（Guo 等，2025；Ouyang 等，2022），训练以包含三个阶段的反馈循环推进。第一，在 rollout 阶段，当前模型与沙箱化环境交互：读取文件、发起工具调用、执行命令、观察输出，并为每个任务产生一条轨迹。第二，在奖励计算阶段，框架利用原生执行信号（如退出码、 stdout、测试通过率或任务特定的验证器）为轨迹打分。第三，在策略更新阶段，RL 算法根据收集到的轨迹与奖励更新模型参数。周期性评测沿用类似的执行路径，只是所得轨迹用于衡量模型能力，而非更新参数。近期系统进一步通过异步 rollout 将生成与策略优化流水线化，持续补充已完成的样本，以维持高并发并缓解长尾拖尾（DeepSeek-AI，2026）。对于智能体负载，这一设计使大量有状态沙箱会话同时处于进行中，并可能因策略更新或调度器抢占而中断并恢复相关 rollout，从而进一步提高平台的并发、生命周期管理与状态一致性要求。

对于每个 rollout 或评测任务，平台都必须创建并准备好一个隔离的任务特定环境，包括其仓库、依赖、服务、评测脚本与编码框架。该环境必须足够接近真实机器，以便运行未经修改的软件栈、包管理器、构建工具、浏览器、模拟器与任务特定服务。因此，一个健壮、高吞吐的沙箱运行时是获得准确且可验证的 RL 与评测结果的基础。

智能体沙箱负载具有若干塑造平台设计的特性：

- **（1）Rollout 与评测作业以突发方式创建沙箱。** 单个作业最多可请求 32K 个沙箱实例，因此平台必须并发接纳并放置大量沙箱。此类突发使水平可扩展性成为全系统需求，并要求调度、镜像分发等共享服务避免集中式瓶颈。

- **（2）沙箱必须以高密度运行。** 在智能体交互过程中，沙箱常常等待 LLM 生成下一个动作，因此 CPU 使用稀疏，天然适合超量分配（overcommit）。例如在生产中，这使单节点最多可承载 800 个 microVM 或 3,200 个容器——但前提是平台能够安全地超量分配资源，并在节点规模上管理生命周期压力。
- **（3）智能体沙箱是有状态且长生命周期的。** 模型可能修改文件、安装依赖并启动服务，后续工具调用依赖这些累积状态。由于沙箱可在多个 LLM 交互轮次间持续存活，内存占用、guest 页缓存、宿主机页缓存与可写状态可能在 CPU 空闲很久之后仍被锁定。在高密度超量分配下，这些常驻开销直接限制集群容量，因此内存共享与回收成为重要的平台需求。
- **（4）智能体负载高度异构。** 平台必须覆盖类 OJ 的脚本执行、基于完整仓库的软件工程任务、安全任务、计算机使用负载、移动开发环境（例如 Android）以及其他全系统环境。这些负载在 CPU 与内存需求、依赖足迹、所需系统功能与隔离强度上差异巨大。单一的沙箱抽象无法高效覆盖全部负载。例如，对于简短的无状态任务，轻量函数调用更合适；而对需要完整商用操作系统的工作负载，虚拟机（VM）更为合适。
- **（5）即使在同一负载类别内，环境多样性也很高。** 训练与评测语料包含许多任务，每个任务可能需要自己的仓库、依赖版本、服务、工具包、评测脚本或 VM 快照。因此，平台必须服务大量不同的镜像与环境产物，其中许多的复用很有限。在突发启动下，从镜像仓库拉取这些多样的任务镜像会将负载集中在分发路径上、抬高启动延迟，并引入额外 I/O 干扰已运行的沙箱。在我们的消融实验中，急切拉取镜像使完成时间延长 1.7×，而按需加载将累计磁盘写入减少 57%。
- **（6）智能体执行是不可信的。** 智能体可能破坏文件系统、耗尽资源或干扰系统组件，进而可能破坏 rollout 或其他同机负载。因此，平台需要细粒度的访问控制与不当行为分析，以遏制并诊断智能体引发的故障。
- **（7）智能体执行是可中断的。** GPU 训练作业可能在长时间运行的 rollout 仍在进行时被抢占。因此，平台必须保留执行状态并支持跨中断的高效恢复。

这些特性界定了智能体沙箱平台的角色。DSec 提供弹性服务伸缩、高密度资源管理、内存共享与回收、面向不同负载类别的多种隔离机制、可扩展的镜像分发，以及与训练框架的显式集成，用于抢占安全恢复、任务特定的网络策略与智能体不当行为的缓解。

本报告其余部分从平台抽象、实现到评测介绍 DSec。第 2 节从用户视角介绍 DSec，包括所支持的负载、沙箱后端与运行规模。第 3 节描述端到端的平台架构。第 4 节刻画生产负载及其带来的平台挑战。第 5 节介绍环境组合、镜像分发与高密度资源管理的核心系统机制。第 6 节描述与 RL 框架的

协同设计，涉及环境构建、状态保留、跨抢占的资源回收，以及针对智能体不当行为的分析与定向访问控制缓解措施。第 7 节总结额外的实现细节。第 8 节评测设计的有效性，第 9 节讨论相关工作。

2 DSec 概览

本章从用户视角介绍 DSec。从平台的角度看，用户是代表研究人员调用软件开发工具包（SDK）的训练框架、评测框架与数据构建流水线；在整份报告中，我们将它们统称为用户。本章涵盖 SDK 入口、平台对外暴露的沙箱后端及其服务的负载类别、沙箱会话的生命周期，以及生产部署的运行规模。

2.1 SDK 入口

用户通过 libdsec（沙箱服务的 Python 客户端库）访问 DSec。libdsec 为用户提供创建与操作沙箱的统一 SDK 入口，但仍要求用户选择适合任务需要的沙箱后端。一个典型请求会指定沙箱类型、镜像或环境标识符、CPU 与内存限制、生命周期设置、网络规则以及初始用户上下文。创建之后，用户可以执行 shell 命令或工具调用，并收集命令输出与返回状态。代码清单 1 展示了一个最简容器会话：客户端连接到服务端点，请求具有所需资源与网络策略的沙箱，运行一条命令，然后释放它。

代码清单 1：通过 libdsec 的最简沙箱会话。（原文第 4 页）

```
client = DSecClient()
await client.open()
args = DSecContainerRunArgs(
    container_image="registry.../sphinx-9658:official",
    memory_limit_mb=4096, cpu_cores_limit=4,
    ttl_running_stop=300,  # idle timeout
    network_rules={"npm": False, "pypi": True},
    init_user="root",
)
sandbox = await client.run_container(args, timeout=120)
result = await sandbox.run_shell("echo hello world")
await sandbox.stop()
```

在该示例中，网络规则允许访问 PyPI（pypi=True）但拒绝访问 NPM（npm=False）。这种细粒度网络控制将在第 6 节详细讨论。该接口有意不做成对所有后端的完整语义抽象。函数调用、容器、microVM 与完整 VM 在启动成本、隔离边界、文件系统语义与操作系统能力上各不相同。libdsec 提供统一的访问路径与相似的操作模型，但调用方仍需负责选择与负载相匹配的后端。

2.2 沙箱后端

沙箱运行时面临一个根本性张力：更强的隔离与更完整的系统功能通常伴随着更高的启动延迟与资源开销。由于没有单一沙箱抽象能适配所有智能体任务，DSec 支持多种后端以覆盖这一权衡空间。表 1 总结了它们的典型适用性。

FnCall 面向简短、无状态的任务，例如类 OJ 负载、代码编译、无服务器程序、GPU 内核与工具代码。FnCall 任务运行在可复用的预创建 CPU 或 GPU 容器中，避免每次调用的供给开销。对于 GPU 负载，FnCall 支持（i）共享模式：多个容器共享一个 GPU 实例，为轻量负载最大化利用率；（ii）独占模式：一个容器在其生命周期内独占一个 GPU 实例，用于性能敏感任务（例如算子评测）。

容器 是软件工程与通用工具使用负载的主要后端。它们提供快速启动与高打包密度，并能运行大多数仓库级任务所用的 Linux 软件栈。其主要局限在于共享宿主机内核，因此并不总是适合安全敏感任务。

【表格】表 1 DSec 各沙箱后端的典型负载适用性。（原文第 5 页）

Firecracker microVM （Agache 等，2020）在保持 Linux 兼容性的同时提供更强的隔离边界。它们适用于安全敏感任务、更强的租户隔离，以及需要兼具 VM 边界与 Linux 兼容性的负载；其代价是比容器更高的内存开销与更慢的启动。

完整 VM 后端覆盖需要完整商用现成操作系统环境的工作负载，例如通过 QEMU（Bellard，2005）运行的 Android VM，以及需要 GUI 或图形渲染的负载。这类后端资源开销最高，但对于依赖 OS 特定 API、移动运行时行为或全系统执行的任务是必要的。

在生产中，容器与 microVM 在实例数量和资源消耗上都占主导。FnCall 以少量常驻环境服务大量轻量调用，而完整 VM 后端覆盖专门但重要的负载类别。

2.3 用户可见的生命周期

尽管所支持的后端在内部各不相同，用户看到的是统一的高层生命周期。首先，调用方通过选择后端并指定环境产物、资源限制、生命周期策略与网络策略来创建沙箱。环境产物因后端与负载而异。对于容器与 microVM，它是基础镜像连同任务特定的工作区与工具包层，平台将其组合为运行环境。对于完整 VM 负载，它是预先准备好的 VM 镜像或快照。对于 FnCall，它是包含任务类型、依赖文件与待运行代码或脚本的任务规范。这些产物构成了本报告后文所述环境组合与镜像分发机制的

基础。其次，平台准备环境并使其就绪以供交互。第三，用户发出命令或工具调用，观察输出，并运行任务特定的检查或测试。沙箱在其整个生命周期内都是有状态的：文件修改、已安装依赖与已启动服务在多次调用之间持续存在，因此后续命令能观察到先前命令的效果。由于沙箱在许多交互轮次间持续存活，而其 CPU 在轮次之间常常空闲，其常驻状态在最后一次命令之后很久仍被锁定——这是第 4 节所刻画的高密度挑战之一。最后，沙箱在显式停止或生存时间（TTL）到期后被回收，从而避免空闲或废弃会话无限期占用资源。

2.4 部署规模

DSec 部署在多个规模单元（scale unit）上，这些单元共享同一套用于基础镜像与工作区存储的 3FS（DeepSeek-AI）分布式文件系统部署。在单个规模单元内，平台跨越近 160 个 CPU 节点、3 万核与约 250 TB DRAM，管理 PB 级的层与镜像。在典型的一天里，单个规模单元服务约 3M 个沙箱实例，峰值并发约 380K，创建速率超过每秒 5,000 个实例。这些数字对于理解本报告余下内容很重要。DSec 不是单一的沙箱运行时，也不是容器的薄封装，而是一个生产级执行平台，必须将面向用户的沙箱抽象、后端特定运行时、可扩展镜像存储、高密度资源管理与训练框架集成结合起来。

3 平台架构

第 2 节展示了用户视角下的 DSec：一个 SDK、一组沙箱后端与一套会话生命周期。本章转向该接口背后的平台，描述一个请求如何从 SDK 到达运行中的沙箱，以及它经过哪些组件。我们按集群级服务与沙箱运行时来阐述架构。集群级服务提供请求入口、身份与访问管理、沙箱放置，以及集群健康与负载视图。沙箱运行时处理节点本地准入、沙箱创建、执行与资源回收，并依赖 3FS 获取镜像数据。

3.1 概述

【图表】图 1 DSec 架构。每个代理负责承载容器或 VM 沙箱与平台其余部分之间的通信。FnCall 采用独立的执行路径，不使用该代理。（原文第 6 页）

在高层面上，创建沙箱的请求首先发送到 IAM 进行认证与授权。一旦被授权，请求便进入放置引擎（placement engine），后者利用 watcher 收集的健康与负载信息选择一个目标节点。放置完成后，apiserver 将请求转发给该节点上的 edge。随后 edge 检查本地容量：若容量允许，则以请求的后端创

建沙箱；否则拒绝该请求。沙箱所需的镜像数据存储在 3FS 中，并在启动与执行期间按需获取。容器、microVM 与完整 VM 沙箱会运行一个每沙箱的代理（aether）以及一个或多个 chronus 实例，用于命令执行、文件系统访问与其他运行时操作。此类沙箱运行后，其操作经由 apiserver、edge、aether 与 chronus 路由。相比之下，FnCall 既不使用 aether 也不使用 chronus，而是走独立的请求路径：提交的任务直接在预创建容器中执行，随后对任务状态进行尽力清理。

3.2 集群级服务

集群级服务管理平台的访问，并在计算节点之间协调沙箱请求。它们包括 IAM、apiserver、放置引擎与 watcher。

IAM 身份与访问管理（IAM）对调用方进行认证，并对发往 DSec 的所有管理请求进行授权。例如，创建或删除沙箱、修改用户资源或并发限制的请求必须在执行前通过 IAM 检查。主体（principal）是与管理请求关联的用户或服务身份。IAM 使用项目（project）来定义资源管理与访问控制的作用域。在项目内，访问策略规定哪些主体可以对其资源执行哪些管理操作，而资源配额则限制资源消耗。

我们支持多级项目嵌套，而非云平台常见的扁平或两级层次结构。获得授权的主体（包括智能体与 harness）可以创建子项目、委派父项目配额的一部分，并在其中授予管理权限。委派受父项目约束：主体不能授予其自身不具备的权限，子项目的策略与配额也不能超过父项目的访问控制或资源限制。人类与智能体使用相同的管理 API 与授权模型。

API Server apiserver 充当沙箱集群的入口代理。训练与评测代码从可信的 GPU 服务器调用 libdsec，而沙箱执行不可信的模型生成代码，并可能访问外部网络。因此两侧在网络上是隔离的，apiserver 是唯一允许的通信路径。所有沙箱请求（包括创建、命令执行与流式 I/O）都经过该入口。apiserver 不维护任何每沙箱状态。它定期从 watcher 刷新 edge 节点集合，而每个沙箱 ID 都编码了其所属的 edge。因此，任意 apiserver 实例都能解析请求并直接转发到目标 edge，使入口层可以水平扩展。

Placement Engine 放置引擎为每个新沙箱选择主机节点。放置分两个阶段进行：过滤与排序。过滤阶段只保留健康状况良好、且能提供请求所需后端与硬件能力的节点。例如，请求启用 GPU 的沙箱会被限制在配备所需 GPU 的节点上。排序阶段随机采样若干合格节点，并选择其中负载最低者。

Watcher 放置引擎的决策质量取决于它对机群（fleet）的视图，而该视图由 watcher 提供。watcher 定期探测每个 edge 与主机的健康状况，并收集与调度相关的状态，例如各后端类型的运行沙

箱数量（按 edge、按用户、按任务细分）。放置引擎定期从 watcher 拉取该状态，并在评估新创建请求时使用最新视图。请注意，放置引擎与 watcher 都不需要持久化状态。放置引擎不保留沙箱执行状态，watcher 在重启后可通过再次轮询 edge 重建其机群视图。这使放置引擎与 watcher 实例易于增加或替换，而无需昂贵的恢复步骤。

3.3 沙箱运行时

沙箱运行时创建并操作单个沙箱、管理其资源。它包括 edge、aether 与 chronus，并依赖 3FS 提供共享镜像存储。

Edge 每个节点运行一个 edge，这是处理来自 apiserver 的容器、microVM、基于 QEMU 的完整 VM 与 FnCall 后端创建请求的每机组件。在接受创建请求之前，edge 会检查节点的当前容量，容量不足时拒绝该请求。这一节点本地准入检查补充了放置引擎基于定期刷新的集群状态所做的放置决策。创建期间，edge 负责供给存储、应用基于 eBPF 的网络策略并启动运行时。FnCall 与容器运行在 QEMU/libvirt VM 内，而非直接运行在宿主机上。该 VM 提供隔离的内核与网络栈，并作为不可信容器与裸金属之间的额外安全边界。为更好地支持图形密集型负载（例如计算机使用的 GUI 应用、浏览器、电子游戏与 3D 渲染），我们利用宿主管理程序的半虚拟化 GPU 接口（例如 virtio-gpu）。在完整 VM 内，我们既支持图形 API 与宿主 OS 原生兼容的负载，也支持其渲染栈可通过 DXVK（DXVK，2018）等兼容层转换为宿主原生 API 的负载。除跟踪沙箱生命周期外，edge 还协调磁盘与内存快照，并在沙箱停止或 TTL 到期时释放节点本地资源。

Aether 容器与 VM 沙箱会运行 aether，这是一个跨平台代理，负责与 edge 建立通信通道。该 edge 到 aether 的通道使用平台特定的传输方式，例如 Linux 容器使用 Unix 域套接字，VM 后端使用 vsock。edge 通过该通道监控沙箱健康状态，若通道关闭则将沙箱标记为失败。对于每个操作，aether 使用该操作的终端会话标识符来创建或定位相应的 chronus 实例，然后通过本地通道转发该操作。会话结束时，aether 终止相应的 chronus 进程树。

Chronus chronus 在沙箱内提供 shell 会话抽象，每个实例代表一个独立的 shell 会话。它暴露用于命令执行、文件系统操作、HTTP 请求与流式 I/O 的跨平台接口。同一沙箱内可以并发运行多个 chronus 实例。aether 与 chronus 共同使 libdsec 能够为容器与 VM 沙箱操作暴露统一接口。

基础镜像与工作区存储 沙箱运行时使用 3FS 作为基础镜像与工作区镜像的共享后备存储。容器镜像离线从 OCI 转换为 EROFS，后者将元数据与数据分离，使元数据保留在本地，而镜像数据留在 3FS 中。microVM 磁盘镜像在同一存储上使用 OverlayBD（Li 等，2020）格式。这些镜像格式共同支

持按需加载与基于共享基础层的增量快照，使 edge 无需先拉取完整镜像即可启动沙箱。第 4 节刻画了使可扩展镜像分发成为必要的工作负载压力，第 5.3 节则描述相应的按需加载机制。

3.4 带选择性卸载的云爆发

DSec 利用云上 VM 吸收沙箱需求的瞬时峰值，同时在本地承载稳态负载。当本地利用率超过 80% 时，放置引擎会将一部分符合条件的入站沙箱创建请求卸载到云上 VM。

我们没有将托管容器服务与对象存储组合使用，而是在云上 VM 中复用本地的容器运行时与基于 EROFS 的镜像加载路径。EROFS 镜像存放在云上托管的分布式文件系统中，并由云上 VM 挂载。生产文件访问轨迹显示，一套紧凑、去重后的 EROFS 镜像集合总计 30 TB，覆盖了 70% 容器任务所访问的镜像文件。我们将这套共享镜像集合离线同步到云上文件系统。镜像依赖完全包含在该集合内的容器任务被归类为可上云（cloud-eligible），其他任务则保留在本地。在生产中，单个规模单元中的 200 台云上 VM 吸收了约 30% 的峰值溢出，从而在无需过度配置本地集群的情况下增加了容量。

4 生产沙箱负载与平台挑战

本章刻画 DSec 所服务的生产负载及其带来的平台挑战。我们只报告容器与 microVM 的测量结果，二者合计占生产中的大部分沙箱实例与资源消耗。测量结果展现出一种对传统执行服务而言并不寻常的组合：请求以大规模突发形式到达；每个沙箱在长时间交互中保留状态；即使大量沙箱同时存活，CPU 需求也很稀疏；且环境工作集过于多样，难以用节点本地镜像缓存覆盖。我们在此将每项负载特性与其系统后果联系起来，并将对应机制留待后续章节讨论。

4.1 生命周期与突发需求

【图表】图 2 每个任务创建的沙箱数量分布。数据采样于 2026 年初的一周内。（原文第 9 页）

Rollout 与评测任务以批而非稳定的速率创建沙箱。图 2 显示，一个典型的容器任务已经会创建数千个沙箱，尾部达到数万个。最大的生产作业最多可请求 32K 个沙箱。这些请求在很短的窗口内到达，因为训练或评测批次在其环境就绪之前无法使用该实例。因此，放置、沙箱创建与环境准备必须吸收急剧的突发，而任一阶段的拖尾任务都会延迟有效的模型交互。

沙箱创建之后，会经历三个宽泛的阶段，如图 3 所示。setup（准备）阶段准备任务依赖、工具与初始化状态。在 tool-call（工具调用）阶段，模型在输出生成与沙箱操作之间交替，产生短暂的 CPU

突发，其间夹杂着沙箱等待下一步动作的时段。test（测试）阶段验证结果，并可能再次短暂提高资源需求。这些阶段没有固定的持续时间，但它们不同的资源特征很重要：setup 成本会随突发规模成倍放大，而后续阶段尽管 CPU 活动是间歇性的，却会保留沙箱状态。

【图表】图 3 一次典型的沙箱执行，包含 setup、tool-call 与 test 三个阶段。setup 之后 CPU 需求是间歇性的，而内存占用与累积状态则持续存在（原文第 10 页）

4.2 环境多样性与启动压力

第一个挑战是由独立演进的软件组件组合每个沙箱所导致的高成本 setup 阶段。沙箱的内容可以分解为三部分：提供操作系统级依赖的基础镜像（base image）（例如 Ubuntu、Python 3.10 或 Java 8 环境）、承载任务代码仓库及其任务特定依赖的工作区（workspace），以及一个或多个频繁更新的工具包（toolkit）（例如 DeepSeek Harness）。在一个生产周内，容器后端服务了 11,266 个基础镜像与 102,171 个工作区，而 microVM 后端使用了两个共享基础镜像与 53,590 个任务特定工作区。平台还服务了 103 个工具包，且 67.8% 的沙箱在基础镜像之外还至少需要一个工作区或工具包。

表 2：2026 年初一个生产周内活跃的环境制品。microVM 工作区以任务特定的磁盘镜像形式存储。（原文第 10 页）

后端	基础镜像	工作区	快照	总大小
容器（Container）	11,266	102,171	–	82.8 TB
microVM	2	53,590	4,889	50.9 TB

将三个组件融合进单个开放容器计划（OCI）（Open Container Initiative，2026）镜像会带来组合式的维护负担。如果平台维护 M 个基础镜像、N 个工作区与 K 个工具包，那么升级 m 个基础镜像可能需要以 O(m·N) 的成本重建其工作区组合；而当工具包与工作区组合时，升级 k 个工具包的成本为 O(k·N)。图 4a 给出了一个具体例子：升级工具包 T1 会强制重建每一个包含它的单体镜像，即使基础镜像与工作区并未改变。目标是通过独立地对三个组件进行版本化与分发，将相应的维护成本降至 O(m) 与 O(k)。

【图表】图 4 在两种环境打包方案下升级工具包 T1 的影响。(a) 使用单体镜像时，即使基础镜像与工作区未变，每个包含 T1 的镜像都必须重建。(b) 使用独立版本化的可组合层时，只需更新 T1 层，然后与现有的基础镜像层和工作区层重新组合（原文第 11 页）

一种直接的替代方案是将工作区与工具包以压缩归档的形式分发，并在启动时于每个沙箱内解压。然而，当这种重复工作集中在突发期间时，会造成大量 CPU 与 I/O 开销，并可能导致沙箱启动超时。

另一种方法是在宿主机上把每个工作区或工具包维护为只读目录，并将其绑定挂载（bind mount）到沙箱中。绑定挂载会完全替换目标路径，而这些组件需要追加语义：它们的文件必须合并到沙箱已有的目录树中，而不能隐藏其下的内容。严格的只读挂载还会与写入自身安装树的工具冲突，例如 Python 创建 __pycache__ 目录。

4.3 稀疏利用率与高密度执行

【图表】图 5 平均与峰值 CPU、内存使用率的分布，按每个沙箱请求的资源归一化。数据采样于 2026 年初的一周（原文第 11 页）

如图 5 所示，容器与 microVM 沙箱中约 90% 平均使用的 CPU 容量不超过其请求量的 5%，这使得超卖（overcommit）成为自然选择。图 6 中 2026 年初的一天采样显示，单节点峰值达到 1,048 个容器与 524 个 microVM。然而在整个生产环境中，我们已观察到单节点至少 3,200 个容器或 800 个 microVM 的稳定运行。这些是已验证的运行点而非硬性上限。在这样的密度下，内存低效与 CPU 干扰变得愈发重要。

【图表】图 6 一天中单个生产节点上活跃沙箱的数量。观测到的峰值为 1,048 个容器与 524 个 microVM。数据采样于 2026 年初的一天（原文第 12 页）

【图表】图 7 从 30K 个容器与 10K 个 microVM 采样的沙箱生命周期分布。中位生命周期分别为 17.4 与 15.5 分钟，两种后端的 p99 生命周期均超过三小时。数据采样于 2026 年初的一周（原文第 12 页）

在内存方面，microVM 存在两类浪费来源。第一，通过虚拟块设备读取的镜像数据可能先被宿主机缓存一次，再被每个 guest 缓存一次，导致同一数据在 guest 与宿主机边界两侧被冗余缓存。第二，guest 内部的空闲页在未显式上报的情况下不会被归还给宿主机。由于请求的内存容量往往超过实际需求，guest 内部几乎没有压力去回收不活跃的页。图 7 表明这些沙箱同样生命周期较长：容器的中位生命周期为 17.4 分钟，microVM 为 15.5 分钟，两种后端的 p99 生命周期均超过三小时。这些较长的生

命周期放大了保留内存的成本。综合来看，这些效应限制了 microVM 的内存超卖，尤其是在高沙箱密度下。

在 CPU 方面，某些任务有严格的单步延迟预算，例如每步有固定时限的博弈智能体。当尽力而为（best-effort）任务与延迟敏感任务运行在相邻的同时多线程（SMT）上下文上并仍共享核心执行资源时，仅给尽力而为任务设置更低的调度优先级是不够的。平台必须通过超卖提升 CPU 利用率，同时不干扰延迟敏感任务。

4.4 大镜像工作集与低扇出

沙箱镜像的绝对数量构成了第三个挑战。如表 2 所示，一个生产周内活跃的制品合计占用超过 130 TB，远超单个工作节点所能存储的容量。如图 8 所示，容器镜像的扇出中位数为 3、p90 为 28，而 microVM 镜像的扇出中位数为 1、p90 为 3。这种低扇出导致本地镜像缓存利用率低下，因为工作集过于多样，无法被单个节点有效吸收。因此，当突发的沙箱创建请求到来时，镜像拉取不可避免，并对镜像分发基础设施造成巨大压力。

【图表】图 8 在超过 150 万个容器与 39 万个 microVM 上测得的每任务镜像扇出。大多数镜像在同一任务内仅被少量沙箱使用。数据采样于 2026 年初的一周（原文第 13 页）

表 3：不同编程语言的采样容器镜像中运行时被访问的文件数据。（原文第 13 页）

镜像类型	C++	Go	Java	JavaScript	Python
访问的数据	8.7%	13.3%	9.2%	4.2%	6.0%
镜像大小	4.9 GB	4.1 GB	12.1 GB	9.6 GB	6.0 GB

由于超卖使集群接近满负荷运行，拉取并物化完整镜像会消耗本可用于服务运行中沙箱的 CPU 与 I/O 资源。预热（pre-warming）只是把这一开销提前，并不能消除它：同样的数据仍需传输与物化，且完整镜像仍会占用本地存储。此外，沙箱通常只访问其镜像数据的一小部分。在表 3 中针对不同编程语言采样的容器镜像里，运行时访问仅覆盖镜像数据的 4.2% 至 13.3%，这使得完整镜像拉取尤其浪费。这些观察促使我们采用按需镜像加载。

5 核心系统机制

第 4 节指出了三个相互耦合的基础设施挑战。第一，突发式创建与独立演进的环境组件要求一条其成本不随每沙箱重复解压而扩展的启动路径。第二，稀疏的 CPU 需求使高密度执行具有价值，但长生命周期、保留内存与混合延迟要求使得不受约束的超卖并不安全。第三，镜像语料庞大、扇出低且运行时访问比例低，使得急切的全镜像分发既昂贵又具有干扰性。

【图表】图 9 核心系统机制概览。LS 表示延迟敏感，BE 表示尽力而为（原文第 14 页）

本章给出环境组合、高密度资源管理与可扩展镜像分发所对应的机制，如图 9 所示。

5.1 可组合的环境层

我们的洞见是：基础操作系统环境、每个工作区与每个工具包都是逻辑上独立的层，各有自己的生命周期，而不是必须融合进单个单体镜像的组件。如图 4b 所示，工具包因此可以独立更新，并与现有的基础镜像层和工作区层重新组合。Overlayfs 原生提供了我们所需的合并语义：当多个只读下层目录堆叠时，内核呈现统一的目录树，来自所有层的文件共存，冲突按优先级顺序解析。一个可写上层目录位于栈顶，透明地吸收任何运行时写入，而不修改下面的只读层。

我们修改容器运行时（即 dockerd），在创建沙箱时动态组合 overlayfs 栈（即 lowerdir）。基础镜像位于最底层，请求的工作区作为只读层插入其上方，每个请求的工具包再堆叠在顶部。这样就把工作区与工具包的文件合并进基础镜像树而非替换路径，将运行时写入导向可写上层，并允许任意层组合而无需耦合其生命周期。因此，升级 m 个基础镜像现在只需重建这 m 个基础层，工作区与工具包不受影响；升级 k 个工具包只需重建这 k 个工具包层。这将单体镜像方案的 O(m·N) 与 O(k·N) 成本降至 O(m) 与 O(k)。

由于已发布的环境层不可变，我们将其存储在 EROFS（Gao 等，2019）中，这是一种专为只读数据设计的文件系统。与 ext4 或 XFS 等可写文件系统相比，EROFS 避免了写入相关的簿记，并使用更简单、更紧凑的磁盘布局。EROFS 还支持数据压缩，同时保持对文件内容的随机访问。与 tar.gz 归档不同，EROFS 可以只读取并解压覆盖所请求数据的压缩块，因此完整镜像不必在使用前被完整传输与解压。

对于 microVM，基础镜像与工具包被打包为独立版本化的 EROFS 镜像，并作为只读块设备暴露给 guest。在 guest 内部，根文件系统使用 overlayfs，将挂载的 EROFS 文件系统作为下层，并将 ext4 格式可写磁盘上的一个目录作为上层。这使 microVM 获得与容器相同的可组合层模型。

5.2 高密度资源管理

内存效率：带 DAX 的 virtio-pmem（QEMU Project，2026）通过将文件访问直接映射到宿主机后备页而不复制到 guest RAM，消除了页缓存重复，使共置的 microVM 共享一份宿主机页缓存副本（§8.4）。然而，virtio-pmem 并不适合所有磁盘。第一，通过带 DAX 的 virtio-pmem 进行冷访问可能需要同步缺页处理来建立映射并使后备数据可用。而带缓冲的 virtio-blk 路径则可以受益于 guest 侧预读与批量块 I/O。第二，guest 必须为整个 pmem 后备地址范围分配 struct page 元数据。以 4 KiB 页、64 字节的 struct page 计算，该元数据所需的 guest RAM 等于 pmem 设备容量的 1/64。例如，一个 128 GB 的 pmem 设备需要 2 GB 的 guest RAM 来存放该元数据。

对于不使用 virtio-pmem 的磁盘，冷文件数据可能在 guest 页缓存中累积，必须单独回收。因此，我们将 DAMON（Data Access MONitor）（Park 等，2019）与 virtio-balloon 空闲页上报结合使用。virtio-balloon 驱动（Waldspurger，2002）支持空闲页上报：guest 定期扫描其伙伴分配器，并主动将空闲页上报给宿主机管理程序，后者通过 madvise(MADV_DONTNEED) 释放相应的宿主机内存。默认情况下，空闲页上报以 order-9 页为单位，即 4 KiB 基础页下的 2 MiB 区域，该 order 可通过内核参数调整。为增强空闲页上报，我们采用 DAMON——Linux 中一个基于采样的内存访问监控框架。DAMON 定期采样页访问位，以识别超过可配置年龄阈值仍未被触碰的冷文件页，并通过内核回收路径将其逐出。这种回收将零散的文件页释放回伙伴分配器，后者将其合并为更高阶的块，以满足空闲页上报的要求。

在 §8.4 的评估中，DAMON 结合 balloon 空闲页上报将内存消耗降低了 21.2%，且没有显著的 CPU 开销。在生产环境中，我们对只读的 EROFS 基础镜像层与工具包层启用带 DAX 的 virtio-pmem，同时使用 DAMON 结合 balloon 空闲页上报为较大的可写磁盘回收内存。

QoS 感知的 CPU 调度：为消除 SMT 层面的 CPU 干扰，DSec 将沙箱分为延迟敏感（LS）与尽力而为（BE）两类。BE 沙箱被置于 SCHED_IDLE 之下，以便只要有 LS 任务可运行就出让 CPU。仅靠调度优先级并不能阻止相邻硬件线程之间的干扰，因此我们还为 LS 沙箱启用 Linux 核心调度（core scheduling）（Zijlstra 等，2021），防止不相关的 BE 工作在同一个物理核的相邻线程上运行。

这一两层策略在保住 LS 每步延迟预算的同时，仍允许 BE 任务利用空闲周期，将 SMT 引起的延迟膨胀从 45.2% 降至 17.3%，详见 §8.5。

5.3 可扩展的镜像分发与按需加载

关键观察是，沙箱通常只访问其镜像数据的一小部分，如表 3 的样本所示。因此，按需拉取不仅解决了时机问题，也解决了体量问题：总 I/O 按实际使用的比例收缩，而不仅仅是被移到另一个阶段。

现有的按需镜像分发系统通常将容器仓库与点对点分发结合，以避免仓库成为瓶颈（Wang 等，2021）。我们则将镜像托管在 3FS 上，它已经在大规模支持我们的生产训练负载。这一选择复用了现有存储基础设施，避免了部署单独的镜像分发层。然而，3FS 表现出高度不对称的 I/O 特性：它对大型顺序读写能维持高吞吐，但在小型随机 I/O 上表现不佳。这种不对称决定了我们的设计：

- **写入留在本地**：沙箱写入不规则且不可控，包括日志文件等小而频繁的写入。我们将可写层放在节点本地磁盘上，从而完全避免 3FS 的小写入惩罚。
- **读取按需且批量**：只读镜像数据仅在访问时从 3FS 获取，并且 I/O 以批量方式执行，以利用 3FS 对大 I/O 请求的高吞吐。
- **元数据尽量保留在本地**：文件系统元数据通常通过小读取访问。当镜像格式允许时，我们将元数据与数据分离，并把元数据预取到本地节点。

对于容器，EROFS 有助于实现这些设计原则。第一，由于 EROFS 严格只读，所有运行时写入都落在本地存储上的 overlayfs 上层目录中。第二，EROFS 通过缓冲 I/O 按需提供文件内容，同时内核预读将相邻块合并为更大的请求。第三，EROFS 提供多设备模式，将文件系统元数据与文件数据分离。Nydus（Dragonfly Community，2020）采用了相关设计：其 EROFS 兼容格式将文件系统元数据与数据 blob 分离并支持懒加载，文件数据通过用户态后端（例如 fscache/FUSE）按需从仓库或对象存储获取。我们则将 EROFS 元数据下载到工作节点本地磁盘，而将文件数据留在 3FS 上，因此元数据遍历与路径名查找不会产生远程 I/O。这把写路径（本地、对小型 I/O 友好）与读路径（远程、按需、批量）分离，匹配 3FS 的不对称性能特征。

尽管这一设计避免了完整镜像拉取，挂载大量 EROFS 层仍会给容器创建增加开销。因此，我们在离线阶段将大小阈值内（例如 3 GB）的连续层合并为单对 EROFS 元数据与数据镜像，同时保留 overlayfs 的 whiteout 语义以正确表示文件删除。这减少了最终挂载数量，避免了过多的文件重复，并

保留了共享公共层的镜像之间的页缓存复用。我们还使用 EROFS 的文件后备挂载模式，以消除 loop 设备块映射层及其开销。

容器式的 EROFS/overlayfs 栈并不能满足 microVM 文件系统的全部兼容性要求。例如，Docker 的 overlay2 驱动无法使用由 overlayfs 后备的数据目录。一种替代方案是通过 virtio-fs 将宿主机挂载的文件系统导出给 guest，但我们的 Firecracker 后端不支持该接口（Agache 等，2020）。因此 microVM 的存储设计并不与容器设计完全相同。只读基础镜像层与工具包层仍使用 EROFS，而 OverlayBD 提供可写的 ext4 磁盘，包括为 Docker-in-microVM 工作负载直接在 Docker 数据根目录挂载的一块独立磁盘。

我们通过 ublk（一个用户态块设备框架）暴露由 OverlayBD 后备的磁盘。这条块级路径提供按需读取与本地写入，并支持增量磁盘快照，而无需将修改过的文件重新打包为 EROFS。与多设备 EROFS 路径不同，ext4 元数据仍嵌入在块镜像中，因此元数据读取可能触发远程 I/O。我们的 ublk 实现通过以 256 KiB 块获取 OverlayBD 数据并将其存储在本地二级文件系统缓存中来缓解这些小的读取。即使某个块被从页缓存中逐出，它仍可在本地缓存中获取，而无需再次从 3FS 拉取。两条路径都将写入保留在本地，并尽量减少对 3FS 的小 I/O 请求。

6 与 RL 框架的协同设计

DSec 服务于从 DeepSeek V3.2（DeepSeek-AI，2025）到 V4.1（DeepSeek-AI，2026）的 RL 训练与评估中所使用的全部沙箱工作负载。除了高效的沙箱执行之外，支持这些工作负载还需要在执行生命周期与安全策略上与 RL 框架协调。本节首先介绍智能体环境的可扩展构建（§6.1）。随后说明智能体循环容器如何将 rollout 执行与 GPU 训练作业解耦（§6.2），以及暂停/恢复如何协调资源回收与训练抢占（§6.3）。最后，考察危害任务完整性或破坏执行环境的智能体行为（§6.4），以及用于缓解这些风险的访问控制（§6.5）。

6.1 属于智能体、由智能体构建、供智能体使用的环境

手工构建智能体 RL 所需的大量环境并不现实。因此，我们让智能体在用于训练与评估的同一基础设施上交互式地构建环境。DSec 通过 pack_diff 支持这一工作流：智能体可以在任意时刻对沙箱做检查点，即获取增量磁盘快照，该快照之后可作为新沙箱恢复。这一检查点-恢复接口将交互式会话直接转化为可复用的环境，使环境可以在同一基础设施上构建、验证与消费，而无需单独的镜像构建流水线。

我们维护一套内部规则，打包的环境必须遵守这些规则，以限制其对共享基础设施的运行时性能影响。这些约束作为指令提供给构建环境的智能体。为追踪所有环境并跟上基础设施的演进，我们的研究人员还构建了一个内部平台，对智能体构建的环境进行质量检查，并以标准格式导出，供 RL 与评估任务使用。

由于环境在同一沙箱基础设施上构建与使用，防止两个阶段之间的信息泄漏非常重要。构建者智能体与运行时智能体使用独立账户；打包前会从可写层中移除构建期残留数据，以免参考答案被带入生成的镜像。

6.2 将智能体循环与 RL 框架分离

我们 GPU 集群中的训练作业会例行被抢占以提升利用率。对于长时间运行的智能体 rollout，将 rollout 执行与训练作业耦合会使抢占的代价尤其高昂：智能体循环可能在取得大量进展后被终止，尽管对应的沙箱状态仍然完好。要恢复这类 rollout，系统必须同时保留智能体的执行状态与沙箱状态。

在训练流水线的早期版本中，智能体循环运行在可被抢占的 GPU，训练 pod 与模型服务及 RL 框架一同部署。当 GPU 作业被抢占时，agent 循环丢失，而沙箱仍然存活。因此，恢复过程依赖命令日志，将训练框架恢复的 rollout 状态与沙箱的执行状态进行对齐。重放时，已完成的操作复用已记录的结果而不再重新执行，从而避免非幂等命令产生重复副作用。

从 DeepSeek-V4.1（DeepSeek-AI，2026）开始，我们改为把 rollout 执行迁移到 DSec，并将其拆分为两个组件：agent 沙箱（agent sandbox），用于托管 scaffold（例如 DeepSeek Harness）及其工具；worker 容器（worker container），用于管理沙箱并为 rollout 提供与 scaffold 无关的控制层。两个组件都运行在可抢占的 GPU 池之外。这一设计将 rollout 的生命周期与训练器的生命周期解耦。worker 容器与 agent 沙箱共同保留完整的 rollout 状态，并作为其唯一真实来源（single source of truth），使被抢占的 GPU 作业能够重新连接并继续运行，而无需通过命令日志重放来重建执行过程。这从 RL 框架中去除了 rollout 状态恢复逻辑，减少了跨组件协调，并简化了故障处理。

6.3 为抢占式 RL 训练挂起沙箱

由于 GPU 作业抢占不可避免（§6.2），沙箱状态必须保留在 DSec 上，直到 rollout 完成。然而，当训练被挂起时，这可能留下许多空闲沙箱占用内存。因此，RL 框架会主动向与被抢占作业关联的所有沙箱发送暂停请求，使 DSec 能够在保留其执行状态的同时回收内存。对于容器与 microVM，对已暂停沙箱的任何后续请求都会在执行所请求操作之前透明地将其恢复。

容器 边缘端（edge）首先发出 docker pause 以冻结容器的进程树，然后通过容器的 memory.swap.max 设置启用交换（swapping），并通过 memory.reclaim 触发主动内存回收。这既回收匿名页也回收文件页，同时保留容器的执行状态。要恢复容器，边缘端对进程的内存映射应用 MADV_WILLNEED 以启动异步预取，然后发出 docker unpause 恢复执行。

microVM 为暂停 microVM，DSec 将其内存与执行状态保存为快照，然后终止正在运行的 Firecracker 进程以释放该 microVM 的运行时内存。恢复时，DSec 启动新进程并恢复快照以继续客户机执行。

6.4 智能体不当行为与系统故障

我们的生产经验揭示了 agent 执行中的两大类风险。任务可能看似通过，但 agent 实际上通过非预期渠道获得了答案，从而破坏训练与评测结果的有效性。agent 的行为也可能破坏执行环境，危及该任务或共享同一基础设施的其他任务。

通过非预期渠道获取答案 在沙箱内，agent 会搜索平台管理的文件与通信渠道以寻找残留答案。例如，它们试图通过直接向 chronus socket（§3.3）发送构造的 RPC 消息来伪造用户请求，并检查 chronus 日志以寻找泄漏的答案。由于 chronus 会为某些操作调用 bash，agent 还试图覆盖 /bin/bash 以绕过检查，或向后续 shell 会话注入命令。在我们引入文件读写与 socket 访问控制（§6.5）之后，一个 agent 试图使用 XFS_IOC_SWAPEXT 绕过这些控制——这是一个交换两个文件数据区映射的 ioctl。其目的是让受保护的文件内容能够通过另一个文件描述符被访问。该尝试破坏了 XFS 元数据并迫使文件系统关闭，说明寻找答案的行为甚至能够扰乱基础设施。

在沙箱之外，agent 会在任务预期的信息来源之外寻找参考实现。例如，它们扫描端口与服务以发现可达的镜像源；还使用 Go 模块代理获取托管在 GitHub 上的代码，并安装更新的软件包版本，以寻找所需功能的既有实现。仅靠最终输出检查无法可靠地判定 agent 是否按预期解决了任务。

篡改执行环境 基础设施故障也源于普通命令与执行失误，而非蓄意破坏系统。在一个案例中，一个 agent 从根目录递归运行 grep，遍历 /proc 并读取 /proc/kpagecgroup，触发内核 bug 导致内核崩溃。类似的故障也出现在一个漏洞利用任务中：本应转发给独立目标 VM 的攻击命令反而在 agent 容器内部执行，导致其自身内核崩溃。除内核崩溃外，无限制的命令输出也可能消耗大量存储。例如，一个 agent 调用了 yes，其持续输出被 chronus 记录，以便用户异步获取命令输出；捕获的 stdout 在存储上累积了数十 GB 的数据。

6.5 访问控制缓解措施

没有任何单一机制能阻止所有 agent 不当行为与系统故障。因此，我们加强可观测性以识别新出现的问题，并随着模型演进而持续加固 DSec。这里我们描述一些访问控制措施，用于限制 agent 通过非预期渠道获取答案的能力，从而缓解奖励攻击（reward hacking）（Amodei 等，2016；Skalse 等，2022）。这些控制只解决了问题的一部分，并不能提供针对破坏性行为（例如触发内核 bug）的通用防御。

文件与 socket 访问控制（AppArmor） 我们使用 AppArmor 配置文件来控制文件读写权限与 socket 访问，包括对 chronus 日志与 Unix 域套接字的访问。这些策略适用于由 agent 控制的进程，即便它们在沙箱内以 root 身份运行。它们限制了从日志中提取残留答案或通过内部通信渠道伪造用户请求的尝试。

细粒度网络控制（eBPF） 训练框架按域名或镜像服务组织，指定任务特定的网络权限。例如，清单 1 允许访问 PyPI，同时拒绝访问 NPM。DSec 通过每个沙箱的 eBPF 程序强制执行相应的允许列表（allowlist），按 IP 地址、端口与协议过滤流量，拒绝允许列表之外的流量。当任务在不同阶段之间移动、连接性需求变化时，这些策略可以动态更新。

7 实现

我们强调若干在实践中被证明重要的实现细节。

放置引擎策略 数千个沙箱在亚秒级时间内的极端尖峰以及严重的超卖，要求以弹性而非固定预留资源的方式均匀分摊增量负载。我们通过以下方面解决该问题。1）我们采用 power-of-k-choices 算法（Mitzenmacher，2001）：调度器随机采样 k 个节点并选择负载最低者，从而避免羊群效应（herding），并减少突发 RL 环境在环境搭建与工具调用期间的相互干扰。2）每个放置引擎实例维护一个局部视图：在周期性 watcher 快照尚未反映其近期放置结果时，将这些结果叠加其上，从而在无需跨实例协调的情况下计入在途负载。3）每个边缘端保留最终准入权：关键资源压力会触发拒绝并选择替代节点，在保持快速路径轻量的同时，防止过时估计覆盖本地资源限制。用户隔离进一步限制了资源尖峰或内核级故障的影响范围（blast radius），代价是牺牲节点级密度。

可靠的服务 辅助服务与集群级服务都必须保持可靠，因为服务中断会导致 agent 任务失败，从而污染奖励信号或评测结果。对于辅助服务（如 API 网关与包镜像）以及控制面入口，我们采用基于 BGP 的负载均衡：每个服务的实例宣告一个共享虚拟 IP，上游交换机在它们之间执行 ECMP 路由。

当某个实例的 BGP 会话中断时，交换机会撤销其路由，并在数秒内将流量重定向到其余实例。包括放置引擎、watcher 与 IAM 在内的集群级服务运行多个独立实例以保证可用性；我们通过定期的集群重置来验证：基础设施即代码（Infrastructure-as-Code）配置能够从零重建所有集群级服务，并在不依赖累积的手工状态的情况下从故障中恢复。

在 dockerd 中动态插入下层 我们修改了开源 Docker 守护进程（基于 Moby 项目（Moby Project，2026）），以在容器创建时动态插入由 EROFS 支撑的下层（lower layers）。具体而言，我们传入一个预先挂载的 EROFS 层的路径，并在挂载之前将其插入 overlayfs 栈，将其置于最上层的 lower layer，从而能够覆盖更低层中的文件。这一改动很小，仅需 30 行 Go 代码。

基于 Rust 的 OverlayBD 与 ublk 库 我们使用由我们贡献的 OverlayBD Rust 移植版，以及我们的 ublk Rust 用户态库，共同构成 §5.3 所述 Firecracker microVM 的按需块存储路径。该存储层支持 3FS、OSS 等对象存储服务以及容器镜像仓库作为远程后端。它还可以使用本地文件系统作为二级缓存，使从页缓存中被逐出的数据无需再次远程获取即可在本地提供。这些存储组件已开源：https://github.com/kvcache-ai/AgentENV/tree/main/storage/overlaybd 。

内存与 CPU QoS 配置 所有机制都利用现有的 Linux 内核特性。带 DAX 的 virtio-pmem 通过 Firecracker 设备配置与客户机内核挂载选项启用。基于 DAMON 的回收通过客户机内核参数与 sysfs 调优来启用。对于 CPU QoS，我们将 best-effort 任务设为 SCHED_IDLE，并通过 prctl(PR_SCHED_CORE) 启用 core scheduling，以按 QoS 类别对任务分组。无需修改内核；实现完全由配置以及与我们沙箱编排器的集成构成。

用于算子基准测试的 GPU FnCall 我们为 FnCall 配备 GPU，以进行无状态算子基准测试。由于 GPU 容量有限，GPU FnCall 使用三种机制在保持性能隔离的同时提升并发。首先，NVIDIA 多实例 GPU（MIG）将每块 GPU 划分为相互隔离的实例，使基准测试能够在独占分配实例的情况下并发运行。其次，CPU FnCall 负责编译，并将生成的产物传给 GPU FnCall，避免不必要的 GPU 占用。最后，Python 进程热池（warm pool）预先初始化运行时并导入库，使请求可以直接开始算子执行。这些机制共同减少了执行路径上的非 GPU 开销，从而同时提升 GPU 利用率与基准测试吞吐。对于非性能敏感的任务，我们还提供共享 GPU 模式，允许多个工作负载共享一个 GPU 实例，以提高并发。

3FS 部署 每台 3FS（An 等，2024）存储服务器配备 20×15 TB SSD 与 $2\times$ 400Gbps RDMA 网卡。CPU 节点通过其基于 FUSE 的客户端访问 3FS，其中 EROFS 元数据存储在本地，文件数据则按需从 3FS 提供。数十台存储服务器即可支撑一个拥有数十万 CPU 核的集群的按需镜像加载。

8 评测

我们的评测衡量 §5 中四项核心机制的有效性与开销：按需镜像加载、可组合环境层、内存优化，以及 QoS 感知的 CPU 调度。实验聚焦于这些基础设施性能机制；§6 中的框架集成本身不在评测范围内。

8.1 实验设置

我们在本章的实验中，使用一个与生产部署分离的、专用的 10 节点 CPU 测试集群。

硬件 为避免嵌套虚拟化，我们的 microVM 直接在裸金属硬件上执行。每个 microVM 节点配备 AMD EPYC 9655 处理器，2 路 × 96 核 × 2 SMT 线程，1.5 TB DRAM，以及 3.4 TB 本地存储。相比之下，基于容器的实验在 QEMU 虚拟机中运行，其配置为：AMD EPYC 9655 处理器，1 路 × 96 核 × 2 SMT 线程（共 192 个硬件线程），512 GB 内存，以及 5.8 TB 本地存储。

内核版本 所有宿主机运行 Linux 7.0，所有 microVM 客户机运行 Linux 6.1。

工作负载 我们的工作负载取自真实的 RL 训练与评测场景。任务套件包括内部软件工程基准、SWE-bench（Jimenez 等，2024）、Terminal-Bench（Merrill 等，2026）、安全漏洞利用任务以及类似领域。

8.2 按需镜像加载

我们评估按需 EROFS 镜像拉取，并与从远程镜像仓库即时拉取完整镜像（Docker Pull (cold)）以及所有镜像层已预缓存在节点上的全本地基线（Docker Pull (cached)）进行对比。实验在需要多样化、数 GB 镜像启动的真实 RL 评测工作负载下，向分布于 10 节点集群的 8,192 个容器发起突发请求。我们测量随时间变化的并发运行容器数、瞬时磁盘写 IOPS 以及累计磁盘写入量。CPU 与内存利用率在各配置间差异可忽略，故未列出。

如图 10 所示，按需 EROFS 拉取达到峰值并发的速度几乎与全本地基线一样快，因为镜像层被直接挂载，数据在沙箱访问其工作集时从 3FS 获取。即时 Docker 拉取必须在容器启动前下载并解压每一层，从而在前 20 分钟内延迟容器创建。按需拉取在约 35 分钟内完成所有任务，与全本地基线相当；而即时拉取需要超过 60 分钟，慢了 1.71 倍。

即时拉取还达到约两倍于按需路径的峰值磁盘写 IOPS，并在每个节点上累计超过 1,600 GB 的磁盘写入。按需拉取只产生一次短暂的初始突发，随后稳定在约 700 GB，比即时拉取少约 57%，接近

约 600 GB 的全本地基线。这些结果验证了 §5.3 的设计：按每个沙箱的工作集按需获取镜像数据，避免了完整镜像的下载与解压，同时接近全本地性能。

【图表】图 10 在 10 节点评测集群上、8,192 个容器突发请求下，按需 EROFS 拉取与即时 Docker 拉取（cold）及全本地 Docker（cached）的对比。左图：每节点随时间变化的运行容器数。右图：瞬时磁盘写 IOPS（实线，左轴）与累计磁盘写入量（虚线，右轴）。（原文第 22 页）

8.3 可组合镜像层：EROFS 与 Tar 对比

对于代码仓库与开发环境，我们比较两种为相同评测工作区与工具包（包括任务仓库、scaffold 二进制文件与命令行工具）提供环境的方式。传统方式将这些文件打包为压缩的 tar.gz 归档，分发到每个沙箱并解压；而 EROFS 方式将相同文件打包为压缩的只读文件系统镜像，可直接挂载为可组合层。我们用预先录制、确定性的工具调用序列替代 LLM 生成，使各次运行只在工作区与工具包的供给方式上不同。

【图表】图 11 分别用每沙箱 tar 解压与 EROFS 层挂载来供给评测工作区时，环境搭建阶段的 CPU 利用率与磁盘写吞吐。（原文第 22 页）

由于 tar.gz 是顺序流格式，每个沙箱都必须解压归档，并在工具调用开始之前将所有工作区与工具包文件写入其本地可写层。这使端到端任务完成时间延长到 79 分钟。EROFS 直接挂载共享层而无需解压，使沙箱能够更早进入工具调用阶段，将完成时间缩短到 45 分钟，加速 1.76 倍。如图 11 所示，基于 tar 的供给产生的磁盘写总流量约为 EROFS 路径的 5.5 倍，峰值磁盘写吞吐约为其 3.4 倍。EROFS 的峰值 CPU 利用率更高，因为更多沙箱更早进入工具调用阶段并并发执行操作。这并不表示环境搭建开销更高，因为 EROFS 避免了反复解压与拆包归档的 CPU 工作。这些结果验证了 §5.1 中可组合层设计的有效性。

8.4 超卖下的内存

【图表】图 12 在真实 agentic-RL 工作负载下，四种 Firecracker 配置的宿主内存使用（左）与 CPU 利用率（右）。CPU 面板对前 10 分钟使用放大的时间尺度，对 10–50 分钟区间使用压缩的时间尺度。（原文第 23 页）

我们在测试集群上运行真实的 agentic-RL 工作负载，并比较四种 Firecracker 配置：未优化的基线、仅启用带 DAX 的 virtio-pmem、仅通过 virtio-balloon 设备启用基于 DAMON 的空闲页报告

（free-page reporting，FPR），以及两者同时启用。带 DAX 的 virtio-pmem 将各客户机冗余的页缓存折叠为单一共享的宿主映射，使峰值宿主内存使用相比基线降低 40.2%。单独使用 DAMON + balloon FPR 基本不改变峰值使用量，但使时间积分（time-integrated）的宿主内存消耗降低 21.2%。两者结合产生最低的总体内存消耗。如图 12 所示，virtio-pmem 将瞬时峰值 CPU 利用率从 26.5% 提高到 41.4%。这一提升可能部分反映了冷访问路径的差异。带 DAX 的 virtio-pmem 可能需要同步的缺页处理来建立映射并使后端数据可用，而缓冲的 virtio-blk 可以受益于客户机侧的预读与批量块 I/O。在 CPU 受限的部署中，运维人员可能更倾向于只启用 FPR 并保留 virtio-blk。总体而言，这些结果验证了 §5.2 中互补的内存优化：virtio-pmem 减少页缓存重复，而 DAMON 与 balloon FPR 回收空闲的客户机内存。

8.5 超卖下的 CPU QoS

我们在真实的评测工作负载中运行延迟敏感（latency-sensitive，LS）任务，同时在同一节点上共置（co-locate）10% 到 50% 节点容量的 best-effort（BE）负载，测量我们的机制在高密度部署下保持每个沙箱性能的程度。我们比较未受保护的基线、单独使用 SCHED_IDLE，以及 SCHED_IDLE 结合 core scheduling。

如图 13 所示，我们使用一个延迟敏感的棋类应用作为测试工作负载。在 50% BE 负载下，若不施加 QoS 控制，其每步延迟相比无共置基线增加 45.2%。单独使用 SCHED_IDLE 至多改善延迟 3.4%，因为 LS 线程仍会与其 SMT 兄弟核上运行的 BE 工作竞争。加入 core scheduling 后，低负载下延迟接近无共置基线，在 50% 负载下把膨胀限制在 17.3%。随着 BE 竞争的加剧，改善幅度增大。这些结果验证了 §5.2 的两级 CPU QoS 设计：SCHED_IDLE 为 LS 任务提供优先级，而 core scheduling 将它们与 SMT 兄弟线程上的 BE 工作隔离开。残余的性能退化主要来自高多核负载下 CPU 睿频（turbo）频率下降、内存带宽与共享末级缓存（LLC）竞争——这些是 core scheduling 无法解决的。由于这种剩余干扰已可容忍，我们不再施加内存带宽隔离。

【图表】图 13 在共置的 best-effort CPU 负载不断增加时，延迟敏感 agent 的耗时；对比未受保护的基线、单独 SCHED_IDLE，以及 SCHED_IDLE 结合 core scheduling。（原文第 24 页）

9 相关工作

无服务器计算（Serverless computing） SAND（Akkus 等，2018）、REAP（Ustiugov 等，2021）、TrEnv（Huang 等，2024）与 RunD（Li 等，2022）等无服务器平台针对短生命周期、无状态的函数优化冷启动延迟与资源共享。这类工作负载通常以高扇出复用有限的镜像集合，且许多系统假设所需镜像已在本地可用。而 agentic 训练使用的是长生命周期、有状态的沙箱，其镜像语料超出单节点存储容量，且每个镜像的扇出很低。

LLM 代码执行平台 近期系统为训练与推理中的 LLM 工作流提供沙箱化代码执行。面向推理的系统包括 OpenAI Code Interpreter（OpenAI，2025）、E2B（E2B，2024）与 Kimi-K2.5 的 Agent Swarm（Kimi Team，2026）。MiMo-V2-Flash（Xiaomi LLM-Core Team，2026）与 ComputerRL（Lai 等，2025）等训练系统提及了其执行环境，但主要关注模型与训练设计。DSec 关注底层的沙箱基础设施，将环境组合、资源超卖、镜像服务以及抢占安全的恢复集成到同一平台中。

容器镜像与文件系统格式 DADI（Li 等，2020）与 CoFS（Wang 等，2026）支持按需加载容器镜像，而 FaaSNet（Wang 等，2021）使用点对点分发来加速镜像分发。EROFS（Gao 等，2019）提供支持随机访问的压缩只读文件系统。DSec 在此基础上将其用于 RL 训练与评测，从 3FS 提供容器与 microVM 镜像，而不是引入单独的镜像仓库与点对点分发层。

轻量级隔离 为运行多样化工作负载，已有多种隔离范式被提出，包括 microVM（Agache 等，2020）或由 VM 支撑的 kata-containers（Kata Containers，2017）、库操作系统（library OSes）（che Tsai 等，2017；LiteBox，2025）、WebAssembly 运行时（Gadepalli 等，2020；Shillaker 和 Pietzuch，2020）、unikernel（Cadden 等，2020）、嵌套内核（Dautenhahn 等，2015；Zhang 等，2025）以及嵌套虚拟化（Huang 等，2023）。它们在隔离性、兼容性与性能之间提供不同权衡。DSec 并未提出另一种隔离机制，而是将多种沙箱后端集成到统一平台之后，让调用方为每个任务选择合适的后端。

RL 训练基础设施 Slime（Zhu 等，2025）、veRL（Sheng 等，2025）、OpenRLHF（Hu，2026；Hu 等，2025）与 Seer（Qin 等，2026）等系统通过高效的 GPU 调度、通信与样本吞吐来扩展 RL 训练。它们将执行环境视为黑盒，假设沙箱可用且配置正确。DSec 工作在互补的基础设施层，管理沙箱供给与生命周期，并与训练框架协调执行状态与安全策略。

10 结论

我们提出了 DSec，一个面向大规模 LLM agentic 训练、评测与环境构建的生产级沙箱平台。DSec 通过统一接口暴露多种沙箱后端，使用户能够针对不同的功能、兼容性与隔离需求选择合适的执行环境。由 EROFS 支撑的可组合层避免了重复的环境重建与解压；互补的内存共享与回收机制支撑高密度部署；QoS 感知的 CPU 调度在超卖下保护延迟敏感任务；由 3FS 支撑的按需镜像加载降低了镜像分发开销。DSec 还与 RL 框架集成，实现抢占安全的恢复与任务特定的网络策略，为 agentic 工作负载提供可扩展的执行基础。

附录

附录：补充材料见英文原文

说明：附录以曲线图与结果表格为主，中文版保留章节与图表标题；完整图表与数据见英文原文第 30–48 页（同目录 PDF）。

参考文献

（保留英文原文，便于检索；编号为原文顺序）

[1] et al., 2015; Zhang et al., 2025), and nested virtualization (Huang et al., 2023). They offer different trade-offs among isolation, compatibility, and performance. Rather than proposing another isolation mechanism, DSec integrates multiple sandbox backends behind a unified platform, allowing callers to choose the appropriate backend for each task.

[2] RL training infrastructure. Systems such as Slime (Zhu et al., 2025), veRL (Sheng et al., 2025), OpenRLHF (Hu, 2026; Hu et al., 2025), and Seer (Qin et al., 2026) focus on scaling RL training through efficient GPU scheduling, communication, and sample throughput. They treat the execution environment as a black box, assuming that sandboxes are available and correctly configured. DSec operates at the complementary infrastructure layer, managing sandbox provisioning and lifecycle while coordinating execution state and security policy with the training framework.

[3] We presented DSec, a production sandbox platform for large-scale LLM agentic training, evaluation and environment construction. DSec exposes multiple sandbox backends through a unified interface, allowing users to select an appropriate execution environment for different functionality, compatibility, and isolation requirements. Composable EROFS-backed layers avoid repeated environment rebuilding and extraction, complementary memory sharing and reclamation mechanisms support high-density deployment, QoS-aware CPU scheduling protects latency-sensitive tasks under oversubscription, and 3FS-backed on-demand image loading reduces image distribution overhead. DSec also integrates with the RL framework for preemption-safe resumption and task-specific network policy, providing a scalable execution foundation for agentic workloads.

[4] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P . Piwonka, and D.-M. Popa. Firecracker: Lightweight virtualization for serverless applications. In 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20), pages 419– 434, Santa Clara, CA, Feb. 2020. USENIX Association. ISBN 978-1-939133-13-7. URL https://www.usenix.org/conference/nsdi20/presentation/agache .

[5] I. E. Akkus, R. Chen, I. Rimac, M. Stein, K. Satzke, A. Beck, P . Aditya, and V . Hilt. SAND: Towards High-Performance serverless computing. In 2018 USENIX Annual Technical Conference (USENIX ATC 18), pages 923–935, Boston, MA, July 2018. USENIX Association. ISBN 978-1- 939133-01-4. URL https://www.usenix.org/conference/atc18/presentation/akk us .

[6] D. Amodei, C. Olah, J. Steinhardt, P . Christiano, J. Schulman, and D. Mané. Concrete problems in ai safety, 2016. URL https://arxiv.org/abs/1606.06565 .

[7] W. An, X. Bi, G. Chen, S. Chen, C. Deng, H. Ding, K. Dong, Q. Du, W. Gao, K. Guan, J. Guo, Y. Guo, Z. Fu, Y. He, P . Huang, J. Li, W. Liang, X. Liu, X. Liu, Y. Liu, Y. Liu, S. Lu, X. Lu, X. Nie, T. Pei, J. Qiu, H. Qu, Z. Ren, Z. Sha, X. Su, X. Sun, Y. Tan, M. Tang, S. Wang, Y. Wang, Y. Wang, Z. Xie, Y. Xiong, Y. Xu, S. Ye, S. Yu, Y. Zha, L. Zhang, H. Zhang, M. Zhang, W. Zhang, Y. Zhang, C. Zhao, Y. Zhao, S. Zhou, S. Zhou, and Y. Zou. Fire-flyer ai-hpc: A cost-effective softwarehardware co-design for deep learning. In Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC ’24. IEEE Press, 2024. ISBN 9798350352917. doi: 10.1109/SC41406.2024.00089. URL https://doi.org/10.1109/SC41406.2024.00089 . Anomaly. OpenCode, 2025. URL https://opencode.ai/ .

[8] F. Bellard. QEMU, a fast and portable dynamic translator. In 2005 USENIX Annual Technical Conference (USENIX ATC 05), Anaheim, CA, Apr. 2005. USENIX Association. URL https: //www.usenix.org/conference/2005-usenix-annual-technical-conference/qe mu-fast-and-portable-dynamic-translator .

[9] J. Cadden, T. Unger, Y. Awad, H. Dong, O. Krieger, and J. Appavoo. Seuss: skip redundant paths to make serverless fast. In Proceedings of the Fifteenth European Conference on Computer Systems, EuroSys ’20, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450368827. doi: 10.1145/3342195.3392698. URL https://doi.org/10.1145/334219 5.3392698 .

[10] C. che Tsai, D. E. Porter, and M. Vij. Graphene-SGX: A practical library OS for unmodified applications on SGX. In 2017 USENIX Annual Technical Conference (USENIX ATC 17), pages 645–658, Santa Clara, CA, July 2017. USENIX Association. ISBN 978-1-931971-38-6. URL https://www.usenix.org/conference/atc17/technical-sessions/presentati on/tsai .

[11] N. Dautenhahn, T. Kasampalis, W. Dietz, J. Criswell, and V . Adve. Nested kernel: An operating system architecture for intra-kernel privilege separation. In Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS ’15, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450328357. doi: 10.1145/2694344.2694386. URL https: //doi.org/10.1145/2694344.2694386 .

[12] DeepSeek-AI. Fire-flyer file sytem. URL https://github.com/deepseek-ai/3fs .

[13] DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556 .

[14] DeepSeek-AI. Deepseek-v4.1-flash: Pushing the limits of kv cache compression, 2026. URL https://arxiv.org/abs/2609.19969 .

[15] Dragonfly Community. Nydus: Dragonfly container image service, 2020. URL https://gith ub.com/dragonflyoss/nydus .

[16] DXVK. DXVK: A vulkan-based implementation of direct3d 8/9/10/11. https://github.c om/doitsujin/dxvk , 2018.

[17] E2B. E2B: Open-source secure sandboxes for AI code execution, 2024. URL https://github .com/e2b-dev/E2B .

[18] P . K. Gadepalli, S. McBride, G. Peach, L. Cherkasova, and G. Parmer. Sledge: a serverlessfirst, light-weight wasm runtime for the edge. In Proceedings of the 21st International Middleware Conference, Middleware '20, page 265–279, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450381536. doi: 10.1145/3423211.3425680. URL https://doi.org/10.1145/3423211.3425680 .

[19] X. Gao, M. Dong, X. Miao, W. Du, C. Yu, and H. Chen. EROFS: A compression-friendly readonly file system for resource-scarce devices. In 2019 USENIX Annual Technical Conference (USENIX ATC 19), pages 149–162, Renton, WA, July 2019. USENIX Association. ISBN 978-1- 939133-03-8. URL http://www.usenix.org/conference/atc19/presentation/gao . D. Guo, D. Yang, H. Zhang, J. Song, P . Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Ding, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Chen, J. Yuan, J. Tu, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. You, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Zhou, M. Li, M. Wang, M. Li, N. Tian, P . Huang, P . Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. Nature, 645(8081):633–638, Sep 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL https://doi.org/10.1038/s41586-025-09422-z .

[20] J. Hu. Reinforce++: A simple and efficient approach for aligning large language models. arXiv preprint arXiv:2501.03262, 2026.

[21] J. Hu, X. Wu, W. Shen, J. K. Liu, Z. Zhu, W. Wang, S. Jiang, H. Wang, H. Chen, B. Chen, W. Fang, Xianyu, Y. Cao, H. Xu, and Y. Liu. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework, 2025. URL https://arxiv.org/abs/2405.11143 .

[22] H. Huang, J. Lai, J. Rao, H. Lu, W. Hou, H. Su, Q. Xu, J. Zhong, J. Zeng, X. Wang, Z. He, W. Han, J. Liu, T. Ma, and S. Wu. Pvm: Efficient shadow paging for deploying secure containers in cloud-native environment. In Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23, page 515–530, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613158. URL https: //doi.org/10.1145/3600006.3613158 .

[23] J. Huang, M. Zhang, T. Ma, Z. Liu, S. Lin, K. Chen, J. Jiang, X. Liao, Y. Shan, N. Zhang, M. Lu, T. Ma, H. Gong, and Y. Wu. Trenv: Transparently share serverless execution environments across different functions and nodes. In Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP '24, page 421–437, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400712517. doi: 10.1145/3694715.3695967. URL https://doi.org/10.1145/3694715.3695967 .

[24] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024. OpenReview.net, 2024. URL https://openreview.net/forum?id=VTF8yNQM66 .

[25] Kata Containers. Kata Containers: Secure Containers with Lightweight Virtual Machines. https://katacontainers.io/ , 2017.

[26] Kimi Team. Kimi K2.5: Visual agentic intelligence, 2026. H. Lai, X. Liu, Y. Zhao, H. Xu, H. Zhang, B. Jing, Y. Ren, S. Yao, Y. Dong, and J. Tang. ComputerRL: Scaling end-to-end online reinforcement learning for computer use agents, 2025.

[27] H. Li, Y. Yuan, R. Du, K. Ma, L. Liu, and W. Hsu. DADI: Block-Level image service for agile and elastic application deployment. In 2020 USENIX Annual Technical Conference (USENIX ATC 20), pages 727–740. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL https://www.usenix.org/conference/atc20/presentation/li-huiba .

[28] Z. Li, J. Cheng, Q. Chen, E. Guan, Z. Bian, Y. Tao, B. Zha, Q. Wang, W. Han, and M. Guo. RunD: A lightweight secure container runtime for high-density deployment and high-concurrency startup in serverless computing. In 2022 USENIX Annual Technical Conference (USENIX ATC 22), pages 53–68, Carlsbad, CA, July 2022. USENIX Association. ISBN 978-1-939133-29- 27. URL https://www.usenix.org/conference/atc22/presentation/li-zijun-r und .

[29] LiteBox. Litebox: A security-focused library os supporting kernel- and user-mode execution. https://github.com/microsoft/litebox , 2025.

[30] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, I. Bercovich, L. Shi, J. Y. Shin, T. Walshe, E. K. Buchanan, J. Shen, G. Ye, H. Lin, J. Poulos, M. Wang, M. Nezhurina, J. Jitsev, D. Lu, O. M. Mastromichalakis, Z. Xu, Z. Chen, Y. Liu, R. Zhang, L. L. Chen, A. Kashyap, J.-L. Uslu, J. Li, J. Wu, M. Yan, S. Bian, V . Sharma, K. Sun, S. Dillmann, A. Anand, A. Lanpouthakoun, B. Koopah, C. Hu, E. Guha, G. H. S. Dreiman, J. Zhu, K. Krauth, L. Zhong, N. Muennighoff, R. Amanfu, S. Tan, S. Pimpalgaonkar, T. Aggarwal, X. Lin, X. Lan, X. Zhao, Y. Liang, Y. Wang, Z. Wang, C. Zhou, D. Heineman, H. Liu, H. Trivedi, J. Yang, J. Lin, M. Shetty, M. Yang, N. Omi, N. Raoof, S. Li, T. Y. Zhuo, W. Lin, Y. Dai, Y. Wang, W. Chai, S. Zhou, D. Wahdany, Z. She, J. Hu, Z. Dong, Y. Zhu, S. Cui, A. Saiyed, A. Kolbeinsson, J. Hu, C. M. Rytting, R. Marten, Y. Wang, A. Dimakis, A. Konwinski, and L. Schmidt.

Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868 .

[31] M. Mitzenmacher. The power of two choices in randomized load balancing. IEEE Transactions on Parallel and Distributed Systems, 12(10):1094–1104, 2001. doi: 10.1109/71.963420.

[32] Moby Project. The Moby Project, 2026. URL https://github.com/moby/moby .

[33] Open Container Initiative. Open Container Initiative Image Format Specification, 2026. URL https://github.com/opencontainers/image-spec .

[34] OpenAI. Code interpreter: Agent harness and sandbox for code execution, 2025. URL https://platform.openai.com/docs/guides/code-interpreter .

[35] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V . Balcom, P . Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzis, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P . Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V . Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene, J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P . Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Ł. Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, J. H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Ł. Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V . Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P . Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P . McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P . Mishkin, V . Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O'Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P . de Oliveira Pinto, Michael, Pokorny, M. Pokrass, V . H. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P . Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P . Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B. Thompson, P . Tillet, A. Tootoonchian, E. Tseng, P . Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P . Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph. Gpt-4 technical report, 2024. URL https://arxiv.org/abs/2303.08774 .

[36] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P . Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P . Welinder, P . Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. In Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

[37] S. Park, J. Ahn, H. Y. Kim, and Y. Lee. Profiling dynamic data access patterns with controlled overhead and quality. In Proceedings of the 20th International Middleware Conference Industrial Track (Middleware '19), pages 29–30, 2019. doi: 10.1145/3366626.3368125.

[38] QEMU Project. VirtIO Persistent Memory, 2026. URL https://www.qemu.org/docs/mast er/system/devices/virtio/virtio-pmem.html .

[39] R. Qin, W. He, W. Huang, Y. Zhang, Y. Zhao, B. Pang, X. Xu, Y. Shan, Y. Wu, and M. Zhang. Seer: Online context learning for fast synchronous llm reinforcement learning, 2026. URL https://arxiv.org/abs/2511.14617 .

[40] G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu. Hybridflow: A flexible and efficient rlhf framework. In Proceedings of the Twentieth European Conference on Computer Systems, EuroSys '25, page 1279–1297, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400711961. doi: 10.1145/3689031.3696075. URL https://doi.org/10.1145/3689031.3696075 .

[41] Y. Shi, W. Zhang, and T. Cui. A programming paradigm for spatiotemporal composability, 2026. URL https://arxiv.org/abs/2608.25512 . S. Shillaker and P . Pietzuch. Faasm: Lightweight isolation for efficient stateful serverless computing. In 2020 USENIX Annual Technical Conference (USENIX ATC 20), pages 419–433. USENIX Association, July 2020. ISBN 978-1-939133-14-4. URL https://www.usenix.org /conference/atc20/presentation/shillaker .

[42] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger. Defining and characterizing reward hacking. In Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

[43] D. Ustiugov, P . Petrov, M. Kogias, E. Bugnion, and B. Grot. Benchmarking, analysis, and optimization of serverless function snapshots. In Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '21, page 559–572, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383172. doi: 10.1145/3445814.3446714. URL https://doi.org/10.1145/34 45814.3446714 .

[44] C. A. Waldspurger. Memory resource management in VMware ESX server. In 5th Symposium on Operating Systems Design and Implementation (OSDI 02), Boston, MA, Dec. 2002. USENIX Association. URL https://www.usenix.org/conference/osdi-02/memory-resourc e-management-vmware-esx-server .

[45] A. Wang, S. Chang, H. Tian, H. Wang, H. Yang, H. Li, R. Du, and Y. Cheng. FaaSNet: Scalable and fast provisioning of custom serverless container runtimes at alibaba cloud function compute. In 2021 USENIX Annual Technical Conference (USENIX ATC 21), pages 443–457. USENIX Association, July 2021. ISBN 978-1-939133-23-6. URL https://www.usenix.org/confere nce/atc21/presentation/wang-ao .

[46] L. Wang, J. Du, Y. Yang, Q. Wu, T. Liu, and H. Wu. CoFS: A filesystem for fast container startup. In 24th USENIX Conference on File and Storage Technologies (FAST 26), pages 415–423, Santa Clara, CA, Feb. 2026. USENIX Association. ISBN 978-1-939133-53-3. URL https://www.usenix.org/conference/fast26/presentation/wang-li .

[47] Xiaomi LLM-Core Team. MiMo-V2-Flash technical report, 2026.

[48] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, Y. Liu, Y. Xu, S. Zhou, S. Savarese, C. Xiong, V . Zhong, and T. Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In Advances in Neural Information Processing Systems, 2024. doi: 10.52202/079017-1650.

[49] C. Zhang, R. Priolkar, Y. Jiang, Y. Xiao, M. Vij, Z. Liang, and A. Ahmad. Erebor: A drop-in sandbox solution for private data processing in untrusted confidential virtual machines. In Proceedings of the Twentieth European Conference on Computer Systems, EuroSys '25, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400711961. doi: 10.1145/3689031.3717464. URL https://doi.org/10.1145/3689031.3717464 .

[50] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. Webarena: A realistic web environment for building autonomous agents. In International Conference on Learning Representations, 2024. URL https://openreview .net/forum?id=oKn9c6ytLx .

[51] Z. Zhu, C. Xie, X. Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. https://github.com/THUDM/slime , 2025. P . Zijlstra, J. Fernandes, and V . Pillai. Core scheduling, 2021. URL https://docs.kernel.or g/admin-guide/hw-vuln/core-scheduling.html .